

SLAM

ĐỊNH VỊ VÀ DỰNG BẢN ĐỒ BẰNG THỊ GIÁC

CÂY TRI THỨC VISUAL SLAM

Trình bày theo mạch

vấn đề → giải pháp → giả định → giá phải trả

“SLAM, SfM, hiệu chuẩn và localization là một bài toán MAP duy nhất. Chúng chỉ khác nhau ở chỗ biến nào được fix, biến nào để tự do, và chuyển động nào làm cho biến đó trở nên quan sát được.”

luận đề trung tâm của cây — Chương 4, 5 và 14

Năm phần A · Nguyên lý và toán học B · Front-end và back-end
C · Hệ thống SLAM đầy đủ D · Thực tế và sản xuất
E · Framework và lộ trình

Bản gộp toàn cây — Ngày 14 tháng 9 năm 2026

Bản đồ cây

TÓM TẮT

Đây là bản đồ điều hướng của cả cây: nó nói cây gồm gì, vì sao được chia thành năm phần theo thứ tự đó, và bốn luận điểm sẽ gặp lại ở gần như mọi chương. Đọc xong bạn biết nên bắt đầu từ đâu, chương nào là xương sống phải tự dẫn lại công thức, chương nào chỉ cần biết là có. Nếu chỉ nhớ một điều: *SLAM, SfM, hiệu chuẩn và localization không phải bốn bài toán* — cả bốn đều là ước lượng hậu nghiệm cực đại trên cùng một đồ thị nhân tử, và chúng chỉ khác nhau ở chỗ biến nào được fix, biến nào để tự do, và chuyển động nào làm biến đó trở nên quan sát được. Toàn bộ cây được dựng để làm rõ câu đó và để chỉ ra khi nào nó hỏng.

Cây này được dựng từ một lộ trình tự soạn, với một mục tiêu hẹp và một mục tiêu rộng. Mục tiêu hẹp: hiểu Visual SLAM tới mức tự dẫn lại được công thức và đoán trước được chỗ hệ thống sẽ hỏng. Mục tiêu rộng: đưa được một hệ thị giác từ nguyên mẫu chạy trên EuRoC lên dây chuyền sản xuất hàng nghìn máy. Hai mục tiêu đó nghe như hai nghề khác nhau, và trong phần lớn tài liệu chúng đúng là hai nghề khác nhau — học thuật viết về cái thứ nhất, còn cái thứ hai gần như không có tài liệu. Cây này cố ý đặt cả hai vào cùng một khung, vì chúng chia nhau đúng một bộ nguyên lý.

Luận đề trung tâm: một bài toán, nhiều cái tên (One Problem, Many Names)

Người mới vào lĩnh vực học Visual SLAM như một danh sách các hệ thống: ORB-SLAM làm thế này, DSO làm thế kia, COLMAP là chuyện khác, Kalibr lại là chuyện khác nữa. Mỗi hệ có bài báo riêng, mã nguồn riêng, ký hiệu riêng. Cách học đó cho cảm giác tiến bộ vì danh sách dài thêm mỗi tuần, nhưng nó không bao giờ hội tụ: hệ thứ hai mươi vẫn phải học lại từ đầu.

Cách nhìn đúng gọn hơn nhiều. Mọi bài toán trong nhóm này đều là: cực tiểu hoá tổng phần dư có trọng số trên một đồ thị nhân tử, với các biến gồm tư thế (quỹ đạo), cấu trúc (bản đồ, landmark), tham số hiệu chuẩn (nội, ngoại, sai lệch thời gian, độ lệch) và gauge. Bộ giải là một; hàm mục tiêu là một; lý thuyết về khi nào nghiệm tồn tại cũng là một. Thứ duy nhất phân biệt các bài toán là biến nào được giữ cố định, như Bảng 1 trình bày.

Bảng 1. Cùng một bài toán MAP, khác nhau ở chỗ fix biến nào. Cột cuối là câu hỏi observability tương ứng — chính là câu hỏi quyết định bài toán có nghiệm hay không.

Bài toán	Fix	Tự do	Điều kiện để nghiệm tồn tại
Hiệu chuẩn ngoại tuyến SLAM	Cấu trúc (target đã biết) Hiệu chuẩn	Nội, ngoại, t_d , quỹ đạo Tư thế + cấu trúc	Chuyển động đủ đa dạng về hướng và độ sâu Đủ parallax; cảnh tĩnh; không xoay thuần
VIO + hiệu chuẩn trực tuyến	Không gì	Tất cả	Kích thích riêng cho từng tham số, kiểm theo thời gian thực
Localization	Bản đồ + hiệu chuẩn	Tư thế	Đủ điểm khớp đúng, hình học không suy biến
SfM	Không gì	Tư thế + cấu trúc, gauge tự do	Đồ thị ảnh liên thông; đủ overlap
Docking bằng marker	Constellation (đã biết)	Tư thế tương đối	Marker đủ lớn trong ảnh; không ở tư thế lưỡng nghĩa

Hệ quả thực tiễn của bảng này lớn hơn vẻ ngoài của nó. Thứ nhất, *mọi kỹ năng bạn học ở một ô đều dùng lại được ở ô khác*: viết được một nhân tử chiếu cho BA thì viết được nó cho hiệu chuẩn, và ngược lại. Thứ hai, và quan trọng hơn, câu hỏi “biến này có hội tụ không” *luôn* là câu hỏi observability, không bao giờ là câu hỏi cài đặt. Khi hiệu chuẩn ngoại tuyến trôi lung tung, khi tỉ lệ đơn mắt sai, khi relocalization cho một tư thế trông hợp lý nhưng lệch nửa mét — ba triệu chứng đó có chung một gốc, và chương 5 là chỗ nói về cái gốc ấy.

TRỰC GIÁC

Hình dung mọi phép đo là một cái lò xo nối các biến chưa biết. Nghiệm của bài toán là vị trí cân bằng của cả hệ lò xo. Các bài toán trong Bảng 1 khác nhau ở chỗ bạn *hàn cứng* biến nào vào tường: hàn cấu trúc thì hệ giải ra tham số hiệu chuẩn, hàn hiệu chuẩn thì hệ giải ra quỹ đạo và bản đồ, không hàn gì thì hệ giải ra tất cả — nếu có đủ lò xo.

Cái “nếu” cuối câu chính là observability, và nó là chỗ khó thật sự. Nếu có một hướng mà dịch chuyển theo hướng đó không làm căng thêm bất kỳ lò xo nào, thì hệ có vô số vị trí cân bằng. Bộ giải vẫn trả về một con số, trông vẫn tự tin, và con số đó vô nghĩa theo đúng hướng ấy.

Một cách nói khác cho cùng ý, dùng được ngay khi debug: *calibration target chỉ là một cách giảm phương sai bằng cách fix cấu trúc*. Khi không fix được cấu trúc, bạn phải dùng chuyển động để thay thế. Đó là toàn bộ khác biệt giữa hiệu chuẩn trong phòng lab và hiệu chuẩn trực tuyến trên robot đang chạy.

Vì sao chia thành năm phần (The Five Parts)

Năm phần không phải năm chủ đề song song mà là năm tầng câu hỏi nối tiếp nhau. Phần A trả lời *camera thật sự đo cái gì, và ngôn ngữ toán nào đủ để nói về chuyển động và cấu trúc*. Phần B trả lời *làm thế nào biến chuỗi ảnh thành một ước lượng dùng được*, và nó chia đôi theo đúng ranh giới mà lĩnh vực tự chia: front-end lo chuyện tìm ra phép đo, back-end lo chuyện ghép phép đo thành trạng thái. Phần C là chỗ các mảnh rời trở thành một *hệ thống*: quản lý bản đồ, hiệu chuẩn như một nhánh của SLAM, hợp nhất đa cảm biến, biểu diễn dày, học máy, và cách đánh giá. Phần D là phần ít tài liệu học thuật nhất và tạo ra giá trị thật nhiều nhất: thế giới thực và dây chuyền sản xuất. Phần E là tầng công cụ và lộ trình — cố ý đặt cuối vì nó hết hạn nhanh nhất.

Bảng 2. Năm phần, câu hỏi mỗi phần trả lời, và mức độ ưu tiên.

	Phần	Câu hỏi nó trả lời	Mức
A	Nguyên lý và toán học	Camera đo cái gì, và cần đúng bao nhiêu toán để nói về nó?	cốt lõi
B	Front-end và back-end	Biến chuỗi ảnh thành trạng thái thế nào, và mỗi cách hỏng ở đâu?	cốt lõi
C	Hệ thống SLAM đầy đủ	Ghép các mảnh thành một hệ chạy được lâu dài thế nào?	cốt lõi
D	Thực tế và sản xuất	Vì sao hệ chạy tốt trên EuRoC lại chết ở nhà khách hàng?	cốt lõi
E	Framework và lộ trình	Ai đã cài sẵn cái gì, và đọc theo thứ tự nào?	công cụ

Bảng 2 cũng là thứ tự đọc đề xuất cho lần đầu, với một ngoại lệ đáng kể: nếu bạn đã có một hệ thống đang chạy và đang đau ở một chỗ cụ thể, hãy đọc chương 24 trước để biết đi thẳng tới đâu. Từ lần thứ hai trở đi, cách đọc hiệu quả hơn là trộn các chương xa nhau và tự kiểm tra chéo, vì gần như mọi chương trong cây đều là một cách nhìn khác của cùng vài nguyên lý.

Một lưu ý về phạm vi. Cây này *không* lặp lại phần quán tính và phần fiducial, vì hai nhánh đó đã có cây riêng: `../IMU_Fusion/` cho tiền tích phân, bộ lọc, khởi tạo thị giác-quán tính và hiệu chuẩn camera-IMU; `../marker/` cho detection marker, lưỡng nghĩa tư thế phẳng, constellation và ngân sách sai số docking. Chỗ nào giao nhau thì cây này trở sang bằng đường dẫn tương đối chứ không chép lại. Đây là quy tắc *một nguồn duy nhất* của kho, và nó quan trọng hơn sự tiện lợi của việc đọc một chỗ.

Ba mức ưu tiên, đánh dấu ngay ở tiêu đề chương (Three Priority Levels)

Không phải chương nào cũng đáng đầu tư như nhau. Cây dùng đúng ba mức, lấy từ quy ước trong lộ trình gốc, và mức xuất hiện ngay ở dòng đầu mỗi chương.

Bảng 3. Ba mức ưu tiên và cách đọc tương ứng.

Mức	Nghĩa	Cách đọc tương ứng
● ● ● xương sống	Không hiểu thì không hiểu được Visual SLAM	Đọc kỹ, tự dẫn lại công thức trên giấy , làm ví dụ nhỏ tính tay, trả lời hết câu hỏi tự kiểm
● ● quan trọng	Cần hiểu ý tưởng và biết khi nào dùng	Đọc hiểu cơ chế và điều kiện áp dụng; không cần tự dẫn lại toàn bộ
● bổ trợ	Biết là có, tra khi gặp vấn đề cụ thể	Đọc lướt một lượt để biết trong đó có gì, quay lại khi cần

Phân bố không đều là có chủ ý. Trong Phần A, chương 3 (Lie group), chương 4 (MAP và BA) và chương 5 (observability) là ● ● ●. Trong Phần B, chương 7 (đặc trưng và data association), chương 8 (mô hình phân dư), chương 9 (paradigm back-end) và chương 11 (loop closure) là ● ● ●. Trong Phần C, chương 13 (quản lý bản đồ) và chương 14 (hiệu chuẩn) là ● ● ● — chương 13 vì nó bị coi nhẹ nhất mà ảnh hưởng production lớn nhất, chương 14 vì nó là chỗ luận đề trung tâm được chứng minh bằng tay. Trong Phần D, chương 19 và 21 là ● ● ● vì mục tiêu của cây có bao gồm mass production.

Bốn nguyên lý xuyên suốt (Recurring Principles)

Bốn ý dưới đây xuất hiện lại ở những chỗ tưởng như không liên quan, và nhận ra chúng là dấu hiệu đã đọc đúng.

Thứ nhất, **camera đo hướng, không đo khoảng cách**. Một điểm ảnh xác định một tia, không xác định một điểm. Mọi thông tin về độ sâu trong hệ thị giác đều đến từ *sự khác biệt giữa hai tia*, tức từ parallax, và parallax chỉ sinh ra khi camera *dịch chuyển*. Đây là gốc của việc xoay thuận không triangulate được, của việc tỉ lệ đơn mắt không quan sát được, của việc điểm xa có bất định độ sâu lớn hơn điểm gần theo một quy luật cụ thể mà chương 6 sẽ tính ra, và của việc stereo có đường cơ sở cố định nên có thước đo còn mono thì không.

Thứ hai, **sai số có cấu trúc là dấu hiệu mô hình sai, sai số ngẫu nhiên là dấu hiệu nhiễu**. Khi phần dư reprojection phân bố ngẫu nhiên quanh không, bạn đang ở chế độ lành mạnh và chỉ cần trọng số đúng. Khi phần dư có hoa văn — tăng dần ra rìa ảnh, xoáy quanh tâm, tương quan với tốc độ góc — thì không có lượng dữ liệu nào cứu được, vì bạn đang khớp một mô hình sai. Phân biệt *model mismatch* với *parameter error* là kỹ năng chẩn đoán có giá trị cao nhất trong cả cây, và nó xuất hiện lại ở chương 2 (mô hình distortion), chương 14 (chất lượng hiệu chuẩn), chương 18 (NEES) và chương 21 (giám sát sức khoẻ tại hiện trường).

Thứ ba, **quan sát được hay không là tính chất của cặp (mô hình, chuyển động), không phải của thuật toán**. Không thuật toán nào ước lượng được một đại lượng mà dữ liệu không chứa thông tin về nó. Khi hệ thống hỏng, câu hỏi đầu tiên phải là “chuyển động vừa rồi có kích thích đủ để đại lượng này quan sát được không”, chứ không phải “có nên đổi sang bộ giải khác không”. Nguyên lý này chung với cây `.. /IMU_Fusion/` và được phát biểu ở đó cho trường hợp quán tính; chương 5 ở đây phát biểu nó cho trường hợp thuần thị giác, nơi nó còn gắt hơn vì mono thiếu cả tỉ lệ.

Thứ tư, **biết mình đang sai quan trọng hơn là không bao giờ sai**. Một hệ thống có tỉ lệ lỗi 2 % nhưng phát hiện được 90 % số lần lỗi thì dùng được trong sản phẩm; một hệ thống có tỉ lệ lỗi 1 % nhưng im lặng khi lỗi thì không. Nguyên lý này chi phối toàn bộ Phần D, và nó là lý do chương 6 (bất định) và chương 18 (đánh giá) được xếp mức cao hơn về ngoài “phụ trợ” của chúng. Nó cũng là lý do hướng nghiên cứu số 2 và số 4 ở chương 22 được đánh giá là còn nhiều khoảng trống thật.

Cách dùng cây (How to Use This Tree)

Mỗi thư mục có một tệp `00-map.tex` nói nhánh đó gồm gì và vì sao chia như vậy; đó là chỗ nên đọc trước khi vào nội dung. Mỗi tài liệu mở đầu bằng một dòng *Phụ thuộc* cho biết cần đọc gì trước, và kết bằng một mục *Kết nối* trỏ sang các nhánh liên quan cùng một câu giải thích quan hệ. Cuối mỗi tài liệu là ba tới bảy câu hỏi tự kiểm; theo lịch ôn của kho, hãy trả lời chúng *trước* khi mở lại bài, sau vài ngày, hai tuần, hai tháng, rồi nửa năm.

Tệp `99-chua-biet.md` ở gốc cây là sổ những điều chưa biết theo tinh thần của Feynman: mỗi mục là một câu hỏi cụ thể mà tôi biết là mình chưa trả lời được. Nó chỉ có ích khi được thu ngắn dần, nên mỗi lần giải quyết xong một mục thì chuyển nó thành một đoạn trong bài chính và ghi ngày.

CẠM BẮY

Cây này hiện ở trạng thái **bản nháp có kiểm soát**, và cần biết rõ điều đó trước khi dựa vào nó.

`refs.bib` được soạn từ trí nhớ về các công trình gốc: tên tác giả, năm và venue đủ chính xác để tra cứu, nhưng *chưa* được đối chiếu với trang gốc của nhà xuất bản. Theo

learning-rules.md mục 5, như vậy là chưa qua cửa kiểm chứng nào. Trước khi trích dẫn ra ngoài cây hoặc dùng để ra quyết định thật, phải mở bản gốc.

Quan trọng không kém: cây này **không chứa con số thực nghiệm nào do tôi đo được**. Mọi con số xuất hiện trong bài đều là một trong ba loại — ký hiệu toán, kết quả tính tay từ một ví dụ tự đặt, hoặc con số được dẫn nguồn kèm điều kiện đo. Chỗ nào tôi suy ra mà chưa đối chiếu được nguồn thì ghi thẳng (*tôi suy ra, chưa đối chiếu nguồn*). Một bảng so sánh APE giữa các hệ thống mà không kèm điều kiện đo là thứ cây này cố ý không có, vì các con số công bố của các nhóm khác nhau thường *không so sánh được với nhau*.

Câu hỏi còn mở (Open Questions)

Bốn điều cần xử lý khi quay lại cây này. Thứ nhất, toàn bộ refs.bib cần một lượt đối chiếu với nguồn gốc, và mỗi entry cần được dán mức tin A/B/C *sau khi* đã mở được ít nhất abstract. Thứ hai, các công thức lỗi — đặc biệt là quan hệ sai số pixel sang sai số độ sâu ở chương 6, phép khử landmark bằng Schur ở chương 4, và điều kiện suy biến của homography ở chương 1 — cần được kiểm bằng một ví dụ số nhỏ chạy được, và mã kiểm nên nằm trong thư mục code/ cạnh chương tương ứng. Thứ ba, Phần E chốt tại tháng 9/2026 và là phần hết hạn nhanh nhất; rà lại mỗi sáu tháng. Thứ tư, và đây là khoản nợ lớn nhất về mặt phương pháp: cây này được viết theo learning-rules.md (kho đã hiểu và sắp xếp xong), *không* theo research-rules.md (khảo sát có search log, ma trận tổng hợp, chỉ tiêu 25–40 công trình). Chương 22 — tám hướng còn trống — là chương duy nhất chạm vào địa hạt nghiên cứu, và nó cần được nâng lên thành một dự án survey/ đúng chuẩn trước khi dùng để chọn đề tài thật.

MỤC LỤC

Bản đồ cây

i

A	NGUYÊN LÝ VÀ TOÁN HỌC	1
1	Hình học đa thị (Multi-View Geometry)	4
1.1	Đặt vấn đề	5
1.2	Toạ độ thuần nhất và phép chiếu	6
1.3	Ràng buộc epipolar: E , F và H	7
1.4	Triangulation: từ hai tia ra một điểm	10
1.5	Từ hai khung sang nhiều khung	12
1.6	Cấu hình suy biến: mục quan trọng nhất của chương	12
1.7	Giới hạn của chương và chỗ nó nối sang	14
1.8	Điều gì chứng minh cách hiểu này sai	16
1.9	Kết nối	17
2	Mô hình camera (Camera Models)	19
2.1	Đặt vấn đề	20
2.2	Pinhole và ma trận nội	21
2.3	Các họ mô hình méo	22
2.4	Mô hình sai so với tham số sai	23
2.5	Ba mô hình mà SLAM hay quên	25
2.6	Điều gì chứng minh cách hiểu này sai	28
2.7	Kết nối	29
3	Lie group và tối ưu trên đa tạp (Lie Groups and Manifold Optimization)	30
3.1	Đặt vấn đề	31
3.2	$SO(3)$: xoay	32
3.3	Nhiều loạn: trái, phải, và vì sao phải chọn	33
3.4	$SE(3)$, $Sim(3)$ và Adjoint	34
3.5	Jacobian của phần dư chiếu: đích đến của chương	36
3.6	$\boxplus$, $\boxminus$ và vòng lặp tối ưu	37
3.7	Điều gì chứng minh cách hiểu này sai	39
3.8	Kết nối	39

4	Ước lượng MAP và bundle adjustment (MAP Estimation and Bundle Adjustment)	41
4.1	Đặt vấn đề	42
4.2	Từ Bayes tới bình phương tối thiểu	43
4.3	Đồ thị nhân tử: ngôn ngữ đúng để mô tả bài toán	45
4.4	Độ thừa và phần bù Schur	45
4.5	Marginalization: sống với thời gian vô hạn	47
4.6	Hàm mất mát bền: sống với dữ liệu thật	48
4.7	Điều gì chứng minh cách hiểu này sai	51
4.8	Kết nối	51
5	Observability, gauge và suy biến (Observability, Gauge and Degeneracy)	53
5.1	Đặt vấn đề	54
5.2	Observability, phát biểu cho đúng một lần	55
5.3	Bảy bậc tự do của SfM đơn mắt	56
5.4	Ba cách cố định gauge	57
5.5	Suy biến do chuyển động: mục thực dụng nhất	59
5.6	Nhất quán: khi bộ ước lượng tự lừa thông tin	60
5.7	Điều gì chứng minh cách hiểu này sai	63
5.8	Kết nối	63
6	Bất định và lan truyền sai số (Uncertainty and Error Propagation)	65
6.1	Đặt vấn đề	66
6.2	Hiệp phương sai từ ma trận thông tin	67
6.3	Lan truyền sai số: chuỗi của Visual SLAM	68
6.4	Bệnh quá tự tin	70
6.5	Điều gì chứng minh cách hiểu này sai	73
6.6	Kết nối	73
B	FRONT-END VÀ BACK-END	75
7	Đặc trưng và data association (Features and Data Association)	78
7.1	Đặt vấn đề	79
7.2	Tìm điểm đáng theo dõi	80
7.3	Khớp: mô tả rồi so, hay theo dõi trực tiếp	82
7.4	Hai thứ bị coi nhẹ nhất	83
7.5	Loại bỏ outlier	84
7.6	Điều gì chứng minh cách hiểu này sai	87
7.7	Kết nối	87

8	Mô hình phần dư (Residual Models)	89
8.1	Đặt vấn đề	90
8.2	Ba họ phần dư	91
8.3	Tham số hoá landmark	92
8.4	Các loại phần dư khác và cái giá ẩn	93
8.5	Điều gì chứng minh cách hiểu này sai	97
8.6	Kết nối	97
9	Paradigm ước lượng back-end (Back-End Estimation Paradigms)	99
9.1	Đặt vấn đề	100
9.2	Bộ lọc và tối ưu là cùng một bài toán	101
9.3	Bốn kiến trúc trên hai trục	102
9.4	Pose graph: bài toán MAP đã nén	103
9.5	Hai hướng phát triển đáng biết	103
9.6	Điều gì chứng minh cách hiểu này sai	107
9.7	Kết nối	107
10	Khởi tạo (Initialization)	109
10.1	Đặt vấn đề	110
10.2	Khởi tạo mono: bài toán khó nhất	111
10.3	Các trường hợp dễ hơn, và vì sao chúng dễ hơn	112
10.4	Tiêu chí từ chối: phần quan trọng nhất	113
10.5	Điều gì chứng minh cách hiểu này sai	117
10.6	Kết nối	117
11	Loop closure và nhất quán toàn cục (Loop Closure and Global Consistency)	119
11.1	Đặt vấn đề	120
11.2	Phát hiện: nhận ra chỗ cũ	121
11.3	Kiểm định hình học: tầng phòng thủ chính	122
11.4	Đóng vòng: áp dụng ràng buộc	123
11.5	Back-end bền: sống sót khi đã nhầm	124
11.6	Điều gì chứng minh cách hiểu này sai	127
11.7	Kết nối	128
12	Relocalization	130
12.1	Đặt vấn đề	131
12.2	Kiến trúc phân cấp	132
12.3	Các hướng học được	133

12.4	Kiểm định chấp nhận	134
12.5	Điều gì chứng minh cách hiểu này sai	137
12.6	Kết nối	138
C	HỆ THỐNG SLAM ĐẦY ĐỦ	140
13	Quản lý bản đồ (Map Management)	143
13.1	Đặt vấn đề	144
13.2	Chính sách chọn và loại	145
13.3	Cấu trúc đồ thị của bản đồ	146
13.4	Bản đồ sống lâu	147
13.5	Ràng buộc kỹ thuật thật	148
13.6	Điều gì chứng minh cách hiểu này sai	151
13.7	Kết nối	152
14	Hiệu chuẩn như một nhánh của SLAM (Calibration as a Branch of SLAM)	154
14.1	Đặt vấn đề	155
14.2	Đi từ ô này sang ô kia	156
14.3	Cổng observability: điều phân biệt có nguyên tắc với mù quáng	158
14.4	Tự đánh giá chất lượng hiệu chuẩn	159
14.5	Suy biến đặc thù của hiệu chuẩn	160
14.6	Điều gì chứng minh cách hiểu này sai	163
14.7	Kết nối	163
15	Hợp nhất đa cảm biến (Multi-Sensor Fusion)	165
15.1	Đặt vấn đề	166
15.2	Nguyên tắc và ba điều kiện	167
15.3	Bảng tra: mỗi cảm biến cho gì, đòi gì	168
15.4	Hai trường hợp hay bị làm sai	168
15.5	Điều gì chứng minh cách hiểu này sai	172
15.6	Kết nối	172
16	Dense mapping và biểu diễn (Dense Mapping and Representation)	174
16.1	Đặt vấn đề	175
16.2	Lấy độ sâu dày	176
16.3	Bốn họ biểu diễn cổ điển	177
16.4	Biểu diễn học được: ảnh chụp tại 9/2026	178

16.5	Đánh đổi bị bỏ qua	179
16.6	Điều gì chứng minh cách hiểu này sai	181
16.7	Kết nối	182
17	Học máy trong SLAM (Learning in SLAM)	183
17.1	Đặt vấn đề	184
17.2	Nơi học máy đã thắng	185
17.3	Nơi nó chưa thắng, và vì sao	186
17.4	Nơi nó nên đóng góp mà chưa	187
17.5	Điều gì chứng minh cách hiểu này sai	190
17.6	Kết nối	191
18	Đánh giá và phương pháp luận (Evaluation and Methodology)	192
18.1	Đặt vấn đề	193
18.2	Chỉ số quỹ đạo, và cái bẫy căn chỉnh	194
18.3	Đo tính trung thực, không chỉ độ chính xác	195
18.4	Đánh giá thành phần có chi phí lỗi bất đối xứng	196
18.5	Kỷ luật phương pháp	196
18.6	Điều gì chứng minh cách hiểu này sai	200
18.7	Kết nối	200
D	THỰC TẾ VÀ SẢN XUẤT	202
19	Bền vững trong thế giới thực (Robustness in the Real World)	205
19.1	Đặt vấn đề	206
19.2	Nhóm một: cảnh không tĩnh	207
19.3	Nhóm hai: thông tin không đủ	208
19.4	Nhóm ba: thông tin sai lệch	208
19.5	Nhóm bốn: cảm biến suy giảm	209
19.6	Phát hiện thất bại và suy giảm có kiểm soát	210
19.7	Điều gì chứng minh cách hiểu này sai	213
19.8	Kết nối	213
20	Tối ưu hiệu năng và hệ thống (Performance and Systems Engineering)	215
20.1	Đặt vấn đề	216
20.2	Thời gian: phân vị, không phải trung bình	217
20.3	Kiến trúc đa luồng và tính tất định	218
20.4	Ngân sách tính toán và bộ nhớ	218

20.5	Điện năng, nhiệt, và một vòng lặp bị bỏ quên	219
20.6	Điều gì chứng minh cách hiểu này sai	222
20.7	Kết nối	223
21	Từ prototype lên mass production (From Prototype to Mass Production)	224
21.1	Đặt vấn đề	225
21.2	A — Hiệu chuẩn ở quy mô sản xuất.	226
21.3	B — Độ tin cậy và fleet analytics	228
21.4	C — Kiểm thử	229
21.5	D — Ràng buộc sản phẩm.	230
21.6	Điều gì chứng minh cách hiểu này sai	233
21.7	Kết nối	234
22	Tám hướng còn khoảng trống (Eight Open Directions)	235
22.1	Vì sao một danh sách như thế này lại có giá trị	235
22.2	Hướng 1 — Hiệu chuẩn trực tuyến có ý thức observability	236
22.3	Hướng 2 — Bất định đáng tin	236
22.4	Hướng 3 — Bản đồ sống lâu trong thế giới thay đổi	237
22.5	Hướng 4 — Dự đoán thất bại thay vì phục hồi sau	237
22.6	Hướng 5 — Bất định được truyền đúng qua ranh giới học/hình học	238
22.7	Hướng 6 — Xử lý suy biến có nguyên tắc	238
22.8	Hướng 7 — Hệ tự nhận biết lệch và tự sửa.	238
22.9	Hướng 8 — Biểu diễn neural cho localization thực dụng	239
22.10	Điều gì chứng minh cách hiểu này sai	240
22.11	Kết nối	240
E	FRAMEWORK VÀ LỘ TRÌNH	241
23	Framework và công cụ (Frameworks and Tools)	243
23.1	Vì sao đọc mã nguồn.	243
23.2	Sáu hệ thống đáng đọc	244
23.3	Các hệ thống khác, theo chủ đề	244
23.4	Thư viện nền	245
23.5	Bộ dữ liệu và công cụ đánh giá.	245
23.6	Kết nối	246

24	Lộ trình (Roadmap)	247
24.1	Vì sao thứ tự quan trọng	247
24.2	Giai đoạn 1 — Nền tảng, không rút ngắn được	247
24.3	Giai đoạn 2 — Đọc hệ thống thật	248
24.4	Giai đoạn 3 — Hợp nhất với hiệu chuẩn	248
24.5	Giai đoạn 4 — Đối mặt thực tế	249
24.6	Giai đoạn 5 — Nghiên cứu	250
24.7	Một gợi ý về chỗ bắt đầu	250
	Số những điều chưa biết	252

NGUYÊN LÝ VÀ TOÁN HỌC



TÓM TẮT

Phần A dựng đúng lượng nền cần thiết cho bốn phần sau: hình học của việc nhìn một cảnh từ nhiều chỗ, mô hình biến tia sáng thành pixel và chỗ mô hình đó sai, ngôn ngữ toán để tối ưu trên đa tạp, khung MAP hợp nhất SLAM với hiệu chuẩn, lý thuyết quyết định khi nào một đại lượng ước lượng được, và cuối cùng là cách sai số lan truyền từ pixel ra tới tư thế. Nếu chỉ nhớ một điều: bốn chương đầu là công cụ, chương 5 giải thích *vì sao* mọi thứ hỏng, và chương 6 giải thích *hỏng bao nhiêu* — hai chương cuối bị bỏ qua nhiều nhất và có giá trị chẩn đoán cao nhất.

Phần này tồn tại để phục vụ bốn phần sau, và nguyên tắc chi phối nó là *học theo nhu cầu*: mỗi chương chỉ đi tới mức dùng được và giải thích được, không chứng minh định lý khi một suy dẫn ngắn đã đủ. Đây là lựa chọn có chủ ý. Một người dừng lại học trọn vẹn hình học xạ ảnh trước khi chạm vào hệ thống SLAM đầu tiên thường không bao giờ tới được hệ thống đầu tiên; ngược lại, một người bỏ hẳn phần nền sẽ đọc mã nguồn ORB-SLAM3 như đọc bùa chú và không đoán được nó sẽ hỏng ở đâu. Ranh giới đúng nằm ở chỗ: học đủ để *tự dẫn lại được khi cần* và để *nhận ra khi một giả định bị vi phạm*.

Sáu chương có vai trò rất khác nhau, và mức đầu tư nên khác nhau theo. Chương 1 là hình học đa thị: nó nói hai ảnh của cùng một cảnh ràng buộc nhau thế nào, và quan trọng hơn, nó liệt kê các cấu hình mà ràng buộc ấy sụp đổ — cảnh phẳng, xoay thuần. Chương 2 là mô hình camera: nó nói tia sáng thành pixel bằng cách nào, và nó dạy một kỹ năng chẩn đoán mà cả cây dùng lại liên tục, là phân biệt *mô hình sai* với *tham số sai*. Chương 3 là ngôn ngữ: không có Lie group thì mọi Jacobian trong BA, trong hiệu chuẩn, trong pose graph đều là hộp đen. Chương 4 dựng khung MAP và cấu trúc thừa của nó, tức chỗ SLAM và hiệu chuẩn gặp nhau, và chỗ phần bù Schur biến một bài toán không giải nổi thành một bài toán chạy thời gian thực. Chương 5 hỏi câu hỏi mà bốn chương trước không hỏi được: trong số các biến ta vừa đặt vào bài toán, biến nào dữ liệu thật sự nói được điều gì đó? Chương 6 hỏi câu tiếp theo, quan trọng không kém: với các biến quan sát được, ta biết chúng chính xác đến đâu, và con số bất định ta báo cáo có trung thực không?

Thứ tự đọc đề xuất là tuần tự, nhưng với hai lưu ý. Chương 3 dài và dễ gây nản nếu đọc một mạch; cách tốt hơn là đọc tới hết phần $SO(3)$ và phép nhiễu loạn, sang chương 4, rồi quay lại lấy nốt Adjoint và $Sim(3)$ khi chương 11 cần đến. Chương 5 thì ngược lại: nên đọc kỹ ngay cả khi chưa hiểu hết chi tiết, vì nó đổi cách bạn nhìn mọi chương sau. Và nếu bạn đã đọc cây . . / IMU_Fusion/, ba chương 3, 4, 5 ở đây có phần chồng lấn với chương 2, 4, 5 ở đó; phần chồng lấn được viết lại từ góc nhìn thị giác chứ không chép sang, và chỗ nào một cây đã nói kỹ hơn thì cây kia trở sang.

Bảng 4. Bản đồ Phần A: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
1	01-hình-học-da-thị	Toạ độ thuần nhất; ràng buộc epipolar, E , F , H ; triangulation DLT/midpoint/optimal; cheirality; phân rã E và H ; cấu hình suy biến	••
2	02-mô-hình-camera	Pinhole và K ; các họ distortion (radtan, KB, UCM, EUCM, double-sphere); mô hình sai so với tham số sai; rolling shutter; mô hình quang trắc; stereo và disparity	••
3	03-lie-group-va-toi-ic	$SO(3)$, $SE(3)$, $Sim(3)$; exp/log, Adjoint, BCH, Jacobian phải; nhiễu loạn trái/phải; $\boxplus/\boxminus$; đạo hàm của phần dư chiều	•••
4	04-uoc-luong-map-va-l	Bayes tới MAP tới bình phương tối thiểu; Gauss-Newton, LM, dogleg; đồ thị nhân tử; độ thừa, Schur, marginalization; M-estimator, IRLS, GNC	•••
5	05-observability-gau	Bảy bậc tự do của SfM đơn mắt; ba cách cố định gauge; chuyển động suy biến; điều kiện kích thích; nhất quán, FEJ	•••
6	06-bat-dinh-va-lan-t	Hiệp phương sai từ Hessian; lan truyền trên đa tạp; pixel $\rightarrow$ độ sâu $\rightarrow$ tư thế; khôi phục hiệp phương sai thừa; NEES và bệnh quá tự tin	••

Chương 1 và 2 có thể đọc lướt nếu bạn đã quen hình học đa thị — nhưng *đừng* lướt hai mục về cấu hình suy biến ở chương 1 và mục “mô hình sai so với tham số sai” ở chương 2, vì hai mục đó là nơi hai chương trả lại giá trị. Ba chương 3, 4, 5 là xương sống và nên đọc theo đúng thứ tự, vì chương 5 chỉ có nghĩa khi đã có ma trận thông tin từ chương 4, và ma trận đó chỉ dựng được khi đã có Jacobian trên đa tạp từ chương 3. Chương 6 đọc sau cùng và đọc kỹ: nó là chương duy nhất trả lời câu “con số bất định tôi vừa in ra có nghĩa gì không”, và câu đó là câu quyết định hệ thống có dùng được trong sản phẩm hay không.

Hình học đa thị (Multi-View Geometry)

PHỤ THUỘC Không có chương trước. Cần sẵn: đại số tuyến tính ở mức SVD, hạng ma trận, không gian rỗng. Chương này là nền cho [A/02-mo-hinh-camera](#) (chỗ tia thành pixel), [B/10-khoi-tao](#) (khởi tạo hai khung) và [B/07-dac-trung-va-data-association](#) (kiểm định hình học sau khớp điểm).

TÓM TẮT

Chương này trả lời một câu hỏi duy nhất: hai ảnh của cùng một cảnh tĩnh ràng buộc nhau như thế nào, và khi nào ràng buộc ấy sụp đổ? Câu trả lời có ba tầng. Tầng đại số: một điểm ở ảnh này bắt buộc nằm trên một đường thẳng ở ảnh kia — đường epipolar — và toàn bộ quan hệ đó gói trong một ma trận 3×3 hạng hai. Tầng hình học: từ ràng buộc đó ta lấy ra được tư thế tương đối tới một hệ số tỉ lệ, rồi lấy ra được cấu trúc ba chiều bằng triangulation. Tầng suy biến, tầng quan trọng nhất trong thực hành: có những cấu hình — cảnh phẳng, xoay thuần, chuyển động dọc trục quang — mà quy trình trên trả về một kết quả trông bình thường nhưng vô nghĩa. Đọc xong bạn tự dựng lại được toàn bộ chuỗi từ hai tập điểm khớp tới một đám mây điểm ba chiều, và quan trọng hơn, bạn biết trước những chuỗi ảnh nào sẽ làm chuỗi đó vỡ. Nếu chỉ nhớ một điều: *camera đo hướng chứ không đo khoảng cách, nên mọi thông tin về độ sâu đều đến từ sự khác biệt giữa hai tia, tức từ parallax, và parallax chỉ sinh ra khi camera dịch chuyển* — xoay đầu không cho biết gì về độ sâu, dù xoay bao nhiêu độ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **toạ độ thuần nhất (homogeneous coordinates)** — biểu diễn một điểm 2D bằng một vectơ 3 thành phần (x, y, w) , trong đó (x, y, w) và $\lambda(x, y, w)$ với $\lambda \neq 0$ là cùng một điểm. Toạ độ Euclid lấy lại bằng $(x/w, y/w)$. Lợi ích: phép chiếu phối cảnh — vốn phi tuyến trong toạ độ Euclid — trở thành một phép nhân ma trận, và các điểm “ở vô hạn” ($w = 0$) có chỗ đứng hợp lệ thay vì là trường hợp ngoại lệ.
- **mặt phẳng chuẩn hoá (normalized plane)** — mặt phẳng $z = 1$ trong hệ toạ độ camera. Một điểm 3D (X, Y, Z) chiếu về $(X/Z, Y/Z, 1)$. Đây là ảnh của cảnh khi đã gỡ bỏ hoàn toàn ảnh hưởng của tham số nội, nên mọi công thức hình học ở chương này viết trên mặt phẳng chuẩn hoá cho gọn.
- **parallax** — độ lệch hướng nhìn tới cùng một điểm 3D khi camera dịch chuyển. Đo bằng góc giữa hai tia. Parallax lớn thì độ sâu xác định tốt; parallax bằng không thì độ sâu hoàn toàn không xác định.
- **epipole** — ảnh của tâm camera này trên mặt phẳng ảnh của camera kia. Mọi đường epipolar trong một ảnh đều đi qua epipole của ảnh đó.
- **cheirality** — ràng buộc rằng điểm 3D phải nằm *trước* camera, tức có độ sâu dương trong cả hai khung. Về mặt hình học thuần tuý, một điểm sau lưng camera vẫn chiếu ra một pixel hợp lệ; cheirality là ràng buộc vật lý loại bỏ các nghiệm như vậy.
- **cấu hình suy biến (degenerate configuration)** — một bố trí cảnh hoặc chuyển động làm cho bài toán mất nghiệm duy nhất: cảnh phẳng, xoay thuần, mọi điểm ở vô hạn, hoặc camera và các điểm cùng nằm trên một mặt bậc hai đặc biệt.
- **DLT (direct linear transform)** — kỹ thuật chung: biến một ràng buộc hình học thành một hệ phương trình tuyến tính thuần nhất $A\theta = 0$, rồi lấy nghiệm là vectơ kỳ dị phải ứng với giá trị kỳ dị nhỏ nhất của A . Nhanh và không cần khởi tạo, nhưng cực tiểu hoá một sai số *đại số* không có ý nghĩa vật lý.

CÂU HỎI MỞ ĐẦU

- Q1. Một camera đơn mắt quay tại chỗ 360° và chụp một nghìn ảnh của một căn phòng đầy ván. Mọi cặp ảnh liên kế khớp điểm hoàn hảo. Vì sao không thể dựng lại được bất kỳ điểm 3D nào, và vì sao “chụp thêm ảnh” không cứu được?
- Q2. Ma trận essential E có chín phần tử nhưng chỉ mã hoá năm bậc tự do. Hai bậc bị mất đi đâu, và vì sao thuật toán tám điểm vẫn chạy được dù nó bỏ qua điều này?
- Q3. Phân rã E cho ra đúng bốn nghiệm, ba trong số đó vô lý về mặt vật lý. Bốn nghiệm đó khác nhau ở chỗ nào, và vì sao phép thử loại chúng lại là một ràng buộc *vật lý* chứ không phải một ràng buộc hình học?
- Q4. Với cùng một cặp điểm khớp và cùng một cặp tư thế, ba phương pháp triangulation — DLT, midpoint, optimal — cho ba kết quả khác nhau. Sự khác nhau đó lớn nhất ở đâu, và vì sao điều đó giải thích được bằng một câu về parallax?
- Q5. Hệ của bạn khởi tạo tốt trong hành lang nhưng thất bại khi quay mặt vào tường. Tường có vân rõ, khớp điểm tốt, RANSAC cho nhiều inlier. Cơ chế nào đang hỏng, và vì sao ORB-SLAM phải chạy *hai* mô hình song song thay vì chọn sẵn một?

1.1 Đặt vấn đề

Bài toán gốc của cả lĩnh vực có thể phát biểu trong một câu: *cho một chuỗi ảnh hai chiều của một cảnh ba chiều, khôi phục lại cảnh đó và đường đi của camera*. Câu hỏi này cũ hơn nhiều so với robot học — nó là bài toán của trắc địa ảnh (photogrammetry) từ đầu thế kỷ hai mươi, khi người ta dựng bản đồ địa hình từ ảnh chụp trên khinh khí cầu — nhưng nó vẫn là bài toán trung tâm, vì một lý do đơn giản: ảnh là loại dữ liệu giàu thông tin nhất trên mỗi đồng chi phí và mỗi watt điện.

Cái khó nằm ở chỗ phép chiếu *mất thông tin một cách không phục hồi được*. Một camera lấy một điểm ba chiều và trả về hai con số. Một chiều đã biến mất, và nó biến mất theo một kiểu đặc biệt khó chịu: toàn bộ tia sáng đi qua tâm quang học chiếu về đúng một pixel. Nói cách khác, *camera đo hướng, không đo khoảng cách*. Từ một ảnh duy nhất của một quả bóng, không có cách nào phân biệt một quả bóng nhỏ ở gần với một quả bóng to ở xa. Đây không phải giới hạn của thuật toán mà là giới hạn của phép đo, và mọi thứ trong chương này là cách đối phó với nó.

Cách đối phó duy nhất là *nhìn từ chỗ khác*. Khi camera dịch chuyển, tia nhìn tới cùng một điểm đổi hướng, và độ đổi hướng ấy — parallax — chứa thông tin về độ sâu. Đó là lý do một người nhắm một mắt vẫn ước lượng được khoảng cách khi lắc đầu, và cũng là lý do người đó không ước lượng được gì khi ngồi im. Toàn bộ hình học đa thị là việc hình thức hoá quan sát đơn giản ấy, và điều đáng ngạc nhiên là nó dẫn tới một cấu trúc đại số gọn gàng bất ngờ.

Trước khi có cấu trúc đó, cách tiếp cận hiển nhiên là thử trực tiếp: đặt ẩn là tư thế tương đối và toạ độ các điểm, viết sai số chiếu, rồi tối ưu. Cách này *đúng* — nó chính là bundle adjustment ở A/04 — nhưng nó hỏng ở một chỗ chí mạng: tối ưu phi tuyến cần một điểm khởi đầu đủ gần, và ta chưa có gì cả. Thiếu khởi tạo thì bộ giải rơi vào cực tiểu địa phương, và với bài toán này cực tiểu địa phương không hiếm mà là quy luật. Vậy câu hỏi thực sự của chương là: *làm sao có được một nghiệm xấp xỉ mà không cần nghiệm xấp xỉ nào trước đó?*

Câu trả lời là khai thác một cấu trúc đại số ẩn trong hình học: ràng buộc epipolar. Nó cho phép ước lượng tư thế tương đối bằng cách giải một hệ tuyến tính — không cần khởi tạo, không có cực tiểu địa phương — rồi từ đó triangulate ra cấu trúc. Cái giá phải trả là hai thứ, và cả hai đều đáng nói ngay vì chúng theo ta suốt cây. Thứ nhất, nghiệm tuyến tính cực tiểu hoá một *sai số đại số* không phải sai số vật lý, nên nó chỉ dùng làm khởi tạo chứ không dùng làm kết quả. Thứ hai, và nghiêm trọng hơn, phương pháp này có những *cấu hình suy biến* mà ở đó nó không chỉ kém chính xác mà hoàn toàn vô nghĩa — trong khi vẫn trả về một ma trận trông bình thường. Mục 6

dành riêng cho chuyện đó, và nếu chỉ có thời gian cho một mục của chương thì đó là mục nên đọc.

TRỰC GIÁC

Đứng ở hai chỗ khác nhau nhìn cùng một vật. Nối tâm mắt trái, tâm mắt phải và điểm vật — ba điểm ấy xác định một mặt phẳng. Toàn bộ hình học epipolar là hệ quả của sự kiện tầm thường này.

Vì điểm vật nằm trên mặt phẳng đó, nên ảnh của nó trên võng mạc phải cũng nằm trên giao tuyến của mặt phẳng đó với võng mạc phải — một *đường thẳng*. Nghĩa là: biết một điểm trong ảnh trái, bạn không biết nó ở đâu trong ảnh phải, nhưng bạn biết nó nằm trên đúng một đường thẳng. Bài toán tìm kiếm hai chiều rút xuống một chiều, miễn phí.

Đường thẳng ấy là đường epipolar, và ma trận biến điểm trái thành đường phải là ma trận essential. Nó không chứa gì hơn ngoài thông tin “hai mắt đặt ở đâu so với nhau” — chính vì thế mà từ nó lấy ngược ra được tư thế tương đối.

Chỗ mọi thứ sụp đổ cũng nhìn ra từ đây. Nếu hai tâm mắt *trùng nhau* — tức camera chỉ xoay chứ không dịch — thì ba điểm không xác định mặt phẳng nào nữa, đường epipolar không tồn tại, và cả cấu trúc biến mất. Đó là lý do xoay thuần là kẻ thù số một của SLAM đơn mắt.

1.2 Toạ độ thuần nhất và phép chiếu

Trước khi vào ràng buộc chính, cần thống nhất ký hiệu, và lựa chọn ký hiệu ở đây không phải chuyện hình thức: dùng toạ độ thuần nhất biến một loạt trường hợp đặc biệt thành trường hợp thường.

Một điểm 3D trong hệ thế giới viết là $\mathbf{P} = (X, Y, Z)^\top$, và dạng thuần nhất của nó là $\tilde{\mathbf{P}} = (X, Y, Z, 1)^\top$. Một camera ở tư thế $(R, \mathbf{t})$ — với quy ước R và $\mathbf{t}$ biến toạ độ thế giới sang toạ độ camera — nhìn thấy điểm đó ở

$$\mathbf{P}_c = R\mathbf{P} + \mathbf{t}, \quad \mathbf{x} = \pi(\mathbf{P}_c) = \frac{1}{Z_c} \begin{pmatrix} X_c \\ Y_c \end{pmatrix},$$

với $\mathbf{x}$ là toạ độ trên mặt phẳng chuẩn hoá. Việc biến $\mathbf{x}$ thành pixel là việc của [A/02](#); chương này làm việc hoàn toàn trên mặt phẳng chuẩn hoá, tức giả định camera đã hiệu chuẩn nội. Giả định đó đáng nhắc lại ở đây vì nó sẽ bị bỏ ở [C/14](#): khi tham số nội trở thành ẩn, mọi thứ trong chương này vẫn đúng nhưng không còn dùng trực tiếp được, và đó chính là ranh giới giữa ma trận essential và ma trận fundamental ở mục 3D.

Sức mạnh của toạ độ thuần nhất nằm ở chỗ nó gộp hai thứ vốn tách rời. Trong toạ độ Euclid, phép chiếu là một phép chia — phi tuyến, và có kỳ dị ở $Z_c = 0$. Trong toạ độ thuần nhất, cùng phép chiếu ấy viết là $\tilde{\mathbf{x}} \sim [R \mid \mathbf{t}] \tilde{\mathbf{P}}$, trong đó $\sim$ nghĩa là “bằng nhau tới một hệ số tỉ lệ khác không”. Phép chia biến mất vào trong quan hệ $\sim$, và toàn bộ biểu thức trở thành tuyến tính. Đây không phải mẹo ký hiệu: chính nhờ tính tuyến tính này mà các phương pháp DLT ở mục 3 và mục 4 tồn tại.

Một lợi ích thứ hai, ít rõ hơn nhưng quan trọng trong SLAM. Toạ độ thuần nhất cho phép biểu diễn *điểm ở vô hạn*: $\tilde{\mathbf{P}} = (X, Y, Z, 0)^\top$ là một hướng chứ không phải một vị trí. Điều này không phải trò chơi toán học, vì trong thực tế các điểm rất xa — toà nhà ở cuối phố, đám mây — hành xử *gần như* điểm ở vô hạn: chúng không cho thông tin gì về tịnh tiến nhưng cho thông tin rất tốt về xoay. Nhận ra điều này giải thích một hiện tượng thực tế quan trọng: một hệ SLAM ngoài trời thường ước lượng hướng rất tốt mà vị trí rất tệ, và lý do là phần lớn đặc trưng nó nhìn thấy đều ở

xa. Cách xử lý đúng là tham số hoá bằng nghịch đảo độ sâu, chuyện của B/08, nơi điểm ở vô hạn tương ứng với nghịch đảo độ sâu bằng không — một giá trị hoàn toàn hợp lệ chứ không phải kỳ dị.

1.3 Ràng buộc epipolar: E , F và H

A. Suy dẫn ràng buộc từ tính đồng phẳng

Xét hai camera. Đặt khung thứ nhất làm gốc, và khung thứ hai ở tư thế tương đối $(R, \mathbf{t})$. Một điểm 3D nhìn từ khung một có tọa độ $\mathbf{P}_1$, nhìn từ khung hai có tọa độ $\mathbf{P}_2 = R\mathbf{P}_1 + \mathbf{t}$.

Ba vectơ sau cùng nằm trên một mặt phẳng, khi biểu diễn trong hệ của khung hai: vectơ $\mathbf{P}_2$ (từ tâm camera hai tới điểm), vectơ $\mathbf{t}$ (từ tâm camera hai tới tâm camera một, sai khác dấu), và vectơ $R\mathbf{P}_1$ (tia của camera một, đã xoay về hệ hai). Đây chính là mặt phẳng epipolar trong khối *Trực giác* ở trên. Điều kiện đồng phẳng của ba vectơ viết bằng tích hỗn tạp bằng không:

$$\mathbf{P}_2^\top (\mathbf{t} \times (R\mathbf{P}_1)) = 0.$$

Vì sao bước này hợp lệ? Vì tích có hướng $\mathbf{t} \times (R\mathbf{P}_1)$ là một vectơ vuông góc với mặt phẳng chứa $\mathbf{t}$ và $R\mathbf{P}_1$; tích vô hướng của nó với $\mathbf{P}_2$ bằng không đúng khi $\mathbf{P}_2$ cũng nằm trong mặt phẳng ấy. Viết tích có hướng thành phép nhân ma trận phản đối xứng $[\mathbf{t}]_\times$, ta có

$$\mathbf{P}_2^\top [\mathbf{t}]_\times R\mathbf{P}_1 = 0.$$

Bước cuối là chỗ tọa độ thuần nhất trả công. Điểm chiếu chuẩn hoá $\mathbf{x}_i$ chỉ khác $\mathbf{P}_i$ một hệ số tỉ lệ (chính là độ sâu), và phương trình trên thuần nhất bậc một theo cả $\mathbf{P}_1$ lẫn $\mathbf{P}_2$, nên hệ số tỉ lệ triệt tiêu hoàn toàn:

$$\boxed{\mathbf{x}_2^\top E \mathbf{x}_1 = 0, \quad E = [\mathbf{t}]_\times R} \quad (1.1)$$

Đây là **ràng buộc epipolar**, và E là **ma trận essential** (*essential matrix*). Kết quả thuộc về Longuet-Higgins [1]; trình bày hiện đại đầy đủ nhất nằm ở [2, §9.6]. Tôi đã tự dẫn lại toàn bộ bước trên (cửa kiểm chứng a) và đối chiếu kết luận với hai nguồn độc lập là [2] và [3] (cửa c).

Điều đáng dừng lại một câu: phương trình (1.1) *không* chứa $\mathbf{P}$. Tọa độ 3D đã biến mất hoàn toàn. Nghĩa là ràng buộc này đúng cho *mọi* điểm khớp, bất kể cấu trúc cảnh — và đó là lý do nó dùng được để ước lượng tư thế tương đối *trước khi* biết gì về cấu trúc. Đây là bước tách bài toán “ước lượng đồng thời tư thế và cấu trúc” thành hai bài toán tuần tự, và sự tách đó là điều làm cho khởi tạo khả thi.

B. Năm bậc tự do trong chín con số

E có chín phần tử nhưng chỉ mã hoá năm bậc tự do, và hiểu vì sao là hiểu bản chất của nó.

Đếm bậc tự do vật lý trước: tư thế tương đối $(R, \mathbf{t})$ có sáu bậc, ba xoay và ba tịnh tiến. Nhưng phương trình (1.1) thuần nhất theo E : nếu E thoả thì λE cũng thoả với mọi $\lambda \neq 0$. Vì E tuyến tính theo $\mathbf{t}$, điều này nghĩa là *độ lớn của $\mathbf{t}$ không quan sát được* — chỉ hướng của nó là. Mất một bậc, còn năm.

Đây là lần đầu tiên trong cây ta gặp **tỉ lệ không quan sát được** (*scale unobservability*), và nó sẽ trở lại ở A/05 dưới dạng tổng quát hơn. Cơ chế thì đã nói ở phần đặt vấn đề: camera đo hướng. Nhân đôi toàn bộ thế giới — mọi điểm, mọi đường cơ sở — thì mọi tia nhìn giữ nguyên, nên mọi ảnh

giữ nguyên. Không có phép đo thị giác nào phân biệt được hai thể giới ấy. Muốn có tỉ lệ tuyệt đối phải *thêm thông tin*: đường cơ sở stereo đã biết ([A/02](#)), gia tốc kế (`./IMU_Fusion/`), một vật có kích thước đã biết như marker (`./marker/`), hoặc một tiên nghiệm học được về kích thước vật thể ([C/17](#)).

Hệ quả đại số của cấu trúc $E = [\mathbf{t}]_{\times} R$ là một ràng buộc đặc trưng: E có hạng hai, và hai giá trị kỳ dị khác không của nó bằng nhau. Lý do ngắn gọn: $[\mathbf{t}]_{\times}$ có hạng hai (không gian rỗng của nó là chính $\mathbf{t}$) và các giá trị kỳ dị khác không của nó đều bằng $\|\mathbf{t}\|$; nhân thêm ma trận trực giao R không đổi phổ giá trị kỳ dị. Ràng buộc này quan trọng trong thực hành vì thuật toán tám điểm *bỏ qua* nó: nó giải hệ tuyến tính tám phương trình và thu được một ma trận bất kỳ, nói chung không có dạng $[\mathbf{t}]_{\times} R$. Cách chữa chuẩn là chiếu về tập hợp hợp lệ bằng SVD: lấy $E = U \text{diag}(\sigma_1, \sigma_2, \sigma_3) V^{\top}$ rồi thay bằng $U \text{diag}(1, 1, 0) V^{\top}$.

CẠM BẦY

Thuật toán **tám điểm** dùng tám ràng buộc cho một ma trận chín phần tử (trừ đi tỉ lệ chung là tám ẩn), nên nó là hệ tuyến tính đúng cỡ — nhưng nó cần tám điểm trong khi bài toán chỉ có năm bậc tự do. Ba ràng buộc thừa ấy là cái giá của việc tuyến tính hoá, và cái giá thật sự không phải số điểm mà là **độ nhạy với nhiễu**.

Thuật toán **năm điểm** [\[4\]](#) khai thác đúng năm bậc tự do và ràng buộc phổ giá trị kỳ dị, nên nó cần ít điểm hơn trong mỗi mẫu RANSAC. Điều này không phải chuyện tiết kiệm: số lần lặp RANSAC cần thiết tăng theo *hàm mũ* của cỡ mẫu tối thiểu, nên đi từ tám xuống năm là một khác biệt lớn khi tỉ lệ outlier cao. Chi tiết ở [B/07](#).

Quan trọng hơn cả: thuật toán năm điểm **không suy biến với cảnh phẳng**, trong khi tám điểm thì có — xem mục 6. Đây là lý do thực dụng chính để dùng nó, và nó bị nhắc tới ít hơn nhiều so với chuyện số điểm.

C. Ví dụ nhỏ tính tay: stereo chuẩn

Lấy trường hợp đơn giản nhất để có số cụ thể. Giả sử hai camera song song, camera hai dịch sang phải một đoạn b so với camera một, không xoay. Trong quy ước “ $R, \mathbf{t}$ biến toạ độ thể giới sang toạ độ camera”, với khung một làm gốc thể giới, ta có $R = I$ và $\mathbf{t} = (-b, 0, 0)^{\top}$ — dấu âm vì camera dịch sang phải thì cảnh dịch sang trái trong hệ của nó.

Tính E trực tiếp:

$$E = [\mathbf{t}]_{\times} R = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & b \\ 0 & -b & 0 \end{pmatrix}.$$

Kiểm hạng: hàng đầu bằng không, hai hàng còn lại độc lập tuyến tính, vậy hạng đúng bằng hai. Kiểm phổ giá trị kỳ dị: khối 2×2 ở góc dưới phải là b lần một ma trận trực giao, nên hai giá trị kỳ dị khác không đều bằng b , bằng nhau đúng như mục B dự đoán. Cả hai tính chất được xác nhận bằng số (cửa kiểm chứng b).

Bây giờ thay vào ràng buộc epipolar với $\mathbf{x}_1 = (u_1, v_1, 1)^{\top}$ và $\mathbf{x}_2 = (u_2, v_2, 1)^{\top}$:

$$\mathbf{x}_2^{\top} E \mathbf{x}_1 = b(v_1 - v_2) = 0 \implies v_1 = v_2.$$

Ràng buộc epipolar rút gọn thành *hai điểm khớp phải có cùng toạ độ dọc*. Đây đúng là tính chất mà stereo rectification tạo ra một cách nhân tạo cho cấu hình bất kỳ, và là lý do tìm kiếm khớp điểm trong stereo đã hiệu chỉnh chỉ cần quét dọc một hàng. Ví dụ nhỏ này vừa kiểm chứng công thức, vừa cho thấy stereo rectification không phải một thủ thuật riêng mà là một trường hợp đặc biệt của hình học epipolar.

D. Fundamental và homography: hai họ hàng cần phân biệt

Ma trận E giả định camera đã hiệu chuẩn nội, tức ta làm việc trên mặt phẳng chuẩn hoá. Khi không có hiệu chuẩn, ta chỉ có pixel, và quan hệ tương ứng là **ma trận fundamental** (*fundamental matrix*):

$$\mathbf{p}_2^\top F \mathbf{p}_1 = 0, \quad F = K_2^{-\top} E K_1^{-1},$$

với $\mathbf{p}_i$ là toạ độ pixel thuần nhất và K_i là ma trận nội. F có bảy bậc tự do (chín phần tử, trừ tỉ lệ chung, trừ ràng buộc $\det F = 0$), nhiều hơn E đúng hai bậc — hai bậc ấy chính là phần thông tin nội mà ta không biết. Trong Visual SLAM, F ít được dùng trực tiếp vì ta gần như luôn đã hiệu chuẩn; chỗ nó có giá trị là SfM từ ảnh internet, nơi tham số nội là ẩn (C/14).

Homography (*homography*) H là chuyện khác hẳn và dễ nhầm. Nó là một ma trận 3×3 khả nghịch biến *điểm thành điểm*, không phải *điểm thành đường*:

$$\mathbf{x}_2 \sim H \mathbf{x}_1.$$

Quan hệ này đúng trong đúng hai trường hợp, và biết rõ hai trường hợp ấy là điều quan trọng nhất cần nhớ về H . Thứ nhất, khi mọi điểm nằm trên *một mặt phẳng* trong không gian — lúc đó $H = R + \frac{1}{d} \mathbf{t} \mathbf{n}^\top$ với $\mathbf{n}$ là pháp tuyến mặt phẳng và d là khoảng cách từ camera tới mặt phẳng đó [2, §13]. Thứ hai, khi camera *chỉ xoay* — lúc đó $\mathbf{t} = 0$ và $H = R$, đúng cho mọi điểm bất kể cấu trúc.

Sự trùng hợp này không phải ngẫu nhiên mà là dấu hiệu của một điều sâu hơn: *cả hai trường hợp đều là trường hợp mà E mất nghĩa*. Với cảnh phẳng, ràng buộc epipolar vẫn đúng nhưng không còn xác định E duy nhất; với xoay thuần, $\mathbf{t} = 0$ nên $E = 0$. Nói cách khác, homography không phải một mô hình cạnh tranh với E mà là mô hình *thay thế cho đúng những cấu hình mà E chết*. Đó là toàn bộ lý do ORB-SLAM chạy song song hai mô hình rồi chọn theo điểm số [5], chuyện sẽ nói kỹ ở B/10.

Bảng 1.1. Ba ma trận, ba phạm vi áp dụng. Cột cuối là chỗ dễ sai nhất trong thực hành.

Ma trận	Bậc tự do	Giả thiết	Cho ra cái gì	Suy biến khi
E	5	Nội đã biết; cảnh không phẳng; có tịnh tiến	$(R, \hat{\mathbf{t}})$ — hướng tịnh tiến, không có độ lớn	Cảnh phẳng; xoay thuần; mọi điểm ở vô hạn
F	7	Nội chưa biết	Hình học epipolar trong pixel; cần thêm giả định để ra tư thế	Như E , cộng thêm mọi cấu hình mà tự hiệu chuẩn suy biến
H	8	Cảnh phẳng <i>hoặc</i> xoay thuần	Ánh xạ điểm-điểm; phân rã cho $(R, \mathbf{t}/d, \mathbf{n})$	Cảnh <i>không</i> phẳng và có tịnh tiến — tức trường hợp thông thường

Bảng 1.1 nên đọc theo cột cuối. Điều đáng chú ý là E và H suy biến ở *những chỗ bù nhau*: nơi này chết thì nơi kia sống. Hệ quả thiết kế trực tiếp là không nên chọn sẵn một mô hình mà phải chạy cả hai và để dữ liệu quyết định — một nguyên tắc sẽ lặp lại ở nhiều chỗ trong cây, dưới cái tên *model selection*.

E. Phân rã E : bốn nghiệm và ràng buộc vật lý loại ba

Có E rồi, lấy $(R, \mathbf{t})$ ra bằng SVD. Viết $E = U \text{diag}(1, 1, 0) V^T$ và đặt

$$W = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix},$$

thì bốn tổ hợp sau đều thoả $E = [\mathbf{t}]_{\times} R$:

$$(R_a, +\mathbf{u}_3), \quad (R_a, -\mathbf{u}_3), \quad (R_b, +\mathbf{u}_3), \quad (R_b, -\mathbf{u}_3),$$

với $R_a = U W V^T$, $R_b = U W^T V^T$, và $\mathbf{u}_3$ là cột thứ ba của U [2, §9.6.2].

Bốn nghiệm này không phải một khiếm khuyết của thuật toán mà là một tính chất thật của ràng buộc epipolar: phương trình (1.1) không biết phân biệt “điểm ở trước camera” với “điểm ở sau camera”. Hai nghiệm khác nhau ở dấu của $\mathbf{t}$ tương ứng với việc đổi chiều đường cơ sở; hai nghiệm khác nhau ở R_a so với R_b tương ứng với phép xoay 180° quanh đường cơ sở — cấu hình gọi là “twisted pair”.

Cách loại là **cheirality**: triangulate một điểm khớp bằng mỗi nghiệm, rồi giữ nghiệm cho độ sâu dương ở *cả hai* camera. Về mặt hình học thuần tuý cả bốn đều hợp lệ; thứ phân biệt chúng là một sự kiện vật lý — ánh sáng đi vào camera từ phía trước. Đây là một ví dụ đẹp và đáng nhớ của một khuôn mẫu lặp lại khắp cây: *ràng buộc hình học để lại nhiều nghiệm, và ràng buộc vật lý là thứ chọn ra nghiệm đúng*. Ta sẽ gặp lại đúng khuôn mẫu này ở lưỡng nghĩa tư thế của target phẳng trong `../marker/`, và ở chỗ trọng lực ghim roll với pitch trong `../IMU_Fusion/`.

CẦN NHỚ

Bốn nghiệm của phân rã E , và cheirality loại ba. Trong cài đặt thật, đừng chỉ triangulate một điểm để quyết định: với nhiễu và với điểm ở gần vô hạn, dấu của độ sâu không đáng tin. Hãy triangulate *tất cả* inlier cho từng nghiệm rồi đếm số điểm có độ sâu dương ở cả hai khung, và chọn nghiệm thắng áp đảo.

Nếu không nghiệm nào thắng áp đảo — chẳng hạn hai nghiệm cho số lượng xấp xỉ nhau — thì đó **không** phải lúc chọn đại. Đó là dấu hiệu parallax quá nhỏ hoặc cảnh phẳng, tức là dấu hiệu bạn đang ở một cấu hình suy biến của mục 6. Câu trả lời đúng lúc đó là *từ chối khởi tạo* và chờ thêm dữ liệu, không phải trả về nghiệm nhiều phiếu hơn một chút.

1.4 Triangulation: từ hai tia ra một điểm

Có tư thế tương đối rồi, bước tiếp theo là khôi phục cấu trúc. Bài toán phát biểu đơn giản: cho hai tia xuất phát từ hai tâm camera đã biết theo hai hướng đã biết, tìm giao điểm. Cái bẫy nằm ở chỗ với dữ liệu thật, *hai tia không cắt nhau* — nhiễu pixel làm chúng chéo nhau trong không gian. Vậy “giao điểm” nghĩa là gì? Ba câu trả lời khác nhau cho ba phương pháp, và chúng khác nhau nhiều nhất đúng ở chỗ quan trọng nhất.

A. Ba phương pháp, ba định nghĩa về “gần nhất”

DLT biến bài toán thành đại số tuyến tính. Điều kiện $\mathbf{x} \sim P\tilde{\mathbf{P}}$ tương đương với $\mathbf{x} \times P\tilde{\mathbf{P}} = 0$; mỗi khung cho hai phương trình độc lập, hai khung cho bốn, và nghiệm là vectơ kỳ dị phải nhỏ nhất của ma trận 4×4 tương ứng. Ưu điểm: một lần SVD, không lặp, không khởi tạo. Nhược điểm: nó cực tiểu hoá một *sai số đại số* — một đại lượng không có đơn vị vật lý, và tệ hơn, một đại lượng có

trọng số ngầm phụ thuộc vào độ sâu. Điểm càng xa thì trọng số ngầm càng lệch, nên DLT thiên vị một cách khó đoán.

Midpoint lấy trung điểm của đoạn vuông góc chung giữa hai tia. Trực giác hình học rất rõ, và nó là phương pháp hay được viết ra đầu tiên. Nhưng nó cực tiểu hoá khoảng cách trong không gian 3D, trong khi nhiều thật nằm ở *mặt phẳng ảnh*. Với parallax lớn hai tiêu chí gần trùng nhau; với parallax nhỏ, chúng phân kỳ mạnh, và midpoint cho kết quả tệ nhất trong ba phương pháp đúng ở chế độ khó nhất.

Optimal (Hartley–Sturm) đặt đúng câu hỏi thống kê: tìm điểm 3D cực tiểu hoá *tổng bình phương sai số chiếu lại* trên hai ảnh, tức ước lượng hợp lý cực đại dưới giả định nhiễu Gauss đẳng hướng trên pixel. Hartley và Sturm [6] chỉ ra bài toán này rút về việc tìm nghiệm của một đa thức bậc sáu, nên giải được chính xác chứ không cần lặp. Đây là phương pháp đúng về mặt nguyên tắc, và nó cũng đắt nhất.

B. Vì sao chúng khác nhau ở điểm xa

Đây là điểm cần hiểu cơ chế chứ không phải thuộc kết luận, vì cùng cơ chế ấy chi phối lựa chọn tham số hoá landmark ở B/08 và ngân sách sai số ở A/06.

Xét hình dạng của *vùng bất định* quanh điểm triangulate. Với parallax lớn — hai tia gần vuông góc — giao của hai chùm tia là một vùng gần tròn, và mọi tiêu chí “gần nhất” đều trở về gần cùng một chỗ. Với parallax nhỏ — hai tia gần song song — vùng ấy là một *điều xi gà* rất dài dọc theo hướng nhìn: ngắn theo phương ngang, dài theo phương sâu. Bất định theo hai hướng lệch nhau nhiều bậc độ lớn.

Trong vùng dẹt như vậy, ba tiêu chí trở về ba chỗ khác nhau dọc theo trục dài, và khoảng cách giữa chúng có thể lớn hơn nhiều so với sai số pixel đã sinh ra chúng. Tệ hơn, phân bố của sai số độ sâu *không còn đối xứng*: nhiều pixel đối xứng ánh xạ thành sai số độ sâu lệch phải, vì quan hệ độ sâu–disparity là nghịch đảo. Trung bình của một đại lượng lệch không bằng ảnh của trung bình, nên mọi ước lượng điểm đều thiên vị theo một hướng nào đó. Đây chính là lý do sâu xa vì sao **nghịch đảo độ sâu** là tham số hoá tốt hơn: trong biến nghịch đảo, quan hệ trở lại gần tuyến tính và phân bố trở lại gần Gauss [7]. Chi tiết ở B/08.

C. Ví dụ nhỏ tính tay: bất định độ sâu của stereo

Lấy cấu hình stereo chuẩn ở mục 3C để có một quan hệ định lượng đáng thuộc lòng. Với đường cơ sở b , tiêu cự f (đơn vị pixel) và disparity $d = u_1 - u_2$ (pixel), độ sâu là

$$z = \frac{bf}{d}. \quad (1.2)$$

Vi phân theo d :

$$\frac{\partial z}{\partial d} = -\frac{bf}{d^2} = -\frac{z^2}{bf} \implies \boxed{\sigma_z \approx \frac{z^2}{bf} \sigma_d}$$

Đây là một trong vài quan hệ trong cả cây đáng thuộc lòng, vì nó chi phối mọi quyết định về chọn cảm biến ở D/21. Nó nói: *sai số độ sâu tăng theo bình phương độ sâu*. Gấp đôi khoảng cách thì sai số gấp bốn.

Số cụ thể. Giả sử $b = 12$ cm, $f = 400$ px, và tinh chỉnh subpixel cho $\sigma_d = 0,2$ px. Ở $z = 2$ m:

$$\sigma_z = \frac{2^2}{0,12 \times 400} \times 0,2 = \frac{4}{48} \times 0,2 \approx 0,017 \text{ m}.$$

Ở $z = 5$ m:

$$\sigma_z = \frac{25}{48} \times 0,2 \approx 0,104 \text{ m.}$$

Ở $z = 10$ m: $\sigma_z \approx 0,417$ m. Khoảng cách tăng năm lần thì sai số tăng hai mươi lăm lần — từ mức centimet lên mức gần nửa mét. (Ví dụ tự đặt; các con số là kết quả tính tay từ công thức trên, cửa kiểm chứng b.)

Bài học thiết kế rút ra ngay, và nó là bài học thật cho sản phẩm: với stereo đường cơ sở ngắn, dữ liệu độ sâu ở xa gần như vô dụng cho việc đo đạc — nhưng vẫn rất hữu ích cho việc ước lượng *hướng*, vì sai số ngang không tăng theo bình phương. Hệ quả là điểm xa nên được đưa vào bài toán với trọng số đúng theo hướng, chứ không nên bị vứt bỏ. Cách làm đúng là tham số hoá nghịch đảo độ sâu và để hiệp phương sai phản ánh hình dạng điều gì, thay vì gán một σ đẳng hướng rồi hy vọng. Đây là một trong những chỗ mà chương 6 và chương 8 nối trực tiếp vào chương này.

1.5 Từ hai khung sang nhiều khung

Hai khung cho ta khởi tạo. Nhưng SLAM là chuyện hàng nghìn khung, và có ba câu hỏi mà hình học hai khung không trả lời được.

Thứ nhất, *tư thế tuyệt đối*. Khi đã có một bản đồ và một khung mới, không cần quay lại hình học epipolar: ta có các tương ứng 3D–2D và bài toán trở thành **PnP** (*perspective-n-point*). Với ba điểm (P3P) bài toán có tối đa bốn nghiệm, giải được ở dạng đóng [8]; với nhiều điểm, EPnP [9] cho nghiệm tuyến tính thời gian $O(n)$. PnP là xương sống của luồng tracking trong mọi hệ thống có bản đồ — ORB-SLAM gọi nó mỗi khung — và cũng là bước cuối của mọi pipeline relocalization ở B/12.

Thứ hai, *ba khung trở lên ràng buộc nhau thế nào*. Câu trả lời đầy đủ là tensor ba tiêu (trifocal tensor), một đối tượng $3 \times 3 \times 3$ đóng vai trò của F cho bộ ba ảnh, và nó có một khả năng mà hình học hai khung không có: chuyển *đường thẳng* giữa các ảnh. Trong Visual SLAM hiện đại tensor này ít được dùng trực tiếp — lý do thực dụng là khi đã có bản đồ thì PnP rẻ hơn và ổn định hơn — nên cây này xếp nó mức •: biết là có, đọc [2, §15] khi gặp bài toán thuần thị giác không có bản đồ.

Thứ ba, và đây là câu quan trọng nhất: *làm sao ghép mọi thứ lại một cách nhất quán*. Ghép từng cặp rồi nối chuỗi là cách hiển nhiên, và nó hỏng theo một kiểu không thể sửa được: sai số tích lũy đơn điệu, và không có cơ chế nào để một phép đo muộn chỉnh lại một ước lượng sớm. Cách đúng là đặt *mọi* tư thế và *mọi* điểm vào cùng một bài toán tối ưu, tức bundle adjustment — nội dung của A/04. Nói cách khác, chương này cho *khởi tạo*, còn chương 4 cho *nghiệm*. Nhầm hai vai trò ấy là một lỗi phổ biến: dùng nghiệm tuyến tính từ chương này làm kết quả cuối là bỏ lại rất nhiều độ chính xác trên bàn.

1.6 Cấu hình suy biến: mục quan trọng nhất của chương

Nếu chỉ đọc một mục của chương thì đọc mục này. Lý do không phải vì nó khó mà vì nó là mục quyết định hệ thống của bạn có sống sót ngoài phòng thí nghiệm hay không — và vì phần lớn tài liệu nói về nó bằng một dòng ở cuối phần liên quan, nếu có nói.

A. Xoay thuần

Khi $\mathbf{t} = 0$, phương trình (1.1) cho $E = [\mathbf{0}] \times R = 0$. Ràng buộc epipolar trở thành $0 = 0$, đúng với mọi cặp điểm, nên nó không ràng buộc gì cả. Hình học hơn: hai tâm camera trùng nhau nên

không có mặt phẳng epipolar, không có đường epipolar, không có parallax. Mỗi điểm 3D có thể ở bất kỳ độ sâu nào dọc theo tia của nó mà không ảnh hưởng tới ảnh nào.

Đây là chế độ hỏng phổ biến nhất và tốn kém nhất của SLAM đơn mắt trong thực tế, vì nó không hiếm chút nào: robot quay tại chỗ để đổi hướng, người cầm điện thoại xoay để quét phòng, drone yaw để đổi tầm nhìn. Trong mọi tình huống đó, camera thu được ảnh chất lượng cao, khớp điểm hoàn hảo, RANSAC báo nhiều inlier — và không một điểm 3D mới nào được sinh ra một cách đáng tin.

Cách xử lý đúng gồm ba phần, và phần thứ ba hay bị bỏ. Một: *phát hiện* bằng cách đo parallax trung vị của các điểm khớp, không phải bằng số inlier. Hai: khi phát hiện, *không tạo landmark mới* và không tạo keyframe mới, chỉ theo dõi bằng mô hình homography thuần xoay. Ba, phần hay bị bỏ: *ghi lại* rằng đoạn đó là đoạn xoay thuần, vì khi robot dịch chuyển trở lại, các điểm nhìn thấy trong lúc xoay vẫn chưa có độ sâu và phải được triangulate lúc đó chứ không phải bị vứt đi. Chi tiết ở B/10 và C/13.

B. Cảnh phẳng

Khi mọi điểm nằm trên một mặt phẳng, hệ phương trình của thuật toán tám điểm mất hạng: có nhiều hơn một ma trận E khớp với dữ liệu. Nghiệm trả về khi đó là một tổ hợp tùy tiện quyết định bởi nhiễu, và nó có thể sai bất kỳ mức nào.

Điều làm cấu hình này nguy hiểm hơn xoay thuần là nó *im lặng hơn*: parallax vẫn có, số inlier vẫn cao, và không có đại lượng đơn giản nào như “parallax trung vị” báo động. Tình huống này rất thường gặp trong nhà: camera quay mặt vào tường, nhìn xuống sàn, nhìn lên trần, nhìn vào cửa. Với robot lau nhà hay AGV kho hàng, nó là chế độ mặc định chứ không phải ngoại lệ.

Có hai cách xử lý và chúng bổ sung nhau. Cách thứ nhất là dùng thuật toán năm điểm, vốn không suy biến với cảnh phẳng vì nó khai thác ràng buộc phổ giá trị kỳ dị [4]. Cách thứ hai, mạnh hơn, là *model selection*: chạy song song cả E và H , tính một điểm số cho mỗi mô hình dựa trên phần dư của inlier, rồi chọn. ORB-SLAM chọn cách này [5], và lý do không chỉ là độ chính xác mà là *thông tin chẩn đoán*: khi H thắng, hệ thống biết nó đang nhìn một mặt phẳng, và nó có thể dùng thông tin đó để đổi hành vi thay vì chỉ đổi công thức.

C. Chuyển động dọc trục quang, và điểm ở vô hạn

Khi camera tiến thẳng về phía trước, epipole nằm *trong* ảnh — tại điểm mà mọi thứ dường như toả ra. Về nguyên tắc hình học vẫn không suy biến, và E vẫn xác định được. Trên thực tế nó là chế độ khó: các điểm gần epipole có parallax rất nhỏ dù camera đã đi một quãng dài, nên độ sâu của chúng rất kém xác định. Với xe tự lái đi thẳng trên đường cao tốc, đây là chế độ hoạt động chính, và nó là một phần lý do vì sao KITTI khó hơn EuRoC về mặt hình học dù cảnh sạch hơn.

Trường hợp cuối: *mọi điểm ở rất xa*. Lúc đó tất cả hành xử như điểm ở vô hạn, ma trận E chỉ ràng buộc được thành phần xoay, còn hướng tịnh tiến thì không xác định. Đây là chuyện của SLAM ngoài trời nhìn ra xa, và của drone bay cao. Một dấu hiệu nhận biết rõ: nếu ước lượng hướng ổn định nhưng hướng tịnh tiến nhảy lung tung giữa các khung, nhiều khả năng bạn đang ở đây, không phải đang có lỗi cài đặt.

Bảng 1.2. Bốn cấu hình suy biến, dấu hiệu nhận biết, và cách xử lý. Cột “dấu hiệu” là cột đáng nhớ nhất, vì nó là thứ bạn cần cài vào hệ thống để nó tự chẩn đoán.

Cấu hình	Cái gì mất	Dấu hiệu đo được	Xử lý
Xoay thuần	Toàn bộ cấu trúc; $E = 0$	Parallax trung vị nhỏ dù số inlier cao; H khớp tốt hơn hẳn E	Không tạo landmark mới; theo dõi bằng H ; ghi lại để triangulate sau
Cảnh phẳng	Nghiệm E duy nhất	Điểm số H cao hơn E rõ rệt; pháp tuyến ổn định qua nhiều khung	Thuật toán 5 điểm; hoặc model selection rồi phân rã H
Đi dọc trục quang	Độ sâu của điểm gần epipole	Epipole nằm trong ảnh; parallax phân bố rất lệch theo bán kính	Ưu tiên điểm ở rìa ảnh; dùng cửa sổ thời gian dài hơn
Mọi điểm ở vô hạn	Hướng tịnh tiến	Hướng $\hat{t}$ nhảy giữa các khung trong khi R ổn định	Tham số hoá nghịch đảo độ sâu; thêm nguồn tỉ lệ (IMU, stereo)

CẠM BẦY

Đừng dùng **số inlier** của RANSAC làm thước đo chất lượng khởi tạo. Đây là lỗi phổ biến nhất liên quan tới chương này, và nó phổ biến vì số inlier là con số dễ lấy nhất.

Lý do nó sai: trong cả bốn cấu hình ở Bảng 1.2, số inlier vẫn *cao*. Với xoay thuần, mọi điểm khớp hoàn hảo với một homography, nên nếu bạn đang chạy mô hình E thì nhiều điểm vẫn nằm trong ngưỡng phần dư; với cảnh phẳng, mọi điểm khớp hoàn hảo với một E nào đó trong họ nghiệm. Số inlier đo *mức độ nhất quán của dữ liệu với mô hình*, không đo *mức độ xác định của mô hình*. Hai thứ đó khác nhau, và chính sự nhầm lẫn giữa chúng là nguồn của kiểu hỏng im lặng mà A/05 mô tả tổng quát.

Thước đo đúng gồm ba con số và nên tính cả ba: **parallax trung vị** của các điểm inlier (không phải trung bình — trung vị bền hơn với vài điểm ở rất gần); **tỉ số điểm số** giữa hai mô hình H và E ; và **số điều kiện** của ma trận thông tin nếu bạn đã dựng nó. Hệ thống nên *từ chối khởi tạo* khi ba con số này không đạt, chứ không nên khởi tạo rồi hy vọng bundle adjustment sẽ sửa. BA không sửa được thứ mà dữ liệu không chứa.

1.7 Giới hạn của chương và chỗ nó nối sang

Toàn bộ chương này đứng trên bốn giả thiết, và hai trong số đó là giả thiết ngầm mà ít tài liệu nói ra. Nêu rõ chúng ở đây vì mỗi giả thiết bị vi phạm là một chương ở phần sau.

Giả thiết một: cảnh tĩnh. Mọi phương trình giả định điểm 3D không di chuyển giữa hai lần chụp. Khi có vật động, ràng buộc epipolar đơn giản là sai với các điểm trên vật đó. RANSAC cứu được khi vật động chiếm thiểu số; khi nó chiếm đa số khung hình — một chiếc xe tải đi ngang trước camera — thì RANSAC sẽ trung thành chọn *vật động* làm inlier và cảnh tĩnh làm outlier, rồi trả về tư thế tương đối so với chiếc xe tải. Kết quả sai nhưng nhất quán, tức im lặng. Xử lý ở D/19.

Giả thiết hai: camera đã hiệu chuẩn nội đúng. Chương làm việc trên mặt phẳng chuẩn hoá, tức giả định đã gỡ được K và distortion. Nếu hiệu chuẩn sai, sai số ấy không biến mất mà *chảy sang* tư thế và cấu trúc, thường theo một hoa văn có hệ thống. Chương 2 dạy cách nhận ra hoa văn ấy; chương 14 dạy cách ước lượng lại tham số ngay trong bài toán.

Giả thiết ba (ngâm): mọi pixel được chụp cùng một lúc. Với cảm biến global shutter thì đúng; với rolling shutter thì sai, và một ảnh chụp lúc đang xoay nhanh không tương ứng với một tư thế nào cả. Chương 2 mô tả mô hình đúng.

Giả thiết bốn (ngâm): điểm khớp là khớp đúng. Cả chương giả định bài toán data association đã giải xong. Trong thực tế đó là phần khó nhất và là nguồn của phần lớn lỗi thảm hoạ — một cặp khớp sai có phần dư khổng lồ sẽ kéo nghiệm bình phương tối thiểu đi rất xa. Chương 7 nói về việc tìm khớp, chương 4 nói về việc chịu đựng khớp sai bằng hàm mất mát bền.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [10]	Chạy song song mô hình H và E , tính điểm số, chọn mô hình; từ chối khởi tạo khi parallax không đủ	Chỗ luận điểm mục 6 xuất hiện nguyên vẹn trong mã nguồn thật
COLMAP [11]	Kiểm định hình học hai khung với nhiều mô hình; phân loại cặp ảnh theo mô hình thẳng	Bước <i>geometric verification</i> chính là Bảng 1.1 được cài đặt
OpenCV findEssentialMat	Thuật toán 5 điểm [4] trong vòng lặp RANSAC; recoverPose làm cheirality	Mục 3E là đúng cái hàm recoverPose làm bên trong
HLLoc [12]	PnP + RANSAC ở bước cuối sau khi retrieval và matching	Mục 5 — khi đã có bản đồ thì PnP thay thế hình học epipolar
VINS-Mono [13]	Khởi tạo bằng SfM hai khung rồi căn với IMU để lấy tỉ lệ	Minh hoạ trực tiếp mục 3B: tỉ lệ phải đến từ ngoài thị giác

MINI PROJECT SLAM · Dựng lại pipeline hai khung từ đầu, rồi phá nó bằng ba cấu hình suy biến · 3 ngày

Mục tiêu	tự viết toàn chuỗi từ hai tập điểm khớp tới đám mây điểm 3D, rồi <i>cố tình</i> đưa nó vào ba cấu hình suy biến để thấy tận mắt kiểu hổng im lặng.
Dữ liệu	một cặp ảnh bất kỳ từ EuRoC hoặc TUM-RGBD kèm hiệu chuẩn đã biết; thêm dữ liệu tổng hợp tự sinh, vì với dữ liệu tổng hợp bạn kiểm soát được chính xác chuyển động.
Các bước	cài thuật toán tám điểm có chuẩn hoá Hartley; chiếu E về tập hợp lệ bằng SVD và kiểm hai giá trị kỳ dị bằng nhau; phân rã ra bốn nghiệm và lọc bằng cheirality trên toàn bộ inlier; cài cả ba phương pháp triangulation; đo sai số chiếu lại của từng phương pháp theo parallax, vẽ đồ thị sai số theo góc parallax.
Xong khi	bạn có một đồ thị cho thấy ba phương pháp trùng nhau ở parallax lớn và phân kỳ ở parallax nhỏ — xác nhận mục 4B bằng số của chính bạn. Và bạn có ba lần chạy trên ba cấu hình suy biến (xoay thuần, cảnh phẳng, đi dọc trục quang), mỗi lần kèm một bảng ba con số chẩn đoán ở khối <i>Cạm bẫy</i> mục 6, cho thấy số inlier không phát hiện được suy biến còn ba con số kia thì có.
Nối với	B/10-khoi-tao dùng trực tiếp kết quả này làm tiêu chí chấp nhận khởi tạo; A/06-bat-dinh-va-lan-truyen-sai-so lấy đồ thị parallax làm đầu vào cho ngân sách sai số.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Vì xoay tại chỗ không sinh parallax: hai tâm camera trùng nhau nên không có mặt phẳng epipolar, $\mathbf{t} = 0$ nên $E = [\mathbf{0}]_{\times} R = 0$, và ràng buộc epipolar trở thành $0 = 0$ — đúng với mọi cặp điểm, nên không ràng buộc gì. Mỗi điểm có thể ở bất kỳ độ sâu nào dọc tia của nó mà không đổi ảnh nào. Chụp thêm ảnh không cứu được vì *mọi* ảnh mới cũng chụp từ đúng tâm đó; thông tin không tồn tại trong dữ liệu, và số lượng không tạo ra thông tin từ chỗ không có. Đây là cùng một cơ chế mà [A/05](#) phát biểu tổng quát.
- Một bậc mất vì tính thuần nhất: E và λE cho cùng ràng buộc, nên độ lớn của $\mathbf{t}$ không quan sát được — chỉ hướng là. Bậc thứ hai mất vì ràng buộc cấu trúc $E = [\mathbf{t}]_{\times} R$, thể hiện thành việc hai giá trị kỳ dị khác không phải bằng nhau. Thuật toán tám điểm vẫn chạy vì nó *cố tình* bỏ qua

ràng buộc thứ hai để giữ bài toán tuyến tính, rồi chữa sau bằng phép chiếu SVD về diag(1, 1, 0). Cái giá là độ nhạy nhiều cao hơn và — quan trọng hơn — suy biến với cảnh phẳng, chỗ mà thuật toán năm điểm không bị.

- A3. Bốn nghiệm chia thành hai cặp: dấu của $\mathbf{t}$ (đổi chiều đường cơ sở) và R_a so với R_b (xoay 180° quanh đường cơ sở, gọi là *twisted pair*). Phép thử loại chúng là *cheirality* — yêu cầu độ sâu dương ở cả hai khung — và nó là ràng buộc *vật lý* chứ không phải hình học vì phương trình epipolar hoàn toàn không biết phân biệt trước với sau camera: một điểm sau lưng vẫn chiếu ra một pixel hợp lệ về mặt đại số. Thứ loại nó là sự kiện ánh sáng đi vào camera từ phía trước. Cùng khuôn mẫu “hình học để lại nhiều nghiệm, vật lý chọn một” xuất hiện ở lưỡng nghĩa target phẳng trong `../marker/` và ở chỗ trọng lực ghim roll-pitch trong `../IMU_Fusion/`.
- A4. Khác nhau lớn nhất ở *parallax nhỏ*, tức điểm xa. Cơ chế: vùng bất định quanh điểm triangulate là một điều gì gần rất dài dọc hướng nhìn khi hai tia gần song song; ba tiêu chí “gần nhất” khác nhau (đại số, khoảng cách 3D, sai số chiếu lại) trở về ba chỗ khác nhau dọc trục dài đó. Với parallax lớn vùng bất định gần tròn nên ba tiêu chí hội tụ. Thêm một lớp nữa: quan hệ độ sâu-disparity là nghịch đảo nên nhiều pixel đối xứng thành sai số độ sâu *lệch*, làm mọi ước lượng điểm thiên vị — đây chính là lý do nghịch đảo độ sâu là tham số hoá tốt hơn [7].
- A5. Tường là cảnh *phẳng*, nên hệ phương trình của thuật toán tám điểm mất hạng và E không còn duy nhất; nghiệm trả về là một tổ hợp tùy tiện do nhiễu quyết định. Khớp điểm tốt và nhiều inlier không mâu thuẫn gì với điều này, vì số inlier đo mức nhất quán của dữ liệu với mô hình chứ không đo mức xác định của mô hình. ORB-SLAM chạy hai mô hình song song vì E và H suy biến ở *những chỗ bù nhau*: chọn sẵn một mô hình là tự nguyện chết ở một nửa số cấu hình. Lợi ích thứ hai, ít được nhắc: khi H thắng, hệ thống *biết* nó đang nhìn mặt phẳng và có thể đổi hành vi, chứ không chỉ đổi công thức.

NÓI LẠI THẬT ĐƠN GIẢN

Nhắm một mắt lại rồi thử với tay lấy một cái cốc. Bạn vẫn làm được, nhưng chậm hơn và hay hụt. Bây giờ vẫn nhắm một mắt nhưng lắc nhẹ đầu sang hai bên — đột nhiên bạn lại biết cái cốc ở đâu. Đó là toàn bộ chương này. Một con mắt đứng yên chỉ biết *hướng* của vật, không biết vật xa hay gần. Muốn biết xa gần thì phải nhìn từ hai chỗ khác nhau và so sánh: vật ở gần thì dịch nhiều trong tầm nhìn khi ta dịch đầu, vật ở xa thì gần như đứng im. Chỉ vậy thôi.

Từ đó suy ra mọi thứ còn lại. Nếu bạn chỉ *quay* đầu chứ không dịch đầu, thì không có hai chỗ khác nhau nào cả — mắt vẫn ở nguyên một chỗ, chỉ nhìn hướng khác. Nên quay đầu không bao giờ cho biết vật xa hay gần, dù quay cả ngày. Máy cũng vậy, và đó là kiểu hỏng hay gặp nhất.

Nếu bạn nhìn vào một bức tường phẳng, mọi thứ trên tường ở cùng một khoảng cách, nên chúng dịch chuyển giống hệt nhau khi bạn lắc đầu. Bạn không đọc được gì từ sự khác biệt, vì không có khác biệt. Máy cũng vậy.

Và nếu vật ở rất xa — một ngọn núi — thì lắc đầu bao nhiêu nó cũng không nhúc nhích. Bạn biết nó ở *hướng* nào rất chính xác, nhưng nó cách bao xa thì hoàn toàn không biết.

Điều nguy hiểm là cái máy trong cả ba trường hợp trên vẫn đọc ra một con số. Nó được lập trình để đọc ra một con số, nên nó đọc. Việc của chương này là dạy bạn nhận ra khi nào con số đó là kết quả của một phép đo, và khi nào nó chỉ là chỗ mà thuật toán tình cờ dừng lại.

Chỗ không rút gọn được: chỗ thật sự không đơn giản hoá được là *ngưỡng*: parallax bao nhiêu thì đủ? Không có câu trả lời tuyệt đối, vì nó phụ thuộc vào mức nhiễu pixel, vào độ sâu cảnh, và vào việc bạn định dùng kết quả để làm gì. Đây là một trong những chỗ mà kinh nghiệm thật sự có giá trị, và cũng là chỗ mà A/06 cố gắng thay kinh nghiệm bằng một phép tính.

Một câu: Muốn biết vật xa hay gần thì phải nhìn nó từ hai chỗ khác nhau — và “khác nhau” nghĩa là dịch đi, không phải quay đi.

1.8 Điều gì chứng minh cách hiểu này sai

Theo quy tắc của kho, một chương phải nói rõ điều gì sẽ bác bỏ nó.

Mệnh đề chính — *độ sâu chỉ khôi phục được từ parallax, và parallax chỉ sinh ra từ tịnh tiến* — bị bác bỏ nếu ai đó khôi phục được cấu trúc metric từ một chuỗi xoay thuần tịnh khiết mà không dùng tiên nghiệm nào khác. Điều kiện cuối là chỗ phải kiểm nghiệm khác. Rất nhiều hệ trông như đang làm điều đó thực ra đang dùng thông tin thêm: một mô hình học được mang theo tiên nghiệm về kích thước vật thể trong cảnh ([C/17](#)), một giả định rằng mặt đất phẳng và cách camera một khoảng cố định, một cảm biến độ sâu. Mỗi thứ đó là một nguồn thông tin hợp lệ và làm độ sâu quan sát được — nhưng chúng không bác bỏ mệnh đề, chúng xác nhận nó bằng cách chỉ ra rằng *phải thêm thứ gì đó*.

Mệnh đề thứ hai — *E có đúng năm bậc tự do và phân rã cho đúng bốn nghiệm* — là một mệnh đề đại số thuần túy, bị bác bỏ nếu ai đó chỉ ra một nghiệm thứ năm độc lập, hoặc chỉ ra rằng một trong bốn nghiệm trùng với nghiệm khác trong trường hợp tổng quát. Tôi đã kiểm bằng suy dẫn trực tiếp (cửa a) và đối chiếu [[2](#), §9.6.2] (cửa c).

Mệnh đề thứ ba — *số inlier không phải thước đo chất lượng khởi tạo* — là mệnh đề thực dụng và dễ kiểm nhất. Nó bị bác bỏ nếu một thí nghiệm có kiểm soát cho thấy, trên một tập đủ lớn các cặp khung gồm cả cấu hình suy biến lẫn cấu hình tốt, số inlier tương quan chặt với sai số tư thế thật — chặt ngang hoặc chặt hơn parallax trung vị. Mini project ở trên chính là phép kiểm đó, và tôi khuyến chạy nó thật chứ không tin lời tôi.

Một mệnh đề mềm hơn nhưng đáng nêu: chương này khẳng định rằng *cấu hình suy biến quan trọng hơn độ chính xác của bộ ước lượng trong thực hành*. Điều đó bị bác bỏ nếu trong triển khai thật, phần lớn ca hỏng hoá ra bắt nguồn từ chất lượng khớp điểm hoặc lỗi cài đặt chứ không từ suy biến hình học. Tôi không có số liệu để khẳng định tỉ lệ, và đây là chỗ tôi phát biểu dựa trên cấu trúc của vấn đề chứ không dựa trên thống kê (*tôi suy ra, chưa đối chiếu nguồn*). Chương 21 đề nghị cách thu số liệu đó từ fleet analytics.

1.9 Kết nối

Chương này là gốc của Phần A và gần như mọi chương sau đều gọi ngược về đây.

Nối xuống dưới trong Phần A trước. [A/02-mô-hình-camera](#) lắp vào chỗ chương này bỏ trống: làm sao từ pixel về tới mặt phẳng chuẩn hoá, và chuyện gì xảy ra khi phép biến đổi ấy sai. [A/03-lie-group-va-toi-uu-tren-da-tap](#) cung cấp ngôn ngữ để (R, t) trở thành một biến tối ưu được chứ không chỉ một kết quả đại số. [A/04-uoc-luong-map-va-bundle-adjustment](#) thay nghiệm tuyến tính của chương này bằng nghiệm hợp lý cực đại — nhớ rằng chương này cho *khởi tạo*, chương 4 cho *nghiệm*. [A/05-observability-gauge-va-suy-bien](#) tổng quát hoá mục 6: bốn cấu hình suy biến ở đây là bốn ví dụ của một hiện tượng duy nhất, và chương 5 cho công cụ để phát hiện chúng một cách hệ thống thay vì theo danh sách. [A/06-bat-dinh-va-lan-truyen-sai-so](#) biến công thức $\sigma_z \approx z^2 \sigma_d / (bf)$ ở mục 4C thành một ngân sách sai số đầy đủ.

Nối sang Phần B. [B/07-dac-trung-va-data-association](#) cung cấp đầu vào cho cả chương — các cặp điểm khớp — và giả thiết bốn ở mục 7 là chỗ nối. [B/10-khoi-tao](#) là ứng dụng trực tiếp nhất: toàn bộ mục 3 và mục 6 hợp lại thành thuật toán khởi tạo hai khung, và tiêu chí từ chối ở khối *Cạm bẫy* mục 6 là phần quan trọng nhất của thuật toán đó. [B/08-mô-hình-phan-du](#) nhận mục 4B: lập luận về điều gì gà bất định là lý do tồn tại của tham số hoá nghịch đảo độ sâu. [B/12-relocalization](#) dùng PnP ở mục 5 làm bước cuối.

Nối ngang sang hai cây khác. `.../marker/` là trường hợp đặc biệt sạch sẽ của chương này: marker phẳng nên homography là mô hình *đúng* chứ không phải mô hình dự phòng, và kích thước marker đã biết nên tỉ lệ quan sát được — hai vấn đề lớn nhất của chương này đều biến mất, đối lại phải sống với lưỡng nghĩa tư thế phẳng. `.../IMU_Fusion/` giải bài toán tỉ lệ theo hướng khác,

bằng cách thêm một cảm biến có đơn vị mét; đọc chương 10 ở đó để thấy vì sao việc thêm ấy không miễn phí mà đòi hỏi chuyển động phải có gia tốc thay đổi.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ràng buộc epipolar không chứa tọa độ 3D, và vì sao chính điều đó làm nó dùng được để khởi tạo?
2. E có chín phần tử và năm bậc tự do. Hai bậc mất đi đâu, và ràng buộc đại số nào thể hiện mỗi bậc bị mất?
3. Hai kỹ sư triangulate cùng một cặp điểm, một người dùng midpoint và một người dùng optimal. Khi nào hai kết quả gần như trùng nhau, khi nào chúng phân kỳ, và cơ chế hình học nào giải thích điều đó?
4. Một cặp khung cho 500 inlier với ngưỡng chặt. Nếu ba tình huống hoàn toàn khác nhau mà con số đó vẫn xảy ra nhưng khởi tạo sẽ sai, và nêu phép đo phân biệt chúng.
5. Vì sao homography là mô hình đúng cho *cả* cảnh phẳng *lẫn* xoay thuần — hai tình huống nghe không liên quan gì tới nhau?
6. Sai số độ sâu của stereo tăng theo bình phương độ sâu. Tự dẫn lại quan hệ đó từ $z = bf/d$, rồi nói nó ảnh hưởng thế nào tới việc chọn đường cơ sở khi thiết kế sản phẩm.
7. Bốn giả thiết của chương là cảnh tĩnh, nội đã biết, global shutter, và khớp điểm đúng. Với mỗi giả thiết, bỏ nó đi thì kết luận nào của chương thay đổi, và bạn có nhận ra được lúc nó bị vi phạm không?

Mô hình camera (Camera Models)

PHỤ THUỘC A/01-hình-hoc-da-thi (mặt phẳng chuẩn hoá, phép chiếu, tọa độ thuần nhất — chương này lấp vào chỗ chương 1 bỏ trống là làm sao từ pixel về tới mặt phẳng chuẩn hoá).

TÓM TẮT

Chương 1 làm việc trên mặt phẳng chuẩn hoá và giả định phép biến đổi từ pixel về đó đã có sẵn. Chương này xây dựng chính phép biến đổi ấy, và quan trọng hơn, nó dạy cách nhận ra khi phép biến đổi ấy *sai*. Nội dung có bốn lớp. Lớp một là mô hình pinhole với ma trận nội K — đơn giản, tuyến tính trong tọa độ thuần nhất, và không đủ cho ống kính thật. Lớp hai là các họ mô hình méo: radial-tangential cho ống kính hẹp, Kannala–Brandt cho fisheye, UCM/EUCM/double-sphere cho ống kính cực rộng — và tiêu chí chọn giữa chúng, vốn là câu hỏi thiết kế chứ không phải câu hỏi tinh chỉnh. Lớp ba, quan trọng nhất, là kỹ năng phân biệt *mô hình sai* với *tham số sai*: hai bệnh có cùng triệu chứng bề mặt nhưng cần hai cách chữa ngược nhau, và nhằm chúng là nguồn của nhiều thág công vô ích. Lớp bốn là những mô hình mà SLAM thường quên: rolling shutter, mô hình quang trắc, và mô hình của stereo. Nếu chỉ nhớ một điều: *phần dư có cấu trúc là dấu hiệu mô hình sai, phần dư ngẫu nhiên là dấu hiệu nhiễu* — và không lượng dữ liệu nào chữa được bệnh thứ nhất.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tham số nội (intrinsic parameters)** — tập tham số mô tả cách camera biến một tia trong hệ tọa độ camera thành một pixel: tiêu cự, điểm chính, và các hệ số méo. Chúng gắn với thiết bị, không gắn với cảnh hay chuyển động.
- **tham số ngoại (extrinsic parameters)** — tư thế (R, t) của camera so với một hệ quy chiếu khác — thế giới, một camera khác, hay IMU. Chúng gắn với việc lắp đặt.
- **điểm chính (principal point)** — giao của trục quang với mặt phẳng ảnh, ký hiệu (c_x, c_y) . Về lý thuyết nằm gần tâm ảnh; trên thực tế lệch vài pixel do dung sai lắp ráp, và độ lệch đó là một trong những thứ hiệu chuẩn phải tìm ra.
- **méo xuyên tâm (radial distortion)** — hiện tượng điểm ảnh bị dịch vào trong (barrel) hay ra ngoài (pincushion) theo khoảng cách tới điểm chính. Nguyên nhân là hình dạng thấu kính; nó là thành phần méo lớn nhất ở hầu hết ống kính.
- **méo tiếp tuyến (tangential distortion)** — thành phần méo sinh ra khi mặt thấu kính không song song hoàn hảo với mặt cảm biến. Thường nhỏ hơn méo xuyên tâm một bậc độ lớn, và với ống kính hiện đại lắp máy thì nhiều khi bỏ được.
- **rolling shutter** — cách đọc cảm biến theo từng hàng chứ không đồng thời. Hàng cuối được phơi sáng muộn hơn hàng đầu một khoảng $H \cdot t_{\text{line}}$, nên một ảnh chụp khi đang chuyển động không tương ứng với *một* tư thế nào cả.
- **hàm đáp ứng camera (camera response function, CRF)** — quan hệ phi tuyến giữa lượng ánh sáng tới cảm biến và giá trị pixel ghi ra. Với phương pháp indirect nó không quan trọng; với phương pháp direct nó là một phần bắt buộc của mô hình.
- **vignetting** — hiện tượng ánh sáng tối dần về phía rìa do ống kính truyền ít ánh sáng hơn ở góc lớn. Cũng là một hiệu ứng mà indirect bỏ qua được còn direct thì không.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn hiệu chuẩn camera, sai số chiếu lại trung bình là 0,3 px — một con số trông rất tốt. Vì sao con số đó gần như không nói gì về việc hiệu chuẩn có đúng không, và bạn phải nhìn vào cái gì thay cho nó?
- Q2. Thêm hệ số méo bậc cao luôn làm sai số chiếu lại giảm. Vì sao điều đó *không* chứng minh mô hình tốt hơn, và chuyện gì xảy ra khi bạn dùng bộ tham số ấy cho một vùng ảnh mà bảng hiệu chuẩn chưa từng đi qua?
- Q3. Hai camera cùng loại, cùng lô sản xuất, cùng nhà cung cấp. Hiệu chuẩn cái thứ nhất rồi dùng chung tham số cho cả lô — một quyết định tiết kiệm rất nhiều tiền. Điều gì quyết định quyết định đó đúng hay sai, và bạn đo nó bằng cách nào *trước khi* chốt thiết kế cơ khí?
- Q4. Hệ thống của bạn chính xác vào buổi sáng và lệch dần tới chiều, rồi hôm sau lại đúng. Không có gì trong phần mềm thay đổi. Nêu cơ chế vật lý nào có thể gây ra chuyện này, và nó thuộc về chương nào trong cây.
- Q5. Rolling shutter làm hỏng giả thiết “một ảnh tương ứng một tư thế”. Vì sao việc chỉ ước lượng thêm một tham số t_{line} lại không đủ để chữa, và phải thêm cái gì nữa vào mô hình?

2.1 Đặt vấn đề

Chương 1 kết thúc bằng một giả thiết ngầm mà nó không tự chữa được: mọi công thức ở đó viết trên mặt phẳng chuẩn hoá, tức giả định ta đã biết cách gỡ bỏ hoàn toàn ảnh hưởng của ống kính và cảm biến. Giả thiết ấy nghe vô hại, nhưng nó chứa gần như toàn bộ phần vật lý của bài toán, và nó là chỗ mà một hệ thống đúng về lý thuyết gặp thế giới thật.

Bài toán gốc là như sau. Một ống kính thật không phải một lỗ kim. Nó là một chồng thấu kính thuỷ tinh hoặc nhựa, mỗi mặt có một sai số gia công, lắp trong một ống có dung sai cơ khí, gắn trước một cảm biến có thể không hoàn toàn vuông góc với trục quang, và toàn bộ cụm ấy giãn nở khi nóng lên. Ánh sáng đi qua chồng thấu kính đó không đi theo đường mà mô hình lỗ kim nói. Câu hỏi của chương là: *mô tả sai lệch ấy bằng bao nhiêu con số, theo dạng hàm nào, và làm sao biết mình đã chọn đúng?*

Ai gặp bài toán này? Tất cả mọi người làm thị giác đo đạc, nhưng mức độ đau thì rất khác nhau. Với phân loại ảnh, méo ống kính gần như không ảnh hưởng. Với SLAM, nó ảnh hưởng theo một kiểu đặc biệt tệ: sai số hiệu chuẩn không sinh ra nhiễu ngẫu nhiên mà sinh ra *sai lệch có hệ thống*, và sai lệch có hệ thống thì bình phương tối thiểu không khử được — nó chảy thẳng vào ước lượng tư thế và tích lũy theo quỹ đạo. Một sai số hiệu chuẩn nhỏ ổn định gây hại nhiều hơn một nhiễu lớn ngẫu nhiên, và đây là điều phản trực giác đầu tiên của chương.

Trước khi có mô hình tham số, cách làm cũ là bảng tra: đo độ lệch tại một lưới điểm rồi nội suy. Cách đó *đúng* và vẫn được dùng trong trắc địa ảnh độ chính xác cao, nhưng nó hỏng ở ba chỗ khi đưa vào SLAM. Nó cần rất nhiều điểm đo, nên quy trình hiệu chuẩn dài — không chấp nhận được trên dây chuyền sản xuất (D/21). Nó không nội suy được ra ngoài vùng đã đo. Và quan trọng nhất, nó không cho ta một *dạng hàm* giải tích để đưa tham số vào bài toán tối ưu, nên không làm hiệu chuẩn trực tuyến được (C/14). Mô hình tham số giải quyết cả ba, với cái giá là ta phải chọn đúng dạng hàm — và chọn sai dạng hàm là bệnh mà mục 4 dành riêng để nói.

Có một lý do thứ hai, sâu hơn, để chương này quan trọng hơn vẻ ngoài của nó. Theo luận đề trung tâm của cây, tham số nội chỉ là một nhóm biến trong cùng bài toán MAP với tư thế và cấu trúc. Fix chúng thì ta có SLAM; thả chúng ra thì ta có tự hiệu chuẩn. Nhưng để thả được, ta phải biết *dạng hàm* — và dạng hàm là thứ không tối ưu ra được, nó phải được chọn. Đây là ranh giới giữa thứ thuật toán làm được và thứ kỹ sư phải quyết định, và chương này nằm đúng trên ranh giới đó.

TRỰC GIÁC

Hình dung mô hình camera là một cái ống dẫn biến tia sáng thành pixel, và hiệu chuẩn là việc đo hình dạng ống dẫn ấy.

Nếu ống dẫn thật có hình dạng nằm trong họ hình dạng mà mô hình của bạn mô tả được, thì hiệu chuẩn chỉ là tìm đúng con số — và với đủ dữ liệu bạn sẽ tìm được, sai số còn lại là nhiễu thuần túy, phân bố ngẫu nhiên.

Nếu ống dẫn thật có hình dạng *không* nằm trong họ ấy — chẳng hạn bạn dùng mô hình radial cho một ống kính có méo bất đối xứng — thì không con số nào đúng cả. Bộ tối ưu sẽ tìm ra bộ tham số làm sai số nhỏ nhất có thể, và sai số nhỏ nhất đó *không phải nhiễu*: nó là hình dạng của phần mô hình không mô tả được, in dấu lên phần dư thành một hoa văn. Đó là toàn bộ nội dung của mục 4, và nó là lý do vì sao phải nhìn *bản đồ* phần dư chứ không nhìn *trung bình* phần dư. Trung bình xoá mất hoa văn; bản đồ giữ nó lại. Một con số trung bình đẹp hoàn toàn tương thích với một mô hình sai hoàn toàn.

2.2 Pinhole và ma trận nội

A. Chuỗi biến đổi

Mô hình pinhole là mô hình nền mà mọi thứ khác là hiệu chỉnh của nó. Chuỗi đầy đủ từ điểm thế giới tới pixel gồm bốn bước, và tách rạch rời bốn bước ấy là điều đáng làm vì mỗi bước hỏng theo một kiểu khác nhau:

$$\mathbf{P}_w \xrightarrow{(R, \mathbf{t})} \mathbf{P}_c \xrightarrow{\pi} \mathbf{x} \xrightarrow{d(\cdot)} \mathbf{x}_d \xrightarrow{K} \mathbf{p}$$

Bước một là ngoại: $\mathbf{P}_c = R\mathbf{P}_w + \mathbf{t}$. Bước hai là phép chiếu phối cảnh về mặt phẳng chuẩn hoá: $\mathbf{x} = (X_c/Z_c, Y_c/Z_c)$. Bước ba là méo: $\mathbf{x}_d = d(\mathbf{x}; \boldsymbol{\theta}_d)$. Bước bốn là affine sang pixel:

$$\mathbf{p} = K \tilde{\mathbf{x}}_d, \quad K = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}.$$

Bốn tham số của K có ý nghĩa vật lý cụ thể. f_x và f_y là tiêu cự tính bằng pixel — chúng khác nhau khi pixel không vuông, điều hiếm ở cảm biến hiện đại nhưng vẫn nên để tự do vì việc ép $f_x = f_y$ khi chúng thật sự khác nhau chính là một dạng *mô hình sai*. (c_x, c_y) là điểm chính. Hệ số trượt s mô tả trục pixel không vuông góc; với mọi cảm biến CMOS hiện đại nó bằng không, và để nó tự do chỉ thêm một hướng gần suy biến vào bài toán — một ví dụ nhỏ của bài học ở [A/05](#) rằng thêm biến không miễn phí.

B. Ví dụ nhỏ tính tay

Giả sử $f_x = f_y = 400$ px, $(c_x, c_y) = (320, 240)$, không méo. Một điểm ở toạ độ camera $\mathbf{P}_c = (1, 0, 4)$ mét.

Chiếu về mặt phẳng chuẩn hoá: $\mathbf{x} = (1/4, 0/4) = (0,25, 0)$. Sang pixel: $u = 400 \times 0,25 + 320 = 420$, $v = 400 \times 0 + 240 = 240$. Vậy $\mathbf{p} = (420, 240)$. (Cửa kiểm chứng b.)

Kiểm ngược lại để thấy điều quan trọng: nếu điểm ở $(2, 0, 8)$ — cùng hướng, gấp đôi khoảng cách — thì $\mathbf{x} = (2/8, 0) = (0,25, 0)$, ra *đúng cùng một pixel*. Đây là sự mất mát thông tin ở [A/01](#) được xác nhận bằng số: phép chiếu là ánh xạ từ một tia vào một điểm, và mọi điểm trên tia đều

cho cùng kết quả. Không có phép hiệu chuẩn nào lấy lại được chiều đã mất; chỉ có parallax làm được.

Một con số đáng nhớ rút ra từ đây: với $f = 400$ px, một pixel tương ứng khoảng $1/400 = 2,5$ mrad, tức khoảng $0,14^\circ$. Ở khoảng cách 4 m, một pixel là $4 \times 2,5$ mm = 1 cm theo phương ngang. Quan hệ “một pixel bằng bao nhiêu mm ở khoảng cách z ” là z/f , và nó là phép tính đầu tiên nên làm khi ai đó hỏi hệ thống chính xác tới đâu.

2.3 Các họ mô hình méo

Chọn mô hình méo là một quyết định thiết kế, và nó nên được quyết định bởi *trường nhìn* của ống kính chứ không bởi thói quen hay bởi mô hình mà thư viện mặc định.

A. Radial-tangential (Brown–Conrady)

Đây là mô hình quen thuộc nhất, và là mặc định của OpenCV và của Zhang [14]. Với $r^2 = x^2 + y^2$ trên mặt phẳng chuẩn hoá:

$$\mathbf{x}_d = (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \mathbf{x} + \mathbf{x}_{\text{tan}},$$

với $\mathbf{x}_{\text{tan}}$ là hai số hạng tiếp tuyến theo p_1, p_2 . Đây là một *mô hình thực nghiệm*, không phải một định lý — nó là khai triển đa thức chẵn của một hàm méo được giả định trơn và đối xứng xuyên tâm. Tôi không tự dẫn lại được nó và cũng không nên cố; nó qua cửa kiểm chứng (c) qua [14] và [2].

Hai tính chất quyết định phạm vi dùng. Thứ nhất, nó không có nghịch đảo ở dạng đóng: từ pixel về mặt phẳng chuẩn hoá phải lặp. Với phương pháp indirect điều này chấp nhận được vì ta chỉ gỡ méo một lần cho mỗi đặc trưng; với direct, nơi ta chiếu hàng nghìn điểm mỗi khung, nó là chi phí thật. Thứ hai, và nghiêm trọng hơn: đa thức *phân kỳ* ngoài vùng dữ liệu hiệu chuẩn. Với ống kính trường nhìn lớn hơn khoảng 120° , r lớn và số hạng r^6 làm mô hình hành xử hoang dã ở rìa. Đây là lý do kỹ thuật khiến radtan không dùng được cho fisheye, và nó không phải chuyện “kém chính xác hơn một chút”.

B. Kannala–Brandt và các mô hình trường nhìn rộng

Với ống kính rộng, cách đúng là mô hình hoá *góc* chứ không phải bán kính trên mặt phẳng chuẩn hoá. Gọi ϑ là góc giữa tia và trục quang; Kannala–Brandt [15] đặt

$$r_d = \vartheta + k_1 \vartheta^3 + k_2 \vartheta^5 + k_3 \vartheta^7 + k_4 \vartheta^9.$$

Khác biệt then chốt so với radtan: ϑ bị chặn bởi trường nhìn (tối đa π), trong khi r trên mặt phẳng chuẩn hoá tiến ra vô hạn khi góc tiến tới 90° . Việc khai triển theo một biến bị chặn là lý do mô hình này ổn định ở trường nhìn rộng, còn khai triển theo một biến không bị chặn thì không. Đây là một ví dụ rõ ràng của việc *chọn đúng biến quan trọng hơn thêm bậc*.

Ba mô hình còn lại đi theo hướng khác: thay vì thêm bậc đa thức, chúng thêm *cấu trúc hình học*. UCM [16] chiếu tia lên một mặt cầu đơn vị rồi chiếu phối cảnh từ một điểm lệch tâm — đúng năm tham số. EUCM tổng quát mặt cầu thành ellipsoid, thêm một tham số. **Double-sphere** [17] dùng hai mặt cầu liên tiếp, cũng sáu tham số, và có lợi thế quyết định là *cả phép chiếu lẫn phép chiếu ngược đều có dạng đóng*. Với hệ thống chạy thời gian thực, đặc biệt là direct, tính chất này thường quan trọng hơn một chút độ chính xác.

Bảng 2.1. Năm họ mô hình méo. Cột cuối là tiêu chí chọn thực dụng, và nó nên được quyết định cùng lúc với việc chọn ống kính, không phải sau đó.

Mô hình	Số tham số	Nguyên lý	Hỏng khi	Chọn khi
Radtan	4–9	Khai triển đa thức theo r trên mặt phẳng chuẩn hoá	Trường nhìn rộng — đa thức phân kỳ ở rìa; không có nghịch đảo dạng đóng	$FOV \lesssim 100^\circ$; hệ sinh thái OpenCV
Kannala–Brandt	4–8	Đa thức lẻ theo góc ϑ	Ống kính rất bất đối xứng; vẫn cần lập để chiếu ngược	Fisheye tới 180° ; mặc định của Kalibr
UCM	5	Chiếu qua mặt cầu đơn vị lệch tâm	Ống kính lệch nhiều khỏi giả định cầu	Ít tham số, cần bền vững
EUCM	6	Mặt ellipsoid thay cho mặt cầu	Như UCM nhưng nhẹ hơn	Thoả hiệp tốt cho FOV rộng
Double-sphere	6	Hai mặt cầu liên tiếp	Rất rộng và rất bất đối xứng	Cần chiếu ngược dạng đóng — direct, thời gian thực

CẦN NHỚ

Chọn mô hình theo **trường nhìn** và theo **việc có cần chiếu ngược dạng đóng hay không** — hai tiêu chí đó quyết định gần hết. Đừng chọn theo số tham số: thêm tham số không phải luôn tốt hơn, và mục 4 giải thích vì sao.

Quy tắc thô dùng được ngay: FOV dưới 100° thì radtan là đủ và tiện vì cả hệ sinh thái hỗ trợ; từ 100° tới 180° thì Kannala–Brandt hoặc double-sphere; cần direct method hoặc cần chiếu ngược hàng nghìn lần mỗi khung thì double-sphere vì nó có dạng đóng cả hai chiều [17].

Và một điều quan trọng hơn việc chọn: **ghi lại lựa chọn ấy cùng lý do**, trong mã nguồn và trong tài liệu sản phẩm. Một tệp hiệu chuẩn không ghi rõ mô hình nào đã sinh ra nó là một quả mìn hẹn giờ — ai đó sẽ nạp nó bằng một mô hình khác, các con số vẫn hợp lệ về kiểu dữ liệu, và hệ thống sẽ sai một cách im lặng.

2.4 Mô hình sai so với tham số sai

Đây là mục quan trọng nhất của chương, và kỹ năng nó dạy được dùng lại ở C/14, C/18 và D/21.

A. Hai bệnh, một triệu chứng

Sau khi hiệu chuẩn, bạn có một tập phần dư — hiệu giữa vị trí góc bảng cờ quan sát được và vị trí chiếu lại từ mô hình. Có hai lý do hoàn toàn khác nhau khiến phần dư không bằng không, và chúng cần hai cách chữa ngược nhau.

Tham số sai (*parameter error*): dạng hàm đúng, nhưng con số chưa tối ưu — vì dữ liệu chưa đủ, vì bộ tối ưu dừng sớm, vì khởi tạo tệ. Dấu hiệu: phần dư phân bố *ngẫu nhiên*, không có hoa văn không gian, và *giảm* khi thêm dữ liệu tốt. Cách chữa: thêm dữ liệu, đa dạng hoá tư thế bảng, tối ưu kỹ hơn.

Mô hình sai (*model mismatch*): dạng hàm không mô tả nổi ống kính thật. Dấu hiệu: phần dư có *cấu trúc không gian* — tăng dần ra rìa, xoáy quanh tâm, khác nhau giữa các vùng ảnh — và *không*

giảm khi thêm dữ liệu; nó hội tụ về một mức sàn khác không. Cách chữa: đổi mô hình. Thêm dữ liệu chỉ làm bạn ước lượng chính xác hơn các tham số của một mô hình sai.

Phân biệt hai bệnh này rẻ tới mức đáng ngạc nhiên, và hầu như không ai làm: *vẽ phần dư lên mặt phẳng ảnh dưới dạng trường vectơ*. Nếu bạn thấy một hoa văn — các mũi tên hướng ra ngoài ở rìa, hoặc xoáy tròn — thì đó là mô hình sai, và bạn vừa nhìn thấy đúng phần mà mô hình không mô tả được. Nếu bạn thấy một đám nhiễu vô hướng thì đó là tham số sai hoặc chỉ là nhiễu, và bạn đã xong.

CẠM BẦY

Sai số chiều lại trung bình (RMS) gần như vô dụng làm thước đo chất lượng hiệu chuẩn, và đây là con số mà mọi công cụ hiệu chuẩn in ra đầu tiên.

Ba lý do. Thứ nhất, nó là một con số tổng hợp — nó xoá đúng cái hoa văn không gian vốn là thông tin chẩn đoán duy nhất. Một mô hình sai với hoa văn rõ rệt có thể có RMS thấp hơn một mô hình đúng với nhiễu cao. Thứ hai, nó *luôn giảm* khi thêm tham số, vì bạn đang khớp nhiều bậc tự do hơn vào cùng lượng dữ liệu; điều này không nói gì về khả năng tổng quát hoá. Thứ ba, nó chỉ đo trên vùng ảnh mà bảng hiệu chuẩn đã đi qua — và ống kính sai nhiều nhất ở rìa, đúng chỗ bảng khó tới nhất.

Bốn thứ đáng nhìn thay cho nó, theo thứ tự giá trị: (1) **bản đồ phần dư** dưới dạng trường vectơ trên mặt phẳng ảnh — chẩn đoán mô hình sai; (2) **độ phủ** của các góc bảng trên toàn khung hình, đặc biệt bốn góc ảnh; (3) **hiệp phương sai của tham số** lấy từ Hessian — tham số nào xác định kém, và có tương quan cao giữa hai tham số nào không ([A/06](#)); (4) **kiểm chéo**: hiệu chuẩn trên một nửa dữ liệu, đo phần dư trên nửa kia. Cái thứ tư bắt được overfitting mà ba cái kia bỏ sót.

B. Vì sao thêm tham số là một cái bẫy

Thêm k_3 , rồi k_4 , rồi số hạng tiếp tuyến bậc cao — RMS giảm mỗi lần. Cảm giác tiến bộ rất rõ ràng, và nó là ảo giác.

Cơ chế: mỗi tham số thêm vào là một bậc tự do để khớp *nhiều* trong dữ liệu hiệu chuẩn, không chỉ khớp tín hiệu. Hệ quả trong vùng có dữ liệu là một cải thiện nhỏ và thật; hệ quả ngoài vùng có dữ liệu là một hàm dao động mạnh. Vì bảng hiệu chuẩn hiếm khi phủ tới sát bốn góc ảnh, “ngoài vùng có dữ liệu” chính là nơi đặc trưng SLAM thường xuất hiện khi robot quay.

Có một hệ quả thứ hai, tinh vi hơn và tổn kém hơn, thuộc về [A/05](#). Các hệ số bậc cao *tương quan mạnh* với nhau và với tiêu cự: nhiều tổ hợp (f, k_1, k_2, k_3) khác nhau cho gần như cùng một ánh xạ trên vùng đã đo. Nghĩa là ma trận thông tin gần suy biến theo một hướng nào đó trong không gian tham số. Bộ tối ưu vẫn trả về một bộ số, nhưng bộ số đó nằm tùy tiện dọc theo một thung lũng phẳng — và hai lần hiệu chuẩn cùng một camera sẽ cho hai bộ số khác nhau đáng kể trong khi cho cùng RMS. Đây là kiểu hổng im lặng mà chương 5 mô tả tổng quát, gặp lần đầu ở đây.

Phép thử rẻ để phát hiện: hiệu chuẩn cùng một camera *ba lần* với ba bộ dữ liệu khác nhau, rồi so sánh các tham số. Nếu f ổn định trong vòng một phần nghìn mà k_2 nhảy ba chục phần trăm giữa các lần, thì k_2 không xác định được từ dữ liệu của bạn, và giữ nó trong mô hình chỉ là giữ một biến nhiễu. Phép thử này nên nằm trong quy trình sản xuất ở [D/21](#), không phải chỉ trong phòng nghiên cứu.

C. Hiệu chuẩn sai chày đi đâu

Câu hỏi cuối của mục, và là câu hỏi nối chương này vào cả phần còn lại của cây: nếu tham số nội sai một chút, hậu quả xuất hiện ở đâu?

Nó không biến mất. Bài toán MAP ở A/04 sẽ tìm cách làm phần dư nhỏ nhất có thể, và với tham số nội bị fix sai, cách duy nhất còn lại là *bỏ cong* các biến còn tự do — tư thế và cấu trúc. Vài đường chảy điển hình:

Tiêu cự sai chảy chủ yếu vào *tỉ lệ* của bản đồ. Nếu f ước lượng lớn hơn thật 1%, toàn bộ cấu trúc thu nhỏ tương ứng — và với mono, nơi tỉ lệ vốn là gauge tự do, điều này *hoàn toàn không nhìn thấy được* cho tới khi bạn so với chân trị hoặc ghép với một cảm biến có đơn vị mét. Đây là một trong những lý do stereo và VIO lộ ra lỗi hiệu chuẩn còn mono thì giấu nó đi.

Điểm chính sai chảy vào tư thế theo một kiểu đặc biệt khó chịu: nó trông *giống hết* một phép xoay nhỏ. Lệch điểm chính Δc_x pixel tương đương xấp xỉ một yaw $\Delta c_x / f$ radian. Với $f = 400$ và $\Delta c_x = 4$ px, đó là 10 mrad, tức $0,57^\circ$ — và vì nó là một sai lệch *hằng*, nó không bị khử qua trung bình mà tích lũy thành quỹ đạo cong. Một quỹ đạo đáng lẽ thẳng trôi thành một cung tròn rộng là dấu hiệu kinh điển của điểm chính sai. (Quan hệ này tôi tự dẫn từ xấp xỉ góc nhỏ của phép chiếu; chưa đối chiếu một nguồn cụ thể nào.)

Hệ số méo sai chảy vào phần dư có cấu trúc theo bán kính, và vào một sai lệch phụ thuộc vị trí trong ảnh — điểm ở rìa bị sai nhiều hơn điểm ở tâm. Trong BA, điều này nghĩa là các quan sát ở rìa mang thông tin sai lệch có hệ thống trong khi được gán cùng trọng số với quan sát ở tâm. Một cách chữa tạm thời thô nhưng hiệu quả là hạ trọng số quan sát theo bán kính; cách chữa đúng là sửa mô hình.

2.5 Ba mô hình mà SLAM hay quên

A. Rolling shutter

Phần lớn cảm biến CMOS giá rẻ đọc theo hàng: hàng r được phơi sáng vào khoảng thời điểm $t_0 + r t_{\text{line}}$. Với cảm biến 480 hàng và $t_{\text{line}} = 30 \mu\text{s}$, hàng cuối muộn hơn hàng đầu khoảng 14,4 ms — so sánh được với cả một chu kỳ khung hình.

Hệ quả với SLAM lớn hơn vẻ ngoài: giả thiết “một ảnh tương ứng một tư thế” — giả thiết nền của cả A/01 và A/04 — *sai*. Mỗi hàng ứng với một tư thế khác nhau, nên một ảnh tương ứng với một *đoạn quỹ đạo*. Với tốc độ góc 1 rad/s và 14,4 ms, chênh lệch hướng giữa hàng đầu và hàng cuối là 14,4 mrad; với $f = 400$ px, đó là gần 6 px dịch chuyển ở rìa ảnh — lớn hơn hàng chục lần so với độ chính xác subpixel mà B/07 vất vả đạt được.

Vì sao chỉ thêm một tham số t_{line} không đủ? Vì t_{line} chỉ cho biết *khi nào* mỗi hàng được chụp; để biết tư thế lúc đó, bạn còn cần một mô hình *chuyển động liên tục* để nội suy giữa các tư thế đã biết. Ba lựa chọn: nội suy tuyến tính giữa hai keyframe (rẻ, đủ cho chuyển động êm), B-spline trên $SE(3)$ [18] (đúng hơn, đắt hơn, là cách Kalibr làm), hoặc dùng IMU để nội suy (chính xác nhất khi có IMU, cách DSO rolling-shutter dùng [19]).

Kết luận thực dụng, và nó thuộc về D/21 nhiều hơn thuộc về chương này: *với sản phẩm đo đạc, hãy mua global shutter*. Chênh lệch giá vài đô la trên mỗi máy đổi lấy việc xóa bỏ hoàn toàn một lớp mô hình, một lớp tham số phải hiệu chuẩn, và một lớp chế độ hỏng chỉ xuất hiện khi rung. Đây là một trong số ít quyết định trong cây mà phần cứng thắng phần mềm một cách rõ ràng.

B. Mô hình quang trắc

Với phương pháp indirect, giá trị pixel chỉ dùng để tìm và mô tả đặc trưng, nên mọi biến đổi cường độ đơn điệu đều vô hại. Với phương pháp direct (B/08), phần dư *chính là* hiệu cường độ, nên mô hình quang trắc trở thành một phần bắt buộc của mô hình đo.

Ba thành phần cần hiệu chuẩn [20]: **vignetting** $V(\mathbf{p})$, hệ số suy giảm theo vị trí trong ảnh; **hàm đáp ứng** $G(\cdot)$, quan hệ phi tuyến giữa năng lượng và giá trị pixel; và **thời gian phơi sáng** e_t ,

thường tự động thay đổi giữa các khung. Mô hình đầy đủ:

$$I(\mathbf{p}) = G(e_t \cdot V(\mathbf{p}) \cdot B(\mathbf{p})),$$

với B là độ sáng thật của cảnh — đại lượng ta muốn so sánh giữa các khung.

Bỏ qua mô hình này thì direct method sẽ diễn giải một thay đổi phơi sáng thành một chuyển động của camera, vì cả hai đều làm cường độ pixel đổi. Đây là kiểu hỏng đặc trưng của direct và là lý do DSO dành hẳn một quy trình hiệu chuẩn riêng cho nó. Cách chữa rẻ hơn — không cần hiệu chuẩn — là đưa hai tham số affine cường độ (a, b) cho mỗi khung vào chính bài toán tối ưu, tức *tự hiệu chuẩn quang trắc trực tuyến*: đúng theo luận đề trung tâm, thả biến ra thay vì fix nó.

C. Stereo và mô hình độ sâu

Cặp stereo đã hiệu chỉnh (rectified) là trường hợp đặc biệt của hình học epipolar mà [A/01](#) mục 3C đã tính: ràng buộc epipolar rút thành $v_1 = v_2$, và độ sâu là $z = bf/d$. Rectification là phép biến đổi ảnh làm điều đó thành đúng cho một cặp camera bất kỳ, và nó bao gồm cả việc gỡ méo — nên mọi bàn luận về mô hình méo ở mục 3 áp dụng nguyên vẹn.

Điều đáng nhấn mạnh ở đây là *tại sao stereo đáng giá trong sản xuất*, và nó không phải lý do hay được nêu. Lý do thường nghe là “có độ sâu trực tiếp”. Lý do thật sâu hơn: đường cơ sở b là một độ dài *đã biết bằng mét*, nên nó phá vỡ gauge tỉ lệ ngay từ khung hình đầu tiên. Khởi tạo trở nên tầm thường — không cần parallax, không cần model selection, không cần chờ chuyển động — và toàn bộ lớp chế độ hỏng ở [A/01](#) mục 6 biến mất. Đó là lý do thật sự khiến stereo là lựa chọn mặc định cho sản phẩm, và vì sao [D/21](#) khuyến nghị stereo cộng IMU cộng odometry bánh xe làm điểm ngọt cho robot mặt đất.

Cái giá cũng phải nói rõ. Đường cơ sở phải *giữ ổn định*: một thay đổi vài chục micromet trong khoảng cách hoặc vài phần trăm độ trong góc giữa hai camera đã đủ làm sai độ sâu ở xa một cách đáng kể. Với khung nhựa và thay đổi nhiệt độ hàng ngày, đây là vấn đề thật chứ không phải lo xa, và nó là lý do hiệu chuẩn ngoại stereo trực tuyến ([C/14](#)) tồn tại. Đồng thời, quan hệ $\sigma_z \approx z^2 \sigma_d / (bf)$ từ [A/01](#) nói rằng lợi ích của stereo tắt dần theo bình phương khoảng cách — ngoài một tầm nào đó, stereo thoái hoá thành mono và bạn phải trở lại mọi thứ ở chương 1.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Kalibr [21]	Hiệu chuẩn nội đa mô hình (pinhole-radtan, KB, UCM...); mô hình liên tục theo thời gian cho rolling shutter	Công cụ chuẩn; đọc tài liệu của nó để hiểu quy trình thu dữ liệu chính là điều kiện kích thích ở A/05
DSO [22]	Mô hình quang trắc đầy đủ: vignette, CRF, exposure; cộng hai tham số affine mỗi khung	Ví dụ rõ nhất của mục 5B; cũng là ví dụ về tự hiệu chuẩn trực tuyến
Basalt [23]	Mặc định double-sphere; spline $SE(3)$ cho hiệu chuẩn	Lý do chọn double-sphere là chiều ngược dạng đóng, đúng như Bảng 2.1
OpenCV calib3d	radtan và fisheye; initUndistortRectifyMap	Hệ sinh thái mặc định; chú ý nó dùng quy ước tham số riêng, dễ sai khi chuyển đổi
COLMAP [11]	Nhiều mô hình camera, ước lượng nội ngay trong BA	Mình hoạ luận đề trung tâm: nội là biến chứ không phải hằng

MINI PROJECT SLAM · Hiệu chuẩn một camera ba lần, rồi chứng minh bằng số rằng RMS nói dối · 2 ngày

Mục tiêu	tự tay thấy sự khác nhau giữa mô hình sai và tham số sai, và thấy vì sao RMS không phân biệt được hai thứ đó.
Dữ liệu	tự chụp: một bảng ChArUco, một camera bất kỳ có FOV càng rộng càng tốt. Thu <i>ba</i> bộ dữ liệu độc lập, mỗi bộ vài chục ảnh với tư thế bảng đa dạng — và cố tình để một bộ có độ phủ kém ở bốn góc ảnh.
Các bước	hiệu chuẩn mỗi bộ với ba cấu hình mô hình: radtan k_1k_2 , radtan $k_1k_2k_3p_1p_2$, và Kannala–Brandt; với mỗi lần chạy ghi lại RMS, bộ tham số, và hiệp phương sai tham số từ Hessian; vẽ bản đồ phần dư dạng trường vectơ cho từng lần; cuối cùng làm kiểm chéo — hiệu chuẩn trên bộ 1 rồi đo phần dư trên bộ 2 và 3.
Xong khi	bạn có một bảng chín hàng (ba bộ dữ liệu $\times$ ba mô hình) với các cột: RMS trong mẫu, RMS kiểm chéo, độ lệch của f giữa ba bộ, độ lệch của k_2 giữa ba bộ. Bạn phải quan sát được ít nhất hai điều: RMS trong mẫu giảm đơn điệu theo số tham số trong khi RMS kiểm chéo thì không, và các hệ số bậc cao dao động mạnh hơn nhiều so với f giữa ba bộ. Nếu không quan sát được, hãy kiểm xem độ phủ ảnh của bạn có quá tốt không — và đó cũng là một kết quả đáng ghi.
Nối với	C/14-hieu-chuan-nhu-mot-nhanh-cua-slam biến quy trình này thành một bài toán MAP có thể thả tham số ra; D/21-tu-prototype-len-mass-production biến bảng trên thành tiêu chí nghiệm thu trên dây chuyền.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì RMS là một con số tổng hợp và nó xoá đúng thứ mang thông tin chẩn đoán: hoa văn không gian của phần dư. Một mô hình sai có hoa văn rõ rệt vẫn có thể cho RMS thấp hơn một mô hình đúng có nhiễu cao. Thêm nữa, RMS luôn giảm khi thêm tham số, và nó chỉ đo trên vùng ảnh mà bảng đã đi qua — trong khi ống kính sai nhiều nhất ở rìa, chỗ bảng khó tới nhất. Bốn thứ nên nhìn thay: bản đồ phần dư dạng trường vectơ, độ phủ góc bảng trên toàn khung, hiệp phương sai tham số từ Hessian, và kiểm chéo trên dữ liệu chưa dùng để khớp.
- A2. Vì mỗi tham số thêm vào là một bậc tự do để khớp *nhiều*, không chỉ khớp tín hiệu — đây là overfitting ở dạng thuần túy nhất. Trong vùng có dữ liệu bạn được một cải thiện nhỏ và thật; ngoài vùng có dữ liệu, đa thức bậc cao dao động mạnh. Vì bảng hiếm khi phủ sát bốn góc ảnh, “ngoài vùng có dữ liệu” chính là nơi đặc trưng SLAM xuất hiện khi robot quay — nên lỗi lớn nhất nằm đúng chỗ nó gây hại nhất. Có một hệ quả thứ hai tinh vi hơn: các hệ số bậc cao tương quan mạnh với nhau và với f , nên ma trận thông tin gần suy biến và hai lần hiệu chuẩn cùng một camera cho hai bộ số khác nhau đáng kể với cùng RMS.
- A3. Quyết định đó đúng hay sai phụ thuộc vào *tỉ số giữa độ tán của tham số trong lô và độ nhạy của hệ thống với tham số ấy* — không phụ thuộc vào việc các camera “cùng loại”. Cách đo trước khi chốt thiết kế: hiệu chuẩn một mẫu vài chục máy, tính độ lệch chuẩn của từng tham số qua lô; rồi chạy phân tích độ nhạy — nhiễu loạn tham số bằng đúng độ lệch chuẩn ấy trong mô phỏng và đo sai số tư thế đầu ra. Nếu sai số vẫn dưới ngân sách thì dùng chung tham số được. Điều quan trọng là việc này phải làm *trước* khi chốt cơ khí, vì kết quả có thể nói rằng dung sai lắp ráp phải siết chặt hơn — và đổi cơ khí sau khi mở khuôn thì đắt. Chi tiết ở [D/21](#).
- A4. Nhiệt độ. Ống kính và khung giãn nở, tiêu cự đổi vài phần vạn, và với stereo thì đường cơ sở cùng góc giữa hai camera đổi — đủ để làm sai độ sâu ở xa. Chu kỳ ngày đêm giải thích tính lặp lại hàng ngày. Nó thuộc về [C/14](#) (hiệu chuẩn trực tuyến để bù trôi) và [D/21](#) (mục về trôi hiệu chuẩn theo nhiệt độ, rung, lão hoá keo). Cần phân biệt với hai nguyên nhân khác có cùng biểu hiện chu kỳ: thay đổi ánh sáng trong ngày làm đổi tập đặc trưng, và auto-exposure đổi theo giờ — phép thử phân biệt là xem sai lệch có tương quan với nhiệt độ vỏ máy hay với độ sáng cảnh.
- A5. Vì t_{line} chỉ nói *khi nào* mỗi hàng được chụp; để biết *tư thế* tại thời điểm đó, cần thêm một *mô hình chuyển động liên tục* để nội suy giữa các tư thế rời rạc. Không có nó thì biết thời điểm cũng vô dụng. Ba lựa chọn: nội suy tuyến tính giữa hai keyframe, B-spline trên $SE(3)$ [\[18\]](#), hoặc dùng IMU nội suy [\[19\]](#). Về mặt sản phẩm, kết luận thực dụng là mua global shutter: vài đô la đổi lấy

việc xoá hẳn một lớp mô hình, một lớp tham số phải hiệu chuẩn, và một lớp chế độ hồng chỉ xuất hiện khi rung.

NÓI LẠI THẬT ĐƠN GIẢN

Một cái máy ảnh giống một cái cửa sổ có kính hơi cong. Bạn nhìn qua nó thấy thế giới, nhưng mọi thứ bị bẻ đi một chút — và bị bẻ nhiều nhất ở gần mép cửa sổ.

Hiệu chuẩn là việc đo xem kính cong như thế nào, để sau này biết đường bẻ ngược lại. Cách đo là giờ một tấm bìa kẻ ô vuông đã biết trước kích thước lên trước cửa sổ ở nhiều tư thế, xem các ô vuông méo ra sao, rồi suy ngược ra độ cong.

Chỗ khó không phải việc đo. Chỗ khó là bạn phải *đoán trước* kính cong theo kiểu gì — cong đều như đáy chai, hay cong xoắn, hay cong một bên nhiều hơn bên kia. Bạn chọn một kiểu, rồi máy tính tìm con số khớp nhất với kiểu đó. Nếu bạn đoán đúng kiểu, kết quả tốt. Nếu bạn đoán sai kiểu, máy tính vẫn đưa ra con số — con số khớp nhất có thể trong cái kiểu sai ấy — và nó vẫn báo “sai lệch trung bình rất nhỏ”.

Cách phân biệt rất đơn giản và hầu như không ai làm. Đừng nhìn con số trung bình. Hãy vẽ chỗ nào còn lệch và lệch về hướng nào, lên cả tấm ảnh. Nếu các mũi tên lệch nằm lung tung không theo quy luật gì, bạn đã xong — phần còn lại chỉ là nhiễu. Nếu các mũi tên xếp thành hình — toả ra ở rìa, hay xoáy quanh tâm — thì bạn đang nhìn thấy chính cái phần mà kiểu bạn đoán không mô tả được. Lúc đó đo thêm bao nhiêu cũng vô ích; phải đổi cách đoán.

Và một chuyện nữa, quan trọng cho việc làm sản phẩm. Kính giãn nở khi trời nóng. Nên cái máy bạn đo chính xác vào buổi sáng có thể lệch đi vào buổi chiều, rồi hôm sau lại đúng. Không có lỗi phần mềm nào cả — chỉ là thế giới vật lý không đứng yên như tài liệu giả định.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *chọn dạng hàm*. Không có thuật toán nào chọn hộ bạn: bạn phải nhìn hoa văn phần dư, biết một chút về quang học của ống kính đang dùng, rồi quyết định. Đây là một trong ít chỗ trong cả cây mà kinh nghiệm của con người thật sự không thay thế được, và cũng là lý do mục 4 được viết dài hơn về ngoài của nó.

Một câu: Sai số ngẫu nhiên là nhiễu và chữa được bằng nhiều dữ liệu hơn; sai số có hình dạng là mô hình sai và chỉ chữa được bằng mô hình khác.

2.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *phần dư có cấu trúc không gian là dấu hiệu mô hình sai, và thêm dữ liệu không chữa được* — bị bác bỏ nếu ai đó cho thấy một trường hợp mà phần dư có hoa văn rõ rệt nhưng hoa văn ấy *biến mất* khi thêm dữ liệu với cùng mô hình. Có một trường hợp cần loại trừ cẩn thận trước khi kết luận: nếu dữ liệu ban đầu có độ phủ rất lệch — chẳng hạn bảng chỉ ở một góc ảnh — thì bộ tối ưu có thể trả về tham số kém và sinh ra hoa văn giả, và thêm dữ liệu đúng là chữa được. Đó là tham số sai chứ không phải mô hình sai, và phép thử phân biệt là xem hoa văn có hội tụ về một mức sàn khác không hay không.

Mệnh đề thứ hai — *RMS không phải thước đo chất lượng hiệu chuẩn* — là mệnh đề dễ kiểm nhất và mini project ở trên chính là phép kiểm. Nó bị bác bỏ nếu trên một tập đủ lớn các lần hiệu chuẩn, RMS trong mẫu tương quan chặt với sai số tư thế thật của hệ SLAM dùng bộ tham số ấy. Tôi cho là không, nhưng tôi *chưa* chạy thí nghiệm đó và không dẫn được nguồn nào đã chạy nó một cách có hệ thống (tôi suy ra, chưa đối chiếu nguồn).

Mệnh đề thứ ba — *lệch điểm chính Δc_x tương đương một yaw $\Delta c_x/f$* — là một xấp xỉ góc nhỏ mà tôi tự dẫn. Nó bị bác bỏ nếu ai đó chỉ ra rằng hai hiệu ứng đó tách được từ dữ liệu trong cấu hình SLAM thông thường. Thực ra tôi ngờ rằng chúng *tách được một phần* khi có chuyển động đủ đa dạng — vì lệch điểm chính gắn với vị trí trong ảnh còn yaw thì không — và làm rõ điều đó là một mục trong 99-chưa-biết.md.

Cuối cùng, một mệnh đề mềm: chương này khuyến nghị global shutter cho sản phẩm đo đạc. Điều đó bị bác bỏ nếu một hệ rolling shutter có mô hình đúng đạt độ chính xác tương đương trong điều kiện rung thật, với chi phí tính toán chấp nhận được trên phần cứng nhúng. Các công trình rolling-shutter [19] cho thấy mô hình hoá được vấn đề; điều chúng *không* cho thấy là chi phí và độ bền trong sản xuất hàng loạt, và đó mới là câu hỏi của D/21.

2.7 Kết nối

Nối lên trên: [A/01-hình-hoc-da-thi](#) là chương mà chương này phục vụ — toàn bộ hình học ở đó giả định phép biến đổi pixel sang mặt phẳng chuẩn hoá đã đúng, và mục 4C ở đây nói chuyện gì xảy ra khi nó không đúng.

Nối xuống dưới trong Phần A. [A/04-uoc-luong-map-va-bundle-adjustment](#) là chỗ tham số nội trở thành biến tối ưu được chứ không còn là hằng số nạp từ tệp. [A/05-observability-gauge-va-suy-bien](#) giải thích tổng quát hiện tượng ở mục 4B: tương quan giữa các hệ số bậc cao là một hướng gần suy biến của ma trận thông tin, và “hiệu chuẩn ba lần ra ba kết quả” là phép thử thực dụng để phát hiện nó. [A/06-bat-dinh-va-lan-truyen-sai-so](#) cho công cụ định lượng: hiệp phương sai tham số lấy từ Hessian là thứ nên in ra thay cho RMS.

Nối sang các phần sau. [B/08-mo-hinh-phan-du](#) là nơi mô hình quang trắc ở mục 5B trở thành bắt buộc, vì direct method dùng cường độ pixel làm phần dư. [C/14-hieu-chuan-nhu-mot-nhanh-cua-slam](#) là chương nối trực tiếp: nó lấy mọi thứ ở đây và đặt vào khung MAP, biến hiệu chuẩn thành một trường hợp của SLAM với cấu trúc bị fix. [C/18-danh-gia-va-phuong-phap-luan](#) mượn nguyên kỹ năng ở mục 4: phân biệt sai lệch có hệ thống với nhiễu là cùng một kỹ năng dù đối tượng là phần dư hiệu chuẩn hay là NEES của bộ ước lượng. [D/21-tu-prototype-len-mass-production](#) là chỗ mọi thứ ở đây gặp thực tế: thời gian hiệu chuẩn mỗi máy, dung sai cơ khí, trôi theo nhiệt độ, và quyết định hiệu chuẩn từng máy hay dùng chung tham số cho cả lô.

Nối ngang: `../marker/` chương 1 bàn cùng chủ đề từ góc nhìn khác — ở đó mục tiêu là độ chính xác milimet trên một target đã biết, nên yêu cầu về mô hình méo còn gắt hơn, và lưỡng nghĩa tư thế phẳng làm mọi sai số nội trở nên nguy hiểm hơn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao khai triển theo góc ϑ ổn định hơn khai triển theo bán kính r khi trường nhìn rộng — và tính chất nào của hai biến ấy tạo ra khác biệt?
2. Nêu hai dấu hiệu phân biệt mô hình sai với tham số sai, và nêu phép đo rẻ nhất thực hiện được cả hai.
3. Thêm k_3 làm RMS giảm. Nêu ba lý do khác nhau khiến điều đó không chứng minh mô hình tốt hơn.
4. Tiêu cự ước lượng lớn hơn thật 1%. Với hệ mono, hệ stereo và hệ VIO, hậu quả xuất hiện ở đâu và hệ nào phát hiện được?
5. Vì sao double-sphere được ưa chuộng trong hệ direct thời gian thực, trong khi Kannala-Brandt chính xác không kém?
6. Rolling shutter với 480 hàng và $t_{\text{line}} = 30 \mu\text{s}$, camera xoay 1 rad/s , $f = 400 \text{ px}$. Tính độ dịch pixel giữa hàng đầu và hàng cuối ở rìa ảnh, rồi so với độ chính xác subpixel mà front-end đạt được.
7. Bốn giả thiết ngầm của mô hình pinhole là gì — kể cả những cái không ai nói ra — và bỏ mỗi cái thì chương nào trong cây phải xử lý?

Lie group và tối ưu trên đa tạp (Lie Groups and Manifold Optimization)

PHỤ THUỘC [A/01-hình-học-da-thi](#) (tư thế (R, t) , phép chiếu); [A/02-mô-hình-camera](#) (chuỗi từ điểm 3D tới pixel — chương này tính đạo hàm của đúng chuỗi đó). Chương tương ứng ở cây quản tính là `.. /IMU_Fusion/` chương 2, nói kỹ hơn về $SE_2(3)$ và bộ lọc bất biến; chỗ nào cây đó đã nói sâu thì ở đây chỉ nhắc.

TÓM TẮT

Chương này giải quyết một vấn đề mà hai chương trước đã dựng ra nhưng không giải được: tư thế camera *không phải một vectơ*. Ma trận xoay có chín số nhưng chỉ ba bậc tự do, và chín số ấy bị ràng buộc bởi $R^T R = I$ — một ràng buộc phi tuyến mà mọi bộ tối ưu không ràng buộc sẽ phá vỡ ngay ở bước đầu tiên. Chương giải quyết bằng cách dựng đúng bộ công cụ tối thiểu: nhóm Lie $SO(3)$, $SE(3)$, $Sim(3)$ và đại số Lie tương ứng; ánh xạ exp và log nối hai thế giới; phép nhiễu loạn trái và phải; toán tử $\boxplus$ và $\boxminus$ biến một bước tối ưu thành một phép toán hợp lệ trên đa tạp; và Adjoint để chuyển nhiễu loạn giữa hai phía. Đích đến cụ thể và kiểm được: tự dẫn ra Jacobian 2×6 của phần dư chiếu theo tư thế camera, vì đó là khối xây dựng của mọi thứ ở [A/04](#) và [B/09](#). Nếu chỉ nhớ một điều: *ta không cộng vào tư thế, ta nhân một nhiễu loạn nhỏ vào nó* — và toàn bộ phần còn lại là hệ quả kỹ thuật của câu đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **nhóm (group)** — một tập hợp với phép toán kết hợp có phần tử đơn vị và phần tử nghịch đảo. Tập các phép xoay là một nhóm: hợp hai phép xoay là một phép xoay, có phép xoay không, và mọi phép xoay đảo ngược được.
- **đa tạp (manifold)** — một không gian mà *cục bộ* trông như không gian Euclid, nhưng toàn cục thì không. Mặt cầu là ví dụ chuẩn: quanh mỗi điểm nó phẳng như một tờ giấy, nhưng không có cách nào trải cả mặt cầu ra một tờ giấy.
- **nhóm Lie (Lie group)** — một nhóm đồng thời là một đa tạp trơn, với phép nhân và phép nghịch đảo trơn. $SO(3)$ và $SE(3)$ là hai ví dụ mà cả cây này dùng.
- **đại số Lie (Lie algebra)** — không gian tiếp tuyến của nhóm tại phần tử đơn vị — một không gian vectơ thật sự, nên cộng trừ được thoải mái. Với $SO(3)$ nó là $\mathfrak{so}(3)$, đồng nhất với $\mathbb{R}^3$ qua toán tử $[\cdot]_{\times}$.
- **ánh xạ mũ (exponential map)** — phép biến một phần tử của đại số Lie (vectơ nhỏ, cộng được) thành một phần tử của nhóm (phép xoay, nhân được). Với $SO(3)$ nó chính là công thức Rodrigues.
- **nhiễu loạn (perturbation)** — một phần tử nhóm rất gần đơn vị, dùng để biểu diễn một thay đổi nhỏ. Nhân nó vào bên trái hay bên phải cho hai quy ước khác nhau, và trộn hai quy ước là nguồn lỗi dấu phổ biến nhất trong lĩnh vực.
- **Adjoint** — ma trận chuyển một nhiễu loạn từ phía này sang phía kia: $T \exp(\xi^{\wedge}) = \exp((\text{Ad}_T \xi)^{\wedge}) T$. Nó là công cụ để đổi giữa quy ước trái và phải mà không phải dẫn lại từ đầu.
- **retraction** — phép ánh xạ một bước đi trong không gian tiếp tuyến trở về đa tạp. Với nhóm Lie, exp là một retraction tự nhiên và chính xác; với đa tạp tổng quát người ta dùng xấp xỉ rẻ hơn.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn tham số hoá tư thế bằng chín phần tử của R và chạy Gauss-Newton. Sau mười vòng lặp, $\det R = 0,94$. Chuyện gì đã xảy ra về mặt hình học, và vì sao “chuẩn hoá lại sau mỗi bước” là một cách chữa tồi chứ không phải một cách chữa tốt?
- Q2. Euler angle có đúng ba số cho ba bậc tự do — tham số hoá tối thiểu, nghe như lựa chọn đúng. Vì sao không ai dùng nó trong bộ tối ưu SLAM, và hiện tượng gimbal lock thực chất là phát biểu gì về đa tạp?
- Q3. Nhiễu loạn trái và nhiễu loạn phải cho hai Jacobian khác nhau, khác cả dấu lẫn dạng. Hai kết quả tối ưu có khác nhau không? Nếu không thì vì sao phải chọn, và chọn sai thì hỏng ở đâu?
- Q4. Loop closure trong SLAM đơn mắt phải dùng $Sim(3)$ chứ không phải $SE(3)$. Bậc tự do thứ bảy đến từ đâu, và vì sao nó *không* xuất hiện khi có stereo hoặc IMU?
- Q5. Jacobian của phần dư chiều theo tư thế là một ma trận 2×6 . Vì sao là 6 chứ không phải 12 hay 16, và con số đó nói gì về cách bộ giải nhìn bài toán?

3.1 Đặt vấn đề

Hai chương trước dựng lên toàn bộ hình học và mô hình đo. Bây giờ đến lúc tối ưu, và ngay ở bước đầu tiên ta gặp một trở ngại mà không chương nào trước đó chuẩn bị cho.

Bài toán MAP ở A/04 sẽ có dạng quen thuộc: tìm θ cực tiểu hoá tổng bình phương phần dư. Gauss-Newton giải bằng cách lặp: tuyến tính hoá quanh ước lượng hiện tại, giải một hệ tuyến tính ra bước đi δ , rồi cập nhật $\theta \leftarrow \theta + \delta$. Bước cuối cùng — phép cộng — chính là chỗ mọi thứ vỡ.

Vì tư thế camera không phải một vectơ. Nếu tôi lấy hai ma trận xoay hợp lệ và cộng chúng lại phần tử với phần tử, kết quả nói chung không phải ma trận xoay: nó không trực giao, định thức không bằng một, và nó biến một vật rắn thành một vật bị kéo méo. Tập các ma trận xoay không đóng dưới phép cộng. Nói bằng ngôn ngữ hình học: $SO(3)$ là một mặt cong trong không gian chín chiều, và phép cộng đưa ta ra khỏi mặt.

Ai gặp bài toán này? Mọi người ước lượng hướng — robot, hàng không, đồ hoạ, sinh học cấu trúc. Và lịch sử cho thấy ba cách chữa thiếu sót, mỗi cách hỏng theo một kiểu riêng đáng ghi nhớ.

Cách thứ nhất: *tham số hoá tối thiểu bằng Euler angle*. Ba số cho ba bậc tự do, cộng thoải mái. Nó hỏng vì *gimbal lock* — ở một số hướng, hai trong ba trục quay trùng nhau và mất một bậc tự do. Điều quan trọng cần hiểu là gimbal lock không phải một khiếm khuyết của cách đánh số mà là một phát biểu về topology: $SO(3)$ không đồng phôi với bất kỳ vùng nào của $\mathbb{R}^3$, nên *mọi* tham số hoá ba số đều có kỳ dị ở đâu đó. Đổi thứ tự trục chỉ dịch chỗ kỳ dị đi, không xoá được nó.

Cách thứ hai: *tham số hoá dư bằng quaternion* với ràng buộc chuẩn bằng một. Không có kỳ dị, nhưng ràng buộc vẫn còn đó, và bộ tối ưu không ràng buộc vẫn phá nó. Chuẩn hoá lại sau mỗi bước thì chạy được, nhưng nó làm hỏng ma trận hiệp phương sai một cách tinh vi: hiệp phương sai bốn chiều mô tả một phân bố trong không gian bốn chiều, trong khi bất định thật chỉ có ba chiều, nên ma trận suy biến theo một hướng và nghịch đảo của nó không tồn tại. Với bộ ước lượng cần hiệp phương sai đáng tin (A/06), đây không phải chuyện nhỏ.

Cách thứ ba: *tối ưu có ràng buộc*. Đúng về nguyên tắc, nhưng nó vứt bỏ toàn bộ cấu trúc thừa mà A/04 phụ thuộc vào, và làm bài toán chậm đi nhiều bậc độ lớn.

Lời giải đúng gọn hơn cả ba, và nó là một trong những ý tưởng đẹp nhất trong cây. Đừng cộng vào tư thế — *nhân một nhiễu loạn nhỏ vào nó*. Nhiễu loạn được tham số hoá bằng một vectơ ba chiều (cộng trừ thoải mái, không kỳ dị vì nó luôn ở gần không), và phép nhân giữ ta ở đúng trên đa tạp. Bộ tối ưu làm việc trong không gian vectơ của nhiễu loạn; đa tạp chỉ xuất hiện ở bước cập

nhật. Toàn bộ chương này là việc làm cho ý tưởng ấy chặt chẽ và tính ra được các đạo hàm cần thiết.

TRỰC GIÁC

Hình dung $SO(3)$ là bề mặt của một quả cầu (không hoàn toàn đúng về topology, nhưng đúng về cái ta cần). Mỗi điểm trên mặt cầu là một hướng khả dĩ của camera. Bây giờ bạn đang ở một điểm và muốn đi tới điểm tốt hơn. Cộng một vectơ vào toạ độ 3D của điểm hiện tại sẽ đưa bạn ra khỏi mặt cầu — vào trong ruột hoặc ra ngoài không gian. Đó chính là điều xảy ra khi cộng vào ma trận xoay.

Cách đúng: đặt một tờ giấy *tiếp xúc* với mặt cầu tại chỗ bạn đang đứng. Trên tờ giấy đó, mọi thứ là hình học phẳng thông thường — cộng vectơ, tính đạo hàm, giải hệ tuyến tính, tất cả đều hợp lệ. Bạn quyết định đi đâu *trên tờ giấy*, rồi *cuộn* tờ giấy trở lại mặt cầu để biết mình thật sự đến điểm nào. Phép cuộn ấy là $\exp$; tờ giấy là đại số Lie; và việc đặt lại tờ giấy tại điểm mới sau mỗi bước là lý do mỗi vòng lặp Gauss-Newton phải tính lại Jacobian.

Hai chi tiết đáng giữ. Một: tờ giấy chỉ xấp xỉ tốt *gần* chỗ tiếp xúc, nên bước đi phải nhỏ — đó chính là lý do trust region ở [A/04](#) tồn tại. Hai: có hai cách dán tờ giấy vào mặt cầu, ứng với nhiễu loạn trái và phải, và chúng cho hai hệ toạ độ khác nhau trên cùng tờ giấy. Kết quả cuối trùng nhau, nhưng công thức trung gian — và ma trận hiệp phương sai — thì không.

3.2 $SO(3)$: xoay

A. Nhóm và đại số của nó

$$SO(3) = \{ R \in \mathbb{R}^{3 \times 3} : R^\top R = I, \det R = +1 \}.$$

Điều kiện $R^\top R = I$ là sáu phương trình độc lập (ma trận $R^\top R$ đối xứng nên chỉ có sáu phần tử độc lập), nên từ chín tham số còn lại ba bậc tự do. Điều kiện $\det R = +1$ loại phản xạ; nó không giảm thêm chiều mà chỉ chọn một trong hai thành phần liên thông.

Đại số Lie $\mathfrak{so}(3)$ là không gian tiếp tuyến tại I . Lấy một đường cong $R(t)$ trong nhóm với $R(0) = I$, đạo hàm ràng buộc $R^\top R = I$ theo t tại $t = 0$:

$$\dot{R}^\top(0) + \dot{R}(0) = 0,$$

tức $\dot{R}(0)$ là ma trận *phản đối xứng*. Vì sao bước này hợp lệ? Vì mọi đường cong nằm trong nhóm phải thoả ràng buộc tại mọi t , nên đạo hàm của ràng buộc cũng phải bằng không — đây là cách chuẩn để tìm không gian tiếp tuyến của một tập được định nghĩa bởi phương trình.

Ma trận phản đối xứng 3×3 có đúng ba tham số độc lập, khớp với ba bậc tự do, và ta đồng nhất chúng với $\mathbb{R}^3$ qua

$$[\phi]_\times = \begin{pmatrix} 0 & -\phi_3 & \phi_2 \\ \phi_3 & 0 & -\phi_1 \\ -\phi_2 & \phi_1 & 0 \end{pmatrix}, \quad [\phi]_\times \mathbf{v} = \phi \times \mathbf{v}.$$

Ý nghĩa vật lý của ϕ đáng nói rõ vì nó làm mọi thứ sau dễ nhớ: *hướng của nó là trục quay, độ lớn là góc quay tính bằng radian*. Đây là biểu diễn trục-góc, và nó là biểu diễn tự nhiên nhất của một phép xoay nhỏ.

B. Rodrigues: từ đại số sang nhóm

Ánh xạ mũ định nghĩa bằng chuỗi lũy thừa $\exp(A) = \sum_k A^k/k!$, nhưng với $[\phi]_\times$ chuỗi ấy có dạng đóng, và việc tự dẫn nó là một bài tập đáng làm một lần.

Đặt $\vartheta = \|\phi\|$ và $\mathbf{a} = \phi/\vartheta$. Tính chất then chốt là $[\mathbf{a}]_\times^3 = -[\mathbf{a}]_\times$ (kiểm trực tiếp bằng nhân ma trận, hoặc bằng đồng nhất thức $\mathbf{a} \times (\mathbf{a} \times (\mathbf{a} \times \mathbf{v})) = -\mathbf{a} \times \mathbf{v}$ khi $\|\mathbf{a}\| = 1$). Nhờ nó, mọi lũy thừa bậc cao rút về bậc một và bậc hai, chuỗi tách thành hai chuỗi quen thuộc, và ta được

$$\exp([\phi]_\times) = I + \frac{\sin \vartheta}{\vartheta} [\phi]_\times + \frac{1 - \cos \vartheta}{\vartheta^2} [\phi]_\times^2. \quad (3.1)$$

Đây là công thức Rodrigues. Tôi tự dẫn lại được toàn bộ (cửa a) và đối chiếu với [24, §7.1] cùng [25] (cửa c).

Hai điều đáng ghi. Thứ nhất, kỳ dị số học tại $\vartheta \rightarrow 0$: hai hệ số có dạng 0/0. Giới hạn tồn tại và hữu hạn ($\sin \vartheta/\vartheta \rightarrow 1$, $(1 - \cos \vartheta)/\vartheta^2 \rightarrow 1/2$), nhưng tính trực tiếp trong số học dấu phẩy động sẽ mất độ chính xác nghiêm trọng khi ϑ nhỏ. Mọi thư viện tốt chuyển sang khai triển Taylor dưới một ngưỡng; ngưỡng đặt ở đâu cho đúng là một câu hỏi tôi chưa trả lời được và nó nằm trong 99-chua-biet.md. Thứ hai, và quan trọng hơn cho SLAM: trong bộ tối ưu, ϑ luôn nhỏ, vì ta chỉ dùng exp cho nhiễu loạn. Nên trường hợp góc nhỏ không phải trường hợp biên hiểm gặp mà là trường hợp thường xuyên, và cài đặt đúng nó không phải chuyện cầu toàn.

Ánh xạ ngược log lấy góc từ vết ma trận, $\vartheta = \arccos \frac{\text{tr}(R)-1}{2}$, rồi lấy trực từ phần phản đối xứng. Nó cũng có kỳ dị, ở $\vartheta \rightarrow 0$ và ở $\vartheta \rightarrow \pi$; trường hợp thứ hai hiếm trong nhiễu loạn nhưng xuất hiện khi tính sai lệch giữa hai hướng cách xa nhau, chẳng hạn trong phần dư của pose graph ở B/11.

C. Ví dụ nhỏ tính tay

Lấy $\phi = (0, 0, \pi/2)$, tức xoay 90° quanh trục z . Ở đây $\vartheta = \pi/2$, $\sin \vartheta = 1$, $\cos \vartheta = 0$.

$$[\phi]_\times = \begin{pmatrix} 0 & -\pi/2 & 0 \\ \pi/2 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad [\phi]_\times^2 = \begin{pmatrix} -\pi^2/4 & 0 & 0 \\ 0 & -\pi^2/4 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Thay vào (3.1) với hệ số $\sin \vartheta/\vartheta = 2/\pi$ và $(1 - \cos \vartheta)/\vartheta^2 = 4/\pi^2$:

$$R = I + \frac{2}{\pi} \begin{pmatrix} 0 & -\pi/2 & 0 \\ \pi/2 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} + \frac{4}{\pi^2} \begin{pmatrix} -\pi^2/4 & 0 & 0 \\ 0 & -\pi^2/4 & 0 \\ 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Kiểm: ma trận này biến $(1, 0, 0)$ thành $(0, 1, 0)$, tức trục x quay sang trục y — đúng là xoay 90° quanh z theo quy tắc bàn tay phải. Trục giao và định thức bằng một, kiểm trực tiếp. (Cửa kiểm chứng b.)

3.3 Nhiễu loạn: trái, phải, và vì sao phải chọn

A. Hai quy ước

Muốn đạo hàm một hàm của R , ta cần định nghĩa “thay đổi nhỏ của R ”. Có hai cách nhân một nhiễu loạn vào, và chúng không tương đương:

$$\text{trái: } R' = \exp([\delta\phi]_{\times}) R, \quad \text{phải: } R' = R \exp([\delta\phi]_{\times}).$$

Ý nghĩa vật lý khác nhau rõ rệt và đáng ghi nhớ theo cách này: *nhiều loạn trái là xoay thêm trong hệ toàn cục (hệ thế giới); nhiều loạn phải là xoay thêm trong hệ cục bộ (hệ thân)*. Với một camera, nhiều loạn phải tương ứng với “camera tự xoay đầu nó một chút”, còn nhiều loạn trái tương ứng với “cả thế giới xoay quanh camera”.

Hệ quả thực hành: một phép đo được biểu diễn tự nhiên trong hệ nào thì nên dùng nhiều loạn của hệ đó, vì Jacobian sẽ gọn hơn và — quan trọng hơn — vì hiệp phương sai thu được sẽ diễn giải được. Với con quay hồi chuyển, vốn đo trong hệ thân, nhiều loạn phải là tự nhiên. Với phần dư chiếu, cả hai đều dùng được và lựa chọn là thói quen thư viện: Sophus và Ceres thường dùng phải, nhiều tài liệu robot học dùng trái.

CẠM BẮY

Trộn hai quy ước trong cùng một hệ thống là nguồn lỗi dấu phổ biến nhất trong cả lĩnh vực, và nó là loại lỗi đặc biệt khó bắt.

Lý do khó bắt: hệ thống *vẫn hội tụ*. Gauss-Newton với Jacobian sai dấu ở một khối vẫn giảm hàm mục tiêu nếu các khối khác đúng, chỉ là chậm hơn và đi đường vòng. Bạn sẽ thấy hệ thống “hơi kém ổn định”, “cần nhiều vòng lặp hơn”, “đôi khi phân kỳ với chuyển động nhanh” — những triệu chứng dễ bị quy cho tinh chỉnh tham số.

Ba cách phòng, theo thứ tự hiệu quả. Một: **ghi quy ước ngay trong tên hàm** — `Jacobian_right`, `boxplus_left` — chứ không ghi trong comment, vì comment không được trình biên dịch kiểm. Hai: **kiểm mọi Jacobian bằng sai phân số** một lần khi viết, và giữ phép kiểm ấy lại như một unit test chạy mãi về sau; đây là cửa kiểm chứng (b) áp cho mã nguồn. Ba: khi phải đổi giữa hai quy ước, **dùng Adjoint** ở mục 4C thay vì dẫn lại bằng tay — vì dẫn lại bằng tay chính là chỗ sai xảy ra.

B. Đạo hàm cơ bản, tự dẫn được

Đây là đạo hàm nền của mọi thứ còn lại, và nó đáng tự dẫn lại vì nó chỉ mất ba dòng.

Cần tính $\partial(R\mathbf{p})/\partial\delta\phi$ với nhiều loạn trái. Thay vào và khai triển $\exp$ tới bậc nhất, dùng $\exp([\delta\phi]_{\times}) \approx I + [\delta\phi]_{\times}$:

$$\exp([\delta\phi]_{\times}) R\mathbf{p} \approx (I + [\delta\phi]_{\times}) R\mathbf{p} = R\mathbf{p} + \delta\phi \times (R\mathbf{p}) = R\mathbf{p} - [R\mathbf{p}]_{\times} \delta\phi.$$

Bước cuối dùng tính phản giao hoán của tích có hướng: $\mathbf{a} \times \mathbf{b} = -\mathbf{b} \times \mathbf{a}$. Vậy

$$\left. \frac{\partial(R\mathbf{p})}{\partial\delta\phi} \right|_{\text{trái}} = -[R\mathbf{p}]_{\times}. \quad (3.2)$$

Với nhiều loạn phải, làm tương tự cho $-R[\mathbf{p}]_{\times}$. Hai kết quả liên hệ qua $[R\mathbf{v}]_{\times} = R[\mathbf{v}]_{\times} R^{\top}$, một đồng nhất thức đáng thuộc vì nó xuất hiện liên tục.

Kiểm bằng số (cửa b): lấy $R = I$, $\mathbf{p} = (1, 0, 0)$, $\delta\phi = (0, 0, \varepsilon)$ với $\varepsilon = 10^{-4}$. Công thức (3.2) dự đoán thay đổi $-[\mathbf{p}]_{\times} \delta\phi = -(1, 0, 0) \times (0, 0, \varepsilon) \cdot (-1)$; tính trực tiếp $(1, 0, 0) \times (0, 0, \varepsilon) = (0 \cdot \varepsilon - 0 \cdot 0, 0 \cdot 0 - 1 \cdot \varepsilon, 0) = (0, -\varepsilon, 0)$, nên $-[\mathbf{p}]_{\times} \delta\phi = (0, \varepsilon, 0)$. Mặt khác xoay $(1, 0, 0)$ quanh z một góc ε cho $(\cos \varepsilon, \sin \varepsilon, 0) \approx (1, \varepsilon, 0)$, tức thay đổi $(0, \varepsilon, 0)$. Khớp.

3.4 $SE(3)$, $Sim(3)$ và Adjoint

A. $SE(3)$: xoay tịnh tiến

$$T = \begin{pmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix} \in SE(3), \quad \xi = \begin{pmatrix} \boldsymbol{\rho} \\ \phi \end{pmatrix} \in \mathbb{R}^6.$$

Sáu bậc tự do: ba xoay, ba tịnh tiến. Ánh xạ mũ có dạng đóng nhưng phần tịnh tiến không đơn giản là $\boldsymbol{\rho}$:

$$\exp(\xi^\wedge) = \begin{pmatrix} \exp([\phi]_\times) & J_l(\phi) \boldsymbol{\rho} \\ \mathbf{0}^\top & 1 \end{pmatrix},$$

với J_l là **Jacobian trái** của $SO(3)$ [24, §7.1.5]. Chỗ này hay gây bối rối, nên đáng một câu giải thích: khi camera vừa xoay vừa tịnh tiến *đồng thời và liên tục*, quỹ đạo là một đường xoắn ốc chứ không phải một đoạn thẳng, và J_l chính là hệ số hiệu chỉnh cho sự cong ấy. Với nhiễu loạn nhỏ, $J_l \approx I$ và ta bỏ qua được — đó là lý do nhiều cài đặt SLAM không bao giờ phải tính J_l một cách tường minh. Nó trở nên quan trọng khi cần exp chính xác trên một khoảng lớn, chẳng hạn trong tiền tích phân IMU (`./IMU_Fusion/` chương 7).

B. $Sim(3)$: thêm tỉ lệ

$$S = \begin{pmatrix} sR & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix}, \quad s > 0.$$

Bảy bậc tự do: ba xoay, ba tịnh tiến, một tỉ lệ. Nhóm này tồn tại trong SLAM vì một lý do rất cụ thể và rất thực dụng: trong SLAM đơn mắt, tỉ lệ là gauge tự do (A/01 mục 3B) và nó *trôi* theo thời gian. Khi đóng vòng — phát hiện rằng ta đã quay lại chỗ cũ — hai đầu của vòng nói chung có tỉ lệ khác nhau, và một phép biến đổi $SE(3)$ không thể ghép chúng lại. Cần $Sim(3)$ [26].

Điều đáng nhấn mạnh, vì nó là một hệ quả trực tiếp của luận đề trung tâm: *bậc tự do thứ bảy chỉ tồn tại khi tỉ lệ không quan sát được*. Với stereo, đường cơ sở cho ta mét; với VIO, gia tốc kế cho ta mét. Trong cả hai trường hợp, tỉ lệ là một đại lượng *quan sát được* chứ không phải một gauge, nên pose graph dùng $SE(3)$ — và với VIO, nơi trọng lực còn ghim thêm roll và pitch, nó rút xuống chỉ còn bốn bậc tự do (`./IMU_Fusion/` chương 5). Số bậc tự do của pose graph không phải một lựa chọn kỹ thuật mà là một hệ quả trực tiếp của việc cảm biến nào có mặt trong hệ.

C. Adjoint: chuyển nhiễu loạn giữa hai phía

Adjoint trả lời câu hỏi: một nhiễu loạn phải tương ứng với nhiễu loạn trái nào?

$$T \exp(\xi^\wedge) = \exp((\text{Ad}_T \xi)^\wedge) T. \quad (3.3)$$

Với $SE(3)$,

$$\text{Ad}_T = \begin{pmatrix} R & [\mathbf{t}]_\times R \\ \mathbf{0} & R \end{pmatrix}.$$

Công thức này tôi lấy từ [25] và [24, §7.1.7] (cửa c); tôi kiểm được trường hợp riêng $\mathbf{t} = 0$ cho $\text{Ad} = \text{diag}(R, R)$ bằng suy dẫn trực tiếp.

Adjoint có ba công dụng thực tế, và biết chúng tiết kiệm rất nhiều công. Một: *đổi quy ước* mà không dẫn lại Jacobian — viết một lần theo quy ước phải rồi chuyển bằng Ad. Hai: *lan truyền hiệp phương sai* qua một phép biến đổi cứng, $\Sigma' = \text{Ad}_T \Sigma \text{Ad}_T^\top$ — đây là điều ta cần ở A/06 khi chuyển bất định giữa các hệ quy chiếu. Ba: nó là thành phần của quy tắc chuỗi cho phép hợp thành hai tư thế, thứ mà pose graph ở B/11 dùng ở mọi cạnh.

3.5 Jacobian của phần dư chiếu: đích đến của chương

Đây là chỗ mọi thứ hội tụ, và là thứ nên tự dẫn lại được trên giấy.

Phần dư chiếu của landmark $\mathbf{P}_w$ quan sát ở khung với tư thế $T = (R, \mathbf{t})$:

$$\mathbf{r} = \mathbf{z} - \pi(R\mathbf{P}_w + \mathbf{t}), \quad \mathbf{r} \in \mathbb{R}^2.$$

Áp quy tắc chuỗi qua ba chặng, đúng như chuỗi ở A/02 mục 2A đọc ngược lại.

Chặng một — đạo hàm của phép chiếu. Với $\mathbf{P}_c = (X, Y, Z)$ và $\pi(\mathbf{P}_c) = (X/Z, Y/Z)$:

$$\frac{\partial \pi}{\partial \mathbf{P}_c} = \begin{pmatrix} 1/Z & 0 & -X/Z^2 \\ 0 & 1/Z & -Y/Z^2 \end{pmatrix} = \frac{1}{Z} \begin{pmatrix} 1 & 0 & -X/Z \\ 0 & 1 & -Y/Z \end{pmatrix}.$$

Dạng thứ hai đáng giữ vì nó cho thấy ngay hai điều: toàn bộ ma trận tỉ lệ với $1/Z$ — nên *điểm càng xa thì ảnh hưởng lên phần dư càng nhỏ*, đúng như trực giác; và các phần tử phụ thuộc vào $X/Z, Y/Z$ tức vị trí trên mặt phẳng chuẩn hoá, nên điểm ở rìa ảnh có Jacobian khác hẳn điểm ở tâm.

Chặng hai — đạo hàm của điểm camera theo nhiễu loạn tư thế. Dùng nhiễu loạn trái, $\mathbf{P}'_c = \exp(\boldsymbol{\xi}^\wedge) T \tilde{\mathbf{P}}_w$. Khai triển bậc nhất và dùng (3.2):

$$\frac{\partial \mathbf{P}_c}{\partial \boldsymbol{\xi}} = \begin{pmatrix} I_{3 \times 3} & -[\mathbf{P}_c]_\times \end{pmatrix} \in \mathbb{R}^{3 \times 6},$$

với thứ tự $\boldsymbol{\xi} = (\boldsymbol{\rho}, \phi)$. Khởi đầu là tịnh tiến — dịch camera một đoạn thì điểm trong hệ camera dịch đúng đoạn đó, nên đạo hàm là đơn vị. Khối sau là xoay, đúng công thức (3.2).

Kết quả:

$$\frac{\partial \mathbf{r}}{\partial \boldsymbol{\xi}} = -\frac{1}{Z} \begin{pmatrix} 1 & 0 & -X/Z \\ 0 & 1 & -Y/Z \end{pmatrix} \begin{pmatrix} I & -[\mathbf{P}_c]_\times \end{pmatrix} \in \mathbb{R}^{2 \times 6}. \quad (3.4)$$

Dấu trừ đến từ việc $\mathbf{r} = \mathbf{z} - \pi(\cdot)$. Với camera có méo, thêm một chặng $\partial d / \partial \mathbf{x}$ ở giữa; với camera có K không tầm thường, thêm $\partial \mathbf{p} / \partial \mathbf{x}_d$ — cả hai là ma trận 2×2 và không đổi cấu trúc của lập luận.

Tương tự, đạo hàm theo landmark là $\partial \mathbf{r} / \partial \mathbf{P}_w = -\frac{\partial \pi}{\partial \mathbf{P}_c} R \in \mathbb{R}^{2 \times 3}$.

CẦN NHỚ

Công thức (3.4) là khối xây dựng của **mọi** thứ ở A/04 và B/09. Ma trận Hessian của bundle adjustment, với hàng triệu phần tử khác không, được lắp từ các khối 2×6 và 2×3 này và không có gì khác.

Ba điều đáng đọc ra từ nó, và cả ba đều có hệ quả thực hành. Thứ nhất, **hệ số** $1/Z$: điểm ở xa đóng góp ít thông tin về tịnh tiến — đó là mục 6 của A/01 phát biểu lại bằng ngôn ngữ đạo hàm, và là lý do tham số hoá nghịch đảo độ sâu ở B/08 hợp lý hơn. Thứ hai, **cấu trúc** 2×6 : mỗi quan sát ràng buộc tư thế theo đúng hai chiều trong sáu, nên cần ít nhất ba điểm để xác định tư thế — đó là lý do P3P cần đúng ba điểm. Thứ ba, **khối** $[\mathbf{P}_c]_\times$: ảnh hưởng của xoay tỉ lệ với khoảng cách tới điểm, nên các điểm ở xa ràng buộc *xoay* rất tốt dù ràng buộc tịnh tiến rất kém.

Điều cuối cùng ấy là một trong những quan sát có giá trị nhất trong cả Phần A: nó nói rằng đừng vớt điểm ở xa đi, mà hãy để hiệp phương sai của chúng phản ánh đúng sự bất đối xứng ấy.

3.6 $\boxplus, \boxminus$ và vòng lặp tối ưu

Đặt mọi thứ lại với nhau. Định nghĩa hai toán tử nối đa tạp với không gian vectơ:

$$T \boxplus \xi = \exp(\xi^\wedge) T, \quad T_1 \boxminus T_2 = \log(T_1 T_2^{-1})^\vee.$$

(Với quy ước trái; quy ước phải đổi thứ tự nhân.) $\boxplus$ nhận một tư thế và một vectơ, trả về một tư thế; $\boxminus$ nhận hai tư thế, trả về một vectơ. Chúng thay thế $+$ và $-$ ở mọi chỗ mà thuật toán tối ưu chuẩn dùng hai phép đó.

Vòng lặp Gauss-Newton trên đa tạp trở thành:

1. Tính phần dư $\mathbf{r}(\boldsymbol{\theta}_k)$ tại ước lượng hiện tại.
2. Tính Jacobian $H = \partial \mathbf{r} / \partial \xi$ theo công thức (3.4) — đây là Jacobian theo nhiều loạn, không phải theo tham số.
3. Giải hệ chuẩn $H^\top \Omega H \delta = -H^\top \Omega \mathbf{r}$ ra $\delta \in \mathbb{R}^n$.
4. Cập nhật $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k \boxplus \delta$.

Bước 3 là đại số tuyến tính thuần túy trong không gian vectơ — mọi công cụ thừa ở A/04 dùng được nguyên vẹn. Bước 4 là chỗ duy nhất đa tạp xuất hiện. Đây chính là điều làm cách tiếp cận này thắng: nó cô lập toàn bộ độ phức tạp hình học vào một dòng mã.

Một hệ quả tinh tế đáng nêu vì nó liên quan tới A/05: vì Jacobian được tính tại điểm hiện tại và nhiều loạn được định nghĩa *quanh điểm đó*, hệ tọa độ của δ đổi sau mỗi vòng lặp. Điều này vô hại cho việc hội tụ nhưng có ý nghĩa cho tính nhất quán: khi ta marginalize một biến rồi sau đó tuyến tính hoá lại tại điểm khác, thông tin đã lưu không còn ở đúng hệ tọa độ nữa. Đó là gốc rễ của vấn đề nhất quán mà FEJ giải quyết, và cây `.. / IMU_Fusion/` chương 5 nói kỹ về nó.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Sophus Sophus	$SO(3)$, $SE(3)$, $Sim(3)$ với $\exp/\log$, Adjoint, Jacobian	Thư viện mặc định trong hệ sinh thái C++ SLAM; đọc mã S03: : $\exp$ để thấy cách xử lý góc nhỏ
manif manif	Cùng phạm vi, API hiện đại hơn, Jacobian tường minh cho mọi phép toán	Đi kèm [25], là tài liệu tham chiếu tốt nhất về quy ước
Ceres Manifold	Trừu tượng hoá $\boxplus/\text{retraction}$; tự động ghép với auto-diff	Mình hoạ trực tiếp mục 6 — bộ giải chỉ cần biết $\boxplus$, không cần biết gì về nhóm
GTSAM [27]	Kiểu Pose3 với $\text{retract}/\text{localCoordinates}$; Jacobian phân tích	Cùng ý tưởng, đặt trong khung đồ thị nhân tử
ORB-SLAM3 [10]	$Sim(3)$ cho loop closing mono, $SE(3)$ khi có stereo hoặc IMU	Xác nhận mục 4B: số bậc tự do phụ thuộc cảm biến, không phải lựa chọn tùy ý

MINI PROJECT SLAM · Tự cài $SE(3)$ và kiểm mọi Jacobian bằng sai phân số · 2 ngày

- Mục tiêu** biến chương này từ kiến thức đọc được thành kiến thức kiểm được, và tạo ra bộ unit test mà bạn sẽ dùng lại suốt phần còn lại của cây.
- Dữ liệu** không cần dữ liệu ngoài; mọi thứ tự sinh.
- Các bước** cài $\exp/\log$ cho $SO(3)$ và $SE(3)$ có xử lý góc nhỏ bằng Taylor; cài $\boxplus, \boxminus, \text{Ad}$; cài phần dư chiếu và Jacobian (3.4) theo cả hai quy ước trái và phải; viết một hàm kiểm sai phân số

trung tâm so sánh Jacobian phân tích với số, chạy trên vài trăm cấu hình ngẫu nhiên; đo sai lệch tương đối và tìm ngưỡng góc nhỏ mà dưới đó công thức Rodrigues trực tiếp mất độ chính xác.

Xong khi mọi Jacobian khớp sai phân số trong sai số tương đối 10^{-6} trở xuống; bạn xác nhận được bằng số rằng Jacobian trái và phải *khác nhau* nhưng liên hệ đúng qua Ad; và bạn có một con số cho ngưỡng góc nhỏ của chính cài đặt của mình — con số đó là một mục trong 99-chua-biet.md được trả lời.

Nối với [A/04-uoc-luong-map-va-bundle-adjustment](#) lắp các Jacobian này thành ma trận thưa; [C/14-hieu-chuan-nhu-mot-nhanh-cua-slam](#) thêm biến hiệu chuẩn vào cùng chuỗi quy tắc chuỗi.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Gauss-Newton đã đẩy R ra khỏi đa tạp $SO(3)$: phép cộng trong không gian chín chiều không giữ ràng buộc $R^T R = I$. Ma trận thu được không còn là phép xoay mà là một phép biến đổi tuyến tính có co giãn và trượt. Chuẩn hoá lại sau mỗi bước là cách chữa tồi vì ba lý do: bước đi mà bộ giải tính ra không còn là bước tối ưu sau khi bị chiếu về đa tạp; ma trận hiệp phương sai chín chiều mô tả một phân bố trong không gian mà bất định thật chỉ có ba chiều, nên nó suy biến; và bạn đang giải một bài toán chín biến khi bài toán chỉ có ba, làm hệ chuẩn thừa sáu hướng phẳng. Cách đúng là không bao giờ rời đa tạp: nhân nhiều loạn thay vì cộng.
- A2. Vì gimbal lock không phải khiếm khuyết của cách đánh số mà là phát biểu về topology: $SO(3)$ không đồng phôi với bất kỳ vùng mở nào của $\mathbb{R}^3$, nên *mọi* tham số hoá bằng ba số đều có kỳ dị ở đâu đó. Đối thứ tự trục chỉ dịch chỗ kỳ dị, không xoá được. Ở gần kỳ dị, Jacobian gần suy biến và bộ tối ưu mất một hướng — đúng kiểu hỏng im lặng ở [A/05](#). Cách thoát là dùng tham số hoá dư (ma trận hoặc quaternion) cho trạng thái nhưng tham số hoá *tối thiểu* cho nhiều loạn, vì nhiều loạn luôn ở gần không nên không bao giờ chạm kỳ dị.
- A3. Kết quả tối ưu *trùng nhau* — cùng một cực tiểu, vì hai quy ước chỉ là hai hệ toạ độ trên cùng một không gian tiếp tuyến. Phải chọn vì hai lý do. Một: trộn hai quy ước trong cùng hệ thống cho Jacobian sai dấu, và triệu chứng là hội tụ chậm hoặc đôi khi phân kỳ chứ không phải lỗi rõ ràng — rất khó bắt. Hai: *ma trận hiệp phương sai* thì khác nhau, vì nó được biểu diễn trong hệ toạ độ tương ứng; chuyển giữa hai hệ phải dùng Ad, và quên chuyển thì con số bất định sai mà không có dấu hiệu.
- A4. Bậc tự do thứ bảy là *tỉ lệ*, và nó tồn tại vì trong SLAM đơn mắt tỉ lệ là gauge tự do — camera đo hướng, nên nhân đôi cả thể giới không đổi ảnh nào ([A/01](#) mục 3B). Tỉ lệ ấy trôi theo thời gian, nên khi đóng vòng hai đầu có tỉ lệ khác nhau và $SE(3)$ không ghép được. Với stereo, đường cơ sở cho mét; với VIO, gia tốc kế cho mét — trong cả hai trường hợp tỉ lệ là đại lượng *quan sát được* chứ không phải gauge, nên $SE(3)$ đủ. Với VIO còn giảm tiếp xuống bốn bậc vì trọng lực ghim roll và pitch.
- A5. Là 6 vì đó là số bậc tự do của $SE(3)$, và Jacobian được tính theo *nhiều loạn* chứ không theo tham số — nhiều loạn sống trong đại số Lie sáu chiều, không phải trong mười sáu phần tử của ma trận thuần nhất. Con số đó nói rằng bộ giải nhìn bài toán ở đúng số chiều thật của nó: hệ chuẩn có đúng sáu ẩn cho mỗi tư thế, không có hướng phẳng giả tạo nào, và ma trận thông tin không suy biến vì lý do tham số hoá. Số 2 ở hàng nói mỗi quan sát ràng buộc tư thế theo đúng hai chiều, nên cần tối thiểu ba điểm — chính là lý do P3P cần ba.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng bạn đang đứng trên mặt đất và muốn đi tới một chỗ tốt hơn ở gần đó. Bạn nhìn xuống chân, thấy dốc nghiêng về hướng nào, rồi bước một bước theo hướng đó. Bây giờ nếu Trái Đất là một quả cầu, có một điều bạn *không* làm: bạn không cộng một vectơ vào toạ độ ba chiều của mình, vì làm thế sẽ đưa bạn xuống lòng đất hoặc lên trời. Bạn đi *dọc theo mặt đất*. Mặt đất quanh chân bạn phẳng như một tờ giấy, nên bạn nghĩ bằng hình học phẳng thông thường — rẽ trái bao nhiêu, đi bao xa — và điều đó hoàn toàn hợp lệ trong phạm vi vài bước chân.

Đó là toàn bộ chương này. Hướng của camera cũng sống trên một mặt cong như vậy. Bạn không được phép cộng thẳng vào nó. Bạn tính xem nên xoay thêm bao nhiêu quanh trục nào — ba con số, hoàn toàn bình thường, cộng trừ thoải mái — rồi *áp* phép xoay nhỏ đó vào hướng hiện tại. Kết quả chắc chắn vẫn là một hướng hợp lệ, vì xoay một hướng thì được một hướng.

Hai chi tiết còn lại cũng đơn giản như vậy. Thứ nhất, “tờ giấy” chỉ phẳng ở gần chân bạn, nên bước phải nhỏ — bước quá dài thì tính toán phẳng không còn đúng, và bộ giải phải tự hãm lại. Thứ hai, có hai cách hiểu “xoay thêm một chút”: xoay so với thế giới, hay xoay so với chính mình. Hai cách đều đúng và đều dùng được, nhưng chúng là hai ngôn ngữ khác nhau, và nói lẫn lộn hai ngôn ngữ trong cùng một câu thì ra câu vô nghĩa. Phần lớn lỗi dấu trong lĩnh vực này là đúng chuyện đó.

Chỗ không rút gọn được: chỗ không rút gọn được là vì sao công thức Rodrigues có đúng dạng ấy, và vì sao phần tịnh tiến của exp trên $SE(3)$ lại cần thêm hệ số J_1 . Hai thứ đó không có hình ảnh trực quan gọn gàng; chúng là kết quả của việc cộng dồn vô hạn các phép xoay nhỏ, và cách duy nhất để thật sự tin là tự dẫn lại chuỗi lũy thừa một lần.

Một câu: Không cộng vào hướng — xoay thêm một chút vào hướng; và “một chút” thì cộng trừ thoải mái được.

3.7 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *mọi tham số hoá $SO(3)$ bằng ba số đều có kỳ dị* — là một định lý topology, không phải một quan sát thực nghiệm. Nó bị bác bỏ nếu ai đó trình bày một tham số hoá ba số trơn, song ánh, không kỳ dị trên toàn $SO(3)$. Điều đó bất khả vì $SO(3)$ không đồng phôi với vùng mở nào của $\mathbb{R}^3$; tôi đối chiếu phát biểu này với [25] và [24] (cửa c) nhưng *không* tự chứng minh được phần topology, và đó là một khoản nợ được ghi trong 99-chua-biet.md.

Mệnh đề thứ hai — *Jacobian (3.4) đúng* — dễ kiểm nhất và mini project chính là phép kiểm: sai phân số phải khớp trong sai số 10^{-6} . Nếu không khớp, hoặc công thức sai hoặc quy ước bị trộn — và phép kiểm không phân biệt được hai khả năng đó, nên phải kiểm cả hai quy ước.

Mệnh đề thứ ba — *chọn quy ước không đổi kết quả tối ưu nhưng đổi hiệp phương sai* — bị bác bỏ nếu một thí nghiệm cho thấy hai quy ước hội tụ về hai cực tiểu khác nhau trên cùng dữ liệu và cùng khởi đầu. Tôi cho là không xảy ra với bài toán trơn và bước đủ nhỏ, nhưng tôi *ngờ* rằng với LM và damping mạnh, hai quy ước có thể đi hai đường khác nhau và rơi vào hai cực tiểu địa phương khác nhau trong bài toán không lồi. Đây là điều tôi chưa kiểm và chưa tìm được nguồn khẳng định (*tôi suy ra, chưa đối chiếu nguồn*).

3.8 Kết nối

Nối lên trên: [A/01-hình-hoc-da-thi](#) cho phép chiếu và [A/02-mo-hinh-camera](#) cho chuỗi đầy đủ tới pixel; chương này tính đạo hàm của đúng chuỗi ấy, nên ba chương hợp thành một khối và nên ôn cùng nhau.

Nối xuống dưới. [A/04-uoc-luong-map-va-bundle-adjustment](#) là nơi Jacobian (3.4) được lắp thành ma trận thưa và nơi vòng lặp ở mục 6 trở thành bundle adjustment thật. [A/05-observability-gauge-va-suy-bien](#) dùng ngôn ngữ nhiễu loạn để phát biểu phép “dịch cả thế giới, xoay cả thế giới” một cách chính xác — không có chương này thì lập luận gauge ở đó chỉ là hình ảnh. [A/06-bat-dinh-va-lan-truyen-sai-so](#) dùng Adjoint để lan truyền hiệp phương sai qua các phép biến đổi cứng, và nhấn mạnh rằng hiệp phương sai trên đa tạp phải hiểu là hiệp phương sai *của nhiễu loạn*, không phải của tham số.

Nối sang các phần sau. [B/09-paradigm-uoc-luong-back-end](#) dùng $Sim(3)$ cho pose graph mono và $SE(3)$ cho các trường hợp còn lại. [B/11-loop-closure-va-nhat-quan-toan-cuc](#) là ứng dụng rõ nhất của $Sim(3)$: hiệu chỉnh trôi tỉ lệ khi đóng vòng. [C/14-hieu-chuan-nhu-mot-nhanh-cua-slam](#) mở rộng quy tắc chuỗi ở mục 5 để bao gồm cả tham số nội và ngoại — và điều đáng chú ý là cấu trúc lập luận không đổi chút nào, chỉ dài thêm một chặng.

Nối ngang: ../IMU_Fusion/ chương 2 bàn cùng chủ đề với trọng tâm khác — nó đi sâu vào $SE_2(3)$ và tính chất group-affine, thứ cần cho bộ lọc bất biến nhưng không cần cho SLAM thuần thị giác. Nếu bạn cần InEKF thì đọc ở đó; chương này cố ý dừng ở $Sim(3)$.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao không thể cộng thẳng vào ma trận xoay, và vì sao “chuẩn hoá lại sau mỗi bước” không phải cách chữa đúng?
2. Tự dẫn lại $\partial(R\mathbf{p})/\partial\phi$ cho nhiễu loạn trái, không nhìn bài. Rồi giải thích dấu trừ đến từ đâu.
3. Nhiễu loạn trái và phải khác nhau về ý nghĩa vật lý thế nào, và tình huống nào khiến bạn buộc phải chọn một cái cụ thể?
4. Trong công thức (3.4), hệ số $1/Z$ nói gì về đóng góp của điểm xa, và khối $[\mathbf{P}_c]_{\times}$ nói gì về đóng góp của chúng cho thành phần xoay? Hai điều đó có mâu thuẫn nhau không?
5. Vì sao pose graph của SLAM mono cần $Sim(3)$ trong khi VIO chỉ cần bốn bậc tự do? Trả lời bằng ngôn ngữ quan sát được, không bằng ngôn ngữ cài đặt.
6. Adjoint làm gì, và nêu ba tình huống cụ thể mà không có nó bạn sẽ phải dẫn lại công thức bằng tay.
7. Ba giả thiết ngầm của cách tiếp cận nhiễu loạn là gì — kể cả cái không ai nói ra — và bỏ mỗi cái thì hỏng ở đâu?

Ước lượng MAP và bundle adjustment (MAP Estimation and Bundle Adjustment)

PHỤ THUỘC A/01-hình-học-da-thi (phép chiếu, triangulation — nguồn của phần dư); A/02-mô-hình-camera (mô hình đo đầy đủ); A/03-lie-group-va-toi-uu-tren-da-tap (Jacobian 2×6 và 2×3 , toán tử $\boxtimes$ — chương này lắp chúng thành ma trận). Chương tương ứng ở cây quản tính là `.. /IMU_Fusion/` chương 4.

TÓM TẮT

Đây là chương dựng khung mà luận đề trung tâm của cả cây phát biểu: *SLAM, SfM, hiệu chuẩn và localization là một bài toán MAP duy nhất, chỉ khác ở chỗ biến nào được fix*. Nội dung đi từ xác suất tới cài đặt qua bốn chặng. Chặng một: từ Bayes tới MAP tới bình phương tối thiểu có trọng số — một chuỗi tương đương mà mỗi bước dựa trên một giả thiết cụ thể, và biết rõ giả thiết nào ở đâu là điều kiện để biết khi nào khung này không dùng được. Chặng hai: đồ thị nhân tử như ngôn ngữ mô tả bài toán, và vì sao nó là ngôn ngữ đúng — nó làm cấu trúc thừa hiện ra thành hình vẽ. Chặng ba: cấu trúc thừa ấy và phân bù Schur, thứ biến một bài toán có độ phức tạp bậc ba không chạy nổi thành một bài toán chạy thời gian thực; đây là kỹ thuật quan trọng nhất trong chương xét về giá trị thực dụng. Chặng bốn: marginalization và hàm mất mát bền, hai thứ cần để bài toán sống được với dữ liệu thật và với thời gian vô hạn. Nếu chỉ nhớ một điều: *cấu trúc thừa không phải một tối ưu hoá, nó là lý do bài toán giải được* — và nó đến từ một sự kiện tầm thường, rằng mỗi quan sát chỉ chạm đúng một tư thế và một điểm.

KIẾN THỨC CẦN CÓ TRƯỚC

- **MAP (maximum a posteriori)** — ước lượng chọn giá trị biến làm cực đại xác suất hậu nghiệm $p(\theta | \mathbf{z})$. Khác với hợp lý cực đại (MLE) ở chỗ nó dùng cả tiên nghiệm $p(\theta)$; khi tiên nghiệm đều thì hai cái trùng nhau.
- **phần dư (residual)** — hiệu giữa phép đo thực tế và phép đo mà mô hình dự đoán, $\mathbf{r} = \mathbf{z} - h(\theta)$. Bài toán là làm cho tập phần dư nhỏ nhất theo một nghĩa có trọng số.
- **ma trận thông tin (information matrix)** — nghịch đảo của hiệp phương sai, $\Omega = \Sigma^{-1}$. Dùng làm trọng số: phép đo càng chính xác thì thông tin càng lớn nên càng được tin.
- **whitening** — phép biến đổi $\mathbf{r} \mapsto \Omega^{1/2} \mathbf{r}$ làm mọi phần dư có hiệp phương sai đơn vị, để tổng bình phương thường trở thành tổng bình phương có trọng số đúng. Sau whitening, mọi phần dư so sánh được với nhau bất kể đơn vị gốc.
- **đồ thị nhân tử (factor graph)** — đồ thị hai phía: một bên là biến, một bên là nhân tử (mỗi nhân tử là một số hạng của hàm mục tiêu), cạnh nối nhân tử với các biến nó chạm. Cấu trúc thừa của bài toán hiện ra trực tiếp thành cấu trúc đồ thị.
- **phần bù Schur (Schur complement)** — kỹ thuật khử một khối biến khỏi hệ tuyến tính bằng phép biến đổi đại số, thu được hệ nhỏ hơn chỉ chứa khối còn lại. Trong BA, ta khử các điểm để còn lại hệ chỉ chứa tư thế.
- **marginalization** — loại một biến khỏi bài toán theo nghĩa xác suất — tích phân nó ra — thay vì chỉ xoá nó. Khác biệt là marginalization *giữ lại* thông tin mà biến ấy mang, dưới dạng một nhân tử mới nối các biến còn lại.
- **M-estimator** — họ ước lượng thay bình phương r^2 bằng một hàm $\rho(r)$ tăng chậm hơn ở đuôi, để một phần dư rất lớn (outlier) không kéo nghiệm đi. Huber, Cauchy, Tukey là ba đại diện với ba mức quyết liệt khác nhau.

CÂU HỎI MỞ ĐẦU

- Q1. Bundle adjustment với 1000 tư thế và 100 000 điểm có hơn ba trăm nghìn ẩn. Phân tích Cholesky một ma trận đặc cỡ đó cần khoảng 10^{16} phép toán — không khả thi. Vậy vì sao nó chạy được trong vài giây, và “mẹo” nằm ở đâu?
- Q2. Bình phương tối thiểu là nghiệm MAP *chỉ khi* nhiều là Gauss. Nhưng nhiều trong SLAM không Gauss chút nào — khớp điểm sai tạo ra outlier với phần dư hàng trăm pixel. Vậy vì sao mọi người vẫn dùng bình phương tối thiểu, và cái giá là gì?
- Q3. Cửa sổ trượt phải bỏ bớt trạng thái cũ để giữ kích thước không đổi. Vì sao *xoá* một tư thế cũ khác hẳn *marginalize* nó, và điều gì bị mất khi chọn cách thứ nhất?
- Q4. Levenberg-Marquardt thêm một số hạng λI vào đường chéo Hessian. Giải thích ý nghĩa hình học của số hạng đó, và nói vì sao nó cứu được bộ giải ở chính những chỗ mà A/01 gọi là cấu hình suy biến.
- Q5. Marginalization biến một bài toán thừa thành ít thừa hơn — nó tạo ra fill-in. Vì sao điều đó xảy ra, và vì sao nó là lý do sâu xa khiến cửa sổ trượt không thể mở rộng vô hạn?

4.1 Đặt vấn đề

Ba chương trước cho ta mọi mảnh: hình học nối hai ảnh ràng buộc nhau thế nào, mô hình camera nối tia thành pixel ra sao, Lie group cho công cụ đạo hàm. Còn thiếu đúng một thứ — cách *ghép tất cả lại*.

Bài toán gốc phát biểu như sau. Ta có hàng nghìn khung hình, hàng trăm nghìn điểm bản đồ, hàng triệu quan sát. Mỗi quan sát nói một điều nhỏ và không chắc chắn: “landmark j xuất hiện ở pixel này trong khung i ”. Không quan sát nào đủ để xác định gì; mọi quan sát đều có nhiễu; và chúng *mâu thuẫn* nhau, theo nghĩa không tồn tại cấu hình nào thoả mãn tất cả một cách chính xác. Câu hỏi: cấu hình tư thế và cấu trúc nào là *tốt nhất*, và “tốt nhất” nghĩa là gì?

Ai gặp bài toán này? Nó vượt xa robot học. Trắc địa ảnh gọi nó là bundle adjustment và đã giải nó từ những năm 1950; thiên văn học giải bài toán cùng dạng khi khớp quỹ đạo; địa vật lý giải nó khi định vị tâm chấn. Thực tế đây là một trong những bài toán được nghiên cứu kỹ nhất trong toán ứng dụng, và Triggs cùng cộng sự [28] viết một tổng hợp vẫn là tài liệu tham chiếu chuẩn sau hơn hai mươi năm.

Trước khi có khung MAP, có hai cách tiếp cận và cả hai đều thiếu sót theo cách đáng học. Cách thứ nhất là *ghép cặp rồi nối chuỗi*: dùng hình học epipolar cho từng cặp khung liền kề, rồi nối các phép biến đổi tương đối lại. Nó đơn giản, chạy được, và hỏng vì sai số tích lũy đơn điệu — không có cơ chế nào để một quan sát muộn chỉnh lại ước lượng sớm. Đây chính là *visual odometry*, và nó hữu ích, nhưng nó không phải SLAM vì chữ M — mapping — đòi hỏi một bản đồ nhất quán toàn cục.

Cách thứ hai là *bộ lọc*: giữ một trạng thái và một hiệp phương sai, cập nhật tuần tự mỗi khi có phép đo mới. EKF-SLAM đi theo hướng này và thống trị lĩnh vực tới khoảng 2010. Nó hỏng vì hai lý do mà chương 9 sẽ nói kỹ: chi phí bậc hai theo số landmark, và bất nhất do tuyến tính hoá một lần rồi không bao giờ sửa lại.

Khung MAP giải quyết cả hai, và nó làm thế bằng cách phát biểu bài toán đúng: *tìm cấu hình biến làm cực đại xác suất hậu nghiệm, cho toàn bộ dữ liệu cùng lúc*. Không có thứ tự thời gian đặc quyền, không có thông tin nào bị vứt đi sau khi dùng, và mọi phép đo đều được tuyến tính hoá lại tại ước lượng tốt nhất hiện có. Cái giá là chi phí tính toán — và toàn bộ mục 5 của chương dành cho việc cái giá ấy hoá ra thấp hơn nhiều so với vẻ ngoài, nhờ cấu trúc thưa.

Có một lý do thứ hai để chương này quan trọng, và nó là lý do chính đáng nhất. Khung MAP là chỗ luận đề trung tâm của cây được phát biểu chính xác. Một khi bài toán đã ở dạng “cực tiểu

hoá tổng phần dư có trọng số trên một tập biến”, thì việc phân biệt SLAM với hiệu chuẩn với localization rút xuống thành một câu hỏi duy nhất: *tập biến gồm những gì, và cái nào bị fix?* Bảng ở 00-map là bảng đó, và chương này là chỗ nó có nghĩa toán học.

TRỰC GIÁC

Hình dung mỗi phép đo là một cái lò xo. Một quan sát “landmark j ở pixel này trong khung i ” là một lò xo nối hai vật: tư thế khung i và vị trí điểm j . Lò xo có độ cứng tỉ lệ với độ tin cậy của phép đo — đo chính xác thì lò xo cứng.

Bài toán là tìm vị trí cân bằng của cả hệ. Không lò xo nào được thoả mãn hoàn toàn, nhưng tổng năng lượng đàn hồi đạt cực tiểu ở đúng một cấu hình, và cấu hình đó là nghiệm MAP. Ba điều đọc được ngay từ hình ảnh này, và cả ba là nội dung của ba mục sau.

Thứ nhất, *độ thưa*: mỗi lò xo chỉ nối hai vật, không nối tất cả. Với hàng triệu lò xo và hàng trăm nghìn vật, ma trận mô tả hệ gần như toàn số không — và đó là thứ làm bài toán giải được.

Thứ hai, *hướng phẳng*: nếu có một cách dịch chuyển cả hệ mà không lò xo nào căng thêm, thì hệ có vô số vị trí cân bằng. Đó là gauge, và A/05 nói về nó.

Thứ ba, *outlier*: một lò xo bị lắp sai chỗ — nối tư thế i với một điểm không phải điểm nó nhìn thấy — sẽ kéo cả hệ lệch đi. Với lò xo tuyến tính, một lò xo lắp sai và căng rất mạnh có thể thắng hàng trăm lò xo đúng. Mục 6 là về việc làm cho lò xo *mềm dần khi căng quá mức*, để một cái lắp sai không phá cả hệ.

4.2 Từ Bayes tới bình phương tối thiểu

Chuỗi tương đương này ngắn nhưng mỗi bước dựa trên một giả thiết, và biết giả thiết nào ở đâu là điều phân biệt người dùng công cụ với người hiểu công cụ.

A. Ba bước

Theo Bayes, hậu nghiệm tỉ lệ với tích của hợp lý và tiên nghiệm:

$$p(\boldsymbol{\theta} \mid \mathbf{z}) \propto p(\mathbf{z} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta}).$$

Giả thiết thứ nhất nằm ở bước tiếp theo: các phép đo *độc lập* với nhau khi đã biết $\boldsymbol{\theta}$, nên hợp lý tách thành tích:

$$p(\mathbf{z} \mid \boldsymbol{\theta}) = \prod_k p(\mathbf{z}_k \mid \boldsymbol{\theta}).$$

Giả thiết này đáng để ý vì nó bị vi phạm thường xuyên hơn người ta nghĩ: hai đặc trưng cạnh nhau trên cùng một góc ảnh chịu chung một sai số hiệu chuẩn cục bộ, nên sai số của chúng tương quan. Hệ quả là bài toán *đếm thừa* thông tin và hiệp phương sai đầu ra quá lạc quan — một trong các nguồn của bệnh quá tự tin ở A/06.

Lấy $-\log$ biến tích thành tổng và biến cực đại thành cực tiểu:

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \left[- \sum_k \log p(\mathbf{z}_k \mid \boldsymbol{\theta}) - \log p(\boldsymbol{\theta}) \right].$$

Giả thiết thứ hai: nhiễu Gauss, $\mathbf{z}_k = h_k(\boldsymbol{\theta}) + \boldsymbol{\varepsilon}_k$ với $\boldsymbol{\varepsilon}_k \sim \mathcal{N}(\mathbf{0}, \Sigma_k)$. Khi đó $-\log p(\mathbf{z}_k \mid \boldsymbol{\theta})$ bằng $\frac{1}{2} \mathbf{r}_k^\top \Sigma_k^{-1} \mathbf{r}_k$ cộng một hằng số không phụ thuộc $\boldsymbol{\theta}$, và ta được

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \frac{1}{2} \sum_k \mathbf{r}_k(\boldsymbol{\theta})^\top \Omega_k \mathbf{r}_k(\boldsymbol{\theta}), \quad \Omega_k = \Sigma_k^{-1}. \quad (4.1)$$

Đây là bài toán bình phương tối thiểu phi tuyến có trọng số. Tôi tự dẫn lại toàn bộ (cửa a) và đối chiếu [24, §4] cùng [29, §11] (cửa c).

CẦN NHỚ

Chuỗi tương đương này dựa trên **hai giả thiết**, và cả hai đều bị vi phạm trong SLAM thật. Biết chúng ở đâu là biết khi nào khung này nói dối.

Độc lập bị vi phạm khi các phép đo chia nhau một nguồn sai số chung — sai số hiệu chuẩn, biến dạng cục bộ của ống kính, một đối tượng động mà nhiều đặc trưng cùng bám vào. Hệ quả: thông tin bị đếm thừa, hiệp phương sai đầu ra quá nhỏ.

Gauss bị vi phạm mạnh hơn nhiều, và theo hướng nguy hiểm hơn. Khớp điểm sai không tạo ra một phần dư “hơi lớn” mà tạo ra một phần dư *hàng trăm pixel*. Phân bố thật có đuôi rất nặng, trong khi Gauss giả định đuôi mỏng theo hàm mũ. Vì hàm mục tiêu bình phương, một phần dư gấp 100 lần đóng góp gấp 10 000 lần — *một outlier có thể thắng mười nghìn* quan sát đúng.

Đây là lý do mục 6 tồn tại, và là lý do không hệ SLAM thật nào dùng bình phương tối thiểu thuần túy. Nhưng cần nói rõ một điều: hàm mất mát bền *không* sửa giả thiết Gauss cho đúng — nó chỉ hạn chế thiệt hại. Ước lượng thu được không còn là MAP của mô hình nào cả, trừ khi ta diễn giải nó là MAP của một mô hình nhiễu có đuôi nặng, và cách diễn giải ấy chính là đối ngẫu Black-Rangarajan [30].

B. Gauss-Newton và hệ chuẩn

Bài toán (4.1) phi tuyến vì h_k phi tuyến. Gauss-Newton giải bằng cách tuyến tính hoá phần dư quanh ước lượng hiện tại. Với δ là nhiễu loạn theo nghĩa A/03:

$$\mathbf{r}_k(\boldsymbol{\theta} \boxplus \boldsymbol{\delta}) \approx \mathbf{r}_k(\boldsymbol{\theta}) + H_k \boldsymbol{\delta}, \quad H_k = \frac{\partial \mathbf{r}_k}{\partial \boldsymbol{\delta}}.$$

Thay vào và lấy đạo hàm theo $\boldsymbol{\delta}$ rồi cho bằng không:

$$\underbrace{\left(\sum_k H_k^\top \Omega_k H_k \right)}_{\Lambda} \boldsymbol{\delta} = - \sum_k H_k^\top \Omega_k \mathbf{r}_k. \quad (4.2)$$

Đây là **hệ chuẩn**, và Λ là ma trận thông tin của bài toán. Ba điều đáng ghi. Thứ nhất, Λ là *xấp xỉ* Hessian — Hessian thật còn một số hạng chứa đạo hàm bậc hai của h_k , và Gauss-Newton bỏ nó đi; xấp xỉ này tốt khi phần dư nhỏ, và tệ khi phần dư lớn, tức chính lúc có outlier. Thứ hai, Λ luôn nửa xác định dương vì nó là tổng các $H_k^\top \Omega_k H_k$; nó xác định dương *trừ khi* có hướng $\mathbf{u}$ với $H_k \mathbf{u} = 0$ cho mọi k — đó chính xác là định nghĩa hướng không quan sát được ở A/05. Thứ ba, Λ là nghịch đảo của hiệp phương sai nghiệm, nội dung của A/06.

Levenberg–Marquardt thêm damping: $(\Lambda + \lambda D) \boldsymbol{\delta} = -\mathbf{g}$, với D thường là $\text{diag}(\Lambda)$. Ý nghĩa hình học đáng nói rõ vì nó trả lời câu hỏi mở đầu thứ tư. Khi $\lambda \rightarrow 0$ ta có Gauss-Newton thuần, bước dài và nhanh nhưng chỉ đúng khi xấp xỉ tuyến tính đúng. Khi $\lambda \rightarrow \infty$ bước tiến về hướng gradient âm và rất ngắn — chậm nhưng an toàn. Quan trọng hơn cả: λD làm *mọi* hướng có độ cong dương, kể cả hướng mà Λ suy biến. Nghĩa là LM *vẫn chạy* trong cấu hình suy biến — nó trả về bước đi ngắn theo hướng không quan sát được thay vì nổ. Đây là tin tốt cho độ ổn định và là tin xấu cho chẩn đoán: nó biến một lỗi ồn ào thành một lỗi im lặng, đúng như A/05 cảnh báo.

I 4.3 Đồ thị nhân tử: ngôn ngữ đúng để mô tả bài toán

Hệ chuẩn (4.2) đúng nhưng nó giấu mất cấu trúc. Đồ thị nhân tử là cách viết cùng bài toán sao cho cấu trúc hiện ra.

Đồ thị có hai loại nút. *Nút biến* là thứ ta muốn ước lượng: tư thế, điểm bản đồ, tham số hiệu chuẩn, độ lệch IMU. *Nút nhân tử* là một số hạng của hàm mục tiêu; mỗi nhân tử nối với đúng những biến mà phần dư của nó phụ thuộc vào.

Trong bundle adjustment thuần thị giác, mỗi quan sát “điểm j thấy ở khung i ” là một nhân tử nối đúng hai nút: tư thế T_i và điểm P_j . Không nối gì khác. Đây là sự kiện quan trọng nhất trong cả chương, và mục 5 sẽ khai thác nó tới tận cùng.

Giá trị của cách biểu diễn này có bốn mặt, và mặt thứ tư là mặt quan trọng nhất cho cây này.

Một — độ thưa hiện ra thành hình. Bậc của một nút biến bằng số khối khác không trong hàng tương ứng của Λ . Nhìn đồ thị là biết ma trận thưa đến đâu.

Hai — module hoá. Thêm một loại cảm biến nghĩa là thêm một loại nhân tử, không phải viết lại bộ giải. Đây là lý do thư viện như GTSAM [27] tổ chức quanh khái niệm này, và là lý do C/15 có thể nói về hợp nhất đa cảm biến bằng một câu: *thêm factor vào graph, với hiệp phương sai đúng và công loại outlier riêng cho từng nguồn*.

Ba — thuật toán đồ thị dùng được. Thứ tự khử biến để giảm fill-in là một bài toán đồ thị (A/04 mục 5D); tính liên thông của đồ thị liên quan trực tiếp tới observability.

Bốn — luận đề trung tâm trở nên hiển nhiên. Khi bài toán đã là một đồ thị, “fix một biến” đơn giản là không đặt nó vào tập biến, và “thả một biến” là đặt nó vào. Hiệu chuẩn ngoại tuyến, SLAM, VIO có hiệu chuẩn trực tuyến — ba đồ thị có cùng loại nhân tử, khác nhau ở danh sách nút biến. Không có gì trong bộ giải phải thay đổi.

Bảng 4.1. Cùng một đồ thị nhân tử, bốn bài toán. Chỉ cột “nút biến” thay đổi; loại nhân tử và bộ giải giữ nguyên. Đây là luận đề trung tâm của cây viết ra dưới dạng bảng.

Bài toán	Nút biến	Nhân tử	Hằng số
Hiệu chuẩn từ target	Tư thế bảng, nội, méo	Chiều góc bảng	Toạ độ 3D góc bảng
Bundle adjustment	Tư thế, điểm	Chiều	Nội, méo
Localization	Tư thế khung hiện tại	Chiều	Nội, méo, toàn bộ bản đồ
VIO + calib trực tuyến	Tư thế, vận tốc, độ lệch, điểm, nội, ngoại, t_d	Chiều, tiền tích phân IMU	Không gì

4.4 Độ thưa và phần bù Schur

Đây là mục có giá trị thực dụng cao nhất của chương, và là câu trả lời cho câu hỏi mở đầu thứ nhất.

A. Vì sao bài toán trông không giải được

Với m tư thế và n điểm, số ẩn là $6m + 3n$. Với $m = 1000$ và $n = 100\,000$, đó là 306\,000 ẩn. Phân tích Cholesky một ma trận đặc cỡ N tốn khoảng $N^3/3$ phép toán; với $N = 3 \times 10^5$ đó là cỡ 10^{16}

phép toán, tức nhiều giờ hoặc nhiều ngày trên CPU thông thường. Bộ nhớ còn tệ hơn: ma trận đặc cỡ đó cần khoảng 750 GB ở độ chính xác kép.

Kết luận nếu dừng ở đây: bundle adjustment không khả thi. Nhưng nó chạy trong vài giây, nên có gì đó sai trong lập luận — và cái sai là giả định ma trận đặc.

B. Cấu trúc thưa đến từ đâu

Quay lại sự kiện ở mục 3: mỗi quan sát chỉ chạm *một* tư thế và *một* điểm. Hệ quả trực tiếp lên Λ : khối ứng với cặp (tư thế i , tư thế i') bằng không khi $i \neq i'$ — không quan sát nào chạm hai tư thế cùng lúc. Khối ứng với cặp (điểm j , điểm j') cũng bằng không khi $j \neq j'$, vì lý do tương tự.

Viết Λ theo khối với thứ tự (tư thế trước, điểm sau):

$$\Lambda = \begin{pmatrix} U & W \\ W^\top & V \end{pmatrix},$$

trong đó U cỡ $6m \times 6m$ là khối chéo với các khối 6×6 ; V cỡ $3n \times 3n$ là khối chéo với các khối 3×3 ; và W cỡ $6m \times 3n$ thưa, chỉ khác không ở cặp (i, j) mà khung i thật sự thấy điểm j .

Tính chất quyết định là V *khối chéo* với khối 3×3 , nên nghịch đảo của nó là nghịch đảo từng khối 3×3 độc lập — chi phí tuyến tính theo n và song song hoá được hoàn hảo. Đây là món quà mà cấu trúc bài toán tặng, và toàn bộ mục này là việc nhận món quà đó.

C. Phần bù Schur

Viết hệ chuẩn theo khối, với δ_c là nhiễu loạn tư thế và δ_p là nhiễu loạn điểm:

$$\begin{pmatrix} U & W \\ W^\top & V \end{pmatrix} \begin{pmatrix} \delta_c \\ \delta_p \end{pmatrix} = \begin{pmatrix} \mathbf{g}_c \\ \mathbf{g}_p \end{pmatrix}.$$

Nhân hàng dưới với WV^{-1} rồi trừ khỏi hàng trên, khối W triệt tiêu:

$$(U - WV^{-1}W^\top) \delta_c = \mathbf{g}_c - WV^{-1}\mathbf{g}_p. \quad (4.3)$$

Ma trận $S = U - WV^{-1}W^\top$ là **phần bù Schur**, cỡ $6m \times 6m$ — chỉ phụ thuộc số tư thế, hoàn toàn không phụ thuộc số điểm. Giải (4.3) ra δ_c , rồi thế ngược lấy $\delta_p = V^{-1}(\mathbf{g}_p - W^\top \delta_c)$ — bước thế ngược cũng tuyến tính theo n .

Đếm lại chi phí với $m = 1000$, $n = 100\,000$. Nghịch đảo V : 100 000 phép nghịch đảo ma trận 3×3 , rẻ và song song. Dựng S : cỡ 6000×6000 , tốn công nhưng S cũng thưa — nó chỉ khác không ở cặp tư thế *cùng nhìn thấy ít nhất một điểm chung*. Đây là chỗ khái niệm **đồ thị đồng quan sát** (covisibility graph) ở C/13 có nguồn gốc toán học: nó *chính là* cấu trúc khác không của S . Giải hệ 6000×6000 thưa: trong tầm vài trăm mili giây.

CẦN NHỚ

Phần bù Schur đưa chi phí từ khoảng $O((6m + 3n)^3)$ xuống khoảng $O(m^3) + O(n)$ khi $n \gg m$ — và trong SLAM thì n luôn lớn hơn m hàng trăm lần. Đây **không** phải một tối ưu hoá thêm vào; nó là lý do bundle adjustment tồn tại được như một kỹ thuật thực dụng [28, §6].

Ba hệ quả đáng nhớ, và cả ba xuất hiện lại ở các chương sau.

Một: ma trận S chính là đồ thị đồng quan sát. Hai tư thế nối với nhau trong S đúng khi chúng cùng thấy một điểm. Khái niệm covisibility ở C/13 không phải một heuristic mà là cấu trúc khác không của một ma trận.

Hai: số điểm gần như miễn phí, số tư thế thì không. Điều này định hình toàn bộ kiến trúc SLAM: người ta giữ hàng trăm nghìn landmark nhưng rất tiết kiệm keyframe. Chính sách chọn và loại keyframe ở C/13 có gốc rễ ở đây.

Ba: kỹ thuật này chỉ hoạt động khi V khối chéo. Nếu có một nhân tử nối hai điểm với nhau — chẳng hạn ràng buộc rằng hai điểm nằm trên cùng một mặt phẳng — thì V không còn khối chéo và toàn bộ lợi thế biến mất. Đây là cái giá ít được nói tới của các ràng buộc cấu trúc ở B/08.

D. Thứ tự khử và fill-in

Giải hệ thừa (4.3) bằng Cholesky, nhưng Cholesky sinh ra **fill-in**: các phần tử vốn bằng không trở thành khác không trong thừa số. Lượng fill-in phụ thuộc mạnh vào thứ tự khử biến, và chênh lệch giữa thứ tự tốt và tệ có thể là nhiều bậc độ lớn.

Tìm thứ tự tối ưu là bài toán NP-khó, nên người ta dùng heuristic. **AMD** (approximate minimum degree) và **COLAMD** là hai lựa chọn chuẩn; mọi thư viện thừa tử tế đều cài sẵn. Điểm cần nhớ không phải chi tiết thuật toán mà là: thứ tự biến là một lựa chọn có hậu quả lớn, và mặc định của thư viện thường không tối ưu cho bài toán có cấu trúc đặc biệt như SLAM. Với bài toán có cấu trúc thời gian rõ ràng — tư thế nối tiếp nhau — thứ tự theo thời gian thường tốt hơn thứ tự tổng quát, và đó là một trong những lý do iSAM2 [31] nhanh: nó khai thác cấu trúc ấy một cách tường minh qua Bayes tree.

4.5 Marginalization: sống với thời gian vô hạn

Full batch BA giải mọi thứ cùng lúc, và nó là nghiệm tốt nhất có thể. Nó cũng bất khả thi cho hệ chạy liên tục: số tư thế tăng vô hạn. Cửa sổ trượt giữ kích thước cố định bằng cách loại bớt trạng thái cũ, và cách loại là chỗ quyết định.

A. Xoá so với marginalize

Xoá một tư thế cũ nghĩa là vứt nó cùng mọi nhân tử chạm nó. Rẻ, đơn giản, và sai: thông tin mà các quan sát ấy mang về các biến còn lại bị vứt theo. Hệ quả là ước lượng kém chính xác hơn và — nghiêm trọng hơn — hiệp phương sai bị phóng đại, vì bài toán quên rằng nó từng biết nhiều hơn.

Marginalize nghĩa là tích phân biến ấy ra khỏi phân bố hậu nghiệm. Với phân bố Gauss, phép tích phân này có dạng đóng và nó chính là phần bù Schur — cùng một công thức (4.3), dùng cho mục đích khác. Kết quả là một **nhân tử prior** mới nối tất cả các biến từng nối với biến bị loại.

Sự trùng nhau giữa hai phép toán đáng dừng lại một câu, vì nó không phải trùng hợp: cả hai đều là “khử một khối biến khỏi hệ tuyến tính trong khi giữ nguyên ảnh hưởng của nó lên phần còn lại”. Trong BA ta khử điểm để giải nhanh rồi thế ngược lấy lại chúng; trong marginalization ta khử tư thế cũ và không lấy lại. Khác biệt duy nhất là ta có quay lại hay không.

B. Cái giá: fill-in

Marginalization có một cái giá mà mọi hệ cửa sổ trượt phải trả, và nó trả lời câu hỏi mở đầu thứ năm.

Khi khử một tư thế nối với k landmark, nhân tử prior mới nối *tất cả* k landmark đó với nhau. Một nút bậc k biến thành một khối đặc $k \times k$. Đồ thị vốn thưa trở nên đặc dần ở vùng biên của cửa sổ, và ma trận thông tin mất cấu trúc khối chéo mà mục 4 dựa vào.

Đây là lý do *sâu xa* khiến cửa sổ trượt không mở rộng vô hạn được: không phải vì số biến lớn, mà vì chi phí của khối prior đặc tăng theo bình phương số biến còn lại nối với nó. Hai cách xử lý trong thực tế. Cách thứ nhất: *chọn khử cái gì*. VINS-Mono marginalize khung cũ nhất khi khung mới là keyframe, và marginalize khung mới thứ hai khi không phải — lựa chọn đó giữ cho prior không nối quá nhiều biến [13]. Cách thứ hai: *làm thưa lại* prior bằng cách xấp xỉ khối đặc bằng một cấu trúc thưa hơn, chấp nhận mất một ít thông tin để đổi lấy chi phí.

C. Cái bẫy nhất quán

Còn một cái giá thứ hai, tinh vi hơn và tổn kém hơn. Nhân tử prior được tính bằng cách tuyến tính hoá tại ước lượng *lúc marginalize*. Các biến nó nối tiếp tục được tối ưu và di chuyển. Nhưng prior thì đóng băng — nó vẫn mang Jacobian của một điểm tuyến tính hoá cũ.

Hậu quả: khi cùng một biến được tuyến tính hoá ở hai điểm khác nhau trong hai phần khác nhau của bài toán, hệ thống có thể *tự tạo ra thông tin* theo một hướng đáng lẽ không quan sát được. Thông tin ấy không đến từ phép đo nào cả; nó là hiện vật của việc tuyến tính hoá không nhất quán. Hệ quả là hiệp phương sai co lại một cách sai trái, tức bộ ước lượng trở nên quá tự tin [32].

Cách chữa chuẩn là FEJ (First-Estimates Jacobian): cố định điểm tuyến tính hoá của các biến bị chạm bởi prior. Cây `.. /IMU_Fusion/` chương 5 và chương 8 nói kỹ về nó; ở đây điều cần nhớ là *marginalization không miễn phí về mặt nhất quán*, và một hệ cửa sổ trượt không xử lý chuyện này sẽ báo cáo hiệp phương sai nhỏ hơn sự thật một cách có hệ thống.

I 4.6 Hàm mất mát bền: sống với dữ liệu thật

A. Vì sao bình phương tối thiểu vỡ

Từ khối *Cần nhớ* ở mục 2: hàm mục tiêu bình phương làm một phần dư gấp 100 lần đóng góp gấp 10 000 lần. Một khớp điểm sai — và với bất kỳ front-end nào, tỉ lệ khớp sai vài phần trăm là chuyện bình thường — có thể kéo nghiệm đi xa tùy ý.

Có hai tuyến phòng thủ và chúng bổ sung nhau chứ không thay thế nhau. Tuyến một là *loại bỏ trước khi tối ưu*: RANSAC [33] và các biến thể ở B/07. Tuyến hai là *chịu đựng trong khi tối ưu*: hàm mất mát bền, nội dung của mục này. Cần cả hai vì RANSAC không bắt hết — đặc biệt là các outlier “vừa phải”, có phần dư đủ nhỏ để lọt ngưỡng nhưng đủ lớn để gây hại.

B. M-estimator và IRLS

Thay $\frac{1}{2}r^2$ bằng $\rho(r)$ tăng chậm hơn ở đuôi. Ba lựa chọn chuẩn:

Huber: bậc hai cho $|r| \leq \delta$, tuyến tính ngoài đó. Lỗi, nên không tạo cực tiểu địa phương mới — tính chất này quan trọng và là lý do Huber là mặc định an toàn.

Cauchy: $\rho(r) = \frac{c^2}{2} \log(1 + r^2/c^2)$. Không lỗi, giảm ảnh hưởng outlier mạnh hơn, nhưng có thể tạo cực tiểu địa phương.

Tukey: bão hoà hoàn toàn — phần dư quá ngưỡng đóng góp *đúng bằng không*. Quyết liệt nhất, và đòi khởi tạo tốt nhất.

Cài đặt qua **IRLS** (iteratively reweighted least squares): ở mỗi vòng lặp, gán trọng số $w_k = \rho'(|r_k|)/|r_k|$ rồi giải bài toán bình phương tối thiểu có trọng số thông thường. Với Huber, $w_k = \min(1, \delta/|r_k|)$.

Ví dụ số (cửa b). Giả sử $\delta = 1,5$ px (whitened). Một phần dư $|r| = 1$ px cho $w = 1$ — không bị hạ trọng số. Một phần dư $|r| = 15$ px cho $w = 0,1$. Đóng góp vào hệ chuẩn tỉ lệ với $w r^2$: với bình phương thuần, tỉ số đóng góp giữa hai phần dư là $15^2/1^2 = 225$; với Huber, là $0,1 \times 225 = 22,5$. Ảnh hưởng của outlier giảm mười lần, đúng bằng hệ số $\delta/|r|$. Cấu trúc thưa hoàn toàn không đổi vì trọng số chỉ nhân vào từng khối — đây là lý do M-estimator gần như miễn phí về mặt tính toán, và là lý do không có cơ gì để không dùng nó.

C. GNC: khi khởi tạo quá tệ

Hàm không lồi như Tukey chỉ hoạt động nếu bắt đầu đủ gần nghiệm đúng. Khi không có điều đó — chẳng hạn với loop closure sai ở B/11, nơi một cạnh sai có thể làm gập cả bản đồ — cần một chiến lược khác.

GNC (graduated non-convexity) [34] bắt đầu bằng một hàm mất mát gần lồi rồi *từ từ* làm nó không lồi hơn qua các vòng lặp. Trực giác: giải bài toán dễ trước để có khởi tạo tốt, rồi dùng nó làm khởi tạo cho bài toán khó hơn một chút, lặp lại. Nền lý thuyết là đối ngẫu Black–Rangarajan [30], cho thấy tối ưu với M-estimator tương đương với tối ưu đồng thời trên biến gốc và một tập biến “trọng số outlier” — nên GNC là một dạng làm mềm dần bài toán gán nhãn inlier/outlier.

Giá trị thực dụng của GNC cao nhất ở pose graph có loop closure sai, và đó là lý do nó xuất hiện lại ở B/11 như một trong các kỹ thuật back-end bền.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Ceres [35]	Schur complement solver, ordering tự động, LossFunction (Huber/Cauchy/Tukey)	Cài đặt tham chiếu của toàn chương; đọc tài liệu phần SPARSE_SCHUR
g2o [36]	Đồ thị nhân tử tường minh, Schur cho cấu trúc BA	Mã ngắn, dễ đọc cấu trúc khối U, W, V ở mục 4B
GTSAM [27]	iSAM2 [31], Bayes tree, khử biến gia tăng	Minh hoạ mục 4D: khai thác thứ tự thời gian để giảm fill-in
ORB-SLAM3 [10]	Local BA trên cửa sổ đồng quan sát; Huber cho mọi cạnh chiếu	S chính là đồ thị đồng quan sát — mục 4C thành kiến trúc
VINS-Mono [13]	Marginalization có chọn lọc khung cũ nhất hay khung mới thứ hai	Mục 5B: chiến lược chọn khử để kiểm soát fill-in
Basalt [37]	Square-root BA — phân tích QR thay vì hệ chuẩn	Ổn định số học tốt hơn; số điều kiện bình phương của Λ là lý do

MINI PROJECT SLAM • Tự viết BA thưa, rồi đo chính xác lợi ích của Schur • 4 ngày

- Mục tiêu** biến “Schur làm nó nhanh hơn” từ một câu nghe được thành một đường cong bạn tự đo.
- Dữ liệu** tự sinh: một quỹ đạo, một đám mây điểm, các quan sát có nhiễu Gauss đã biết σ . Tự sinh là bắt buộc ở bài này vì bạn cần chân trị để đo sai số.
- Các bước** dựng bài toán BA với Jacobian từ A/03; cài ba bộ giải trên cùng bài toán — Cholesky đặc, Cholesky thưa không Schur, và Schur + Cholesky thưa; đo thời gian và bộ nhớ theo n từ 100 tới 100 000 điểm với m cố định, rồi theo m với n cố định; thêm 5% outlier và chạy lại với bình phương thuần rồi với Huber; cuối cùng cài marginalization và đo fill-in của khối prior theo số lần khử.

- Xong khi** bạn có ba đường cong thời gian theo n trên trục log-log, và độ dốc của chúng khớp với phân tích ở mục 4C: đặc gần bậc ba, Schur gần tuyến tính. Bạn có một bảng sai số tư thế với và không có Huber ở 5% outlier, cho thấy khác biệt bậc độ lớn. Và bạn có một đường cong mật độ khối prior tăng theo số lần marginalize — xác nhận mục 5B bằng số của chính mình.
- Nối với** [A/05-observability-gauge-va-suy-bien](#) dùng chính ma trận Λ bạn vừa dựng để xem phổ giá trị riêng; [A/06-bat-dinh-va-lan-truyen-sai-so](#) lấy Λ^{-1} làm hiệp phương sai; [B/09-paradigm-uoc-luong-back-end](#) đặt bộ giải này vào các kiến trúc khác nhau.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1.** Mẹo nằm ở *cấu trúc thưa*, và cụ thể là ở việc khối V — ứng với các điểm — là khối chéo với các khối 3×3 . Lý do: mỗi quan sát chỉ chạm một tư thế và một điểm, nên không có nhân tử nào nối hai điểm với nhau. Phân bù Schur khai thác điều đó để khử toàn bộ $3n$ biến điểm với chi phí tuyến tính theo n , để lại một hệ chỉ cỡ $6m \times 6m$. Chi phí đi từ $O((6m + 3n)^3)$ xuống khoảng $O(m^3) + O(n)$. Với $m = 1000$, $n = 100\,000$, đó là khác biệt giữa nhiều ngày và vài trăm mili giây.
- A2.** Vì bình phương tối thiểu có hai thứ mà không lựa chọn nào khác có cả hai: nghiệm dạng đóng cho bài toán tuyến tính hoá, và *bảo toàn cấu trúc thưa*. Một mô hình nhiều đuôi nặng đúng đắn sẽ phá vỡ cả hai. Cái giá là cực nhạy với outlier — phần dư gấp 100 lần đóng góp gấp 10\,000 lần. Cách xử lý là hai tuyến: RANSAC loại bỏ trước khi tối ưu, và M-estimator chịu đựng trong khi tối ưu. Nhưng phải nói rõ: M-estimator *không* sửa giả thiết Gauss cho đúng, nó chỉ hạn chế thiệt hại; nghiệm thu được là MAP của một mô hình nhiều đuôi nặng, theo đối ngẫu Black-Rangarajan [30].
- A3.** Xoá vứt cả biến lẫn mọi nhân tử chạm nó, nên thông tin mà các quan sát ấy mang về các biến *còn lại* mất theo. Marginalize tích phân biến ra khỏi hậu nghiệm — với Gauss thì đó chính là phân bù Schur — và giữ lại thông tin đó dưới dạng một nhân tử prior mới nối các biến còn lại. Cái mất khi xoá: độ chính xác giảm, và hiệp phương sai bị phóng đại vì bài toán quên rằng nó từng biết nhiều hơn. Cái giá khi marginalize: fill-in (prior là khối đặc) và nguy cơ bất nhất do điểm tuyến tính hoá bị đóng băng, chữa bằng FEJ.
- A4.** λD cộng độ cong dương vào *mọi* hướng, kể cả hướng mà Λ suy biến. Hình học: nó biến một cái máng phẳng thành một cái bát nông. Khi $\lambda \rightarrow 0$ ta có Gauss-Newton (bước dài, nhanh, chỉ đúng khi xấp xỉ tuyến tính đúng); khi $\lambda \rightarrow \infty$ ta có gradient descent bước ngắn (chậm, an toàn). Nó “cứu” bộ giải ở cấu hình suy biến theo nghĩa hệ thống không nổ — nhưng đó là tin xấu cho chẩn đoán: nó trả về một bước ngắn theo hướng không quan sát được và biến một lỗi ồn ào thành lỗi im lặng. LM che triệu chứng chứ không chữa bệnh; bệnh thuộc về [A/05](#).
- A5.** Vì khi khử một tư thế nối với k landmark, mọi cặp trong k landmark ấy trở nên tương quan — thông tin “chúng cùng được nhìn từ một chỗ” phải được lưu ở đâu đó, và chỗ đó là một khối đặc $k \times k$ trong prior. Đồ thị thưa trở nên đặc dần ở vùng biên cửa sổ, và V mất tính khối chéo mà Schur dựa vào. Đó là lý do sâu xa: không phải số biến lớn, mà là chi phí khối prior đặc tăng theo bình phương. Hai cách xử lý: chọn khử cái gì để prior không nối quá nhiều biến (VINS-Mono), hoặc làm thưa lại prior bằng xấp xỉ.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng một tấm lưới làm bằng hàng nghìn sợi chun, mỗi sợi nối hai cái đinh ghim trên một tấm bảng. Mỗi sợi chun là một lần cái máy “nhìn thấy” một thứ gì đó; mỗi cái đinh là một chỗ mà máy từng đứng, hoặc một điểm mốc trong phòng. Không sợi chun nào chính xác cả — mỗi sợi hơi dài hoặc hơi ngắn so với chỗ nó cần. Nên không có cách nào đặt các đinh để mọi sợi đều thoải mái. Việc phải làm là tìm chỗ đặt đinh sao cho *tổng* độ căng nhỏ nhất. Đó là toàn bộ bài toán.

Điều tưởng như làm nó bất khả thi: có hàng trăm nghìn đinh. Nhưng có một sự kiện cứu tất cả. Mỗi sợi chun chỉ nối *hai* đinh, và một trong hai luôn là một điểm mốc chứ không phải một chỗ đứng. Nghĩa là không có sợi nào nối điểm mốc này với điểm mốc kia. Các điểm mốc chỉ “nói chuyện” với nhau *gián tiếp*, qua các chỗ đứng.

Nhờ đó, ta có thể giải quyết từng điểm mốc một cách độc lập — rất nhanh, và làm song song được — rồi chỉ còn lại một bài toán nhỏ giữa vài trăm chỗ đứng. Chuyện từ nhiều ngày rút xuống còn một phần giây. Đó là ý tưởng duy nhất đáng nhớ của chương, và nó không phải toán cao siêu mà là việc nhận ra một sự kiện tầm thường rồi khai thác nó triệt để.

Còn một chuyện nữa. Nếu có một sợi chun bị mắc nhầm chỗ — nối một chỗ đứng với một điểm mốc mà nó chưa bao giờ nhìn thấy — thì sợi đó sẽ căng cực mạnh và kéo lệch cả tấm lưới. Cách chữa không phải tìm cho ra sợi hỏng, vì có hàng triệu sợi. Cách chữa là làm cho mọi sợi chun *mềm dần khi bị kéo quá căng*, để không sợi nào một mình thắng được số đông.

Chỗ không rút gọn được: chỗ không rút gọn được là *marginalization*. Ý tưởng “bỏ bớt đỉnh cũ nhưng giữ lại điều ta đã học được từ chúng” nghe đơn giản, nhưng cái giá của nó — các đỉnh còn lại trở nên dính vào nhau, và thông tin cũ được ghi lại ở một toạ độ đã lỗi thời — thì không có hình ảnh nào diễn tả gọn được. Đó là lý do các hệ cửa sổ trượt phức tạp hơn nhiều so với vẽ ngoài, và là lý do B/09 phải viết riêng một chương.

Một câu: Mỗi phép đo chỉ chạm hai thứ — và chính sự nghèo nàn đó làm bài toán không lồ trở nên giải được.

4.7 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — MAP với nhiều Gauss tương đương bình phương tối thiểu có trọng số — là một suy dẫn toán học thuần tuý dưới hai giả thiết đã nêu rõ, nên nó không bị bác bỏ bởi thực nghiệm mà chỉ bởi lỗi suy dẫn. Cái *có thể* bị bác bỏ là mệnh đề ẩn rằng hai giả thiết ấy đủ gần đúng để khung này dùng được. Nó bị bác bỏ nếu một hệ dùng mô hình nhiều đuôi nặng đúng đắn — chứ không phải M-estimator — cho kết quả tốt hơn đáng kể với chi phí chấp nhận được. Tôi chưa thấy công trình nào chứng minh điều đó trong SLAM thời gian thực, nhưng tôi cũng chưa khảo sát có hệ thống (tôi suy ra, chưa đối chiếu nguồn).

Mệnh đề thứ hai — Schur đưa chi phí từ bậc ba theo tổng số biến xuống gần tuyến tính theo số điểm — là kết quả đếm phép toán dưới giả thiết $n \gg m$ và V khối chéo. Nó bị bác bỏ nếu đường cong đo được không có độ dốc như dự đoán, và mini project chính là phép đo. Một trường hợp mà nó *thật sự* không đúng và đáng biết: khi đồ thị đồng quan sát rất đặc — mọi khung nhìn thấy mọi điểm, như trong một quét vòng quanh một vật nhỏ — thì S đặc và lợi thế giảm mạnh.

Mệnh đề thứ ba — *marginalization* gây bất nhất nếu không dùng FEJ — đã được thiết lập trong ngữ cảnh bộ lọc [32]. Điều tôi *chưa* kiểm được là mức độ nghiêm trọng trong ngữ cảnh cửa sổ trượt tối ưu hoá, nơi các biến vẫn được tái tuyến tính hoá trong cửa sổ. Có lý do để nghĩ nó nhẹ hơn trong bộ lọc, nhưng tôi không có số liệu, và đó là một mục trong 99-chưa-biết.md.

4.8 Kết nối

Chương này là trung tâm của Phần A và là chỗ luận đề trung tâm có nghĩa toán học.

Nối lên trên: A/03-lie-group-va-toi-uu-tren-da-tap cung cấp các khối Jacobian 2×6 và 2×3 mà chương này lắp thành ma trận; A/02-mo-hinh-camera cung cấp hàm đo h ; A/01-hinh-hoc-da-thi cung cấp khởi tạo, thứ mà Gauss-Newton bắt buộc phải có.

Nối xuống dưới trong Phần A. A/05-observability-gauge-va-suy-bien lấy chính ma trận Λ ở (4.2) và hỏi khi nào nó suy biến — toàn bộ chương 5 là một câu hỏi về ma trận của chương 4. A/06-bat-dinh-va-lan-truyen-sai-so lấy Λ^{-1} làm hiệp phương sai và hỏi con số đó có đáng tin không.

Nối sang các phần sau. B/09-paradigm-uoc-luong-back-end là chương mở rộng trực tiếp nhất: nó cho thấy bộ lọc, cửa sổ trượt, smoothing gia tăng và full batch đều là cùng

bài toán MAP với các lựa chọn khác nhau về việc marginalize cái gì và tuyến tính hoá lại bao nhiêu lần. [B/08-mo-hinh-phan-du](#) thay đổi hàm h — indirect, direct, hay lai — mà không đổi gì trong khung này, và đó là bằng chứng cho tính tổng quát của nó. [B/11-loop-closure-va-nhat-quan-toan-cuc](#) dùng GNC và các kỹ thuật bền ở mục 6 để sống với loop closure sai. [C/13-quan-ly-ban-do](#) có gốc rễ toán học ở mục 4C: covisibility graph là cấu trúc khác không của S . [C/14-hieu-chuan-nhu-mot-nhanh-cua-slam](#) là chương chứng minh Bảng 4.1 bằng cách thực sự đi từ một ô sang ô khác.

Nối ngang: ../IMU_Fusion/ chương 4 trình bày cùng khung MAP từ góc quán tính, và chương 9 ở đó nói kỹ hơn về marginalization trong VIO. Chỗ nào cần chi tiết về tiền tích phân như một nhân tử thì đọc ở đó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Chuỗi từ Bayes tới bình phương tối thiểu dựa trên hai giả thiết. Nêu cả hai, nêu chỗ chúng bị vi phạm trong SLAM thật, và nêu hậu quả cụ thể của mỗi vi phạm.
2. Vì sao khối V trong Hessian BA là khối chéo, và điều gì trong cấu trúc bài toán đảm bảo điều đó? Nêu một loại ràng buộc sẽ phá vỡ nó.
3. Tự dẫn lại phần bù Schur từ hệ khối 2×2 , không nhìn bài. Rồi đếm chi phí cho $m = 500$, $n = 50\,000$.
4. Vì sao ma trận S chính là đồ thị đồng quan sát, và điều đó có hệ quả gì cho chính sách chọn keyframe?
5. Nêu ba khác biệt giữa xoá và marginalize một tư thế cũ, và nêu cái giá riêng của marginalize.
6. Levenberg-Marquardt giúp bộ giải chạy được trong cấu hình suy biến. Vì sao đó vừa là tin tốt vừa là tin xấu?
7. Huber với $\delta = 1,5$. Tính trọng số cho phần dư 0,5, 3, và 30 pixel, rồi nói ảnh hưởng tương đối của ba quan sát ấy lên hệ chuẩn so với trường hợp bình phương thuần.

Observability, gauge và suy biến (Observability, Gauge and Degeneracy)

PHỤ THUỘC A/01-hinh-hoc-da-thi (bốn cấu hình suy biến cụ thể — chương này tổng quát hoá chúng); A/03-lie-group-va-toi-uu-tren-da-tap (ngôn ngữ nhiễu loạn để viết “dịch cả thế giới” thành phép toán); A/04-uoc-luong-map-va-bundle-adjustment (ma trận thông tin $\Lambda = \sum H^\top \Omega H$ — toàn bộ chương này là câu hỏi về chỗ ma trận đó suy biến). Phần quán tính tương ứng ở ../IMU_Fusion/ chương 5.

TÓM TẮT

Chương này trả lời câu hỏi mà bốn chương trước không hỏi được: trong số các biến ta vừa đặt vào bài toán MAP, biến nào dữ liệu *thật sự* nói được điều gì đó? Câu trả lời có ba tầng, và ba tầng ấy đòi ba cách xử lý ngược nhau nên trộn chúng là nguồn của phần lớn chẩn đoán sai. Tầng cấu trúc: SfM đơn mắt luôn có đúng bảy bậc tự do không quan sát được — ba tịnh tiến, ba xoay, một tỉ lệ, hợp thành $Sim(3)$ — bất kể chuyển động tốt đến đâu; đó là *quy ước*, không phải thiếu dữ liệu, và cách chữa là chọn một quy ước. Tầng chuyển động: ngoài bảy bậc đó, một số đại lượng chỉ quan sát được nếu chuyển động kích thích đúng chiều; xoay thuần và cảnh phẳng ở A/01 là hai ví dụ, và cách chữa là đổi chuyển động hoặc thêm nguồn thông tin. Tầng thuật toán: ngay cả khi lý thuyết nói một hướng không quan sát được, bộ ước lượng vẫn có thể tự bịa ra thông tin theo hướng ấy do tuyến tính hoá không nhất quán, và hậu quả là hiệp phương sai nhỏ hơn sự thật. Nếu chỉ nhớ một điều: *quan sát được hay không là tính chất của cặp (mô hình, chuyển động), không phải của thuật toán* — đổi bộ giải không cứu được một hướng mà dữ liệu không chứa thông tin.

KIẾN THỨC CẦN CÓ TRƯỚC

- **quan sát được (observability)** — tính chất của một hệ: hai trạng thái khác nhau thì sinh ra hai chuỗi phép đo khác nhau. Nếu hai trạng thái khác nhau sinh ra *chính xác* cùng chuỗi đo thì không phép xử lý nào phân biệt được chúng, và hướng nổi chúng gọi là hướng không quan sát được.
- **không gian rỗng (null space)** — tập các vectơ $\mathbf{u}$ với $\Lambda \mathbf{u} = 0$. Đi theo $\mathbf{u}$ thì hàm mục tiêu không đổi tới bậc hai, nên bộ giải không có lý do gì để ở lại chỗ nó đang đứng.
- **tự do gauge (gauge freedom)** — một hướng không quan sát được sinh ra từ *quy ước*, không từ thiếu dữ liệu. Ta phải chọn một hệ quy chiếu ở đâu đó; chọn chỗ nào cũng được, và chính cái “cũng được” ấy là bậc tự do dư.
- **chuyển động suy biến (degenerate motion)** — một chuyển động làm mất thêm bậc quan sát được so với trường hợp tổng quát: xoay thuần, cảnh phẳng, đi dọc trục quang, đứng yên.
- **kích thích (excitation)** — mức độ mà chuyển động “chạm” vào một biến. Đủ kích thích nghĩa là tồn tại một phần quỹ đạo mà nếu đổi biến đó thì phép đo đổi rõ rệt so với mức nhiễu.
- **số điều kiện (condition number)** — tỉ số giữa giá trị riêng lớn nhất và nhỏ nhất khác không của Λ . Nó đo mức “dẹt” của bát; số điều kiện rất lớn nghĩa là gần suy biến, tức bệnh chưa thành nhưng sắp thành.
- **tính nhất quán (consistency)** — một bộ ước lượng nhất quán khi sai số có trung bình bằng không và hiệp phương sai nó khai báo đúng bằng hiệp phương sai thật [38]. Chính xác nhưng khai hiệp phương sai quá nhỏ là bất nhất, và nguy hiểm hơn kém chính xác mà trung thực.

CÂU HỎI MỞ ĐẦU

- Q1. Bộ ước lượng trả về tỉ lệ $s = 1,03$ với độ lệch chuẩn 0,004. Con số đó có thể hoàn toàn vô nghĩa mà không có bất kỳ dấu hiệu nào trong đầu ra. Cơ chế nào cho phép một bộ ước lượng vừa sai vừa tự tin, và vì sao nó không thể tự phát hiện?
- Q2. SfM đơn mắt có bảy bậc tự do không quan sát được, VIO có bốn, còn SLAM stereo có sáu. Ba con số khác nhau ấy đến từ đâu — và chúng nói gì về việc pose graph nên tối ưu mấy bậc?
- Q3. Hai kỹ sư chạy cùng một bộ giải trên cùng dữ liệu, một người cố định tư thế đầu và một người dùng giả nghịch đảo. Kết quả giống nhau ở đâu và khác nhau ở đâu — và nếu *quỹ đạo* của họ khác nhau thì điều đó nói lên chuyện gì?
- Q4. Bạn phát hiện tỉ lệ không hội tụ nên thêm một prior mạnh để ghim nó lại. Hệ thống ngừng trôi và mọi thứ trông ổn định. Vì sao đó có thể là điều tệ nhất bạn vừa làm, và phép thử nào phân biệt trường hợp nó hợp lệ với trường hợp nó là bẫy?
- Q5. Ba loại suy biến — do quy ước, do chuyển động, do số học — đều biểu hiện thành “ma trận thông tin gần suy biến”. Nêu phép thử phân biệt ba loại, và nói vì sao dùng cách chữa của loại này cho loại kia thì hỏng.

5.1 Đặt vấn đề

Có một kiểu hỏng mà mọi kỹ sư SLAM gặp sớm hay muộn, và nó khác mọi kiểu hỏng khác ở một điểm: nó không phát ra tiếng động nào.

Hình dung tình huống cụ thể. Robot của bạn đang chạy trong một hành lang dài, camera nhìn thẳng về phía trước. Ảnh sắc nét, hàng trăm đặc trưng được trích ra và khớp hoàn hảo giữa các khung, RANSAC báo hơn 90% inlier, sai số chiếu lại trung bình dưới một pixel. Bundle adjustment hội tụ trong năm vòng lặp. Bản đồ trông đẹp. Hệ thống in ra một quỹ đạo kèm hiệp phương sai, và hiệp phương sai ấy nhỏ.

Rồi bạn so với chân trị và phát hiện quỹ đạo dài hơn thực tế 15%.

Bài toán gốc là như vậy, và nó thuộc về bất kỳ ai ước lượng trạng thái từ cảm biến: người làm SLAM đơn mắt, người hiệu chuẩn camera trên dây chuyền, người làm localization trong bản đồ cũ. Trước khi có lý thuyết, cách xử lý phổ biến là kinh nghiệm truyền miệng: “đi chữ chi khi khởi tạo”, “dừng để robot quay tại chỗ quá lâu”, “hiệu chuẩn thì nghiêng bảng nhiều hướng”. Những lời khuyên đó *đúng*, nhưng chúng là các mảnh rời của một định lý mà không ai phát biểu ra. Hệ quả là chúng không tổng quát hoá được: gặp một cấu hình cảm biến mới, không ai biết phải đi kiểu gì, hay đi như vậy đã đủ chưa.

Chỗ khác biệt với mọi kiểu hỏng khác nằm ở đây. Nếu một điểm khớp sai, phần dư sẽ lớn và bạn nhìn thấy. Nếu trọng số sai, NEES lệch và bạn đo được (C/18). Nhưng nếu dữ liệu *không chứa thông tin* về một đại lượng, thì hàm mục tiêu phẳng theo hướng đó, mọi giá trị đều cho phần dư như nhau, và bộ giải — vốn chỉ biết giảm hàm mục tiêu — dừng lại ở một điểm tùy tiện trong thung lũng phẳng ấy rồi báo cáo điểm đó như một kết quả. Tệ hơn, nếu bộ ước lượng bị bệnh bất nhất mà mục 6 sẽ mô tả, nó còn khai một hiệp phương sai *nhỏ*. Sai lầm được đóng gói trong một con số trông tự tin.

Đây là lý do chương này nằm ở vị trí gần cuối Phần A chứ không phải một phụ lục. Bốn chương trước cho công cụ: hình học, mô hình đo, đạo hàm, bộ giải. Chương này hỏi câu đứng trước tất cả: *bài toán MAP bạn vừa dựng có nghiệm duy nhất không, và nếu không thì thiếu ở hướng nào?* Không trả lời được câu này thì mọi kỹ thuật ở các phần sau chỉ là những cách khác nhau để tính ra một con số vô nghĩa nhanh hơn.

Cách hiển nhiên hơn — và cách gần như ai cũng thử trước — là chữa triệu chứng: thấy tỉ lệ trôi thì thêm prior, thấy hiệu chuẩn ngoại không hội tụ thì khoá nó, thấy bộ giải dao động thì tăng

damping. Ba cách đó đều *hoạt động* theo nghĩa hệ thống ngừng gây rối, và chính vì thế chúng nguy hiểm: chúng che mất chẩn đoán. Thêm prior mạnh cho tỉ lệ nghĩa là bạn đã tự quyết định tỉ lệ và bộ ước lượng chỉ xác nhận lại quyết định đó. Cả ba đều hợp lệ *nếu* bạn biết mình đang làm gì, và đều là bẫy nếu không. Phân biệt hai trường hợp là toàn bộ nội dung của chương.

TRỰC GIÁC

Hình dung hàm mục tiêu MAP như một cảnh quan trong không gian nhiều chiều: mỗi điểm là một cấu hình biến, độ cao là tổng bình phương phần dư có trọng số. Bộ giải là một quả bóng lăn xuống.

Trường hợp lành mạnh: cảnh quan là một cái bát. Có đúng một đáy, quả bóng dừng ở đó, và độ cong theo mỗi hướng cho biết bạn chắc chắn tới mức nào theo hướng ấy — đó chính là ma trận thông tin.

Trường hợp bệnh: cảnh quan không phải bát mà là một *máng*. Cong theo vài hướng, nhưng có ít nhất một hướng mà đi dọc theo nó thì độ cao không đổi chút nào. Quả bóng vẫn dừng — ma sát số học, điều kiện dừng của bộ giải, một chút nhiễu làm nghiêng máng — và nó dừng ở một chỗ hoàn toàn tùy tiện dọc theo máng.

Điều then chốt: quả bóng không biết mình đang ở trong máng. Nó chỉ biết nó không lăn nữa. Toàn bộ lý thuyết observability là cách trả lời câu “cảnh quan này có máng không, và máng chạy theo hướng nào” *trước khi* thả bóng.

Và một chi tiết quyết định về cách chữa: có loại máng là *cố hữu* — nó tồn tại vì bài toán thật sự có nhiều nghiệm tương đương, và cách chữa là chọn một nghiệm rồi đi tiếp. Có loại máng là *tạm thời* — nó xuất hiện vì dữ liệu vừa rồi nghèo, và nó sẽ biến mất nếu bạn đi kiểu khác. Dùng cách chữa của loại này cho loại kia là sai lầm tốn kém nhất trong chương.

5.2 Observability, phát biểu cho đúng một lần

A. Phát biểu theo phép đo

Cho một hệ với biến θ và hàm đo $\mathbf{z} = h(\theta)$. Hệ không quan sát được nếu tồn tại $\theta \neq \theta'$ mà $h(\theta) = h(\theta')$ — hai cấu hình khác nhau sinh ra *chính xác* cùng tập phép đo.

Định nghĩa này có một hệ quả cần nói rõ ngay vì nó là trục của cả chương. Nếu hai cấu hình sinh ra cùng phép đo, thì *không có thuật toán nào* phân biệt được chúng. Không phải “thuật toán hiện tại chưa đủ tốt”. Không phải “cần thêm dữ liệu cùng loại”. Thông tin không tồn tại trong dữ liệu, và không phép biến đổi nào tạo ra thông tin từ chỗ không có. Đây là lý do câu “quan sát được là tính chất của cặp (mô hình, chuyển động)” được ghi vào **00-map** như một trong bốn nguyên lý xuyên suốt.

B. Phát biểu theo ma trận thông tin

Trong khung MAP của **A/04**, quanh nghiệm $\hat{\theta}$ ta có xấp xỉ bậc hai

$$J(\hat{\theta} \boxplus \delta) \approx J(\hat{\theta}) + \frac{1}{2} \delta^\top \Lambda \delta, \quad \Lambda = \sum_k H_k^\top \Omega_k H_k.$$

Nếu tồn tại $\mathbf{u} \neq 0$ với $\Lambda \mathbf{u} = 0$, thì đi theo $\mathbf{u}$ không làm hàm mục tiêu tăng tới bậc hai. Vì sao bước này hợp lệ? Vì Λ là tổng các ma trận nửa xác định dương, nên

$$\mathbf{u}^\top \Lambda \mathbf{u} = \sum_k \|\Omega_k^{1/2} H_k \mathbf{u}\|^2 = 0$$

buộc *từng* số hạng bằng không, tức $H_k \mathbf{u} = 0$ với mọi k . Nói cách khác, *không một phép đo nào* thay đổi khi ta nhiễu loạn theo $\mathbf{u}$. Đó đúng là định nghĩa ở mục A, viết ở dạng tuyến tính hoá.

Sự gặp nhau này đáng dừng lại một câu. Nó chuyển câu hỏi trừu tượng “hệ có quan sát được không” thành một phép thử số học cụ thể: lấy ma trận thông tin, tính phổ giá trị riêng, đếm xem bao nhiêu giá trị bằng không hoặc gần không. Và nó cho biết ngay hậu quả: hiệp phương sai là Λ^{-1} , nên một giá trị riêng bằng không nghĩa là phương sai *vô hạn* theo hướng tương ứng. Bộ giải nào báo hiệp phương sai hữu hạn theo hướng đó thì hoặc đã thêm một prior, hoặc đang bịa.

C. Ba mức suy biến cần tách bạch

Trong thực hành, ba thứ khác nhau đều biểu hiện thành “ Λ gần suy biến”, và trộn chúng là nguồn của rất nhiều chẩn đoán sai. Đây là câu trả lời cho câu hỏi mở đầu thứ năm.

- **Suy biến do quy ước (gauge).** Bài toán *thật sự* có vô số nghiệm, và điều đó đúng về mặt toán học — ta chỉ chưa chọn hệ quy chiếu. Bảy bậc của SfM đơn mắt ở mục 3 thuộc loại này. Cách chữa: chọn quy ước; mọi cách chọn đều hợp lệ và cho cùng kết quả có nghĩa vật lý. Dấu hiệu: số chiều không gian rộng *không đổi* dù chuyển động thế nào.
- **Suy biến do chuyển động.** Mô hình quan sát được với chuyển động tổng quát, nhưng chuyển động *cụ thể* vừa rồi nằm trong một tập suy biến. Xoay thuần và cảnh phẳng thuộc loại này. Cách chữa: đổi chuyển động; nếu không đổi được thì thừa nhận và thêm thông tin từ nguồn khác. Dấu hiệu: số chiều không gian rộng *thay đổi* theo đoạn quỹ đạo.
- **Suy biến do số học.** Ma trận về nguyên tắc khả nghịch nhưng số điều kiện quá lớn so với độ chính xác máy hoặc mức nhiễu. Đây là trường hợp xấu nhất để chẩn đoán vì nó nằm giữa hai loại trên và di chuyển theo dữ liệu. Cách chữa: đổi tham số hoá (nghịch đảo độ sâu thay cho XYZ), scaling biến, hoặc dùng dạng căn bậc hai [37].

Ba loại đòi ba cách xử lý khác nhau, và dùng cách của loại này cho loại kia thì hỏng. Cổ định gauge cho một hướng suy biến do chuyển động — ví dụ khoa tỉ lệ vì nó không hội tụ — là tự áp đặt một giá trị mà bạn không có cơ sở nào để chọn, và đó là câu trả lời cho câu hỏi mở đầu thứ tư. Ngược lại, đổi chuyển động để cứu một tự do gauge là vô ích: bảy bậc của SfM không biến mất dù bạn đi kiểu gì.

I 5.3 Bảy bậc tự do của SfM đơn mắt

A. Trực giác: dịch, xoay, co giãn cả thế giới

Đặt câu hỏi ngược: có phép biến đổi nào của *toàn bộ* bài toán — mọi tư thế, mọi điểm — mà sau đó mọi phép đo vẫn giữ nguyên?

Phép đo là toạ độ ảnh $\mathbf{x}_{ij} = \pi(R_i(\mathbf{P}_j - \mathbf{c}_i))$, với $\mathbf{c}_i$ là tâm camera. Xét phép biến đổi $Sim(3)$ áp cho mọi thứ:

$$\mathbf{P}_j \mapsto s R_g \mathbf{P}_j + \mathbf{t}_g, \quad \mathbf{c}_i \mapsto s R_g \mathbf{c}_i + \mathbf{t}_g, \quad R_i \mapsto R_i R_g^\top.$$

Thay vào:

$$R_i R_g^\top (s R_g \mathbf{P}_j + \mathbf{t}_g - s R_g \mathbf{c}_i - \mathbf{t}_g) = R_i R_g^\top s R_g (\mathbf{P}_j - \mathbf{c}_i) = s R_i (\mathbf{P}_j - \mathbf{c}_i).$$

Và vì π là phép chia cho thành phần z , hệ số s triệt tiêu hoàn toàn: $\pi(s\mathbf{v}) = \pi(\mathbf{v})$. Phép đo *không đổi*, với mọi R_g , mọi $\mathbf{t}_g$, mọi $s > 0$.

Đếm: ba tịnh tiến, ba xoay, một tỉ lệ — **bảy bậc tự do không quan sát được**, hợp thành nhóm $Sim(3)$ ở A/03 mục 4B. Tôi tự dẫn lại toàn bộ (cửa a) và đối chiếu [28, §9] cùng [2] (cửa c).

B. Vì sao con số thay đổi theo cấu hình cảm biến

Đây là chỗ đáng dừng lại, vì nó là một hệ quả trực tiếp và đẹp của luận đề trung tâm — và là câu trả lời cho câu hỏi mở đầu thứ hai.

Bảy bậc trên là của SfM *thuần thị giác đơn mắt*. Thêm cảm biến thì con số giảm, và giảm đúng theo thứ cảm biến ấy đo được:

Thêm *stereo* với đường cơ sở đã biết bằng mét: tỉ lệ trở thành quan sát được, vì nhân đôi thể giới sẽ làm disparity giảm một nửa và điều đó *nhìn thấy được*. Còn sáu bậc, tức $SE(3)$.

Thêm *IMU*: gia tốc kể cho đơn vị mét (nếu chuyển động có gia tốc thay đổi — điều kiện quan trọng, xem .../IMU_Fusion/ chương 5), và trọng lực ghim roll với pitch vì gia tốc kể đo lực riêng nên nó “thấy” hướng xuống. Còn bốn bậc: ba tịnh tiến và yaw.

Thêm GNSS hoặc *marker có tọa độ đã biết*: tịnh tiến và yaw cũng được ghim. Còn không bậc — bài toán có nghiệm tuyệt đối.

CẦN NHỚ

Số bậc tự do gauge **không phải một lựa chọn kỹ thuật mà là hệ quả trực tiếp của việc cảm biến nào có mặt**. Bảng dưới là thứ nên thuộc:

Mono thuần thị giác: **7** ($Sim(3)$). Stereo: **6** ($SE(3)$). VIO: **4** (tịnh tiến + yaw). Có GNSS hoặc anchor tuyệt đối: **0**.

Hai hệ quả thực hành theo ngay. Thứ nhất, *pose graph phải tối ưu đúng số bậc ấy*: $Sim(3)$ cho mono [26], $SE(3)$ cho stereo, bốn bậc cho VIO. Xiết thêm bậc là xiết thứ dữ liệu đã xác định rồi — vô hại nhưng vô ích; để dư bậc là tạo thêm đường cho sai số tích lũy.

Thứ hai, và đây là chỗ hay sai nhất: *căn chỉnh quỹ đạo khi đánh giá phải dùng đúng số bậc ấy*. Căn bảy bậc (có tỉ lệ) cho một hệ VIO là **che mất lỗi tỉ lệ** — tức che mất đúng thứ bạn cần đo. Căn bốn bậc cho một hệ mono là vô nghĩa vì bốn bậc không đủ hấp thụ gauge. Chi tiết ở C/18 và [39].

C. Vì sao đây là tin tốt

Phản ứng tự nhiên khi gặp kết quả này lần đầu là coi nó như khiếm khuyết. Thật ra nó là điều kiện lành mạnh nhất có thể có.

Bảy bậc ấy không phải thiếu thông tin mà là *dư quy ước*. Thế giới không có gốc tọa độ; ta phải chọn một. Không có hướng bắc trong bài toán thuần thị giác; ta phải chọn một. Không có đơn vị chiều dài; ta phải chọn một. Bảy bậc đó chính xác là số cách chọn, và mọi cách chọn cho cùng một quỹ đạo tương đối, cùng một hình dạng bản đồ, cùng mọi thứ có ý nghĩa vật lý.

Điều cần để ý là chúng *không biến mất khi đi lâu hơn hay giỏi hơn*. Đây là điểm phân biệt quan trọng nhất với mục 4. Tỉ lệ dưới xoay thuần là vấn đề của chuyển động và chữa được bằng cách đổi chuyển động; bảy bậc gauge thì không — chúng là tính chất của mô hình.

5.4 Ba cách cố định gauge

Biết có bảy bậc dư là một chuyện; xử lý trong bộ giải là chuyện khác. Để nguyên thì Λ có bảy giá trị riêng bằng không, Cholesky thất bại, và Gauss-Newton không có bước đi xác định. Có đúng ba cách, và chúng không tương đương ở mọi khía cạnh [40].

A. Cố định tư thế đầu (và một khoảng cách)

Chọn khung đầu làm gốc, khoá sáu bậc tự thế của nó; rồi khoá thêm một đại lượng có đơn vị chiều dài — thường là khoảng cách giữa hai khung đầu, đặt bằng một. Bấy ràng buộc cho bấy bậc dư.

Ưu điểm: rõ ràng và rẻ; ma trận còn lại xác định dương. Nhược điểm ít người để ý: hiệp phương sai đầu ra là hiệp phương sai *tương đối so với khung đầu*, nên nó tăng đơn điệu theo khoảng cách tới khung đó. Một điểm gần khung đầu trông rất chắc chắn, một điểm cuối quỹ đạo trông rất bất định, và sự bất đối xứng ấy là *nhân tạo*. Khi bạn lấy hiệp phương sai đó dùng cho việc khác — quyết định đóng vòng, đặt trọng số, lập kế hoạch — bạn đang mang một quy ước tùy tiện vào một quyết định vật lý.

B. Thêm prior

Không khoá cứng mà thêm một số hạng phạt mềm nới vào khung đầu với hiệp phương sai nhỏ. Cộng một ma trận xác định dương vào Λ , làm nó khả nghịch.

Ưu điểm: mọi biến vẫn tự do, quan trọng khi ta không hoàn toàn chắc khung đầu ở đâu. Nhược điểm: *độ mạnh* của prior trở thành một núm vặn không có cơ sở vật lý. Quá mạnh thì hành xử như khoá cứng nhưng làm hỏng hiệp phương sai theo cách khó lẩn; quá yếu thì ma trận vẫn gần suy biến. Ranh giới đúng phụ thuộc vào tỉ lệ của các trọng số khác, nên phải chỉnh lại khi đổi cấu hình cảm biến — đúng loại tinh chỉnh mà cây này cố tránh.

C. Gauge tự do với giả nghịch đảo

Không cố định gì. Chấp nhận Λ suy biến và giải bằng giả nghịch đảo Moore–Penrose, lấy nghiệm chuẩn nhỏ nhất. Hình học: chọn điểm trên máng gần nhất với điểm khởi đầu hiện tại.

Ưu điểm: không áp đặt quy ước, và hiệp phương sai thu được là “trung bình trên gauge”, không thiên vị khung nào. Nhược điểm là chi phí: cần SVD hoặc xử lý thừa tính vi hơn, đắt hơn Cholesky rõ rệt. Với bài toán cỡ lớn, đây là khác biệt quyết định.

Bảng 5.1. Ba cách cố định gauge. Cả ba cho *cùng một quỹ đạo tương đối*; chúng khác nhau ở hiệp phương sai báo cáo, ở chi phí, và ở chỗ có đưa thêm một hằng số tùy ý vào hệ hay không [40].

Cách	Cơ chế	Giá phải trả	Khi nào chọn
Cố định tư thế đầu	Xoá 7 cột khỏi Jacobian; Λ rút gọn xác định dương	Hiệp phương sai là tương đối so với khung đầu, tăng đơn điệu theo khoảng cách — bất đối xứng nhân tạo	Mặc định cho BA cửa sổ trượt; đơn giản nhất, rẻ nhất
Thêm prior	Cộng nhân tử phạt mềm vào khung đầu	Độ mạnh prior là hằng số không có cơ sở vật lý, phải chỉnh lại khi đổi cấu hình	Khi cần giữ mọi biến tự do, hoặc khi thật sự có tiên nghiệm
Gauge tự do	Giả nghịch đảo Moore–Penrose; nghiệm chuẩn nhỏ nhất	Cần SVD hoặc xử lý thừa phức tạp; đắt hơn Cholesky	Khi cần hiệp phương sai không thiên vị để so sánh — thường là nghiên cứu

Lưu ý quan trọng để tránh hiểu nhầm, và là câu trả lời cho câu hỏi mở đầu thứ ba: *cả ba cho cùng một quỹ đạo tương đối*. Chúng khác nhau ở hiệp phương sai báo cáo và ở chi phí, không ở nghiệm

có nghĩa vật lý. Nếu ba cách cho ba quỹ đạo khác nhau về hình dạng, thì bạn *không* đang gặp vấn đề gauge mà đang gặp một vấn đề khác — nhiều khả năng là một hướng suy biến do chuyển động, hoặc một lỗi cài đặt.

5.5 Suy biến do chuyển động: mục thực dụng nhất

Bảy bậc gauge là chuyện cấu trúc và không đổi. Phần này nói về chuyện ngược lại: những bậc *lẽ ra* quan sát được nhưng mất đi vì chuyển động cụ thể vừa rồi không kích thích chúng.

A. Bốn cấu hình từ chương 1, nhìn bằng ngôn ngữ mới

A/01 mục 6 liệt kê bốn cấu hình suy biến bằng ngôn ngữ hình học. Với công cụ của chương này, cả bốn trở thành phát biểu về không gian rỗng của Λ , và việc thống nhất ấy có giá trị: nó cho một *phép thử duy nhất* thay cho bốn phép thử riêng.

Xoay thuận: mọi điểm có thể ở bất kỳ độ sâu nào dọc tia của nó mà không đổi ảnh. Nếu có n điểm, ta được thêm n hướng không quan sát được — không gian rỗng phình từ bảy lên $7 + n$. Phép thử: đếm số giá trị riêng gần không, thấy nó tăng theo số điểm.

Cánh phẳng: họ nghiệm E không duy nhất; về mặt Λ , có thêm hướng tương ứng với việc nghiêng mặt phẳng và bù bằng tư thế.

Đi dọc trục quang: không suy biến hoàn toàn, nhưng số điều kiện rất lớn — các điểm gần epipole gần như không đóng góp thông tin về tịnh tiến. Đây là ví dụ của loại suy biến số học ở mục 2C.

Mọi điểm ở vô hạn: hướng tịnh tiến mất; không gian rỗng phình thêm ba hướng (hoặc hai, tùy ràng buộc còn lại).

B. Ví dụ nhỏ tính tay: hai giả thuyết, một chuỗi đo

Dựng ví dụ nhỏ nhất chứng minh tỉ lệ không quan sát được bằng số cụ thể.

Giả sử bài toán một chiều: camera ở vị trí p , một điểm ở vị trí f trên cùng trục, và camera đo góc nhìn tới điểm — nhưng vì là một chiều, ta đo đại lượng tỉ lệ nghịch với khoảng cách, $z = c/(f - p)$, đúng như kích thước ảnh của một vật tỉ lệ nghịch với độ sâu.

Giả thuyết một: $c = 1$, $p_0 = 0$, camera đi với $p(t) = t$, điểm ở $f = 10$. Lấy mẫu tại $t = 0, 1, 2, 3$:

$$z_0 = \frac{1}{10} = 0,1, \quad z_1 = \frac{1}{9} \approx 0,1111, \quad z_2 = \frac{1}{8} = 0,125, \quad z_3 = \frac{1}{7} \approx 0,1429.$$

Giả thuyết hai: nhân đôi mọi thứ có đơn vị mét — $p'(t) = 2t$, $f' = 20$, và giữ nguyên tia nhìn bằng cách để $c' = 2$:

$$z'_k = \frac{2}{20 - 2t_k} = \frac{2}{2(10 - t_k)} = \frac{1}{10 - t_k} = z_k.$$

Chuỗi đo trùng khít, từng chữ số. Hai giả thuyết khác nhau hoàn toàn về mét, không phân biệt được từ dữ liệu. (Cửa kiểm chứng b.)

Một câu giải thích về c , vì nó dễ gây hiểu nhầm. Trong bài toán thật, c là tiêu cự đã biết nên không được nhân lên. Điều đó *không* cứu tình hình, và lý do đáng ghi: bất biến thật của bài toán thị giác đơn mắt là phép co giãn đồng thời *quỹ đạo và bản đồ*, giữ nguyên tham số nội — và phép co giãn ấy giữ nguyên mọi *tia nhìn* từ camera tới điểm, nên giữ nguyên mọi toạ độ ảnh. Đó chính là suy dẫn ở mục 3A. Ví dụ một chiều là cách rẻ nhất để thấy cơ chế, với cái giá là phải cho c co giãn theo để mô phỏng “tia nhìn không đổi”.

C. Điều kiện kích thích cho tham số hiệu chuẩn

Khi thả tham số hiệu chuẩn ra ([C/14](#)), mỗi tham số cần một loại kích thích riêng, và đây là bảng đáng thuộc vì nó biến những lời khuyên truyền miệng thành điều kiện kiểm được.

Bảng 5.2. Điều kiện kích thích cho từng nhóm tham số. Cột cuối là cách phát biểu lại dưới dạng thao tác — và đó chính là nội dung của mọi hướng dẫn thu dữ liệu hiệu chuẩn.

Tham số	Cần gì để quan sát được	Phát biểu thành thao tác
Tiêu cự f	Đa dạng <i>độ sâu</i> — target ở nhiều khoảng cách	Đưa bảng lại gần rồi ra xa
Điểm chính (c_x, c_y)	Target xuất hiện ở <i>nhiều vị trí</i> trong khung, nhất là bốn góc	Quét bảng khắp khung hình
Hệ số méo	Target ở <i>rìa ảnh</i> , nơi méo lớn nhất	Đẩy bảng ra sát mép
Tỉ lệ (mono)	Gia tốc thay đổi (có IMU) hoặc một chiều dài đã biết	Chuyển động có tăng giảm tốc
Ngoại tình tiến camera-IMU	Xoay quanh <i>nhiều trục</i> (số hạng $\omega \times \mathbf{p}$)	Xoay quanh cả ba trục
Ngoại xoay stereo	Cảnh có <i>độ sâu đa dạng</i>	Cảnh có cả vật gần và xa
Sai lệch thời gian t_d	Tốc độ góc <i>cao và biến thiên</i>	Lắc nhanh

Điều đáng nói về bảng này: nó giải thích vì sao mọi hướng dẫn hiệu chuẩn đều bảo bạn “ngiênêng bảng nhiều hướng, đưa ra sát rìa, lại gần rồi ra xa”. Ba thao tác đó không phải mê tín — chúng là ba điều kiện kích thích ở ba hàng đầu, phát biểu bằng ngôn ngữ thao tác tay. Và nó cho một tiêu chí để *tự động đánh giá* một bộ dữ liệu hiệu chuẩn trước khi chạy tối ưu, điều mà [D/21](#) cần cho đây chuyên.

I 5.6 Nhất quán: khi bộ ước lượng tự bịa thông tin

Tầng thứ ba, và là tầng tinh vi nhất. Ngay cả khi lý thuyết nói một hướng không quan sát được, bộ ước lượng vẫn có thể báo cáo hiệp phương sai hữu hạn theo hướng ấy.

Cơ chế là **thông tin giả** (spurious information). Trong một hệ có marginalization ([A/04](#) mục 5), thông tin về một biến được lưu lại dưới dạng nhân tử prior, tính bằng Jacobian tại một điểm tuyến tính hoá. Biến ấy tiếp tục di chuyển; khi các phần khác của bài toán tuyến tính hoá nó ở điểm mới, hai Jacobian *không tương thích*. Tích của các ma trận chuyển không còn giữ đúng không gian rỗng mà lý thuyết dự đoán, và kết quả là ma trận thông tin có hạng cao hơn hạng thật [\[32\]](#).

Hậu quả đi theo chiều nguy hiểm chứ không phải chiều an toàn: thông tin *tăng*, nên hiệp phương sai *giảm*, nên bộ ước lượng trở nên *quá tự tin*. Một hệ báo sai số ± 2 cm khi sai thật là ± 20 cm gây hại nhiều hơn một hệ báo ± 50 cm — vì bộ lập kế hoạch dựa vào con số ấy để quyết định đi qua một khe hẹp.

Hai cách chữa. **FEJ**: cố định điểm tuyến tính hoá của các biến bị chạm bởi prior, chấp nhận Jacobian hơi lỗi thời để đổi lấy tính nhất quán của không gian rỗng. **Công thức bất biến** (InEKF): chọn một cấu trúc nhóm mà động học sai số không phụ thuộc trạng thái, nên vấn đề không phát sinh [\[41\]](#). Cây . . /IMU_Fusion/ chương 5 và 8 nói kỹ cả hai; ở đây điều cần nhớ là *hiệp phương sai do một bộ ước lượng có marginalization báo ra phải bị nghi ngờ mặc định*, và [C/18](#) cho công cụ đo mức độ nghi ngờ ấy bằng NEES.

CẠM BÃY

Bốn phép thử để phát hiện suy biến, xếp theo độ rẻ. Không phép nào là số inlier — xem A/01 mục 6.

(1) **Chạy lại với nhiều khởi đầu khác nhau.** Rẻ nhất, mạnh bất ngờ. Nếu nghiệm *đi theo* giá trị khởi đầu thì hướng đó không quan sát được. Đây là phép thử duy nhất không cần chân trị và không cần dùng vào ma trận.

(2) **Phổ giá trị riêng của Λ .** Đếm số giá trị riêng gần không, so với số bậc gauge lý thuyết ở mục 3B. Nhiều hơn dự đoán nghĩa là có suy biến do chuyển động. Xem cả *vectơ riêng* để biết hướng nào — đó là chẩn đoán, không chỉ báo động.

(3) **Số điều kiện theo thời gian.** Rẻ hơn phổ đầy đủ và dùng được tại runtime. Nó bắt được suy biến số học, tức trường hợp bệnh chưa thành nhưng sắp thành — và đó là thứ bạn muốn biết *trước* khi hỏng.

(4) **NEES với chân trị.** Đắt nhất vì cần chân trị, nhưng là phép thử duy nhất bắt được bệnh bất nhất ở mục 6. Chỉ làm được trong mô phỏng hoặc trên bộ dữ liệu có chân trị — và đó là một lý do chính đáng để dựng mô phỏng, xem C/18.

Điểm chung của cả bốn: chúng đo *mức xác định của mô hình*, không đo *mức nhất quán của dữ liệu với mô hình*. Nhằm hai thứ đó là gốc của kiểu hỏng im lặng.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenVINS [42]	FEJ bật/tắt bằng cờ; hiệu chuẩn trực tuyến có kiểm observability; bộ mô phỏng có chân trị	Nơi tốt nhất để đọc mã liên quan chương này; cũng là chỗ tính được NEES
Kalibr [21]	Quy trình thu dữ liệu có yêu cầu chuyển động cụ thể	Mỗi yêu cầu trong tài liệu là một hàng của Bảng 5.2 viết thành thao tác
ORB-SLAM3 [10]	Từ chối khởi tạo khi parallax không đủ; $Sim(3)$ cho loop mono, $SE(3)$ khi có stereo/IMU	Mục 3B thành quyết định kiến trúc
rpg_trajectory_evaluation [39] [40]	Căn chỉnh theo đúng số bậc tự do của cấu hình cảm biến So sánh có hệ thống ba cách xử lý gauge trong VIO	Khối <i>Cần nhớ</i> mục 3B cài thành công cụ Nguồn trực tiếp của Bảng 5.1

MINI PROJECT SLAM · Xác nhận bảy bậc tự do bằng phổ, rồi làm nó phình ra bằng ba chuyển động suy biến · 3 ngày

Mục tiêu	nhìn tận mắt bảy giá trị riêng bằng không, rồi nhìn chúng thành nhiều hơn khi chuyển động suy biến — tự kiểm chứng mục 3 và mục 5 bằng số.
Dữ liệu	tự sinh, vì bạn cần kiểm soát chính xác chuyển động: một quỹ đạo tổng quát, một quỹ đạo xoay thuần, một cảnh phẳng, và một quỹ đạo đi dọc trục quang. Một bản đồ điểm ngẫu nhiên.
Các bước	dựng bài toán BA nhỏ với vài chục tư thế và vài trăm điểm dùng mã từ A/04; <i>không</i> cố định gauge; dựng Λ dạng đặc; tính toàn bộ phổ giá trị riêng; với quỹ đạo tổng quát, xác nhận đúng bảy giá trị riêng ở mức không số học và kiểm rằng bảy vectơ riêng tương ứng đúng là ba tịnh tiến, ba xoay, một tỉ lệ; lặp với ba quỹ đạo suy biến và đếm lại; cuối cùng thêm tham số nội vào tập biến rồi đếm thêm một lượt.
Xong khi	bạn có một bảng: bốn hàng quỹ đạo, mỗi hàng ghi số giá trị riêng gần không và <i>diễn giải vật lý</i> của các vectơ riêng. Kiểm tra chéo bắt buộc: với quỹ đạo tổng quát và có thêm tham số nội, số chiều không gian rộng phải <i>vẫn</i> là bảy — nếu ra nhiều hơn thì quỹ đạo chưa đủ đa dạng theo Bảng 5.2, và đó cũng là một kết quả đáng ghi. Thêm một cột: với

mỗi quỹ đạo, chạy lại từ ba khởi đầu khác nhau và ghi độ tán của nghiệm — xác nhận phép thử (1) ở khối *Cạm bẫy*.

Nối với B/10-khoi-tao dùng bảng này làm tiêu chí quyết định khi nào được phép khởi tạo; C/14-hieu-chuan-nhu-mot-nhanh-cua-slam dùng cột cuối để quyết định thả biến nào ra.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Cơ chế là hàm mục tiêu phẳng theo một hướng: mọi giá trị của tỉ lệ dọc hướng đó cho *chính xác* cùng tập phân dư, nên bộ giải không có tín hiệu nào để chọn và nó dừng ở chỗ nhiều số học đầy tới. Nó không tự phát hiện được vì thứ duy nhất bộ giải quan sát là hàm mục tiêu, và hàm mục tiêu không phân biệt các điểm trên máng. Độ lệch chuẩn 0,004 khi đó đến từ một prior ngấm, từ damping LM, hoặc từ thông tin giả do tuyến tính hoá không nhất quán — ba nguồn đều không phải phép đo. Cách bắt rẻ nhất: chạy lại với nhiều khởi đầu; nếu nghiệm đi theo khởi đầu thì hướng đó không quan sát được.
- A2. Ba con số đến từ việc cảm biến nào cung cấp thông tin gì. Mono thuần thị giác: không có đơn vị chiều dài, không có hướng tuyệt đối, không có gốc — bảy bậc, đúng $Sim(3)$. Stereo: đường cơ sở đã biết bằng mét nên tỉ lệ quan sát được, còn sáu. VIO: gia tốc kế cho mét (nếu gia tốc thay đổi) và trọng lực ghim roll-pitch vì gia tốc kế đo lực riêng, còn bốn. Hệ quả: pose graph phải tối ưu đúng số bậc ấy — $Sim(3)$, $SE(3)$, hay bốn bậc. Và quan trọng không kém, *cần chỉnh khi đánh giá* phải dùng đúng số đó; cần bảy bậc cho hệ VIO là che mất lỗi tỉ lệ.
- A3. Giống nhau: *quỹ đạo tương đối* — hình dạng đường đi và hình dạng bản đồ trùng khít sau khi căn chỉnh. Khác nhau: hiệp phương sai báo cáo (cách 1 cho hiệp phương sai tương đối so với khung đầu, tăng đơn điệu theo khoảng cách; cách 3 cho hiệp phương sai không thiên vị) và chi phí tính toán. Nếu *quỹ đạo* của họ khác nhau về hình dạng thì đó **không** phải vấn đề gauge — gauge chỉ làm khác nhau một phép biến đổi $Sim(3)$ toàn cục. Khác nhau về hình dạng nghĩa là có một hướng suy biến do chuyển động, hoặc một lỗi cài đặt, và phải điều tra chứ không được bỏ qua.
- A4. Vì bạn vừa *tự quyết định* tỉ lệ, và bộ ước lượng chỉ xác nhận lại quyết định đó — con số đầu ra bây giờ phản ánh prior của bạn chứ không phản ánh dữ liệu. Nếu prior đúng thì tốt; nếu sai thì bạn đã đóng băng một sai số có hệ thống và nó sẽ chảy vào mọi thứ khác. Tệ hơn, hiệp phương sai bây giờ trông nhỏ vì prior mạnh, nên không có gì cảnh báo. Phép thử phân biệt: hỏi *prior này đến từ đâu*. Nếu từ một phép đo thật — đường cơ sở stereo, một vật có kích thước đã biết, IMU — thì nó hợp lệ và nên được mô hình hoá thành một nhân tử có hiệp phương sai đúng. Nếu nó chỉ là một con số bạn chọn để hệ thống ngừng trôi thì đó là bẫy: bạn đã che chẩn đoán thay vì chữa bệnh.
- A5. Phép thử: (a) *số chiều không gian rộng có đổi theo chuyển động không*? Không đổi thì là gauge; đổi thì là suy biến do chuyển động. (b) *giá trị riêng có đúng bằng không hay chỉ rất nhỏ*? Đúng bằng không (tới mức không số học) thì là hai loại đầu; chỉ nhỏ thì là suy biến số học. Dùng sai cách chữa thì hỏng: cố định gauge cho hướng suy biến do chuyển động là tự áp đặt một giá trị không có cơ sở; đổi chuyển động để cứu gauge là vô ích vì bảy bậc không biến mất dù đi kiểu gì; và đổi tham số hoá để chữa một gauge thật thì chỉ dịch vấn đề đi chỗ khác.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng bạn tỉnh dậy trong một căn phòng trắng không cửa sổ, không đồ đạc, và người ta hỏi bạn đang ở đâu.

Có những câu bạn không bao giờ trả lời được, dù ngồi bao lâu. Phòng này ở toạ độ nào trên Trái Đất? Nó quay mặt về hướng nào? Nếu mọi thứ trong phòng — kể cả bạn — được thu nhỏ đúng một nửa cùng lúc, bạn có nhận ra không? Không, không, và không. Ba câu đó không phải vì bạn quan sát kém mà vì *trong phòng không có thông tin đó*. Đây là bảy điều mà một cái máy chỉ có camera cũng không bao giờ biết được, và điều quan trọng là: nó *không cần* biết. Ta chỉ cần chọn đại một gốc, một hướng, một đơn vị, rồi mọi thứ còn lại vẫn đúng so với nhau.

Rồi có loại câu hỏi thứ hai, thú vị hơn. “Cái bàn kia cách tôi bao xa?” Nếu bạn ngồi im, bạn không biết — một cái bàn nhỏ ở gần trông giống hệt một cái bàn to ở xa. Nhưng nếu bạn *đi vài bước*, bạn biết ngay. Đây là loại khác hẳn loại trên: hôm nay không biết nhưng sẽ biết nếu làm đúng việc. Và có loại thứ ba, khó chịu nhất. Bạn đi vài bước, nhưng đi theo đúng hướng cái bàn — nên nó chỉ to dần lên chứ không lệch sang bên. Bạn *có* thông tin nhưng rất ít, và ước lượng của bạn chập chờn. Đây không phải không biết, cũng không phải biết chắc, mà là ở giữa — và ở giữa là chỗ khó chẩn đoán nhất.

Điều nguy hiểm nhất không phải việc không biết. Nó là việc *nói ra một con số kèm sự tự tin mà mình không có quyền có*. Cái máy trong cả ba tình huống trên vẫn đọc ra một con số, vì người ta bảo nó phải đọc ra một con số. Và nếu lập trình sơ ý, nó còn kèm thêm “tôi khá chắc”.

Toàn bộ chương này là cách trả lời: trong những con số máy đang đọc ra, con số nào là kết quả của một phép đo, con số nào là một quy ước ta tự chọn, và con số nào chỉ là chỗ ngẫu nhiên mà thuật toán dừng lại.

Chỗ không rút gọn được: ranh giới giữa “không biết được vĩnh viễn” và “chưa biết được vì chuyển động chưa đủ” không nhìn ra được từ đầu ra của bộ ước lượng — hai trường hợp cho cùng triệu chứng. Phân biệt chúng đòi phân tích chính mô hình, và đó là việc phải làm bằng tay một lần cho mỗi cấu hình cảm biến. Không có công cụ tự động nào làm hộ, và việc đó vẫn còn là một khoảng trống nghiên cứu — xem [D/22](#).

Một câu: Trước khi hỏi bộ ước lượng trả lời đúng không, hãy hỏi dữ liệu có chứa câu trả lời không.

5.7 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *SfM đơn mắt có đúng bảy bậc tự do không quan sát được* — bị bác bỏ nếu ai đó chỉ ra một phép biến đổi thứ tám, độc lập với bảy phép kia, giữ nguyên mọi phép đo trong mô hình chuẩn (cảnh tĩnh, camera đã hiệu chuẩn nội, không có cảm biến khác). Nó cũng bị bác bỏ theo hướng ngược lại nếu ai đó chỉ ra rằng một trong bảy hướng thực ra *có* ảnh hưởng tới một phép đo. Tôi đã kiểm tra thuận bằng cách thay trực tiếp vào phương trình chiếu ở mục 3A (cửa a) và đối chiếu [\[28\]](#) cùng [\[2\]](#) (cửa c).

Mệnh đề thứ hai — *ba cách cổ định gauge cho cùng quỹ đạo tương đối* — bị bác bỏ nếu một thí nghiệm có kiểm soát cho thấy chúng cho ba hình dạng quỹ đạo khác nhau sau khi căn chỉnh $Sim(3)$, trên một bài toán không có suy biến do chuyển động. [\[40\]](#) khảo sát đúng câu hỏi này trong ngữ cảnh VIO; tôi chưa đọc tới lượt ba nên chưa kiểm được liệu kết luận của họ có điều kiện nào mà tôi đang bỏ sót không.

Mệnh đề thứ ba — *thông tin giả làm bộ ước lượng quá tự tin* — đã được thiết lập cho bộ lọc [\[32\]](#), [\[43\]](#). Nó bị bác bỏ trong ngữ cảnh của sổ trượt tối ưu hoá nếu một thí nghiệm Monte-Carlo cẩn thận cho thấy ANEES nằm trong dải chấp nhận trong thời gian dài mà *không* nhờ nhiều quá trình được nói rộng hay một prior bù vào. Đây là mệnh đề dễ kiểm nhất trong ba, và tôi chưa kiểm.

Một mệnh đề mềm hơn nhưng đáng nêu: chương này khẳng định rằng *chẩn đoán qua observability có giá trị thực dụng cao hơn việc đổi bộ giải*. Điều đó bị bác bỏ nếu trong thực tế, phần lớn ca hỏng mà kỹ sư gặp bắt nguồn từ lỗi cài đặt và lỗi số học chứ không từ thiếu kích thích. Tôi không có số liệu để khẳng định tỉ lệ, và đây là chỗ tôi phát biểu dựa trên cấu trúc của vấn đề chứ không dựa trên thống kê (*tôi suy ra, chưa đối chiếu nguồn*). [D/21](#) đề nghị cách thu số liệu đó từ fleet analytics, và đó là cách duy nhất tôi biết để trả lời câu này một cách trung thực.

5.8 Kết nối

Chương này đứng ở chỗ Phần A gấp lại, và gần như mọi chương sau đều gọi ngược về đây.

Nổi lên trên. [A/01-hinh-hoc-da-thi](#) cung cấp bốn cấu hình suy biến cụ thể mà mục 5A tổng quát hoá; điều đáng chú ý là chương 1 cho bốn phép thử riêng còn chương này cho *một* phép thử duy nhất thay thế được cả bốn. [A/02-mo-hinh-camera](#) cung cấp một ví dụ sớm ở mục 4B của nó: tương quan giữa các hệ số méo bậc cao là một hướng gần suy biến, và “hiệu chuẩn ba lần ra ba kết quả” chính là phép thử (1) ở khối *Cam bẫy*. [A/03-lie-group-va-toi-uu-tren-da-tap](#) cho ngôn ngữ để “co giãn cả thế giới” viết thành một phép toán trên $Sim(3)$. [A/04-uoc-luong-map-va-bundle-adjustment](#) cho ma trận Λ ; chương này chỉ làm một việc là hỏi chỗ nó suy biến.

Nổi xuống dưới, theo thứ tự cấp bách. [B/10-khoi-tao](#) là nơi mục 5 trở thành điều kiện chấp nhận: bộ khởi tạo tốt không phải bộ luôn trả về kết quả mà là bộ biết *từ chối* khi kích thích chưa đủ. [C/14-hieu-chuan-nhu-mot-nhanh-cua-slam](#) nhận Bảng 5.2: mọi thảo luận về hiệu chuẩn trực tuyến đều phải kèm điều kiện kích thích, nếu không nó chỉ là mở rộng trạng thái một cách mù quáng. [B/09-paradigm-uoc-luong-back-end](#) nhận mục 6: FEJ và các biến thể tồn tại *chỉ vì* bệnh bất nhất mô tả ở đây. [C/18-danh-gia-va-phuong-phap-luan](#) nhận hai thứ: quy tắc căn chỉnh theo đúng số bậc tự do, và NEES như thước đo bổ sung cho ATE — hai con số đo hai thứ khác nhau, ATE hỏi bạn đúng bao nhiêu, NEES hỏi bạn có thành thật về việc mình đúng bao nhiêu không.

Nổi xa hơn: [D/19-ben-vung-trong-the-gioi-thuc](#) và [D/22-huong-nghien-cuu-con-trong](#) đều lấy chương này làm nền. Hướng nghiên cứu số một — hiệu chuẩn trực tuyến có ý thức về observability — chính là việc cài Bảng 5.2 thành một cơ chế chạy theo thời gian thực, và nó còn thô sơ trong mọi hệ thống hiện có.

Nổi ngang: `../IMU_Fusion/` chương 5 là chương sinh đôi của chương này, viết cho trường hợp có quán tính. Bốn bậc tự do của VIO, FEJ, InEKF và NEES được nói kỹ ở đó; ở đây tôi chỉ nhắc kết quả và trở sang, theo quy tắc một nguồn duy nhất của kho.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Tự dẫn lại bảy bậc tự do của SfM đơn mắt bằng cách thay phép biến đổi $Sim(3)$ vào phương trình chiếu, không nhìn bài.
2. Vì sao số bậc tự do gauge giảm từ bảy xuống sáu khi thêm stereo, và xuống bốn khi thêm IMU? Trả lời bằng ngôn ngữ “cảm biến nào đo được gì”.
3. Ba loại suy biến — quy ước, chuyển động, số học — phân biệt bằng hai phép thử nào, và mỗi loại chữa thế nào?
4. Một bộ ước lượng cho tỉ lệ rất ổn định qua hai mươi lần chạy. Vì sao điều đó *không* chứng minh tỉ lệ quan sát được, và phép thử nào mới chứng minh được?
5. Nếu chỉ xoay quanh đúng một trục trong suốt quá trình hiệu chuẩn camera–IMU, thành phần nào của phép biến đổi ngoại không xác định được, và vì sao?
6. Bộ ước lượng tự tạo thông tin theo hướng không quan sát được bằng cơ chế nào, và vì sao hậu quả đi theo chiều nguy hiểm chứ không phải chiều an toàn?
7. Ba giả thiết ngầm của chương là cảnh tĩnh, camera đã hiệu chuẩn nội, và nhiễu Gauss. Với mỗi giả thiết, bỏ nó thì kết luận nào thay đổi, và bạn có nhận ra lúc nó bị vi phạm không?

Bất định và lan truyền sai số (Uncertainty and Error Propagation)

PHỤ THUỘC A/01-hinh-hoc-da-thi (quan hệ $\sigma_z \approx z^2 \sigma_d / (bf)$, hình dạng điều xi gà); A/03-lie-group-va-toi-uu-tren-da-tap (Adjoint, và hiệp phương sai trên đa tạp là hiệp phương sai của *nhiều loạn*); A/04-uoc-luong-map-va-bundle-adjustment (ma trận thông tin Λ); A/05-observability-gauge-va-suy-bien (khi Λ suy biến thì hiệp phương sai theo hướng đó vô nghĩa — điều kiện tiên quyết cho cả chương này).

TÓM TẮT

Bốn chương trước cho một ước lượng. Chương này hỏi câu tiếp theo, và là câu quyết định hệ thống có dùng được trong sản phẩm hay không: *ước lượng ấy chính xác tới đâu, và con số bất định ta in ra có trung thực không?* Nội dung có bốn phần. Một: hiệp phương sai đến từ đâu — $\Sigma = \Lambda^{-1}$ quanh nghiệm — và vì sao điều đó chỉ đúng dưới những giả thiết cụ thể. Hai: lan truyền sai số qua các phép biến đổi, với lưu ý rằng trên đa tạp phải lan truyền qua nhiễu loạn chứ không qua tham số, và Adjoint là công cụ. Ba: chuỗi lan truyền cụ thể của Visual SLAM — từ sai số pixel, qua sai số độ sâu, tới sai số tư thế — kèm các quan hệ tỉ lệ đáng thuộc lòng vì chúng quyết định mọi lựa chọn thiết kế phần cứng. Bốn: bệnh mãn tính của lĩnh vực, là hiệp phương sai được báo cáo nhỏ hơn sự thật một cách hệ thống; nguyên nhân, cách đo bằng NEES, và vì sao nó nguy hiểm hơn việc kém chính xác. Nếu chỉ nhớ một điều: *một hệ thống biết mình sai bao nhiêu thì dùng được trong sản phẩm; một hệ thống chính xác hơn nhưng im lặng khi sai thì không.*

KIẾN THỨC CẦN CÓ TRƯỚC

- **hiệp phương sai (covariance)** — ma trận Σ mô tả mức tán và tương quan của sai số. Đường chéo là phương sai từng thành phần, ngoài đường chéo là tương quan — và phần ngoài đường chéo thường mang nhiễu loạn thông tin hơn đường chéo.
- **ellipsoid bất định** — tập $\{\mathbf{x} : \mathbf{x}^\top \Sigma^{-1} \mathbf{x} \leq c\}$, hình dạng hình học của Σ . Trục chính là vectơ riêng, độ dài trục tỉ lệ căn bậc hai giá trị riêng.
- **hiệp phương sai biên (marginal covariance)** — bất định của một biến khi đã tính tới việc mọi biến khác cũng chưa biết. Lấy bằng khối con của Λ^{-1} — *không* phải nghịch đảo của khối con của Λ .
- **hiệp phương sai điều kiện (conditional covariance)** — bất định của một biến khi *giả sử* mọi biến khác đã biết chính xác. Lấy bằng nghịch đảo khối con của Λ . Luôn nhỏ hơn hoặc bằng biên, nên nhầm hai cái là nguồn của việc quá tự tin.
- **lan truyền bậc nhất** — xấp xỉ $\Sigma_y \approx J \Sigma_x J^\top$ cho $\mathbf{y} = f(\mathbf{x})$, với J là Jacobian. Đúng khi f gần tuyến tính trên vùng bất định; sai khi f cong mạnh trên vùng đó.
- **NEES** — *normalized estimation error squared*, đại lượng $\varepsilon = \tilde{\mathbf{x}}^\top \Sigma^{-1} \tilde{\mathbf{x}}$ với $\tilde{\mathbf{x}}$ là sai số so với chân trị. Nếu Σ trung thực thì $\mathbb{E}[\varepsilon]$ đúng bằng số chiều của $\tilde{\mathbf{x}}$.
- **quá tự tin (overconfidence)** — trạng thái Σ được báo nhỏ hơn hiệp phương sai thật của sai số. Biểu hiện bằng NEES lớn hơn số chiều. Đây là bệnh mãn tính của SLAM và nguy hiểm hơn hẳn chiều ngược lại.

CÂU HỎI MỞ ĐẦU

- Q1. Bộ giải in ra tư thế kèm một ma trận 6×6 . Nếu ba điều kiện phải đúng thì ma trận đó mới có nghĩa, và nói mỗi điều kiện bị vi phạm ra sao trong SLAM thật.
- Q2. Bạn cần bất định của *một* landmark. Lấy khối 3×3 của nó trong Λ rồi nghịch đảo — rẻ và hiển nhiên. Vì sao kết quả đó sai, sai theo chiều nào, và đại lượng đúng là gì?
- Q3. Sai số độ sâu của stereo tăng theo *bình phương* khoảng cách. Dẫn lại quan hệ đó, rồi nói nó quyết định thế nào tới việc chọn đường cơ sở và tiêu cự khi thiết kế sản phẩm.
- Q4. Hệ của bạn báo sai số vị trí ± 2 cm; đo với chân trị thì sai thật là ± 20 cm. Nếu bốn cơ chế khác nhau có thể gây ra chuyện đó, và nêu phép thử phân biệt chúng — không dùng thêm chân trị nào.
- Q5. NEES của bạn gấp bốn lần số bậc tự do. Phản xạ tự nhiên là nói hiệp phương sai lên bốn lần và con số sẽ đẹp lại. Vì sao phản xạ đó có thể đang giấu một lỗi nghiêm trọng hơn nhiều?

6.1 Đặt vấn đề

Năm chương trước dựng một cỗ máy trả về ước lượng. Chương này hỏi câu mà người dùng cỗ máy ấy thật sự cần trả lời, và nó không phải câu “ước lượng có đúng không” — vì nếu biết được điều đó thì đã không cần ước lượng.

Câu đúng là: *ta biết ước lượng này chính xác tới đâu?* Và nó không phải một câu hỏi học thuật. Một robot quyết định có đi qua khe cửa rộng 80 cm hay không cần biết sai số vị trí của nó là ± 2 cm hay ± 30 cm — và với con số thứ hai thì hành vi đúng là dừng lại, chứ không phải đi qua và hy vọng. Một hệ AR quyết định có vẽ vật ảo lên bàn hay không cần biết bất định của mặt bàn. Một hệ thống quyết định có chấp nhận một loop closure hay không cần biết bất định của tư thế tương đối để đặt ngưỡng kiểm định hình học (B/11).

Ai gặp bài toán này? Về nguyên tắc là mọi người; trên thực tế là rất ít người, và đó chính là vấn đề. Phần lớn công trình SLAM báo cáo ATE và dừng ở đó. Rất ít công trình báo cáo liệu hiệp phương sai của họ có *thực* hay không, và khi có ai đo thì kết quả thường không đẹp. Đây là một trong những khoảng trống thật của lĩnh vực, và D/22 xếp nó là hướng số hai còn nhiều chỗ trống.

Trước khi có khung xác suất, cách xử lý phổ biến là kinh nghiệm: “hệ này thường sai khoảng mười centimet”. Cách đó hỏng ở ba chỗ. Nó không phân biệt các *hướng* — nhưng như A/01 đã chỉ ra, bất định của một điểm triangulate có dạng điều xì gà, ngắn ngang và rất dài dọc, nên một con số vô hướng là vô nghĩa. Nó không thích ứng — sai số thật thay đổi hàng chục lần giữa lúc nhiều đặc trưng và lúc ít. Và nó không lan truyền được sang các phép tính sau.

Khung xác suất giải quyết cả ba, và nó gần như miễn phí: ma trận thông tin Λ ở A/04 đã có sẵn như sản phẩm phụ của việc giải. Đó là điều làm chương này đặc biệt: thứ ta cần đã nằm trong tay, chỉ cần biết cách đọc nó và biết khi nào không nên tin nó.

Và đó là chỗ cái khó thật sự nằm. Hiệp phương sai đúng đòi hỏi mô hình đúng: trọng số đúng, nhiễu độc lập, tuyến tính hoá hợp lệ, không có sai lệch có hệ thống, và không có hướng suy biến. Trong SLAM thật, *mọi* điều kiện ấy đều bị vi phạm ít nhiều. Hệ quả là con số Λ^{-1} có xu hướng nhỏ hơn sự thật một cách có hệ thống — và vì nó nhỏ theo hướng lạc quan, nó là loại sai nguy hiểm.

TRỰC GIÁC

Hình dung hàm mục tiêu MAP như cái bát ở A/05. Đáy bát là ước lượng. Nhưng có một câu hỏi thứ hai quan trọng không kém đáy nằm ở đâu: *bát rộng hay hẹp?*

Một cái bát hẹp và dốc nghĩa là chỉ cần lệch một chút khỏi đáy thì phần dư đã tăng mạnh — dữ liệu ràng buộc chặt, ta chắc chắn. Một cái bát nông và rộng nghĩa là có thể đi khá xa mà phần dư gần như không đổi — dữ liệu ràng buộc lỏng, ta không chắc.

Độ cong của bát chính là Λ , và độ rộng là Λ^{-1} . Đó là toàn bộ ý tưởng của mục 2.

Ba hệ quả đáng giữ. Thứ nhất, bát cong *khác nhau theo hướng*, nên bất định là một ellipsoid chứ không phải một con số — và với điểm triangulate, ellipsoid ấy dẹt tới mức một con số vô hướng là vô nghĩa.

Thứ hai, bát có thể *phẳng hoàn toàn* theo một hướng — đó là **A/05**, và lúc đó Λ^{-1} không tồn tại. Nghĩa là chương này phụ thuộc hoàn toàn vào chương trước: *không được hỏi “rộng bao nhiêu” trước khi chắc rằng bát có đáy*.

Thứ ba, và đây là chỗ tinh vi nhất: độ cong của bát mô tả *hàm mục tiêu*, không mô tả *sự thật*. Nếu hàm mục tiêu sai — trọng số sai, mô hình sai, một outlier lọt vào — thì bát vẫn có một độ cong hoàn toàn xác định, và ta vẫn tính ra một hiệp phương sai trông đẹp. Nó chỉ không liên quan gì tới sai số thật. Toàn bộ mục 5 là về khoảng cách giữa hai thứ đó.

6.2 Hiệp phương sai từ ma trận thông tin

A. Kết quả cơ bản

Quanh nghiệm $\hat{\theta}$, hậu nghiệm xấp xỉ Gauss với

$$p(\theta \mid \mathbf{z}) \propto \exp\left(-\frac{1}{2}(\theta \ominus \hat{\theta})^\top \Lambda (\theta \ominus \hat{\theta})\right),$$

nên

$$\Sigma = \Lambda^{-1} = \left(\sum_k H_k^\top \Omega_k H_k\right)^{-1}. \quad (6.1)$$

Suy dẫn trực tiếp từ định nghĩa Gauss (cửa a), đối chiếu [24, §4] (cửa c).

Ba giả thiết nằm ẩn trong công thức này, và đây là câu trả lời cho câu hỏi mở đầu thứ nhất. *Một*: hậu nghiệm xấp xỉ Gauss quanh nghiệm, tức hàm mục tiêu xấp xỉ bậc hai trên vùng bất định — sai khi bất định lớn và mô hình cong mạnh, chẳng hạn với điểm rất xa. *Hai*: trọng số Ω_k đúng — tức ta thật sự biết σ của mỗi phép đo, và mục 5 sẽ chỉ ra ta thường không biết. *Ba*: Λ khả nghịch, tức không có hướng không quan sát được — **A/05**. Vi phạm giả thiết ba thì công thức không chỉ kém chính xác mà *không tồn tại*.

B. Biên so với điều kiện: cái bẫy đất nhất của chương

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là lỗi tôi cho là phổ biến nhất liên quan tới hiệp phương sai.

Muốn bất định của biến θ_a trong một bài toán có cả θ_b . Có hai đại lượng khác nhau:

$$\Sigma_a^{\text{dk}} = \Lambda_{aa}^{-1} \quad \text{so với} \quad \Sigma_a^{\text{biên}} = [\Lambda^{-1}]_{aa} = (\Lambda_{aa} - \Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba})^{-1}.$$

Cái thứ nhất là *điều kiện*: bất định của θ_a *giả sử* ta biết θ_b chính xác. Cái thứ hai là *biên*: bất định thật, khi θ_b cũng chưa biết.

Vì $\Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba}$ nửa xác định dương, ta luôn có $\Sigma_a^{\text{biên}} \succeq \Sigma_a^{\text{dk}}$. Nghĩa là dùng điều kiện thay cho biên *luôn* cho kết quả quá lạc quan — và mức lạc quan có thể lớn tùy ý khi hai biến tương quan mạnh, đúng tình huống của tư thế và landmark trong SLAM.

Cái khó thực hành: Λ thưa nhưng Λ^{-1} đặc, nên không thể nghịch đảo cả ma trận cho bài toán lớn. Cách đúng là **ngịch đảo thưa**: tính chỉ những phần tử của Λ^{-1} tương ứng với phần khác không của thừa số Cholesky, bằng một truy hồi ngược. Kaess và Dellaert [44] trình bày thuật toán cho ma trận căn bậc hai; chi phí tỉ lệ với số phần tử khác không của thừa số, tức chấp nhận được.

CẠM BÃY

Ngịch đảo khối con $\neq$ khối con của nghịch đảo. Đây là lỗi phổ biến nhất về hiệp phương sai và nó luôn sai theo chiều nguy hiểm: kết quả quá nhỏ, tức quá tự tin.

Ví dụ số tối thiểu để thấy rõ. Giả sử $\Lambda = \begin{pmatrix} 2 & 1,8 \\ 1,8 & 2 \end{pmatrix}$, tức hai biến vô hướng tương quan mạnh.

Điều kiện: $\Lambda_{aa}^{-1} = 1/2 = 0,5$.

Biên: $\det \Lambda = 4 - 3,24 = 0,76$, nên $\Lambda^{-1} = \frac{1}{0,76} \begin{pmatrix} 2 & -1,8 \\ -1,8 & 2 \end{pmatrix}$, và $[\Lambda^{-1}]_{aa} = 2/0,76 \approx 2,63$.

Chênh lệch hơn *năm lần* về phương sai, tức hơn hai lần về độ lệch chuẩn. Và tương quan 0,9 giữa một tư thế và một landmark nó nhìn thấy hoàn toàn bình thường trong SLAM — thực tế nó là quy luật, vì hai đại lượng ấy chỉ được xác định *cùng nhau*. (Ví dụ tự đặt, tính tay, cửa kiểm chứng b.)

Quy tắc: **luôn dùng hiệp phương sai biên** trừ khi bạn thật sự đang hỏi một câu hỏi có điều kiện. Và khi ai đó đưa bạn một hiệp phương sai, hãy hỏi cho rõ đó là loại nào — nếu họ không biết câu hỏi nghĩa là gì thì gần như chắc chắn đó là điều kiện.

C. Hiệp phương sai trên đa tạp

Theo A/03, tư thế sống trên đa tạp, nên hiệp phương sai của nó phải hiểu là hiệp phương sai của *nhiều loạn*:

$$T_{\text{thật}} = \hat{T} \boxplus \xi, \quad \xi \sim \mathcal{N}(\mathbf{0}, \Sigma_{6 \times 6}).$$

Ma trận 6×6 này sống trong đại số Lie, không trong không gian tham số. Ba hệ quả thực hành.

Một: nó phụ thuộc vào quy ước trái hay phải, và chuyển giữa hai quy ước phải dùng Adjoint: $\Sigma_{\text{trái}} = \text{Ad}_T \Sigma_{\text{phải}} \text{Ad}_T^\top$. Quên chuyển thì con số sai mà không có dấu hiệu nào.

Hai: lan truyền qua một phép biến đổi cứng $T' = T_0 T$ dùng $\Sigma' = \text{Ad}_{T_0} \Sigma \text{Ad}_{T_0}^\top$, không phải một phép cộng ma trận nào đó.

Ba: xấp xỉ Gauss trên đa tạp chỉ tốt khi bất định *nhỏ so với độ cong*. Với bất định xoay cỡ vài độ thì hoàn toàn ổn; với bất định cỡ vài chục độ thì phân bố thật “quấn quanh” đa tạp và Gauss không mô tả nổi. Trường hợp sau xuất hiện lúc mất bám hoặc lúc relocalization thất bại — tức đúng lúc ta cần hiệp phương sai nhất.

6.3 Lan truyền sai số: chuỗi của Visual SLAM

A. Quy tắc bậc nhất

Cho $\mathbf{y} = f(\mathbf{x})$ với $\mathbf{x} \sim \mathcal{N}(\hat{\mathbf{x}}, \Sigma_x)$. Khai triển bậc nhất quanh $\hat{\mathbf{x}}$:

$$\Sigma_y \approx J \Sigma_x J^\top, \quad J = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}}.$$

Đơn giản, và nó là công cụ chính. Điều đáng nhớ là *khi nào nó sai*: khi f cong mạnh trên vùng bất định. Trong SLAM có đúng một chỗ điều này xảy ra thường xuyên và gây hại, và đó là quan hệ độ sâu–disparity ở mục B.

B. Pixel sang độ sâu: quan hệ đáng thuộc lòng

Từ A/01 mục 4C, với stereo đã hiệu chỉnh:

$$z = \frac{bf}{d} \implies \sigma_z \approx \frac{z^2}{bf} \sigma_d \quad (6.2)$$

Đây là quan hệ quan trọng nhất trong chương xét về hệ quả thiết kế, và nó là câu trả lời cho câu hỏi mở đầu thứ ba. Đọc nó theo ba chiều:

Theo z : sai số tăng theo *bình phương* khoảng cách. Gấp đôi khoảng cách thì sai số gấp bốn. Đây là lý do mọi cảm biến độ sâu dựa trên tam giác đều có một tầm hữu ích khá rõ ràng, ngoài tầm đó thì số liệu vẫn có nhưng vô dụng cho đo đạc.

Theo b : sai số tỉ lệ nghịch với đường cơ sở. Gấp đôi b thì sai số giảm một nửa — nhưng cái giá là vùng chồng lấn giữa hai camera giảm, và việc khớp điểm khó hơn vì hai góc nhìn khác nhau nhiều hơn. Đây là một đánh đổi thiết kế thật, quyết định ở D/21.

Theo f : sai số tỉ lệ nghịch với tiêu cự, nhưng tiêu cự lớn nghĩa là trường nhìn hẹp. Lại một đánh đổi, và nó tương tác với lựa chọn mô hình camera ở A/02.

Nhưng có một chi tiết mà công thức (6.2) *giấu đi*, và nó quan trọng hơn bản thân công thức. Vì $z = bf/d$ là hàm *ngịch đảo*, lan truyền bậc nhất chỉ đúng khi $\sigma_d \ll d$. Khi d nhỏ — tức điểm ở xa — thì σ_d/d không còn nhỏ, hàm cong mạnh trên vùng bất định, và phân bố của z trở nên *lệch*: đuôi phía xa dài hơn đuôi phía gần rất nhiều. Ở giới hạn $d \rightarrow \sigma_d$, phân bố của z không còn có kỳ vọng hữu hạn.

Hệ quả: với điểm xa, σ_z *không mô tả được bất định thật*, và một ellipsoid Gauss trong toạ độ XYZ là một mô tả sai. Cách đúng là làm việc trong biến $\rho = 1/z$, nơi quan hệ trở lại tuyến tính ($\rho = d/(bf)$) và phân bố trở lại gần Gauss. Đây là lập luận đầy đủ đằng sau tham số hoá nghịch đảo độ sâu [7], và nó là một trong những chỗ mà một chi tiết thống kê quyết định một lựa chọn kiến trúc ở B/08.

C. Ví dụ nhỏ tính tay: ngân sách sai số

Giả sử một robot với stereo $b = 12$ cm, $f = 400$ px, $\sigma_d = 0,2$ px (tính chính subpixel tốt). Nó nhìn thấy đặc trưng ở các khoảng cách khác nhau.

Từ (6.2): ở $z = 1$ m, $\sigma_z = 1/(0,12 \times 400) \times 0,2 \approx 0,004$ m. Ở $z = 3$ m: $9/48 \times 0,2 \approx 0,038$ m. Ở $z = 8$ m: $64/48 \times 0,2 \approx 0,27$ m. Ở $z = 15$ m: $225/48 \times 0,2 \approx 0,94$ m.

Còn sai số *ngang* thì sao? Nó là $\sigma_\perp \approx (z/f) \sigma_u$, tuyến tính theo z chứ không bình phương. Với $\sigma_u = 0,2$ px: ở $z = 8$ m, $\sigma_\perp = 8/400 \times 0,2 = 0,004$ m.

So sánh ở $z = 8$ m: sai số dọc 0,27 m, sai số ngang 0,004 m. *Tỉ số gần bảy mươi lần*. Đó là điều xì gà mà A/01 mô tả bằng lời, giờ đã có con số. (Ví dụ tự đặt, tính tay, cửa b.)

Bài học thiết kế rút ra ngay và nó thật sự dùng được: *đừng vút điểm xa đi, và cũng đừng gán cho chúng một σ đẳng hướng*. Điểm ở 15 m gần như vô dụng cho việc đo khoảng cách nhưng vẫn xác định *hướng* với độ chính xác vài milimet ở khoảng cách đó, tức vài phần vạn radian. Đúng như A/03 mục 5 chỉ ra qua khối $[\mathbf{P}_c]_\times$: điểm xa ràng buộc xoay rất tốt. Cách xử lý đúng là tham số hoá nghịch đảo độ sâu và để hiệp phương sai phản ánh sự bất đối xứng — không phải lọc bỏ theo một ngưỡng khoảng cách, vốn là cách làm phổ biến và lãng phí.

6.4 Bệnh quá tự tin

A. Đo bằng NEES

Cách duy nhất biết hiệp phương sai có trung thực không là so với chân trị. Đại lượng chuẩn là NEES [38]:

$$\varepsilon = \tilde{\mathbf{x}}^\top \Sigma^{-1} \tilde{\mathbf{x}}, \quad \tilde{\mathbf{x}} = \mathbf{x}_{\text{thật}} \ominus \hat{\mathbf{x}}.$$

Nếu Σ trung thực thì $\mathbb{E}[\varepsilon] = \dim(\tilde{\mathbf{x}})$. Suy dẫn một dòng: với $\tilde{\mathbf{x}} \sim \mathcal{N}(\mathbf{0}, \Sigma)$,

$$\mathbb{E}[\tilde{\mathbf{x}}^\top \Sigma^{-1} \tilde{\mathbf{x}}] = \mathbb{E}[\text{tr}(\Sigma^{-1} \tilde{\mathbf{x}} \tilde{\mathbf{x}}^\top)] = \text{tr}(\Sigma^{-1} \Sigma) = \dim.$$

(Cửa a; đối chiếu [38], cửa c.)

Cách dùng: chạy nhiều lần Monte-Carlo, tính NEES trung bình (ANEES), so với số chiều. Lớn hơn nhiều nghĩa là quá tự tin; nhỏ hơn nhiều nghĩa là quá bảo thủ. Với sáu bậc tự do của một tư thế, ANEES nên gần 6; giá trị 24 nghĩa là độ lệch chuẩn bị đánh giá thấp khoảng hai lần.

Cái giá: cần chân trị. Nên trong thực tế NEES chỉ đo được trong mô phỏng hoặc trên bộ dữ liệu có hệ đo ngoài. Đây là một lý do chính đáng và hay bị bỏ qua để dựng mô phỏng, chuyện của C/18.

B. Bốn nguồn của quá tự tin

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và cả bốn nguồn xuất hiện lại trong các chương sau.

Một — phụ thuộc thống kê bị bỏ qua. Khung MAP giả định phép đo độc lập (A/04 mục 2A). Trong thực tế, các đặc trưng gần nhau chia nhau sai số hiệu chuẩn cục bộ; các quan sát liên tiếp của cùng một đặc trưng chia nhau sai số mô tả của nó. Thông tin bị *đếm thừa*, nên Λ quá lớn, nên Σ quá nhỏ.

Hai — sai lệch có hệ thống bị coi là nhiễu. Hiệu chuẩn sai một chút (A/02 mục 4C) tạo ra phần dư có thành phần *hằng*. Bình phương tối thiểu xử lý nó như nhiễu ngẫu nhiên và “khử” nó bằng trung bình, trong khi thực tế nó không khử được. Ước lượng lệch, mà hiệp phương sai lại không phản ánh độ lệch — vì hiệp phương sai chỉ đo độ tán, không đo độ lệch.

Ba — thông tin giả do tuyến tính hoá. Cơ chế ở A/05 mục 6: marginalization với các điểm tuyến tính hoá không nhất quán tạo ra thông tin không đến từ phép đo nào.

Bốn — outlier đã bị loại nhưng chưa tính vào. Sau RANSAC và M-estimator, ta có một tập inlier sạch. Nhưng ta *chọn* tập ấy dựa trên chính dữ liệu, và bất định của bước chọn đó không được đưa vào Σ . Hệ quả là ta hành xử như thể data association chắc chắn, trong khi nó không. Đây là nguồn khó định lượng nhất trong bốn.

C. Vì sao chiều lạc quan nguy hiểm hơn

Một điểm đáng dừng lại vì nó định hình cách thiết kế hệ thống.

Quá bảo thủ — báo hiệp phương sai lớn hơn thật — làm hệ thống kém hiệu quả: nó từ chối loop closure hợp lệ, nó đi chậm hơn cần thiết, nó yêu cầu nhiều dữ liệu hơn. Khó chịu, nhưng *an toàn*.

Quá tự tin làm hệ thống đưa ra quyết định sai một cách tự tin: chấp nhận loop closure sai và gập cả bản đồ (B/11), đi qua khe hẹp mà nó không thật sự biết mình đang ở đâu, khai báo relocalization thành công khi không phải (B/12). Trong mỗi trường hợp, hậu quả không phải một ước lượng kém mà là một *hỏng hóc*.

Hệ quả thiết kế: khi không chắc, *nói hiệp phương sai*. Điều này nghe phản khoa học — ta đang cố tình báo cáo một con số ta biết là sai — và nó đúng là một sự thoả hiệp. Nhưng nó là sự thoả

hiệp đúng, vì phân bố hậu quả không đối xứng. Điều kiện là phải *ghi lại* rằng mình đã nói và nói bao nhiêu; nói lằng lẽ là cách nhanh nhất để một hệ số $\times 5$ tùy tiện trở thành một hằng số bất khả xâm phạm mà ba năm sau không ai biết từ đâu ra.

CẠM BẦY

ANEES lớn không tự động nghĩa là hiệp phương sai quá nhỏ. Đây là cái bẫy quan trọng nhất của mục này, và là câu trả lời cho câu hỏi mở đầu thứ năm.

NEES giả định mô hình *đúng dạng* và chỉ có thể sai *độ lớn*. Nếu bạn có một sai lệch có hệ thống — hiệu chuẩn sai, sai lệch thời gian chưa bù, một vật động chưa loại — thì phần dư mang thành phần có hệ thống và NEES lớn *vì lý do đó*, không phải vì Σ nhỏ.

Nói Σ lên bốn lần sẽ làm con số đẹp lại và **giấu lỗi gốc vào trọng số**. Ba tháng sau, khi hệ thống hỏng ở một điều kiện khác, không ai biết vì sao.

Phép thử phân biệt, và nó không cần thêm chân trị: **kiểm tương quan giữa phần dư và các đại lượng khác**. Phần dư tương quan với tốc độ góc thì là sai lệch thời gian hoặc rolling shutter. Tương quan với vị trí trong ảnh thì là mô hình camera ([A/02](#) mục 4). Tương quan với nhiệt độ thì là trôi hiệu chuẩn. Tăng đều theo thời gian chạy thì là một biến chưa được ước lượng. Không tương quan với gì cả — lúc đó, và chỉ lúc đó, mới thật sự là chuyện trọng số.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
GTSAM [27]	Marginals tính hiệp phương sai biên từ Bayes tree	Cài đặt của mục 2B — chú ý API phân biệt rõ biên và điều kiện
Ceres [35]	Covariance với nghịch đảo thừa; cảnh báo khi ma trận suy biến	Việc nó <i>từ chối</i> khi suy biến là đúng và hiếm; xem A/05
OpenVINS [42]	Mô phỏng có chân trị; tính NEES trực tiếp	Một trong ít hệ mã nguồn mở đo nhất quán chứ không chỉ đo độ chính xác
[7]	Nghịch đảo độ sâu cho phân bố gần Gauss với điểm xa	Nguồn của lập luận mục 3B — một chi tiết thống kê quyết định một lựa chọn kiến trúc
[44]	Khôi phục hiệp phương sai từ ma trận căn bậc hai	Thuật toán làm mục 2B khả thi cho bài toán lớn

MINI PROJECT SLAM · Đo NEES của chính bộ giải BA của bạn, rồi tìm ra vì sao nó lớn · 3 ngày

Mục tiêu	chuyển từ “hệ thống của tôi chính xác” sang “hệ thống của tôi trung thực về việc nó chính xác tới đâu” — hai khẳng định rất khác nhau.
Dữ liệu	tự sinh, vì bạn cần chân trị hoàn hảo. Sinh quỹ đạo, bản đồ, và quan sát với nhiều Gauss <i>đã biết</i> σ . Đây là chỗ mô phỏng không thay thế được.
Các bước	dùng bộ giải BA từ A/04 ; chạy 100 lần Monte-Carlo với nhiều khác nhau; mỗi lần tính sai số so với chân trị và hiệp phương sai biên; tính ANEES cho tư thế và so với 6; rồi <i>cố tình</i> đưa vào từng nguồn quá tự tin ở mục 4B một cách riêng lẻ — thêm tương quan giữa các quan sát gần nhau, thêm một sai lệch hiệu chuẩn nhỏ, thêm 2% outlier vượt qua bộ lọc — và đo lại ANEES sau mỗi lần.
Xong khi	bạn có một bảng: nguồn sai lệch $\times$ ANEES, cho thấy mỗi nguồn đẩy ANEES lên bao nhiêu. Bạn phải quan sát được rằng với dữ liệu <i>hoàn hảo</i> và trọng số <i>đúng</i> , ANEES rất gần 6 — nếu không thì có lỗi trong cài đặt hiệp phương sai, nhiều khả năng là nhầm biên với điều kiện, và đó là một kết quả đáng giá hơn cả bài tập. Thêm một cột: với mỗi

nguồn, ghi lại phần dư tương quan với đại lượng nào — đó là bảng chẩn đoán ở khối *Cạm bẫy*, do chính bạn dựng.

Nối với [C/18-danh-gia-va-phuong-phap-luan](#) biến quy trình này thành phương pháp đánh giá chuẩn; [D/21-tu-prototype-len-mass-production](#) biến nó thành giám sát sức khoẻ tại hiện trường, nơi không có chân trị và phải dùng đại lượng thay thế.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Ba điều kiện. (1) *Hậu nghiệm xấp xỉ Gauss* quanh nghiệm — bị vi phạm với điểm rất xa, nơi quan hệ độ sâu–disparity cong mạnh và phân bố lệch. (2) *Trọng số Ω đúng* — gần như luôn bị vi phạm, vì ta hiếm khi biết σ thật của mỗi phép đo, và mục 4B liệt kê bốn cách nó sai. (3) *Λ khả nghịch*, tức không có hướng không quan sát được — bị vi phạm trong mọi cấu hình suy biến ở [A/05](#), và khi đó công thức không chỉ kém chính xác mà *không tồn tại*.
- A2. Vì $[\Lambda^{-1}]_{aa} \neq [\Lambda_{aa}]^{-1}$. Nghịch đảo khối con cho hiệp phương sai *điều kiện* — bất định giả sử mọi biến khác đã biết chính xác. Đại lượng đúng là hiệp phương sai *biên*, khối con của nghịch đảo. Vì $\Sigma^{\text{biên}} \succeq \Sigma^{\text{dk}}$, sai luôn theo chiều quá lạc quan, và mức chênh lớn tùy ý khi tương quan mạnh — ví dụ ở khối *Cạm bẫy* cho chênh hơn năm lần với tương quan 0,9, vốn là chuyện bình thường giữa một tư thế và landmark nó nhìn thấy. Cách tính đúng cho bài toán lớn là nghịch đảo thừa [\[44\]](#).
- A3. Từ $z = bf/d$, vi phân cho $\partial z/\partial d = -bf/d^2 = -z^2/(bf)$, nên $\sigma_z \approx z^2 \sigma_d/(bf)$. Hệ quả thiết kế: tăng b giảm sai số tuyến tính nhưng giảm vùng chồng lấn và làm khớp điểm khó hơn; tăng f giảm sai số tuyến tính nhưng thu hẹp trường nhìn. Cả hai là đánh đổi thật phải quyết ở [D/21](#) dựa trên tầm làm việc mục tiêu. Quan trọng không kém: công thức này *giấu* một điều — nó là lan truyền bậc nhất qua một hàm nghịch đảo, nên nó mất hiệu lực khi σ_d/d không còn nhỏ, tức đúng với điểm xa. Lúc đó phân bố lệch và σ_z không mô tả nổi bất định thật; cách đúng là làm việc trong $\rho = 1/z$.
- A4. Bốn cơ chế: (1) phụ thuộc thống kê bị bỏ qua nên thông tin bị đếm thừa; (2) sai lệch có hệ thống bị xử lý như nhiễu — hiệp phương sai đo độ tán chứ không đo độ lệch; (3) thông tin giả do tuyến tính hoá không nhất quán trong marginalization; (4) bất định của bước chọn inlier không được tính vào. Phép thử phân biệt không cần chân trị: kiểm tương quan của phần dư với các đại lượng khác — với tốc độ góc thì là t_d hoặc rolling shutter; với vị trí trong ảnh thì là mô hình camera; với nhiệt độ thì là trôi hiệu chuẩn; tăng đều theo thời gian thì là một biến chưa ước lượng; không tương quan với gì thì mới là chuyện trọng số.
- A5. Vì NEES giả định mô hình *đúng dạng* và chỉ sai *độ lớn*. Với một sai lệch có hệ thống, phần dư mang thành phần có hệ thống và NEES lớn vì lý do đó chứ không phải vì Σ nhỏ. Nới Σ lên bốn lần làm con số đẹp lại và giấu lỗi gốc vào trọng số — ba tháng sau, khi hệ hỏng ở điều kiện khác, không ai biết vì sao. Cần chạy phép thử tương quan ở câu trên *trước* khi nới. Và nếu sau cùng vẫn quyết định nới, phải *ghi lại* đã nới bao nhiêu và vì sao; nới lặng lẽ là cách nhanh nhất biến một hệ số tùy tiện thành một hằng số bất khả xâm phạm.

NÓI LẠI THẬT ĐƠN GIẢN

Nếu ai đó hỏi bạn bây giờ mấy giờ và bạn nói “khoảng ba giờ chiều”, bạn đã nói hai điều: một con số, và một mức độ tin. Chữ “khoảng” mới là phần quan trọng. Nếu bạn vừa nhìn đồng hồ thì “khoảng” nghĩa là sai vài phút; nếu bạn vừa ngủ dậy trong một căn phòng không cửa sổ thì “khoảng” nghĩa là sai vài giờ. Cùng một câu trả lời, hai ý nghĩa hoàn toàn khác.

Cái máy cũng phải nói hai điều như vậy. Nó nói “tôi đang ở đây”, và nó phải nói thêm “tôi tin điều đó tới mức nào”. Phần thứ hai quyết định người ta dùng câu trả lời của nó ra sao: nếu máy nói “tôi ở đây, sai số hai centimet” thì robot đi qua khe cửa; nếu nói “sai số ba mươi centimet” thì robot nên dừng lại nhìn kỹ hơn.

Có ba chuyện cần biết về phần thứ hai.

Chuyện thứ nhất: mức tin *không giống nhau theo mọi hướng*. Cái máy nhìn thấy một cái cây ở xa: nó biết cây đó nằm về *hướng* nào cực kỳ chính xác, nhưng cây đó cách bao xa thì nó mơ hồ. Nói “sai

số một mét” là vô nghĩa — một mét theo hướng nào? Phải nói “sai số vài milimet ngang, một mét dọc”.

Chuyện thứ hai: mức tin ấy được tính ra từ chính phép tính tìm vị trí, gần như miễn phí. Không cần làm gì thêm. Nhưng — và đây là chuyện thứ ba, quan trọng nhất — *con số đó chỉ đúng nếu mọi giả định đằng sau nó đúng*. Nếu cái thước bạn dùng để đo bị cong mà bạn không biết, bạn sẽ đo rất nhất quán và rất tự tin, và rất sai. Máy cũng vậy, và nó không có cách nào tự phát hiện.

Hậu quả là một bệnh rất phổ biến: máy nói “tôi khá chắc” nhiều hơn mức nó có quyền nói. Và cái bệnh này nguy hiểm theo một chiều chứ không phải hai chiều. Một cái máy quá thận trọng thì chỉ chậm chạp và phiền phức. Một cái máy quá tự tin thì đâm vào tường — một cách rất quả quyết.

Chỗ không rút gọn được: chỗ không rút gọn được: cách duy nhất để *biết* máy có quá tự tin hay không là so nó với sự thật, và trong đời thật thì không có sự thật để so. Đó là lý do mô phỏng quan trọng hơn về ngoài, và là lý do việc tìm ra một cách tự chẩn đoán mà không cần chân trị vẫn là một bài toán mở — xem [D/22](#).

Một câu: Một con số không kèm mức tin thì chưa phải một phép đo; và một mức tin không được kiểm thì chỉ là một con số nữa.

6.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — $\Sigma = \Lambda^{-1}$ dưới ba giả thiết đã nêu — là suy dẫn toán học, nên nó chỉ sai nếu tôi dẫn sai. Cái *có thể* bị bác bỏ là mệnh đề ẩn rằng ba giả thiết đủ gần đúng để con số dùng được. Nó bị bác bỏ nếu một hệ đo được ANEES gần đúng số chiều trong thời gian dài trên dữ liệu thật — không phải mô phỏng — mà không nhờ một hệ số nói thủ công nào. Tôi chưa thấy công trình nào báo cáo điều đó, nhưng tôi cũng chưa khảo sát có hệ thống (*tôi suy ra, chưa đổi chiều nguồn*); nếu nó tồn tại thì nó sẽ làm thay đổi đánh giá của [D/22](#) về hướng nghiên cứu số hai.

Mệnh đề thứ hai — *hiệp phương sai biên luôn lớn hơn hoặc bằng điều kiện* — là một bất đẳng thức đại số, kiểm được bằng ví dụ số ở mục 2B (cửa b) và chứng minh được từ tính nửa xác định dương của phần bù Schur (cửa a).

Mệnh đề thứ ba — *SLAM quá tự tin một cách hệ thống* — là mệnh đề thực nghiệm và là mệnh đề tôi tin nhất trong chương, nhưng cũng là mệnh đề tôi có ít bằng chứng trực tiếp nhất. Nó được thiết lập cho EKF-SLAM [[32](#)], [[43](#)]. Với các hệ tối ưu hoá hiện đại, tôi *suy ra* từ bốn cơ chế ở mục 4B rằng bệnh vẫn còn, nhưng tôi không có số liệu về mức độ. Nó bị bác bỏ nếu một khảo sát trên nhiều hệ và nhiều bộ dữ liệu cho thấy ANEES phân bố quanh số chiều chứ không lệch lên. Đó là một thí nghiệm đáng làm và tôi không biết ai đã làm.

Mệnh đề thứ tư — *quá tự tin nguy hiểm hơn quá bảo thủ* — là một phán đoán về sự bất đối xứng của hậu quả, không phải một sự kiện đo được. Nó có thể sai trong ứng dụng mà chi phí của việc quá thận trọng rất cao — một robot phải hoàn thành nhiệm vụ trong thời gian giới hạn, hoặc một hệ phải chấp nhận đủ số loop closure để bản đồ không vỡ. Nếu rõ điều này vì quy tắc “khi không chắc thì nói” ở mục 4C không phải quy tắc phổ quát mà là một lựa chọn phụ thuộc ứng dụng.

6.6 Kết nối

Chương này khép lại Phần A, và nó phụ thuộc vào tất cả các chương trước theo một cách rất trực tiếp.

Nói lên trên. [A/05-observability-gauge-va-suy-bien](#) là điều kiện tiên quyết tuyệt đối: không được hỏi “bất định bao nhiêu” trước khi chắc rằng Λ khả nghịch. Hai chương này nên đọc liền và ôn cùng nhau. [A/04-uoc-luong-map-va-bundle-adjustment](#) cho Λ , và

kỹ thuật nghịch đảo thừa ở mục 2B dùng đúng thừa số Cholesky mà bộ giải đã tính. [A/03-lie-group-va-toi-uu-tren-da-tap](#) cho Adjoint và lời nhắc rằng hiệp phương sai trên đa tạp là của nhiễu loạn. [A/01-hinh-hoc-da-thi](#) cho quan hệ độ sâu, và chương này biến nó thành một ngân sách sai số có số. [A/02-mo-hinh-camera](#) cho một trong bốn nguồn quá tự tin: sai lệch hiệu chuẩn bị xử lý như nhiễu.

Nổi xuống dưới. [B/08-mo-hinh-phan-du](#) nhận lập luận ở mục 3B: tham số hoá nghịch đảo độ sâu tồn tại vì phân bố trong biến z lệch còn trong biến $1/z$ thì không. [B/11-loop-closure-va-nhat-quan-toan-cuc](#) dùng hiệp phương sai để đặt ngưỡng kiểm định hình học, và bệnh quá tự tin ở mục 4 là lý do ngưỡng ấy phải bảo thủ hơn lý thuyết. [B/12-relocalization](#) có cùng vấn đề ở dạng gắt hơn: một reloc sai có hại hơn không reloc, nên ngưỡng chấp nhận phải dựa trên một hiệp phương sai đáng tin. [C/15-hop-nhat-da-cam-bien](#) là nơi hiệp phương sai đúng trở thành điều kiện bắt buộc chứ không phải điều tốt nên có: khi ghép nhiều nguồn, trọng số tương đối giữa chúng *chính là* tỉ số hiệp phương sai, và sai một nguồn thì cả hệ lệch.

[C/18-danh-gia-va-phuong-phap-luan](#) là chương mở rộng trực tiếp: nó đặt NEES bên cạnh ATE và lập luận rằng hai con số đo hai thứ khác nhau, không thay thế nhau. [D/21-tu-prototype-len-mass-production](#) là chỗ chương này gặp thực tế khắc nghiệt nhất: tại hiện trường không có chân trị, nên phải tìm đại lượng thay thế — số inlier, số điều kiện, vết hiệp phương sai, tỉ lệ tracking — và chương 21 bàn về việc chọn chúng. [D/22-huong-nghien-cuu-con-trong](#) xếp “bất định đáng tin” là hướng số hai, và lý do nằm trọn trong mục 4 của chương này.

Nổi ngang: `../IMU_Fusion/` chương 5 bàn NEES và tính nhất quán cho trường hợp quán tính, với FEJ và InEKF là hai cách chữa; `../marker/` chương 5 làm ngân sách sai số pixel-sang-tư thế cho bài toán docking, nơi yêu cầu là milimet nên mọi thứ trong chương này phải làm chặt hơn một bậc.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba giả thiết của $\Sigma = \Lambda^{-1}$, và với mỗi cái nói nó bị vi phạm thế nào trong SLAM thật và hậu quả theo chiều nào.
2. Vì sao nghịch đảo khối con khác khối con của nghịch đảo, và sai theo chiều nào? Tự làm lại ví dụ số 2×2 với tương quan 0,9.
3. Tự dẫn lại $\sigma_z \approx z^2 \sigma_d / (bf)$, rồi tính σ_z và $\sigma_\perp$ ở $z = 10$ m với $b = 12$ cm, $f = 400$ px, $\sigma = 0,2$ px. Tỉ số bao nhiêu, và nó nói gì về hình dạng ellipsoid?
4. Vì sao lan truyền bậc nhất mất hiệu lực với điểm xa, và tham số hoá nào chữa được? Trả lời bằng ngôn ngữ hình dạng phân bố.
5. Nêu bốn nguồn của quá tự tin, và với mỗi nguồn nêu một phép thử phát hiện không cần chân trị.
6. ANEES bằng ba lần số chiều. Nêu hai cách giải thích hoàn toàn khác nhau và phép đo phân biệt chúng.
7. Vì sao quá tự tin nguy hiểm hơn quá bảo thủ, và nêu một ứng dụng mà điều ngược lại đúng?



FRONT-END VÀ BACK-END

TÓM TẮT

Phần B là phần dài nhất và là chỗ lĩnh vực này thật sự sống. Nó chia đôi theo đúng ranh giới mà mọi hệ thống SLAM tự chia: front-end lo việc *tìm ra phép đo* từ pixel, back-end lo việc *ghép phép đo thành trạng thái*. Sáu chương đi theo thứ tự một khung hình được xử lý: trích và khớp đặc trưng (chương 7), chọn dạng phân dư (chương 8), chọn kiến trúc ước lượng (chương 9), khởi động hệ thống từ số không (chương 10), phát hiện và đóng vòng khi quay lại chỗ cũ (chương 11), và tìm lại mình khi lạc (chương 12). Nếu chỉ nhớ một điều: *front-end quyết định hệ thống hỏng theo kiểu nào, back-end quyết định nó chính xác tới đâu* — và trong triển khai thật, kiểu hỏng quan trọng hơn độ chính xác.

Ranh giới front-end / back-end không phải một quy ước tùy tiện mà là một đường phân chia có cơ sở. Front-end giải bài toán **data association** — pixel nào ở khung này tương ứng với pixel nào ở khung kia, hoặc với landmark nào trong bản đồ. Đó là một bài toán *tổ hợp*, có lời giải đúng hoặc sai, và sai thì sai thảm họa. Back-end giải bài toán **ước lượng** — cho các tương ứng, tìm trạng thái tốt nhất. Đó là bài toán *liên tục*, có lời giải chính xác hơn hoặc kém chính xác hơn, và sai thì sai từ từ.

Hai loại sai ấy đòi hai loại công cụ, và đó là lý do sâu xa của sự phân chia. Chống lỗi tổ hợp cần dư thừa và kiểm định: RANSAC, kiểm tra hình học, ngưỡng chấp nhận bảo thủ. Chống lỗi liên tục cần mô hình đúng và trọng số đúng. Dùng công cụ của bên này cho bên kia thì hỏng — và một trong những lỗi thiết kế phổ biến là hy vọng back-end sẽ “chịu đựng được” một front-end tồi. Nó chịu được vài phần trăm outlier; nó không chịu được một loop closure sai, và B/11 nói vì sao.

Sáu chương có mức đầu tư rất khác nhau. Chương 7 và 8 là hai chương mà mọi hệ thống đều phải quyết định, và quyết định của chúng định hình mọi thứ còn lại — đó là lý do chúng đứng đầu. Chương 9 là chương hợp nhất: nó cho thấy bộ lọc, cửa sổ trượt, smoothing gia tăng và full batch đều là cùng một bài toán MAP của A/04 với các lựa chọn khác nhau về marginalize cái gì và tuyến tính hoá lại bao nhiêu lần; hiểu điều đó thì việc chuyển qua lại giữa các thể giới trở nên tự nhiên. Chương 10 và 11 là hai chương về hai khoảnh khắc đặc biệt của một hệ thống — lúc bắt đầu và lúc quay lại — và cả hai đều là chỗ A/05 trả công. Chương 12 là chương về khoảnh khắc thứ ba, lúc lạc đường, và nó là chương gần với sản phẩm nhất trong phần này.

Thứ tự đọc đề xuất là tuần tự. Một ngoại lệ: nếu bạn đã đọc ../IMU_Fusion/, chương 9 ở đây chồng lấn với chương 8 và 9 ở đó; phần chồng lấn được viết lại từ góc nhìn thị giác, và chỗ nào cây kia nói sâu hơn thì ở đây trở sang.

Bảng 6.2. Bản đồ Phần B: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
7	07-dac-trung-va-dat	Detector và descriptor cổ điển lẫn học được; KLT và optical flow; phân bố đặc trưng; RANSAC và họ của nó; tinh chỉnh subpixel	• • •
8	08-mo-hinh-phan-du	Indirect, direct, semi-direct; tham số hoá landmark; structureless; phần dư độ sâu, đường thẳng, mặt phẳng; prior học được	• • •
9	09-paradigm-uoc-luo	Bộ lọc (EKF, MSCKF, InEKF); cửa sổ trượt; fixed-lag; iSAM2; full batch; pose graph; dạng căn bậc hai; certifiable	• • •
10	10-khoi-tao	Khởi tạo mono hai khung và model selection; khởi tạo từ stereo/depth; khởi tạo VI; từ prior học được; phát hiện thất bại và khởi tạo lại	• •
11	11-loop-closure-va-	BoW và place recognition học được; kiểm định hình học; $Sim(3)$; pose graph; back-end bền (switchable, DCS, GNC, RRR); perceptual aliasing	• • •
12	12-relocalization	BoW + PnP; localization phân cấp; scene coordinate regression; vì sao APR không tốt; kidnapped robot; kiểm định chống reloc sai	• •

Chương 7, 8, 9 là bộ ba xương sống và nên đọc liền nhau: chúng lần lượt trả lời “phép đo đến từ đâu”, “phép đo có dạng gì”, và “ghép phép đo thế nào”. Chương 11 cũng là xương sống nhưng vì lý do khác — nó là chương duy nhất mà một lỗi đơn lẻ có thể phá huỷ toàn bộ bản đồ, nên phần về back-end bền ở đó đáng đọc kỹ hơn về ngoài. Chương 10 và 12 có thể đọc lướt lần đầu rồi quay lại khi bạn thật sự phải xây dựng hai mô-đun đó; cả hai đều là nơi [A/05](#) trả công và là nơi nguyên tắc “biết từ chối quan trọng hơn luôn trả lời” được áp dụng cụ thể nhất.

Đặc trưng và data association (Features and Data Association)

PHỤ THUỘC A/01-hình-học-da-thi (kiểm định hình học là bước cuối của mọi pipeline khớp điểm); A/02-mô-hình-camera (gỡ méo trước khi dùng hình học); A/06-bat-dinh-va-lan-truyen-sai-so (σ của phép đo pixel chính là thứ chương này quyết định, và nó là trọng số cho mọi thứ ở back-end).

TÓM TẮT

Chương này về bài toán mà phần còn lại của cây coi như đã giải xong: *pixel nào ở khung này là cùng một điểm vật lý với pixel nào ở khung kia?* Đây là bài toán *tổ hợp* — có lời giải đúng hoặc sai — khác hẳn bài toán ước lượng liên tục ở back-end, và khác biệt ấy quyết định mọi công cụ dùng để giải nó. Nội dung có bốn phần. Một: cách tìm điểm đáng theo dõi, từ tiêu chí Harris/Shi-Tomasi tới detector học được, và tiêu chí thật để chọn giữa chúng. Hai: cách khớp chúng — mô tả rồi so khớp, hay theo dõi trực tiếp bằng optical flow — và vì sao hai cách phù hợp với hai chế độ hoạt động khác nhau. Ba: hai thứ bị coi nhẹ nhất mà ảnh hưởng lớn nhất, là *phân bố không gian* của đặc trưng và *độ chính xác subpixel*; cái thứ nhất quyết định điều kiện số của Hessian, cái thứ hai quyết định trực tiếp σ tức trọng số của mọi phần dư. Bốn: loại bỏ outlier bằng RANSAC và họ của nó, với phân tích vì sao cỡ mẫu tối thiểu quan trọng hơn người ta nghĩ. Nếu chỉ nhớ một điều: *front-end quyết định hệ thống hỏng theo kiểu nào, back-end chỉ quyết định nó chính xác tới đâu* — và trong sản phẩm, kiểu hỏng quan trọng hơn.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đặc trưng (feature)** — một vùng ảnh nhỏ có tính chất đủ đặc biệt để nhận ra lại ở ảnh khác. Gồm hai phần tách rời: *detector* tìm ra nó ở đâu, *descriptor* mô tả nó trông thế nào.
- **bài toán cửa sổ (aperture problem)** — nhìn qua một lỗ nhỏ vào một cạnh thẳng, ta chỉ biết cạnh dịch chuyển vuông góc với nó bao nhiêu, không biết nó trượt dọc bao nhiêu. Đây là lý do góc theo dõi được còn cạnh thì không, và nó là gốc của mọi tiêu chí detector.
- **ma trận cấu trúc (structure tensor)** — ma trận $M = \sum_w \nabla I \nabla I^\top$ lấy trên một cửa sổ. Hai giá trị riêng của nó nói vùng đó là vùng phẳng (cả hai nhỏ), cạnh (một lớn một nhỏ), hay góc (cả hai lớn).
- **optical flow** — trường vectơ mô tả chuyển động biểu kiến của các mẫu cường độ giữa hai ảnh. KLT theo dõi một mảng nhỏ bằng cách giải trực tiếp cho độ dịch làm cường độ khớp nhất.
- **ratio test** — quy tắc của Lowe: chỉ chấp nhận khớp nếu khoảng cách tới ứng viên gần nhất nhỏ hơn đáng kể so với ứng viên gần thứ hai. Nó loại các đặc trưng lặp lại, thứ mà khoảng cách tuyệt đối không loại được.
- **RANSAC** — lặp: lấy mẫu tối thiểu, khớp mô hình, đếm inlier, giữ mô hình tốt nhất. Nó tìm *tập nhất quán lớn nhất*, không phải nghiệm chính xác nhất — nên nó luôn phải theo sau bởi một bước tinh chỉnh.
- **tinh chỉnh subpixel** — ước lượng vị trí đặc trưng chính xác hơn một pixel, thường bằng cách khớp parabol vào hàm điểm số quanh cực đại. Nó quyết định trực tiếp σ của phép đo.

CÂU HỎI MỞ ĐẦU

- Q1. Thuật toán năm điểm cần năm mẫu, tám điểm cần tám. Với 50% outlier, chênh lệch số vòng lặp RANSAC giữa hai lựa chọn ấy là bao nhiêu — và vì sao con số đó lớn hơn nhiều so với trực giác “chỉ hơn ba điểm”?
- Q2. Hai hệ thống trích cùng số lượng đặc trưng với cùng chất lượng khớp. Một hệ chính xác hơn hẳn. Nếu một khác biệt về front-end có thể giải thích điều đó mà *không* liên quan gì tới detector hay descriptor.
- Q3. Tinh chỉnh subpixel cải thiện σ từ 0,5 px xuống 0,15 px. Vì sao con số đó ảnh hưởng tới *mọi* thứ ở back-end, và nó ảnh hưởng thế nào tới hiệp phương sai đầu ra?
- Q4. SuperPoint và LightGlue cho nhiều khớp đúng hơn ORB một cách rõ rệt trong điều kiện khó. Vì sao nhiều hệ sản phẩm vẫn dùng ORB, và câu trả lời “vì GPU đắt” là chưa đủ?
- Q5. Hành lang văn phòng có hàng chục cánh cửa giống hệt nhau. Front-end khớp điểm giữa hai cánh cửa khác nhau một cách hoàn hảo — khoảng cách descriptor rất nhỏ. Vì sao ratio test không cứu được ở đây, và thứ gì mới cứu được?

7.1 Đặt vấn đề

Phần A xây dựng toàn bộ bộ máy ước lượng trên một giả thiết mà nó không bao giờ kiểm: rằng ta biết pixel nào tương ứng với pixel nào. A/01 mục 7 nêu nó là giả thiết ngầm thứ tư; chương này là chỗ trả nợ.

Bài toán gốc: cho hai ảnh của cùng một cảnh chụp từ hai chỗ khác nhau, tìm các cặp pixel là ảnh của cùng một điểm vật lý. Nghe đơn giản, và nó khó vì bốn lý do chồng lên nhau. *Thứ nhất, không gian tìm kiếm lớn*: một ảnh 640×480 có ba trăm nghìn pixel, nên ghép cặp vét cạn là chín mươi tỉ phép so sánh. *Thứ hai, phần lớn pixel không phân biệt được*: một pixel giữa bức tường trắng giống hết hàng nghìn pixel khác. *Thứ ba, vẻ ngoài thay đổi*: cùng một điểm nhìn từ góc khác, dưới ánh sáng khác, ở tỉ lệ khác, trông không giống nhau. *Thứ tư, và đây là lý do nghiêm trọng nhất, có những thứ trông giống nhau mà không phải cùng một điểm* — cửa sổ trên một toà nhà, viên gạch trên sàn, cánh cửa trong hành lang.

Ba lý do đầu là lý do *khó*; lý do thứ tư là lý do *nguy hiểm*, và nó khác về bản chất. Với ba lý do đầu, thất bại nghĩa là không tìm được khớp, và hệ thống biết mình không tìm được. Với lý do thứ tư, thất bại nghĩa là tìm được một khớp *sai* trông rất thuyết phục, và hệ thống không biết. Đây là **perceptual aliasing**, và nó là kẻ thù số một của SLAM trong môi trường nhân tạo — chính là môi trường mà robot sản phẩm hoạt động.

Ai gặp bài toán này? Mọi ứng dụng thị giác hình học, và giải pháp được phát triển chủ yếu trong cộng đồng thị giác máy tính chứ không phải robot học. Lịch sử có ba giai đoạn đáng biết vì mỗi giai đoạn để lại một bài học. Giai đoạn một, trước SIFT: theo dõi mảng ảnh trực tiếp (KLT), hoạt động tốt khi chuyển động nhỏ và hỏng hoàn toàn khi lớn. Giai đoạn hai, SIFT [45] và con cháu: mô tả bất biến với tỉ lệ và xoay, cho phép khớp giữa hai ảnh bất kỳ; đây là bước nhảy làm SfM quy mô lớn khả thi. Giai đoạn ba, từ khoảng 2018: detector và matcher học được, cải thiện rõ rệt ở điều kiện khó.

Cách tiếp cận hiển nhiên — và cách gần như ai cũng nghĩ tới đầu tiên — là làm descriptor *tốt hơn* cho tới khi không còn khớp sai. Cách đó không bao giờ thành công, và lý do đáng nói rõ vì nó định hình toàn bộ kiến trúc: *không có descriptor nào phân biệt được hai cánh cửa thật sự giống hệt nhau*, vì thông tin phân biệt không nằm trong vùng ảnh cục bộ. Nó nằm ở *ngữ cảnh* — cánh cửa này ở đâu so với các thứ khác — và ngữ cảnh là thông tin hình học, không phải thông tin về ngoại.

Đó là lý do mọi pipeline khớp điểm nghiêm túc đều có *hai tầng*: tầng vẻ ngoài đề xuất ứng viên, tầng hình học kiểm định chúng. Tầng thứ hai không phải một bước làm sạch tùy chọn mà là một phần thiết yếu, và nó là chỗ A/01 quay lại phục vụ chương này.

TRỰC GIÁC

Hình dung bạn phải nối hai bức ảnh chụp cùng một căn phòng từ hai chỗ đứng, bằng cách vẽ các đường nối những điểm tương ứng.

Bạn không bắt đầu bằng cách nhìn từng pixel. Bạn tìm những chỗ *đáng nhớ*: góc bàn, mép tranh, vết ố trên tường. Những chỗ ấy có hai tính chất — chúng khác biệt với xung quanh, và chúng vẫn nhận ra được từ góc nhìn khác. Đó là công việc của detector.

Rồi với mỗi chỗ đáng nhớ, bạn ghi lại nó *trông thế nào* theo một cách không phụ thuộc vào việc bạn đứng đâu — “một góc vuông, sáng ở trên, tối ở dưới”. Đó là descriptor.

Rồi bạn ghép: với mỗi chỗ ở ảnh một, tìm chỗ ở ảnh hai có mô tả giống nhất. Và đây là chỗ bạn sẽ mắc lỗi, vì trong phòng có bốn cái ghế giống hệt nhau.

Cách bạn — một con người — phát hiện lỗi ấy rất đáng chú ý: bạn *không* nhìn kỹ hơn vào cái ghế. Bạn lùi lại và nhìn *tất cả* các đường nối cùng lúc. Nếu hai mươi đường nối đều đi theo một quy luật chung và một đường đi ngược hẳn, bạn biết ngay đường đó sai — không phải vì nó trông sai, mà vì nó *không phù hợp với số đông*.

Đó chính xác là RANSAC, và đó là lý do tăng hình học không thể thiếu. Thông tin để phát hiện khớp sai không nằm trong khớp đó; nó nằm trong quan hệ giữa nó với tất cả các khớp khác.

7.2 Tìm điểm đáng theo dõi

A. Vì sao góc chứ không phải cạnh

Tiêu chí chọn đặc trưng có một cơ sở toán học gọn và đáng tự dẫn một lần.

Xét một cửa sổ nhỏ và hỏi: dịch nó đi một đoạn $\Delta \mathbf{u}$ thì cường độ đổi bao nhiêu? Tổng bình phương chênh lệch, khai triển bậc nhất:

$$E(\Delta \mathbf{u}) = \sum_w [I(\mathbf{u} + \Delta \mathbf{u}) - I(\mathbf{u})]^2 \approx \Delta \mathbf{u}^\top M \Delta \mathbf{u}, \quad M = \sum_w \nabla I \nabla I^\top.$$

Ma trận M cỡ 2×2 là **ma trận cấu trúc**. Hai giá trị riêng của nó phân loại vùng ảnh: cả hai nhỏ nghĩa là vùng phẳng — dịch đi đâu cũng không đổi, không theo dõi được; một lớn một nhỏ nghĩa là cạnh — dịch vuông góc thì đổi, dịch dọc thì không, đây chính là **bài toán cửa sổ**; cả hai lớn nghĩa là góc — dịch theo hướng nào cũng đổi, theo dõi được theo cả hai chiều.

Shi và Tomasi [46] đề nghị dùng trực tiếp $\min(\lambda_1, \lambda_2)$ làm điểm số; Harris dùng một tổ hợp của định thức và vết để tránh tính giá trị riêng. Hai tiêu chí rất gần nhau trong thực hành.

Điều đáng nhận ra, và nó nối chương này với A/04: M chính là *ma trận thông tin của bài toán ước lượng vị trí đặc trưng*. Bài toán “đặc trưng này dịch đi đâu” là một bài toán MAP nhỏ, và M là Hessian của nó. Cạnh không theo dõi được vì M suy biến theo hướng dọc cạnh — đó đúng là hiện tượng ở A/05, gặp ở quy mô một cửa sổ 7×7 pixel. Cùng một nguyên lý xuất hiện ở hai thang bậc khác nhau, và nhận ra điều đó là dấu hiệu đã đọc đúng cả hai chương.

B. Từ cổ điển tới học được

FAST [47] kiểm một vòng tròn 16 pixel quanh ứng viên và hỏi có đủ số pixel liên tiếp sáng hơn hoặc tối hơn tâm không — nhanh tới mức gần như miễn phí, không bắt biến gì cả. **ORB** [48] thêm hướng vào FAST và ghép với descriptor nhị phân BRIEF có xoay; kết quả là một cặp detector-descriptor cực nhanh, khớp bằng khoảng cách Hamming. **SIFT** [45] tìm cực trị trong không gian tỉ lệ và mô tả bằng histogram gradient; chậm hơn nhiều bậc nhưng bất biến tỉ lệ và xoay thật sự.

SuperPoint [49] học cả detector lẫn descriptor bằng tự giám sát. **SuperGlue** [50] thay bước khớp bằng một mạng chú ý nhìn *toàn bộ* hai tập điểm cùng lúc, nên nó khai thác được ngữ cảnh — đây là bước tiến khái niệm quan trọng, vì nó là lần đầu thông tin ngữ cảnh được đưa vào tầng về ngoài thay vì chỉ ở tầng hình học. **LightGlue** [51] làm cùng việc rẻ hơn nhiều với cơ chế thoát sớm. **LoFTR** [52] bỏ hẳn detector và khớp trực tiếp ở mức dày — hoạt động tốt ở vùng ít vân, nơi mọi detector đều thất bại.

CẠM BÃY

Tôi **không** đặt bảng so sánh số liệu giữa các detector ở đây, và lý do thuộc về phương pháp chứ không phải sự lười.

Các con số công bố (số khớp đúng, độ lặp lại, tỉ lệ thành công) phụ thuộc cực mạnh vào bộ dữ liệu, vào ngưỡng, vào cách đếm, và vào việc ai chạy baseline. Hai bài báo cáo hai con số cho cùng một cặp phương pháp trên cùng bộ dữ liệu thường *không so sánh được với nhau*, và việc đặt chúng cạnh nhau trong một bảng tạo ra ảo giác so sánh — đúng thứ mà `research-rules.md` mục 6.2 cảnh báo.

Điều *có thể* nói một cách trung thực là các đánh đổi về cơ chế, và đó là nội dung Bảng 7.1. Điều *không thể* nói là “X tốt hơn Y bao nhiêu phần trăm” mà không kèm điều kiện đo đầy đủ.

Cách đúng để chọn cho dự án của bạn không phải đọc bảng của người khác: đó là **đo trên dữ liệu của chính bạn**, trong chính điều kiện triển khai, với chính phần cứng mục tiêu. Chương D/21 lập luận rằng đây không phải lời khuyên cầu toàn mà là yêu cầu bắt buộc, vì khoảng cách giữa EuRoC và một nhà kho thật lớn hơn khoảng cách giữa hai detector bất kỳ.

Bảng 7.1. Các họ front-end theo cơ chế, không theo con số. Cột cuối là tiêu chí chọn thực dụng.

Họ	Cơ chế	Hỏng khi	Chọn khi
FAST + BRIEF/ORB	Kiểm vòng tròn cường độ; descriptor nhị phân	Đổi tỉ lệ mạnh; đổi ánh sáng lớn; vùng ít vân	CPU hạn chế; tốc độ khung cao; chuyển động vừa phải
SIFT và họ	Cực trị không gian tỉ lệ; histogram gradient	Vùng ít vân; đổi về ngoài lớn (mùa, ngày đêm)	Cần bất biến tỉ lệ thật; SfM ngoại tuyến
KLT / optical flow	Theo dõi mảng trực tiếp bằng gradient	Chuyển động lớn; đổi ánh sáng; che khuất	Khung liên tiếp gần nhau; muốn rẻ và chính xác subpixel
Học được (SuperPoint+Li)	Detector và matcher học từ dữ liệu; matcher dùng ngữ cảnh	Ngoài phân bố huấn luyện; cần GPU; khó dự đoán kiểu hỏng	Có GPU; điều kiện khó; cần độ bền với đổi về ngoài
Detector-free (LoFTR, MAST3R)	Khớp dày trực tiếp, không qua điểm	Rất đắt; khó đưa vào vòng lặp thời gian thực	Vùng ít vân; localization ngoại tuyến

Về câu hỏi mở đầu thứ tư — vì sao sản phẩm vẫn dùng ORB — câu trả lời “vì GPU đắt” chỉ là một phần. Ba lý do còn lại đáng giá hơn. *Tính dự đoán được*: ORB hỏng theo những cách đã biết rõ và lặp lại được, còn mạng học được hỏng theo những cách phụ thuộc vào việc dữ liệu hiện tại có giống dữ liệu huấn luyện không — và không ai đo được điều đó tại hiện trường. *Tính tất định*: cùng đầu vào cho cùng đầu ra, bit-exact, điều kiện bắt buộc để debug một lỗi từ hiện trường

(D/20). *Chi phí cập nhật*: đổi một mô hình học được đòi hỏi tái kiểm định toàn bộ, còn đổi một ngưỡng thì không. Ba lý do đó thuộc về D/21 và chúng thường quyết định hơn độ chính xác.

7.3 Khớp: mô tả rời so, hay theo dõi trực tiếp

A. Hai chế độ hoạt động

Có hai cách hoàn toàn khác nhau để tìm tương ứng, và chúng phù hợp với hai tình huống khác nhau — nhằm chúng là một lỗi thiết kế phổ biến.

Khớp bằng descriptor không giả định gì về vị trí: trích đặc trưng ở cả hai ảnh, mô tả chúng, rồi tìm ứng viên gần nhất trong không gian descriptor. Dùng được cho hai ảnh *bất kỳ*, kể cả cách nhau hàng giờ hoặc chụp từ hai thiết bị khác nhau. Chi phí: trích đặc trưng ở cả hai ảnh, và tìm kiếm trong không gian nhiều chiều.

Theo dõi bằng optical flow giả định chuyển động nhỏ: lấy vị trí đặc trưng ở khung trước làm khởi đầu, rồi giải cho độ dịch làm cường độ khớp nhất. KLT làm điều này bằng Gauss-Newton trên một mảng nhỏ, với thấp ảnh để chịu được chuyển động lớn hơn [53]. Rẻ hơn nhiều vì không phải trích và mô tả lại; chính xác subpixel tốt hơn vì nó tối ưu trực tiếp trên cường độ. Hồng khi chuyển động quá lớn hoặc ánh sáng đổi.

Quy tắc chọn gọn: *theo dõi giữa các khung liên tiếp, khớp descriptor khi cần nối hai khung xa nhau*. Đây là lý do VINS-Mono dùng KLT cho tracking [13] còn ORB-SLAM dùng descriptor — hai lựa chọn kiến trúc khác nhau phù hợp với hai ưu tiên khác nhau, không phải một cái đúng một cái sai. Nhưng mọi hệ thống đều cần descriptor ở đâu đó, vì loop closure (B/11) và relocalization (B/12) bản chất là khớp giữa hai khung rất xa nhau về thời gian.

Baker và Matthews [54] chỉ ra một biến thể quan trọng: *inverse compositional*, trong đó Hessian được tính *một lần* trên mảng tham chiếu thay vì tính lại mỗi vòng lặp. Cùng kết quả, chi phí thấp hơn nhiều bậc. Đây là loại tối ưu hoá đáng biết vì nó không đánh đổi gì cả.

B. Ratio test và giới hạn của nó

Sau khi tìm ứng viên gần nhất, câu hỏi là có chấp nhận không. Dùng ngưỡng tuyệt đối trên khoảng cách descriptor hoạt động kém, vì thang khoảng cách khác nhau giữa các loại đặc trưng và giữa các vùng ảnh.

Ratio test của Lowe so khoảng cách tới ứng viên gần nhất với ứng viên gần *thứ hai*, và chỉ chấp nhận nếu tỉ số đủ nhỏ. Trực giác: nếu đặc trưng thật sự đặc biệt thì ứng viên đúng phải gần hơn hẳn mọi ứng viên khác; nếu có hai ứng viên gần ngang nhau thì ta không có cơ sở chọn. Đây là một phép thử *tương đối*, nên nó tự thích ứng với thang khoảng cách địa phương — và đó là lý do nó bền hơn ngưỡng tuyệt đối rất nhiều.

Nhưng nó có một giới hạn quan trọng, và đó là câu trả lời cho câu hỏi mở đầu thứ năm. Ratio test loại được trường hợp *có nhiều ứng viên giống nhau trong tầm nhìn*. Nó không loại được trường hợp cánh cửa thứ hai không nằm trong tập ứng viên — chẳng hạn vì nó ở ngoài khung hình hiện tại, hoặc vì ta chỉ tìm kiếm trong một vùng nhỏ. Lúc đó ứng viên sai là ứng viên duy nhất, tỉ số rất tốt, và khớp được chấp nhận với độ tin cậy cao.

Thứ cứu được chỉ có thể là hình học: kiểm định rằng khớp này nhất quán với đa số các khớp khác (mục 5), hoặc nhất quán với một tiên nghiệm về chuyển động. Đây lại là bài học ở khối *Trực giác*: thông tin để phát hiện khớp sai không nằm trong khớp đó.

7.4 Hai thứ bị coi nhẹ nhất

A. Phân bố không gian

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một trong những điểm có tỉ lệ giá trị trên độ phức tạp cao nhất trong cả cây.

Cách cài đặt ngây thơ: tính điểm số cho mọi ứng viên, sắp xếp, lấy N cái tốt nhất. Kết quả gần như luôn là các đặc trưng *tụ lại* ở vùng có vân mạnh — một tấm poster trên tường, một tán lá — và cả vùng còn lại của ảnh không có đặc trưng nào.

Vì sao điều đó tệ? Quay lại Jacobian ở A/03 công thức (3.4): đóng góp của một quan sát vào ma trận thông tin phụ thuộc vào *vị trí của nó trong ảnh* qua các số hạng X/Z và Y/Z . Các đặc trưng tụ ở một chỗ đóng góp các hàng Jacobian gần như *tuyến tính phụ thuộc* nhau, nên chúng thêm rất ít thông tin mới. Hệ quả là Λ có số điều kiện lớn, và bài toán gần suy biến theo một hướng — đúng loại suy biến số học ở A/05 mục 2C. Một trăm đặc trưng trải đều cho nhiều thông tin hơn năm trăm đặc trưng tụ một chỗ.

Có một lý do thứ hai, độc lập và cũng quan trọng: *độ bền*. Nếu mọi đặc trưng ở trên một vật, và vật đó bị che hoặc bắt đầu di chuyển, hệ thống mất tất cả cùng lúc. Phân bố đều là một dạng đa dạng hoá rủi ro.

Ba cách cài đặt, theo thứ tự tinh vi. *Lưới*: chia ảnh thành ô, giữ tối đa k đặc trưng tốt nhất mỗi ô. Rẻ, hiệu quả, và đủ cho hầu hết trường hợp. *Quadtree*: chia thích ứng, tinh hơn ở vùng đông — ORB-SLAM dùng cách này [5]. *ANMS* (adaptive non-maximal suppression): giữ đặc trưng theo bán kính triệt tiêu giảm dần, cho phân bố đều nhất nhưng đắt nhất.

B. Độ chính xác subpixel

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và nó là điểm nối trực tiếp nhất giữa front-end và A/06.

Vị trí đặc trưng ở độ phân giải pixel nguyên cho $\sigma \approx 0,5$ px (sai số lượng tử hoá của một biến phân bố đều trên một pixel là $1/\sqrt{12} \approx 0,29$ px, nhưng trong thực tế sai số lớn hơn thế vì còn nhiễu và lượng tử hoá không hoàn hảo). Tinh chỉnh subpixel — khớp một parabol vào hàm điểm số quanh cực đại, hoặc chạy vài vòng KLT — giảm con số đó xuống mức thường được nêu là 0,1 tới 0,2 px.

CẠM BẮY

Hai con số vừa nêu cần một cảnh báo. Khoảng 0,1–0,2 px là con số **thường được nêu trong cộng đồng** cho tinh chỉnh subpixel trên ảnh có vân tốt, nhưng **tôi không dẫn được một nguồn đo có điều kiện đầy đủ cho nó**, và nó phụ thuộc mạnh vào độ tương phản, mức nhiễu cảm biến, và độ sắc nét của đặc trưng.

Theo `learning-rules.md` mục 5, một con số không có điều kiện đo là một con số vô nghĩa, nên cách đúng là: **tự đo σ của chính front-end của bạn**, trên chính dữ liệu của bạn. Cách đo rẻ nhất: chụp một cảnh tĩnh với camera cố định trên chân máy, theo dõi đặc trưng qua vài trăm khung, và tính độ lệch chuẩn của vị trí từng đặc trưng. Con số thu được là σ *thật* của bạn, đã bao gồm cả nhiễu cảm biến lẫn giới hạn thuật toán.

Con số đó đáng đo vì nó là **trọng số $\Omega = \sigma^{-2}$** cho mọi phần dư chiếu ở back-end. Đặt sai nó thì mọi hiệp phương sai đầu ra sai theo, và bạn rơi vào đúng nguồn số hai của bệnh quá tự tin ở A/06 mục 4B. Đây là một trong những phép đo rẻ nhất và bị bỏ qua nhiều nhất trong cả quy trình.

Vì sao σ quan trọng tới vậy? Ba đường. *Một*: nó vào thẳng Ω_k trong hệ chuẩn (4.2), nên nó quyết định trọng số tương đối giữa nhân tử thị giác và mọi nhân tử khác — IMU, odometry, GNSS. *Hai*: hiệp phương sai đầu ra tỉ lệ với σ^2 , nên giảm σ ba lần thì hiệp phương sai giảm chín lần; đó là một cải thiện lớn hơn hầu hết các thay đổi thuật toán. *Ba*: qua σ_d trong công thức (6.2), nó quyết định trực tiếp tầm hữu ích của stereo. Với $b = 12$ cm, $f = 400$ px, ở $z = 5$ m: $\sigma = 0,5$ px cho $\sigma_z \approx 26$ cm, còn $\sigma = 0,15$ px cho $\sigma_z \approx 8$ cm. Cùng phần cứng, ba lần khác biệt về chất lượng độ sâu.

7.5 Loại bỏ outlier

A. RANSAC và vì sao cỡ mẫu quan trọng

RANSAC [33] lặp bốn bước: lấy ngẫu nhiên s tương ứng (cỡ mẫu tối thiểu của mô hình), khớp mô hình, đếm inlier, giữ mô hình nhiều inlier nhất.

Số vòng lặp cần để có xác suất p lấy được ít nhất một mẫu toàn inlier, với tỉ lệ inlier w :

$$N = \frac{\log(1-p)}{\log(1-w^s)}. \quad (7.1)$$

Suy dẫn: xác suất một mẫu toàn inlier là w^s ; xác suất N mẫu đều hỏng là $(1-w^s)^N$; đặt bằng $1-p$ và lấy log. (Cửa a.)

Ví dụ số trả lời câu hỏi mở đầu thứ nhất. Với $w = 0,5$ và $p = 0,99$:

Thuật toán năm điểm ($s = 5$): $w^s = 0,5^5 = 0,03125$, nên $N = \log(0,01)/\log(0,96875) \approx -4,605/(-0,03175) \approx 146$ vòng.

Thuật toán tám điểm ($s = 8$): $w^s = 0,5^8 = 0,00391$, nên $N \approx -4,605/(-0,003914) \approx 1177$ vòng.

Tám lần khác biệt cho ba điểm mẫu thêm. (Cửa kiểm chứng b, tính tay.) Và với $w = 0,3$, chênh lệch thành 170 so với 6300 vòng — gần bốn mươi lần. Đây là lý do thực dụng chính để dùng thuật toán năm điểm, và nó lớn hơn nhiều so với ấn tượng “chỉ bớt ba điểm”. Quan hệ là hàm mũ theo s , không tuyến tính.

B. Cải tiến và giới hạn

PROSAC [55] lấy mẫu theo thứ tự chất lượng khớp thay vì ngẫu nhiên đều — vì khớp có điểm số tốt có xác suất là inlier cao hơn. Hội tụ nhanh hơn nhiều khi thang điểm có ý nghĩa. **MAGSAC++** [56] bỏ ngưỡng inlier cứng bằng cách tích phân trên các mức ngưỡng — giải quyết một vấn đề thực tế đáng kể, là ngưỡng inlier thường được chọn tùy tiện và kết quả nhạy với nó. **LO-RANSAC** chạy một bước tinh chỉnh cục bộ trên tập inlier mỗi khi tìm được mô hình tốt hơn, cải thiện rõ với chi phí nhỏ.

Ba giới hạn của RANSAC đáng nhớ vì chúng là nơi nó thất bại một cách im lặng.

Một: nó tìm *tập nhất quán lớn nhất*, không phải nghiệm đúng. Nếu outlier có cấu trúc — tất cả nằm trên một vật đang di chuyển — thì chúng nhất quán với nhau, và nếu vật đó chiếm đa số khung hình, RANSAC sẽ trung thành chọn vật động làm inlier. Kết quả sai nhưng nhất quán, tức im lặng. Xử lý ở D/19.

Hai: nó cần một mô hình. Trong cấu hình suy biến ở A/01 mục 6, mô hình không xác định duy nhất, nên RANSAC báo nhiều inlier cho một mô hình vô nghĩa. Đây chính là lý do khối *Cắm bẫy* ở chương đó cấm dùng số inlier làm thước đo chất lượng.

Ba: nghiệm của nó chỉ để khởi tạo. Mô hình khớp từ mẫu tối thiểu không dùng *tất cả* inlier, nên nó không tối ưu. Luôn phải theo sau bởi một bước tinh chỉnh trên toàn bộ inlier — và bước đó chính là bài toán MAP nhỏ của A/04.

C. Tiên nghiệm chuyển động: cách rẻ nhất cải thiện front-end

Một kỹ thuật đơn giản mà hiệu quả bất ngờ, và nó là một trong những chỗ mà kiến trúc hệ thống trả công.

Nếu có tiên nghiệm về chuyển động giữa hai khung — từ IMU, từ odometry bánh xe, hoặc chỉ từ mô hình vận tốc không đổi — ta dự đoán được mỗi landmark sẽ chiếu về *đâu* trong khung mới. Thay vì tìm kiếm toàn ảnh, ta tìm trong một vùng nhỏ quanh vị trí dự đoán.

Ba lợi ích cùng lúc, và cả ba đều lớn. *Rẻ hơn*: vùng tìm kiếm nhỏ hơn hàng chục lần. *Ít khớp sai hơn*: bớt ứng viên nghĩa là bớt cơ hội nhầm — và điều này tấn công trực tiếp vào perceptual aliasing, vì cánh cửa thứ hai thường nằm ngoài vùng tìm kiếm. *Tỉ lệ inlier cao hơn*: w tăng, nên theo (7.1) số vòng RANSAC giảm theo hàm mũ.

Đây là một ví dụ rõ ràng của việc hợp nhất cảm biến giúp *front-end* chứ không chỉ back-end — một điểm hay bị bỏ qua khi người ta nghĩ về IMU chỉ như một nguồn ràng buộc bổ sung trong đồ thị. Chi tiết ở C/15 và ../IMU_Fusion/.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [10]	ORB + quadtree phân bố; tìm kiếm có hướng dẫn bằng mô hình chuyển động; RANSAC PnP	Mục 4A và 5C trong một hệ thống hoàn chỉnh
VINS-Mono [13]	KLT tracking + lưới phân bố; IMU dự đoán vùng tìm kiếm	Lựa chọn kiến trúc đối lập với ORB-SLAM — xem mục 3A
COLMAP [11]	SIFT + ratio test + kiểm định hình học đa mô hình	Pipeline hai tầng đầy đủ nhất trong phần mềm mã nguồn mở
HLoc [12]	SuperPoint + SuperGlue/LightGlue	Front-end học được trong pipeline localization thật
SVO-Pro [57]	Trích đặc trưng thưa + căn chỉnh direct; tinh chỉnh subpixel bằng tối ưu cường độ	Mình hoạ mục 4B: độ chính xác subpixel đến từ tối ưu trực tiếp trên cường độ

MINI PROJECT SLAM · Đo σ thật của front-end của bạn, rồi đo ảnh hưởng của phân bố đặc trưng · 3 ngày

Mục tiêu	biến hai con số mà mọi người đoán — σ subpixel và ảnh hưởng của phân bố — thành hai con số bạn đo được trên phần cứng của mình.
Dữ liệu	camera của bạn trên chân máy cố định, chụp một cảnh tĩnh có vân đa dạng (vài trăm khung); cộng thêm một chuỗi EuRoC để có chân trị.
Các bước	<i>Phần một</i> : theo dõi đặc trưng qua toàn bộ chuỗi tĩnh, tính độ lệch chuẩn vị trí của từng đặc trưng, vẽ histogram — đó là σ thật của bạn; lặp lại có và không có tinh chỉnh subpixel, và với đặc trưng có độ tương phản cao và thấp riêng. <i>Phần hai</i> : trên chuỗi EuRoC, chạy BA với hai chiến lược chọn đặc trưng — “top-N theo điểm số” và “lưới phân bố” — với <i>cùng tổng số</i> đặc trưng; đo số điều kiện của Λ và ATE cho cả hai.
Xong khi	bạn có một con số σ của riêng mình kèm điều kiện đo, và bạn dùng nó làm trọng số trong mọi thứ về sau thay vì một hằng số đoán. Và bạn có hai đường cong số điều kiện cho thấy phân bố lưới cho Λ tốt hơn đáng kể ở cùng số đặc trưng — nếu không quan sát được, hãy kiểm xem cảnh của bạn có vân quá đều không, và đó cũng là một kết quả đáng ghi.

Nối với [A/06-bat-dinh-va-lan-truyen-sai-so](#) dùng σ này làm đầu vào cho toàn bộ ngân sách sai số; [C/18-danh-gia-va-phuong-phap-luan](#) biến phần hai thành một ablation study có kỷ luật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Từ $N = \log(1 - p) / \log(1 - w^s)$ với $w = 0,5$, $p = 0,99$: năm điểm cần khoảng 146 vòng, tám điểm cần khoảng 1177 — *tám lần*. Với $w = 0,3$ thì chênh gần bốn mươi lần. Lớn hơn trực giác vì quan hệ là *hàm mũ* theo s : mỗi điểm thêm vào nhân xác suất thành công của một mẫu với w , nên với $w = 0,5$ thì ba điểm thêm chia xác suất cho tám. Đây là lý do thực dụng chính để dùng thuật toán năm điểm, cùng với lý do quan trọng không kém là nó không suy biến với cảnh phẳng ([A/01](#) mục 6B).
- A2. *Phân bố không gian* của đặc trưng. Hệ tụ đặc trưng ở vùng vùn mạnh cho các hàng Jacobian gần tuyến tính phụ thuộc — theo công thức Jacobian ở [A/03](#), đóng góp phụ thuộc vị trí trong ảnh — nên Λ có số điều kiện lớn và bài toán gần suy biến. Một trăm đặc trưng trải đều cho nhiều thông tin hơn năm trăm đặc trưng tụ một chỗ. Lý do thứ hai, độc lập: nếu mọi đặc trưng ở trên một vật thì khi vật bị che hoặc di chuyển, hệ mất tất cả cùng lúc. Cách chữa rẻ nhất là lưới; tinh hơn thì quadtree hoặc ANMS.
- A3. Vì σ là trọng số $\Omega = \sigma^{-2}$ cho mọi phần dư chiều, nên nó quyết định trọng số *tương đối* giữa nhân tử thị giác và mọi nhân tử khác trong đồ thị. Hiệp phương sai đầu ra tỉ lệ σ^2 , nên giảm σ hơn ba lần thì hiệp phương sai giảm hơn mười lần — lớn hơn hầu hết cải tiến thuật toán. Và qua σ_d trong $\sigma_z = z^2 \sigma_d / (bf)$, nó quyết định tầm hữu ích của stereo: ở $z = 5$ m với $b = 12$ cm, $f = 400$ px, $\sigma = 0,5$ px cho $\sigma_z \approx 26$ cm còn $\sigma = 0,15$ px cho ≈ 8 cm. Cùng phần cứng, ba lần khác biệt.
- A4. “GPU đắt” chỉ là một phần. Ba lý do quan trọng hơn, và cả ba thuộc về [D/21](#). *Tính dự đoán được*: ORB hỏng theo cách đã biết và lặp lại được; mạng học được hỏng khi dữ liệu lệch khỏi phân bố huấn luyện, và không ai đo được điều đó tại hiện trường. *Tính tất định*: cùng đầu vào cho cùng đầu ra bit-exact, điều kiện bắt buộc để tái hiện một lỗi từ hiện trường. *Chi phí cập nhật*: đổi mô hình đòi tái kiểm định toàn bộ; đổi ngưỡng thì không. Trong sản phẩm, ba yếu tố đó thường quyết định hơn độ chính xác trung bình.
- A5. Vì ratio test so ứng viên gần nhất với ứng viên gần *thứ hai trong tập ứng viên*. Nếu cánh cửa thứ hai không nằm trong tập — ngoài khung hình, hoặc ngoài vùng tìm kiếm — thì ứng viên sai là ứng viên duy nhất, tỉ số rất tốt, và khớp được chấp nhận với độ tin cậy cao. Thứ cứu được chỉ có thể là *hình học*: kiểm định rằng khớp này nhất quán với đa số khớp khác (RANSAC), hoặc nhất quán với tiên nghiệm chuyển động. Nguyên tắc chung: thông tin để phát hiện một khớp sai không nằm trong khớp đó mà nằm trong quan hệ của nó với các khớp khác — và ở quy mô lớn hơn, đây chính là vấn đề perceptual aliasing ở [B/11](#).

NÓI LẠI THẬT ĐƠN GIẢN

Đưa cho một người hai bức ảnh chụp cùng một căn phòng từ hai chỗ đứng và bảo họ nối các điểm tương ứng. Họ sẽ làm ba việc, theo đúng thứ tự.

Đầu tiên họ tìm những chỗ *đáng nhớ*. Không phải giữa bức tường trắng — chỗ đó nhìn đâu cũng như nhau. Họ tìm góc bàn, mép khung tranh, vết bẩn trên sàn. Nguyên tắc: một chỗ đáng nhớ là chỗ mà dịch đi một chút theo *bất kỳ* hướng nào thì nó trông khác ngay. Một đoạn mép bàn thẳng thì không đạt — trượt dọc theo mép, nó vẫn y như cũ.

Thứ hai họ ghép. Với mỗi chỗ đáng nhớ ở ảnh một, tìm chỗ giống nhất ở ảnh hai. Và ở bước này họ chắc chắn sẽ mắc lỗi, vì trong phòng có bốn cái ghế giống hệt nhau.

Thứ ba — và đây là bước quan trọng nhất — họ *lùi lại nhìn tất cả các đường nối cùng lúc*. Nếu hai mươi đường đi theo một quy luật chung và một đường đi ngược hẳn, họ gạch đường đó đi. Không phải vì nhìn kỹ hơn vào cái ghế, mà vì đường đó không hợp với số đông.

Đó là toàn bộ chương này, và bài học lớn nhất nằm ở bước ba: *không có cách nào nhìn kỹ hơn vào một cái ghế để biết nó là cái nào*. Thông tin đó không tồn tại trong cái ghế. Nó chỉ tồn tại trong việc cái ghế ấy ở đâu so với mọi thứ khác.

Còn hai chi tiết nhỏ mà ít ai để ý nhưng lại quan trọng hơn vẻ ngoài. Một: đừng chọn tất cả các chỗ đáng nhớ ở cùng một góc ảnh — trải chúng ra khắp bức ảnh, vì mỗi vùng nói cho bạn một chuyện khác nhau. Hai: hãy xác định vị trí của chúng *chính xác nhất có thể*, chính xác hơn cả một ô pixel. Nghe như chuyện vụn vặt, nhưng con số ấy nhân lên qua toàn bộ phép tính phía sau, và cải thiện nó ba lần thì kết quả cuối cải thiện gấp mười.

Chỗ không rút gọn được: chỗ không rút gọn được là những căn phòng mà *mọi thứ* đều lặp lại — một hành lang kho hàng dài với các kệ giống hệt nhau. Ở đó bước ba cũng thất bại, vì số đông cũng có thể sai một cách nhất quán. Đó là bài toán khó nhất trong triển khai thật, và B/11 nói rằng nó vẫn chưa có lời giải tốt.

Một câu: Không có gì trong một khớp cho biết nó đúng hay sai — chỉ có quan hệ của nó với mọi khớp khác mới cho biết.

I 7.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *không descriptor nào phân biệt được hai vật thật sự giống hệt nhau, nên tầng hình học là bắt buộc* — gần như đúng theo định nghĩa nếu “giống hệt” hiểu chặt. Nó trở nên có thể bác bỏ khi ta hỏi về trường hợp thực tế: hai cánh cửa gần giống nhau. Ở đó, một descriptor đủ tốt có thể khai thác các khác biệt nhỏ — vết xước, khác biệt ánh sáng — và mệnh đề bị bác bỏ nếu một hệ thuần về ngoài đạt tỉ lệ khớp sai thấp ngang hệ có kiểm định hình học trên môi trường lặp lại. Tôi cho là không, nhưng LoFTR và MAST3R đi theo hướng khai thác ngữ cảnh rộng hơn, nên ranh giới đang dịch chuyển và tôi không chắc nó dừng ở đâu (*tôi suy ra, chưa đối chiếu nguồn*).

Mệnh đề thứ hai — *phân bố không gian ảnh hưởng tới số điều kiện của Λ* — suy ra trực tiếp từ cấu trúc Jacobian (cửa a) và kiểm được bằng mini project (cửa b). Nó bị bác bỏ nếu thí nghiệm cho thấy số điều kiện không khác nhau đáng kể giữa hai chiến lược chọn ở cùng số đặc trưng. Một trường hợp mà nó *thật sự* không đúng: khi cảnh có vân đều khắp nơi, top-N tự nhiên đã trải đều, và hai chiến lược trùng nhau.

Mệnh đề thứ ba — *tính chỉnh subpixel cho σ khoảng 0,1–0,2 px* — là con số tôi không có nguồn đo, và tôi đã nói rõ điều đó ở khối *Cạm bẫy*. Nó dễ bác bỏ nhất trong chương: chỉ cần đo, và mini project chính là phép đo. Nếu σ thật của bạn là 0,4 px thì mọi tính toán ngân sách sai số dựa trên 0,15 px đều sai gấp bảy lần về phương sai.

Một mệnh đề mềm: chương này khẳng định *front-end quyết định kiểu hồng, back-end quyết định độ chính xác*. Điều đó bị bác bỏ nếu trong một khảo sát ca hồng thật, phần lớn thất bại bắt nguồn từ back-end — phân kỳ số học, cực tiểu địa phương — chứ không từ data association. Tôi không có số liệu và đây là chỗ tôi phát biểu dựa trên cấu trúc của vấn đề (*tôi suy ra, chưa đối chiếu nguồn*); D/21 đề nghị cách thu số liệu đó.

I 7.7 Kết nối

Nói lên trên. A/01-hình-học-da-thì cung cấp tầng kiểm định hình học ở mục 5, và đồng thời nhận từ chương này đầu vào của nó — hai chương nối vòng với nhau. A/02-mo-hình-camera là bước gỡ méo phải chạy trước khi dùng bất kỳ ràng buộc hình học nào. A/03-lie-group-va-toi-uu-tren-da-tap giải thích vì sao phân bố đặc trưng ảnh hưởng tới điều kiện số: qua cấu trúc của Jacobian. A/05-observability-gauge-va-suy-bien là chỗ ma trận cấu trúc M ở mục 2A được nhìn đúng bản chất — một ma trận thông tin suy biến ở quy mô nhỏ. A/06-bat-dinh-va-lan-truyen-sai-so là chương nhận σ từ mục 4B và biến nó thành ngân sách sai số.

Nổi xuống dưới. [B/08-mo-hinh-phan-du](#) là lựa chọn tiếp theo: một khi có tương ứng, phần dư viết thế nào — và với direct method thì chương này gần như biến mất, vì không còn bước trích đặc trưng. [B/09-paradigm-uoc-luong-back-end](#) nhận đầu ra của chương này làm đầu vào, và giả định rằng outlier đã được loại — giả định mà mục 5B cảnh báo là không bao giờ đúng hoàn toàn. [B/11-loop-closure-va-nhat-quan-toan-cuc](#) là chương mà bài toán của chương này xuất hiện lại ở quy mô lớn hơn và với hậu quả nghiêm trọng hơn: khớp sai giữa hai khung liên kế chỉ làm hỏng một quan sát, còn khớp sai giữa hai địa điểm làm hỏng cả bản đồ. [B/12-relocalization](#) cũng vậy.

Nổi xa hơn. [C/15-hop-nhat-da-cam-bien](#) là chỗ mục 5C mở rộng: tiên nghiệm chuyển động từ bất kỳ cảm biến nào đều cải thiện front-end theo ba đường cùng lúc. [C/17-hoc-may-trong-slam](#) bàn kỹ hơn về front-end học được và lập luận rằng đây là chỗ học máy đóng góp rõ ràng nhất cho SLAM. [D/19-ben-vung-trong-the-gioi-thuc](#) xử lý ba giới hạn của RANSAC ở mục 5B, đặc biệt là trường hợp vật động chiếm đa số. [D/20-toi-uu-hieu-nang-va-he-thong](#) là chỗ chi phí của front-end trở thành ràng buộc cứng — nó thường chiếm phần lớn ngân sách tính toán của cả hệ.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao cạnh không theo dõi được còn góc thì được? Trả lời bằng ma trận cấu trúc, rồi nói nó liên hệ thế nào với [A/05](#).
2. Tự dẫn lại công thức số vòng RANSAC, rồi tính cho $s = 4$ (homography) với $w = 0,4$, $p = 0,99$.
3. Nêu hai chế độ khớp — descriptor và optical flow — và nói mỗi cái phù hợp với tình huống nào và vì sao.
4. Ratio test cứu được trường hợp nào và không cứu được trường hợp nào? Nêu cơ chế duy nhất cứu được trường hợp thứ hai.
5. Vì sao phân bố đặc trưng ảnh hưởng tới độ chính xác nhiều hơn số lượng đặc trưng? Nêu hai lý do độc lập.
6. σ subpixel của bạn là bao nhiêu, bạn đo nó thế nào, và nó ảnh hưởng tới ba thứ nào ở back-end?
7. Nêu ba giới hạn của RANSAC, và với mỗi cái nêu một tình huống thực tế mà nó thất bại im lặng.

Mô hình phần dư (Residual Models)

PHỤ THUỘC A/02-mo-hinh-camera (mô hình quang trắc — bắt buộc cho direct); A/04-uoc-luong-map-va-bundle-adjustment (khung MAP — chương này chỉ thay hàm h mà không đổi gì khác); A/06-bat-dinh-va-lan-truyen-sai-so (phân bố lệch của độ sâu — lập luận đầy đủ cho nghịch đảo độ sâu); B/07-dac-trung-va-data-association (đầu vào cho indirect; direct thì bỏ qua bước này).

TÓM TẮT

Khung MAP ở A/04 là một cái khuôn: đưa vào một hàm đo h và một trọng số Ω , nhận về một bộ ước lượng. Chương này bàn về việc chọn h , và nó cho thấy khuôn ấy tổng quát tới mức nào — toàn bộ sự khác nhau giữa ORB-SLAM và DSO nằm ở một lựa chọn duy nhất trong chương này, chứ không nằm ở bộ giải. Nội dung có ba phần. Một: ba họ phần dư — indirect (sai số chiếu lại), direct (sai số cường độ), semi-direct — với phân tích cơ chế chứ không phải bảng số liệu, vì các con số công bố của hai họ này gần như không so sánh được. Hai: tham số hoá landmark, nơi một chi tiết thống kê ở A/06 quyết định một lựa chọn kiến trúc; và kỹ thuật structureless, nơi landmark biến mất hoàn toàn khỏi vector trạng thái nhờ một phép chiếu. Ba: các loại phần dư khác — độ sâu, đường thẳng, mặt phẳng, prior học được — và cái giá ẩn của chúng, vốn là phá vỡ cấu trúc thừa mà A/04 phụ thuộc vào. Nếu chỉ nhớ một điều: *lựa chọn phần dư quyết định hệ thống cần gì từ thế giới* — indirect cần vận có cấu trúc, direct cần ánh sáng ổn định, và đó là lý do không cái nào thắng ở mọi nơi.

KIẾN THỨC CẦN CÓ TRƯỚC

- **phần dư chiếu lại (reprojection residual)** — hiệu giữa tọa độ pixel quan sát được của một đặc trưng và tọa độ mà mô hình dự đoán. Đơn vị pixel, hai chiều cho mỗi quan sát.
- **phần dư quang trắc (photometric residual)** — hiệu giữa giá trị cường độ của một pixel ở khung này và giá trị của pixel tương ứng ở khung kia. Đơn vị mức xám, một chiều cho mỗi pixel.
- **ngịch đảo độ sâu (inverse depth)** — tham số hoá một điểm bằng $\rho = 1/z$ thay vì z , thường kèm một khung neo và một hướng tia. Điểm ở vô hạn tương ứng $\rho = 0$, một giá trị hoàn toàn hợp lệ chứ không phải kỳ dị.
- **khung neo (anchor frame)** — khung hình mà tọa độ của landmark được biểu diễn tương đối với nó. Chọn khung đầu tiên nhìn thấy điểm làm neo giữ cho tham số hoá ổn định số học.
- **structureless** — kỹ thuật loại landmark khỏi vector trạng thái bằng cách chiếu phần dư lên không gian rỗng của Jacobian theo landmark. Kết quả là một ràng buộc chỉ giữa các tư thế.
- **độ sáng affine (affine brightness)** — mô hình $I' = aI + b$ cho thay đổi cường độ giữa hai khung do phơi sáng hoặc gain tự động. Hai tham số mỗi khung, đưa vào chính bài toán tối ưu.
- **gradient ảnh** — điều kiện tồn tại của direct method: phần dư quang trắc chỉ ràng buộc được tư thế ở những pixel có gradient khác không. Vùng phẳng đóng góp đúng bằng không.

CÂU HỎI MỞ ĐẦU

- Q1. Direct method dùng *mọi* pixel có gradient, indirect chỉ dùng vài trăm đặc trưng. Vì sao direct không thắng tuyệt đối, và thứ gì nó phải trả giá để có được nhiều dữ liệu như vậy?
- Q2. Tham số hoá một điểm bằng z và bằng $1/z$ mô tả cùng một điểm và cho cùng một nghiệm. Vậy vì sao lựa chọn giữa chúng lại đổi kết quả, và đổi ở đâu?

- Q3. MSCKF khử landmark khỏi vector trạng thái bằng phép chiếu null-space. Nếu landmark biến mất thì thông tin về nó đi đâu, và vì sao kỹ thuật này cho một bộ lọc chi phí tuyến tính thay vì bậc hai?
- Q4. Bạn thêm một ràng buộc “hai landmark này nằm trên cùng một mặt phẳng”. Ràng buộc đúng, và nó làm bài toán chậm đi nhiều bậc. Vì sao?
- Q5. Một mô hình học được cho độ sâu metric của mỗi pixel. Nghe như nó giải quyết bài toán tỉ lệ của mono. Nêu điều kiện để đưa nó vào đồ thị một cách hợp lệ, và nói vì sao đưa vào sai còn tệ hơn không dùng.

8.1 Đặt vấn đề

A/04 dựng một khuôn tổng quát: cực tiểu hoá $\sum_k \mathbf{r}_k^\top \Omega_k \mathbf{r}_k$. Khuôn ấy không nói $\mathbf{r}_k$ là gì. Chương này bàn đúng câu hỏi đó, và câu trả lời hoá ra là chỗ phân chia lớn nhất trong lịch sử Visual SLAM.

Bài toán gốc phát biểu thế này. Ta có một chuỗi ảnh — những mảng số. Ta muốn một hàm biến giả thuyết về tư thế và cấu trúc thành một dự đoán *kiểm được với ảnh*. Có hai cách làm việc đó, và chúng khác nhau ở chỗ *trừu tượng hoá ở đâu*.

Cách thứ nhất: *trừu tượng hoá sớm*. Biến ảnh thành một tập điểm có toạ độ và mô tả (B/07), rồi so sánh toạ độ dự đoán với toạ độ quan sát. Đây là **indirect**, và nó thống trị từ PTAM [58] tới ORB-SLAM [5].

Cách thứ hai: *không trừu tượng hoá*. So sánh trực tiếp giá trị cường độ: nếu tư thế đúng thì pixel này ở khung một phải có cùng độ sáng với pixel tương ứng ở khung hai. Đây là **direct**, và nó được LSD-SLAM [59] và DSO [22] phát triển thành một dòng hoàn chỉnh.

Ai gặp sự lựa chọn này? Mọi người thiết kế một hệ SLAM, và đó là một trong hai hoặc ba quyết định kiến trúc lớn nhất — nó lan toả vào mọi thứ khác, từ yêu cầu phần cứng tới cách hệ thống hỏng.

Điều đáng nói ngay là vì sao tranh luận này *không* giải quyết được bằng cách đo. Các con số công bố cho hai họ này gần như không so sánh được: chúng chạy trên các bộ dữ liệu khác nhau, với các mức hiệu chuẩn quang trắc khác nhau (DSO cần nó, ORB-SLAM không), và với các định nghĩa thất bại khác nhau. Đặt hai con số cạnh nhau tạo ra ảo giác so sánh — đúng thứ *research-rules.md* mục 6.2 cảnh báo là chỗ dễ tự lừa mình nhất.

Điều *có thể* nói một cách trung thực là mỗi họ *cần gì từ thế giới*, và cần gì thì hỏng khi thiếu cái đó. Indirect cần vắn có cấu trúc đủ để trích ra đặc trưng lặp lại được; nó bất chấp thay đổi ánh sáng trong giới hạn rộng. Direct cần ánh sáng ổn định hoặc một mô hình quang trắc đúng; nó bất chấp việc thiếu góc cạnh, chỉ cần có gradient. Hai yêu cầu ấy độc lập với nhau, nên có những môi trường mà cái này sống cái kia chết, và ngược lại. Đó là phát biểu trung thực nhất mà tôi đưa ra được, và nó hữu ích hơn một bảng số liệu.

TRỰC GIÁC

Hình dung bạn phải kiểm tra xem một bản đồ có khớp với thực địa không.

Cách indirect: bạn chọn ra hai mươi mốc dễ nhận — ngã tư, tháp nước, cây đa đầu làng — rồi kiểm xem từng mốc có ở đúng chỗ bản đồ nói không. Ưu điểm: mỗi mốc là một phép kiểm rõ ràng, và bạn không quan tâm hôm nay trời nắng hay mưa. Nhược điểm: bạn đã *vứt bỏ* toàn bộ phần còn lại của thực địa — những con đường, những cánh đồng — và nếu vùng đó không có mốc nào dễ nhận thì bạn bó tay.

Cách direct: bạn không chọn mốc nào. Bạn chồng bản đồ lên ảnh chụp từ trên cao và đo xem chúng lệch nhau bao nhiêu ở *mọi* chỗ có sự biến đổi — mọi ranh giới, mọi bờ ruộng. Ưu điểm: bạn dùng được rất nhiều thông tin, kể cả ở vùng không có mốc nào. Nhược điểm: nếu ảnh chụp hôm nay sáng hơn ảnh gốc — vì trời nắng hơn, vì máy chỉnh phơi sáng — thì mọi chỗ đều lệch, và bạn sẽ kết luận nhầm rằng bản đồ sai. Đó là toàn bộ sự đánh đổi. Cách một bên với thay đổi bề ngoài nhưng vứt bỏ phần lớn dữ liệu. Cách hai dùng gần hết dữ liệu nhưng phải biết chính xác điều kiện chụp. Và một chi tiết quyết định về cách hai: ở giữa một cánh đồng phẳng lì, không có gì để so. Vùng không có sự biến đổi đóng góp *đúng bằng không*. Nên direct không thật sự dùng “mọi pixel” — nó dùng mọi pixel *có gradient*, và ở một bức tường trắng thì con số đó cũng bằng không, y như indirect.

8.2 Ba họ phần dư

A. Indirect: sai số chiếu lại

$$\mathbf{r}_{ij}^{\text{ind}} = \mathbf{z}_{ij} - \pi(T_i^{-1} \mathbf{P}_j) \in \mathbb{R}^2.$$

Jacobian đã dẫn ở A/03 công thức (3.4). Trọng số $\Omega = \sigma^{-2}I$ với σ là độ chính xác subpixel đo ở B/07 mục 4B.

Ba tính chất quyết định. *Bên với vẻ ngoài:* descriptor được thiết kế bất biến với ánh sáng và một phần với góc nhìn, nên thay đổi phơi sáng không ảnh hưởng gì. *Ít phần dư:* vài trăm quan sát mỗi khung thay vì vài chục nghìn — rẻ, và cấu trúc thưa của A/04 giữ nguyên hoàn hảo. *Mất thông tin:* toàn bộ ảnh bị rút xuống vài trăm tọa độ; mọi thứ ở vùng ít vân bị vứt.

Điểm thứ ba là điểm đáng suy nghĩ nhất. Một bức tường trắng có gradient rất yếu nhưng *không phải bằng không*, và direct khai thác được phần thông tin ít ỏi đó trong khi indirect thì không — vì detector cần một cực đại rõ ràng để tồn tại. Đây là lý do direct thường sống lâu hơn trong hành lang trống, và nó là một lợi thế thật chứ không phải lý thuyết.

B. Direct: sai số cường độ

$$r_{ij}^{\text{dir}} = I_i(\mathbf{u}) - I_j(\pi(T_{ij} \pi^{-1}(\mathbf{u}, \rho))) \in \mathbb{R}.$$

Một chiều cho mỗi pixel. Trọng số dựa trên nhiễu cảm biến và độ tin cậy của gradient.

Ba yêu cầu mà direct đặt ra, và chúng là cái giá của nó.

Một — mô hình quang trắc. Nếu phơi sáng đổi giữa hai khung thì *mọi* phần dư lệch cùng một lượng, và bộ giải sẽ diễn giải điều đó thành một chuyển động. DSO xử lý bằng hai cách đồng thời: hiệu chuẩn ngoại tuyến vignette và CRF (A/02 mục 5B), và ước lượng trực tuyến hai tham số affine (a, b) cho mỗi khung [22]. Cách thứ hai đáng chú ý vì nó là luận đề trung tâm áp dụng ở quy mô nhỏ: thay vì fix một tham số ta không biết, thả nó ra làm biến.

Hai — khởi tạo tốt. Hàm mục tiêu quang trắc *rất* không lồi: bề mặt cường độ của một ảnh thật có hàng nghìn cực tiểu địa phương. Direct chỉ hội tụ trong một bán kính hẹp, nên nó bắt buộc phải dùng thấp ảnh (coarse-to-fine) và một tiên nghiệm chuyển động. Đây là giới hạn nghiêm trọng nhất và là lý do direct dễ mất bám khi chuyển động nhanh.

Ba — gradient. Pixel không có gradient cho Jacobian bằng không, đóng góp đúng bằng không. Nên “dùng mọi pixel” thực ra là “dùng mọi pixel có gradient”, và DSO chọn có chọn lọc vài nghìn pixel phân bố đều thay vì dùng hết — một quyết định hội tụ về chính nguyên tắc phân bố ở B/07 mục 4A.

C. Semi-direct

SVO [57] kết hợp theo một cách không phải thoả hiệp mà là phân công theo thể mạnh: *direct* cho *tracking*, *indirect* cho *mapping*.

Logic: *tracking* cần *nhANH* và chỉ giải sáu ẩn với khởi tạo tốt từ khung trước — đúng chế độ mà *direct* mạnh. *Mapping* cần *bền* vì landmark sống lâu và phải nhận ra lại sau nhiều khung — đúng chế độ mà *indirect* mạnh, vì descriptor cho phép nhận ra lại sau khi mất bám.

Kết quả là một hệ rất nhanh với độ bền chấp nhận được. Đây là một ví dụ tốt về việc *nhận ra hai bài toán con có yêu cầu khác nhau* thay vì chọn một công cụ cho cả hai — một khuôn mẫu thiết kế xuất hiện lại ở B/09 và C/13.

Bảng 8.1. Ba họ phần dư, so sánh theo *cơ chế và yêu cầu*, không theo con số. Cột “cần gì từ thế giới” là cột quyết định lựa chọn.

Họ	Cần gì từ thế giới	Hỏng khi	Được gì
Indirect	Vân có cấu trúc đủ để trích đặc trưng lặp lại được	Vùng ít vân; vân lặp lại (perceptual aliasing)	Bền với đổi ánh sáng và phơi sáng; rẻ; cấu trúc thừa hoàn hảo; dễ nhận ra lại sau mất bám
Direct	Ánh sáng ổn định hoặc mô hình quang trắc đúng; gradient khác không	Đổi phơi sáng không mô hình hoá; chuyển động nhanh (không lỗi); vật động	Dùng được vùng gradient yếu; độ chính xác subpixel ngăm; không cần bước trích đặc trưng
Semi-direct	Cả hai, nhưng ở hai chỗ khác nhau	Kế thừa cả hai chế độ hỏng, nhẹ hơn	Nhanh như <i>direct</i> , bền như <i>indirect</i> ở chỗ cần bền

I 8.3 Tham số hoá landmark

A. Vì sao XYZ không đủ

Cách hiển nhiên là ba toạ độ Euclid. Nó hỏng với điểm xa, và lý do đã được dẫn đầy đủ ở A/06 mục 3B — đây là câu trả lời cho câu hỏi mở đầu thứ hai.

Tóm tắt lập luận. Quan hệ giữa phép đo (disparity, hoặc parallax) và z là *nghịch đảo*. Nhiều Gauss trên phép đo, đi qua một hàm nghịch đảo, cho phân bố *lệch* trên z : đuôi phía xa dài hơn nhiều. Ở giới hạn parallax rất nhỏ, phân bố của z thậm chí không có kỳ vọng hữu hạn. Hệ quả: một ellipsoid Gauss trong toạ độ XYZ là mô tả *sai* về bất định của một điểm xa, và bộ giải làm việc với một mô hình sai.

Có một hệ quả thứ hai, thực dụng hơn: *điểm ở vô hạn không biểu diễn được*. Trong SLAM ngoài trời, nhiều điểm gần như ở vô hạn, và chúng cực kỳ hữu ích cho việc ước lượng xoay (A/03 mục 5, khối $[P_c]_{\times}$). Với tham số hoá XYZ, ta buộc phải hoặc gán cho chúng một z hữu hạn tuỳ tiện, hoặc vứt bỏ chúng. Cả hai đều lãng phí.

B. Nghịch đảo độ sâu có neo

Tham số hoá đúng [7]: biểu diễn điểm bằng một khung *neo* a , một hướng tia $\mathbf{m}$ trong khung đó, và một nghịch đảo độ sâu ρ :

$$\mathbf{P} = \mathbf{c}_a + \frac{1}{\rho} R_a \mathbf{m}.$$

Nếu hướng tia lấy trực tiếp từ quan sát đầu tiên và giữ cố định, chỉ còn *một* ẩn ρ — giảm từ ba xuống một, một khoản tiết kiệm đáng kể khi có hàng trăm nghìn landmark.

Ba lợi ích. *Phân bố gần Gauss*: ρ tỉ lệ tuyến tính với disparity, nên nhiều Gauss trên phép đo cho phân bố gần Gauss trên ρ — xấp xỉ của A/06 trở lại hợp lệ. *Điểm ở vô hạn hợp lệ*: $\rho = 0$ là một giá trị bình thường, không phải kỳ dị, nên một điểm rất xa vẫn nằm trong bài toán và vẫn đóng góp cho ước lượng xoay. *Tuyến tính hoá tốt hơn*: hàm đo gần tuyến tính theo ρ hơn nhiều so với theo z , nên Gauss-Newton hội tụ tốt hơn ở chính chế độ khó.

Cái giá: cần theo dõi khung neo, và khi khung neo bị marginalize thì phải chuyển landmark sang neo mới — một phép toán không miễn phí và là một nguồn lỗi cài đặt. Đây là loại phức tạp mà một hệ sản phẩm phải chấp nhận, và là lý do C/13 coi quản lý bản đồ là một chương riêng.

C. Structureless: khi landmark biến mất

MSCKF [60] đi xa hơn: *không* đưa landmark vào vector trạng thái. Đây là câu trả lời cho câu hỏi mở đầu thứ ba.

Ý tưởng. Với một landmark quan sát từ n khung, xếp toàn bộ phần dư thành một vectơ:

$$\mathbf{r} \approx H_x \delta_x + H_f \delta_f + \mathbf{n},$$

với δ_f là nhiễu loạn của landmark. Lấy N là ma trận có các cột trực chuẩn sinh ra *không gian rộng trái* của H_f , rồi nhân trái:

$$N^\top \mathbf{r} = N^\top H_x \delta_x + N^\top \mathbf{n}.$$

Landmark biến mất hoàn toàn. Cái còn lại là một ràng buộc *chỉ giữa các tư thế* — chính là thông tin mà landmark mang về hình học tương đối, tách khỏi vị trí cụ thể của nó.

Thông tin đi đâu? Nó không mất: nó được chuyển thành ràng buộc giữa các tư thế. Về mặt đại số, đây là phần bù Schur của A/04 mục 4C viết ở dạng khác — khử một khối biến nhưng giữ nguyên ảnh hưởng của nó lên phần còn lại. Khác biệt duy nhất là ở đây ta *không* thể ngược để lấy lại landmark, vì ta không cần nó.

Vì sao điều này quan trọng cho bộ lọc: chi phí của EKF là bậc ba theo kích thước trạng thái, nên một trạng thái chứa hàng nghìn landmark là bất khả thi. Structureless giữ trạng thái chỉ gồm một cửa sổ tư thế, nên chi phí không phụ thuộc số landmark. Đây chính là điều làm MSCKF khả thi trong khi EKF-SLAM thì không — và là một trong những kỹ thuật đẹp nhất trong cả cây, vì nó đổi một bài toán bậc ba thành bậc thấp bằng một phép chiếu.

Cái giá: landmark không được ước lượng, nên không có bản đồ để tái sử dụng. MSCKF là một hệ *odometry*, không phải SLAM đầy đủ — nó không đóng vòng được vì không có gì để nhận ra lại. Đây là một đánh đổi kiến trúc thật, và B/09 bàn về nó.

8.4 Các loại phần dư khác và cái giá ẩn

A. Phần dư độ sâu và ICP

Với RGB-D hoặc stereo, ta có phép đo độ sâu trực tiếp. Cách đưa vào đơn giản: thêm một chiều vào phần dư chiều, $r_z = z_{\text{đo}} - z_{\text{dự đoán}}$, với trọng số theo công thức $\sigma_z \approx z^2 \sigma_d / (bf)$ ở A/06.

Điểm đáng nhấn mạnh: trọng số ấy *phải* phụ thuộc z . Gán một σ_z không đổi cho mọi điểm là một trong những lỗi phổ biến nhất với RGB-D, và nó gây hại theo cả hai chiều — điểm gần bị tin quá ít, điểm xa bị tin quá nhiều. Cái thứ hai nguy hiểm hơn: nó bơm sai lệch có hệ thống vào bài toán và là một nguồn của bệnh quá tự tin ở A/06 mục 4B.

ICP dùng phần dư hình học giữa hai đám mây điểm. Nó là chuẩn mực trong LiDAR SLAM và xuất hiện trong RGB-D SLAM dày. Điểm cần biết cho chương này: ICP có bài toán data association của riêng nó — việc ghép điểm gần nhất — và bài toán đó có cùng cấu trúc và cùng chế độ hỏng như B/07.

B. Đường thẳng và mặt phẳng: cái giá ẩn

Trong môi trường nhân tạo, đường thẳng và mặt phẳng có mặt khắp nơi và ổn định hơn điểm. Việc thêm chúng vào có vẻ hiển nhiên có lợi, và nó có — nhưng có một cái giá ít được nói tới, và đó là câu trả lời cho câu hỏi mở đầu thứ tư.

Quay lại A/04 mục 4B: phần bù Schur hoạt động được vì khối V là khối chéo, và V khối chéo vì không nhân tử nào nối hai landmark với nhau. Một ràng buộc “ba điểm này cùng nằm trên một mặt phẳng” là đúng một nhân tử nối ba landmark. Khối V mất tính khối chéo, nghịch đảo của nó không còn tách được thành các phép nghịch đảo 3×3 độc lập, và toàn bộ lợi thế tính toán của Schur biến mất.

Đây là một ví dụ quan trọng của một nguyên tắc chung: *thêm ràng buộc đúng không miễn phí*. Ràng buộc cấu trúc cho thông tin thêm nhưng phá vỡ tính thưa. Cách xử lý thực tế là mô hình hoá mặt phẳng như một *landmark riêng* với các điểm nối vào nó — lúc đó cấu trúc thưa khôi phục được một phần, vì các điểm lại chỉ nối với một biến chung thay vì nối với nhau. Ràng buộc Manhattan hoặc Atlanta (các mặt phẳng vuông góc hoặc song song với nhau) thì mạnh hơn nữa và cũng nguy hiểm hơn: chúng đúng trong nhiều toà nhà và sai trong nhiều toà nhà khác, và khi sai thì chúng bơm vào một sai lệch có hệ thống mà không có gì báo.

C. Prior độ sâu học được

Các mô hình độ sâu metric đơn ảnh gần đây — Depth Anything v2 [61], UniDepth [62] — cho một ước lượng độ sâu có đơn vị mét từ một ảnh duy nhất. Nghe như nó giải quyết bài toán tỉ lệ của mono, và đây là câu trả lời cho câu hỏi mở đầu thứ năm.

Cách đưa vào đồ thị đúng nguyên tắc: thêm một nhân tử prior trên ρ của mỗi landmark, với hiệp phương sai phản ánh độ tin cậy của mô hình. Điều này hợp lệ và hữu ích. Nhưng nó có ba điều kiện, và vi phạm bất kỳ điều kiện nào thì đưa vào còn tệ hơn không dùng.

Một — hiệp phương sai phải trung thực. Nếu mô hình sai nhiều hơn ta khai báo, prior kéo nghiệm đi và không có gì cảnh báo. Vấn đề: các mô hình độ sâu học được *không* đi kèm một bất định đã hiệu chuẩn, và bất định của chúng thay đổi mạnh theo loại cảnh — tốt trong nhà kiểu phòng khách, tệ với bề mặt phản chiếu hoặc cảnh ngoài phân bố huấn luyện. C/17 bàn về điều này.

Hai — sai số phải gần độc lập giữa các pixel. Không đúng chút nào: một mô hình đơn ảnh sai cả một vùng cùng lúc, theo một cách tương quan mạnh. Đưa vào như các prior độc lập là *đếm thừa* thông tin nghiêm trọng — nguồn số một của bệnh quá tự tin ở A/06 mục 4B.

Ba — phải có cổng phát hiện khi mô hình ra ngoài phân bố. Đây là điều kiện khó nhất và chưa ai giải tốt.

Cách dùng an toàn nhất trong thực tế: dùng prior học được để *khởi tạo* tỉ lệ và độ sâu, rồi giảm mạnh trọng số của nó (hoặc bỏ hẳn) khi đã có đủ quan sát hình học. Như vậy ta lấy được phần giá trị lớn nhất — vượt qua chỗ khó là khởi tạo — mà không để một mô hình không được hiệu chuẩn chi phối nghiệm dài hạn.

CẠM BÃY

Một nhân tử có hiệp phương sai sai còn tệ hơn không có nhân tử đó, và đây là nguyên tắc quan trọng nhất của mục 4.

Lý do nằm ở chính khung MAP. Trọng số $\Omega = \Sigma^{-1}$ quyết định nhân tử này được tin bao nhiêu *so với* mọi nhân tử khác. Khai Σ quá nhỏ nghĩa là bạn ra lệnh cho bộ giải tin nguồn này hơn tất cả — và nếu nguồn ấy có sai lệch có hệ thống, sai lệch đó sẽ chảy vào toàn bộ nghiệm, kéo theo cả những phần vốn được ước lượng tốt.

Điều làm nó đặc biệt nguy hiểm: *không có gì cảnh báo*. Phần dư của các nhân tử khác sẽ tăng lên, nhưng bộ giải diễn giải điều đó là “các nguồn kia kém chính xác” chứ không phải “nguồn này sai”. Bạn thấy một hệ thống hội tụ đẹp về một nghiệm sai.

Quy tắc thực hành: trước khi thêm bất kỳ loại nhân tử mới nào vào đồ thị, hãy trả lời được *hiệp phương sai của nó từ đâu ra*. Nếu câu trả lời là “tôi chỉnh cho tới khi nó chạy” thì bạn đã có một hằng số tùy tiện trong hệ thống, và ba năm sau không ai biết vì sao nó bằng con số đó. Với nguồn học được, câu trả lời trung thực thường là “tôi không biết”, và lúc đó cách đúng là dùng nó làm khởi tạo chứ không làm ràng buộc.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [10]	Indirect thuần; landmark XYZ với quản lý cẩn thận	Chứng minh indirect vẫn cạnh tranh khi front-end và quản lý bản đồ tốt
DSO [22]	Direct thừa; nghịch đảo độ sâu có neo; hai tham số affine mỗi khung	Cài đặt tham chiếu của mục 2B; đọc cách nó chọn pixel để thấy nguyên tắc phân bố
SVO-Pro [57]	Semi-direct: direct cho tracking, indirect cho mapping	Mục 2C — phân công theo thể mạnh thay vì thỏa hiệp
MSCKF [60]	Structureless qua chiếu null-space	Mục 3C — kỹ thuật làm bộ lọc khả thi
VINS-Mono [13]	Nghịch đảo độ sâu có neo trong cửa sổ trượt	Cho thấy chi phí thật của việc chuyển neo khi marginalize
Kimera [63]	Thêm nhân tử mặt phẳng và ngữ nghĩa vào đồ thị GTSAM	Ví dụ về cái giá ẩn ở mục 4B trong một hệ thật

MINI PROJECT SLAM · So sánh ba tham số hoá landmark trên cùng dữ liệu, đặc biệt ở điểm xa · 3 ngày

Mục tiêu	tự thấy vì sao nghịch đảo độ sâu thắng, và thấy nó thắng ở đâu — vì nó không thắng ở mọi nơi.
Dữ liệu	tự sinh, để kiểm soát được phân bố độ sâu: một quỹ đạo, và ba bản đồ — toàn điểm gần (1–3 m), toàn điểm xa (20–100 m), và trộn cả hai.
Các bước	dựng BA với ba tham số hoá: XYZ, nghịch đảo độ sâu ba ẩn, và nghịch đảo độ sâu một ẩn có neo cố định tia; với mỗi tổ hợp (tham số hoá $\times$ bản đồ), đo số vòng lặp tới hội tụ, ATE cuối, số điều kiện của Λ , và — quan trọng nhất — chạy Monte-Carlo để so <i>phân bố</i> sai số độ sâu của các điểm xa với một Gauss.
Xong khi	bạn có một bảng ba nhân ba, và bạn quan sát được: với bản đồ gần, ba tham số hoá gần như tương đương; với bản đồ xa, XYZ hội tụ chậm hơn rõ rệt và phân bố sai số của nó <i>lệch</i> trong khi phân bố theo ρ thì gần Gauss. Cái sau là kết quả quan trọng nhất và là thứ xác nhận A/06 mục 3B bằng số của chính bạn.
Nối với	B/09-paradigm-uoc-luong-back-end chọn kiến trúc dùng các tham số hoá này; C/13-quan-ly-ban-do xử lý chi phí thật của việc chuyển khung neo.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì “mọi pixel” thực ra là “mọi pixel có gradient khác không” — vùng phẳng đóng góp Jacobian bằng không, nên ở một bức tường trắng direct cũng nghèo thông tin như indirect. Và cái giá phải trả có ba phần: (a) *mô hình quang trắc* phải đúng, vì thay đổi phơi sáng làm mọi phần dư lệch cùng lượng và bộ giải diễn giải đó là chuyển động; (b) *khởi tạo tốt* là bắt buộc, vì hàm mục tiêu quang trắc cực kỳ không lồi với hàng nghìn cực tiểu địa phương — đây là giới hạn nghiêm trọng nhất và là lý do direct dễ mất bám khi chuyển động nhanh; (c) khó nhận ra lại sau mất bám, vì không có descriptor để so.
- A2. Vì nghiệm thì trùng nhau nhưng *đường đi tới nghiệm* và *mô tả bất định* thì không. Quan hệ phép đo- z là nghịch đảo, nên nhiều Gauss trên phép đo cho phân bố *lệch* trên z (ở giới hạn parallax nhỏ, không có cả kỳ vọng hữu hạn), trong khi trên $\rho = 1/z$ thì gần Gauss. Hệ quả: với XYZ, xấp xỉ Gauss của hiệp phương sai là mô tả sai, và tuyến tính hoá tệ hơn nên Gauss-Newton hội tụ chậm hơn — cả hai đúng ở chế độ khó nhất là điểm xa. Thêm nữa, XYZ không biểu diễn được điểm ở vô hạn, trong khi $\rho = 0$ là giá trị hợp lệ, nên ta giữ được các điểm xa vốn rất tốt cho ước lượng xoay.
- A3. Thông tin không mất — nó được chuyển thành một ràng buộc *chỉ giữa các tư thế*, chính là phần thông tin mà landmark mang về hình học tương đối, tách khỏi vị trí cụ thể của nó. Về đại số, đây là phần bù Schur ở A/04 viết ở dạng khác, khác duy nhất ở chỗ ta không thể ngược để lấy lại landmark. Nó cho bộ lọc chi phí không phụ thuộc số landmark vì chi phí EKF là bậc ba theo kích thước trạng thái, và trạng thái bây giờ chỉ gồm một cửa sổ tư thế. Cái giá: không có bản đồ để tái sử dụng, nên MSCKF là odometry chứ không phải SLAM — nó không đóng vòng được.
- A4. Vì nó phá vỡ tính khối chéo của khối V trong Hessian. Phần bù Schur ở A/04 mục 4C hoạt động được *chỉ vì* không nhân tử nào nối hai landmark với nhau, nên V^{-1} tách thành các phép nghịch đảo 3×3 độc lập. Một ràng buộc đồng phẳng là một nhân tử nối nhiều landmark, và nó phá đúng tính chất đó — toàn bộ lợi thế tính toán biến mất. Cách xử lý: mô hình hoá mặt phẳng như một *landmark riêng* với các điểm nối vào nó, để các điểm lại chỉ nối với một biến chung thay vì nối trực tiếp với nhau.
- A5. Ba điều kiện. (a) *Hiệp phương sai trung thực* — nhưng các mô hình độ sâu học được không đi kèm bất định đã hiệu chuẩn, và bất định của chúng thay đổi mạnh theo loại cảnh. (b) *Sai số gần độc lập giữa các pixel* — hoàn toàn không đúng, vì một mô hình đơn ảnh sai cả một vùng theo cách tương quan mạnh, nên đưa vào như các prior độc lập là đếm thừa thông tin nghiêm trọng. (c) *Có cổng phát hiện khi ra ngoài phân bố*. Đưa vào sai tệ hơn không dùng vì trọng số quá lớn sẽ kéo cả nghiệm theo một sai lệch có hệ thống, và bộ giải sẽ diễn giải phần dư tăng của các nguồn khác là “các nguồn kia kém chính xác”. Cách an toàn: dùng làm khởi tạo rồi hạ mạnh trọng số khi đã có đủ quan sát hình học.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn muốn biết mình đã đi bao xa, bằng cách so hai bức ảnh chụp trước và sau. Có hai cách làm, và chúng khác nhau ở một điểm duy nhất.

Cách một: chọn ra vài chục thứ dễ nhận trong ảnh — góc cửa sổ, mép bàn — rồi xem chúng dịch đi đâu. Bạn không quan tâm hôm nay đèn sáng hơn hay tối hơn, vì góc cửa sổ vẫn là góc cửa sổ. Nhưng nếu bạn đang nhìn một bức tường trắng trơn thì không có gì để chọn, và bạn bó tay.

Cách hai: không chọn gì cả. Chồng hai ảnh lên nhau và trượt qua trượt lại cho tới khi chúng khớp nhất. Bạn dùng được cả những chỗ mờ nhạt mà cách một bỏ qua. Nhưng nếu ai đó vừa bật thêm đèn, hai ảnh sẽ không bao giờ khớp — và bạn sẽ kết luận nhầm rằng mình đã di chuyển.

Cả hai cách đều đúng, và không cách nào thắng ở mọi nơi. Cách một cần thể giới *có hình thù rõ*; cách hai cần ánh sáng *đúng thay đổi*. Hai đòi hỏi đó chẳng liên quan gì tới nhau, nên có phòng cách này sống cách kia chết, và ngược lại. Đó là lý do tranh luận “cái nào tốt hơn” kéo dài hai mươi năm không có kết quả — vì nó là câu hỏi sai.

Còn một chuyện nữa, về cách ghi lại vị trí của một điểm mốc. Nếu điểm mốc ở gần, ghi “cách ba mét” là hợp lý. Nếu nó ở rất xa — một ngọn núi — thì “cách mười ki-lô-mét” hay “cách hai mươi ki-lô-mét” đều là những phỏng đoán tệ như nhau, và ghi con số nào cũng gây hiểu lầm rằng ta biết. Cách khôn hơn là ghi *độ gần*: “rất gần”, “khá gần”, “gần như không gần chút nào”. Với ngọn núi,

câu cuối là một mô tả hoàn toàn chính xác — và quan trọng là, nó vẫn cho biết ngọn núi nằm về hướng nào, thứ mà ta biết rất rõ.

Chỗ không rút gọn được: chỗ không rút gọn được là việc ước lượng bất định của một mô hình học được. Một cái máy nói “chỗ này cách ba mét” — nhưng nó chắc tới mức nào? Nó không biết, và ta cũng không có cách nào biết hộ nó. Đó là lý do phần cuối chương này khuyên dùng các mô hình ấy để khởi động chứ không để quyết định, và là một trong tám khoảng trống ở D/22.

Một câu: Chọn phần dư là chọn hệ thống của bạn sẽ đòi hỏi gì ở thế giới — và thế giới không phải lúc nào cũng đáp ứng.

8.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *indirect và direct đòi hỏi hai thứ độc lập từ thế giới, nên không cái nào thắng ở mọi nơi* — bị bác bỏ nếu một hệ thuộc một họ vượt trội hệ thuộc họ kia trên mọi loại môi trường trong một khảo sát có kiểm soát, với cùng phần cứng và cùng mức hiệu chuẩn. Tôi không biết một khảo sát như vậy tồn tại, và tôi nghi ngờ nó sẽ khó thực hiện công bằng vì DSO cần hiệu chuẩn quang trắc mà ORB-SLAM không cần — nên “cùng mức hiệu chuẩn” không định nghĩa được rõ ràng. Đây là một lý do cấu trúc khiến tranh luận này khó giải quyết bằng thực nghiệm (*tôi suy ra, chưa đổi chiều nguồn*).

Mệnh đề thứ hai — *ngịch đảo độ sâu cho phân bố gần Gauss còn XYZ thì lệch* — là hệ quả toán học của việc lan truyền qua hàm nghịch đảo (cửa a, dẫn ở A/06) và kiểm được bằng Monte-Carlo trong mini project (cửa b). Nó bị bác bỏ nếu thí nghiệm cho thấy hai phân bố lệch như nhau — điều sẽ xảy ra nếu parallax luôn lớn, tức nếu bạn không có điểm xa thật sự.

Mệnh đề thứ ba — *ràng buộc cấu trúc phá vỡ tính thừa* — là một sự kiện về cấu trúc ma trận, kiểm được bằng cách nhìn mẫu khác không. Điều có thể sai là kết luận thực dụng rằng cái giá đó đáng kể: với bài toán nhỏ hoặc với số ràng buộc ít, nó có thể không đo được. Tôi chưa đo.

Mệnh đề thứ tư — *một nhân tử có hiệp phương sai sai tệ hơn không có nhân tử đó* — là mệnh đề tôi tin nhất và cũng là mệnh đề khó kiểm nhất, vì nó đòi so sánh với một chân trị. Nó bị bác bỏ nếu một hệ thêm prior học được với hiệp phương sai đoán bừa vẫn cho ATE tốt hơn hệ không dùng, một cách nhất quán qua nhiều điều kiện. Điều này có thể xảy ra khi prior đủ tốt và bài toán gốc đủ thiếu ràng buộc — và nếu nó xảy ra thì kết luận đúng không phải “hiệp phương sai không quan trọng” mà là “bài toán gốc thiếu thông tin tới mức bất kỳ prior nào cũng giúp”, tức một chẩn đoán thuộc A/05.

8.6 Kết nối

Nổi lên trên. A/04-uoc-luong-map-va-bundle-adjustment là khuôn mà chương này đổ vào, và điều đáng chú ý nhất là khuôn ấy không đổi gì dù ta thay indirect bằng direct — đó là bằng chứng mạnh nhất cho tính tổng quát của nó. A/02-mo-hinh-camera mục 5B là điều kiện tiên quyết cho direct. A/06-bat-dinh-va-lan-truyen-sai-so mục 3B cho lập luận đầy đủ về nghịch đảo độ sâu, và mục 4B cho lý do vì sao hiệp phương sai sai lại nguy hiểm. B/07-dac-trung-va-data-association là đầu vào của indirect, và điều thú vị là với direct thì cả chương đó gần như biến mất — direct không có bước trích đặc trưng, nhưng nó trả giá bằng yêu cầu khởi tạo tốt.

Nổi xuống dưới. B/09-paradigm-uoc-luong-back-end nhận structureless ở mục 3C như một kỹ thuật nền tảng, và bàn về hệ quả kiến trúc của nó — odometry so với SLAM đầy đủ. B/10-khoi-tao chịu ảnh hưởng trực tiếp: direct cần khởi tạo tốt hơn nhiều so với indirect, nên

chiến lược khởi tạo phụ thuộc vào lựa chọn ở chương này. [C/15-hop-nhat-da-cam-bien](#) mở rộng nguyên tắc ở khối *Cam bẫy* mục 4: mỗi nguồn thêm vào đồ thị cần hiệp phương sai đúng và cố gắng loại outlier riêng. [C/16-dense-mapping-va-bieu-dien](#) dùng phần dư quang trắc và độ sâu ở quy mô dày. [C/17-hoc-may-trong-slam](#) bàn kỹ hơn về prior học được ở mục 4C, đặc biệt về bài toán hiệu chuẩn bất định.

[D/19-ben-vung-trong-the-gioi-thuc](#) là chỗ hai họ phần dư lộ ra chế độ hỏng thật: direct rất nhạy với vật động (vì một vật động làm hỏng cường độ của cả một vùng) và với thay đổi ánh sáng đột ngột. [D/20-toi-uu-hieu-nang-va-he-thong](#) là chỗ chi phí tính toán của lựa chọn này được cân: direct có nhiều phần dư hơn hàng chục lần nhưng mỗi phần dư rẻ hơn, và kết quả ròng phụ thuộc vào phần cứng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu điều mà indirect cần từ thế giới và điều mà direct cần, rồi nói vì sao hai điều đó độc lập với nhau.
2. “Direct dùng mọi pixel.” Câu đó sai ở chỗ nào, và sự thật là gì?
3. Vì sao nghịch đảo độ sâu tuyến tính hoá tốt hơn XYZ với điểm xa? Trả lời bằng ngôn ngữ hình dạng phân bố, không bằng ngôn ngữ “nó chạy tốt hơn”.
4. Structureless khử landmark khỏi trạng thái. Thông tin đi đâu, và kỹ thuật này liên hệ thế nào với phần bù Schur ở [A/04](#)?
5. Vì sao thêm một ràng buộc đồng phẳng — một ràng buộc *đúng* — lại làm bài toán chậm đi nhiều bậc?
6. Nêu ba điều kiện để đưa một prior độ sâu học được vào đồ thị một cách hợp lệ, và nói điều kiện nào khó thoả nhất.
7. Vì sao một nhân tử có hiệp phương sai sai lại tệ hơn không có nhân tử đó, và vì sao không có gì cảnh báo khi điều đó xảy ra?

Paradigm ước lượng back-end (Back-End Estimation Paradigms)

PHỤ THUỘC A/04-uoc-luong-map-va-bundle-adjustment (MAP, Schur, marginalization — chương này là các lựa chọn khác nhau trong khung đó); A/05-observability-gauge-va-suy-bien (nhất quán và FEJ); B/08-mo-hinh-phan-du (structureless là kỹ thuật nền cho MSCKF). Phần quán tính tương ứng ở ../IMU_Fusion/ chương 8 và 9.

TÓM TẮT

Người mới học SLAM gặp một danh sách các kiến trúc back-end trông không liên quan: EKF, MSCKF, cửa sổ trượt, fixed-lag, iSAM2, full batch BA, pose graph. Mỗi cái có bài báo riêng, ký hiệu riêng, và cộng đồng riêng. Chương này lập luận rằng chúng là *một* bài toán MAP của A/04 với các lựa chọn khác nhau trên đúng hai trục: *marginalize những gì*, và *tuyến tính hoá lại bao nhiêu lần*. Hiểu hai trục đó thì danh sách sụp thành một bảng, và việc chuyển qua lại giữa các thể giới trở nên tự nhiên. Nội dung có bốn phần. Một: chứng minh sự tương đương, và chỉ ra chính xác chỗ bộ lọc đánh đổi cái gì. Hai: bốn kiến trúc chính đặt trên hai trục ấy. Ba: pose graph như một trường hợp riêng quan trọng — nó là một bài toán MAP mà mọi landmark đã bị marginalize. Bốn: hai hướng phát triển đáng biết là dạng căn bậc hai (ổn định số học) và certifiable (bảo đảm nghiệm toàn cục). Nếu chỉ nhớ một điều: *bộ lọc là bài toán tối ưu đã marginalize toàn bộ quá khứ và chỉ tuyến tính hoá một lần* — mọi khác biệt về chi phí và về tính nhất quán đều là hệ quả của câu đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bộ lọc (filter)** — bộ ước lượng chỉ giữ trạng thái hiện tại và hiệp phương sai của nó, cập nhật tuần tự khi có phép đo mới. Mọi thông tin quá khứ được nén vào hai đại lượng đó.
- **smoothing** — bộ ước lượng giữ *nhiều* trạng thái quá khứ và giải lại cho tất cả khi có phép đo mới. Kết quả tốt hơn vì một phép đo muộn có thể sửa một ước lượng sớm.
- **cửa sổ trượt (sliding window)** — smoothing trên một cửa sổ kích thước cố định; trạng thái ra khỏi cửa sổ được marginalize thành nhân tử prior.
- **fixed-lag smoothing** — tên khác của cửa sổ trượt, nhấn mạnh rằng ước lượng cho một thời điểm được chốt sau một độ trễ cố định.
- **smoothing gia tăng (incremental smoothing)** — giữ *toàn bộ* lịch sử nhưng chỉ tính lại phần bị ảnh hưởng khi có phép đo mới. iSAM2 làm điều này qua cấu trúc Bayes tree.
- **pose graph** — đồ thị nhân tử chỉ gồm các tư thế, với các cạnh là ràng buộc tư thế tương đối. Landmark đã bị khử; đây là dạng nén nhất của bài toán còn giữ được tính nhất quán toàn cục.
- **dạng căn bậc hai (square-root form)** — làm việc với thừa số R sao cho $\Lambda = R^T R$ thay vì với Λ . Số điều kiện giảm còn căn bậc hai, nên ổn định số học tốt hơn hẳn.
- **certifiable** — thuật toán không chỉ trả về nghiệm mà còn trả về *chứng chỉ* rằng nghiệm đó là cực tiểu toàn cục — điều mà Gauss-Newton không bao giờ cho được.

CÂU HỎI MỞ ĐẦU

- Q1. EKF-SLAM và bundle adjustment cùng giải một bài toán MAP. Vậy khác biệt thật nằm ở đâu, và nêu chính xác hai lựa chọn phân biệt chúng.

- Q2. EKF-SLAM có chi phí bậc hai theo số landmark mỗi bước. Vì sao lại là bậc hai, khi mà trận thông tin của bài toán thì thưa?
- Q3. MSCKF giữ một cửa sổ tư thế và khử landmark. Nó rẻ và chính xác. Vì sao nó *không* phải một hệ SLAM, và điều đó có hậu quả gì cho sản phẩm?
- Q4. iSAM2 giữ toàn bộ lịch sử nhưng chạy thời gian thực. Nghe như nó có tất cả mà không mất gì. Điều gì nó vẫn phải trả giá, và khi nào cái giá đó trở nên nghiêm trọng?
- Q5. Dạng căn bậc hai cho cùng nghiệm với cùng chi phí bậc độ lớn, nhưng ổn định số học tốt hơn hẳn. “Tốt hơn hẳn” ở đây là bao nhiêu, và nó đến từ đâu?

9.1 Đặt vấn đề

Ai học SLAM theo thứ tự lịch sử sẽ gặp một danh sách gây bối rối. Đầu tiên là EKF-SLAM, thống trị những năm 1990 và 2000. Rồi PTAM [58] chứng minh rằng bundle adjustment chạy được thời gian thực, và cộng đồng chuyển sang tối ưu hoá. Rồi MSCKF [60] cho thấy bộ lọc vẫn cạnh tranh nếu làm đúng. Rồi iSAM2 [31] cho thấy smoothing đầy đủ cũng chạy thời gian thực được. Bốn câu chuyện, bốn cộng đồng, bốn bộ ký hiệu.

Cách học đó để lại một ấn tượng sai: rằng đây là những phương pháp *cạnh tranh* và ta phải chọn một. Ấn tượng ấy làm người học phải học lại từ đầu mỗi lần đổi hệ thống, và tệ hơn, nó che mất việc các đánh đổi là gì.

Bài toán gốc của chương: *tồn tại một khung nào mà trong đó cả bốn đều là trường hợp riêng không?* Câu trả lời là có, và khung đó chính là bài toán MAP ở A/04. Mọi kiến trúc trong danh sách trên đều cực tiểu hoá cùng một hàm mục tiêu; chúng khác nhau ở hai lựa chọn kỹ thuật:

Trục một — *marginalize* những gì. Giữ lại bao nhiêu quá khứ trong bài toán? Từ “không giữ gì” (bộ lọc) tới “giữ tất cả” (full batch).

Trục hai — *tuyến tính hoá lại bao nhiêu lần*. Jacobian được tính tại điểm nào, và có tính lại khi ước lượng di chuyển không? Từ “một lần, không bao giờ sửa” (EKF) tới “mỗi vòng lặp, cho mọi biến” (full batch).

Hai trục ấy giải thích *tất cả* khác biệt về chi phí, độ chính xác và tính nhất quán. Và quan trọng hơn, chúng cho một cách suy nghĩ: khi thiết kế hệ thống, bạn không chọn giữa bốn cái tên mà chọn một điểm trên một mặt phẳng hai chiều.

Ai gặp bài toán này? Mọi người thiết kế back-end, và quyết định này định hình yêu cầu tính toán, độ trễ, và khả năng đóng vòng. Nó cũng là quyết định khó đảo ngược nhất trong cả hệ thống — đổi detector thì mất một tuần, đổi kiến trúc back-end thì viết lại gần hết.

TRỰC GIÁC

Hình dung bạn đang ghi chép một chuyến đi dài.

Cách bộ lọc: bạn chỉ giữ *một* trang giấy, ghi “tôi đang ở đây, và tôi chắc chắn chừng này”. Mỗi khi có thông tin mới, bạn cập nhật trang đó rồi *xoá* thông tin cũ — vì nó đã được gộp vào con số hiện tại. Rất rẻ, bộ nhớ không đổi. Cái giá: nếu ba giờ sau bạn phát hiện một suy luận lúc sáng là sai, bạn không sửa được. Trang giấy không còn ghi suy luận đó nữa; nó chỉ còn kết quả.

Cách full batch: bạn giữ *toàn bộ* nhật ký, mọi quan sát từ đầu chuyến đi. Mỗi khi có thông tin mới, bạn đọc lại tất cả và suy luận lại từ đầu. Kết quả tốt nhất có thể, vì một thông tin muộn có thể sửa một hiểu lầm sớm. Cái giá: chi phí tăng theo độ dài chuyến đi, nên tới ngày thứ mười thì không kịp nữa.

Cách của sổ trượt: bạn giữ nhật ký của *ba ngày gần nhất* và tóm tắt mọi thứ cũ hơn thành một đoạn ghi chú. Chi phí không đổi, và bạn vẫn sửa được các hiểu lầm trong ba ngày qua. Cái giá nằm ở đoạn tóm tắt: một khi đã tóm tắt, bạn không mở lại được, và nếu lúc tóm tắt bạn đang hiểu sai thì cái hiểu sai ấy bị đóng băng.

Cách smoothing gia tăng: bạn giữ toàn bộ nhật ký nhưng tổ chức nó *rất khéo*, sao cho khi có thông tin mới bạn chỉ phải đọc lại đúng những trang bị ảnh hưởng. Nếu thông tin mới chỉ liên quan tới hôm nay thì bạn chỉ đọc hôm nay. Nhưng nếu bạn nhận ra hôm nay mình đang ở đúng chỗ đã đi qua tuần trước — một loop closure — thì bạn phải đọc lại cả tuần, và lúc đó chi phí vọt lên.

Bốn cách, một bài toán. Khác biệt duy nhất: giữ lại bao nhiêu, và có cho phép sửa lại suy luận cũ hay không.

9.2 Bộ lọc và tối ưu là cùng một bài toán

A. Chứng minh sự tương đương

Xét bài toán MAP với các trạng thái $\mathbf{x}_{0:k}$ và phép đo tới thời điểm k . Full batch giải cho tất cả:

$$\hat{\mathbf{x}}_{0:k} = \arg \min \left[\underbrace{\|\mathbf{x}_0 - \bar{\mathbf{x}}_0\|_{\Sigma_0}^2}_{\text{prior}} + \sum_i \|\mathbf{r}_i^{\text{do}}\|_{\Omega_i}^2 + \sum_i \|\mathbf{r}_i^{\text{chuyển}}\|_{Q_i}^2 \right].$$

Bây giờ áp dụng marginalization của A/04 mục 5 một cách cực đoan: sau mỗi bước thời gian, marginalize *mọi* trạng thái trừ trạng thái mới nhất. Theo mục 5A ở chương đó, marginalize một khối biến Gauss chính là phần bù Schur, và kết quả là một nhân tử prior trên các biến còn lại. Vì chỉ còn một trạng thái, prior ấy là một Gauss đơn giản: một trung bình và một hiệp phương sai.

Đó *chính xác* là những gì EKF giữ. Bước dự đoán của EKF là việc lan truyền prior qua nhân tử chuyển động; bước cập nhật là việc gộp nhân tử đo mới vào prior. Cùng công thức, cùng kết quả.

Nhưng có một khác biệt thứ hai, và nó là khác biệt quan trọng hơn. Trong full batch, khi ta thêm phép đo mới, ta *tuyến tính hoá lại* mọi phần dư tại ước lượng mới nhất. Trong EKF, Jacobian của một phép đo được tính *một lần* tại ước lượng lúc đó và không bao giờ sửa. Khi ước lượng di chuyển — và nó luôn di chuyển — Jacobian cũ trở nên lỗi thời, nhưng thông tin đã được nén vào hiệp phương sai và không tách ra được nữa.

CẦN NHỚ

Bộ lọc = bài toán tối ưu đã marginalize toàn bộ quá khứ, và chỉ tuyến tính hoá một lần.

Hai vế của câu này giải thích mọi thứ còn lại.

Về *marginalize toàn bộ quá khứ* giải thích chi phí: trạng thái không lớn lên theo thời gian, nên bộ lọc chạy được vô hạn. Nó cũng giải thích một hệ quả ít được nói: bộ lọc *không thể* sửa lại một ước lượng cũ, vì ước lượng cũ không còn tồn tại như một biến.

Về *tuyến tính hoá một lần* giải thích độ chính xác và tính nhất quán. Với mô hình phi tuyến mạnh — và phép chiếu camera phi tuyến mạnh — Jacobian tính tại một ước lượng tệ sẽ sai, và cái sai ấy bị đóng băng vĩnh viễn. Tệ hơn, như A/05 mục 6 chỉ ra, các Jacobian tính tại các điểm khác nhau *không tương thích* và tạo ra thông tin giả theo hướng không quan sát được. Đó là gốc của bệnh quá tự tin, và là lý do FEJ tồn tại.

Hệ quả thiết kế: nếu bạn cần chi phí không đổi và chấp nhận không đóng vòng, dùng bộ lọc và **bắt buộc** phải xử lý nhất quán (FEJ hoặc InEKF). Nếu bạn cần đóng vòng và sửa lại quá khứ, bạn cần giữ quá khứ, tức smoothing ở một dạng nào đó.

B. Vì sao EKF-SLAM có chi phí bậc hai

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và nó là một nghịch lý đáng hiểu.

Trong bài toán MAP, ma trận *thông tin* Λ của SLAM *thừa* — đó là toàn bộ nội dung của A/04 mục 4B. Nhưng EKF không giữ Λ ; nó giữ $\Sigma = \Lambda^{-1}$. Và như A/06 mục 2B đã nêu, *nghịch đảo của một ma trận thừa nói chung là đặc*.

Lý do trực giác: hai landmark chưa bao giờ được nhìn thấy cùng lúc vẫn *tương quan* với nhau, vì cả hai đều tương quan với quỹ đạo nối chúng. Thông tin “chúng không nối trực tiếp” nằm trong Λ ; thông tin “chúng vẫn tương quan” nằm trong Σ . Bộ lọc chọn biểu diễn thứ hai, và phải trả giá.

Hậu quả: với n landmark, Σ có $O(n^2)$ phần tử, và mỗi bước cập nhật EKF phải chạm vào tất cả — chi phí $O(n^2)$ mỗi bước, $O(n^2)$ bộ nhớ. Với vài nghìn landmark là hết. Đây là lý do kỹ thuật khiến EKF-SLAM không mở rộng được, và là lý do MSCKF quan trọng: bằng cách khử landmark qua *structureless* (B/08 mục 3C), nó giữ trạng thái chỉ gồm một cửa sổ tư thế, và chi phí không còn phụ thuộc số landmark.

Có một họ thứ ba đáng biết: *information filter*, giữ Λ thay vì Σ . Nó giữ được tính thừa — nhưng để lấy ra ước lượng thì phải giải một hệ, và để lấy hiệp phương sai thì phải nghịch đảo. Nói cách khác, nó chuyển chi phí từ lúc cập nhật sang lúc truy vấn. Với SLAM, nơi ta cập nhật liên tục nhưng chỉ thỉnh thoảng cần hiệp phương sai đầy đủ, đây thường là đánh đổi tốt hơn — và đó chính là cách mà các phương pháp smoothing hiện đại làm việc.

I 9.3 Bốn kiến trúc trên hai trục

Bảng 9.1. Bốn kiến trúc đặt trên hai trục: giữ lại bao nhiêu quá khứ, và tuyến tính hoá lại bao nhiêu. Mọi khác biệt về chi phí và nhất quán đều là hệ quả của hai cột đầu.

Kiến trúc	Giữ lại	Tuyến tính hoá lại	Hệ quả	Đại diện
Bộ lọc (EKF)	Chỉ trạng thái hiện tại	Không bao giờ	Chi phí không đổi theo thời gian; bắt nhất nếu không FEJ; không sửa được quá khứ	EKF-SLAM, ESKF
Bộ lọc cửa sổ (MSCKF)	Cửa sổ tư thế, không có landmark	Không bao giờ	Chi phí không phụ thuộc số landmark; không có bản đồ nên không đóng vòng	MSCKF, OpenVINS
Cửa sổ trượt	Cửa sổ tư thế + landmark; cũ hơn thì <i>marginalize</i>	Trong cửa sổ, mỗi vòng lặp	Chi phí không đổi; sửa được quá khứ gần; prior bị đóng băng gây <i>fill-in</i> và bắt nhất	VINS-Mono, OKVIS, Basalt
Smoothing gia tăng	Toàn bộ lịch sử	Phần bị ảnh hưởng, mỗi lần	Chính xác như full batch; chi phí thấp khi cập nhật cục bộ, vọt lên khi đóng vòng	iSAM2, GTSAM
Full batch	Toàn bộ	Tất cả, mỗi vòng lặp	Nghiem tốt nhất có thể; không chạy thời gian thực	BA toàn cục, COLMAP

Ba nhận xét đọc ra từ bảng, và chúng quan trọng hơn bản thân bảng.

Thứ nhất, hai cột đầu là nguyên nhân, cột thứ ba là kết quả. Không có gì trong cột “hệ quả” là một lựa chọn thiết kế độc lập — tất cả đều suy ra từ hai lựa chọn trước. Điều này có nghĩa là nếu bạn muốn một hệ có tính chất nào đó ở cột ba, bạn phải tìm nó ở hai cột đầu.

Thứ hai, MSCKF không phải SLAM. Đây là câu trả lời cho câu hỏi mở đầu thứ ba. Vì landmark bị khử ngay sau khi dùng, không có bản đồ nào tồn tại, nên không có gì để nhận ra lại khi quay về chỗ cũ. MSCKF là một hệ *odometry* xuất sắc: trôi chậm, rẻ, ổn định. Nhưng trôi của nó là đơn điệu — nó không bao giờ tự sửa. Với sản phẩm chạy trong một không gian hữu hạn và lặp lại — robot lau nhà, AGV kho hàng — điều đó không chấp nhận được, và ta cần một lớp SLAM đầy đủ ở trên. Cách phổ biến trong thực tế là ghép: MSCKF hoặc cửa sổ trượt cho odometry tần số cao, cộng một pose graph cho nhất quán toàn cục (B/11).

Thứ ba, iSAM2 vẫn phải trả giá. Đây là câu trả lời cho câu hỏi mở đầu thứ tư. Nó giữ toàn bộ lịch sử và chỉ tính lại phần bị ảnh hưởng, nhờ Bayes tree tổ chức biến theo một thứ tự khử tốt [31]. Khi cập nhật là cục bộ theo thời gian — thêm một khung, thêm vài landmark — phần bị ảnh hưởng nhỏ và chi phí thấp. Nhưng một *loop closure* nối một tư thế mới với một tư thế rất cũ, và nó làm mất hiệu lực một phần lớn của cây; chi phí vọt lên. Với bản đồ lớn và loop closure thường xuyên, cái giá đó trở nên nghiêm trọng, và đó là lý do các hệ thực tế thường tách loop closure ra một luồng riêng chạy chậm hơn (C/13).

9.4 Pose graph: bài toán MAP đã nén

Pose graph là một trường hợp riêng quan trọng tới mức đáng một mục riêng.

Ý tưởng: khử *mọi* landmark, giữ chỉ tư thế. Mỗi cạnh của đồ thị là một ràng buộc tư thế tương đối $\hat{T}_{ij}$ kèm hiệp phương sai, và phần dư là

$$\mathbf{r}_{ij} = \log(\hat{T}_{ij}^{-1} T_i^{-1} T_j)^\vee.$$

Các ràng buộc ấy đến từ hai nguồn: odometry (giữa hai tư thế liên tiếp) và loop closure (giữa hai tư thế xa nhau về thời gian).

Ba lý do pose graph quan trọng.

Một — kích thước. Với m tư thế và n landmark, bài toán đầy đủ có $6m + 3n$ ẩn với $n \gg m$; pose graph chỉ có $6m$. Điều này biến một bài toán không chạy nổi thành một bài toán giải được trong vài giây, kể cả với hàng chục nghìn tư thế.

Hai — tách mỗi quan tâm. Việc sửa trôi tích lũy là bài toán *toàn cục* và không cần độ chính xác cao ở quy mô nhỏ; việc tinh chỉnh cấu trúc cục bộ là bài toán *địa phương*. Tách chúng ra cho phép chạy hai luồng ở hai tần số khác nhau — một kiến trúc mà mọi hệ SLAM thật đều dùng.

Ba — nền cho certifiable. Vì pose graph chỉ chứa tư thế, nó có cấu trúc đại số đặc biệt cho phép các phương pháp bảo đảm nghiệm toàn cục (mục 5B).

Số bậc tự do của pose graph phải khớp với cấu hình cảm biến, đúng như A/05 mục 3B: $Sim(3)$ cho mono [26], $SE(3)$ cho stereo, và bốn bậc cho VIO vì trọng lực ghim roll với pitch. Đây không phải tối ưu hoá mà là tính đúng đắn: tối ưu một bậc tự do mà dữ liệu đã xác định chỉ tạo thêm đường cho sai số tích lũy.

Cái giá của pose graph, và nó đáng nói rõ: *thông tin bị nén một cách không phục hồi được*. Một cạnh với hiệp phương sai 6×6 không mô tả đầy đủ những gì hàng trăm quan sát landmark đã nói. Cụ thể, các cạnh khác nhau *tương quan* với nhau (vì chúng chia nhau các landmark), và pose graph chuẩn coi chúng độc lập — đúng nguồn số một của bệnh quá tự tin ở A/06 mục 4B. Đây là một xấp xỉ được chấp nhận rộng rãi vì lợi ích tính toán quá lớn, nhưng nên biết rằng nó là một xấp xỉ.

9.5 Hai hướng phát triển đáng biết

A. Dạng căn bậc hai

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó là một trong những cải tiến có tỉ lệ lợi ích trên chi phí cao nhất trong cả cây.

Thay vì dựng và phân tích $\Lambda = \sum H^\top \Omega H$, làm việc trực tiếp với một thừa số R sao cho $\Lambda = R^\top R$, thu được bằng phân tích QR của ma trận Jacobian đã whitened.

Lợi ích số học định lượng được, và đó là điểm mạnh của lập luận. Số điều kiện của Λ là *bình phương* số điều kiện của R :

$$\text{cond}(\Lambda) = \text{cond}(R^\top R) = \text{cond}(R)^2.$$

Nghĩa là việc dựng Λ tương minh *phá huỷ một nửa số chữ số có nghĩa*. Với bài toán có số điều kiện 10^8 — hoàn toàn bình thường trong BA có điểm ở nhiều thang độ sâu khác nhau — ta mất tám chữ số, và số học độ chính xác kép chỉ có mười sáu. Làm việc với R giữ được tám chữ số đó. (Cửa kiểm chứng a: suy dẫn trực tiếp từ định nghĩa số điều kiện qua giá trị kỳ dị.)

Lợi ích này lớn nhất đúng ở chỗ bài toán gần suy biến — tức đúng các cấu hình ở A/05. Basalt [23] và $\sqrt{\text{BA}}$ [37] khai thác điều này, và $\sqrt{\text{BA}}$ còn cho thấy nó cho phép dùng số học *độ chính xác đơn* trong nhiều trường hợp — tức nhanh gấp đôi và tốn nửa bộ nhớ, một lợi ích thật cho hệ nhúng (D/20).

Cái giá: phân tích QR đắt hơn Cholesky khoảng hai lần cho cùng bài toán, và cài đặt phức tạp hơn, đặc biệt khi kết hợp với phần bù Schur. Đây là loại đánh đổi mà một hệ sản phẩm nên cân nhắc nghiêm túc, vì tính ổn định thường quan trọng hơn tốc độ.

B. Certifiable: bảo đảm nghiệm toàn cục

Gauss-Newton và LM chỉ tìm được cực tiểu *địa phương*, và chúng không bao giờ cho biết nghiệm tìm được có phải toàn cục hay không. Với pose graph có loop closure, hàm mục tiêu *rất* không lồi và cực tiểu địa phương xấu tồn tại thật — một bản đồ “gập” vào chính nó là một cực tiểu địa phương hoàn toàn hợp lệ về mặt toán học.

SE-Sync [64] khai thác một cấu trúc đặc biệt: bài toán pose graph có thể viết lại thành một bài toán tối ưu trên đa tạp các ma trận bán xác định, và trong điều kiện nhiễu vừa phải, một nơi lồi của nó có nghiệm *trùng* với nghiệm của bài toán gốc. Khi điều kiện thoả, thuật toán trả về nghiệm kèm một *chứng chỉ* rằng đó là cực tiểu toàn cục.

Điều kiện “nhiều vừa phải” là chỗ đáng chú ý và cũng là giới hạn: với nhiễu lớn hoặc với outlier, bảo đảm mất hiệu lực. Nên certifiable không thay thế được back-end bền ở B/11 — hai thứ giải hai bài toán khác nhau, một cái lo cực tiểu địa phương và một cái lo dữ liệu sai.

Một hướng liên quan và thực dụng hơn: **rotation averaging** [65]. Tách bài toán thành hai bước — giải cho mọi phép xoay trước (bài toán này dễ hơn nhiều và có nghiệm gần toàn cục), rồi giải cho tịnh tiến với xoay đã cố định (bài toán này gần tuyến tính). Đây là nền của SfM toàn cục, đối lập với SfM tuần tự của COLMAP, và nó cho thấy rằng việc *chọn thứ tự giải* có thể quan trọng ngang việc chọn bộ giải.

CẠM BẦY

Đừng chọn kiến trúc back-end theo độ chính xác công bố. Ba lý do.

Thứ nhất, các con số không so sánh được: chúng chạy trên các bộ dữ liệu khác nhau, với front-end khác nhau, và front-end thường ảnh hưởng tới ATE nhiều hơn back-end.

Thứ hai, và quan trọng hơn, *độ chính xác trung bình không phải tiêu chí quyết định*. Bốn tiêu chí sau thường quyết định hơn trong sản phẩm: (a) **chi phí xấu nhất** — một hệ nhanh trung bình nhưng thỉnh thoảng mất một giây là không dùng được trong vòng điều khiển;

(b) **có đóng vòng được không** — quyết định hệ có trôi vô hạn hay không, và đó là khác biệt giữa dùng được và không dùng được trong không gian lặp lại; (c) **tính nhất quán** — hiệp phương sai có đáng tin không ([A/06](#)); (d) **hành vi khi hỏng** — nó phân kỳ ồn ào hay trôi im lặng?

Thứ ba, quyết định này khó đảo ngược nhất trong cả hệ thống. Đổi detector mất một tuần; đổi kiến trúc back-end là viết lại gần hết. Nên nó xứng đáng được quyết định dựa trên yêu cầu sản phẩm, không dựa trên bảng xếp hạng.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenVINS [42]	MSCKF với FEJ, hiệu chuẩn trực tuyến, mô phỏng có chân trị	Cài đặt bộ lọc sạch nhất; đọc để thấy cái giá của “tuyến tính hoá một lần”
VINS-Fusion [13]	Cửa sổ trượt + marginalization + pose graph riêng	Kiến trúc hai tầng điển hình: odometry nhanh + nhất quán toàn cục chậm
GTSAM [31]	iSAM2, Bayes tree, khử biến gia tăng	Cài đặt tham chiếu của smoothing gia tăng; đọc để hiểu vì sao loop closure đắt
Basalt [23]	Cửa sổ trượt dạng căn bậc hai; chất lượng kỹ thuật rất cao	Mục 5A trong một hệ thật; cũng là ví dụ tốt về kỹ thuật phần mềm
SE-Sync [64]	Pose graph certifiable với chứng chỉ toàn cục	Mục 5B; đọc để biết ranh giới của điều kiện “nhiều vừa phải”
ORB-SLAM3 [10]	Local BA + pose graph + BA toàn cục, ba luồng	Cho thấy các paradigm <i>kết hợp</i> chứ không loại trừ nhau

MINI PROJECT SLAM · Cài cùng một bài toán bằng ba kiến trúc, rồi đo cái mà bảng xếp hạng không đo · 4 ngày

Mục tiêu	tự thấy rằng ba kiến trúc cho cùng nghiệm khi mọi thứ thuận lợi, và thấy chúng phân kỳ ở <i>đâu</i> — vì chỗ phân kỳ mới là thứ quyết định lựa chọn.
Dữ liệu	tự sinh để có chân trị, cộng một chuỗi EuRoC để có dữ liệu thật.
Các bước	cài ba back-end trên cùng front-end và cùng dữ liệu: (a) EKF với toàn bộ landmark trong trạng thái, (b) cửa sổ trượt có marginalization, (c) full batch chạy lại từ đầu mỗi khung (chậm, làm chuẩn vàng); đo bốn thứ theo thời gian — ATE, thời gian mỗi bước (<i>cả trung bình lẫn phân vị 99</i>), bộ nhớ, và NEES so với chân trị; rồi bật/tắt FEJ cho (a) và (b) và đo lại NEES; cuối cùng thêm một loop closure và xem cái nào sửa được trôi.
Xong khi	bạn có bốn đường cong theo thời gian cho ba kiến trúc. Bạn phải quan sát được: (1) chi phí của (a) tăng bậc hai theo số landmark trong khi (b) không đổi; (2) NEES của (a) và (b) lệch lên khi không có FEJ và gần số chiều khi có — xác nhận A/05 mục 6; (3) chỉ (c) và (b) sửa được trôi khi có loop closure, còn (a) thì không nếu landmark đã ra khỏi trạng thái. Điểm quan trọng nhất: phân vị 99 của thời gian có thể kể một câu chuyện rất khác với trung bình.
Nối với	B/11-loop-closure-va-nhat-quan-toan-cuc xây trên pose graph ở mục 4; D/20-toi-uu-hieu-nang-va-he-thong lấy phân vị 99 làm tiêu chí thiết kế chứ không phải trung bình.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Chúng cực tiểu hoá *cùng một* hàm mục tiêu MAP. Hai lựa chọn phân biệt chúng: (1) *marginalize những gì* — EKF marginalize toàn bộ quá khứ sau mỗi bước, BA giữ tất cả; (2) *tuyến tính hoá lại bao nhiêu lần* — EKF tính Jacobian một lần tại ước lượng lúc đó và không bao giờ sửa, BA tính lại mọi Jacobian tại ước lượng tốt nhất hiện có ở mỗi vòng lặp. Về một giải thích chi phí (trạng thái không lớn lên) và giới hạn (không sửa được quá khứ). Về hai giải thích độ chính xác và tính nhất quán — Jacobian lỗi thời bị đóng băng, và các Jacobian tại các điểm khác nhau không tương thích nên tạo ra thông tin giả.
- A2. Vì EKF giữ $\Sigma = \Lambda^{-1}$ chứ không giữ Λ , và *ngịch đảo của một ma trận thưa nói chung là đặc*. Trực giác: hai landmark chưa bao giờ nhìn thấy cùng lúc vẫn tương quan, vì cả hai đều tương quan với quỹ đạo nổi chúng; Λ ghi “không nối trực tiếp”, còn Σ ghi “vẫn tương quan”. Với n landmark, Σ có $O(n^2)$ phần tử và mỗi cập nhật chạm tất cả. Có một lối thoát là information filter (giữ Λ , thưa) nhưng nó chuyển chi phí sang lúc truy vấn — và đó chính là cách các phương pháp smoothing hiện đại làm.
- A3. Vì landmark bị khử ngay sau khi dùng (structureless, B/08 mục 3C), nên *không có bản đồ nào tồn tại* — không có gì để nhận ra lại khi quay về chỗ cũ. MSCKF là odometry xuất sắc: trôi chậm, rẻ, ổn định, chi phí không phụ thuộc số landmark. Nhưng trôi của nó *đơn điệu* — nó không bao giờ tự sửa. Với sản phẩm chạy trong không gian hữu hạn và lặp lại (robot lau nhà, AGV kho), điều đó không chấp nhận được. Cách xử lý thực tế là ghép hai tầng: MSCKF hoặc cửa sổ trượt cho odometry tần số cao, cộng một pose graph cho nhất quán toàn cục.
- A4. Nó trả giá ở *loop closure*. Bayes tree cho phép chỉ tính lại phần bị ảnh hưởng, và khi cập nhật cục bộ theo thời gian — thêm một khung — phần đó nhỏ. Nhưng một loop closure nổi tư thế mới với tư thế rất cũ, làm mất hiệu lực một phần lớn cây, và chi phí vọt lên. Cái giá nghiêm trọng khi bản đồ lớn và loop closure thường xuyên — đúng tình huống của robot chạy lâu trong không gian lặp lại. Đó là lý do các hệ thực tế tách loop closure ra một luồng riêng chạy chậm hơn thay vì để nó chặn luồng tracking.
- A5. Tốt hơn *một cách định lượng được*: số điều kiện của R là căn bậc hai của số điều kiện của Λ , vì $\Lambda = R^T R$. Dựng Λ tường minh phá huỷ một nửa số chữ số có nghĩa. Với bài toán có $\text{cond} = 10^8$ — bình thường trong BA có điểm ở nhiều thang độ sâu — ta mất tám chữ số trong khi độ chính xác kép chỉ có mười sáu. Lợi ích lớn nhất đúng ở chỗ bài toán gần suy biến, tức các cấu hình ở A/05. Nó còn cho phép dùng độ chính xác đơn trong nhiều trường hợp [37], tức nhanh gấp đôi và tốn nửa bộ nhớ. Cái giá: QR đắt hơn Cholesky khoảng hai lần và cài đặt phức tạp hơn.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đang đi bộ đường dài và ghi lại mình đã đi qua đâu. Có bốn cách ghi, và chúng khác nhau đúng ở một điểm: bạn giữ lại bao nhiêu, và bạn có cho phép mình sửa lại những gì đã ghi hay không.

Cách một: chỉ giữ một mẫu giấy ghi “tôi đang ở đây”. Mỗi lần biết thêm điều gì, bạn sửa mẫu giấy rồi quên thông tin cũ đi. Rất nhẹ — mẫu giấy không dày lên dù bạn đi bao lâu. Nhưng chiều nay nếu bạn nhận ra rằng buổi sáng mình đã hiểu nhầm một ngã rẽ, bạn không sửa lại được, vì trên giấy không còn buổi sáng nữa.

Cách hai: giữ toàn bộ nhật ký từ đầu. Mỗi lần biết thêm điều gì, bạn đọc lại tất cả và nghĩ lại từ đầu. Kết quả luôn tốt nhất có thể. Nhưng đến ngày thứ mười, đọc lại hết mất cả buổi, và bạn không còn đi được nữa.

Cách ba: giữ nhật ký ba ngày gần nhất, còn cũ hơn thì tóm tắt thành một đoạn. Nhẹ, và vẫn sửa được những gì trong ba ngày. Cái bẫy nằm ở đoạn tóm tắt: một khi viết xong thì không mở lại được, và nếu lúc viết bạn đang hiểu nhầm thì cái hiểu nhầm đó đóng băng luôn ở đó.

Cách bốn: giữ hết, nhưng sắp xếp cực kỳ khéo — đánh dấu, phân mục — sao cho khi biết thêm điều gì, bạn chỉ phải đọc lại đúng những trang liên quan. Thường thì chỉ vài trang. Nhưng có một loại thông tin đặc biệt đắt: khi bạn nhận ra *chỗ mình đang đứng hôm nay chính là chỗ đã đi qua tuần trước*. Lúc đó mọi thứ ở giữa đều bị ảnh hưởng, và bạn phải đọc lại cả tuần.

Bốn cách, một việc. Và điều đáng nhớ nhất là: không có cách nào tốt hơn tuyệt đối. Nếu bạn chỉ cần biết mình đang ở đâu ngay bây giờ, cách một là đủ và rẻ nhất. Nếu bạn cần một tấm bản đồ đúng của cả chuyến đi, bạn buộc phải giữ lại quá khứ, và buộc phải trả giá cho nó.

Chỗ không rút gọn được: chỗ không rút gọn được là cái bẫy của đoạn tóm tắt ở cách ba. Ý tưởng “nén quá khứ lại nhưng giữ nguyên điều đã học được” nghe đơn giản, nhưng khi nén, các trang còn lại trở nên dính vào nhau theo những cách khó theo dõi, và điều đã học được thì ghi bằng một hệ quy chiếu đã lỗi thời. Đó là lý do các hệ cửa sổ trượt phức tạp hơn nhiều so với vẻ ngoài, và là lý do FEJ tồn tại.

Một câu: Bộ lọc là bài toán tối ưu đã quên hết quá khứ và chỉ được nghĩ một lần — mọi ưu và nhược của nó đều là hệ quả của câu đó.

I 9.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *bộ lọc là MAP đã marginalize toàn bộ quá khứ và tuyến tính hoá một lần* — là một tương đương toán học dưới giả thiết Gauss và mô hình đúng (cửa a, đối chiếu [24] và [29], cửa c). Nó bị bác bỏ nếu ai đó chỉ ra một bộ lọc mà hành vi của nó *không* tái tạo được bằng một bài toán MAP với marginalization tương ứng. Các bộ lọc phi Gauss — particle filter — nằm ngoài phạm vi này, và đó là một giới hạn thật của khung ở đây chứ không phải một ngoại lệ nhỏ.

Mệnh đề thứ hai — *EKF-SLAM có chi phí bậc hai* — là kết quả đếm phép toán, kiểm được bằng mini project (cửa b). Nó bị bác bỏ nếu một cài đặt khai thác cấu trúc nào đó trong Σ để đạt chi phí thấp hơn — và thực tế có những công trình làm điều này bằng cách xấp xỉ Σ bằng một ma trận thưa, với cái giá là mất tính chính xác của hiệp phương sai.

Mệnh đề thứ ba — *dạng căn bậc hai giảm số điều kiện xuống căn bậc hai* — là một đồng nhất thức đại số. Điều *có thể* sai là kết luận thực dụng rằng điều đó đáng kể: với bài toán có số điều kiện nhỏ, lợi ích không đo được. Mini project ở trên không kiểm điều này; đó là một thí nghiệm riêng đáng làm, và nó nằm trong 99-chua-biet.md.

Một mệnh đề mềm hơn: chương này khẳng định *bốn tiêu chí ở khối Cam bẫy quyết định hơn độ chính xác trung bình trong sản phẩm*. Đó là một phán đoán về ưu tiên kỹ thuật, không phải một sự kiện đo được, và nó có thể sai trong ứng dụng mà độ chính xác là yêu cầu chính — chẳng hạn quét đo đạc, nơi độ trễ không quan trọng và chỉ độ chính xác cuối cùng mới tính. Nêu rõ điều này vì cây này viết với mục tiêu sản phẩm robot, và ưu tiên ấy không phổ quát (*tôi suy ra từ mục tiêu đã nêu ở 00-map, chưa đối chiếu nguồn*).

I 9.7 Kết nối

Nối lên trên. [A/04-uoc-luong-map-va-bundle-adjustment](#) là khung mà toàn bộ chương này nằm trong; đặc biệt mục 5 ở đó về marginalization là công cụ chính của mục 2 ở đây. [A/05-observability-gauge-va-suy-bien](#) mục 6 giải thích bệnh bất nhất mà FEJ chữa — không có chương đó thì FEJ trông như một thủ thuật kỳ quặc. [A/06-bat-dinh-va-lan-truyen-sai-so](#) giải thích vì sao Σ đặc còn Λ thưa, tức gốc của chi phí bậc hai ở mục 2B. [B/08-mo-hinh-phan-du](#) mục 3C cho structureless, kỹ thuật làm MSCKF khả thi.

Nối xuống dưới. [B/10-khoi-tao](#) là chỗ mọi kiến trúc đều cần một khởi đầu, và yêu cầu về chất lượng khởi đầu khác nhau giữa chúng — bộ lọc nhạy hơn vì nó tuyến tính hoá một lần. [B/11-loop-closure-va-nhat-quan-toan-cuc](#) xây trực tiếp trên pose graph ở mục 4, và nó là chương giải thích vì sao khả năng đóng vòng — thứ mà MSCKF không có — lại quyết định tính dùng được của một hệ trong không gian lặp lại. [C/13-quan-ly-ban-do](#) là chỗ kiến trúc nhiều luồng trở thành một vấn đề kỹ thuật thật: tracking, local mapping và loop closing chạy ở ba tần số khác nhau và phải chia nhau một bản đồ.

Nói xa hơn. [C/15-hop-nhat-da-cam-bien](#) cho thấy lựa chọn kiến trúc ở đây quyết định việc thêm cảm biến dễ hay khó: với đồ thị nhân tử thì thêm một loại nhân tử là đủ, còn với bộ lọc thì phải sửa cả vector trạng thái và các ma trận. [D/20-toi-uu-hieu-nang-va-he-thong](#) lấy phân vị thời gian — không phải trung bình — làm tiêu chí, và giải thích vì sao một hệ nhanh trung bình nhưng thỉnh thoảng chậm là không dùng được trong vòng điều khiển.

Nói ngang: ../IMU_Fusion/ chương 8 và 9 bàn cùng chủ đề với trọng tâm quán tính, và nói kỹ hơn về ESKF, InEKF và cấu trúc nhân tử tiền tích phân. Chỗ nào cần chi tiết về bộ lọc bất biến thì đọc ở đó; chương này cố ý dừng ở mức nêu nó tồn tại và nó chữa cái gì.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu hai trục phân biệt mọi kiến trúc back-end, và đặt EKF, MSCKF, VINS-Mono và iSAM2 lên hai trục đó.
2. Vì sao Λ thừa mà Σ đặc? Trả lời bằng một câu về ý nghĩa vật lý, không bằng đại số.
3. MSCKF rẻ và chính xác nhưng không phải SLAM. Vì sao, và hậu quả gì cho một robot chạy tám tiếng trong một nhà kho?
4. iSAM2 giữ toàn bộ lịch sử mà vẫn chạy thời gian thực. Loại cập nhật nào làm nó đắt, và vì sao?
5. Tự dẫn lại vì sao $\text{cond}(\Lambda) = \text{cond}(R)^2$, rồi nói điều đó nghĩa là mất bao nhiêu chữ số với bài toán có $\text{cond}(\Lambda) = 10^{10}$.
6. Pose graph nén thông tin một cách không phục hồi được. Cụ thể cái gì bị mất, và nó liên hệ thế nào với bệnh quá tự tin ở [A/06](#)?
7. Nêu bốn tiêu chí quyết định lựa chọn back-end trong sản phẩm, và nói vì sao độ chính xác trung bình không nằm trong bốn cái đó.

Khởi tạo (Initialization)

PHỤ THUỘC A/01-hình-hoc-da-thi (E so với H , bốn cấu hình suy biến — chương này là ứng dụng trực tiếp nhất của mục 6 ở đó); A/05-observability-gauge-va-suy-bien (điều kiện kích thích, và nguyên tắc từ chối khi chưa đủ); B/07-dac-trung-va-data-association (tương ứng đầu vào); B/09-paradigm-uoc-luong-back-end (mỗi kiến trúc có yêu cầu khởi tạo khác nhau).

TÓM TẮT

Mọi bộ tối ưu phi tuyến cần một điểm khởi đầu đủ gần, và SLAM bắt đầu từ chỗ không biết gì. Chương này về khoảnh khắc chuyển từ “không biết gì” sang “có một ước lượng đủ tốt để Gauss-Newton hội tụ”. Nội dung có ba phần. Một: khởi tạo mono hai khung — bài toán khó nhất — với kỹ thuật model selection chạy song song E và H rồi để dữ liệu quyết định; đây là chỗ bốn cấu hình suy biến ở A/01 chuyển từ lý thuyết thành mã. Hai: các trường hợp dễ hơn — stereo, RGB-D, thị giác-quán tính, prior học được — và điều đáng chú ý là mỗi trường hợp dễ hơn vì cùng một lý do: có thêm một nguồn thông tin phá vỡ một gauge. Ba: phần quan trọng nhất và bị bỏ qua nhiều nhất, là tiêu chí từ chối và cơ chế phát hiện khởi tạo hỏng. Nếu chỉ nhớ một điều: một bộ khởi tạo tốt không phải bộ luôn trả về kết quả mà là bộ biết từ chối khi dữ liệu chưa đủ — vì một khởi tạo sai không tự sửa mà nhiễm độc toàn bộ bản đồ về sau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **khởi tạo (initialization)** — quá trình tạo ra ước lượng đầu tiên của tư thế và cấu trúc từ chỗ không có tiên nghiệm nào. Khác với tracking ở chỗ tracking luôn có một dự đoán từ khung trước.
- **model selection** — chạy song song nhiều mô hình hình học (E và H), tính điểm số cho mỗi mô hình dựa trên phần dư của inlier, rồi chọn mô hình thắng. Cần thiết vì hai mô hình suy biến ở những chỗ bù nhau.
- **parallax** — góc giữa hai tia nhìn tới cùng một điểm từ hai vị trí camera. Đây là đại lượng quyết định độ sâu có xác định được không, và là tiêu chí đúng để đánh giá một cặp khung khởi tạo.
- **bootstrapping** — giai đoạn trước khi khởi tạo thành công, trong đó hệ thống thu thập khung và theo dõi đặc trưng nhưng chưa có bản đồ 3D nào.
- **khởi tạo thị giác-quán tính** — quá trình giải cho hướng trọng lực, tỉ lệ metric, vận tốc ban đầu và độ lệch con quay bằng cách căn chỉnh quỹ đạo thị giác (không có tỉ lệ) với tiền tích phân IMU. Chi tiết ở ../IMU_Fusion/ chương 10.
- **khởi tạo lại (re-initialization)** — quá trình bắt đầu lại sau khi hệ thống mất bám hoàn toàn. Khác với relocalization ở chỗ relocalization tìm lại vị trí trong bản đồ cũ, còn khởi tạo lại tạo một bản đồ mới.

CÂU HỎI MỞ ĐẦU

- Q1. Bộ khởi tạo của bạn thành công 100% số lần được gọi — nó chưa bao giờ từ chối. Vì sao đó là một tin xấu chứ không phải tin tốt?
- Q2. ORB-SLAM chạy song song hai mô hình E và H rồi chọn. Vì sao không đơn giản là luôn dùng thuật toán năm điểm, vốn không suy biến với cảnh phẳng?

- Q3.** Khởi tạo stereo gần như tầm thường: có độ sâu ngay từ khung đầu. Nêu chính xác điều gì stereo cung cấp mà mono không có, và nói điều đó liên hệ thế nào với [A/05](#).
- Q4.** Một khởi tạo sai với tỉ lệ lệch 20% nhưng hình học tự nhất quán. Bundle adjustment chạy sau đó sẽ sửa được không, và vì sao?
- Q5.** Hệ thống của bạn khởi tạo tốt trong phòng thí nghiệm và thất bại ở hành lang khách hàng. Nêu ba nguyên nhân hình học khác nhau, và nêu đại lượng phân biệt chúng.

10.1 Đặt vấn đề

[A/04](#) dựng một bộ giải mạnh và nêu một điều kiện mà nó không tự thoả được: Gauss-Newton cần một điểm khởi đầu đủ gần nghiệm. Với bài toán không lồi như BA, “đủ gần” không phải một hình thức — xa quá thì bộ giải hội tụ về một cực tiểu địa phương hoàn toàn sai, và nó làm điều đó một cách hoàn toàn tự tin.

Bài toán gốc: hệ thống vừa được bật lên. Nó có một chuỗi ảnh và không có gì khác — không tư thế, không bản đồ, không biết mình đang ở đâu hay cảnh trông thế nào. Từ chỗ đó, phải tạo ra một ước lượng đủ tốt.

Điều làm chương này quan trọng hơn về ngoài là *hậu quả của việc làm sai không tự giới hạn*. Một khung tracking sai sẽ được sửa ở khung sau. Một khởi tạo sai thì không: bản đồ ban đầu trở thành hệ quy chiếu cho mọi thứ sau đó, và một bản đồ khởi tạo méo sẽ làm mọi landmark thêm vào sau bị méo theo cho khớp với nó. Hệ thống hội tụ, phần dư nhỏ, và toàn bộ bản đồ sai. Đây là câu trả lời cho câu hỏi mở đầu thứ tư: BA *không* sửa được, vì BA tìm cầu hình nhất quán nhất với dữ liệu, và một bản đồ méo với tỉ lệ sai là nhất quán — đó chính là gauge tỉ lệ ở [A/05](#) mục 3.

Ai gặp bài toán này? Mọi hệ SLAM, và nó gay gắt nhất với mono. Với stereo hay RGB-D thì gần như tầm thường, và *vì sao* nó tầm thường là một bài học đáng rút ra chứ không phải một may mắn.

Trước khi có model selection, cách làm phổ biến là chọn sẵn một mô hình — thường là E — và chờ tới khi nó cho kết quả trông hợp lý. Cách đó hỏng trong nhà, nơi cảnh phẳng là quy luật chứ không phải ngoại lệ: camera nhìn vào tường, sàn, trần, cửa. Với cảnh phẳng, E không xác định duy nhất ([A/01](#) mục 6B), và nghiệm trả về là một tổ hợp tùy tiện do nhiễu quyết định — nhưng số inlier vẫn cao, nên “trông hợp lý”.

Đó là lý do sâu xa để chạy hai mô hình. Và có một lợi ích thứ hai ít được nhắc nhưng quan trọng hơn cho sản phẩm: khi H thắng, hệ thống *biết* nó đang nhìn một mặt phẳng hoặc đang xoay thuần, và nó có thể đổi *hành vi* — chờ thêm, báo cho người dùng di chuyển — chứ không chỉ đổi công thức.

TRỰC GIÁC

Hình dung bạn tỉnh dậy trong một căn phòng lạ và phải vẽ bản đồ của nó chỉ bằng cách nhìn.

Nhìn một lần không cho biết gì về chiều sâu — mọi thứ có thể là một bức tranh tường phẳng. Bạn phải *đi vài bước* rồi nhìn lại, và so sánh: thứ gì dịch nhiều thì gần, thứ gì gần như đứng im thì xa.

Ba chuyện có thể làm hỏng việc này, và biết ba chuyện đó là biết cả chương.

Bạn xoay đầu thay vì bước đi. Cảnh đổi hoàn toàn, nhưng không có gì dịch tương đối với nhau, nên bạn không học được gì về chiều sâu. Tệ hơn, bạn có cảm giác đã quan sát rất nhiều.

Bạn đang nhìn vào một bức tường phẳng. Bạn bước đi, mọi thứ dịch — nhưng chúng dịch giống hệt nhau, vì chúng ở cùng khoảng cách. Bạn không đọc được gì từ sự khác biệt, vì không có khác biệt nào.

Bạn đi thẳng về phía thứ mình đang nhìn. Nó to dần lên nhưng không lệch sang bên. Bạn có một chút thông tin, rất ít, và ước lượng của bạn chậm chạp.

Trong cả ba trường hợp, một người khôn ngoan sẽ nói “khoan đã, tôi chưa đủ dữ liệu” rồi bước sang bên vài bước. Một cái máy được lập trình kém sẽ vẽ ra một bản đồ, vì người ta bảo nó phải vẽ ra một bản đồ.

Toàn bộ chương này là về việc dạy cái máy nói câu “khoan đã”.

10.2 Khởi tạo mono: bài toán khó nhất

A. Quy trình

Bốn bước, và bước thứ tư mới là bước phân biệt một bộ khởi tạo tốt với một bộ tồi.

Bước một — thu thập và theo dõi. Chọn một khung tham chiếu, theo dõi đặc trưng qua các khung sau ([B/07](#)). Tích lũy cho tới khi có đủ tương ứng và đủ parallax.

Bước hai — chạy song song hai mô hình. Với cùng tập tương ứng, chạy RANSAC cho E (hoặc F) và cho H . Mỗi mô hình cho một tập inlier và một tổng phần dư.

Bước ba — chọn mô hình. Tính một điểm số cho mỗi mô hình phản ánh cả số inlier lẫn độ khớp, rồi chọn. ORB-SLAM [5] dùng một tỉ số điểm số; tôi *không* chép con số ngưỡng cụ thể của họ vào đây, vì ngưỡng ấy phụ thuộc vào thang phần dư, vào ngưỡng RANSAC, và vào loại đặc trưng — chép nó sang một hệ khác mà không hiểu là cách nhanh nhất để có một hằng số kỳ bí. Nguyên tắc thì chép được: nếu H khớp tốt hơn đáng kể thì cảnh phẳng hoặc xoay thuận, và trong cả hai trường hợp bạn chưa nên khởi tạo bằng E .

Bước bốn — kiểm tra và quyết định. Phân rã mô hình thẳng, triangulate, rồi kiểm kết quả trước khi chấp nhận. Đây là mục 4.

B. Vì sao không chỉ dùng thuật toán năm điểm

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và câu trả lời có ba tầng.

Tầng một: thuật toán năm điểm không suy biến với cảnh phẳng *theo nghĩa nó vẫn trả về một E xác định*. Nhưng nó không giải quyết được vấn đề thật — với cảnh phẳng, *bài toán* có nhiều nghiệm, không phải *thuật toán* có nhiều nghiệm. Thuật toán năm điểm chọn một nghiệm trong họ đó dựa trên năm điểm được lấy mẫu, và nghiệm ấy nhảy với nhiều theo cách không kiểm soát được.

Tầng hai, và đây là tầng quan trọng: với cảnh *gần* phẳng — một bức tường có vài vật nhô ra — bài toán không suy biến hoàn toàn nhưng có số điều kiện rất lớn. Đây là suy biến số học ở [A/05](#) mục 2C, và đúng kiểu khó chẩn đoán nhất. Homography cho một cách *đo* mức độ phẳng: nếu H khớp gần như hoàn hảo thì cảnh gần phẳng, và ta biết mình đang ở vùng nguy hiểm.

Tầng ba, thực dụng nhất: khi H thẳng, ta *vẫn khởi tạo được* bằng cách phân rã H — nó cho $(R, \mathbf{t}/d, \mathbf{n})$, tức tư thế tương đối và pháp tuyến mặt phẳng, với tỉ lệ tương đối so với khoảng cách tới mặt phẳng. Đó là một khởi tạo hợp lệ cho cảnh phẳng, và nó tốt hơn nhiều so với việc ép E hoạt động. Nói cách khác, model selection không chỉ là một bộ phát hiện suy biến mà còn là một bộ chọn *thuật toán phù hợp*.

C. Tiêu chí chọn cặp khung

Đây là chỗ [A/01](#) mục 6 trả công trực tiếp nhất.

Cần bao nhiêu parallax? Không có ngưỡng phổ quát, vì nó phụ thuộc vào σ của front-end ([B/07](#) mục 4B) và vào độ sâu cảnh. Nhưng có một *cách tính* ngưỡng thay vì đoán nó, và cách ấy đáng biết.

Từ [A/06](#), bất định độ sâu của một điểm triangulate với góc parallax α nhỏ tỉ lệ xấp xỉ $\sigma_z/z \approx \sigma_\theta/\alpha$, với $\sigma_\theta \approx \sigma_{px}/f$ là bất định góc của một quan sát. Đặt yêu cầu $\sigma_z/z \leq \epsilon$ — chẳng hạn $\epsilon = 0,1$ cho 10% sai số độ sâu tương đối — ta được

$$\alpha \geq \frac{\sigma_{px}}{f \epsilon}.$$

Với $\sigma_{px} = 0,2$, $f = 400$, $\epsilon = 0,1$: $\alpha \geq 0,005 \text{ rad} \approx 0,29^\circ$. (Tôi tự dẫn quan hệ này từ hình học tam giác nhỏ; chưa đối chiếu một nguồn cụ thể, và nó là xấp xỉ bậc nhất.)

Điều đáng nói không phải con số mà là *phương pháp*: ngưỡng parallax nên được *tính ra* từ σ đã đo và từ yêu cầu độ chính xác của ứng dụng, chứ không nên là một hằng số chép từ mã nguồn của người khác. Đây là một ví dụ nhỏ của nguyên tắc lớn hơn ở [D/21](#): mỗi hằng số trong hệ thống nên truy được về một phép đo hoặc một yêu cầu.

Hai tiêu chí nữa. *Số lượng tương ứng*: cần đủ để RANSAC có ý nghĩa thống kê, và đủ để bản đồ ban đầu không quá nghèo. *Phân bố*: các điểm khởi tạo nên trải đều khung hình, vì lý do ở [B/07](#) mục 4A — điều kiện số của bài toán khởi tạo cũng chịu ảnh hưởng như mọi bài toán khác.

10.3 Các trường hợp dễ hơn, và vì sao chúng dễ hơn

Điều đáng chú ý về mục này: mọi trường hợp dễ hơn đều dễ hơn vì *cùng một lý do*, và nhận ra điều đó là hiểu được cả chương.

A. Stereo và RGB-D

Với stereo đã hiệu chỉnh, khung đầu tiên đã cho một đám mây điểm có đơn vị mét: khớp dọc hàng, tính disparity, lấy $z = bf/d$. Không cần chuyển động, không cần parallax theo thời gian, không cần model selection. Hệ thống có bản đồ ngay ở khung một.

Vì sao dễ như vậy? Đây là câu trả lời cho câu hỏi mở đầu thứ ba. Theo [A/05](#) mục 3B, đường cơ sở stereo là một *độ dài đã biết bằng mét*, nên nó phá vỡ gauge tỉ lệ ngay lập tức. Bảy bậc tự do của mono rút xuống sáu, và bậc bị mất chính là bậc khó nhất — bậc mà mono cần chuyển động để lấy. Nói khác đi: stereo mua thông tin bằng *phân cứng* thay vì bằng *chuyển động*.

Đây là một trong những lập luận mạnh nhất cho stereo trong sản phẩm, và nó lớn hơn lợi ích “có độ sâu trực tiếp” thường được nêu. Cả một lớp chế độ hồng — toàn bộ mục 2 của chương này, cộng bốn cấu hình suy biến ở [A/01](#) — biến mất. Với robot phải khởi động ở một vị trí bất kỳ do khách hàng đặt, trong một môi trường mà ta không kiểm soát được, đó là khác biệt giữa dùng được và không.

Cái giá, đã nêu ở [A/02](#) mục 5C: đường cơ sở phải giữ ổn định, và lợi ích tắt dần theo bình phương khoảng cách. Ngoài tầm hữu ích của stereo, ta trở lại chính bài toán mono — nên hệ stereo vẫn cần logic của mục 2, chỉ là ít khi phải dùng tới.

B. Thị giác-quán tính

Khởi tạo VI giải đồng thời cho hướng trọng lực, tỉ lệ metric, vận tốc ban đầu và độ lệch con quay, bằng cách căn chỉnh một quỹ đạo thị giác không có tỉ lệ với tiền tích phân IMU. VINS-Mono làm theo cách tuyến tính từng bước [13]; ORB-SLAM3 làm bằng một bài toán MAP thuần quán tính [10]. Chi tiết đầy đủ nằm ở `../IMU_Fusion/` chương 10 và không lặp lại ở đây.

Điều cần nhớ cho chương này là điều kiện: *gia tốc phải thay đổi*. Nếu gia tốc thật là hằng — đứng yên, hoặc chuyển động thẳng đều — thì thành phần gia tốc nhập hoàn toàn vào vectơ trọng lực chưa biết, và tỉ lệ không quan sát được dù có IMU. Đây là cùng nguyên lý ở [A/05](#): cảm biến cho thông tin chỉ khi chuyển động kích thích nó.

Hệ quả thực hành đáng nhớ: một hệ VI khởi tạo trên một drone đang đậu trên bàn sẽ thất bại, và nó thất bại một cách *im lặng* nếu không có kiểm tra kích thích. Đây là ví dụ mở đầu của `../IMU_Fusion/` chương 5, và nó là lý do mọi bộ khởi tạo VI nghiêm túc đều có một cổng kiểm tra phương sai gia tốc.

C. Prior độ sâu học được

Một mô hình độ sâu metric đơn ảnh cho một ước lượng độ sâu ngay từ khung đầu, nên về nguyên tắc nó chơi vai trò của stereo. Điều này hữu ích và ngày càng được dùng.

Nhưng theo [B/08](#) mục 4C, phải cẩn thận về cách đưa vào. Dùng đúng: *khởi tạo* tỉ lệ và độ sâu từ mô hình, rồi để hình học tiếp quản khi đã có đủ parallax. Dùng sai: giữ prior với trọng số cao mãi mãi, lúc đó một sai lệch có hệ thống của mô hình — và mọi mô hình đều có, phụ thuộc vào loại cảnh — sẽ chảy vào toàn bộ bản đồ.

Có một lợi ích thứ hai ít được nhắc và có thể quan trọng hơn: mô hình học được cho một ước lượng *ngay lập tức*, nên hệ thống có thể bắt đầu tracking trong khi chờ đủ parallax. Điều này biến giai đoạn bootstrapping từ “mù hoàn toàn” thành “ước lượng thô”, và với trải nghiệm người dùng thì đó là khác biệt lớn.

Bảng 10.1. Bốn cấu hình và cái mà mỗi cấu hình cung cấp. Cột thứ hai là cột giải thích tất cả: mỗi trường hợp dễ hơn vì có một nguồn phá vỡ gauge tỉ lệ.

Cấu hình	Nguồn phá gauge tỉ lệ	Điều kiện	Chế độ hỏng còn lại
Mono	Không có — tỉ lệ là gauge tự do	Cần parallax; cần cảnh không phẳng	Cả bốn cấu hình ở A/01 mục 6
Stereo	Đường cơ sở đã biết bằng mét	Không có — khung đầu là đủ	Ngoài tầm hữu ích thì thoái hoá về mono
RGB-D	Cảm biến độ sâu	Trong tầm và bề mặt hợp tác	Kính, gương, bề mặt đen, ngoài trời
Thị giác-quán tính	Gia tốc kế (qua tích phân)	Gia tốc phải thay đổi	Đứng yên hoặc thẳng đều thì thất bại im lặng
Prior học được	Tiên nghiệm về kích thước vật thể	Cảnh nằm trong phân bố huấn luyện	Sai lệch có hệ thống, không có bất định đã hiệu chuẩn

10.4 Tiêu chí từ chối: phần quan trọng nhất

A. Vì sao từ chối quan trọng hơn thành công

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là luận điểm trung tâm của chương.

Một bộ khởi tạo có hai loại lỗi. *Từ chối khi lẽ ra chấp nhận được*: hệ thống chờ thêm vài khung. Chi phí là một chút độ trễ, và người dùng thấy hệ thống “hơi chậm khởi động”. *Chấp nhận khi lẽ ra phải từ chối*: hệ thống tạo ra một bản đồ sai, và mọi thứ sau đó xây trên bản đồ ấy. Chi phí là một quỹ đạo sai không tự sửa, và thường là mất bám hoàn toàn sau vài chục khung — nhưng đôi khi tệ hơn, là một quỹ đạo trông hợp lý mà sai tỉ lệ.

Hai chi phí ấy chênh nhau nhiều bậc độ lớn. Nên ngưỡng chấp nhận nên đặt *bảo thủ*, và một bộ khởi tạo từ chối thường xuyên là dấu hiệu của thiết kế đúng chứ không phải thiết kế kém. Một hệ chưa bao giờ từ chối thì hoặc nó chỉ chạy trong điều kiện lý tưởng, hoặc nó đang chấp nhận những thứ lẽ ra không nên — và bạn không biết là cái nào.

Nguyên tắc này lặp lại ở B/12 với relocalization, nơi chênh lệch còn lớn hơn, và ở B/11 với loop closure, nơi nó lớn nhất.

B. Bốn phép kiểm

Kiểm một — *parallax trung vị*. Không phải trung bình: trung vị bền hơn với vài điểm ở rất gần. Ngưỡng tính theo mục 2C từ σ đã đo. Đây là phép kiểm bắt được xoay thuần và chuyển động quá nhỏ.

Kiểm hai — *tỉ số điểm số giữa H và E*. Bắt được cảnh phẳng. Khi H thắng rõ rệt, hoặc chuyển sang khởi tạo bằng phân rã H, hoặc chờ thêm.

Kiểm ba — *tỉ lệ cheirality*. Sau khi phân rã và triangulate, đếm tỉ lệ điểm có độ sâu dương ở cả hai khung. Với nghiệm đúng, tỉ lệ này phải áp đảo. Nếu hai trong bốn nghiệm cho tỉ lệ xấp xỉ nhau thì đó *không* phải lúc chọn cái nhiều phiếu hơn — đó là dấu hiệu parallax quá nhỏ (A/01 mục 3E), và câu trả lời đúng là từ chối.

Kiểm bốn — *số điều kiện hoặc phổ giá trị riêng của Λ* . Sau khi dựng bài toán BA nhỏ cho hai khung, kiểm xem có bao nhiêu hướng gần suy biến ngoài bảy bậc gauge. Đắt hơn ba phép trên nhưng là phép kiểm tổng quát nhất — nó bắt được cả những trường hợp mà ba phép kia bỏ sót. A/05 khối *Cạm bẫy* liệt kê đầy đủ.

CẠM BÃY

Số inlier không nằm trong bốn phép kiểm, và đó là có chủ ý.

Như A/01 mục 6 đã lập luận, trong cả bốn cấu hình suy biến, số inlier vẫn cao. Nó đo mức nhất quán của dữ liệu với mô hình, không đo mức xác định của mô hình. Dùng nó làm tiêu chí chấp nhận là đúng loại nhầm lẫn mà A/05 mô tả.

Một biểu hiện cụ thể và rất hay gặp: hệ thống khởi tạo “thành công” với năm trăm inlier khi camera đang xoay tại chỗ trước một bức tường có vân. Mọi điểm khớp hoàn hảo — vì chúng thật sự khớp. Bản đồ tạo ra có mọi điểm ở những độ sâu ngẫu nhiên, và nó hoàn toàn nhất quán với dữ liệu.

Số inlier *vẫn* hữu ích, nhưng như một *điều kiện cần*, không phải điều kiện đủ: quá ít inlier thì chắc chắn không khởi tạo được, nhưng nhiều inlier không nói gì về việc có nên khởi tạo hay không.

C. Phát hiện khởi tạo hỏng sau khi đã chấp nhận

Bốn phép kiểm trên chạy *trước* khi chấp nhận. Cần thêm một tầng chạy *sau*, vì không bộ kiểm nào hoàn hảo.

Ba dấu hiệu cho thấy bản đồ ban đầu có vấn đề, quan sát được trong vài chục khung đầu. *Tỉ lệ tracking sụt nhanh*: nếu bản đồ ban đầu sai, các landmark không chiếu về đúng chỗ ở khung sau và bị loại dần. *Phần dư trung bình tăng dần* thay vì ổn định. *Nhiều landmark bị loại vì cheirality* — điểm rơi ra sau camera khi camera di chuyển, dấu hiệu độ sâu ban đầu sai.

Khi phát hiện, hành động đúng là *vứt bản đồ và khởi tạo lại* chứ không phải cố cứu. Đây là một quyết định kiến trúc quan trọng: hệ thống phải có khả năng vứt bỏ trạng thái của chính nó và bắt đầu lại sạch sẽ. Nhiều hệ không có khả năng đó và phải khởi động lại cả tiến trình — chấp nhận được trong nghiên cứu, không chấp nhận được trong sản phẩm (D/21).

ORB-SLAM3 đi xa hơn với kiến trúc Atlas: khi mất bám, nó bắt đầu một bản đồ *mới* và giữ bản đồ cũ; nếu sau này phát hiện hai bản đồ chồng lấn, nó ghép chúng [10]. Đây là cách xử lý tình tế hơn nhiều so với vứt đi, và nó là chủ đề của C/13.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [10]	Model selection H so với E ; từ chối khi parallax không đủ; Atlas cho nhiều bản đồ	Cài đặt đầy đủ nhất của mục 2 và mục 4C
VINS-Mono [13]	SfM thuần thị giác rồi căn với IMU; kiểm phương sai gia tốc trước khi khởi tạo	Mục 3B — và công kiểm tra kích thích là một ví dụ tốt
DSO [22]	Khởi tạo direct coarse-to-fine với độ sâu ngẫu nhiên ban đầu	Cho thấy direct cần chiến lược khác hẳn: không có tương ứng rời rạc để dùng E hay H
SVO-Pro [57]	Khởi tạo bằng homography cho cảnh phẳng (giả định mặt đất)	Ví dụ về việc dùng tiên nghiệm ứng dụng — hợp lệ khi tiên nghiệm đúng
COLMAP [11]	Chọn cặp khung khởi tạo theo tiêu chí hình học, thử nhiều cặp	Ngoại tuyến nên có thể thử lại — một xa xỉ mà hệ thời gian thực không có

MINI PROJECT SLAM • Xây một bộ khởi tạo biết từ chối, rồi đo tỉ lệ từ chối đúng và sai • 3 ngày

Mục tiêu	biến “khởi tạo” từ một bước bạn hy vọng thành công thành một bước có tiêu chí đo được và có tỉ lệ lỗi biết trước.
Dữ liệu	một chuỗi EuRoC hoặc TUM-RGBD có chân trị, cộng <i>ba chuỗi tự quay</i> : một chuỗi xoay thuần, một chuỗi nhìn tường phẳng, một chuỗi đi thẳng về phía trước. Ba chuỗi sau là phần quan trọng nhất của bài này.
Các bước	cài khởi tạo mono hai khung với model selection; cài cả bốn phép kiểm ở mục 4B; chạy trên cả bốn chuỗi và ghi lại quyết định (chấp nhận/từ chối) kèm giá trị của bốn đại lượng kiểm; với các lần chấp nhận trên chuỗi có chân trị, đo sai số tỉ lệ và sai số tư thế tương đối; vẽ đường cong ROC — tỉ lệ chấp nhận sai so với tỉ lệ từ chối sai — khi quét ngưỡng parallax.
Xong khi	bạn có một đường cong ROC và một ngưỡng parallax chọn từ đường cong đó chứ không chép từ đâu. Bạn phải quan sát được rằng trên ba chuỗi suy biến, số inlier vẫn cao trong khi ba đại lượng kiểm kia thì không — xác nhận khối <i>Cạm bẫy</i> bằng dữ liệu của chính bạn. Và bạn có một con số: tỉ lệ chấp nhận sai của hệ thống bạn ở ngưỡng đã chọn.
Nối với	B/12-relocalization dùng cùng nguyên tắc từ chối với chênh lệch chi phí còn lớn hơn; D/21-tu-prototype-len-mass-production biến tỉ lệ chấp nhận sai thành một chỉ tiêu nghiệm thu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì một bộ khởi tạo có hai loại lỗi với chi phí chênh nhau nhiều bậc. Từ chối sai chỉ tốn một chút độ trễ; chấp nhận sai tạo ra một bản đồ sai mà mọi thứ sau đó xây trên nó, và nó *không tự sửa*. Một hệ chưa bao giờ từ chối thì hoặc chỉ chạy trong điều kiện lý tưởng, hoặc đang chấp nhận những thứ lẽ ra không nên — và bạn không biết là cái nào, tức bạn đang bay mù. Thiết kế đúng là đặt ngưỡng bảo thủ và *đo* tỉ lệ chấp nhận sai, chứ không phải tối đa hoá tỉ lệ thành công.
- A2. Ba tầng. (a) Thuật toán năm điểm vẫn trả về một E xác định với cảnh phẳng, nhưng vấn đề là *bài toán* có nhiều nghiệm chứ không phải thuật toán — nghiệm nó chọn nhảy với nhiễu theo cách không kiểm soát được. (b) Với cảnh *gần* phẳng, bài toán có số điều kiện rất lớn — suy biến số học, khó chẩn đoán nhất — và homography cho một cách *đo* mức độ phẳng. (c) Khi H thẳng, ta vẫn khởi tạo được bằng phân rã H , vốn cho $(R, t/d, n)$ và là một khởi tạo hợp lệ cho cảnh phẳng. Nên model selection không chỉ phát hiện suy biến mà còn chọn thuật toán phù hợp.
- A3. Stereo cung cấp một *độ dài đã biết bằng mét* — đường cơ sở. Theo A/05 mục 3B, nó phá vỡ gauge tỉ lệ ngay lập tức, đưa bảy bậc tự do của mono xuống sáu, và bậc bị mất chính là bậc khó nhất — bậc mà mono phải dùng chuyển động để lấy. Nói khác đi: stereo mua thông tin bằng *phần cứng* thay vì bằng *chuyển động*. Hệ quả là cả một lớp chế độ hồng biến mất. Cái giá: đường cơ sở phải ổn định, và lợi ích tắt dần theo bình phương khoảng cách — ngoài tầm hữu ích thì ta trở lại đúng bài toán mono.
- A4. Không, BA *không* sửa được, và lý do là A/05: tỉ lệ là một gauge tự do của SLAM mono. BA tìm cấu hình nhất quán nhất với dữ liệu, và một bản đồ cơ giản đều với tỉ lệ sai là hoàn toàn nhất quán — mọi phần dư chiếu đều bằng đúng như với tỉ lệ đúng. Hàm mục tiêu phẳng theo hướng đó, nên BA không có tín hiệu nào để sửa. Muốn sửa thì phải *thêm thông tin*: một cảm biến có đơn vị mét, một vật có kích thước đã biết, hoặc một loop closure $Sim(3)$ nối về một phần bản đồ có tỉ lệ đúng.
- A5. Ba nguyên nhân: (a) *xoay thuận* — robot quay để vào hành lang, phát hiện bằng parallax trung vị thấp dù số inlier cao; (b) *cảnh phẳng* — camera nhìn vào tường cuối hành lang, phát hiện bằng tỉ số điểm số H thẳng E ; (c) *đi dọc trục quang* — robot đi thẳng dọc hành lang, epipole nằm trong ảnh, phát hiện bằng phân bố parallax rất lệch theo bán kính và bằng số điều kiện lớn. Ba đại lượng phân biệt chúng là ba đại lượng kiểm khác nhau ở mục 4B, và không cái nào là số inlier. Lưu ý thêm: hành lang thường có *cả ba* cùng lúc, và đó là lý do nó là môi trường khó nhất cho SLAM mono.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn bị bịt mắt đưa vào một căn phòng lạ, rồi tháo băng ra và bảo vẽ bản đồ căn phòng. Nhìn một lần không đủ. Một bức tranh vẽ cảnh sâu hun hút và một cái hành lang thật, từ một chỗ đứng, trông y hệt nhau. Bạn phải bước sang bên vài bước rồi nhìn lại: thứ gì dịch nhiều là gần, thứ gì gần như đứng im là xa.

Ba chuyện có thể làm hỏng việc này. Bạn *quay người* thay vì bước đi — cảnh đổi hoàn toàn nhưng chẳng học được gì. Bạn đang nhìn vào một *bức tường phẳng* — bước đi thì mọi thứ dịch, nhưng dịch giống hệt nhau nên chẳng đọc được gì từ sự khác biệt. Hoặc bạn *đi thẳng* về phía đang nhìn — mọi thứ to dần chứ không lệch sang bên, nên bạn chỉ có rất ít thông tin.

Điều mà một người tỉnh táo làm trong cả ba trường hợp: nói “khoan đã”, rồi bước sang bên vài bước nữa.

Điều mà một cái máy lập trình kém làm: vẽ ra một bản đồ. Vì người ta bảo nó phải vẽ ra một bản đồ, và nó không có khái niệm “chưa đủ dữ liệu”.

Và đây là chỗ nguy hiểm: bản đồ sai ấy không tự hỏng. Mọi thứ máy nhìn thấy về sau sẽ được đặt vào cho *khớp* với bản đồ sai ban đầu, nên bản đồ vẫn luôn tự nhất quán. Nó chỉ không giống căn phòng thật.

Nên phần khó nhất của chương này không phải cách vẽ bản đồ. Đó là cách dạy cái máy nhận ra lúc nó chưa nên vẽ — và chịu đựng việc phải chờ.

Chỗ không rút gọn được: chỗ không rút gọn được là *ngưỡng*: bước sang bên bao nhiêu thì đủ? Câu trả lời phụ thuộc vào camera của bạn chính xác tới đâu, cảnh xa hay gần, và bạn cần bản đồ chính xác

tới mức nào. Mục 2C cho một cách *tính* nó thay vì đoán, và đó là cách đúng — nhưng nó đòi bạn phải biết σ của chính mình, và đó là một phép đo mà ít ai làm.

Một câu: Một bộ khởi tạo tốt là bộ biết nói “chưa đủ dữ liệu” — vì cái nó tạo ra sẽ là hệ quy chiếu cho mọi thứ về sau.

10.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *khởi tạo sai không tự sửa được* — là hệ quả của [A/05](#): tỉ lệ là gauge tự do nên BA không có tín hiệu để sửa nó. Nó bị bác bỏ nếu ai đó cho thấy một hệ mono thuần thị giác phục hồi được tỉ lệ đúng sau một khởi tạo sai tỉ lệ, mà không dùng thêm thông tin nào. Điều đó bất khả theo lý thuyết; điều *có thể* xảy ra và cần phân biệt là hệ phục hồi được sau một khởi tạo sai về *hình học* (không phải tỉ lệ) — chẳng hạn khi các landmark sai bị loại dần và thay bằng landmark mới đúng. Đó là một cơ chế thật và nó làm mệnh đề cần được phát biểu chặt hơn: *sai tỉ lệ không sửa được; sai hình học thì có thể, nếu hệ có cơ chế loại landmark tốt*.

Mệnh đề thứ hai — *số inlier không phải tiêu chí chấp nhận* — là mệnh đề kế thừa từ [A/01](#) mục 6 và kiểm được bằng mini project (cửa b). Nó bị bác bỏ nếu trên một tập đủ lớn các lần khởi tạo gồm cả cấu hình suy biến, số inlier tương quan chặt với sai số tỉ lệ thật — chặt ngang hoặc hơn parallax trung vị.

Mệnh đề thứ ba — *ngưỡng parallax tính được từ σ và yêu cầu độ chính xác* — dựa trên một xấp xỉ bậc nhất mà tôi tự dẫn ở mục 2C và *chưa đối chiếu nguồn*. Nó có thể sai ở chỗ: quan hệ $\sigma_z/z \approx \sigma_\theta/\alpha$ bỏ qua ảnh hưởng của số lượng quan sát (nhiều quan sát bù được parallax nhỏ ở mức độ nào đó) và bỏ qua hình dạng phân bố lệch ở [A/06](#). Một công thức đúng hơn nên tính từ ma trận thông tin đầy đủ. Đây là một mục trong 99-chua-biet.md.

Một mệnh đề mềm: chương này khẳng định *tỉ lệ từ chối cao là dấu hiệu thiết kế đúng*. Điều đó có thể sai trong ứng dụng mà độ trễ khởi động là yêu cầu cứng — một thiết bị AR mà người dùng không chờ quá hai giây. Ở đó, đánh đổi dịch chuyển, và cách đúng là dùng một cấu hình cảm biến khác (stereo, hoặc prior học được) chứ không phải nới ngưỡng. Nêu rõ điều này vì nguyên tắc ở mục 4A không phổ quát mà phụ thuộc vào tỉ số chi phí của hai loại lỗi.

10.6 Kết nối

Nối lên trên. [A/01-hình-hoc-da-thi](#) là chương mà chương này áp dụng trực tiếp nhất — mục 6 ở đó liệt kê bốn cấu hình suy biến, và mục 4B ở đây biến chúng thành bốn phép kiểm chạy được. [A/05-observability-gauge-va-suy-bien](#) cho nguyên tắc lớn hơn: tỉ lệ là gauge nên không sửa được, và điều kiện kích thích quyết định khi nào được phép tin. [A/06-bat-dinh-va-lan-truyen-sai-so](#) cho công cụ tính ngưỡng parallax từ σ . [B/07-dac-trung-va-data-association](#) cho σ đó.

Nối xuống dưới. [B/11-loop-closure-va-nhat-quan-toan-cuc](#) dùng cùng nguyên tắc từ chối với chênh lệch chi phí còn lớn hơn: một loop closure sai phá cả bản đồ. [B/12-relocalization](#) cũng vậy, và cả ba chương — 10, 11, 12 — có chung một cấu trúc lập luận: hai loại lỗi với chi phí bất đối xứng, nên ngưỡng phải bảo thủ và tỉ lệ lỗi phải được đo. [C/13-quan-ly-ban-do](#) nhận vấn đề ở mục 4C: hệ thống phải vứt bỏ được bản đồ của chính nó, và Atlas là cách xử lý tinh tế hơn.

Nối xa hơn. [C/15-hop-nhat-da-cam-bien](#) là chỗ Bảng 10.1 được đọc như một bảng thiết kế: chọn cảm biến nào chính là chọn phá gauge nào. [D/19-ben-vung-trong-the-gioi-thuc](#) bàn về việc khởi tạo trong điều kiện xấu — ít vân, ánh sáng kém, vật động.

[D/21-tu-prototype-len-mass-production](#) biến tỉ lệ chấp nhận sai thành một chỉ tiêu nghiệm thu, và lập luận rằng khởi động nhanh và đáng tin là một yêu cầu sản phẩm chứ không phải một tính năng.

Nổi ngang: ../IMU_Fusion/ chương 10 là chương đầy đủ về khởi tạo thị giác-quán tính, với suy dẫn các bước giải cho trọng lực, tỉ lệ và vận tốc. Chương này chỉ nêu kết quả và điều kiện.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một bộ khởi tạo chưa bao giờ từ chối là một tin xấu? Nêu tỉ số chi phí giữa hai loại lỗi.
2. Nêu ba lý do để chạy song song E và H thay vì chỉ dùng thuật toán năm điểm.
3. Tự dẫn lại ngưỡng parallax từ σ_{px} , f và yêu cầu ϵ , rồi tính cho $\sigma = 0,3$ px, $f = 600$, $\epsilon = 0,05$.
4. Điều gì stereo cung cấp mà mono không có? Trả lời bằng ngôn ngữ gauge, không bằng ngôn ngữ “có độ sâu”.
5. Khởi tạo VI cần gia tốc thay đổi. Vì sao, và chuyện gì xảy ra nếu bỏ qua điều kiện đó?
6. Nêu bốn phép kiểm trước khi chấp nhận khởi tạo, và nói mỗi phép bắt được cấu hình suy biến nào.
7. Bản đồ khởi tạo sai tỉ lệ 20%. Nêu chính xác vì sao BA không sửa được, và nêu ba cách thêm thông tin để sửa.

Loop closure và nhất quán toàn cục (Loop Closure and Global Consistency)

PHỤ THUỘC A/03-lie-group-va-toi-uu-tren-da-tap ($Sim(3)$ và log của sai lệch tư thế); A/04-uoc-luong-map-va-bundle-adjustment (M-estimator, GNC); A/05-observability-gauge-va-suy-bien (số bậc tự do của pose graph theo cấu hình cảm biến); B/07-dac-trung-va-data-association (perceptual aliasing — ở đây nó xuất hiện lại với hậu quả nghiêm trọng hơn nhiều); B/09-paradigm-uoc-luong-back-end (pose graph).

TÓM TẮT

Odometry trôi, và trôi đơn điệu — không cơ chế nào trong B/09 tự sửa được điều đó. Loop closure là cơ chế duy nhất: khi hệ thống nhận ra nó đã quay lại một chỗ từng đi qua, nó thêm một ràng buộc nổi hiện tại với quá khứ, và ràng buộc ấy phân phối lại sai số tích lũy trên toàn quỹ đạo. Đây là thứ biến *odometry* thành *SLAM*. Nội dung có bốn phần. Một: nhận ra chỗ cũ — từ túi từ trực quan tới các mô hình học được — và vì sao đây là một bài toán truy hồi chứ không phải một bài toán khớp. Hai: kiểm định hình học, tăng bắt buộc biến một ứng viên thành một ràng buộc, và vì sao ngưỡng của nó phải bảo thủ hơn mọi ngưỡng khác trong hệ thống. Ba: áp dụng ràng buộc — $Sim(3)$ cho mono, pose graph, và việc hoà nhập kết quả với luồng tracking đang chạy. Bốn, và là phần quan trọng nhất: back-end bền, tức làm sao sống sót khi một ràng buộc sai lọt qua. Nếu chỉ nhớ một điều: *đây là chương duy nhất trong cây mà một lỗi đơn lẻ phá huỷ toàn bộ sản phẩm của hàng giờ làm việc* — nên chi phí của hai loại lỗi bất đối xứng tới mức mọi thiết kế phải xoay quanh sự bất đối xứng đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **loop closure** — việc nhận ra rằng vị trí hiện tại trùng với một vị trí đã đi qua, và thêm một ràng buộc tương ứng vào bài toán ước lượng. Gồm hai bước tách rời: *phát hiện* (nhận ra chỗ cũ) và *đóng* (áp dụng ràng buộc).
- **trôi (drift)** — sai số tích lũy của odometry, tăng đơn điệu theo quãng đường đi. Không cơ chế odometry nào tự sửa được trôi, vì nó không có thông tin nào về vị trí tuyệt đối.
- **túi từ trực quan (bag of visual words)** — biểu diễn một ảnh bằng histogram tần suất của các “từ” — các cụm descriptor đã học trước. Cho phép so sánh hai ảnh bằng một vectơ thưa thay vì bằng hàng trăm descriptor.
- **TF-IDF** — trọng số cho mỗi từ: từ xuất hiện nhiều trong ảnh này (term frequency) và hiếm trong toàn bộ kho (inverse document frequency) thì mang nhiều thông tin phân biệt hơn.
- **perceptual aliasing** — hai địa điểm khác nhau trông giống nhau tới mức không phân biệt được bằng vẻ ngoài. Đây là nguyên nhân gốc của loop closure sai, và nó là quy luật chứ không phải ngoại lệ trong môi trường nhân tạo.
- **kiểm định hình học (geometric verification)** — bước kiểm một ứng viên loop closure bằng cách tìm một phép biến đổi nhất quán giữa hai khung. Đây là tầng phòng thủ chính chống lại perceptual aliasing.
- **back-end bền (robust back-end)** — họ kỹ thuật cho phép bài toán tối ưu sống sót khi một số cạnh là sai, bằng cách cho phép bộ giải “tắt” chúng thay vì buộc phải thoả mãn chúng.

CÂU HỎI MỞ ĐẦU

- Q1. Loop closure sai làm gập cả bản đồ; loop closure bỏ lỡ chỉ làm hệ thống trôi thêm một chút. Ước lượng tỉ số chi phí giữa hai loại lỗi, rồi nói nó quyết định gì về thiết kế.
- Q2. Kiểm định hình học đã cho một phép biến đổi với hai trăm inlier. Vì sao đó vẫn chưa đủ để chấp nhận, và nêu phép kiểm bổ sung.
- Q3. Hành lang kho hàng có ba mươi kệ giống hệt nhau, cách nhau ba mét. Mọi phương pháp dựa trên vẽ ngoài đều thất bại ở đây. Nêu hai nguồn thông tin khác vẫn phân biệt được, và nói vì sao chúng không phải lời giải hoàn chỉnh.
- Q4. SLAM mono cần $Sim(3)$ cho loop closure còn SLAM stereo chỉ cần $SE(3)$. Nếu bạn dùng $SE(3)$ cho mono thì chuyện gì xảy ra cụ thể — không phải “kém chính xác hơn” mà là gì?
- Q5. Back-end bền cho phép bộ giải tắt các cạnh nghi ngờ. Nghe như nó giải quyết hoàn toàn bài toán loop closure sai. Nêu chính xác điều kiện để nó hoạt động, và nói khi nào điều kiện đó bị vi phạm.

11.1 Đặt vấn đề

Mọi bộ ước lượng ở B/09 có chung một tính chất: sai số của chúng *tích lũy*. Mỗi bước thêm một chút sai số, và không có gì kéo nó về. Với một robot chạy tám tiếng trong một nhà kho, sai số cuối ngày có thể là hàng chục mét — đủ để bản đồ trở nên vô dụng.

Điều làm vấn đề này đặc biệt khó chịu: *robot thường xuyên quay lại chỗ cũ*. Nó đi qua cùng một lối đi mười lần một ngày. Ở lần thứ mười, nó “biết” rằng nó đang ở chỗ đã đi qua — theo nghĩa thông tin đó có trong dữ liệu — nhưng nếu không ai nói cho nó, nó sẽ tiếp tục vẽ một lối đi thứ mười cách lối đi thứ nhất năm mét.

Bài toán gốc: phát hiện việc quay lại, và dùng nó để sửa. Đây là chữ “M” trong SLAM — không có nó, ta chỉ có odometry.

Ai gặp bài toán này? Mọi hệ chạy lâu trong không gian hữu hạn. Với một xe tự lái đi một chiều trên đường cao tốc, loop closure ít quan trọng; với robot dịch vụ, nó là yêu cầu sống còn.

Trước khi có các phương pháp truy hồi hiệu quả, cách làm là so khung hiện tại với *mọi* khung quá khứ. Chi phí tuyến tính theo số khung mỗi lần, tức bậc hai theo thời gian — không khả thi. Tút từ trực quan giải quyết bằng cách biến mỗi ảnh thành một vectơ thưa và dùng chỉ mục ngược, đưa chi phí truy hồi xuống gần hằng số [66].

Nhưng vấn đề thật không phải chi phí. Đó là *perceptual aliasing*, và nó nghiêm trọng hơn ở đây so với ở B/07 vì hai lý do. Thứ nhất, không gian tìm kiếm lớn hơn nhiều — ta so với toàn bộ bản đồ, không chỉ với khung trước. Thứ hai, và quan trọng hơn, *hậu quả lớn hơn nhiều*. Một khớp điểm sai giữa hai khung liền kề tạo ra một phần dư lớn mà M-estimator hạ trọng số. Một loop closure sai tạo ra một ràng buộc nói “hai chỗ cách nhau năm mươi mét này thực ra là một chỗ”, và bộ giải — vốn tin mọi ràng buộc — sẽ gập bản đồ lại để thỏa mãn nó.

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và nó định hình cả chương: chi phí của một dương tính giả lớn hơn chi phí của một âm tính giả nhiều bậc độ lớn. Bỏ lỡ một loop closure nghĩa là trôi thêm một chút, và sẽ có cơ hội khác ở vòng sau. Chấp nhận một loop closure sai nghĩa là phá huỷ sản phẩm của hàng giờ làm việc, và thường không phục hồi được.

TRỰC GIÁC

Hình dung bạn đi bộ quanh một khu phố, vừa đi vừa vẽ bản đồ bằng cách đếm bước và ghi hướng rẽ.

Sau một vòng, bạn quay về điểm xuất phát — nhưng bản đồ của bạn nói bạn đang ở cách đó ba mươi mét. Sai số đếm bước đã tích lũy.

Bây giờ bạn *nhận ra* cái cửa hàng trước mặt chính là cửa hàng bạn xuất phát. Đây là thông tin cực kỳ quý: nó nói điểm đầu và điểm cuối trùng nhau. Bạn lấy bản đồ ra và *kéo* nó cho hai điểm khớp, phân phối ba mươi mét sai lệch đều ra cả vòng. Bản đồ đột nhiên đúng hơn nhiều — không chỉ ở điểm cuối mà ở *mọi* chỗ.

Đó là loop closure, và nó là phép thuật thật sự của SLAM.

Nhưng bây giờ hình dung bạn nhầm. Cái cửa hàng trước mặt thuộc cùng một chuỗi, cùng biển hiệu, cùng cách bày hàng — nhưng nó ở một khu phố khác cách đó nửa cây số. Bạn kéo bản đồ cho hai điểm khớp.

Kết quả không phải một bản đồ hơi sai. Đó là một bản đồ *gập đôi*: nửa cây số bị nén thành không, hai con phố khác nhau chồng lên nhau, và mọi thứ bạn đã ghi đúng suốt buổi sáng giờ nằm sai chỗ. Và bạn không có cách nào biết, vì bản đồ mới vẫn tự nhất quán — nó nhất quán với một giả thuyết sai.

Chênh lệch giữa hai kịch bản ấy là toàn bộ lý do chương này dài như vậy. Việc nhận ra chỗ cũ thì dễ. Việc *chắc chắn* rằng mình không nhầm mới là bài toán.

11.2 Phát hiện: nhận ra chỗ cũ

A. Vì sao đây là bài toán truy hồi

Điểm khái niệm quan trọng đầu tiên: phát hiện loop closure *không* phải bài toán khớp điểm mà là bài toán *truy hồi* (retrieval). Ta không hỏi “điểm này khớp với điểm nào” mà hỏi “khung này giống khung nào nhất trong hàng chục nghìn khung đã lưu”.

Khác biệt ấy quyết định công cụ. Khớp điểm dùng khoảng cách descriptor và ratio test; truy hồi dùng một *biểu diễn toàn cục* cho mỗi ảnh, một chỉ mục để tìm nhanh, và một thang điểm tương đồng. Đây là bài toán mà cộng đồng tìm kiếm thông tin đã giải cho văn bản, và túi từ trực quan là việc mượn nguyên giải pháp đó.

B. Túi từ trực quan

Quy trình [66], [67]. *Ngoại tuyến*: gom một tập lớn descriptor từ dữ liệu đại diện, phân cụm chúng thành một *từ điển* có cấu trúc cây, mỗi lá là một “từ”. *Trực tuyến*: với mỗi khung, gán mỗi descriptor vào từ gần nhất, thu được một histogram thưa; tính điểm tương đồng giữa hai histogram bằng khoảng cách có trọng số TF-IDF.

Hai chi tiết kỹ thuật đáng biết vì chúng là lý do phương pháp này khả thi.

Chỉ mục ngược (inverse index): với mỗi từ, lưu danh sách các khung chứa nó. Khi truy vấn, chỉ cần duyệt các khung chia sẻ ít nhất một từ với khung hiện tại — thường là một phần rất nhỏ của bản đồ. Đây là thứ đưa chi phí từ tuyến tính theo số khung xuống gần hằng số.

Chỉ mục thuận (direct index): với mỗi khung, lưu descriptor nào thuộc nhánh nào của cây. Khi đã có ứng viên, chỉ so khớp các descriptor thuộc cùng nhánh thay vì so tất cả với tất cả — tăng tốc bước kiểm định hình học ở mục 3 lên nhiều lần. Chi tiết này ít được nhắc nhưng nó là lý do DBoW2 nhanh trong thực tế.

TF-IDF đáng một câu giải thích vì nó là chỗ trực giác đẹp. Một từ xuất hiện trong *mọi* khung — chẳng hạn một mẫu vân sàn phổ biến — không mang thông tin phân biệt nào, nên IDF hạ trọng số của nó gần về không. Một từ hiếm — một vật đặc biệt trong phòng — mang rất nhiều thông

tin. Đây chính là cơ chế mà một người dùng khi tìm đường: bạn không định vị bằng “có tường trắng” mà bằng “có cái cây bonsai kỳ lạ ở góc”.

C. Place recognition học được

NetVLAD [68] và các hệ sau học một biểu diễn toàn cục trực tiếp từ dữ liệu, tối ưu cho việc nhận ra cùng một địa điểm qua các điều kiện khác nhau — mùa, thời tiết, ngày đêm. Đây là cải thiện rõ rệt so với BoW ở điều kiện thay đổi mạnh, vì BoW dựa trên descriptor cục bộ vốn không bền với thay đổi về ngoài lớn.

Tôi *không* dẫn số precision/recall so sánh các hệ ở đây, vì cùng lý do ở B/07 khối *Cận bầy*: các con số ấy phụ thuộc cực mạnh vào bộ dữ liệu, vào cách định nghĩa “cùng một địa điểm”, và vào ngưỡng. Điều có thể nói trung thực: học được thắng rõ ở *thay đổi về ngoài lớn* (mùa, ngày đêm) và cái giá là GPU cùng tính khó dự đoán khi ra ngoài phân bố huấn luyện.

D. Nhất quán chuỗi

Một kỹ thuật đơn giản và rất hiệu quả, dựa trên một quan sát tầm thường: robot di chuyển liên tục, nên nếu khung t khớp với khung s trong bản đồ, thì khung $t + 1$ nên khớp với khung gần $s + 1$.

SeqSLAM [69] khai thác điều này triệt để: thay vì tìm khớp cho từng ảnh, nó tìm khớp cho cả một *chuỗi* ảnh. Kết quả là nó chịu được những ảnh riêng lẻ rất kém — và nó hoạt động trong điều kiện mà khớp từng ảnh hoàn toàn thất bại.

Ý tưởng chung đáng nhớ hơn thuật toán cụ thể: *ràng buộc thời gian là một nguồn thông tin miễn phí chống lại perceptual aliasing*. Ba mươi cái kệ giống hệt nhau, nhưng chỉ có *một* cái mà robot có thể tới được từ chỗ nó đứng một giây trước. Hầu hết các hệ khai thác điều này ở dạng đơn giản — yêu cầu vài khung liên tiếp cùng đề xuất một vùng — và đó là một trong những phép kiểm rẻ nhất có sẵn.

11.3 Kiểm định hình học: tầng phòng thủ chính

A. Quy trình

Một ứng viên từ truy hồi *chưa phải* một loop closure. Nó phải qua kiểm định hình học, và đây là câu trả lời một phần cho câu hỏi mở đầu thứ hai.

Quy trình: khớp descriptor giữa khung hiện tại và khung ứng viên (dùng chỉ mục thuận để tăng tốc); chạy RANSAC để tìm phép biến đổi — PnP nếu ứng viên có landmark 3D, hoặc $Sim(3)$ nếu hai bên đều có cấu trúc; đếm inlier và đo phần dư.

Vì sao hai trăm inlier vẫn chưa đủ? Ba lý do, và cả ba đều là bài học đã gặp.

Một — perceptual aliasing thật sự. Nếu hai địa điểm giống nhau về mặt hình học *lẫn* về ngoài — hai hành lang xây giống hệt nhau — thì một phép biến đổi nhất quán tồn tại thật, và RANSAC tìm ra nó. Đây là trường hợp mà kiểm định hình học không cứu được, và nó là lý do cần thêm các tầng ở mục B.

Hai — cấu hình suy biến. Như A/01 mục 6 đã nói, số inlier cao tương thích hoàn hảo với một mô hình không xác định. Nếu hai khung đều nhìn vào một bức tường phẳng, phép biến đổi tìm được có thể có một hướng tự do.

Ba — chất lượng phân bố. Hai trăm inlier tụ ở một góc ảnh cho một phép biến đổi có số điều kiện tẻ, đúng như B/07 mục 4A.

B. Bốn tầng phòng thủ

Thiết kế đúng có nhiều tầng, mỗi tầng bắt một loại lỗi khác nhau. Đây là phần còn lại của câu trả lời cho câu hỏi thứ hai.

Tầng một — ngưỡng inlier bảo thủ. Cần thiết nhưng không đủ. Đặt cao hơn nhiều so với ngưỡng dùng cho tracking, vì chi phí lỗi khác nhau.

Tầng hai — nhất quán chuỗi. Yêu cầu nhiều khung liên tiếp cùng đề xuất một vùng bản đồ, với các tư thế tương đối nhất quán với odometry giữa chúng. Rẻ và mạnh — nó tấn công trực tiếp vào perceptual aliasing bằng ràng buộc thời gian.

Tầng ba — nhất quán với hiệp phương sai hiện tại. Nếu hệ thống nghĩ nó đang ở A với bất định ± 5 m, thì một loop closure nói nó đang ở cách A năm mươi mét là *không nhất quán* với chính ước lượng của nó. Về nguyên tắc, đây là phép kiểm mạnh nhất: nó dùng toàn bộ thông tin hệ thống có. Trong thực tế nó bị hạn chế bởi bệnh quá tự tin ở $A/06$ — nếu hiệp phương sai nhỏ hơn sự thật, phép kiểm này sẽ loại cả các loop closure đúng. Cách dùng an toàn là đặt ngưỡng rất rộng, chấp nhận rằng nó chỉ bắt được các trường hợp sai rành rành.

Tầng bốn — nhất quán với các loop closure khác. Nếu một cạnh mâu thuẫn với một nhóm cạnh đã được chấp nhận, nó đáng ngờ. Đây là ý tưởng của RRR [70]: nhóm các loop closure thành các cụm và kiểm tính nhất quán giữa các cụm, loại cả cụm nếu nó mâu thuẫn với phần còn lại. Mạnh nhất trong bốn tầng và cũng đắt nhất.

CẦN NHỚ

Chi phí của hai loại lỗi bất đối xứng tới mức nó phải chi phối mọi quyết định thiết kế trong chương này.

Bỏ lỡ một loop closure: hệ thống trôi thêm một chút, và sẽ có cơ hội khác ở vòng sau. Chi phí tuyến tính và có thể phục hồi.

Chấp nhận một loop closure sai: bản đồ gấp lại, hàng giờ làm việc bị phá huỷ, và thường không phục hồi được vì không có cơ chế nào phát hiện sau đó — bản đồ gấp vẫn tự nhất quán.

Ba hệ quả thiết kế trực tiếp. Một: **ngưỡng phải bảo thủ**, và tỉ lệ dương tính giả nên được đặt làm mục tiêu chứ không phải tỉ lệ phát hiện. Hai: **nhiều tầng phòng thủ độc lập** tốt hơn một tầng tốt, vì các tầng bắt các loại lỗi khác nhau. Ba: **luôn có back-end bền** như tầng cuối, vì không bộ phát hiện nào hoàn hảo.

Và một hệ quả về cách đo: khi đánh giá một hệ loop closure, đường cong precision-recall quan trọng hơn một con số duy nhất, và **vùng precision rất cao** mới là vùng đáng quan tâm. Một hệ có recall 80% ở precision 99% tốt hơn nhiều so với một hệ có recall 95% ở precision 90% — dù con số thứ hai trông đẹp hơn. Chi tiết ở [C/18](#).

11.4 Đóng vòng: áp dụng ràng buộc

A. Vì sao mono cần $Sim(3)$

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và nó là ứng dụng trực tiếp nhất của [A/05](#) mục 3B.

Trong SLAM mono, tỉ lệ là gauge tự do, và nó *trôi*: mỗi lần triangulate một landmark mới từ các landmark cũ, một sai số tỉ lệ nhỏ được đưa vào, và các sai số ấy tích lũy. Sau một vòng dài, tỉ lệ ở cuối vòng có thể lệch vài phần trăm so với ở đầu.

Hệ quả cụ thể nếu dùng $SE(3)$: phép biến đổi $SE(3)$ bảo toàn khoảng cách, nên nó *không thể* ghép hai đầu vòng có tỉ lệ khác nhau. Bộ giải sẽ tìm một thoả hiệp — và thoả hiệp ấy không phải

một sai số nhỏ phân bố đều mà là một *biến dạng* của bản đồ: một phần bị nén, một phần bị kéo, và các ràng buộc odometry bị vi phạm một cách có hệ thống. Đây không phải “kém chính xác hơn” mà là một chế độ hỏng khác về chất.

Với $Sim(3)$ [26], phép biến đổi có bảy bậc tự do gồm cả tỉ lệ, nên nó ghép được hai đầu và *đồng thời* sửa trôi tỉ lệ. Quy trình đầy đủ: tính $Sim(3)$ giữa hai khung, áp dụng nó để hiệu chỉnh các tư thế và landmark trong vùng ảnh hưởng, rồi chạy pose graph $Sim(3)$ để phân phối sai số.

Với stereo hoặc VIO, tỉ lệ quan sát được nên nó không trôi, và $SE(3)$ là đủ — và với VIO thì còn rút xuống bốn bậc vì trọng lực ghim roll với pitch. Số bậc tự do lại một lần nữa là hệ quả của cấu hình cảm biến, không phải một lựa chọn.

B. Pose graph và phân phối sai số

Sau khi có ràng buộc, chạy tối ưu pose graph (B/09 mục 4). Điều quan trọng cần hiểu: tối ưu này *không* làm cho sai số biến mất — nó *phân phối lại* sai số một cách tối ưu theo trọng số.

Trực giác: ba mươi mét sai lệch ở điểm cuối không phải lỗi của riêng bước cuối. Nó là tổng của hàng nghìn sai số nhỏ dọc đường. Pose graph phân bổ phần sửa cho từng cạnh theo tỉ lệ nghịch với độ tin cậy của nó — cạnh có hiệp phương sai lớn nhận phần sửa nhiều hơn. Đây chính là bài toán MAP đang làm đúng việc của nó, và nó cho thấy vì sao hiệp phương sai của các cạnh odometry phải đúng: nếu chúng sai, phần sửa được phân bổ sai chỗ.

C. Hoà nhập với luồng tracking

Một vấn đề kỹ thuật thật mà ít tài liệu học thuật nói tới, và nó thuộc về C/13 nhiều hơn nhưng cần nêu ở đây.

Tối ưu pose graph mất từ hàng chục mili giây tới hàng giây tùy kích thước; BA toàn cục sau đó còn lâu hơn. Trong lúc đó, luồng tracking *vẫn phải chạy* — không được dừng vì camera vẫn gửi khung và robot vẫn di chuyển. Nhưng tracking đang dùng bản đồ mà loop closing sắp sửa đổi.

Ba cách xử lý trong thực tế. *Đóng băng*: dừng tracking trong lúc cập nhật — đơn giản, và không chấp nhận được nếu vòng điều khiển phụ thuộc vào tư thế. *Sao chép*: loop closing làm việc trên một bản sao, rồi hoán đổi nhanh; tốn bộ nhớ và cần xử lý các khung xử lý trong lúc đó. *Cập nhật gia tăng*: áp dụng phần hiệu chỉnh dần dần. ORB-SLAM3 dùng một biến thể của cách thứ hai với một giai đoạn hoà nhập cẩn thận [10].

Điều đáng nhớ: đây là loại vấn đề *không xuất hiện trong bài báo nhưng chiếm phần lớn thời gian kỹ thuật*. Nó là một trong những lý do khoảng cách giữa một bài báo và một sản phẩm lớn hơn về ngoài, và là chủ đề của D/20.

11.5 Back-end bền: sống sót khi đã nhầm

Không bộ phát hiện nào hoàn hảo. Tầng cuối là làm cho bài toán tối ưu chịu đựng được các cạnh sai.

A. Ba họ kỹ thuật

Switchable constraints [71]: gán cho mỗi cạnh loop closure một biến “công tắc” $s \in [0, 1]$ và đưa nó vào bài toán tối ưu, với một prior kéo s về một. Nếu một cạnh mâu thuẫn mạnh với phần còn lại, bộ giải thấy rẻ hơn khi hạ s xuống — tức tắt cạnh đó. Ý tưởng đẹp: để bộ giải tự quyết định tin cạnh nào, thay vì quyết định trước.

DCS [72]: cùng ý tưởng nhưng loại bỏ biến công tắc bằng cách giải cho nó ở dạng đóng và thay vào — kết quả là một hàm mất mát bền có dạng cụ thể. Rẻ hơn và dễ cài hơn, cho hành vi tương tự.

GNC [34]: bắt đầu với hàm mất mát gần lỗi rồi làm nó không lỗi dần (A/04 mục 6C). Giải bài toán dễ trước để có khởi tạo tốt, rồi dùng nó cho bài toán khó hơn. Mạnh hơn hai cách trên khi tỉ lệ outlier cao.

Cả ba là các biến thể của cùng một ý tưởng, và đối ngẫu Black–Rangarajan [30] là chỗ sự thống nhất ấy được phát biểu: tối ưu với một hàm mất mát bền *tương đương* với tối ưu đồng thời trên biến gốc và một tập biến trọng số outlier. Switchable constraints làm điều đó một cách tường minh; DCS và M-estimator làm ngầm.

B. Điều kiện để nó hoạt động

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó là giới hạn quan trọng nhất của mục này.

Back-end bền hoạt động khi các cạnh sai là *thiếu số* và *không nhất quán với nhau*. Lúc đó, đa số đúng tạo ra một “sự thật” mà thiếu số sai mâu thuẫn với, và bộ giải tắt thiếu số.

Điều kiện ấy bị vi phạm trong đúng tình huống khó nhất: *perceptual aliasing có hệ thống*. Trong hành lang kho hàng với ba mươi kệ giống nhau, các loop closure sai không ngẫu nhiên — chúng đều nói “bạn đang ở kệ nào đó”, và chúng *nhất quán với nhau* theo một cách sai. Nếu số cạnh sai vượt số cạnh đúng, back-end bền sẽ tắt các cạnh *đúng*. Nó không chỉ thất bại mà thất bại theo hướng ngược lại.

Đây là lý do back-end bền là tầng *cuối cùng* chứ không phải tầng duy nhất, và là lý do bốn tầng ở mục 3B không thể bỏ qua. Nó cũng là lý do vì sao perceptual aliasing được liệt kê ở D/22 như một bài toán chưa giải quyết: nó phá vỡ giả định nền của mọi kỹ thuật bền.

C. Hai nguồn thông tin ngoài về ngoài

Đây là câu trả lời cho câu hỏi mở đầu thứ ba. Trong môi trường lặp lại có hệ thống, hai nguồn vẫn phân biệt được.

Một — ràng buộc thời gian và odometry. Robot không dịch chuyển tức thời. Nếu một giây trước nó ở kệ thứ năm, nó không thể đang ở kệ thứ hai mươi. Đây là thông tin mạnh và miễn phí, và nó là cơ sở của tầng hai và tầng ba ở mục 3B. Giới hạn: nó chỉ giúp khi odometry còn đáng tin — tức đúng lúc ta chưa cần loop closure. Sau tám tiếng trôi, bất định đã lớn tới mức ràng buộc này không loại được gì.

Hai — neo tuyệt đối. Một marker fiducial đặt ở các vị trí đã biết, một tag UWB, một tín hiệu wifi. Đây là cách mà phần lớn hệ thống kho hàng thật giải quyết vấn đề, và nó là một lời thú nhận đáng chú ý: *bài toán thuần thị giác trong môi trường lặp lại chưa có lời giải tốt, nên công nghiệp giải nó bằng cách thay đổi môi trường*. Cây `.. /marker/` bàn về nhánh này. Giới hạn: nó đòi sửa đổi cơ sở hạ tầng, không phải lúc nào cũng làm được, và nó chuyển bài toán sang việc bảo trì các neo đó.

Cả hai đều không phải lời giải hoàn chỉnh, và đó là phát biểu trung thực nhất tôi đưa ra được.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [10]	DBoW2 + kiểm định $Sim(3)$ + pose graph + BA toàn cục; Atlas ghép nhiều bản đồ	Cài đặt đầy đủ nhất; đọc phần hoà nhập với tracking ở mục 4C
VINS-Fusion [13]	Pose graph 4-DoF riêng biệt với luồng VIO	Kiến trúc hai tầng; 4-DoF vì có IMU, đúng A/05 mục 3B

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DBoW2 [66]	Từ điển nhị phân, chỉ mục ngược và thuận	Đọc để hiểu chỉ mục thuận — chỉ tiết ít được nhắc nhưng quyết định tốc độ
RRR [70]	Phân cụm loop closure và kiểm nhất quán giữa các cụm	Tầng bốn ở mục 3B — mạnh nhất và đắt nhất
GTSAM [34]	GNC cài sẵn cho pose graph	Mục 5A; dễ thử nhất trong ba họ
maplab [73]	Loop closure đa phiên; quản lý bản đồ dài hạn	Hiếm framework xử lý tốt mảng nhiều phiên — xem C/13

MINI PROJECT SLAM · Đo đường cong precision-recall của hệ loop closure, rồi cố tình tìm cạnh sai · 4 ngày

Mục tiêu	chuyển từ “hệ của tôi đóng vòng được” sang “hệ của tôi có tỉ lệ dương tính giả là X ở ngưỡng Y , và nó sống sót được $Z\%$ cạnh sai”.
Dữ liệu	một chuỗi có loop thật và có chân trị (KITTI hoặc EuRoC); cộng một chuỗi tự quay trong môi trường lặp lại — một hành lang, một dãy phòng giống nhau. Chuỗi thứ hai là phần quan trọng.
Các bước	<i>Phần một:</i> cài phát hiện bằng BoW, ghi lại điểm số cho mọi cặp khung, dùng chân trị để gán nhãn đúng/sai, vẽ đường cong precision-recall; chú ý vùng precision > 99%. <i>Phần hai:</i> thêm từng tầng phòng thủ ở mục 3B và vẽ lại đường cong sau mỗi tầng. <i>Phần ba:</i> chạy pose graph với các cạnh đã thêm $k\%$ cạnh sai ngẫu nhiên, với k từ 0 tới 50; so ATE giữa pose graph thường, với Huber, và với GNC.
Xong khi	bạn có ba kết quả. (1) Một đường cong precision-recall và một ngưỡng chọn từ vùng precision cao chứ không từ điểm F1 tối đa — và bạn giải thích được vì sao. (2) Một bảng cho thấy mỗi tầng phòng thủ đóng góp bao nhiêu, và tầng nào rẻ nhất trên mỗi đơn vị cải thiện. (3) Một đường cong ATE theo tỉ lệ cạnh sai, cho thấy GNC giữ được tới một ngưỡng rồi sụp — và bạn xác định được ngưỡng đó, vốn chính là điều kiện ở mục 5B.
Nối với	C/13-quan-ly-ban-do xử lý vấn đề hoà nhập ở mục 4C; C/18-danh-gia-va-phuong-phap-luan chuẩn hoá cách đo precision-recall; D/22-huong-nghien-cuu-con-trong lấy perceptual aliasing làm một trong tám khoảng trống.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Tỉ số ít nhất vài bậc độ lớn, và nó bất đối xứng về *bản chất* chứ không chỉ về độ lớn. Bỏ lỡ: trôi thêm một chút, có cơ hội khác ở vòng sau, phục hồi được. Chấp nhận sai: bản đồ gấp lại, hàng giờ làm việc bị phá, và *không phục hồi được* vì bản đồ gấp vẫn tự nhất quán nên không cơ chế nào phát hiện sau đó. Ba hệ quả thiết kế: ngưỡng phải bảo thủ và mục tiêu nên là tỉ lệ dương tính giả chứ không phải tỉ lệ phát hiện; nhiều tầng phòng thủ độc lập tốt hơn một tầng tốt; luôn phải có back-end bền lâu tầng cuối. Và khi đo, vùng precision rất cao mới là vùng đáng quan tâm, không phải điểm F1 tối đa.
- A2. Vì ba lý do. (a) *Perceptual aliasing thật sự*: hai hành lang xây giống hệt nhau thì một phép biến đổi nhất quán tồn tại thật, và RANSAC tìm ra nó — kiểm định hình học không cứu được. (b) *Cấu hình suy biến*: nếu cả hai khung nhìn vào tường phẳng, phép biến đổi có thể có hướng tự do, và số inlier cao tương thích hoàn hảo với điều đó (A/01 mục 6). (c) *Phân bố kém*: hai trăm inlier tự một góc cho phép biến đổi có số điều kiện tẻ. Phép kiểm bổ sung, theo thứ tự rẻ: nhất quán chuỗi (nhiều khung liên tiếp cùng đề xuất một vùng, với tư thế tương đối khớp odometry), nhất quán với hiệp phương sai hiện tại, và nhất quán với các loop closure đã chấp nhận (RRR).
- A3. Hai nguồn: (a) *ràng buộc thời gian và odometry* — robot không dịch chuyển tức thời, nên chỉ một trong ba mươi kè là tới được từ chỗ nó đứng một giây trước; (b) *neo tuyệt đối* — marker, UWB, wifi. Cả hai đều không hoàn chỉnh. Nguồn (a) chỉ giúp khi odometry còn đáng tin, tức đúng lúc ta chưa cần loop closure; sau tám tiếng trôi, bất định đã lớn tới mức nó không loại được gì. Nguồn (b) đòi sửa đổi cơ sở hạ tầng và chuyển bài toán sang việc bảo trì các neo. Việc công

nghiệp chủ yếu dùng (b) là một lời thú nhận đáng chú ý: bài toán thuần thị giác trong môi trường lặp lại chưa có lời giải tốt.

- A4. Cụ thể là bản đồ bị *biến dạng*, không phải “kém chính xác hơn”. Trong mono, tỉ lệ trôi, nên hai đầu vòng có tỉ lệ khác nhau vài phần trăm. Phép biến đổi $SE(3)$ bảo toàn khoảng cách nên nó không ghép được hai đầu có tỉ lệ khác nhau; bộ giải tìm một thoả hiệp, và thoả hiệp ấy nén một phần bản đồ và kéo phần khác, vi phạm các ràng buộc odometry một cách có hệ thống. $Sim(3)$ có bảy bậc gồm cả tỉ lệ nên nó ghép được và *đồng thời* sửa trôi tỉ lệ [26]. Với stereo hoặc VIO thì tỉ lệ quan sát được nên không trôi, và $SE(3)$ — hoặc bốn bậc với VIO — là đủ.
- A5. Điều kiện: các cạnh sai phải là *thiếu số* và *không nhất quán với nhau*. Lúc đó đa số đúng tạo ra một “sự thật” mà thiếu số sai mâu thuẫn với, và bộ giải tắt thiếu số. Điều kiện bị vi phạm trong đúng tình huống khó nhất: *perceptual aliasing có hệ thống*. Trong hành lang ba mươi kệ giống nhau, các cạnh sai không ngẫu nhiên — chúng nhất quán với nhau theo một cách sai. Nếu chúng vượt số cạnh đúng, back-end bền sẽ tắt các cạnh *đúng*, tức thất bại theo hướng ngược lại. Đó là lý do nó là tầng cuối chứ không phải tầng duy nhất, và là lý do perceptual aliasing nằm trong danh sách bài toán chưa giải ở D/22.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đi bộ quanh khu phố và vẽ bản đồ bằng cách đếm bước. Quay về chỗ xuất phát, bản đồ nói bạn còn cách đó ba mươi mét — vì đếm bước có sai số và sai số cộng dồn.

Nhưng bạn *nhận ra* cửa hàng trước mặt chính là cửa hàng lúc xuất phát. Thế là bạn lấy bản đồ và kéo nó cho hai điểm trùng nhau, chia đều ba mươi mét sai lệch ra cả vòng. Bản đồ đột nhiên đúng hơn hẳn — không chỉ ở điểm cuối mà ở mọi chỗ. Đây là phép thuật thật sự, và nó là lý do chữ M trong SLAM tồn tại.

Bây giờ hình dung bạn nhầm. Cửa hàng trước mặt cùng chuỗi, cùng biển hiệu, nhưng ở khu phố khác cách đó nửa cây số. Bạn vẫn kéo bản đồ cho hai điểm trùng.

Kết quả không phải một bản đồ hơi sai. Đó là một bản đồ *gập đôi*: nửa cây số bị nén thành không, hai con phố chồng lên nhau, và mọi thứ bạn ghi đúng suốt buổi sáng giờ nằm sai chỗ. Tệ nhất: bạn không có cách nào biết, vì bản đồ mới *vẫn tự nhất quán* — chỉ là nó nhất quán với một giả thuyết sai.

Đó là lý do chương này dài. Nhận ra chỗ cũ thì dễ. Chắc chắn rằng mình không nhầm mới là bài toán.

Và cách chắc chắn không phải nhìn kỹ hơn vào cửa hàng. Đó là hỏi thêm vài câu khác: *Tôi có thể đi từ chỗ tôi đứng một phút trước tới đây trong một phút không? Nếu đây thật là cửa hàng xuất phát, thì mọi thứ khác trong bản đồ có còn hợp lý không? Mấy phút vừa qua tôi có liên tục thấy những thứ quen thuộc, hay chỉ đúng một khoảnh khắc này?*

Còn một chỗ mà mọi câu hỏi đều thất bại: một dãy kho hàng có ba mươi cửa giống hệt nhau, cách nhau ba mét. Ở đó bạn không phân biệt được, và cũng không có gì trong khung cảnh giúp bạn phân biệt. Đó là bài toán khó nhất trong lĩnh vực này, và câu trả lời thật thà là nó chưa được giải.

Chỗ không rút gọn được: chỗ không rút gọn được là chính cái dãy kho hàng ấy. Mọi kỹ thuật trong chương này đều giả định rằng “số đồng thì đúng”. Nhưng khi môi trường lặp lại một cách có hệ thống, số đồng cũng có thể sai một cách nhất quán, và toàn bộ lập luận sụp đổ. Cách duy nhất thoát ra là thêm thông tin từ bên ngoài thị giác — và đó là lý do các nhà kho thật dán marker lên tường.

Một câu: *Đóng vòng đúng sửa cả một ngày làm việc; đóng vòng sai phá cả một ngày làm việc — và hai việc đó trông giống hệt nhau ở thời điểm quyết định.*

11.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *chi phí của dương tính giả lớn hơn âm tính giả nhiều bậc* — là một lập luận cấu trúc, và nó bị bác bỏ nếu một hệ có cơ chế *phát hiện và hủy* một loop closure sai sau khi đã áp dụng. Điều đó không bất khả: RRR [70] làm gần như vậy bằng cách kiểm nhất quán giữa các cụm và

có thể loại cả cụm đã chấp nhận. Nếu một cơ chế như vậy đủ tin cậy, sự bất đối xứng giảm và ngưỡng có thể nới. Tôi cho rằng nó chưa đủ tin cậy trong thực tế, nhưng đây là chỗ tôi có ít bằng chứng nhất (*tôi suy ra, chưa đối chiếu nguồn*).

Mệnh đề thứ hai — *mono cần $Sim(3)$, stereo cần $SE(3)$, VIO cần bốn bậc* — kế thừa từ A/05 mục 3B và kiểm được bằng cách chạy pose graph với số bậc sai rồi xem biến dạng. Nó bị bác bỏ nếu một hệ mono dùng $SE(3)$ mà không biểu hiện biến dạng — điều sẽ xảy ra nếu trôi tỉ lệ đủ nhỏ, chẳng hạn vòng rất ngắn hoặc có prior tỉ lệ ngầm từ đâu đó.

Mệnh đề thứ ba — *back-end bền cần cạnh sai là thiếu số và không nhất quán* — là điều kiện được nêu trong các công trình gốc. Điều tôi chưa kiểm là ngưỡng định lượng: bao nhiêu phần trăm cạnh sai thì GNC sụp? Mini project phần ba đo đúng điều đó, và con số thu được sẽ phụ thuộc mạnh vào cấu trúc đồ thị. Đây là một mục trong 99-chua-biet.md.

Một mệnh đề mềm và là mệnh đề tôi ít chắc nhất: chương này khẳng định *perceptual aliasing có hệ thống chưa có lời giải tốt*. Nó bị bác bỏ nếu một hệ đạt precision cao trong môi trường lặp lại có hệ thống mà chỉ dùng thị giác, được kiểm định trên dữ liệu thật chứ không phải benchmark. Các mô hình học được gần đây khai thác ngữ cảnh rộng hơn, nên ranh giới đang dịch chuyển, và tôi không chắc phát biểu này còn đúng trong hai năm nữa.

11.7 Kết nối

Nổi lên trên. B/07-dac-trung-va-data-association là chương mà bài toán này xuất hiện lần đầu ở quy mô nhỏ; điều đáng chú ý là *cùng một nguyên lý* — thông tin để phát hiện khớp sai nằm trong quan hệ với các khớp khác — áp dụng ở cả hai quy mô, chỉ khác hậu quả. B/09-paradigm-uoc-luong-back-end cho pose graph và giải thích vì sao MSCKF không đóng vòng được. A/03-lie-group-va-toi-uu-tren-da-tap cho $Sim(3)$; A/04-uoc-luong-map-va-bundle-adjustment cho GNC và M-estimator; A/05-observability-gauge-va-suy-bien cho số bậc tự do và cho lý do trôi tỉ lệ tồn tại.

Nổi xuống dưới. B/12-relocalization dùng cùng công nghệ phát hiện — BoW, kiểm định hình học — cho một mục đích khác, và có cùng cấu trúc chi phí bất đối xứng. C/13-quan-ly-ban-do xử lý vấn đề hoà nhập ở mục 4C và mở rộng sang ghép nhiều bản đồ, nhiều phiên. C/18-danh-gia-va-phuong-phap-luan chuẩn hoá cách đo precision-recall và lập luận vì sao vùng precision cao mới quan trọng.

Nổi xa hơn. D/19-ben-vung-trong-the-gioi-thuc bàn về loop closure khi môi trường đã thay đổi — cùng một chỗ nhưng đồ đạc đã dịch, ánh sáng đã khác. D/22-huong-nghien-cuu-con-trong lấy perceptual aliasing làm một trong tám khoảng trống, và mục 5B của chương này là lập luận vì sao nó khó: nó phá vỡ giả định nền của mọi kỹ thuật bền.

Nổi ngang: ../marker/ là nhánh giải bài toán này bằng cách thay đổi môi trường — đặt các neo có danh tính duy nhất. Đây là lời giải thực dụng nhất hiện có cho môi trường lặp lại, và đọc cây đó cho thấy cái giá của nó: hiệu chuẩn, bảo trì, và giới hạn tầm nhìn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Ước lượng tỉ số chi phí giữa loop closure sai và loop closure bỏ lỡ, rồi nêu ba hệ quả thiết kế của nó.
2. Vì sao phát hiện loop closure là bài toán truy hồi chứ không phải bài toán khớp, và điều đó quyết định công cụ thể nào?
3. Nêu bốn tầng phòng thủ chống loop closure sai, và nói mỗi tầng bắt được loại lỗi nào mà ba tầng kia bỏ sót.

4. Dùng $SE(3)$ thay cho $Sim(3)$ trong SLAM mono. Mô tả *cụ thể* bản đồ sẽ trông thế nào, không dùng cụm từ “kém chính xác hơn”.
5. Pose graph không làm sai số biến mất mà phân phối lại nó. Theo nguyên tắc nào, và điều gì phải đúng thì việc phân phối mới đúng?
6. Nêu điều kiện để back-end bền hoạt động, và mô tả một tình huống thực tế mà điều kiện đó bị vi phạm.
7. Khi đánh giá hệ loop closure, vì sao điểm F1 tối đa là tiêu chí sai, và tiêu chí nào đúng?

Relocalization

PHỤ THUỘC A/01-hình-học-da-thi (PnP — bước cuối của mọi pipeline); A/06-bat-dinh-va-lan-truyen-sai-so (hiệp phương sai để đặt ngưỡng chấp nhận); B/07-dac-trung-va-data-association (khớp điểm giữa hai khung rất xa nhau); B/11-loop-closure-va-nhat-quan-toan-cuc (cùng công nghệ phát hiện, cùng cấu trúc chi phí bất đối xứng).

TÓM TẮT

Có ba khoảnh khắc đặc biệt trong đời một hệ SLAM: lúc bắt đầu (B/10), lúc quay lại chỗ cũ (B/11), và lúc lạc đường — chương này. Relocalization trả lời câu hỏi “tôi đang ở đâu trong bản đồ này” khi không có tiên nghiệm nào, và nó là chương gần sản phẩm nhất trong Phần B vì nó quyết định điều mà khách hàng thật sự quan tâm: hệ thống có tự phục hồi được không, và mất bao lâu. Nội dung có bốn phần. Một: hai bài toán khác nhau thường bị gộp làm một — phục hồi sau mất bám ngắn, và định vị từ đầu trong một bản đồ đã có — với các yêu cầu rất khác nhau. Hai: kiến trúc phân cấp *truy hồi* → *khớp* → *PnP* đã trở thành chuẩn công nghiệp, và vì sao nó thắng. Ba: các hướng học được, gồm cả một trường hợp đáng học là hồi quy tư thế tuyệt đối — một ý tưởng hấp dẫn hoá ra không làm điều người ta tưởng. Bốn: kiểm định chấp nhận, với cùng lập luận bất đối xứng như B/11 nhưng gay gắt hơn. Nếu chỉ nhớ một điều: *một relocalization sai hại hơn không relocalization* — vì hệ thống đang lạc thì biết mình lạc, còn hệ thống tin sai thì không.

KIẾN THỨC CẦN CÓ TRƯỚC

- **relocalization** — xác định tư thế camera trong một bản đồ đã có, không dùng tiên nghiệm về vị trí. Khác với tracking ở chỗ tracking luôn có dự đoán từ khung trước.
- **kidnapped robot** — bài toán kinh điển: robot bị nhấc lên và đặt ở một chỗ khác trong khi tắt nguồn, rồi bật lên và phải tự tìm ra mình ở đâu. Tên gọi cho trường hợp khó nhất của relocalization.
- **cold start** — khởi động hệ thống trong một bản đồ đã lưu, không biết vị trí ban đầu. Cùng bài toán với kidnapped robot nhưng thường có ràng buộc thời gian gắt hơn vì người dùng đang chờ.
- **localization phân cấp (hierarchical localization)** — kiến trúc ba bước: truy hồi vài ứng viên toàn cục, khớp đặc trưng cục bộ với chúng, rồi PnP. Tách bài toán lớn thành hai bài toán nhỏ hơn mà mỗi cái đều có công cụ tốt.
- **scene coordinate regression** — học một hàm ánh xạ trực tiếp từ một mảng ảnh sang tọa độ 3D của nó trong bản đồ, rồi dùng PnP trên các tương ứng thu được. Bỏ qua bước truy hồi và bước mô tả.
- **hồi quy tư thế tuyệt đối (absolute pose regression, APR)** — học một mạng ánh xạ thẳng từ ảnh sang tư thế 6-DoF. Ý tưởng đơn giản nhất có thể, và mục 4B giải thích vì sao nó không hoạt động như kỳ vọng.

CÂU HỎI MỞ ĐẦU

- Q1. Hệ thống mất bám. Nó có thể (a) tiếp tục báo tư thế cuối cùng biết được, (b) báo “tôi không biết”, hoặc (c) relocalize và báo một tư thế mới. Xếp ba lựa chọn theo mức nguy hiểm, và giải thích thứ tự.

- Q2.** HLoc tách bài toán thành truy hồi rồi khớp rồi PnP, thay vì khớp trực tiếp với toàn bộ bản đồ. Nêu hai lý do khác nhau vì sao việc tách đó thắng — một lý do về chi phí và một lý do về chất lượng.
- Q3.** PoseNet học ánh xạ thẳng từ ảnh sang tư thế. Nó hoạt động trên tập kiểm tra. Nêu điều nó thực sự học được, và vì sao điều đó khác với điều người ta tưởng nó học được.
- Q4.** Robot lau nhà bật lên trong một căn hộ đã được lập bản đồ tuần trước. Từ tuần trước, một cái ghế đã bị dịch và rèm cửa đã được kéo. Nêu ba thứ có thể làm relocalization thất bại, xếp theo mức khó xử lý.
- Q5.** Relocalization của bạn trả về một tư thế với hai trăm inlier PnP. Nêu ba phép kiểm bổ sung trước khi chấp nhận nó, và nói mỗi phép bắt được lỗi gì.

12.1 Đặt vấn đề

Mọi hệ SLAM mất bám. Không phải nếu mà là khi nào: camera bị che, ánh sáng tắt, robot bị đẩy, chuyển động quá nhanh, một người đi ngang chắn hết khung hình. Câu hỏi thiết kế không phải làm sao để không bao giờ mất bám — điều đó bất khả — mà là *chuyện gì xảy ra sau đó*.

Bài toán gốc: cho một ảnh và một bản đồ, xác định tư thế camera mà không có tiên nghiệm nào về vị trí. Nghe giống loop closure, và công nghệ thì giống thật, nhưng bối cảnh khác ở một điểm quyết định: trong loop closure, hệ thống có một ước lượng vị trí (dù trôi) để kiểm chéo; trong relocalization, nó không có gì.

Mất đi phép kiểm chéo ấy là mất tầng phòng thủ thứ ba ở B/11 mục 3B — phép kiểm “có nhất quán với chỗ tôi nghĩ mình đang đứng không”. Nên relocalization khó hơn loop closure, dù dùng cùng công cụ.

Ai gặp bài toán này? Ba nhóm với ba yêu cầu khác nhau, và gộp chúng làm một là một lỗi thiết kế phổ biến.

Nhóm một — phục hồi sau mất bám ngắn. Hệ đang chạy, mất bám vài giây. Nó có một ước lượng thô từ trước khi mất, và nó chỉ cần tìm lại trong một vùng nhỏ. Yêu cầu: rất nhanh (người dùng không nên nhận ra), và có thể dùng tiên nghiệm.

Nhóm hai — cold start trong bản đồ đã có. Robot bật lên, không biết mình ở đâu. Không có tiên nghiệm, phải tìm trong toàn bộ bản đồ. Yêu cầu: nhanh vừa phải (vài giây chấp nhận được), và đúng — một khởi động sai vị trí là một lỗi mà khách hàng nhìn thấy ngay.

Nhóm ba — localization thuần túy. Không có SLAM, chỉ có một bản đồ lớn đã dựng sẵn và một ảnh cần định vị. Đây là bài toán của thị giác máy tính (định vị ảnh trong thành phố) hơn là của robot, và nó có quy mô lớn hơn nhiều — hàng triệu ảnh trong bản đồ.

Ba nhóm chia nhau công nghệ nhưng khác nhau về ràng buộc, và một hệ sản phẩm thường cần cả nhóm một lẫn nhóm hai với hai đường xử lý khác nhau.

Trước khi có kiến trúc phân cấp, cách làm là khớp đặc trưng của ảnh truy vấn với *toàn bộ* bản đồ. Với bản đồ vài nghìn landmark thì được; với bản đồ vài triệu thì không — cả về chi phí lẫn về chất lượng, vì tỉ lệ khớp sai tăng theo kích thước không gian tìm kiếm. Đây là câu trả lời cho câu hỏi mở đầu thứ hai và là lý do kiến trúc ba bước tồn tại.

TRỰC GIÁC

Bạn tỉnh dậy trong một thành phố bạn đã đi qua, không nhớ mình đang ở đâu, và có một cuốn sổ đầy ảnh chụp các góc phố bạn từng đi.

Cách ngây thơ: giờ từng trang, so ảnh trong sổ với cảnh trước mặt. Với sổ mười trang thì được. Với sổ mười nghìn trang thì hết ngày — và tệ hơn, xác suất bạn gặp một trang *trông giống* mà không phải đúng chỗ tăng theo số trang.

Cách người ta thật sự làm có hai bước. Đầu tiên nhìn tổng thể: kiến trúc kiểu gì, cây gì, màu tường ra sao — và lật nhanh tới vài trang có *ấn tượng chung* giống nhất. Rồi với vài trang đó, so kỹ từng chi tiết: cái biển hiệu này, vết nứt kia.

Hai bước ấy dùng hai loại thông tin khác nhau. Bước một dùng *ấn tượng toàn cục* — thô, nhưng so được rất nhanh với mười nghìn trang. Bước hai dùng *chi tiết cục bộ* — chính xác, nhưng chỉ so được với vài trang.

Đó là kiến trúc phân cấp, và nó thắng vì hai lý do chứ không phải một. Rõ ràng là nó *nhANH hơn*. Ít rõ hơn nhưng quan trọng không kém: nó *chính xác hơn*, vì khi chỉ so chi tiết với ba trang thay vì mười nghìn, cơ hội nhầm giảm hàng nghìn lần.

Còn một chuyện nữa, và nó là chuyện quan trọng nhất của chương. Giả sử sau cả hai bước bạn vẫn không chắc — có hai góc phố trông giống nhau. Bạn có hai lựa chọn: đoán một cái và đi tiếp, hoặc thừa nhận “tôi chưa biết” và đứng nhìn thêm.

Đoán và đi tiếp nguy hiểm hơn nhiều. Vì một khi đã đi, bạn sẽ diễn giải mọi thứ thấy sau đó theo giả định sai, và bạn càng lúc càng tin vào nó. Còn nếu bạn thừa nhận chưa biết, ít nhất bạn vẫn đang tìm.

12.2 Kiến trúc phân cấp

A. Ba bước

HLoc [12] hệ thống hoá kiến trúc đã trở thành chuẩn:

Bước một — truy hồi toàn cục. Tính một biểu diễn toàn cục cho ảnh truy vấn (BoW, NetVLAD, hoặc tương đương) và lấy k khung giống nhất trong bản đồ, với k thường vài chục. Chi phí gần hằng số nhờ chỉ mục.

Bước hai — khớp cục bộ. Với mỗi ứng viên, khớp đặc trưng cục bộ giữa ảnh truy vấn và khung ấy (B/07). Vì chỉ có vài chục ứng viên, ta có thể dùng matcher đắt như SuperGlue mà vẫn chấp nhận được.

Bước ba — PnP. Các khớp cho tương ứng 2D–3D với landmark của bản đồ; chạy PnP trong RANSAC (A/01 mục 5) để lấy tư thế và tập inlier.

B. Vì sao tách lại thẳng

Hai lý do độc lập, và lý do thứ hai quan trọng hơn nhưng ít được nhắc.

Chi phí: bước một là $O(1)$ nhờ chỉ mục ngược; bước hai là $O(k)$ với k nhỏ và không phụ thuộc kích thước bản đồ. Tổng chi phí gần như không tăng khi bản đồ lớn lên — một tính chất thiết yếu cho hệ chạy lâu.

Chất lượng: đây là lý do tinh tế hơn. Khi khớp một đặc trưng với toàn bộ bản đồ, số ứng viên sai tăng tuyến tính theo kích thước bản đồ, nên xác suất một khớp sai lọt qua ratio test cũng tăng. Với mười nghìn landmark, ratio test đủ tốt; với mười triệu, nó không còn đủ. Bằng cách giới hạn tìm kiếm vào vài chục khung, ta giữ không gian tìm kiếm nhỏ và ratio test lấy lại hiệu lực. Nói cách khác: *bước truy hồi không chỉ tăng tốc mà còn khôi phục điều kiện hoạt động của bước khớp.*

Điều này là một trường hợp của một nguyên tắc lặp lại trong cả cây: *thu hẹp không gian tìm kiếm là cách rẻ nhất để cải thiện data association* — cùng nguyên tắc với tiên nghiệm chuyển động ở B/07 mục 5C.

C. Dùng tiên nghiệm thô khi có

Với nhóm một — phục hồi sau mất bám ngắn — ta có tiên nghiệm, và bỏ qua nó là lãng phí.

Tư thế cuối cùng biết được, cộng với thời gian đã trôi qua và một giới hạn vận tốc, cho một vùng khả dĩ. Chỉ tìm trong vùng đó: rẻ hơn nhiều, và tỉ lệ dương tính giả giảm mạnh vì phần lớn các ứng viên gây nhầm lẫn nằm ngoài vùng.

Các nguồn tiên nghiệm thô khác đáng biết trong sản phẩm: odometry bánh xe tiếp tục chạy trong lúc thị giác mất bám; IMU cho hướng và một ước lượng dịch chuyển ngắn hạn; wifi hoặc bluetooth beacon cho vị trí ở mức phòng; một marker trong tầm nhìn cho vị trí tuyệt đối (`.../marker/`). Mỗi nguồn thu hẹp không gian tìm kiếm, và chúng *cộng dồn*.

Đây là lý do thực dụng quan trọng nhất để có nhiều cảm biến, và nó khác với lý do thường được nêu (độ chính xác cao hơn). Cảm biến phụ có giá trị lớn nhất không phải khi mọi thứ chạy tốt mà khi thị giác thất bại — chúng cho hệ thống một chỗ đứng để phục hồi.

12.3 Các hướng học được

A. Scene coordinate regression

Thay vì truy hồi rồi khớp, học một hàm ánh xạ trực tiếp từ một mảng ảnh sang tọa độ 3D của nó trong bản đồ. DSAC* [74] và ACE [75] theo hướng này: mạng cho mỗi mảng một tọa độ 3D, và PnP trong RANSAC chạy trên các tương ứng đó.

Ưu điểm: bỏ được cả bước truy hồi lẫn bước mô tả; và vì mạng đã học cảnh cụ thể, nó xử lý được các vùng mà descriptor tổng quát thất bại. ACE còn giảm thời gian huấn luyện cho một cảnh mới xuống mức phút, thay vì giờ.

Giới hạn quan trọng: mô hình *gắn với một cảnh cụ thể*, phải huấn luyện lại cho mỗi bản đồ. Với ứng dụng có một bản đồ cố định — một nhà máy, một bảo tàng — điều đó chấp nhận được. Với robot phải hoạt động ở nhà bất kỳ khách hàng nào, nó là một rào cản triển khai thật: mỗi lần lập bản đồ lại là một lần huấn luyện lại, và điều đó phải chạy trên thiết bị hoặc trên đám mây với mọi hệ quả về quyền riêng tư (C/13).

B. Hồi quy tư thế tuyệt đối, và vì sao nó không làm điều ta tưởng

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và tôi đưa nó vào vì nó là một bài học phương pháp có giá trị vượt ra ngoài chủ đề.

Ý tưởng đơn giản nhất có thể: học một mạng ánh xạ thẳng từ ảnh sang sáu (hoặc bảy) con số tư thế. PoseNet [76] làm điều đó và hoạt động — trên tập kiểm tra.

Sattler và cộng sự [77] đặt câu hỏi mà lẽ ra phải đặt sớm hơn: *mạng thực sự học được gì?* Kết quả của họ là mạng hành xử gần với *nội suy giữa các tư thế huấn luyện* hơn là với một bộ ước lượng hình học. Cụ thể, độ chính xác của nó không vượt hơn đáng kể so với một đường cơ sở rất đơn giản: truy hồi ảnh huấn luyện giống nhất rồi trả về tư thế của ảnh đó.

Điều đó có nghĩa gì? Mạng học được một ánh xạ *từ vẻ ngoài sang tư thế* trên tập huấn luyện, và nó nội suy giữa chúng. Nó *không* học hình học chiều, nên nó không tổng quát hoá ra các tư thế xa tập huấn luyện — đúng chỗ mà một bộ ước lượng hình học lại mạnh nhất.

Bài học phương pháp đáng rút ra, và nó áp dụng rộng hơn nhiều so với chủ đề này: *một phương pháp đạt điểm số tốt trên benchmark không chứng minh rằng nó làm điều ta nghĩ nó làm*. Cách kiểm là so với một đường cơ sở tầm thường thiết kế để có cùng khả năng nội suy nhưng không có gì khác. Nếu phương pháp mới không thắng đường cơ sở ấy, thì phần “mới” chưa được chứng minh. Đây là đúng tinh thần của `research-rules.md` mục 7 về việc phải làm baseline từ tế trước, và đó là lý do tôi đưa trường hợp này vào một chương học tập.

CẠM BẦY

Tôi không dẫn số liệu benchmark relocalization ở đây, và cần nói rõ lý do.

Các con số trên Aachen Day-Night, 7-Scenes hay Cambridge phụ thuộc cực mạnh vào: bản đồ được dựng thế nào (SfM từ chính tập huấn luyện hay từ nguồn khác), ngưỡng nào được coi là “thành công” (5 cm/5° hay 25 cm/2° — các bài khác nhau dùng khác nhau), và có bao nhiêu ảnh trong bản đồ. Hai bài báo báo cáo cho cùng phương pháp trên cùng bộ dữ liệu thường không so sánh được.

Nghiêm trọng hơn, với ứng dụng robot thì các benchmark ấy *đo sai thứ*. Chúng đo độ chính xác trung bình trên các truy vấn có câu trả lời trong bản đồ. Chúng không đo: tỉ lệ trả về một tư thế *sai* một cách tự tin; hành vi khi truy vấn *không* có trong bản đồ; và thời gian xấu nhất. Với sản phẩm, ba thứ đó quyết định hơn.

Cách đúng, và nó lặp lại lời khuyên ở [B/07](#): đo trên dữ liệu của chính bạn, trong chính điều kiện triển khai, và đo *tỉ lệ dương tính giả ở ngưỡng bạn định dùng* chứ không phải độ chính xác trung bình. [C/18](#) bàn cách thiết kế phép đo đó.

12.4 Kiểm định chấp nhận

A. Vì sao reloc sai hại hơn không reloc

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là luận điểm trung tâm của chương.

Xếp ba lựa chọn theo mức nguy hiểm, từ ít tới nhiều:

(b) Báo “tôi không biết” — an toàn nhất. Hệ thống ở trạng thái đã biết là xấu, và mọi module phía trên có thể phản ứng phù hợp: robot dừng lại, người dùng được thông báo, một chiến lược phục hồi được kích hoạt.

(a) Tiếp tục báo tư thế cuối cùng — nguy hiểm vừa. Tư thế ngày càng sai theo thời gian, nhưng sai một cách *dự đoán được*: nó cũ, và nếu hệ thống ghi lại độ tuổi của ước lượng thì module phía trên biết mà chiết khấu.

(c) Relocalize sai — nguy hiểm nhất. Hệ thống báo một tư thế *mới*, với độ tin cậy cao, và hoàn toàn sai. Mọi thứ phía trên tin nó. Tệ hơn cả: hệ thống sẽ bắt đầu tracking từ tư thế sai đó, khớp landmark của vùng sai, và *củng cố* niềm tin sai — vì các quan sát mới sẽ khớp với giả thuyết sai một cách nhất quán trong một thời gian. Nếu nó còn ghi cập nhật vào bản đồ thì bản đồ cũng bị hỏng.

Chênh lệch giữa (b) và (c) là lý do chương này tồn tại. Và nó gay gắt hơn [B/11](#) vì ở đó, hệ thống ít nhất còn một ước lượng để kiểm chéo; ở đây nó không có gì.

B. Bốn phép kiểm

Đây là câu trả lời cho câu hỏi mở đầu thứ năm.

Một — số và tỉ lệ inlier PnP. Cần thiết, không đủ; cùng lý do ở [A/01](#) mục 6. Đặt ngưỡng cao hơn nhiều so với tracking.

Hai — phân bố không gian của inlier. Hai trăm inlier tụ ở một góc ảnh cho một tư thế có bất định lớn theo vài hướng, dù số đếm trông tốt. Kiểm bằng diện tích bao lồi của inlier hoặc bằng số ô lưới có inlier — rẻ và hiệu quả, và nó là [B/07](#) mục 4A áp dụng ở đây.

Ba — hiệp phương sai của tư thế. Tính Λ của bài toán PnP và xem phổ của nó ([A/06](#)). Đây là phép kiểm đúng về nguyên tắc vì nó đo trực tiếp mức xác định. Giới hạn: nó chịu bệnh quá tự tin, nên ngưỡng phải bảo thủ.

Bốn — nhất quán thời gian. Đây là phép kiểm mạnh nhất và rẻ bất ngờ: *đừng chấp nhận một relocalization từ một khung duy nhất*. Yêu cầu vài khung liên tiếp cùng cho tư thế nhất quán với nhau và nhất quán với chuyển động đo được giữa chúng (từ odometry, IMU, hay optical flow). Một dương tính giả hiếm khi lặp lại nhất quán qua nhiều khung với chuyển động đúng.

Cái giá của phép kiểm bốn là độ trễ — vài trăm mili giây. Với nhóm hai (cold start) đó là chấp nhận được và nên làm mặc định. Với nhóm một (phục hồi nhanh) thì cần cân nhắc, và đó là một lý do nữa để tách hai đường xử lý.

C. Ba chế độ hồng trong sản phẩm

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, xếp theo mức khó xử lý.

Dễ nhất — thay đổi ánh sáng. Rèm kéo, đèn bật. Descriptor cổ điển kém bền với điều này; place recognition học được bền hơn nhiều. Có lời giải, dù tốn tài nguyên.

Khó vừa — vật thể dịch chuyển. Cái ghế đã dịch. Các landmark trên nó giờ ở sai chỗ, và chúng sẽ là outlier trong PnP. RANSAC xử lý được nếu chúng là thiểu số. Vấn đề thật là *tích lũy*: sau vài tháng, một tỉ lệ đáng kể bản đồ đã lỗi thời, và có lúc thiểu số thành đa số. Cần cơ chế cập nhật bản đồ ([C/13](#)), và đó là bài toán chưa được giải tốt.

Khó nhất — thay đổi cấu trúc. Bàn ghế được sắp lại hoàn toàn, tường được sơn lại, một phòng được cải tạo. Bản đồ cũ không còn mô tả thế giới. Không phép kiểm nào phân biệt được “tôi đang ở chỗ khác” với “tôi ở đúng chỗ nhưng chỗ này đã đổi”, và hai kết luận đòi hai hành động ngược nhau. Đây là bài toán *lifelong mapping*, và [D/22](#) xếp nó là một trong tám khoảng trống lớn nhất.

CẦN NHỚ

Một hệ relocalization dùng được trong sản phẩm phải trả lời được ba câu, không phải một.

Câu một: *tôi ở đâu?* — đây là câu mà mọi bài báo trả lời.

Câu hai: *tôi có chắc không?* — một tư thế kèm hiệp phương sai đáng tin, hoặc ít nhất một mức tin cậy đã được hiệu chuẩn trên dữ liệu thật. Rất ít công trình trả lời câu này.

Câu ba: *tôi có nên nói gì không?* — tức khả năng trả về “không biết”. Đây là câu quan trọng nhất cho sản phẩm và là câu ít được nghiên cứu nhất.

Ba câu ấy tương ứng với ba chỉ tiêu đo khác nhau, và một hệ tối ưu cho câu một có thể rất tệ ở câu ba. Khi đánh giá, hãy đo cả ba — và với sản phẩm, hãy đặt chỉ tiêu cho câu ba trước, rồi tối ưu câu một trong ràng buộc đó.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [10]	Reloc bằng DBoW2 + PnP RANSAC + tối ưu tư thế; yêu cầu nhất quán qua nhiều khung	Cài đặt của mục 2 và phép kiểm bốn ở mục 4B
HLoc [12]	NetVLAD/SuperPoint/SuperGlue + PnP; pipeline chuẩn công nghiệp	Kiến trúc phân cấp đầy đủ; đọc để thấy mỗi bước tách bạch thể nào

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ACE [75]	Scene coordinate regression với huấn luyện nhanh cho cảnh mới	Mục 3A; chú ý rằng buộc “một mô hình một cảnh”
[77]	Phân tích cho thấy APR gần với truy hồi ảnh hơn là với ước lượng hình học	Mục 3B — bài học phương pháp quan trọng hơn kết quả cụ thể
RTAB-Map	Reloc và quản lý bộ nhớ cho bản đồ dài hạn	Một trong ít hệ mã nguồn mở nghĩ về hoạt động nhiều phiên

MINI PROJECT SLAM · Đo tỉ lệ dương tính giả của relocalization, rồi hạ nó xuống bằng bốn phép kiểm · 3 ngày

Mục tiêu	chuyển từ “hệ của tôi relocalize được” sang “hệ của tôi trả về tư thế sai $X\%$ số lần ở ngưỡng Y , và nói không biết $Z\%$ ”.
Dữ liệu	một chuỗi có chân trị để dựng bản đồ, và một chuỗi thứ hai cùng môi trường để truy vấn. Quan trọng: thêm <i>các truy vấn không có câu trả lời</i> — ảnh từ một phòng khác không có trong bản đồ. Phần này là phần bị bỏ qua trong mọi benchmark và là phần quyết định trong sản phẩm.
Các bước	cài pipeline ba bước ở mục 2A; chạy trên cả truy vấn có và không có câu trả lời; với ngưỡng inlier quét qua một dải, vẽ ba đường cong theo ngưỡng — tỉ lệ đúng, tỉ lệ sai, tỉ lệ “không biết”; rồi thêm từng phép kiểm ở mục 4B và vẽ lại; cuối cùng đo độ trễ thêm vào bởi phép kiểm bốn.
Xong khi	bạn có một bảng: mỗi phép kiểm giảm tỉ lệ dương tính giả bao nhiêu, tăng tỉ lệ “không biết” bao nhiêu, và tốn thêm bao nhiêu mili giây. Bạn chọn được một điểm làm việc <i>từ yêu cầu sản phẩm</i> — “tôi chấp nhận sai không quá $0,1\%$ ” — chứ không từ một ngưỡng chép ở đâu. Và bạn có con số cho các truy vấn không có câu trả lời, thứ mà không benchmark nào cho bạn.
Nối với	C/13-quan-ly-ban-do xử lý vấn đề bản đồ lỗi thời ở mục 4C; D/21-tu-prototype-len-mass-production biến điểm làm việc này thành một chỉ tiêu nghiệm thu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Từ an toàn tới nguy hiểm: (b) báo “không biết” — hệ ở trạng thái đã biết là xấu, mọi module phía trên phản ứng được; (a) báo tư thế cũ — sai dần nhưng sai *dự đoán được*, và nếu ghi kèm độ tuổi thì phía trên chiết khấu được; (c) reloc sai — nguy hiểm nhất vì hệ báo một tư thế *mới* với độ tin cậy cao và hoàn toàn sai. Tệ hơn cả, nó sẽ bắt đầu tracking từ đó, khớp landmark vùng sai, và *củng cố* niềm tin sai vì các quan sát mới khớp nhất quán với giả thuyết sai trong một thời gian. Nếu còn ghi vào bản đồ thì bản đồ cũng hỏng.
- A2. Lý do *chi phí*: bước truy hồi là $O(1)$ nhờ chỉ mục, bước khớp là $O(k)$ với k nhỏ và không phụ thuộc kích thước bản đồ — nên tổng chi phí gần không đổi khi bản đồ lớn lên. Lý do *chất lượng*, tinh tế hơn và quan trọng hơn: khi khớp với toàn bộ bản đồ, số ứng viên sai tăng tuyến tính theo kích thước, nên xác suất một khớp sai lọt qua ratio test cũng tăng — với mười triệu landmark, ratio test không còn đủ. Giới hạn tìm kiếm vào vài chục khung giữ không gian nhỏ và khôi phục điều kiện hoạt động của bước khớp. Đây là cùng nguyên tắc với tiên nghiệm chuyển động ở [B/07](#) mục 5C.
- A3. Nó học một ánh xạ *từ vẻ ngoài sang tư thế* trên tập huấn luyện, rồi *nhội suy* giữa chúng — chứ không học hình học chiều. Sattler và cộng sự [77] cho thấy nó không vượt hơn đáng kể một đường cơ sở rất đơn giản là truy hồi ảnh huấn luyện giống nhất rồi trả về tư thế của ảnh đó. Hệ quả: nó không tổng quát hoá ra các tư thế xa tập huấn luyện — đúng chỗ mà bộ ước lượng hình học mạnh nhất. Bài học phương pháp rộng hơn: điểm số tốt trên benchmark không chứng minh phương pháp làm điều ta nghĩ nó làm; cách kiểm là so với một đường cơ sở tầm thường có cùng khả năng nội suy nhưng không có gì khác.
- A4. Từ dễ tới khó: (a) *thay đổi ánh sáng* (rèm kéo) — descriptor cổ điển kém bền, place recognition học được bền hơn nhiều; có lời giải. (b) *vật thể dịch chuyển* (cái ghế) — landmark trên nó thành outlier, RANSAC xử lý được khi chúng là thiểu số; vấn đề thật là *tích lũy* qua nhiều tháng cho

tới khi thiếu số thành đa số, và cần cơ chế cập nhật bản đồ. (c) *thay đổi cấu trúc* — không phép kiểm nào phân biệt được “tôi ở chỗ khác” với “tôi ở đúng chỗ nhưng chỗ này đã đổi”, và hai kết luận đòi hai hành động ngược nhau. Đây là bài toán lifelong mapping, một trong tám khoảng trống ở D/22.

- A5. Bốn phép: (1) *số và tỉ lệ inlier* — cần nhưng không đủ, ngưỡng cao hơn tracking nhiều. (2) *phân bố không gian của inlier* — hai trăm inlier tụ một góc cho tư thế bất định theo vài hướng dù số đếm tốt; kiểm bằng số ô lưới có inlier. (3) *hiệp phương sai từ Λ của bài toán PnP* — đúng về nguyên tắc vì nó đo trực tiếp mức xác định, nhưng chịu bệnh quá tự tin nên ngưỡng phải bảo thủ. (4) *nhất quán thời gian* — mạnh nhất và rẻ bất ngờ: đừng chấp nhận từ một khung duy nhất, yêu cầu vài khung liên tiếp cho tư thế nhất quán với nhau và với chuyển động đo được giữa chúng. Cái giá của (4) là vài trăm mili giây độ trễ.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn tỉnh dậy giữa một thành phố từng đi qua, không nhớ mình ở đâu, và có một cuốn sổ ảnh các góc phố từng đi.

Cách làm hợp lý có hai bước. Đầu tiên nhìn tổng thể — kiến trúc, cây cối, màu tường — rồi lật nhanh tới vài trang có ấn tượng chung giống nhất. Rồi với vài trang đó, so kỹ từng chi tiết.

Hai bước, hai loại thông tin. Ấn tượng chung thì thô nhưng so được với mười nghìn trang trong vài giây. Chi tiết thì chính xác nhưng chỉ so được với vài trang. Ghép hai lại thì vừa nhanh vừa ít nhầm — và cái “ít nhầm” quan trọng hơn cái “nhanh”, vì so chi tiết với ba trang thay vì mười nghìn thì cơ hội nhầm giảm hàng nghìn lần.

Nhưng chuyện quan trọng nhất của chương không phải cách tìm. Đó là chuyện gì xảy ra khi bạn *không chắc*.

Giả sử có hai góc phố trông giống nhau và bạn không phân biệt được. Hai lựa chọn: đoán một cái rồi đi tiếp, hoặc nói “tôi chưa biết” rồi đứng nhìn thêm một lúc.

Đoán rồi đi tiếp tệ hơn nhiều, và không phải vì lý do hiển nhiên. Một khi đã tin mình ở góc phố A, bạn sẽ diễn giải *mọi thứ* thấy sau đó theo giả định ấy. Cái cửa hàng kia hơi khác? Chắc họ đổi biển. Con đường không rõ như mong đợi? Chắc mình nhớ nhầm. Bạn càng đi càng tin, và càng khó quay lại. Trong khi nếu bạn thừa nhận chưa biết, ít nhất bạn vẫn đang tìm.

Cái máy cũng hệt vậy, và tệ hơn: nó không có cảm giác gợn khi mọi thứ hơi không khớp. Nó chỉ thấy con số phù hợp đủ tốt và đi tiếp.

Nên câu hỏi thiết kế đúng không phải “làm sao để máy luôn tìm được chỗ đúng” mà là “làm sao để máy biết nói tôi chưa biết”. Và cách làm, hoá ra, đơn giản đến bất ngờ: *đừng quyết định từ một cái nhìn duy nhất*. Nhìn thêm vài giây, đi thêm vài bước, và kiểm xem những gì thấy được có khớp với cả việc mình vừa di chuyển thế nào không. Một sự nhầm lẫn hiếm khi sống sót qua phép kiểm đó.

Chỗ không rút gọn được: chỗ không rút gọn được là khi *chỗ cũ đã thay đổi*. Bạn về đúng nhà mình nhưng ai đó đã kê lại toàn bộ đồ đạc. Máy không có cách nào phân biệt “tôi đang ở nhầm nhà” với “tôi ở đúng nhà nhưng nhà đã đổi” — và hai kết luận đòi hai hành động trái ngược. Đó là bài toán khó nhất trong triển khai dài hạn, và nó chưa có lời giải tốt.

Một câu: Một hệ đang lạc thì biết mình lạc; một hệ tin sai thì không — và đó là toàn bộ lý do phải biết nói “chưa biết”.

12.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *reloc sai hại hơn không reloc* — là một lập luận cấu trúc dựa trên việc hệ thống sẽ củng cố niềm tin sai. Nó bị bác bỏ nếu một hệ có cơ chế *phát hiện và huỷ* một relocalization sai sau khi đã chấp nhận, đủ tin cậy để chi phí của (c) không còn vượt (b). Cơ chế như vậy có thể tồn tại — theo dõi tỉ lệ tracking và phần dư trong vài giây sau reloc, và huỷ nếu chúng xấu đi — và nó nên được cài. Tôi ngờ rằng nó không bắt được trường hợp khó nhất, là reloc vào một vùng *thật sự giống*, vì ở đó tracking sẽ chạy tốt một thời gian. Nhưng tôi chưa đo (*tôi suy ra, chưa đổi chiều nguồn*).

Mệnh đề thứ hai — *kiến trúc phân cấp thắng vì cả chi phí lẫn chất lượng* — kiểm được bằng cách so pipeline ba bước với khớp trực tiếp toàn bản đồ, ở nhiều kích thước bản đồ. Nó bị bác bỏ nếu chất lượng không khác nhau ở bản đồ nhỏ — điều tôi *dự đoán* là đúng, vì lập luận về chất lượng chỉ có hiệu lực khi bản đồ đủ lớn. Kiểm điều đó sẽ làm rõ ngưỡng kích thước mà kiến trúc phân cấp bắt đầu đáng giá.

Mệnh đề thứ ba — *APR hành xử như nội suy chứ không như ước lượng hình học* — là kết quả chính của [77] và là một mệnh đề đã được kiểm bởi người khác (cửa c). Nó có thể đã thay đổi với các kiến trúc mới hơn, và tôi chưa khảo sát tài liệu sau 2019 về điểm này. Đây là một mục trong 99-chua-biet.md.

Một mệnh đề mềm: chương này khẳng định *benchmark relocalization* đo sai thứ cho ứng dụng robot. Đó là một phán đoán về mức phù hợp của phép đo, không phải một sự kiện. Nó bị bác bỏ nếu ai đó cho thấy thứ hạng trên benchmark tương quan chặt với tỉ lệ dương tính giả trong triển khai thật. Tôi cho là không, vì benchmark không chứa truy vấn ngoài bản đồ, nhưng tôi không có dữ liệu.

12.6 Kết nối

Nổi lên trên. [B/11-loop-closure-va-nhat-quan-toan-cuc](#) chia sẻ gần như toàn bộ công nghệ và toàn bộ cấu trúc lập luận; khác biệt duy nhất và quyết định là ở đây không có ước lượng hiện tại để kiểm chéo, nên mất một tầng phòng thủ. [B/07-dac-trung-va-data-association](#) cho bước khớp và cho nguyên tắc thu hẹp không gian tìm kiếm. [A/01-hinh-hoc-da-thi](#) cho PnP. [A/06-bat-dinh-va-lan-truyen-sai-so](#) cho phép kiểm thứ ba và cảnh báo về giới hạn của nó.

Nổi xuống dưới. [C/13-quan-ly-ban-do](#) là chương xử lý ba vấn đề mà chương này nêu ra nhưng không giải: bản đồ lỗi thời theo thời gian, kích thước bản đồ tăng vô hạn, và việc ghép nhiều phiên. [C/18-danh-gia-va-phuong-phap-luan](#) chuẩn hoá cách đo ba câu hỏi ở khối *Cần nhớ*, đặc biệt câu ba vốn không có benchmark nào đo.

Nổi xa hơn. [D/19-ben-vung-trong-the-gioi-thuc](#) bàn về ba chế độ hồng ở mục 4C một cách đầy đủ hơn. [D/21-tu-prototype-len-mass-production](#) là chỗ chương này quan trọng nhất: khả năng tự phục hồi quyết định MTBF cảm nhận được của sản phẩm, và thời gian cold start là thứ khách hàng đo bằng đồng hồ. [D/22-huong-nghien-cuu-con-trong](#) lấy lifelong mapping — bài toán ở mục 4C — làm một trong tám khoảng trống.

Nổi ngang: ../marker/ cung cấp giải pháp thực dụng nhất cho cold start trong môi trường có thể sửa đổi: một marker trong tầm nhìn cho tư thế tuyệt đối ngay lập tức, với độ tin cậy cao hơn nhiều so với bất kỳ phương pháp thuần thị giác nào. Cái giá là phải đặt và bảo trì marker.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Xếp ba lựa chọn khi mất bám — báo tư thế cũ, báo không biết, relocalize — theo mức nguy hiểm, và giải thích cơ chế làm lựa chọn nguy hiểm nhất trở nên nguy hiểm.
2. Nêu hai lý do độc lập vì sao kiến trúc phân cấp thắng, và nói lý do nào chỉ có hiệu lực khi bản đồ đủ lớn.
3. PoseNet thực sự học được gì? Nêu đường cơ sở tầm thường dùng để phát hiện điều đó, và bài học phương pháp rộng hơn.
4. Nêu bốn phép kiểm trước khi chấp nhận một relocalization, và nói phép nào mạnh nhất và vì sao nó rẻ bất ngờ.
5. Ba chế độ hồng khi bản đồ cũ đi. Vì sao chế độ thứ ba không có lời giải tốt, và hai kết luận nào bị lẫn lộn?

6. Ba câu hỏi mà một hệ relocalization sản phẩm phải trả lời. Câu nào ít được nghiên cứu nhất, và vì sao nó quan trọng nhất?
7. Vì sao benchmark relocalization đo sai thứ cho ứng dụng robot? Nêu ba thứ chúng không đo.



HỆ THỐNG SLAM ĐẦY ĐỦ

TÓM TẮT

Phần B cho các mảnh rời. Phần C là chỗ chúng trở thành một *hệ thống* chạy được lâu dài. Sáu chương xử lý sáu câu hỏi mà không chương nào ở Phần B trả lời được: bản đồ được quản lý thế nào khi nó lớn lên vô hạn và thế giới thì thay đổi (chương 13); hiệu chuẩn — vốn được coi là hằng số suốt Phần A và B — thực ra là một nhánh của chính SLAM (chương 14); thêm cảm biến vào thế nào cho đúng (chương 15); làm sao có một bản đồ dày chứ không chỉ một đám mây điểm thưa (chương 16); học máy đóng góp ở đâu và ở đâu thì không (chương 17); và làm sao biết hệ thống của mình có thật sự tốt hơn không (chương 18). Nếu chỉ nhớ một điều: *chương 14 là chỗ luận đề trung tâm của cây được chứng minh bằng tay* — ta đi từ ô “hiệu chuẩn” sang ô “SLAM” trong bảng ở 00-map bằng cách thả dần các biến ra, và không có gì trong bộ giải phải thay đổi.

Phần này khác Phần B ở một điểm về bản chất. Phần B trả lời các câu hỏi có lời giải *đúng hoặc sai*: thuật toán này khớp điểm tốt hơn thuật toán kia, kiến trúc này chi phí thấp hơn kiến trúc kia. Phần C trả lời các câu hỏi có lời giải *phù hợp hoặc không phù hợp*: giữ bao nhiêu keyframe là đủ, cập nhật bản đồ theo chu kỳ nào, thêm cảm biến nào cho ứng dụng nào. Đó là các câu hỏi kỹ thuật hệ thống, và chúng phụ thuộc vào ràng buộc sản phẩm chứ không chỉ vào toán học.

Đây cũng là lý do phần này ít tài liệu học thuật hơn hẳn. Một bài báo có thể chứng minh thuật toán khớp điểm mới tốt hơn; rất khó chứng minh một chính sách quản lý bản đồ tốt hơn, vì “tốt hơn” phụ thuộc vào việc robot chạy bao lâu, trong không gian nào, và ai trả tiền cho bộ nhớ. Hệ quả thực tế là kiến thức trong phần này chủ yếu nằm trong mã nguồn của vài hệ thống trưởng thành chứ không nằm trong bài báo — và đó là lý do E/23 khuyến nghị đọc mã ORB-SLAM3 và maplab chứ không chỉ đọc bài của họ.

Sáu chương có vai trò rất khác nhau. Chương 13 là chương bị coi nhẹ nhất trong toàn lĩnh vực và có ảnh hưởng lớn nhất tới sản phẩm; nó nên được đọc kỹ ngay cả khi bạn chưa thấy vì sao. Chương 14 là chương hợp nhất của cả cây — nếu chỉ đọc một chương của Phần C thì đọc nó. Chương 15 là chương ngắn nhất vì phần lớn nội dung nằm ở các cây khác, nhưng nó chứa một nguyên tắc quan trọng về cách thêm nguồn thông tin. Chương 16 và 17 là hai chương nóng nhất về mặt nghiên cứu và cũng là hai chương hết hạn nhanh nhất. Chương 18 khép phần lại bằng câu hỏi “làm sao biết mình đúng”, và nó là chương mà mọi chương trước đều nợ.

Bảng 12.2. Bản đồ Phần C: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
13	13-quan-ly-ban-do	Chọn và loại keyframe; loại landmark; đồ thị đồng quan sát và essential graph; cửa sổ bản đồ cục bộ; làm thưa bản đồ; ghép nhiều phiên và Atlas; cập nhật dài hạn; nén và tuần tự hoá; quyền riêng tư	● ● ●
14	14-hieu-chuan-nhu-mc	Hiệu chuẩn nội từ target; tự hiệu chuẩn; ngoại stereo và camera-IMU; hiệu chuẩn trực tuyến có cổng observability; tự đánh giá chất lượng; suy biến trong hiệu chuẩn	● ● ●
15	15-hop-nhat-da-cam-b	Nguyên tắc chung để thêm một nguồn; IMU, odometry bánh xe, LiDAR, GNSS, marker, UWB; ràng buộc nonholonomic; căn chỉnh hệ toàn cục	● ●
16	16-dense-mapping-va-	Stereo và MVS; TSDF và voxel hashing; surfel và mesh; octree và ESDF cho planning; NeRF-SLAM; 3D Gaussian Splatting; đánh đổi dày so với thưa	● ●
17	17-hoc-may-trong-sla	Front-end học được; prior độ sâu; DROID-SLAM và luồng học được; bất định học được; foundation model 3D; SLAM khả vi; semantic SLAM; vì sao hybrid là hướng đúng	● ●
18	18-danh-gia-va-phuor	ATE và RPE, căn chỉnh đúng số bậc; drift theo đoạn; NEES; đánh giá reloc và loop closure; ablation có kỷ luật; mô phỏng; ý nghĩa thống kê; đánh giá ở đuôi phân bố	● ● ●

Chương 14 là chương nên đọc trước nếu chỉ có thời gian cho một chương — nó là chỗ luận đề trung tâm của cây được chứng minh, và nó thay đổi cách bạn nhìn cả Phần A. Chương 13 và 18 là hai chương có tỉ lệ giá trị trên độ nổi tiếng cao nhất: cả hai đều bị coi nhẹ trong tài liệu và cả hai đều quyết định hệ thống có dùng được ngoài phòng thí nghiệm hay không. Chương 15 ngắn và chủ yếu trở sang các cây khác; đọc nó để lấy nguyên tắc, rồi sang `../IMU_Fusion/` và `../marker/` khi cần chi tiết. Chương 16 và 17 nên đọc lướt để biết bản đồ lĩnh vực, rồi quay lại khi thật sự cần — chúng là hai chương hết hạn nhanh nhất và được chốt tại tháng 9/2026.

Quản lý bản đồ (Map Management)

PHỤ THUỘC [A/04-uoc-luong-map-va-bundle-adjustment](#) (phần bù Schur — đồ thị đồng quan sát có gốc rễ toán học ở đó, và chi phí bất đối xứng giữa tư thế và điểm là gốc của mọi chính sách trong chương này); [B/09-paradigm-uoc-luong-back-end](#) (kiến trúc nhiều luồng); [B/11-loop-closure-va-nhat-quan-toan-cuc](#) (vấn đề hoà nhập); [B/12-relocalization](#) (bản đồ lỗi thời).

TÓM TẮT

Đây là chương bị coi nhẹ nhất trong toàn lĩnh vực và có ảnh hưởng lớn nhất tới việc một hệ SLAM có dùng được trong sản phẩm hay không. Lý do bị coi nhẹ thì dễ hiểu: không ai viết được một bài báo chứng minh chính sách chọn keyframe của mình tốt hơn, vì “tốt hơn” phụ thuộc vào robot chạy bao lâu, ở đâu, và ai trả tiền cho bộ nhớ. Lý do quan trọng thì cũng dễ hiểu: mọi thứ ở Phần B giả định có một bản đồ, và không chương nào ở đó nói bản đồ ấy được tạo ra, cắt tỉa, lưu trữ, cập nhật và chia sẻ thế nào. Nội dung có bốn phần. Một: chính sách chọn và loại — keyframe và landmark — với lập luận rằng cả hai chính sách đều bắt nguồn từ một sự kiện toán học duy nhất ở [A/04](#). Hai: cấu trúc đồ thị của bản đồ, và vì sao đồ thị đồng quan sát không phải một heuristic mà là cấu trúc khác không của một ma trận. Ba: bản đồ sống lâu — nhiều phiên, ghép bản đồ, và bài toán chưa giải là thế giới thay đổi. Bốn: các ràng buộc kỹ thuật thật — nén, tuần tự hoá, phiên bản, và quyền riêng tư. Nếu chỉ nhớ một điều: *tư thế đắt, điểm rẻ* — và gần như mọi chính sách trong chương này là hệ quả của câu đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **keyframe** — một khung hình được giữ lại trong bản đồ như một biến tư thế, khác với các khung chỉ dùng để tracking rồi bỏ. Việc chọn khung nào thành keyframe là quyết định quan trọng nhất trong chương này.
- **đồ thị đồng quan sát (covisibility graph)** — đồ thị có đỉnh là keyframe và cạnh nối hai keyframe cùng nhìn thấy đủ nhiều landmark chung. Nó xác định “vùng lân cận” của một keyframe trong bản đồ.
- **essential graph** — một đồ thị con thừa hơn của đồ thị đồng quan sát, giữ lại cây bao trùm cộng các cạnh mạnh nhất và các cạnh loop closure. Dùng cho tối ưu pose graph, nơi ta cần liên thông chứ không cần mọi cạnh.
- **bản đồ cục bộ (local map)** — tập con của bản đồ mà tracking đang dùng ở thời điểm hiện tại — thường là các keyframe lân cận trong đồ thị đồng quan sát cùng landmark của chúng.
- **làm thưa bản đồ (map sparsification)** — chọn một tập con landmark để giữ sao cho khả năng định vị không giảm đáng kể trong khi kích thước giảm nhiều lần. Bài toán tối ưu tổ hợp, giải bằng heuristic.
- **Atlas** — kiến trúc giữ *nhiều* bản đồ độc lập cùng lúc thay vì một, với cơ chế ghép khi phát hiện chồng lấn. Cách xử lý mất bám tinh tế hơn so với vứt bản đồ và làm lại.
- **map inversion** — tấn công tái tạo lại ảnh gốc từ một bản đồ gồm điểm 3D và descriptor. Đây là một vấn đề quyền riêng tư thật, không phải lo xa.

CÂU HỎI MỞ ĐẦU

- Q1. Giữ nhiều keyframe hơn cho bản đồ chi tiết hơn và tracking bền hơn. Vì sao mọi hệ thống lại tiết kiệm keyframe một cách gắt gao trong khi hào phóng với landmark?
- Q2. Đồ thị đồng quan sát nghe như một cấu trúc dữ liệu tiện lợi do ai đó nghĩ ra. Nêu nguồn gốc toán học của nó, và nói vì sao điều đó có nghĩa là nó không tùy tiện.
- Q3. Robot chạy tám tiếng mỗi ngày trong cùng một toà nhà, ba trăm ngày một năm. Nêu ba thứ sẽ hỏng nếu bạn chỉ đơn giản tiếp tục thêm vào cùng một bản đồ, và nói cái nào khó xử lý nhất.
- Q4. Hệ thống mất bám. Lựa chọn hiển nhiên là vứt bản đồ và khởi tạo lại. Nêu một lựa chọn tốt hơn và nói nó đòi hỏi gì về kiến trúc.
- Q5. Bản đồ của bạn chỉ gồm tọa độ 3D và descriptor nhị phân — không có ảnh nào. Vì sao đó *không* đủ để nói nó không chứa thông tin riêng tư?

13.1 Đặt vấn đề

Mọi chương ở Phần B giả định có một bản đồ. Chúng nói làm sao tạo ra nó (B/10), làm sao dùng nó (B/12), làm sao sửa nó khi đóng vòng (B/11). Không chương nào nói nó *lớn lên thế nào và ta làm gì với việc đó*.

Bài toán gốc phát biểu đơn giản: một hệ SLAM chạy liên tục thêm keyframe và landmark vào bản đồ. Nếu không làm gì, bản đồ tăng tuyến tính theo thời gian, và ba thứ hỏng theo. *Bộ nhớ cạn. Chi phí tính toán tăng* — bundle adjustment cục bộ chậm dần, truy hồi loop closure chậm dần. Và, ít rõ hơn nhưng nghiêm trọng hơn, *chất lượng giảm*: một bản đồ đầy các keyframe gần trùng nhau và các landmark chất lượng kém không tốt hơn một bản đồ gọn gàng — nó tệ hơn, vì tỉ lệ khớp sai tăng theo số ứng viên (B/12 mục 2B).

Ai gặp bài toán này? Chỉ những người chạy hệ thống lâu hơn một chuỗi benchmark. Đây là lý do cấu trúc khiến chương này thiếu tài liệu: EuRoC dài hai phút, KITTI dài vài phút. Không bộ dữ liệu chuẩn nào dài tám tiếng, nên không ai cần giải bài toán này để đăng bài — và mọi người cần giải nó để bán sản phẩm.

Trước khi có các chính sách có nguyên tắc, cách làm phổ biến là ngưỡng thủ công: “tạo keyframe nếu đi quá x mét hoặc xoay quá y độ”. Cách đó hoạt động và nó là cách đúng để bắt đầu, nhưng nó hỏng ở hai chỗ. Nó không thích ứng với cảnh — x mét trong một hành lang hẹp khác hẳn x mét ngoài bãi đỗ xe. Và nó không có cơ chế *loại bỏ*, nên bản đồ vẫn tăng vô hạn, chỉ chậm hơn.

Chương này lập luận rằng phần lớn các chính sách đúng đều suy ra từ một sự kiện toán học duy nhất, đã được thiết lập ở A/04 mục 4C: *chi phí bundle adjustment tăng theo lũy thừa ba của số tư thế nhưng gần tuyến tính theo số điểm*. Tư thế đắt, điểm rẻ. Một khi nắm được điều đó, các chính sách không còn là heuristic tùy tiện mà là hệ quả kinh tế.

TRỰC GIÁC

Hình dung bản đồ như một cuốn album ảnh kèm chú thích, và bạn phải mang nó theo suốt đời.

Mỗi trang album là một keyframe — một chỗ bạn từng đứng, kèm danh sách những gì nhìn thấy từ đó. Mỗi chú thích là một landmark.

Bây giờ một sự kiện quyết định: *thêm một trang thì đắt, thêm một chú thích vào trang có sẵn thì rẻ*. Vì mỗi khi bạn muốn chỉnh lại album cho nhất quán, bạn phải so từng trang với từng trang khác — chi phí tăng theo bình phương số trang. Còn chú thích thì chỉ thuộc về một trang, nên thêm bao nhiêu cũng không làm việc so trang khó hơn.

Từ sự kiện đó suy ra gần hết chính sách. Thêm trang mới chỉ khi nó cho thấy thứ mà các trang cũ không thấy — nếu bạn chỉ nhích sang một bước thì trang mới gần như trùng trang cũ, không đáng. Ngược lại, ghi thật nhiều chú thích vào mỗi trang, vì chúng gần như miễn phí và chúng làm trang đó dễ nhận ra lại hơn.

Rồi có ba chuyện xảy ra theo thời gian, và chúng khó dần.

Album dày lên. Sau vài năm nó nặng quá không mang nổi. Bạn phải xé bớt trang — nhưng xé trang nào? Trang ở chỗ bạn ít khi quay lại, hoặc trang mà các trang lân cận đã ghi hết những gì nó ghi.

Bạn có nhiều album. Mỗi chuyến đi một cuốn. Có lúc bạn nhận ra hai cuốn ghi cùng một nơi, và bạn muốn ghép chúng.

Thế giới thay đổi. Cái nhà trong ảnh đã bị phá. Chú thích của bạn vẫn nói nó ở đó. Bạn không có cách nào biết cho tới khi quay lại — và khi quay lại, bạn không phân biệt được “mình nhớ nhầm chỗ” với “chỗ này đã đổi”.

Chuyện thứ ba là chuyện khó nhất, và nó chưa có lời giải tốt.

13.2 Chính sách chọn và loại

A. Vì sao tư thế đắt còn điểm rẻ

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là nền của cả chương.

Từ A/04 mục 4C: sau phần bù Schur, hệ còn lại có kích thước $6m \times 6m$ với m là số tư thế; chi phí giải nó là $O(m^3)$ trong trường hợp xấu (thực tế tốt hơn nhờ độ thưa, nhưng vẫn siêu tuyến tính). Việc khử n landmark tốn $O(n)$ vì khối V là khối chéo.

Hệ quả kinh tế trực tiếp: *gấp đôi số landmark làm chi phí tăng gấp đôi; gấp đôi số keyframe làm chi phí tăng nhiều lần hơn thế.* Với một ngân sách tính toán cố định, tối ưu là giữ ít keyframe và nhiều landmark trên mỗi keyframe.

Có một lý do thứ hai, độc lập và cũng quan trọng: *thông tin*. Hai keyframe cách nhau một bước chân nhìn thấy gần như cùng một thứ từ gần như cùng một chỗ, nên keyframe thứ hai thêm rất ít thông tin — các hàng Jacobian của nó gần tuyến tính phụ thuộc với hàng của keyframe thứ nhất. Nó tốn chi phí đầy đủ mà cho lợi ích gần bằng không. Ngược lại, thêm một landmark mới nhìn từ một hướng mới thật sự thêm ràng buộc.

Hai lý do ấy hội tụ về cùng một chính sách, và đó là lý do nó vững chắc.

B. Tiêu chí tạo keyframe

Ba tiêu chí, và điều đáng chú ý là cả ba đều là *tiêu chí thông tin*, không phải tiêu chí khoảng cách.

Một — đủ parallax so với keyframe gần nhất. Đây là tiêu chí đúng thay cho “đi quá x mét”, vì nó tự thích ứng với độ sâu cảnh: trong hành lang hẹp, một mét cho nhiều parallax; ngoài bãi đỗ xe, một mét cho rất ít. Cách đo: parallax trung vị của các landmark đang theo dõi, đúng đại lượng đã dùng ở B/10 mục 4B.

Hai — tỉ lệ theo dõi giảm. Nếu số landmark cũ còn nhìn thấy được đã giảm dưới một tỉ lệ nào đó, ta đang rời khỏi vùng mà bản đồ hiện tại mô tả, và cần một keyframe để neo vùng mới.

Ba — đã đủ xa về thời gian. Một điều kiện an toàn: dù hai tiêu chí trên chưa kích hoạt, vẫn tạo keyframe sau một số khung nhất định, để tránh trường hợp hệ thống trôi mà không có neo nào.

Tôi không đưa ra các giá trị ngưỡng cụ thể, và đó là có chủ ý. Chúng phụ thuộc vào σ của front-end (B/07), vào độ sâu điển hình của cảnh, vào tốc độ di chuyển, và vào ngân sách tính toán. Cách

đúng là *tính* chúng từ những đại lượng đó — ngưỡng parallax tính theo B/10 mục 2C — rồi hiệu chỉnh bằng đo đạc trên dữ liệu của chính mình. Chép một hằng số từ mã nguồn của hệ khác là cách nhanh nhất để có một con số kỳ bí mà ba năm sau không ai biết vì sao.

C. Loại bỏ: phần bị bỏ quên

Tạo thì ai cũng làm; loại thì ít hơn nhiều. Nhưng không có cơ chế loại thì bản đồ vẫn tăng vô hạn.

Loại keyframe dư thừa. Tiêu chí của ORB-SLAM [5] đáng học vì nó đúng nguyên tắc: loại một keyframe nếu phần lớn landmark của nó *cũng* được nhìn thấy bởi ít nhất ba keyframe khác. Lập luận: keyframe ấy không neo landmark nào một cách độc nhất, nên loại nó không mất thông tin về cấu trúc. Chú ý điều này khác hẳn với “loại keyframe cũ” — nó loại theo *mức dư thừa* chứ không theo tuổi, nên một keyframe cũ ở một góc ít ai qua sẽ được giữ, còn mười keyframe mới chồng lên nhau ở một chỗ sẽ bị tía.

Loại landmark chất lượng kém. Một landmark bị loại nếu nó được quan sát quá ít lần so với số lần *lẽ ra* được quan sát (tức nó nằm trong tầm nhìn nhưng không khớp được — dấu hiệu nó là một khớp sai hoặc một vật động), hoặc nếu phần dư của nó liên tục lớn, hoặc nếu bất định của nó không giảm dù đã quan sát nhiều lần (dấu hiệu suy biến hình học).

Điều đáng nhấn mạnh về chính sách loại landmark: *ngưỡng nên gặt lúc mới tạo và nới dần theo tuổi.* Một landmark vừa được triangulate chưa được kiểm chứng và nên bị loại dễ dàng; một landmark đã sống sót qua hàng trăm quan sát là tài sản quý và nên được bảo vệ. Chính sách này rẻ và hiệu quả bất ngờ, vì phần lớn landmark sai bị loại trong vài khung đầu.

I 13.3 Cấu trúc đồ thị của bản đồ

A. Đồ thị đồng quan sát và gốc rễ toán học của nó

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một trong những kết nối đẹp nhất trong cây.

Đồ thị đồng quan sát nối hai keyframe khi chúng cùng nhìn thấy đủ nhiều landmark chung. Nó trông như một cấu trúc dữ liệu tiện lợi do ai đó nghĩ ra vì nó hữu ích.

Nó không phải vậy. Quay lại A/04 mục 4C: sau phần bù Schur, ma trận còn lại là

$$S = U - WV^{-1}W^T,$$

và khối (i, i') của S khác không *đúng khi* tồn tại một landmark j mà cả tư thế i lẫn tư thế i' đều quan sát — vì chỉ lúc đó cả W_{ij} lẫn $W_{i'j}$ mới khác không.

Nói cách khác: *đồ thị đồng quan sát chính xác là cấu trúc khác không của S .* Nó không phải một heuristic mà là hình ảnh của một ma trận. (Cửa kiểm chứng a: suy trực tiếp từ công thức Schur.)

Điều này có ba hệ quả thực dụng. *Một:* hai keyframe nối nhau trong đồ thị chính là hai keyframe mà bộ giải *phải* xử lý cùng nhau; tách chúng ra hai bài toán con là làm mất một khối khác không. *Hai:* chọn bản đồ cục bộ cho tracking bằng cách lấy lân cận trong đồ thị đồng quan sát là đúng theo nghĩa toán học — đó chính là tập biến có tương tác trực tiếp. *Ba:* mật độ của đồ thị quyết định chi phí, nên chính sách keyframe ở mục 2 cũng chính là chính sách kiểm soát chi phí.

B. Essential graph và vì sao cần một đồ thị thưa hơn

Đồ thị đồng quan sát đầy đủ có thể rất đặc: trong một căn phòng nhỏ, mọi keyframe nhìn thấy mọi landmark, nên mọi cặp đều nối nhau. Với tối ưu pose graph ở B/11, điều đó làm bài toán đắt không cần thiết.

Essential graph là một đồ thị con giữ lại: cây bao trùm (đảm bảo liên thông), các cạnh có số landmark chung cao nhất, và *mọi* cạnh loop closure. Lập luận: với pose graph, mục tiêu là phân phối sai số trên toàn quỹ đạo, và điều đó cần *liên thông* chứ không cần mọi cạnh. Các cạnh yếu đóng góp ít và có thể bỏ.

Đây là một ví dụ của một khuôn mẫu chung: *dùng biểu diễn đầy đủ cho tối ưu cục bộ, biểu diễn thưa cho tối ưu toàn cục*. Cùng khuôn mẫu xuất hiện ở việc chia luồng ở mục 4A.

C. Làm thưa bản đồ

Với bản đồ dùng cho localization lâu dài, ta không cần *mọi* landmark — ta cần một tập con đủ để định vị ở mọi chỗ với độ chính xác yêu cầu. Bài toán chọn tập con ấy là một bài toán tối ưu tổ hợp, và nó có một cấu trúc đẹp: ta muốn tối đa hoá *độ phủ* (mọi vùng đều có đủ landmark nhìn thấy được) với ràng buộc về kích thước.

Dymczyk và cộng sự [78] đề nghị chọn theo điểm số kết hợp số lần quan sát, độ bền qua các phiên, và độ phủ không gian. Kết quả là một bản đồ nhỏ hơn nhiều lần mà khả năng định vị giảm ít.

Điều đáng nhớ về mặt nguyên tắc: tiêu chí chọn nên là *độ phủ*, không phải *chất lượng trung bình*. Một trăm landmark xuất sắc tụ ở một góc phòng kém hữu ích hơn ba mươi landmark trung bình trải khắp phòng — cùng lập luận với phân bố đặc trưng ở B/07 mục 4A, ở một quy mô khác.

13.4 Bản đồ sống lâu

A. Kiến trúc nhiều luồng

Một hệ SLAM thật có ít nhất ba nhịp độ khác nhau, và cố ép chúng vào một luồng là một lỗi thiết kế.

Tracking chạy ở tần số khung hình — 30 Hz nghĩa là 33 ms cho mỗi khung, và nó *không bao giờ được chặn*, vì camera vẫn gửi khung và robot vẫn di chuyển. *Local mapping* chạy khi có keyframe mới — vài lần mỗi giây, và có thể mất vài trăm mili giây. *Loop closing* chạy hiếm nhưng lâu — có thể vài giây.

Ba nhịp độ ấy chia nhau một bản đồ, và đó là nguồn của phần lớn độ phức tạp kỹ thuật trong một hệ SLAM thật. Ba vấn đề cụ thể: tracking đọc bản đồ trong khi local mapping ghi vào nó; loop closing sắp sửa dịch chuyển các keyframe mà tracking đang dùng; và khi loop closing hoàn thành, các khung đã xử lý trong lúc đó cần được hoà nhập.

CẠM BẮY

Các vấn đề ở mục 4A không xuất hiện trong bài báo nhưng chiếm phần lớn thời gian kỹ thuật của một hệ SLAM thật.

Một bài báo mô tả thuật toán như thể nó chạy tuần tự: tracking, rồi mapping, rồi loop closing. Mã nguồn thật thì đầy hàng đợi, khoá, bản sao, và cờ trạng thái — và phần lớn lỗi khó tái hiện nhất nằm ở đó chứ không nằm trong toán.

Ba hệ quả thực hành. *Một*: đọc mã ORB-SLAM3 hay VINS-Fusion để học *kiến trúc* có giá trị cao hơn đọc bài báo của chúng, và E/23 khuyến nghị điều này. *Hai*: khi thiết kế, hãy quyết định sớm ai sở hữu bản đồ và ai được ghi vào nó — sửa điều này về sau rất đắt. *Ba*: **tính tất định** là một yêu cầu thật chứ không phải một điều xa xỉ; nếu hệ thống cho kết quả khác nhau giữa hai lần chạy trên cùng dữ liệu vì thứ tự luồng khác nhau, bạn không

debug được một lỗi từ hiện trường. Đây là chủ đề của D/20 và là một trong những lý do chính đáng nhất để chấp nhận chạy chậm hơn một chút.

B. Nhiều phiên và Atlas

Đây là câu trả lời cho câu hỏi mở đầu thứ tư.

Khi hệ thống mất bám, lựa chọn hiển nhiên là vứt bản đồ và khởi tạo lại (B/10 mục 4C). Nó lãng phí: bản đồ cũ có thể hoàn toàn tốt, chỉ là ta tạm thời không biết mình đang ở đâu trong nó.

Atlas [10] làm khác: khi mất bám, bắt đầu một bản đồ *mới* và giữ bản đồ cũ ở trạng thái không hoạt động. Hệ thống tiếp tục chạy trên bản đồ mới. Song song, nó liên tục thử relocalize vào các bản đồ cũ; nếu thành công, nó *ghép* hai bản đồ bằng phép biến đổi tìm được và tiếp tục với bản đồ hợp nhất.

Ba lợi ích. *Không mất dữ liệu*: công sức lập bản đồ trước đó được giữ. *Phục hồi mềm*: hệ thống vẫn hoạt động ngay cả khi chưa tìm lại được. *Nhiều phiên tự nhiên*: bản đồ từ hôm qua chỉ là một bản đồ không hoạt động trong atlas, và cùng cơ chế ghép hoạt động.

Yêu cầu về kiến trúc: bản đồ phải là một đối tượng *tách rời và ghép được* — có hệ quy chiếu riêng, có thể serialize, và có phép toán ghép. Nhiều hệ thống có bản đồ gắn chặt với trạng thái toàn cục và không làm được điều này. Đây là quyết định thiết kế nên đưa ra sớm.

C. Thế giới thay đổi: bài toán chưa giải

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và là chỗ tôi phải thành thật rằng lĩnh vực chưa có lời giải tốt.

Ba thứ hỏng khi chạy lâu trong cùng một môi trường, xếp theo mức khó.

Dễ — *kích thước*. Giải được bằng các chính sách ở mục 2 và mục 3C. Không tầm thường nhưng có lời giải.

Khó vừa — *thay đổi về ngoài theo chu kỳ*. Ánh sáng ban ngày và ban đêm, rèm mở và đóng. Lời giải một phần: lưu *nhiều biểu diễn* cho cùng một địa điểm — mỗi điều kiện một tập descriptor — và chọn theo điều kiện hiện tại. Tồn bộ nhớ và cần một cách phân loại điều kiện, nhưng khả thi. OpenLORIS [79] là một trong ít bộ dữ liệu đo được điều này.

Khó nhất — *thay đổi cấu trúc*. Đồ đạc được sắp lại, một bức tường được dựng, một phòng được cải tạo. Đây là bài toán chưa giải, và lý do nó khó nằm ở một chỗ cụ thể: *không có cách nào phân biệt “bản đồ sai” với “tôi ở sai chỗ” từ bên trong hệ thống*. Cả hai cho cùng triệu chứng — landmark không khớp — và hai kết luận đòi hai hành động ngược nhau: cập nhật bản đồ, hay relocalize.

Các cách tiếp cận hiện có đều là thỏa hiệp. *Quên theo thời gian*: hạ trọng số landmark cũ dần — đơn giản, nhưng nó quên cả những thứ không đổi. *Đếm phiếu*: landmark nào liên tục không khớp qua nhiều lần đi qua thì loại — tốt hơn, nhưng cần nhiều lần đi qua và cần chắc rằng ta thật sự đi qua đúng chỗ, tức cần relocalization đáng tin, tức vòng tròn. *Lưu nhiều phiên bản*: giữ cả bản đồ cũ và mới — tồn bộ nhớ và chuyển bài toán thành việc chọn phiên bản.

Không cách nào giải quyết trường hợp khó nhất, và D/22 xếp đây là một trong tám khoảng trống lớn nhất. Tôi cho rằng nó là khoảng trống *đau nhất* trong triển khai thực tế, vì nó là điểm khác biệt lớn nhất giữa một hệ chạy được một ngày và một hệ chạy được một năm.

13.5 Ràng buộc kỹ thuật thật

A. Nén và tuần tự hoá

Một bản đồ phải lưu được xuống đĩa và tải lại — cho cold start (B/12), cho nhiều phiên, cho cập nhật OTA. Ba yêu cầu thường xung đột.

Kích thước: descriptor chiếm phần lớn. Với ORB, mỗi descriptor là 32 byte; với một bản đồ triệu landmark có vài quan sát mỗi cái, đó là hàng trăm megabyte. Các cách giảm: chỉ lưu một descriptor đại diện cho mỗi landmark thay vì mọi quan sát; lượng tử hoá; hoặc lưu chỉ số từ điển BoW thay vì descriptor đầy đủ — cách cuối làm mất khả năng khớp chính xác nhưng giữ được khả năng truy hồi.

Tốc độ tải: quan trọng cho cold start, nơi người dùng đang chờ. Định dạng nhị phân có cấu trúc phẳng tải nhanh hơn nhiều so với định dạng có nhiều con trỏ phải dựng lại.

Tính tương thích: bản đồ lưu bằng phiên bản phần mềm này phải đọc được bằng phiên bản sau. Đây là yêu cầu thật cho sản phẩm có cập nhật OTA, và nó đòi hỏi *phiên bản hoá định dạng* ngay từ đầu — thêm vào sau rất đau.

B. Quyền riêng tư và map inversion

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó là một vấn đề thật chứ không phải lo xa.

Trực giác thông thường: một bản đồ chỉ gồm tọa độ 3D và descriptor thì không chứa ảnh, nên nó không riêng tư. Trực giác đó *sai*.

Pittaluga và cộng sự [80] cho thấy có thể tái tạo lại ảnh nhận ra được từ một bản đồ điểm kèm descriptor. Lý do: descriptor mã hoá về ngoài cục bộ, và với đủ điểm, một mô hình học được có thể ghép chúng lại thành một ảnh. Nói cách khác, *bản đồ SLAM của một căn nhà là một dạng ảnh chụp căn nhà đó*, chỉ ở một mã hoá khác.

Hệ quả cho sản phẩm là thật và đáng nghĩ sớm. Nếu bản đồ được tải lên đám mây — cho xử lý, cho chia sẻ giữa các thiết bị, cho cập nhật — thì đó là truyền dữ liệu riêng tư, với mọi nghĩa vụ pháp lý kèm theo. Nếu bản đồ được lưu trên thiết bị, nó là một mục tiêu.

Các hướng giảm thiểu đang được nghiên cứu: thay điểm bằng *đường* (làm mất thông tin vị trí chính xác trong khi vẫn định vị được), làm nhiều descriptor, hoặc lưu bản đồ ở dạng chỉ dùng được để định vị chứ không dùng được để tái tạo. Không hướng nào hoàn thiện. Điều tối thiểu nên làm: *biết rằng bản đồ là dữ liệu riêng tư và đối xử với nó như vậy*.

CẦN NHỚ

Mọi chính sách trong chương này suy ra từ một câu: tư thế đất, điểm rẻ.

Chi phí BA là $O(m^3)$ theo tư thế và $O(n)$ theo điểm (A/04 mục 4C). Từ đó:

Tạo keyframe **tiết kiệm**, theo tiêu chí thông tin (parallax, tỉ lệ theo dõi) chứ không theo khoảng cách; tạo landmark **hào phóng**.

Loại keyframe theo **mức dư thừa**, không theo tuổi — một keyframe cũ ở góc hiếm người qua đáng giá hơn mười keyframe mới chồng lên nhau.

Chọn bản đồ cục bộ theo **lân cận trong đồ thị đồng quan sát**, vì đó chính là tập biến tương tác trực tiếp trong ma trận S .

Làm thưa bản đồ theo **độ phủ**, không theo chất lượng trung bình.

Và một điều không suy ra từ toán mà từ thực tế: **ngưỡng loại landmark nên gắt lúc mới tạo và nới dần theo tuổi**, vì phần lớn landmark sai lệch ra trong vài khung đầu, còn một landmark sống sót qua hàng trăm quan sát là tài sản đã được kiểm chứng.

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [10]	Covisibility + essential graph + spanning tree; culling keyframe theo dư thừa; Atlas nhiều bản đồ	Cài đặt đầy đủ nhất của cả chương; đọc mã để học kiến trúc ba luồng
maplab [73]	Nhiều phiên, ghép bản đồ, làm thưa, quản lý dài hạn	Hiếm framework coi quản lý bản đồ là chủ đề chính chứ không phải phụ
[78]	Làm thưa bản đồ theo độ phủ và độ bền qua các phiên	Mục 3C — nguyên tắc chọn theo độ phủ
RTAB-Map	Quản lý bộ nhớ có cấu trúc: working memory và long-term memory	Một cách tiếp cận khác cho bài toán kích thước, lấy cảm hứng từ mô hình trí nhớ người
[80]	Tái tạo ảnh từ point cloud kèm descriptor	Mục 5B — đọc để hết ảo tưởng rằng bản đồ không riêng tư
[79]	Bộ dữ liệu đo thay đổi môi trường theo thời gian	Một trong ít nguồn dữ liệu để nghiên cứu mục 4C

MINI PROJECT SLAM · Chạy một hệ SLAM trong tám tiếng, rồi xem cái gì hỏng · 4 ngày

Mục tiêu	thấy tận mắt ba chế độ hỏng ở mục 4C, vốn không xuất hiện trên bất kỳ benchmark nào vì mọi benchmark đều quá ngắn.
Dữ liệu	tự thu: đặt camera trên một xe đẩy hoặc robot, đi lặp lại một tuyến trong văn phòng hoặc nhà bạn, <i>nhiều lần trong nhiều ngày</i> — ít nhất năm phiên trải trên một tuần, và cố ý dịch chuyển vài đồ vật giữa các phiên.
Các bước	chạy một hệ có sẵn (ORB-SLAM3) trên toàn bộ dữ liệu theo trình tự thời gian; ghi lại theo thời gian: số keyframe, số landmark, bộ nhớ, thời gian mỗi khung của cả ba luồng, tỉ lệ tracking, tỉ lệ relocalization thành công ở mỗi phiên; rồi so bản đồ cuối với bản đồ sau phiên đầu và đếm bao nhiêu landmark của phiên đầu còn sống.
Xong khi	bạn có ba đường cong theo thời gian và một con số. Đường cong: kích thước bản đồ, thời gian mỗi khung, và tỉ lệ tracking. Con số: tỉ lệ landmark từ phiên đầu còn sống sau phiên cuối — đó là <i>tốc độ lão hoá</i> của bản đồ trong môi trường của bạn, và tôi không biết ai đã công bố con số này cho một môi trường thật. Nếu tỉ lệ tracking giảm dần qua các phiên, bạn vừa đo được bài toán ở mục 4C bằng dữ liệu của chính mình.
Nối với	D/19-ben-vung-trong-the-gioi-thuc và D/21-tu-prototype-len-mass-production xây trực tiếp trên kết quả này; D/22-huong-nghien-cuu-con-trong lấy lifelong mapping làm một khoảng trống, và số liệu của bạn là bằng chứng cho nó.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì chi phí bất đối xứng: sau phần bù Schur, hệ còn lại có kích thước $6m \times 6m$ và chi phí giải là siêu tuyến tính theo m , trong khi việc khử n landmark chỉ tốn $O(n)$ vì khối V là khối chéo (A/04 mục 4C). Gấp đôi landmark làm chi phí tăng gấp đôi; gấp đôi keyframe làm nó tăng nhiều lần hơn. Có lý do thứ hai độc lập và cũng quan trọng: hai keyframe cách nhau một bước chân cho các hàng Jacobian gần tuyến tính phụ thuộc nên thêm rất ít *thông tin* trong khi tốn chi phí đầy đủ. Hai lý do hội tụ về cùng chính sách, và đó là lý do nó vững.
- A2. Nó *chính xác* là cấu trúc khác không của ma trận $S = U - WV^{-1}W^T$ sau phần bù Schur: khối (i, i') của S khác không đúng khi tồn tại một landmark mà cả hai tư thế cùng quan sát. Nghĩa là nó không tùy tiện — nó là hình ảnh của một ma trận. Ba hệ quả: hai keyframe nối nhau là hai keyframe bộ giải *phải* xử lý cùng nhau; chọn bản đồ cục bộ bằng lân cận trong đồ thị là đúng theo nghĩa toán học vì đó là tập biến tương tác trực tiếp; và mật độ đồ thị quyết định chi phí, nên chính sách keyframe cũng là chính sách kiểm soát chi phí.
- A3. Ba thứ, theo mức khó tăng dần. (a) *Kích thước* — bộ nhớ cạn, tính toán chậm dần, và chất lượng giảm vì tỉ lệ khớp sai tăng theo số ứng viên; giải được bằng chính sách loại và làm thưa. (b) *Thay đổi về ngoài theo chu kỳ* — ánh sáng ngày đêm; giải một phần được bằng cách lưu nhiều

biểu diễn cho cùng địa điểm. (c) *Thay đổi cấu trúc* — đồ đạc sắp lại, tường dựng thêm; **khó nhất và chưa có lời giải**, vì không cách nào phân biệt “bản đồ sai” với “tôi ở sai chỗ” từ bên trong hệ thống — cả hai cho cùng triệu chứng nhưng đòi hai hành động ngược nhau.

- A4. Lựa chọn tốt hơn là *Atlas*: bắt đầu một bản đồ mới và giữ bản đồ cũ ở trạng thái không hoạt động, tiếp tục chạy trên bản đồ mới, đồng thời liên tục thử relocalize vào các bản đồ cũ — nếu thành công thì ghép. Ba lợi ích: không mất công sức đã bỏ ra, hệ vẫn hoạt động ngay cả khi chưa tìm lại được, và nhiều phiên trở thành trường hợp tự nhiên của cùng cơ chế. Yêu cầu kiến trúc: bản đồ phải là một *đối tượng tách rời và ghép được* — có hệ quy chiếu riêng, serialize được, và có phép toán ghép. Nhiều hệ có bản đồ gắn chặt với trạng thái toàn cục và không làm được; đây là quyết định nên đưa ra sớm.
- A5. Vì descriptor mã hoá về ngoài cục bộ, và với đủ điểm, một mô hình học được có thể ghép chúng lại thành một ảnh nhận ra được [80]. Nói cách khác, bản đồ SLAM của một căn nhà là một dạng ảnh chụp căn nhà đó, chỉ ở mã hoá khác. Hệ quả cho sản phẩm: tải bản đồ lên đám mây là truyền dữ liệu riêng tư với mọi nghĩa vụ pháp lý kèm theo; lưu trên thiết bị thì nó là một mục tiêu tấn công. Các hướng giảm thiểu — thay điểm bằng đường, làm nhiều descriptor — đều chưa hoàn thiện. Điều tối thiểu: biết rằng bản đồ là dữ liệu riêng tư và đối xử với nó như vậy.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ về bản đồ như một cuốn album ảnh có chú thích mà bạn phải mang theo suốt đời. Mỗi trang là một chỗ bạn từng đứng; mỗi chú thích là một thứ bạn nhìn thấy từ đó. Và có một sự thật quyết định mọi chuyện: *thêm một trang thì đắt, thêm một chú thích thì rẻ*. Vì mỗi lần muốn chỉnh album cho nhất quán, bạn phải so từng trang với từng trang khác — càng nhiều trang càng tốn công theo cấp số. Chú thích thì chỉ thuộc về một trang nên không làm việc so trang khó hơn.

Từ đó suy ra: keo kiệt với trang, hào phóng với chú thích. Chỉ thêm trang mới khi nó cho thấy thứ mà các trang cũ không thấy — nhích sang một bước thì không đáng. Và khi phải xé bớt trang, đừng xé theo tuổi mà xé theo mức *thừa*: một trang cũ chụp cái góc mà không trang nào khác có thì quý, còn mười trang mới chụp cùng một chỗ thì tĩa chín.

Rồi có ba chuyện xảy ra theo năm tháng, và chúng khó dần.

Album dày lên cho tới lúc không mang nổi — chuyện này giải được, chỉ cần kỷ luật xé bớt.

Bạn có nhiều album từ nhiều chuyến đi, và có lúc nhận ra hai cuốn ghi cùng một nơi — chuyện này cũng giải được, chỉ cần biết cách ghép.

Và chuyện thứ ba: *thế giới đổi*. Cái nhà trong ảnh đã bị phá, cái ghế đã dời chỗ. Chú thích của bạn vẫn nói chúng ở đó. Bạn chỉ phát hiện khi quay lại — và khi quay lại, bạn không phân biệt được “mình nhớ nhầm chỗ” với “chỗ này đã đổi”. Hai kết luận ấy đòi hai việc hoàn toàn trái ngược: một bên là đi tìm lại, một bên là sửa album.

Chuyện thứ ba chưa ai giải được, và nó là lý do chính khiến một hệ chạy tốt một ngày lại không chạy tốt được một năm.

Chỗ không rút gọn được: chỗ không rút gọn được là cái vòng luẩn quẩn của chuyện thứ ba: để biết bản đồ sai, bạn phải chắc rằng mình đang ở đúng chỗ; để chắc mình ở đúng chỗ, bạn phải tin bản đồ. Mọi cách tiếp cận hiện có đều là cách phá vòng ấy một cách thô — đếm phiếu qua nhiều lần đi qua, quên dần theo thời gian — và không cách nào thoả đáng.

Một câu: Tư thế thì đắt, điểm thì rẻ — và gần như mọi quyết định về bản đồ là hệ quả của câu đó.

13.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *đồ thị đồng quan sát là cấu trúc khác không của S* — là một sự kiện đại số suy trực tiếp từ công thức Schur (cửa a), kiểm được bằng cách dựng S và so mẫu khác không với đồ thị. Nó bị bác bỏ nếu có nhân tử nào nổi hai tư thế trực tiếp — chẳng hạn một ràng buộc odometry hay tiền tích phân IMU — vì lúc đó S có thêm khối khác không không đến từ landmark chung.

Đây là một điều kiện thật và đáng nêu rõ: với hệ có odometry, đồ thị đồng quan sát là một tập con của cấu trúc S , không phải toàn bộ.

Mệnh đề thứ hai — *tư thế đất, điểm rẽ* — suy từ A/04 và kiểm được bằng đo đạc (mini project ở chương 4). Nó mất hiệu lực khi đồ thị đồng quan sát rất đặc, vì lúc đó S đặc và chi phí theo m còn tệ hơn dự đoán — tức mệnh đề đúng theo hướng *mạnh hơn*, không phải yếu hơn.

Mệnh đề thứ ba — *thay đổi cấu trúc chưa có lời giải tốt* — là một phát biểu về trạng thái của lĩnh vực, và nó là mệnh đề tôi ít chắc nhất vì tôi *chưa khảo sát có hệ thống* tài liệu về lifelong SLAM theo chuẩn `research-rules.md`. Nó bị bác bỏ nếu một hệ duy trì được tỉ lệ relocalization ổn định qua nhiều tháng trong một môi trường thay đổi thật, được kiểm định độc lập. Đây là một trong những mục quan trọng nhất trong `99-chua-biet.md`, và nó nên được nâng lên thành một dự án khảo sát riêng.

Một mệnh đề mềm: chương này khẳng định *quản lý bản đồ bị coi nhẹ trong tài liệu*. Đó là một quan sát về văn hoá nghiên cứu chứ không phải một sự kiện đo được. Nó bị bác bỏ nếu một khảo sát cho thấy tỉ lệ đáng kể các bài SLAM có phần thực nghiệm về hoạt động dài hạn. Tôi cho là không, dựa trên việc các bộ dữ liệu chuẩn đều ngắn, nhưng tôi không đếm (*tôi suy ra, chưa đổi chiều nguồn*).

13.7 Kết nối

Nói lên trên. [A/04-uoc-luong-map-va-bundle-adjustment](#) là nguồn của toàn bộ chương; phần bù Schur cho cả đồ thị đồng quan sát lẫn chi phí bất đối xứng. [B/09-paradigm-uoc-luong-back-end](#) cho kiến trúc nhiều luồng và pose graph. [B/11-loop-closure-va-nhat-quan-toan-cuc](#) nêu vấn đề hoà nhập mà mục 4A ở đây xử lý. [B/12-relocalization](#) nêu vấn đề bản đồ lỗi thời mà mục 4C ở đây mở rộng — và vẫn không giải được.

Nói xuống dưới. [C/14-hieu-chuan-nhu-mot-nhanh-cua-slam](#) chia sẻ một vấn đề với chương này: cái gì được coi là cố định và cái gì được phép thay đổi theo thời gian. [C/16-dense-mapping-va-bieu-dien](#) là chỗ bài toán kích thước trở nên gay gắt hơn nhiều bậc, vì biểu diễn dày tốn bộ nhớ hơn biểu diễn thưa hàng trăm lần. [C/18-danh-gia-va-phuong-phap-luan](#) là chỗ ta cần các chỉ tiêu để so sánh hai chính sách quản lý bản đồ — và chương đó phải thừa nhận rằng các chỉ tiêu hiện có không đo được điều này.

Nói xa hơn. [D/20-toi-uu-hieu-nang-va-he-thong](#) lấy vấn đề nhiều luồng ở mục 4A làm chủ đề chính, đặc biệt là tính bất định và ngân sách bộ nhớ. [D/21-tu-prototype-len-mass-production](#) là chỗ mục 5 trở thành yêu cầu thật: nén bản đồ cho thiết bị nhúng, phiên bản hoá cho cập nhật OTA, và quyền riêng tư cho nghĩa vụ pháp lý. [D/22-huong-nghien-cuu-con-trong](#) lấy lifelong mapping ở mục 4C làm khoảng trống số ba, và tôi cho rằng đó là khoảng trống đau nhất trong triển khai thực tế.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao mọi hệ thống tiết kiệm keyframe mà hào phóng landmark? Nêu hai lý do độc lập.
2. Nêu gốc rễ toán học của đồ thị đồng quan sát, và nêu một điều kiện khiến phát biểu đó không còn đúng hoàn toàn.
3. Tiêu chí loại keyframe của ORB-SLAM loại theo mức dư thừa chứ không theo tuổi. Vì sao điều đó quan trọng, và nêu một trường hợp mà loại theo tuổi sẽ sai.
4. Ba nhịp độ của một hệ SLAM là gì, và nêu ba vấn đề kỹ thuật sinh ra từ việc chúng chia nhau một bản đồ.

5. Atlas giải quyết vấn đề gì mà “vứt bản đồ và làm lại” không giải quyết được, và nó đòi hỏi gì về kiến trúc?
6. Vì sao “bản đồ sai” và “tôi ở sai chỗ” không phân biệt được từ bên trong hệ thống, và vì sao điều đó làm lifelong mapping khó?
7. Bản đồ chỉ gồm điểm và descriptor. Vì sao nó vẫn là dữ liệu riêng tư, và hệ quả gì cho thiết kế sản phẩm?

Hiệu chuẩn như một nhánh của SLAM (Calibration as a Branch of SLAM)

PHỤ THUỘC A/02-mô-hình-camera (mô hình nội và méo, và kỹ năng phân biệt mô hình sai với tham số sai); A/04-uoc-luong-map-va-bundle-adjustment (khung MAP — chương này chỉ đổi danh sách biến); A/05-observability-gauge-va-suy-bien (điều kiện kích thích, Bảng điều kiện — không có nó thì hiệu chuẩn trực tuyến là mở rộng trạng thái mù quáng); A/06-bat-dinh-va-lan-truyen-sai-so (hiệp phương sai tham số là thứ nên in ra thay cho RMS).

TÓM TẮT

Đây là chương hợp nhất của cả cây. Suốt Phần A và Phần B, tham số hiệu chuẩn được coi là hằng số nạp từ một tệp — một giả định tiện lợi mà không chương nào biện minh. Chương này bỏ giả định ấy và cho thấy điều gì xảy ra: *không có gì trong bộ giải phải thay đổi*. Hiệu chuẩn và SLAM là cùng một bài toán MAP; chúng khác nhau đúng ở danh sách biến tự do. Nội dung đi theo một mạch duy nhất: bắt đầu từ hiệu chuẩn nội với target đã biết — bài toán dễ nhất vì cấu trúc bị fix — rồi thả dần các biến ra, qua tự hiệu chuẩn, qua hiệu chuẩn ngoại, tới hiệu chuẩn trực tuyến trong một hệ VIO đang chạy, tức tới chính bài toán SLAM đầy đủ. Mỗi bước thả một biến, và mỗi bước đòi một điều kiện kích thích cụ thể — đó là chỗ A/05 trả công toàn bộ. Hai mục cuối bàn về hai thứ mà sản phẩm cần mà nghiên cứu ít nói: cách tự đánh giá chất lượng hiệu chuẩn, và cách phát hiện khi nó đã trôi. Nếu chỉ nhớ một điều: *calibration target chỉ là một cách giảm phương sai bằng cách fix cấu trúc; khi không fix được cấu trúc, bạn phải dùng chuyển động để thay thế* — và đó là toàn bộ khác biệt giữa phòng lab và robot đang chạy.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hiệu chuẩn nội (intrinsic calibration)** — xác định các tham số mô tả cách camera biến tia thành pixel — tiêu cự, điểm chính, hệ số méo (A/02).
- **hiệu chuẩn ngoại (extrinsic calibration)** — xác định tư thế tương đối giữa hai cảm biến — hai camera trong cặp stereo, camera và IMU, camera và LiDAR.
- **sai lệch thời gian (time offset, t_d)** — độ lệch giữa mốc thời gian mà hai cảm biến gán cho cùng một sự kiện vật lý. Nguyên nhân: độ trễ truyền, độ trễ phơi sáng, đồng hồ khác nhau.
- **target hiệu chuẩn** — một vật có hình học đã biết chính xác — bảng cờ vua, ChArUco, lưới vòng tròn. Vai trò thật của nó là *fix cấu trúc*, tức loại bỏ một nhóm biến khỏi bài toán.
- **tự hiệu chuẩn (self-calibration)** — ước lượng tham số hiệu chuẩn mà không có target, bằng cách coi chúng là biến trong chính bài toán SLAM hoặc SfM.
- **hiệu chuẩn trực tuyến (online calibration)** — tự hiệu chuẩn chạy liên tục trong lúc hệ thống hoạt động, để bù trôi theo nhiệt độ, rung, lão hoá.
- **cổng observability (observability gating)** — cơ chế chỉ cập nhật một tham số khi chuyển động hiện tại kích thích nó đủ, và đóng băng nó khi không. Đây là điều phân biệt hiệu chuẩn trực tuyến có nguyên tắc với việc mở rộng trạng thái mù quáng.

CÂU HỎI MỞ ĐẦU

- Q1. Hiệu chuẩn dùng bảng cờ vua, SLAM dùng cảnh tự nhiên. Nêu chính xác điều mà bảng cờ vua cung cấp, và nói vì sao điều đó làm bài toán dễ hơn — không phải “vì nó chính xác” mà là gì?
- Q2. Hiệu chuẩn trực tuyến thường được mô tả là “chỉ cần thêm biến vào vector trạng thái”. Nếu đơn giản như vậy thì vì sao phép biến đổi ngoại của bạn trôi lung tung trên một chuỗi và đứng im trên chuỗi khác?
- Q3. Bạn hiệu chuẩn camera-IMU bằng cách xoay thiết bị quanh đúng một trục trong suốt quá trình. Thành phần nào của phép biến đổi ngoại không xác định được, và vì sao tích có hướng là thủ phạm?
- Q4. Một kỹ sư báo rằng hiệu chuẩn ngoại trực tuyến của anh ta hội tụ rất nhanh và rất ổn định. Nêu hai cách giải thích — một tốt một xấu — và nêu phép thử phân biệt.
- Q5. Bạn cần biết hiệu chuẩn đã trôi để yêu cầu hiệu chuẩn lại. Nhưng tại hiện trường không có target và không có chân trị. Nêu ba đại lượng đo được có thể phát hiện trôi.

14.1 Đặt vấn đề

Suốt hai phần đầu của cây, có một giả định được dùng liên tục mà không chương nào biện minh: rằng tham số hiệu chuẩn là *hằng số đã biết*. A/01 làm việc trên mặt phẳng chuẩn hoá, tức giả định đã gỡ được K và méo. A/04 đặt tư thế và cấu trúc làm biến, và lạng lẽ để hiệu chuẩn ra ngoài. B/08 viết phần dư chiếu với một hàm π cố định.

Giả định ấy sai theo hai cách, và cả hai đều tốn kém.

Sai cách thứ nhất — nó không bao giờ đúng chính xác. Mọi phép hiệu chuẩn đều có sai số, và như A/02 mục 4C đã chỉ ra, sai số ấy không biến mất mà *chảy* vào tư thế và cấu trúc theo một hoa văn có hệ thống. Bình phương tối thiểu không khử được sai lệch có hệ thống.

Sai cách thứ hai — nó thay đổi theo thời gian. Ống kính giãn nở khi nóng lên. Khung stereo biến dạng. Keo dán module lão hoá. Thiết bị bị va đập. Tham số đúng vào ngày xuất xưởng không còn đúng sau sáu tháng ở nhà khách hàng.

Bài toán gốc của chương: *làm sao ước lượng các tham số ấy, và làm sao giữ chúng đúng theo thời gian?*

Ai gặp bài toán này? Ai làm thị giác đo đạc. Nhưng mức độ đau khác nhau rất nhiều giữa nghiên cứu và sản xuất, và đó là điều đáng nói. Trong nghiên cứu, ta hiệu chuẩn một lần bằng Kalibr, lưu tệp YAML, và quên đi — vì thí nghiệm kéo dài vài giờ và thiết bị không rời khỏi bàn. Trong sản xuất, ta phải hiệu chuẩn hàng nghìn thiết bị trong dưới một phút mỗi cái, theo dõi trôi trong nhiều năm, và phát hiện khi một thiết bị cần hiệu chuẩn lại mà không có target trong tay (D/21).

Trước khi có khung hợp nhất, hiệu chuẩn và SLAM được học như hai môn riêng. Hiệu chuẩn có bàn xoay, bảng cờ vua, một phần mềm chạy một lần rồi ra một tệp. SLAM có camera chuyển động, điểm đặc trưng, quỹ đạo. Hai thứ trông không liên quan, và chúng được dạy ở hai chỗ khác nhau.

Cách nhìn đó sai, và nó sai theo một kiểu tốn kém: nó làm người ta học hai lần cùng một thứ, và tệ hơn, nó che mất việc *cùng một lý thuyết observability quyết định cả hai*. Khi một kỹ sư hỏi “vì sao hiệu chuẩn ngoại trực tuyến của tôi không hội tụ”, câu trả lời nằm ở đúng chỗ đã trả lời câu “vì sao tỉ lệ mono của tôi sai”: chuyển động không đủ kích thích thì ma trận thông tin suy biến theo một hướng, và bộ ước lượng không có cách nào biết mình đang đoán mò theo hướng đó.

Chương này lập luận rằng cả hai là *một* bài toán, và nó chứng minh điều đó bằng cách đi từ ô này sang ô kia trong bảng ở 00-map, mỗi lần thả thêm một biến.

TRỰC GIÁC

Hình dung lại hệ lò xo ở A/04: mỗi phép đo là một lò xo nối các ẩn số, và nghiệm là vị trí cân bằng.

Hiệu chuẩn bằng target và SLAM khác nhau ở *đúng một chuyện*: bạn hàn cứng cái gì vào tường.

Với target, bạn hàn cứng *cấu trúc* — bạn biết chính xác các góc bằng cờ cách nhau bao nhiêu milimet, vì bạn đã in nó ra. Hệ lò xo còn lại giải ra tham số camera và quỹ đạo của bảng. Với SLAM, bạn hàn cứng *tham số camera* — bạn đã đo chúng hôm trước. Hệ giải ra quỹ đạo và cấu trúc.

Với hiệu chuẩn trực tuyến, bạn *không hàn gì cả*. Hệ giải ra tất cả — nếu có đủ lò xo.

Cái “nếu” cuối câu là toàn bộ nội dung khó của chương, và nó chính là observability ở A/05. Khi bạn tháo một cái hàn, bạn thêm một bậc tự do; nếu không có lò xo nào căng khi bậc đó thay đổi, hệ có vô số nghiệm và bộ giải dừng ở một chỗ tùy tiện.

Từ đây rút ra câu quan trọng nhất của chương, và nó đáng thuộc lòng: *target không phải một dụng cụ chính xác kỳ diệu — nó chỉ là một cách giảm phương sai bằng cách fix cấu trúc*. Khi bạn không có target, bạn phải thay cái hàn ấy bằng thứ khác, và thứ duy nhất có sẵn là *chuyển động*. Chuyển động đa dạng làm nhiều lò xo căng theo nhiều hướng, và nó thay thế được cái hàn.

Đó là lý do mọi hướng dẫn hiệu chuẩn đều bảo bạn nghiêng bảng nhiều hướng, đưa ra sát rìa, lại gần rồi ra xa — và cũng là lý do một robot đi thẳng mãi không bao giờ tự hiệu chuẩn được.

14.2 Đi từ ô này sang ô kia

Đây là mục trung tâm của chương. Ta bắt đầu từ bài toán dễ nhất rồi thả dần các biến, và theo dõi xem mỗi lần thả đòi hỏi thêm điều gì.

A. Bước không: hiệu chuẩn nội với target

Biến: tham số nội θ_K (tiêu cự, điểm chính, hệ số méo) và tư thế của bảng ở mỗi ảnh $T_1, \dots, T_N$.

Hằng số: tọa độ 3D của các góc bảng trong hệ của bảng — *đã biết chính xác vì ta in ra nó*.

Nhân tử: với mỗi góc bảng nhìn thấy ở mỗi ảnh, một phần dư chiếu

$$\mathbf{r}_{ij} = \mathbf{z}_{ij} - \pi(T_i \mathbf{P}_j; \theta_K).$$

So với bundle adjustment ở A/04, khác biệt duy nhất là $\mathbf{P}_j$ là hằng số thay vì biến, và θ_K là biến thay vì hằng số. Cùng một bộ giải, cùng một cấu trúc thừa, cùng mọi thứ. (Cửa kiểm chứng a: dựng đồ thị nhân tử cho cả hai và so sánh.)

Điều target thực sự cung cấp — và đây là câu trả lời cho câu hỏi mở đầu thứ nhất — *không phải độ chính xác mà là việc loại bỏ một nhóm biến*. Với N ảnh và M góc bảng, SfM thuần túy sẽ có $6N + 3M$ ẩn; hiệu chuẩn bằng target chỉ có $6N + |\theta_K|$. Với M cỡ vài trăm, đó là khác biệt lớn — và quan trọng hơn số lượng, nó loại bỏ gauge tỉ lệ, vì kích thước bảng cho đơn vị mét.

B. Bước một: bỏ target, thả cấu trúc

Bây giờ bỏ bảng đi và dùng cảnh tự nhiên. Cấu trúc trở thành biến, và ta có *tự hiệu chuẩn*: bài toán SfM với tham số nội nằm trong tập biến. COLMAP làm chính xác điều này [11].

Cái được: không cần target, nên làm được ở bất cứ đâu và bất cứ lúc nào — kể cả trên robot đang chạy.

Cái mất: *phương sai tăng*, và gauge tỉ lệ quay lại. Bài toán bây giờ có nhiều biến hơn và ít ràng buộc hơn trên mỗi biến, nên bất định của θ_K lớn hơn. Và vì không còn kích thước đã biết, tỉ lệ lại là gauge tự do (A/05 mục 3).

Điều kiện mới xuất hiện: *chuyển động phải đủ đa dạng*. Bảng 5.2 ở A/05 liệt kê chính xác cái gì cần gì: tiêu cự cần đa dạng độ sâu, điểm chính cần đặc trưng ở nhiều vị trí trong khung hình, hệ số méo cần đặc trưng ở rìa. Với target, người thao tác cung cấp sự đa dạng ấy một cách có chủ ý; với cảnh tự nhiên, ta phải chờ nó xảy ra hoặc kiểm xem nó đã xảy ra chưa.

Đây là lần đầu trong chương mà việc thả một biến đòi thêm một điều kiện, và khuôn mẫu ấy lặp lại ở mọi bước sau.

C. Bước hai: thả ngoại

Với nhiều cảm biến, thêm phép biến đổi ngoại vào tập biến. Hai trường hợp đáng nói riêng vì điều kiện của chúng khác nhau.

Stereo: thả $T_{\text{trái} \rightarrow \text{phải}}$. Điều kiện: cảnh phải có *độ sâu đa dạng*. Lý do trực giác: nếu mọi điểm ở cùng một khoảng cách, thì một sai lệch nhỏ trong góc giữa hai camera có thể được bù bằng một sai lệch trong khoảng cách chung, và hai giả thuyết không phân biệt được. Có cả gần lẫn xa thì không bù được.

Camera-IMU: thả $T_{\text{cam} \rightarrow \text{imu}}$ và t_d . Đây là trường hợp được nghiên cứu kỹ nhất, và nó chứa một kết quả đẹp đáng dẫn lại.

Vận tốc của camera và vận tốc của IMU liên hệ qua

$$\mathbf{v}_{\text{cam}} = \mathbf{v}_{\text{imu}} + \boldsymbol{\omega} \times \mathbf{p}_{\text{cam}}^{\text{imu}},$$

với $\mathbf{p}$ là phần tịnh tiến của phép biến đổi ngoại. Số hạng $\boldsymbol{\omega} \times \mathbf{p}$ là *thứ duy nhất* trong toàn bộ hệ phương trình chứa $\mathbf{p}$.

Từ đó suy ra ngay câu trả lời cho câu hỏi mở đầu thứ ba. Nếu $\boldsymbol{\omega} = 0$ — tịnh tiến thuần — thì số hạng đó bằng không và $\mathbf{p}$ hoàn toàn không quan sát được. Nếu $\boldsymbol{\omega}$ luôn cùng một hướng — xoay quanh đúng một trục — thì tích có hướng $\boldsymbol{\omega} \times \mathbf{p}$ chỉ nhạy với thành phần của $\mathbf{p}$ *vuông góc* với $\boldsymbol{\omega}$; thành phần *song song* với trục quay không ảnh hưởng gì tới tích có hướng, nên nó không quan sát được. Cần xoay quanh ít nhất hai trục độc lập để xác định cả ba thành phần. (Cửa a: suy trực tiếp từ tính chất của tích có hướng.)

Đây là một trong những ví dụ sạch nhất trong cả cây về việc một điều kiện thực hành — “xoay thiết bị quanh cả ba trục khi hiệu chuẩn” — là hệ quả trực tiếp của một tính chất đại số. Và nó cho thấy vì sao lời khuyên truyền miệng không tổng quát hoá được: ai chỉ nghe “lắc thiết bị” mà không biết vì sao sẽ không biết phải làm gì với một cấu hình cảm biến mới.

D. Bước ba: thả tất cả, trong lúc đang chạy

Bước cuối: đặt mọi thứ vào tập biến — tư thế, vận tốc, độ lệch IMU, cấu trúc, nội, ngoại, t_d — và chạy liên tục trong khi hệ thống hoạt động. Đây chính là ô cuối cùng của bảng ở 00-map, và OpenVINS [42] là cài đặt tham chiếu.

Đây là câu trả lời cho câu hỏi mở đầu thứ hai. Mô tả “chỉ cần thêm biến vào vector trạng thái” đúng về *cơ chế* và sai về *điều kiện*. Công thức đầy đủ là:

hiệu chuẩn trực tuyến = mở rộng trạng thái + đủ kích thích + công để biết khi nào không đủ.

Về thứ hai giải thích vì sao cùng một hệ thống hội tụ trên chuỗi này và trôi trên chuỗi kia: chuỗi có xoay quanh nhiều trục kích thích $\mathbf{p}$ qua số hạng $\omega \times \mathbf{p}$, chuỗi tịnh tiến thuần thì không. Về thứ ba là mục 3.

CẦN NHỚ

Target không phải một dụng cụ chính xác kỳ diệu — nó là một cách giảm phương sai bằng cách fix cấu trúc.

Khi bạn không có target, bạn phải thay cái đó bằng thứ khác, và thứ duy nhất có sẵn là *chuyển động*. Đó là toàn bộ khác biệt giữa hiệu chuẩn trong phòng lab và hiệu chuẩn trên một robot đang chạy.

Hệ quả là một cách nhìn thống nhất cho mọi bài toán trong Bảng ở 00-map: mỗi bài toán là cùng một đồ thị nhân tử, và việc đi từ ô này sang ô kia là việc *tháo một cái hàn và thay nó bằng một điều kiện chuyển động*.

Hệ quả thực hành trực tiếp: khi thiết kế một quy trình hiệu chuẩn — dù trong lab hay trên dây chuyền — câu hỏi đúng không phải “dùng target nào” mà là “**tham số nào tôi cần, và chuyển động nào làm nó quan sát được**”. Bảng 5.2 ở A/05 là bảng trả lời câu đó, và mọi hướng dẫn thu dữ liệu hiệu chuẩn đều là bảng ấy viết lại thành thao tác tay.

14.3 Cổng observability: điều phân biệt có nguyên tắc với mù quáng

A. Vấn đề

Thả một tham số ra mà không kiểm điều kiện kích thích là một sai lầm có hậu quả cụ thể và nghiêm trọng hơn vẻ ngoài.

Khi một tham số không quan sát được, nó không đơn giản là “không hội tụ”. Nó *trôi tự do*, và tệ hơn, nó *hấp thụ sai số từ các nguồn khác*. Bộ giải thấy một hướng phẳng trong hàm mục tiêu, và nó dùng hướng đó để giảm phần dư gây ra bởi những nguyên nhân hoàn toàn khác — một vật động chưa loại, một mô hình méo sai, một sai lệch thời gian.

Kết quả là tình huống tệ nhất có thể: tham số hiệu chuẩn nhận một giá trị sai, phần dư trông đẹp, và sai số thật đã được giấu vào một biến mà không ai kiểm tra. Khi điều kiện chuyển động thay đổi và tham số ấy trở lại quan sát được, nó phải dịch chuyển một quãng lớn, và hệ thống trải qua một giai đoạn bất ổn không giải thích được.

B. Cơ chế cổng

Ý tưởng đơn giản và có nguyên tắc [81], [82]: theo dõi mức quan sát được của từng tham số theo thời gian thực, và chỉ cập nhật nó khi mức ấy đủ.

Cài đặt cụ thể có vài mức tinh vi.

Mức đơn giản nhất — cổng theo chuyển động. Với mỗi tham số, xác định trước một đại lượng chuyển động mà nó cần (Bảng 5.2), đo đại lượng ấy trên cửa sổ hiện tại, và đóng băng tham số nếu dưới ngưỡng. Với $\mathbf{p}_{\text{cam}}^{\text{imu}}$, đại lượng là mức đa dạng của hướng ω ; với t_d , là độ lớn của tốc độ góc.

Mức đúng hơn — cổng theo phổ ma trận thông tin. Dạng khối của Λ tương ứng với các tham số hiệu chuẩn, tính giá trị riêng nhỏ nhất hoặc số điều kiện, và đóng băng khi nó dưới ngưỡng. Đắt hơn nhưng tổng quát — nó không cần biết trước tham số nào cần chuyển động gì.

Mức thực dụng — prior mềm thích ứng. Thay vì đóng băng cứng, đặt một prior quanh giá trị hiện tại với độ mạnh *tỉ lệ nghịch* với mức quan sát được. Khi kích thích tốt, prior yếu và dữ liệu quyết

định; khi kích thích kém, prior mạnh và tham số giữ nguyên. Mượt hơn, tránh được các bước nhảy khi cổng đóng mở.

C. Vì sao hội tụ nhanh có thể là tin xấu

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và là một phép chẩn đoán đáng nhớ.

Hai cách giải thích cho việc hiệu chuẩn ngoại trực tuyến “hội tụ rất nhanh và rất ổn định”.

Cách tốt: chuyển động thật sự kích thích mạnh, ma trận thông tin xác định dương tốt, và tham số hội tụ về giá trị đúng. Điều này xảy ra và là điều ta muốn.

Cách xấu: tham số thực ra không quan sát được, nhưng có một prior mạnh (có thể là một prior bạn đặt, hoặc damping LM, hoặc một khởi tạo được tin quá mức) đang ghim nó. Nó “ổn định” vì nó không đi đâu cả — không phải vì dữ liệu xác nhận nó.

Phép thử phân biệt, và nó rẻ: *chạy lại với vài giá trị khởi đầu khác nhau*. Nếu kết quả hội tụ về cùng một chỗ bất kể khởi đầu, đó là cách tốt. Nếu kết quả đi theo giá trị khởi đầu, đó là cách xấu — và đây chính là phép thử (1) ở A/05 khối *Cạm bẫy*, áp dụng cho một tình huống cụ thể.

Phép thử thứ hai, đắt hơn nhưng nhiều thông tin hơn: *nhìn hiệp phương sai của tham số*, không phải giá trị của nó. Một tham số quan sát được tốt có hiệp phương sai *giảm dần* theo thời gian; một tham số bị ghim bởi prior có hiệp phương sai *đứng yên* ở đúng giá trị prior.

14.4 Tự đánh giá chất lượng hiệu chuẩn

Đây là mục mà nghiên cứu ít nói và sản phẩm bắt buộc phải có.

A. Bốn đại lượng thay cho RMS

A/02 khối *Cạm bẫy* đã lập luận vì sao sai số chiều lại trung bình gần như vô dụng. Bốn đại lượng thay thế, theo thứ tự giá trị:

Một — *bản đồ phần dư dạng trường vectơ*. Chẩn đoán mô hình sai. Nếu phần dư có hoa văn, không lượng dữ liệu nào cứu được.

Hai — *hiệp phương sai của tham số, từ Λ* . Đây là đại lượng đúng để trả lời “tôi biết tham số này chính xác tới đâu”. Quan trọng: nhìn cả *tương quan* chứ không chỉ đường chéo. Tương quan cao giữa f và k_1 nghĩa là hai tham số không tách được từ dữ liệu này, và bộ số thu được nằm tùy tiện dọc một thung lũng (A/02 mục 4B).

Ba — *độ phủ của dữ liệu*. Các góc bằng có tới sát bốn góc ảnh không? Có đủ đa dạng độ sâu không? Đây là kiểm *đầu vào* chứ không phải kiểm *đầu ra*, và nó rẻ hơn nhiều vì nó chạy được *trước* khi tối ưu — một quy trình dây chuyền tốt sẽ từ chối một bộ dữ liệu kém ngay lúc thu, không phải sau khi chạy xong (D/21).

Bốn — *kiểm chéo*. Hiệu chuẩn trên một nửa dữ liệu, đo phần dư trên nửa kia. Bắt được overfitting mà ba cái trên bỏ sót.

B. Phát hiện trôi tại hiện trường

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và là bài toán khó nhất trong mục này: tại hiện trường không có target, không có chân trị, và thiết bị đang phục vụ khách hàng.

Ba đại lượng đo được, xếp theo độ nhảy.

Một — phần dư có cấu trúc theo thời gian. Nếu hiệu chuẩn trôi, phần dư chiếu sẽ phát triển một hoa văn không gian giống như ở A/02 mục 4A — chỉ là bây giờ trên các đặc trưng tự nhiên thay vì trên góc bằng. Cách theo dõi rẻ: tích lũy phần dư trung bình theo từng ô lưới trên mặt phẳng ảnh, trên một cửa sổ dài, và cảnh báo khi độ lệch giữa các ô vượt ngưỡng. Rẻ vì chỉ cần vài chục con số.

Hai — tham số hiệu chuẩn trực tuyến tự nó. Nếu bạn đang chạy hiệu chuẩn trực tuyến có cổng, thì giá trị của tham số là một tín hiệu. Theo dõi nó theo thời gian; một xu hướng đơn điệu (không phải dao động) là dấu hiệu trôi vật lý. Điều kiện: cổng phải hoạt động, nếu không bạn đang theo dõi nhiễu.

Ba — tương quan với nhiệt độ. Nếu có cảm biến nhiệt — và thiết bị nhúng thường có — thì tương quan giữa tham số ước lượng và nhiệt độ vỏ máy là bằng chứng mạnh cho trôi nhiệt. Nó cũng cho một khả năng đáng giá: bù trước bằng một mô hình tham số theo nhiệt độ, thay vì chỉ phát hiện.

CẠM BÃY

Một hệ thống hiệu chuẩn trực tuyến không có cổng observability tệ hơn không có hiệu chuẩn trực tuyến.

Lý do: khi tham số không quan sát được, nó không đứng yên mà *hấp thụ sai số từ các nguồn khác*. Bộ giải thấy một hướng phẳng và dùng nó để giảm phần dư gây ra bởi những nguyên nhân hoàn toàn khác — một vật động chưa loại, một mô hình méo sai, một sai lệch thời gian chưa bù.

Kết quả là tình huống tệ nhất: tham số nhận giá trị sai, *phần dư trông đẹp*, và sai số thật bị giấu vào một biến không ai kiểm. Hệ thống trở nên khó chẩn đoán hơn so với khi tham số bị fix, vì giờ bạn có thêm một nguồn nghi ngờ.

Ba yêu cầu tối thiểu cho một hệ hiệu chuẩn trực tuyến dùng được. *Một*: có cổng, ở bất kỳ mức tinh vi nào trong ba mức ở mục 3B. *Hai*: **ghi lại** khi nào tham số được cập nhật và khi nào bị đóng băng — không có nhật ký này thì không debug được. *Ba*: có giới hạn cứng — nếu tham số đi quá xa giá trị hiệu chuẩn ban đầu một lượng không hợp lý về mặt vật lý, đó là lỗi, không phải một ước lượng; hãy báo động thay vì chấp nhận.

14.5 Suy biến đặc thù của hiệu chuẩn

Ngoài các trường hợp ở A/05, hiệu chuẩn có vài suy biến riêng đáng biết vì chúng hay gặp trong thực tế.

Chuyển động phẳng. Một robot mặt đất di chuyển trên sàn phẳng: mọi tư thế nằm trong một mặt phẳng và mọi phép xoay quanh trục thẳng đứng. Hậu quả nghiêm trọng hơn vẻ ngoài: thành phần thẳng đứng của phép biến đổi ngoại không quan sát được, và roll với pitch của nó cũng vậy. Đây là tình huống mặc định của AGV và robot dịch vụ, tức phần lớn robot thương mại — nên nó không phải một trường hợp biên mà là trường hợp thường gặp nhất. Cách xử lý: hoặc chấp nhận và fix các thành phần ấy từ thiết kế cơ khí, hoặc tạo một quy trình hiệu chuẩn riêng có nâng/nghiêng thiết bị.

Xoay thuần. Không có tịnh tiến nên không có parallax, nên cấu trúc không quan sát được (A/01); và phần tịnh tiến của ngoại thì có quan sát được qua $\omega \times \mathbf{p}$. Đây là một trường hợp thú vị vì nó ngược với trực giác thông thường: xoay thuần tệ cho SLAM nhưng *tốt* cho một phần của hiệu chuẩn ngoại.

Target phẳng và lưỡng nghĩa tư thế. Khi bảng hiệu chuẩn ở gần song song với mặt phẳng ảnh và chiếm phần nhỏ khung hình, bài toán PnP có hai nghiệm gần nhau. Đây là chủ đề của `.. /marker/`

chương 4, và trong hiệu chuẩn nó biểu hiện thành các tư thế bảng bị lật, gây phần dư lớn ở vài ảnh. Cách xử lý: nghiêng bảng đủ nhiều, và dùng bộ giải PnP trả về cả hai nghiệm rồi chọn theo phần dư.

Mọi đặc trưng ở cùng độ sâu. Làm ngoại stereo không quan sát được theo một hướng, như mục 2C đã nêu. Trong thực tế: hiệu chuẩn stereo bằng cách nhìn vào một bức tường phẳng ở một khoảng cách cố định là một sai lầm phổ biến.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Kalibr [21]	Hiệu chuẩn camera-IMU và đa camera bằng batch MAP với spline liên tục thời gian	Hiện thân rõ nhất của luận đề chương này; đọc tài liệu quy trình thu dữ liệu và đối chiếu với Bảng kích thích ở A/05
OpenVINS [82]	Hiệu chuẩn trực tuyến nội, ngoại và t_d trong MSCKF	Mục 2D và 3B trong một hệ chạy thật; có cờ bật tắt từng nhóm tham số
COLMAP [11]	Tham số nội là biến trong BA; nhiều mô hình camera	Mục 2B — tự hiệu chuẩn ở dạng thuần túy nhất
Basalt [23]	Hiệu chuẩn bằng spline $SE(3)$; double-sphere	Chất lượng kỹ thuật cao; cách xử lý thời gian liên tục cho rolling shutter
[81]	Chỉ cập nhật các hướng quan sát được, dựa trên phân tích phổ	Nguồn của ý tưởng cổng ở mục 3B

MINI PROJECT SLAM · Đi từ hiệu chuẩn tới SLAM bằng cách thả dần biến, trên cùng một bộ mã · 5 ngày

Mục tiêu	đây là bài tập làm sáng tỏ luận đề trung tâm của cả cây rõ nhất. Bạn sẽ viết <i>một</i> bộ giải và dùng nó cho bốn bài toán khác nhau, chỉ bằng cách đổi danh sách biến.
Dữ liệu	một chuỗi hiệu chuẩn với target (tự quay, hoặc từ Kalibr), và một chuỗi EuRoC có chân trị.
Các bước	dựng bài toán MAP bằng Ceres hoặc GTSAM với một hàm phần dư chiếu duy nhất dùng chung; rồi chạy bốn cấu hình: (1) <i>hiệu chuẩn</i> : fix cấu trúc (toạ độ góc bảng), thả nội và tư thế; (2) <i>SfM</i> : thả cấu trúc, fix nội; (3) <i>tự hiệu chuẩn</i> : thả cả hai; (4) <i>tất cả</i> : thêm ngoại và t_d nếu có IMU. Với mỗi cấu hình, in ra hiệp phương sai của mọi tham số hiệu chuẩn và phổ giá trị riêng của Λ ; rồi chạy mỗi cấu hình trên <i>ba</i> đoạn quỹ đạo khác nhau — một đa dạng, một chỉ tịnh tiến, một chỉ xoay quanh một trục.
Xong khi	bạn có một bảng bốn cấu hình nhân ba quỹ đạo, mỗi ô ghi: số chiều không gian rỗng, hiệp phương sai của các tham số, và giá trị hội tụ. Bạn phải quan sát được ba điều: (a) cùng một bộ mã chạy cả bốn bài toán — luận đề trung tâm được chứng minh bằng tay; (b) thả thêm biến làm hiệp phương sai tăng và số chiều không gian rỗng có thể tăng; (c) trên quỹ đạo chỉ xoay quanh một trục, thành phần của $\mathbf{p}$ song song trục ấy có hiệp phương sai lớn hơn hẳn hai thành phần kia — xác nhận mục 2C bằng số của chính bạn.
Nối với	D/21-tu-prototype-len-mass-production biến quy trình này thành một quy trình dây chuyền có chỉ tiêu thời gian và chỉ tiêu chất lượng; D/22-huong-nghien-cuu-con-trong lấy hiệu chuẩn có ý thức observability làm hướng số một.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Target cung cấp *cấu trúc đã biết* — toạ độ 3D của các góc, chính xác vì ta in ra nó. Điều đó làm bài toán dễ hơn không phải vì độ chính xác của bảng mà vì nó *loại bỏ một nhóm biến*: với M góc, bài toán bớt $3M$ ẩn, và quan trọng hơn, kích thước bảng cho đơn vị mét nên gauge tỉ lệ biến

mất. Nói đúng nhất: *target là một cách giảm phương sai bằng cách fix cấu trúc*. Khi không có target, phải thay cái fix ấy bằng *chuyển động* — chuyển động đa dạng làm nhiều ràng buộc căng theo nhiều hướng và thay thế được. Đó là toàn bộ khác biệt giữa lab và robot đang chạy.

- A2. Vì mô tả ấy đúng về *cơ chế* và thiếu về *điều kiện*. Công thức đầy đủ: hiệu chuẩn trực tuyến = mở rộng trạng thái + đủ kích thích + công để biết khi nào không đủ. Trên chuỗi có xoay quanh nhiều trục, phần tịnh tiến của ngoại được kích thích qua số hạng $\omega \times \mathbf{p}$ và nó hội tụ. Trên chuỗi tịnh tiến thuần, số hạng đó bằng không nên $\mathbf{p}$ trôi tự do — và tệ hơn, nó hấp thụ sai số từ các nguồn khác. Chuỗi mà nó “đứng im” thường không phải chuỗi tốt mà là chuỗi mà bạn đã vô tình ghim nó bằng một prior mạnh hoặc bằng damping.
- A3. Thành phần của $\mathbf{p}$ song song với trục quay không xác định được. Vì số hạng duy nhất trong hệ phương trình chứa $\mathbf{p}$ là $\omega \times \mathbf{p}$, và tích có hướng *triệt tiêu* thành phần song song với ω : thay $\mathbf{p}$ bằng $\mathbf{p} + \lambda\omega$ cho đúng cùng một tích có hướng, nên cùng một phép đo. Đó là thủ phạm. Cần xoay quanh ít nhất hai trục độc lập để cả ba thành phần quan sát được. Đây là ví dụ sạch nhất trong cây về việc một lời khuyên thực hành — “xoay quanh cả ba trục” — là hệ quả trực tiếp của một tính chất đại số.
- A4. *Cách tốt*: chuyển động thật sự kích thích mạnh, ma trận thông tin xác định dương tốt, tham số hội tụ về giá trị đúng. *Cách xấu*: tham số không quan sát được nhưng bị ghim bởi một prior mạnh, damping LM, hoặc một khởi tạo được tin quá mức — nó “ổn định” vì nó không đi đâu cả, không phải vì dữ liệu xác nhận. Phép thử rẻ: chạy lại với vài giá trị khởi đầu khác nhau; hội tụ về cùng chỗ bất kể khởi đầu là cách tốt, đi theo khởi đầu là cách xấu. Phép thử thứ hai nhiều thông tin hơn: nhìn *hiệp phương sai* chứ không phải giá trị — tham số quan sát được có hiệp phương sai giảm dần, tham số bị ghim có hiệp phương sai đứng yên ở đúng giá trị prior.
- A5. Ba đại lượng. (1) *Phần dư có cấu trúc theo thời gian*: tích lũy phần dư trung bình theo từng ô lưới trên mặt phẳng ảnh trên một cửa sổ dài, cảnh báo khi độ lệch giữa các ô vượt ngưỡng — rẻ vì chỉ vài chục con số. (2) *Chính giá trị tham số hiệu chuẩn trực tuyến*: một xu hướng đơn điệu (không phải dao động) là dấu hiệu trôi vật lý; điều kiện là công phải hoạt động, nếu không bạn đang theo dõi nhiễu. (3) *Tương quan với nhiệt độ vỏ máy*: bằng chứng mạnh cho trôi nhiệt, và nó còn cho khả năng bù trước bằng một mô hình tham số theo nhiệt độ thay vì chỉ phát hiện.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn có một cái thước và muốn đo một căn phòng. Nhưng cái thước của bạn có thể bị co giãn, và bạn không biết chắc nó dài bao nhiêu.

Có hai cách xử lý.

Cách một: mượn một vật mà bạn *biết chắc* kích thước — một tờ giấy A4 chẳng hạn. Đo nó bằng cái thước của mình, so với kích thước thật, và suy ra thước của mình sai bao nhiêu. Đó là hiệu chuẩn bằng target: bạn mượn một thứ đã biết để đo một thứ chưa biết.

Cách hai: không có tờ giấy nào. Bạn chỉ có căn phòng và cái thước không đáng tin. Nghe như bó tay — nhưng không hẳn. Nếu bạn đo cùng một bức tường *theo nhiều cách khác nhau*, từ nhiều chỗ, theo nhiều hướng, thì các kết quả phải khớp với nhau. Và chính việc chúng *không* khớp cho bạn biết thước sai ở đâu.

Đó là toàn bộ chương này. Có vật chuẩn thì dễ. Không có vật chuẩn thì vẫn làm được, nhưng phải *đo nhiều kiểu khác nhau* — và “nhiều kiểu khác nhau” là chỗ mọi thứ có thể hỏng.

Vì nếu bạn đo mười lần mà lần nào cũng theo đúng một cách, mười kết quả sẽ khớp nhau hoàn hảo — và chúng cùng sai một kiểu. Bạn sẽ rất tự tin và rất nhầm.

Đây là chỗ quan trọng nhất cần hiểu, và nó áp dụng cho mọi thứ chứ không riêng camera: *sự nhất quán giữa nhiều phép đo giống nhau không chứng minh gì cả*. Chỉ sự nhất quán giữa các phép đo *thật sự khác nhau* mới chứng minh được.

Và có một hệ quả thực tế đáng nhớ. Nếu một tham số nào đó không được kiểm bởi cách bạn đang đo, thì đừng để máy tự do chỉnh nó. Nó sẽ chỉnh — và nó sẽ chỉnh sai, đồng thời dùng cái sai ấy để che giấu những lỗi khác. Một cái núm bị vặn bừa còn tệ hơn một cái núm bị khoá.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *biết chuyển động nào kiểm được tham số nào*. Không có trực giác chung cho việc này — với mỗi cấu hình cảm biến mới, phải ngồi xuống viết ra phương trình và xem tham số ấy xuất hiện ở đâu. Với camera-IMU thì nó xuất hiện trong một tích có hướng, và từ đó suy ra phải xoay quanh hai trục. Với một cấu hình khác, câu trả lời sẽ khác.

Một câu: Hiệu chuẩn và SLAM là một bài toán — khác nhau ở chỗ bạn hàn cứng cái gì, và chuyển động là thứ thay thế cho cái hàn.

14.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *hiệu chuẩn và SLAM là cùng một bài toán MAP, khác ở danh sách biến* — là một phát biểu cấu trúc, và mini project chính là phép chứng minh: nếu một bộ mã chạy được cả bốn cấu hình thì mệnh đề đúng cho phạm vi đó. Nó bị bác bỏ nếu có một bài toán hiệu chuẩn không viết được thành một đồ thị nhân tử với cùng loại nhân tử — và có một trường hợp như vậy đáng nêu: hiệu chuẩn quang trắc (A/02 mục 5B) có phần dư dạng khác (cường độ thay vì tọa độ) và một số thành phần (đường cong đáp ứng) được biểu diễn phi tham số. Nó vẫn là MAP, nhưng nó không dùng chung nhân tử với SLAM indirect, nên phát biểu cần được giới hạn: *đúng cho các bài toán hình học; hiệu chuẩn quang trắc là một họ riêng*.

Mệnh đề thứ hai — *thành phần của $\mathbf{p}$ song song trục quay không quan sát được khi chỉ xoay quanh một trục* — suy trực tiếp từ tính chất tích có hướng (cửa a) và kiểm được bằng hiệp phương sai trong mini project (cửa b). Nó bị bác bỏ nếu có một số hạng khác trong hệ phương trình cũng chứa $\mathbf{p}$ — và thực tế có: với gia tốc, số hạng $\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{p})$ và $\dot{\boldsymbol{\omega}} \times \mathbf{p}$ cũng xuất hiện. Số hạng thứ hai đưa vào một cơ chế quan sát khác nếu tốc độ góc *thay đổi*. Nên phát biểu chặt hơn là: *với tốc độ góc hằng quanh một trục, thành phần song song không quan sát được; với tốc độ góc biến thiên, nó có thể quan sát được yếu*. Tôi chưa kiểm điều này bằng số và nó là một mục trong 99-chua-biet.md.

Mệnh đề thứ ba — *hiệu chuẩn trực tuyến không công tẻ hơn không hiệu chuẩn* — là một phán đoán về đánh đổi. Nó bị bác bỏ nếu một thí nghiệm cho thấy hiệu chuẩn trực tuyến không công vẫn cải thiện độ chính xác trung bình so với tham số fix, trên một tập chuỗi đa dạng. Điều đó *có thể* xảy ra nếu phần lớn chuỗi có kích thích tốt và tham số ban đầu khá sai. Nếu vậy, kết luận đúng không phải “công không cần thiết” mà là “lợi ích trung bình che mất rủi ro đuôi” — và với sản phẩm thì đuôi mới là thứ quan trọng (C/18).

14.7 Kết nối

Chương này là chỗ cây gập lại. Nối lên trên gần như mọi chương của Phần A: A/02-mô-hình-camera cho mô hình và cho kỹ năng chẩn đoán ở mục 4A; A/04-uoc-luong-map-va-bundle-adjustment cho khung MAP mà chương này chỉ đổi danh sách biến; A/05-observability-gauge-va-suy-bien cho toàn bộ phần điều kiện — đây là chương mà Bảng kích thích ở đó trả công hoàn toàn; A/06-bat-dinh-va-lan-truyen-sai-so cho hiệp phương sai tham số, đại lượng nên in ra thay cho RMS.

Nối ngang trong Phần C. C/13-quan-ly-ban-do chia sẻ một câu hỏi với chương này: cái gì được coi là cố định và cái gì được phép đổi theo thời gian — ở đó là bản đồ, ở đây là tham số. C/15-hop-nhat-da-cam-bien là chương mà hiệu chuẩn ngoại trở thành điều kiện tiên quyết: không thể ghép hai cảm biến mà không biết chúng đặt ở đâu so với nhau. C/18-danh-gia-va-phuong-phap-luan cho công cụ để so sánh hai quy trình hiệu chuẩn một cách có kỷ luật.

Nối xuống dưới. D/21-tu-prototype-len-mass-production là chỗ chương này gập thực tế khắc nghiệt nhất: thiết kế đồ gá hiệu chuẩn tự động dưới sáu mươi giây mỗi máy, quyết định hiệu chuẩn từng máy hay dùng tham số chung cho cả lô, phân tích độ nhạy trước khi chốt cơ khí, và thống kê SPC trên tham số qua từng lô hàng. D/22-huong-nghien-cuu-con-trong lấy hiệu

chuẩn trực tuyến có ý thức observability làm hướng số một, và lý do nằm trọn trong mục 3 của chương này: hầu hết hệ thống hoặc hiệu chuẩn mù quá hoặc không hiệu chuẩn, và việc giám sát observability theo thời gian thực rồi quyết định cập nhật biến nào vẫn còn rất thô sơ.

Nổi ngang sang cây khác: . . /IMU_Fusion/ chương 6 bàn hiệu chuẩn quán tính — đặc trưng hoá nhiễu, phương sai Allan, độ lệch — và chương 5 ở đó cho phần observability cho trường hợp quán tính. . . /marker/ chương 12 bàn hiệu chuẩn cho bài toán docking, nơi yêu cầu là milimet nên mọi thứ trong chương này phải làm chặt hơn một bậc.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu chính xác điều mà calibration target cung cấp, và nói vì sao “vì nó chính xác” là câu trả lời sai.
2. Viết công thức đầy đủ cho hiệu chuẩn trực tuyến, gồm cả về điều kiện, và giải thích vì sao thiếu về đó là nguy hiểm.
3. Tự dẫn lại vì sao thành phần của $\mathbf{p}_{\text{cam}}^{\text{imu}}$ song song với trục quay không quan sát được khi chỉ xoay quanh một trục.
4. Hiệu chuẩn trực tuyến của bạn hội tụ nhanh và ổn định. Nêu hai cách giải thích và hai phép thử phân biệt.
5. Nêu bốn đại lượng nên nhìn thay cho RMS khi đánh giá hiệu chuẩn, và nói cái nào chạy được trước khi tối ưu.
6. Nêu ba đại lượng phát hiện trôi hiệu chuẩn tại hiện trường, nơi không có target và không có chân trị.
7. Robot mặt đất đi trên sàn phẳng. Thành phần nào của hiệu chuẩn ngoại không quan sát được, và vì sao đây không phải một trường hợp biên mà là trường hợp thường gặp nhất?

Hợp nhất đa cảm biến (Multi-Sensor Fusion)

PHỤ THUỘC [A/04-uoc-luong-map-va-bundle-adjustment](#) (đồ thị nhân tử — thêm cảm biến là thêm loại nhân tử); [A/06-bat-dinh-va-lan-truyen-sai-so](#) (hiệp phương sai đúng là điều kiện bắt buộc chứ không phải điều tốt nên có); [C/14-hieu-chuan-nhu-mot-nhanh-cua-slam](#) (không ghép được hai cảm biến mà không biết chúng đặt ở đâu so với nhau). Chi tiết về IMU nằm ở `../IMU_Fusion/`, về marker ở `../marker/`.

TÓM TẮT

Chương này ngắn có chủ ý. Phần lớn nội dung chi tiết về từng loại cảm biến đã nằm ở các cây khác, và lặp lại nó ở đây sẽ vi phạm quy tắc một nguồn duy nhất của kho. Cái chương này đóng góp là *nguyên tắc chung* — một nguyên tắc gọn tới mức đáng ngạc nhiên — cộng một bảng tra về việc mỗi cảm biến cho gì và đòi gì. Nguyên tắc: *thêm một nguồn thông tin là thêm một loại nhân tử vào đồ thị, với hiệp phương sai đúng và cổng loại outlier riêng cho từng nguồn*. Ba vế của câu đó đều quan trọng và vế thứ ba hay bị bỏ. Nội dung gồm: nguyên tắc và ba điều kiện của nó; bảng tra sáu nhóm cảm biến với cái chúng cho, điều kiện để cái đó quan sát được, và chế độ hỏng riêng; hai trường hợp đáng nói kỹ hơn vì chúng hay bị làm sai — odometry bánh xe và GNSS; và cuối cùng là một cách nhìn về việc chọn cấu hình cảm biến như một bài toán phá gauge. Nếu chỉ nhớ một điều: *giá trị lớn nhất của một cảm biến phụ không phải độ chính xác cao hơn khi mọi thứ chạy tốt, mà là một chỗ đứng để phục hồi khi thị giác thất bại*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **nhân tử (factor)** — một số hạng của hàm mục tiêu, nối các biến mà phần dư của nó phụ thuộc vào ([A/04](#)). Thêm một cảm biến nghĩa là định nghĩa một loại nhân tử mới.
- **ràng buộc nonholonomic** — ràng buộc lên *vận tốc* chứ không lên vị trí. Với robot bánh xe không trượt ngang, vận tốc theo phương ngang thân xe bằng không — một ràng buộc miễn phí và rất hữu ích.
- **odometry bánh xe** — ước lượng chuyển động từ số vòng quay bánh xe. Cho tỉ lệ metric, rẻ, tần số cao; hỏng khi trượt.
- **de-skew** — hiệu chỉnh méo của một lượt quét LiDAR do cảm biến di chuyển trong lúc quét. Cùng bản chất với vấn đề rolling shutter ở [A/02](#).
- **hệ toàn cục so với hệ cục bộ** — SLAM làm việc trong một hệ tùy ý do nó tự chọn (gauge); GNSS cho toạ độ trong một hệ toàn cục cố định. Ghép hai hệ đòi một phép biến đổi có bốn bậc tự do phải ước lượng.
- **cổng loại outlier (outlier gating)** — cơ chế từ chối một phép đo trước khi đưa vào bộ giải. Mỗi nguồn cần cổng riêng vì phân bố outlier của chúng khác nhau.

CÂU HỎI MỞ ĐẦU

- Q1. Thêm một cảm biến vào đồ thị là thêm một loại nhân tử — nghe như một việc cơ học. Nếu ba điều kiện phải thoả, và nói điều kiện nào hay bị bỏ nhất.
- Q2. Một GNSS rẻ tiền có sai số vài mét. Thị giác của bạn chính xác tới centimet trong ngắn hạn. Nêu lý do vẫn đáng thêm GNSS, và nói lý do đó không phải “để chính xác hơn”.
- Q3. Odometry bánh xe cho tỉ lệ metric miễn phí, chính xác, tần số cao. Vì sao nó vẫn không thay thế được IMU trong hầu hết hệ thống, và chế độ hỏng của nó khác IMU thế nào?

- Q4.** Bạn dùng chung một ngưỡng Mahalanobis cho mọi loại nhân tử — cách làm gọn gàng và thống nhất. Nêu vì sao đó là một sai lầm, với một ví dụ cụ thể.
- Q5.** Chọn cấu hình cảm biến cho một robot mặt đất. Nêu cách đặt câu hỏi này sao cho nó trở thành một bài toán có tiêu chí thay vì một cuộc tranh luận về sở thích.

15.1 Đặt vấn đề

Thị giác một mình có ba điểm yếu cố hữu mà không thuật toán nào chữa được, vì chúng là tính chất của phép đo chứ không của cách xử lý.

Một — không có tỉ lệ với mono (A/05 mục 3). Hai — hỏng khi không có vân: hành lang trắng, sương mù, bóng tối, camera bị che. Ba — tần số thấp và độ trễ cao: ba mươi khung mỗi giây với vài chục mili giây xử lý là quá chậm cho một vòng điều khiển bay.

Ba điểm yếu ấy được chữa bởi ba loại cảm biến khác nhau, và đó là lý do gần như mọi hệ thống thật đều có nhiều hơn một cảm biến.

Bài toán gốc: *ghép chúng lại thế nào cho đúng?* Câu hỏi này nghe mở, và lịch sử của lĩnh vực có nhiều câu trả lời phức tạp — các sơ đồ lọc phân tầng, các cơ chế trọng số thủ công, các máy trạng thái chuyển đổi giữa các nguồn. Phần lớn chúng đã lỗi thời.

Câu trả lời hiện đại gọn tới mức đáng ngạc nhiên, và nó là hệ quả trực tiếp của [A/04: đồ thị nhân tử không quan tâm nhân tử đến từ đâu](#). Một nhân tử là một hàm phần dư cộng một hiệp phương sai. Bộ giải nhìn thấy một tập nhân tử và một tập biến; nó không biết và không cần biết nhân tử nào đến từ camera, nhân tử nào từ IMU, nhân tử nào từ GNSS.

Hệ quả là việc thêm một cảm biến trở thành một việc *cục bộ*: định nghĩa biến mới nếu cần, định nghĩa phần dư, cung cấp Jacobian và hiệp phương sai. Không có gì trong bộ giải thay đổi. Đây là lý do kiến trúc đồ thị nhân tử thắng, và nó là một lợi thế lớn so với bộ lọc, nơi thêm một cảm biến nghĩa là sửa vector trạng thái và mọi ma trận liên quan.

Nhưng — và đây là toàn bộ nội dung còn lại của chương — sự gọn gàng ấy che mất ba điều kiện. Bỏ qua chúng thì việc thêm cảm biến làm hệ thống *tệ đi*, và tệ theo cách khó chẩn đoán.

TRỰC GIÁC

Hình dung một cuộc họp mà mọi người cùng ước lượng một con số — giá của một ngôi nhà chẳng hạn.

Mỗi người đưa ra một con số kèm một mức tự tin: “khoảng ba tỉ, tôi khá chắc” hoặc “khoảng bốn tỉ, nhưng tôi chỉ đoán”. Cách hợp lý để tổng hợp là trung bình có trọng số, với trọng số theo mức tự tin.

Đó chính xác là điều đồ thị nhân tử làm, và nó giải thích cả ba điều kiện.

Điều kiện một — mức tự tin phải trung thực. Nếu một người nói “tôi rất chắc” trong khi thực ra chỉ đoán, ý kiến của người đó sẽ lấn át cả phòng. Và điều tệ nhất: cuộc họp sẽ kết luận rằng *những người khác đang sai*, chứ không phải người kia đang nói quá.

Điều kiện hai — phải biết mọi người đang nói về cùng một thứ. Nếu một người nói giá tính bằng đồng và người khác tính bằng đô la, việc trung bình hoá là vô nghĩa. Trong cảm biến, “cùng một thứ” nghĩa là cùng hệ quy chiếu và cùng mốc thời gian — tức hiệu chuẩn ngoại và đồng bộ thời gian.

Điều kiện ba — phải loại người nói nhảm, và mỗi người nói nhảm theo kiểu khác nhau. Một người thỉnh thoảng nhầm một chữ số; một người khác đôi khi nói về một ngôi nhà hoàn toàn

khác. Hai kiểu sai ấy cần hai cách phát hiện, và một quy tắc chung “loại ai lệch quá xa” sẽ bắt được kiểu thứ hai mà bỏ sót kiểu thứ nhất.

Và một điều nữa, quan trọng nhất cho thực tế. Giá trị lớn nhất của việc có nhiều người trong phòng không phải là trung bình chính xác hơn khi mọi người đều tỉnh táo. Đó là khi *một người ngất xỉu* — cuộc họp vẫn tiếp tục được.

15.2 Nguyên tắc và ba điều kiện

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất.

Nguyên tắc: *thêm một nguồn thông tin là thêm một loại nhân tử vào đồ thị*. Ba điều kiện:

A. Hiệp phương sai phải trung thực

Trọng số $\Omega = \Sigma^{-1}$ quyết định nguồn này được tin bao nhiêu so với mọi nguồn khác. Khai Σ quá nhỏ nghĩa là ra lệnh cho bộ giải tin nguồn này hơn tất cả.

Hậu quả khi sai, và nó đã được nêu ở B/08 khối *Cạm bẫy*: nếu nguồn ấy có sai lệch có hệ thống, sai lệch đó chảy vào toàn bộ nghiệm, kéo theo cả những phần vốn được ước lượng tốt. Và *không có gì cảnh báo* — phần dư của các nguồn khác tăng lên, nhưng bộ giải diễn giải điều đó là “các nguồn kia kém chính xác”.

Quy tắc thực hành: trước khi thêm một loại nhân tử, trả lời được *hiệp phương sai của nó từ đâu ra*. Ba câu trả lời hợp lệ: từ datasheet của cảm biến (đã kiểm bằng đo đặc); từ một phép đo bạn tự làm (như đo σ subpixel ở B/07 mục 4B); hoặc từ một mô hình có cơ sở vật lý. Câu trả lời *không* hợp lệ: “tôi chỉnh cho tới khi nó chạy”.

B. Hệ quy chiếu và thời gian phải đúng

Hai cảm biến đặt ở hai chỗ khác nhau trên cùng thiết bị đo hai thứ khác nhau, và quan hệ giữa chúng là phép biến đổi ngoại (C/14). Sai ngoại thì mọi phần dư mang một sai lệch có hệ thống.

Sai lệch thời gian còn nguy hiểm hơn vì nó *phụ thuộc chuyển động*: một t_d sai không gây hại gì khi thiết bị đứng yên và gây hại lớn khi nó chuyển động nhanh. Hệ quả chẩn đoán đáng nhớ: nếu hệ thống của bạn chính xác khi đi chậm và tệ khi đi nhanh, nghi ngờ t_d trước khi nghi ngờ thuật toán.

Cả hai đều nên được *thả ra làm biến* theo C/14, với cổng observability.

C. Cổng loại outlier riêng cho từng nguồn

Đây là điều kiện hay bị bỏ nhất, và là câu trả lời cho câu hỏi mở đầu thứ tư.

Mỗi nguồn có *phân bố outlier riêng*, và một cổng chung không bắt được cả hai.

Ví dụ cụ thể. Outlier thị giác là các khớp sai: chúng rải rác, độc lập, và thường có phần dư rất lớn — một cổng Mahalanobis đơn giản bắt tốt. Outlier odometry bánh xe là trượt: nó *kéo dài* (cả một đoạn đường), *tương quan* (mọi phép đo trong đoạn đó đều sai cùng chiều), và phần dư của mỗi phép đo riêng lẻ có thể *nhỏ*. Một cổng Mahalanobis trên từng phép đo sẽ không bắt được nó, vì không phép đo nào lệch đủ xa — nhưng tổng của chúng thì lệch rất xa.

Phát hiện trượt cần một cơ chế khác: so tích lũy odometry với tích lũy từ một nguồn độc lập (IMU hoặc thị giác) trên một cửa sổ, và cảnh báo khi chúng phân kỳ. Đây là một công ở *mức chuỗi* chứ không ở mức phép đo, và nó không thể suy ra từ công thị giác.

Bài học tổng quát: *cổng phải được thiết kế theo cơ chế hỏng của từng nguồn*, không theo một công thức thống nhất. Một hệ thống có n loại cảm biến cần n cơ chế phát hiện, và đó là chi phí kỹ thuật thật của việc hợp nhất.

15.3 Bảng tra: mỗi cảm biến cho gì, đòi gì

Bảng 15.1. Sáu nhóm cảm biến. Cột “cho gì” nên đọc theo ngôn ngữ gauge của A/05; cột “chế độ hỏng riêng” là cột quyết định thiết kế cổng ở mục 2C.

Cảm biến	Cho gì	Điều kiện	Chế độ hỏng riêng
IMU	Tỉ lệ metric, hướng trọng lực (roll, pitch), chuyển động tần số cao	Gia tốc phải <i>thay đổi</i> để tỉ lệ quan sát được	Trôi độ lệch; bão hoà khi va đập; nhiễu rung
Odometry bánh xe	Tỉ lệ metric, chuyển động phẳng, ràng buộc nonholonomic	Bánh không trượt; đường kính bánh đã hiệu chuẩn	Trượt — kéo dài, tương quan, phần dư từng phép đo nhỏ
Stereo	Tỉ lệ metric ngay từ khung đầu	Trong tầm hữu ích; đường cơ sở ổn định	Ngoài tầm thì thoái hoá về mono; trôi đường cơ sở theo nhiệt
LiDAR	Cấu trúc metric dày, không cần ánh sáng	Bề mặt phản xạ đủ; de-skew đúng	Hành lang dài (suy biến dọc trục); mưa, bụi, kính
GNSS	Vị trí <i>tuyệt đối</i> trong hệ toàn cục	Thấy trời; ít đa đường	Đa đường trong phố; mất tín hiệu trong nhà; sai lệch tương quan
Marker / UWB	Vị trí <i>tuyệt đối</i> trong hệ đã thiết lập	Trong tầm nhìn / tầm sóng; đã lắp đặt	Ngoài tầm; marker bị che, bẩn, tháo mất

Bảng này nên đọc cùng với Bảng ở 00-map: mỗi cảm biến trong cột “cho gì” thực chất là *phá một gauge*. IMU và odometry và stereo phá gauge tỉ lệ; IMU còn ghim roll với pitch; GNSS và marker phá cả gauge tịnh tiến lẫn yaw. Nhìn theo cách đó, việc chọn cấu hình cảm biến trở thành một câu hỏi có tiêu chí — đó là câu trả lời cho câu hỏi mở đầu thứ năm.

15.4 Hai trường hợp hay bị làm sai

A. Odometry bánh xe

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và odometry bánh xe đáng một mục riêng vì nó là cảm biến *bị đánh giá thấp nhất* cho robot mặt đất.

Cái nó cho, và ba thứ này đều đáng giá. *Tỉ lệ metric* — đường kính bánh đã biết cho một thước đo, giống đường cơ sở stereo. *Tần số cao và độ trễ thấp* — thường vài trăm Hz, tốt cho vòng điều khiển. *Ràng buộc nonholonomic* — và đây là phần miễn phí ít người khai thác: với robot bánh xe không trượt ngang, vận tốc theo phương ngang thân xe *bằng không*, và với robot chạy trên sàn

phẳng, vận tốc thẳng đứng cũng bằng không. Đó là hai ràng buộc không cần cảm biến nào cả — chỉ cần biết loại robot.

Vì sao nó vẫn không thay thế IMU? Vì chế độ hỏng của hai cảm biến *bù nhau* chứ không trùng nhau. IMU trôi nhưng không bao giờ “nói dối” — nó luôn đo đúng cái nó đo, chỉ là có độ lệch. Odometry không trôi khi mọi thứ bình thường, nhưng khi bánh trượt thì nó nói dối một cách tự tin: nó báo robot đã đi ba mét trong khi robot đứng yên quay bánh.

Hai kiểu sai ấy đòi hai cách xử lý ngược nhau. Với IMU, ta *mô hình hoá* độ lệch và ước lượng nó. Với odometry, ta phải *phát hiện và loại* — không có mô hình nào cho việc trượt. Và như mục 2C đã nói, việc phát hiện ấy cần một cổng ở mức chuỗi chứ không ở mức phép đo.

Kết luận thực dụng cho robot mặt đất: *dùng cả hai*. IMU cho hướng và cho chuyển động ngắn hạn khi bánh trượt; odometry cho tỉ lệ và cho chuyển động khi IMU trôi. Chúng kiểm chéo lẫn nhau, và chính việc kiểm chéo ấy là cơ chế phát hiện trượt.

B. GNSS và hai hệ quy chiếu

Đây là câu trả lời cho câu hỏi mở đầu thứ hai.

Vấn đề đầu tiên khi thêm GNSS không phải độ chính xác mà là *hệ quy chiếu*. SLAM làm việc trong một hệ tuỳ ý mà nó tự chọn khi khởi tạo — đó chính là gauge ở A/05. GNSS cho toạ độ trong một hệ toàn cục cố định. Hai hệ liên hệ qua một phép biến đổi *chưa biết* với bốn bậc tự do: ba tịnh tiến và một yaw (roll và pitch đã bị trọng lực ghim nếu có IMU).

Phép biến đổi ấy phải được *ước lượng*, và nó là một biến trong đồ thị như mọi biến khác. Điều kiện để nó quan sát được: quỹ đạo phải đủ dài và đủ cong — một đoạn thẳng không xác định được yaw của phép căn chỉnh.

Vì sao vẫn đáng thêm GNSS dù nó kém chính xác hơn thị giác? Lý do *không* phải độ chính xác mà là *bản chất sai số*. Sai số thị giác *tích lũy* — nó nhỏ ở ngắn hạn và lớn vô hạn ở dài hạn. Sai số GNSS *bị chặn* — nó vài mét ở mọi thời điểm, hôm nay cũng như một năm sau. Hai đặc tính ấy bù nhau hoàn hảo: thị giác cho độ chính xác cục bộ, GNSS cho việc không trôi.

Đây là một ví dụ của một nguyên tắc chung đáng nhớ: *khi ghép cảm biến, hãy tìm nguồn có đặc tính sai số bù nhau, không phải nguồn chính xác nhất*. Hai cảm biến cùng trôi theo cùng kiểu không giúp được nhau; một cảm biến kém chính xác nhưng không trôi lại vô giá.

Cảnh báo về cổng, đúng theo mục 2C: sai số GNSS trong phố có *đa đường* — tín hiệu phản xạ từ toà nhà — và nó gây sai lệch *tương quan theo thời gian*, có thể hàng chục mét và kéo dài nhiều giây. Một cổng dựa trên từng phép đo sẽ không bắt được, vì hiệp phương sai mà máy thu báo cũng lạc quan trong tình huống ấy. Cần một cổng so với dự đoán từ thị giác trên một cửa sổ.

CẦN NHỚ

Giá trị lớn nhất của một cảm biến phụ không phải độ chính xác cao hơn khi mọi thứ chạy tốt, mà là một chỗ đứng để phục hồi khi nguồn chính thất bại.

Đây là điều mà các bảng so sánh độ chính xác không nắm bắt được, và nó là lý do quan trọng nhất để hợp nhất cảm biến trong sản phẩm.

Ba biểu hiện cụ thể, đã xuất hiện rải rác trong cây. *Một*: khi thị giác mất bám, odometry và IMU giữ cho hệ thống có một ước lượng thô — đủ để tiếp tục vòng điều khiển, và đủ để thu hẹp không gian tìm kiếm khi relocalize (B/12 mục 2C). *Hai*: tiên nghiệm chuyển động từ cảm biến phụ cải thiện chính front-end thị giác, theo ba đường cùng lúc (B/07 mục 5C). *Ba*: khi hai nguồn bất đồng, đó là một tín hiệu chẩn đoán — và một hệ chỉ có một nguồn thì không bao giờ có tín hiệu ấy.

Hệ quả cho việc đánh giá: đo một hệ hợp nhất bằng ATE trung bình trên một chuỗi sạch sẽ bỏ sót phần lớn giá trị của nó. Phải đo trên các chuỗi có sự cố — che camera, tắt đèn, bánh trượt — và đo tỉ lệ sống sót chứ không chỉ độ chính xác. C/18 bàn cách thiết kế phép đo đó.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
VINS-Fusion [13]	Camera + IMU + GNSS tùy chọn; căn chỉnh hệ cục bộ với hệ toàn cục	Mục 4B trong một hệ thật; mã ngắn dễ đọc
GTSAM [27]	Thư viện nhân tử cho nhiều loại cảm biến; dễ thêm loại mới	Hiện thân của nguyên tắc ở mục 2 — thêm cảm biến là viết một lớp nhân tử
Kimera [63]	Camera + IMU + ngữ nghĩa trong một đồ thị	Cho thấy cả thông tin ngữ nghĩa cũng chỉ là một loại nhân tử
maplab [73]	Nhiều cảm biến, nhiều phiên, hiệu chuẩn ngoại trong cùng khung	Kết hợp chương này với C/13 và C/14
OpenVINS [42]	Camera + IMU với hiệu chuẩn ngoại và t_d trực tuyến	Mục 2B — thả ngoại và thời gian ra làm biến

MINI PROJECT SLAM · Thêm odometry bánh xe vào một hệ VIO, rồi đo giá trị thật của nó · 3 ngày

Mục tiêu	đo giá trị của một cảm biến phụ theo cách đúng — không phải bằng ATE trung bình mà bằng tỉ lệ sống sót khi có sự cố.
Dữ liệu	một robot mặt đất có bánh xe encoder, camera và IMU; tự thu ba loại chuỗi: chuỗi sạch, chuỗi có che camera vài giây, và chuỗi có bánh trượt (chạy lên một tấm thảm trơn hoặc nâng một bánh).
Các bước	thêm nhân tử odometry và ràng buộc nonholonomic vào một hệ VIO có sẵn; hiệu chuẩn đường kính bánh và ngoại bánh–camera theo C/14; cài một cổng phát hiện trượt ở mức chuỗi theo mục 2C — so tích lũy odometry với tích lũy IMU trên một cửa sổ; chạy bốn cấu hình (VIO thuần, VIO + odom không cổng, VIO + odom có cổng, VIO + odom + nonholonomic) trên cả ba loại chuỗi.
Xong khi	bạn có một bảng bốn cấu hình nhân ba chuỗi với hai cột: ATE và <i>có sống sót không</i> . Bạn phải quan sát được: (a) trên chuỗi sạch, thêm odometry cải thiện ATE ít — đúng như dự đoán, vì đó không phải giá trị chính của nó; (b) trên chuỗi che camera, chỉ các cấu hình có odometry sống sót; (c) trên chuỗi trượt, cấu hình <i>không cổng</i> tệ hơn cả VIO thuần — xác nhận mục 2C bằng dữ liệu của chính bạn, và đó là kết quả đáng giá nhất của bài này.
Nối với	C/18-danh-gia-va-phuong-phap-luan chuẩn hoá cách đo “tỉ lệ sống sót”; D/19-ben-vung-trong-the-gioi-thuc mở rộng danh mục sự cố cần thử.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Ba điều kiện: (1) *hiệp phương sai trung thực* — trọng số quyết định nguồn này được tin bao nhiêu so với mọi nguồn khác, và khai quá nhỏ làm sai lệch có hệ thống của nó chảy vào toàn bộ nghiệm mà không có gì cảnh báo; (2) *hệ quy chiếu và thời gian đúng* — ngoại và t_d , cả hai nên thả ra làm biến theo C/14; (3) *cổng loại outlier riêng cho từng nguồn*. Điều kiện (3) hay bị bỏ nhất, vì nó đòi hiểu cơ chế hỏng của từng cảm biến chứ không suy ra được từ một công thức chung.
- Vì *bản chất sai số bù nhau*, không phải vì độ chính xác. Sai số thị giác tích lũy — nhỏ ở ngắn hạn, lớn vô hạn ở dài hạn. Sai số GNSS bị chặn — vài mét ở mọi thời điểm, hôm nay cũng như một năm sau. Ghép lại: thị giác cho độ chính xác cục bộ, GNSS cho việc không trôi. Nguyên tắc chung đáng nhớ: khi ghép cảm biến, tìm nguồn có *đặc tính sai số bù nhau*, không phải nguồn

chính xác nhất — hai cảm biến cùng trôi theo cùng kiểu không giúp được nhau. Lưu ý: thêm GNSS đòi ước lượng một phép biến đổi bốn bậc tự do giữa hệ cục bộ và hệ toàn cục, và nó cần quỹ đạo đủ cong để quan sát được yaw.

- A3. Vì chế độ hỏng của hai cảm biến *bù nhau* chứ không trùng nhau. IMU trôi nhưng không bao giờ nói dối — nó luôn đo đúng cái nó đo, chỉ có độ lệch, nên ta *mô hình hoá* độ lệch và ước lượng nó. Odometry không trôi khi bình thường, nhưng khi bánh trượt thì nó nói dối một cách tự tin — báo đã đi ba mét trong khi robot đứng yên quay bánh — và không có mô hình nào cho việc trượt, nên phải *phát hiện và loại*. Hai kiểu sai đòi hai cách xử lý ngược nhau. Kết luận: dùng cả hai, và chính việc chúng kiểm chéo lẫn nhau là cơ chế phát hiện trượt.
- A4. Vì mỗi nguồn có *phân bố outlier riêng*. Ví dụ cụ thể: outlier thị giác là khớp sai — rải rác, độc lập, phần dư rất lớn — nên công Mahalanobis trên từng phép đo bất tốt. Outlier odometry là trượt — *kéo dài* cả một đoạn, *tương quan* (mọi phép đo sai cùng chiều), và phần dư của từng phép đo riêng lẻ có thể *nhỏ*. Công Mahalanobis không bắt được vì không phép đo nào lệch đủ xa, nhưng tổng của chúng thì lệch rất xa. Cần một công ở *mức chuỗi*: so tích lũy odometry với tích lũy từ nguồn độc lập trên một cửa sổ. Bài học: công phải thiết kế theo cơ chế hỏng của từng nguồn; n loại cảm biến cần n cơ chế phát hiện.
- A5. Đặt nó thành một bài toán *phá gauge*: liệt kê các bậc tự do không quan sát được của cấu hình thuần thị giác (bây với mono, theo A/05), rồi hỏi mỗi cảm biến phá được bậc nào và với điều kiện gì — Bảng 15.1 là bảng trả lời. Thêm hai tiêu chí nữa để bài toán đầy đủ: (a) *đặc tính sai số có bù nhau không* (tích lũy so với bị chặn); (b) *chế độ hỏng có độc lập không* — hai cảm biến cùng chết khi mất điện hoặc cùng chết trong sương mù thì không cho khả năng phục hồi. Với robot mặt đất, kết quả thường là stereo + IMU + odometry bánh xe, và D/21 lập luận vì sao đó là điểm ngọt.

NÓI LẠI THẬT ĐƠN GIẢN

Hình dung một cuộc họp để đoán giá một ngôi nhà. Mỗi người đưa ra một con số kèm mức tự tin. Cách hợp lý để tổng hợp là trung bình có trọng số theo mức tự tin. Đó chính xác là điều một hệ hợp nhất cảm biến làm.

Từ hình ảnh đó suy ra được cả ba chỗ có thể hỏng.

Nếu một người nói “tôi rất chắc” trong khi thực ra chỉ đoán bừa, ý kiến người đó lấn át cả phòng — và tệ hơn, cuộc họp sẽ kết luận rằng những người kia đang sai chứ không phải người này đang nói quá. Đây là chuyện xảy ra khi bạn khai mức tin cậy của một cảm biến quá cao.

Nếu một người tính bằng đồng và người khác tính bằng đô la, trung bình hoá là vô nghĩa. Với cảm biến, điều này nghĩa là phải biết chúng đặt ở đâu so với nhau và đo vào lúc nào.

Và nếu có người nói nhầm, phải loại — nhưng mỗi người nói nhầm theo một kiểu. Một người thỉnh thoảng nhầm một chữ số, dễ thấy ngay. Một người khác thì nói rất hợp lý nhưng về một ngôi nhà hoàn toàn khác, và cả buổi không ai phát hiện. Hai kiểu ấy cần hai cách bắt. Đây là chỗ mà một quy tắc chung “loại ai lệch quá xa” sẽ bắt kiểu thứ nhất và bỏ sót kiểu thứ hai — đúng cái xảy ra với bánh xe bị trượt.

Nhưng điều quan trọng nhất thì không nằm ở việc trung bình. Giá trị lớn nhất của việc có nhiều người trong phòng không phải là con số chính xác hơn khi mọi người đều tỉnh táo. Đó là lúc *một người ngất xỉu* — cuộc họp vẫn tiếp tục được.

Với một cái máy, “ngất xỉu” nghĩa là camera bị che, đèn tắt, hay đi vào một hành lang trắng trơn. Một hệ chỉ có camera sẽ dừng lại ở đó. Một hệ có thêm vài nguồn khác thì vẫn đi tiếp được — kém chính xác hơn, nhưng vẫn đi. Và với một sản phẩm, khác biệt giữa “kém chính xác” và “dừng lại” là khác biệt giữa được và không được.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *biết mỗi cảm biến nói dối theo kiểu nào*. Không có quy tắc chung; phải hiểu vật lý của từng cảm biến. Bánh xe nói dối khi trượt; GPS nói dối khi tín hiệu dội vào toà nhà; camera nói dối khi có một chiếc xe tải đi ngang chiếm hết khung hình. Ba kiểu nói dối ấy chẳng liên quan gì tới nhau, và mỗi kiểu cần một cách bắt riêng.

Một câu: Thêm một cảm biến là thêm một ý kiến vào cuộc họp — và điều quan trọng nhất không phải nó nói gì, mà là nó có thành thật về mức nó chắc chắn hay không.

15.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *thêm cảm biến là thêm nhân tử, không đổi bộ giải* — là hệ quả cấu trúc của đồ thị nhân tử (cửa a). Nó bị bác bỏ nếu có một loại phép đo không viết được thành một nhân tử với Jacobian và hiệp phương sai — và có: các phép đo có *cấu trúc rời rạc*, chẳng hạn “robot đang ở trong phòng A” từ một bộ phân loại, không phải một phần dư liên tục. Chúng cần các kỹ thuật khác (ước lượng đa giả thuyết), và đó là một giới hạn thật của khung này chứ không phải một ngoại lệ nhỏ.

Mệnh đề thứ hai — *mỗi nguồn cần cổng riêng* — kiểm được bằng mini project phần chuỗi trượt. Nó bị bác bỏ nếu một cổng chung đủ tốt cho mọi nguồn trên một tập sự cố đa dạng. Tôi cho là không, dựa trên lập luận về phân bố outlier, nhưng mức độ nghiêm trọng thì tôi chưa đo cho các cặp cảm biến ngoài odometry–thị giác.

Mệnh đề thứ ba — *giá trị chính của cảm biến phụ là khả năng phục hồi, không phải độ chính xác* — là một phán đoán về ưu tiên, và nó phụ thuộc ứng dụng. Với một hệ quét đo đặc chạy trong điều kiện kiểm soát, độ chính xác đúng là tiêu chí chính và mệnh đề này sai. Nó đúng cho robot hoạt động không giám sát, tức cho mục tiêu đã nêu ở [00-map](#) (tôi suy ra từ mục tiêu ấy, chưa đổi chiều nguồn).

15.6 Kết nối

Nói lên trên. [A/04-uoc-luong-map-va-bundle-adjustment](#) cho khung mà nguyên tắc ở mục 2 là hệ quả trực tiếp. [A/05-observability-gauge-va-suy-bien](#) cho ngôn ngữ để đọc Bảng 15.1: mỗi cảm biến phá một gauge. [A/06-bat-dinh-va-lan-truyen-sai-so](#) là chương mà điều kiện một ở mục 2A phụ thuộc hoàn toàn vào — và nó cũng cảnh báo rằng hiệp phương sai đáng tin là thứ khó có. [C/14-hieu-chuan-nhu-mot-nhanh-cua-slam](#) là điều kiện hai.

Nói xuống dưới. [C/18-danh-gia-va-phuong-phap-luan](#) là chỗ khối *Cần nhớ* trở thành phương pháp: đo một hệ hợp nhất bằng ATE trung bình là bỏ sót phần lớn giá trị của nó. [D/19-ben-vung-trong-the-gioi-thuc](#) mở rộng danh mục sự cố mà cảm biến phụ giúp vượt qua. [D/21-tu-prototype-len-mass-production](#) là chỗ việc chọn cấu hình cảm biến trở thành một quyết định có chi phí BOM, có ngân sách điện, và có hệ quả về dung sai cơ khí — và là chỗ lập luận “stereo + IMU + odometry là điểm ngọt cho robot mặt đất” được trình bày đầy đủ.

Nói ngang. `.../IMU_Fusion/` là cây đầy đủ về nhánh quán tính: tiền tích phân (chương 7), bộ lọc và tối ưu (chương 8, 9), khởi tạo (chương 10), ràng buộc chuyển động và đa cảm biến (chương 12). Chương này cố ý không lặp lại. `.../marker/` là cây về nhánh fiducial, nơi marker đóng vai trò neo tuyệt đối — và nó cho thấy cái giá thật của giải pháp ấy: lắp đặt, hiệu chuẩn, bảo trì.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu nguyên tắc chung để thêm một cảm biến, và ba điều kiện của nó. Điều kiện nào hay bị bỏ nhất và vì sao?
2. Vì sao một nhân tử có hiệp phương sai sai lại tệ hơn không có nhân tử đó, và vì sao không có gì cảnh báo?
3. Outlier thị giác và outlier odometry khác nhau thế nào về phân bố, và vì sao một cổng chung không bắt được cả hai?
4. Odometry bánh xe và IMU đều cho tỉ lệ metric. Chế độ hỏng của chúng khác nhau thế nào, và vì sao điều đó nghĩa là nên dùng cả hai?

5. Thêm GNSS đòi ước lượng một phép biến đổi mấy bậc tự do, và điều kiện nào để nó quan sát được?
6. Nêu nguyên tắc chọn cảm biến theo “đặc tính sai số bù nhau”, và cho một ví dụ về hai cảm biến *không* bù nhau.
7. Vì sao đo một hệ hợp nhất bằng ATE trung bình trên chuỗi sạch là bỏ sót phần lớn giá trị của nó?

Dense mapping và biểu diễn (Dense Mapping and Representation)

PHỤ THUỘC A/01-hình-học-da-thi (triangulation, stereo); A/06-bat-dinh-va-lan-truyen-sai-so ($\sigma_z \approx z^2 \sigma_d / (bf)$) — quan hệ này quyết định chất lượng mọi bản đồ dày); C/13-quan-ly-ban-do (bài toán kích thước, ở đây gay gắt hơn nhiều bậc).

TÓM TẮT

Bản đồ thưa gồm vài nghìn điểm đủ để định vị nhưng không đủ để *làm gì với thế giới*. Một robot cần biết chỗ nào đi được; một hệ AR cần biết bề mặt ở đâu để đặt vật ảo lên; một hệ quét cần một mô hình có thể in ra. Chương này về các biểu diễn dày và về một câu hỏi mà ít ai hỏi thẳng: *ai cần chúng, để làm gì, và cái giá là bao nhiêu?* Nội dung có bốn phần. Một: lấy độ sâu dày từ ảnh — stereo, multi-view, và giới hạn vật lý mà không thuật toán nào vượt qua được. Hai: bốn họ biểu diễn cổ điển — TSDF, surfel, octree, mesh — với tiêu chí chọn dựa trên *việc bạn định dùng bản đồ để làm gì*, không dựa trên chất lượng hình ảnh. Ba: các biểu diễn học được — NeRF và Gaussian splatting — là hướng nóng nhất hiện nay, với một đánh giá thẳng thắn về chỗ chúng đã tốt và chỗ chưa. Bốn: một đánh đổi mà tài liệu hay bỏ qua. Nếu chỉ nhớ một điều: *bản đồ dày đẹp hơn nhưng độ chính xác tư thế thường không hơn bản đồ thưa* — và hiểu vì sao là hiểu được vị trí đúng của cả chương này trong một hệ thống.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bản đồ thưa (sparse map)** — tập vài nghìn tới vài trăm nghìn điểm 3D rời rạc, mỗi điểm ứng với một đặc trưng ảnh. Đủ để định vị, không đủ để mô tả bề mặt.
- **bản đồ dày (dense map)** — biểu diễn mô tả *bề mặt* hoặc *thể tích* của cảnh, không chỉ vài điểm rời rạc. Tốn bộ nhớ hơn hàng trăm tới hàng nghìn lần.
- **TSDF** — *truncated signed distance function* — mỗi voxel lưu khoảng cách có dấu tới bề mặt gần nhất, cắt ở một ngưỡng. Bề mặt là tập mức không. Hợp nhất nhiều lượt quét bằng trung bình có trọng số.
- **surfel** — một mảnh bề mặt nhỏ có vị trí, pháp tuyến, bán kính và màu. Biểu diễn không cần lưới, nên dễ biến dạng khi bản đồ được hiệu chỉnh.
- **octree** — cấu trúc cây chia không gian thành tám phần đệ quy, chỉ chia sâu ở chỗ có nội dung. Tiết kiệm bộ nhớ cho không gian phần lớn là trống.
- **ESDF** — *euclidean signed distance field* — mỗi voxel lưu khoảng cách tới chương ngại gần nhất. Đây là thứ mà bộ lập kế hoạch đường đi thật sự cần, không phải một mô hình đẹp.
- **NeRF** — *neural radiance field* — biểu diễn cảnh bằng một mạng ánh xạ từ (vị trí, hướng nhìn) sang (màu, mật độ), render bằng ray marching.
- **3D Gaussian Splatting** — biểu diễn cảnh bằng hàng triệu Gaussian 3D dị hướng có màu và độ trong, render bằng phép chiếu và trộn — nhanh hơn NeRF nhiều bậc vì không cần lấy mẫu dọc tia.

CÂU HỎI MỞ ĐẦU

- Q1. Bản đồ dày dùng nhiều dữ liệu hơn bản đồ thưa hàng trăm lần. Vì sao nó thường *không* cho tư thế chính xác hơn?
- Q2. Bộ lập kế hoạch đường đi của bạn cần biết chỗ nào đi được. Nêu biểu diễn nó thật sự cần, và nói vì sao một mesh đẹp không phải câu trả lời.
- Q3. Stereo cho độ sâu ở mọi pixel có vấn. Nêu ba vùng mà nó chắc chắn thất bại, và nói vì sao đó là giới hạn vật lý chứ không phải giới hạn thuật toán.
- Q4. 3D Gaussian Splatting cho hình ảnh đẹp hơn hẳn TSDF với cùng dữ liệu. Nêu ba lý do vì sao một robot sản phẩm năm 2026 có thể vẫn nên dùng TSDF.
- Q5. Bạn có một bản đồ dày và loop closure vừa xảy ra, dịch chuyển các keyframe đi vài mét. Chuyện gì phải xảy ra với bản đồ dày, và vì sao đó là vấn đề khó hơn với TSDF so với surfel?

16.1 Đặt vấn đề

Mọi chương trước làm việc với bản đồ thưa: vài nghìn điểm 3D, mỗi điểm ứng với một góc hoặc một đốm trong ảnh. Bản đồ ấy đủ cho việc định vị — đó là toàn bộ nội dung Phần B — và nó rẻ.

Nó không đủ cho gần như mọi thứ khác. Một robot không thể lập kế hoạch đường đi từ một đám mây điểm thưa, vì giữa hai điểm có thể là không gian trống hoặc có thể là một bức tường và bản đồ không nói. Một hệ AR không thể đặt một vật ảo lên bàn nếu nó không biết mặt bàn ở đâu. Một hệ quét không thể xuất ra một mô hình.

Bài toán gốc: từ cùng những ảnh ấy, dựng một mô tả bề mặt hoặc thể tích của cảnh.

Ai gặp bài toán này? Ba nhóm với ba yêu cầu rất khác nhau, và gộp chúng là một lỗi phổ biến vì nó dẫn tới việc chọn sai biểu diễn.

Nhóm một — điều hướng. Robot cần biết chỗ nào đi được. Yêu cầu: đúng về mặt an toàn (thà báo có chướng ngại khi không có còn hơn ngược lại), cập nhật nhanh, và ở một biểu diễn mà bộ lập kế hoạch dùng trực tiếp được. *Không cần đẹp.*

Nhóm hai — tương tác. AR, robot gấp vật. Cần bề mặt chính xác ở vùng quan tâm, với pháp tuyến. Cần đẹp ở mức đủ để không gây khó chịu.

Nhóm ba — tái tạo. Quét để lưu trữ, để in 3D, để dựng mô hình. Cần chính xác và đẹp; *không cần* thời gian thực.

Ba nhóm ấy dẫn tới ba lựa chọn biểu diễn khác nhau, và phần lớn nhầm lẫn trong lĩnh vực này đến từ việc đọc một bài báo viết cho nhóm ba rồi áp dụng cho nhóm một.

Có một câu hỏi thứ hai, ít được hỏi thẳng và quan trọng hơn: *bản đồ dày có giúp việc định vị không?* Trực giác nói có — nhiều dữ liệu hơn thì tốt hơn. Mục 4 lập luận rằng câu trả lời chủ yếu là *không*, và hiểu vì sao là hiểu được vị trí đúng của cả chương này.

TRỰC GIÁC

Hình dung sự khác nhau giữa một tấm bản đồ du lịch và một mô hình sa bàn.

Bản đồ du lịch có vài chục ký hiệu: nhà thờ ở đây, ga tàu ở kia, ngã tư ở đó. Nó đủ để bạn biết mình đang ở đâu và đi hướng nào. Nó nhẹ, bạn gấp bỏ túi được, và nó không nói gì về việc vỉa hè cao bao nhiêu centimet.

Sa bàn thì mô phỏng cả thành phố — từng bức tường, từng bậc thềm. Nó nặng, tốn công dựng, nhưng nó trả lời được những câu mà bản đồ không trả lời được: cái xe đẩy của tôi có qua được chỗ này không? Cái kệ này có đủ chỗ đặt vật kia không?

Hai thứ phục vụ hai mục đích, và đây là chỗ trực giác hay sai: *sa bàn không giúp bạn định vị tốt hơn bản đồ*. Để biết mình đang ở đâu, bạn vẫn nhìn nhà thờ và ga tàu — những thứ dễ nhận. Cả một bức tường phẳng của sa bàn không nói gì thêm về vị trí của bạn, vì mọi chỗ trên bức tường ấy trông như nhau.

Đó là toàn bộ mục 4 của chương, phát biểu bằng hình ảnh: thông tin về *vị trí* nằm ở những chỗ *khác biệt*, và bản đồ thưa đã lấy hết những chỗ ấy rồi. Phần dày thêm vào là phần *không khác biệt* — hữu ích để biết hình dạng thế giới, vô ích để biết mình ở đâu trong đó.

Còn một chuyện nữa. Khi bạn phát hiện mình vẽ sai và phải kéo bản đồ cho đúng (loop closure), thì với tấm bản đồ du lịch, bạn chỉ cần dịch vài chục ký hiệu. Với sa bàn, bạn phải *làm lại* cả một vùng — và đó là lý do bản đồ dày khó sống chung với SLAM hơn nhiều so với vẽ ngoài.

16.2 Lấy độ sâu dày

A. Stereo và giới hạn vật lý

Với cặp stereo đã hiệu chỉnh, bài toán là tìm disparity cho *mọi* pixel. Khác với B/07, nơi ta chọn vài trăm điểm tốt, ở đây ta phải xử lý cả những pixel không có gì đặc biệt.

SGM [83] là phương pháp cổ điển vẫn được dùng rộng rãi, và ý tưởng của nó đáng biết. Bài toán đúng là tối ưu một hàm năng lượng hai chiều với số hạng dữ liệu (độ khớp) và số hạng trơn (disparity của pixel cạnh nhau nên gần nhau) — nhưng tối ưu năng lượng 2D là NP-khó. SGM xấp xỉ bằng cách tối ưu chính xác dọc *nhiều đường một chiều* qua mỗi pixel rồi cộng chi phí lại. Rẻ, song song hoá tốt, và chất lượng đủ cho hầu hết ứng dụng.

Ba vùng mà stereo *chắc chắn* thất bại, và đây là câu trả lời cho câu hỏi mở đầu thứ ba — điều quan trọng là cả ba là giới hạn *vật lý*, không phải giới hạn thuật toán.

Một — vùng không có vân. Một bức tường trắng: mọi pixel trong một hàng trông giống hệt nhau, nên không có thông tin nào xác định disparity. Không thuật toán nào tạo ra thông tin từ chỗ không có; cái tốt nhất có thể làm là *nội suy* từ biên và *thừa nhận* rằng đó là nội suy.

Hai — vùng bị che khuất. Một phần cảnh nhìn thấy từ camera trái nhưng bị che với camera phải. Không có tương ứng nào tồn tại. Phát hiện được bằng kiểm tra trái-phải, và câu trả lời đúng là đánh dấu “không biết”.

Ba — bề mặt phản chiếu hoặc trong suốt. Gương, kính, nước, kim loại bóng. Ở đây tệ hơn hai trường hợp trên: khớp điểm *thành công* nhưng nó khớp với ảnh phản chiếu, nên độ sâu thu được là độ sâu của một vật không tồn tại ở đó. Đây là kiểu hỏng im lặng, và nó là lý do robot đâm vào cửa kính.

Ngoài ba vùng ấy, có một giới hạn định lượng chi phối mọi thứ: $\sigma_z \approx z^2 \sigma_d / (bf)$ từ A/06. Nó nói rằng dù thuật toán hoàn hảo, chất lượng độ sâu suy giảm theo bình phương khoảng cách. Một bản đồ dày ở xa là một bản đồ dày *sai*, và vẽ nó ra trông vẫn đẹp.

B. Multi-view và độ sâu học được

Với nhiều khung thay vì hai, ta có multi-view stereo. Lợi ích: nhiều đường cơ sở khác nhau nên nhiều tầm độ sâu được phục vụ tốt; và nhiều góc nhìn nên ít bị che khuất hơn. PatchMatch stereo và các biến thể là chuẩn cho tái tạo ngoại tuyến.

Độ sâu học được (B/08 mục 4C) giải quyết được vấn đề vùng không có vân — vì mô hình mang tiên nghiệm về việc các bề mặt thường trông thế nào. Đây là một đóng góp thật và là chỗ học máy giúp rõ ràng.

Cảnh báo vẫn như ở B/08: kết quả *trông* chắc chắn ở mọi pixel, kể cả những pixel mà hình học không nói gì. Một bản đồ dày hoàn toàn từ độ sâu học được là một bản đồ mà một phần là phép đo và một phần là phỏng đoán, *không có gì phân biệt hai phần*. Với ứng dụng an toàn — robot tránh chướng ngại — đó là điều nguy hiểm. Cách xử lý đúng: giữ hai kênh, một cho giá trị và một cho nguồn gốc (đo được hay suy ra), và để bộ lập kế hoạch đối xử với chúng khác nhau.

16.3 Bốn họ biểu diễn cổ điển

Bảng 16.1. Bốn họ biểu diễn. Cột “dùng cho” là tiêu chí chọn đúng — chọn theo việc bạn định làm gì với bản đồ, không theo chất lượng hình ảnh.

Biểu diễn	Cơ chế	Dùng cho	Hỏng ở
TSDF	Voxel lưu khoảng cách có dấu; bề mặt là tập mức không; hợp nhất bằng trung bình có trọng số	Tái tạo bề mặt chất lượng cao; quét trong nhà	Bộ nhớ theo thể tích; khó biến dạng khi loop closure
Surfel	Các mảnh bề mặt rời có vị trí, pháp tuyến, bán kính	Khi bản đồ phải <i>biến dạng</i> — ElasticFusion	Không có bề mặt liền; khó truy vấn “điểm này trông hay đặc”
Octree / occupancy	Chia không gian đệ quy; mỗi ô lưu xác suất bị chiếm	Điều hướng — trả lời trực tiếp “chỗ này đi được không”	Không mô tả bề mặt chính xác; độ phân giải hữu hạn
Mesh	Lưới tam giác	Hiển thị, in 3D, mô phỏng vật lý	Đắt để cập nhật gia tăng; khó xử lý topology thay đổi

Hai điểm đáng nói kỹ hơn bảng.

A. Điều hướng cần ESDF, không cần mesh

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một trong những nhầm lẫn tốn kém nhất trong lĩnh vực.

Bộ lập kế hoạch đường đi không cần biết bề mặt trông thế nào. Nó cần trả lời hai câu: *điểm này có trống không?* và *điểm này cách chướng ngại gần nhất bao xa?* Câu thứ hai đặc biệt quan trọng vì phần lớn bộ lập kế hoạch hiện đại dùng một hàm chi phí dựa trên khoảng cách tới chướng ngại, và chúng cần cả *gradient* của hàm đó.

ESDF trả lời trực tiếp cả hai. Một mesh đẹp thì không — để trả lời “cách chướng ngại bao xa” từ một mesh, bạn phải tính khoảng cách tới tam giác gần nhất, và làm điều đó hàng nghìn lần mỗi chu kỳ lập kế hoạch là quá đắt.

Bài học tổng quát: *chọn biểu diễn theo truy vấn bạn cần trả lời, không theo cái trông giống thế giới nhất*. Một bản đồ occupancy độ phân giải thô mà trả lời truy vấn trong micro giây hữu ích hơn một mesh hoàn hảo mà mỗi truy vấn tốn mili giây.

B. Vấn đề biến dạng khi loop closure

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và là vấn đề kỹ thuật khó nhất của bản đồ dày.

Khi loop closure xảy ra (B/11), các keyframe dịch chuyển — có thể vài mét. Bản đồ thưa xử lý dễ: mỗi landmark gắn với một keyframe neo, nên nó dịch theo. Bản đồ dày thì khó, và mức khó phụ thuộc vào biểu diễn.

Với TSDF, dữ liệu được *hợp nhất* vào một lưới voxel cố định trong không gian. Sau khi các keyframe dịch, lưới ấy không còn đúng — và không có cách nào “dịch” một lưới voxel đã trộn dữ liệu từ nhiều góc nhìn. Cách duy nhất đúng là *tích hợp lại từ đầu* từ các khung gốc, tức phải lưu toàn bộ ảnh độ sâu, tức tốn bộ nhớ khổng lồ. Các cách thoả hiệp: chia bản đồ thành các khối gắn với keyframe (mỗi khối dịch theo keyframe của nó, chấp nhận các đường nối không khớp), hoặc chỉ tích hợp lại vùng bị ảnh hưởng nhiều nhất.

Với surfel, mỗi surfel gắn với một keyframe nên nó dịch theo một cách tự nhiên — đây chính là lý do ElasticFusion [84] chọn surfel. Cái giá là mất bề mặt liền và khó trả lời truy vấn thể tích.

Đây là một ví dụ rõ về việc *lựa chọn biểu diễn bị chi phối bởi một yêu cầu hệ thống mà tài liệu về biểu diễn không nhắc tới*. Nếu bạn đọc một bài về TSDF mà không nghĩ tới loop closure, bạn sẽ chọn nó rồi phát hiện vấn đề sau nhiều tháng.

16.4 Biểu diễn học được: ảnh chụp tại 9/2026

CẠM BÃY

Mục này chốt tại tháng 9/2026 và là phần hết hạn nhanh nhất của cả cây. Lĩnh vực này chuyển động rất nhanh; mọi đánh giá dưới đây nên được rà lại sau sáu tháng.

Ngoài ra, tôi **không dẫn con số so sánh chất lượng** giữa các phương pháp. Lý do như ở B/07: các chỉ số (PSNR, độ chính xác bề mặt) phụ thuộc mạnh vào bộ dữ liệu và vào cách đo, và các bài báo trong lĩnh vực này thường so với baseline được cài lại chứ không phải số công bố gốc — đúng cỡ đồ ở research-rules.md mục 4. Điều tôi đưa ra là đánh giá về *cơ chế* và về *mức trưởng thành cho sản phẩm*, và cả hai đều là phán đoán của tôi chứ không phải kết quả đo.

A. NeRF-SLAM

iMAP [85] và NICE-SLAM [86] thay biểu diễn tường minh bằng một mạng: cảnh là các trọng số của mạng, và cả tư thế lẫn trọng số được tối ưu cùng lúc bằng phần dư quang trắc — về bản chất là một phương pháp direct (B/08) với một biểu diễn cấu trúc rất khác.

Cái được: biểu diễn liên tục, nội suy tự nhiên vào vùng chưa quan sát, bộ nhớ không tỉ lệ với thể tích.

Cái mất, và đây là ba điểm quyết định cho sản phẩm: *chậm* — render cần lấy mẫu nhiều điểm dọc mỗi tia; *quên* — một mạng huấn luyện tăng dần có xu hướng quên vùng cũ khi học vùng mới, và cách chữa (giữ lại các khung cũ để huấn luyện lại) làm mất lợi thế bộ nhớ; và *không có bảo đảm* — không cách nào biết một vùng trong biểu diễn là phép đo hay là ảo giác nội suy.

B. 3D Gaussian Splatting

3DGS [87] thay mạng bằng hàng triệu Gaussian 3D dị hướng có màu và độ trong, render bằng phép chiếu và trộn theo thứ tự độ sâu. Điểm quyết định: *không cần lấy mẫu dọc tia*, nên nhanh hơn NeRF nhiều bậc trong khi chất lượng hình ảnh tương đương hoặc hơn.

SplaTAM [88] và MonoGS [89] đưa nó vào SLAM: Gaussian là bản đồ, và tư thế được ước lượng bằng cách tối ưu phần dư quang trắc qua render khả vi.

Đây là hướng nóng nhất tại thời điểm chốt, và lý do dễ hiểu: nó cho hình ảnh đẹp hơn mọi biểu diễn cổ điển với cùng dữ liệu, và nó đủ nhanh để nghĩ tới thời gian thực.

Ba lý do vì sao một robot sản phẩm năm 2026 vẫn có thể nên dùng TSDF — đây là câu trả lời cho câu hỏi mở đầu thứ tư.

Một — truy vấn. 3DGS tối ưu cho việc *render*, không cho việc trả lời “điểm này có trống không”. Một đám Gaussian không phải một hàm occupancy, và trích xuất bề mặt hoặc ESDF từ nó là một bước thêm, không rẻ và không được bảo đảm. Với điều hướng, đây là điểm chặn.

Hai — bộ nhớ và phần cứng. Hàng triệu Gaussian, mỗi cái vài chục tham số, cộng với việc render cần GPU. Với một robot dịch vụ chạy trên SoC nhúng, đây là ngân sách không có.

Ba — tính dự đoán được và độ trưởng thành. Chế độ hồng của TSDF đã được hiểu rõ sau mười lăm năm; chế độ hồng của 3DGS-SLAM thì chưa. Với sản phẩm, đây là lập luận nặng ký, và nó là cùng lập luận với việc chọn ORB thay vì SuperPoint ở B/07 mục 2B.

Đánh giá thẳng thắn của tôi tại 9/2026: 3DGS là một bước tiến thật và rất có thể là tương lai của biểu diễn cảnh; nó *chưa* là lựa chọn cho một robot điều hướng chạy trên phần cứng hạn chế. Với nhóm ba — tái tạo ngoại tuyến — nó đã rất mạnh. D/22 xếp “3DGS cho localization thực dụng” làm một trong tám khoảng trống, và lý do nằm ở ba điểm trên.

16.5 Đánh đổi bị bỏ qua

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là mục mà tôi cho là có giá trị cao nhất của chương.

Bản đồ dày thường không cho tư thế chính xác hơn bản đồ thưa. Điều này phản trực giác và đáng dẫn ra lý do.

Thông tin về tư thế nằm ở đâu? Quay lại Jacobian ở A/03: một quan sát đóng góp vào ma trận thông tin tỉ lệ với *gradient* của hàm đo. Với phần dư quang trắc, pixel không có gradient đóng góp *đúng bằng không* (B/08 mục 2B). Với phần dư chiếu, một điểm không phân biệt được với lân cận của nó không cho tương ứng đáng tin.

Nói cách khác: *thông tin về tư thế tập trung ở những chỗ có cấu trúc*, và một front-end thưa tốt đã lấy gần hết những chỗ ấy. Phần dày thêm vào chủ yếu là các vùng trơn — chúng cho biết *hình dạng* bề mặt giữa các điểm đã biết, nhưng chúng không thêm ràng buộc mạnh cho tư thế.

Có ngoại lệ và đáng nêu rõ để công bằng. Trong môi trường rất nghèo đặc trưng — một hành lang trắng có gradient rất yếu nhưng khác không — direct method dày khai thác được phần thông tin ít ỏi mà front-end thưa bỏ qua, vì detector cần một cực đại rõ ràng để tồn tại. Đây là lợi thế thật của direct, đã nêu ở B/08 mục 2A. Nhưng đó là lợi thế của *phần dư dày*, không phải của *bản đồ dày* — hai thứ khác nhau: ta có thể dùng phần dư dày để ước lượng tư thế mà vẫn chỉ lưu một bản đồ thưa.

Kết luận thiết kế, và nó gọn: *xây bản đồ dày vì bạn cần nó cho một truy vấn cụ thể — điều hướng, tương tác, hiển thị — không phải vì bạn kỳ vọng nó cải thiện định vị.* Nếu mục tiêu là định vị chính xác hơn, hãy đầu tư vào front-end (B/07), vào hiệu chuẩn (C/14), hoặc vào quản lý bản đồ (C/13) — ba chỗ đó cho lợi ích lớn hơn nhiều trên mỗi đơn vị công sức.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OctoMap [90]	Octree occupancy có xác suất; truy vấn nhanh	Lựa chọn mặc định cho điều hướng; mục 3A
KinectFusion [91]	TSDF thời gian thực trên GPU	Công trình khai sinh dòng TSDF; đọc để hiểu cơ chế hợp nhất
ElasticFusion [84]	Surfel để bản đồ biến dạng được khi loop closure	Mục 3B — lựa chọn biểu diễn bị chi phối bởi yêu cầu hệ thống
NICE-SLAM [86]	NeRF phân cấp cho SLAM	Mục 4A; chú ý vấn đề quên và tốc độ
SplaTAM [88], MonoGS [89]	3DGS-SLAM với render khả vi	Mục 4B; ảnh chụp tại 9/2026, rà lại sau sáu tháng
SGM trong OpenCV	Stereo dày cổ điển, chạy được trên CPU	Mục 2A; vẫn là baseline đáng tin cho sản phẩm

MINI PROJECT SLAM · Đo xem bản đồ dày có cải thiện định vị không — trên dữ liệu của chính bạn · 3 ngày

Mục tiêu	kiểm mệnh đề ở mục 5 thay vì tin nó, và đồng thời đo cái giá thật của bản đồ dày.
Dữ liệu	một chuỗi RGB-D có chân trị (TUM-RGBD), cộng một chuỗi tự quay trong môi trường <i>nghèo đặc trưng</i> — một hành lang trắng, một phòng trống. Chuỗi thứ hai là chỗ ngoại lệ ở mục 5 có thể xuất hiện.
Các bước	chạy ba cấu hình trên cùng dữ liệu: (a) SLAM thừa thuần (ORB-SLAM3 RGB-D), (b) SLAM thừa + dựng TSDF <i>sau</i> khi đã có tư thế, (c) một hệ dày dùng phần dư quang trắc dày để ước lượng tư thế; đo ATE, bộ nhớ, thời gian mỗi khung cho cả ba; rồi làm một loop closure và đo thời gian cùng chất lượng của việc cập nhật bản đồ dày.
Xong khi	bạn có một bảng ba cấu hình với bốn cột. Bạn phải quan sát được: (1) trên chuỗi có đặc trưng tốt, ATE của (a) và (c) gần nhau trong khi bộ nhớ chênh nhau hai bậc — xác nhận mục 5; (2) trên chuỗi nghèo đặc trưng, (c) có thể thắng rõ — xác nhận ngoại lệ; (3) chi phí cập nhật bản đồ dày sau loop closure lớn hơn bạn nghĩ, và đó là con số ít ai công bố.
Nối với	C/13-quan-ly-ban-do cho bài toán kích thước ở quy mô này; D/20-toi-uu-hieu-nang-va-he-thong cho ngân sách bộ nhớ thật trên phần cứng nhúng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì thông tin về tư thế tập trung ở những chỗ *có cấu trúc*, và một front-end thừa tốt đã lấy gần hết những chỗ ấy. Theo Jacobian ở [A/03](#), đóng góp vào ma trận thông tin tỉ lệ với gradient của hàm đo; pixel không có gradient đóng góp đúng bằng không. Phần dày thêm vào chủ yếu là các vùng trơn — chúng cho biết *hình dạng* bề mặt giữa các điểm đã biết nhưng không thêm ràng buộc mạnh cho tư thế. Ngoại lệ: môi trường rất nghèo đặc trưng, nơi gradient yếu nhưng khác không và detector thừa không tồn tại được. Nhưng đó là lợi thế của *phần dư dày*, không phải của *bản đồ dày* — ta có thể dùng phần dư dày mà vẫn chỉ lưu bản đồ thừa.
- A2. Nó cần một trường khoảng cách — ESDF — trả lời trực tiếp hai câu: điểm này có trống không, và cách chướng ngại gần nhất bao xa (kèm gradient, vì phần lớn bộ lập kế hoạch hiện đại dùng hàm chi phí theo khoảng cách). Một mesh đẹp không phải câu trả lời vì để tính khoảng cách tới chướng ngại từ mesh, bạn phải tìm tam giác gần nhất, và làm điều đó hàng nghìn lần mỗi chu kỳ là quá đắt. Bài học tổng quát: *chọn biểu diễn theo truy vấn bạn cần trả lời, không theo cái trông giống thế giới nhất*.
- A3. Ba vùng: (a) *không có vân* — mọi pixel trong một hàng giống hệt nhau nên không có thông tin xác định disparity; (b) *bị che khuất* — không tương ứng nào tồn tại; (c) *phản chiếu hoặc trong suốt* — tệ nhất, vì khớp *thành công* nhưng khớp với ảnh phản chiếu, cho độ sâu của một vật không có ở đó, tức hổng im lặng. Cả ba là giới hạn vật lý vì thông tin *không tồn tại trong dữ liệu* — không thuật toán nào tạo ra thông tin từ chỗ không có. Cái tốt nhất có thể làm với (a) là nội suy và *thừa nhận* rằng đó là nội suy; với (b) là đánh dấu “không biết”; với (c) thì chưa có lời giải tốt.

- A4. (1) *Truy vấn*: 3DGS tối ưu cho render, không cho việc trả lời “điểm này có trống không”; một đám Gaussian không phải hàm occupancy, và trích ESDF từ nó là một bước thêm không rẻ và không bảo đảm — với điều hướng đây là điểm chặn. (2) *Bộ nhớ và phần cứng*: hàng triệu Gaussian cộng yêu cầu GPU, ngoài ngân sách của một SoC nhúng. (3) *Tính dự đoán được*: chế độ hồng của TSDF đã hiểu rõ sau mười lăm năm, của 3DGS-SLAM thì chưa — cùng lập luận với việc chọn ORB thay vì SuperPoint ở B/07. Đánh giá của tôi tại 9/2026: 3DGS rất mạnh cho tái tạo ngoại tuyến, chưa phải lựa chọn cho robot điều hướng trên phần cứng hạn chế.
- A5. Bản đồ dày phải *biến dạng theo*. Với **TSDF** điều này khó vì dữ liệu đã được *hợp nhất* vào một lưới voxel cố định trong không gian, và không có cách nào “dịch” một lưới đã trộn dữ liệu từ nhiều góc nhìn — cách duy nhất đúng là tích hợp lại từ đầu, tức phải lưu toàn bộ ảnh độ sâu gốc. Các thỏa hiệp: chia thành khối gắn với keyframe (chấp nhận đường nối không khớp), hoặc chỉ tích hợp lại vùng ảnh hưởng nhiều nhất. Với **surfel** thì dễ hơn nhiều vì mỗi surfel gắn với một keyframe nên dịch theo tự nhiên — đó chính là lý do ElasticFusion chọn surfel. Đây là ví dụ rõ về việc lựa chọn biểu diễn bị chi phối bởi một yêu cầu hệ thống mà tài liệu về biểu diễn không nhắc.

NÓI LẠI THẬT ĐƠN GIẢN

Có hai loại bản đồ. Một tấm bản đồ du lịch gấp bỏ túi, ghi vài chục địa điểm để nhận. Và một cái sa bàn mô phỏng cả thành phố tới từng bậc thềm.

Bản đồ du lịch đủ để bạn biết mình ở đâu. Sa bàn thì trả lời được những câu khác: cái xe đẩy của tôi có lọt qua chỗ này không, cái kệ kia có đủ chỗ đặt vật này không.

Và đây là chỗ trực giác hay sai: *sa bàn không giúp bạn định vị tốt hơn*. Để biết mình ở đâu, bạn vẫn nhìn nhà thờ và ga tàu — những thứ dễ nhận ra. Cả một bức tường phẳng trong sa bàn không nói gì thêm về vị trí của bạn, vì mọi chỗ trên bức tường ấy trông như nhau.

Nên câu hỏi đúng không phải “bản đồ nào tốt hơn” mà là “*tôi định dùng bản đồ để làm gì*”. Nếu để biết mình ở đâu, tấm gấp bỏ túi là đủ và rẻ hơn hàng trăm lần. Nếu để quyết định đi lối nào, bạn cần thứ khác — nhưng không phải sa bàn đẹp, mà là một thứ trả lời nhanh câu hỏi “chỗ này có đi được không”. Hai thứ đó không giống nhau: một cái nhìn giống thế giới, một cái trả lời nhanh câu hỏi bạn cần hỏi.

Còn một chuyện nữa, ít ai nghĩ tới. Khi bạn phát hiện mình vẽ sai và phải kéo bản đồ lại cho đúng, thì với tấm gấp bỏ túi bạn chỉ cần dịch vài chục ký hiệu. Với sa bàn, bạn phải đập đi làm lại cả một khu — vì các mảnh đã dính chặt vào nhau và không dịch riêng được. Đó là lý do bản đồ chi tiết khó sống chung với một hệ thống hay phải sửa lại chính mình.

Chỗ không rút gọn được: chỗ không rút gọn được là các bề mặt phản chiếu. Một tấm gương hay một cửa kính cho hệ thống một phép đo *trông hoàn toàn hợp lệ* về một vật không tồn tại ở đó. Khác với vùng trống trơn — nơi hệ thống biết mình không biết — ở đây nó tự tin và sai. Chưa có lời giải tốt, và đó là lý do robot vẫn đâm vào cửa kính.

Một câu: *Xây bản đồ dày vì bạn cần trả lời một câu hỏi cụ thể, không vì bạn kỳ vọng nó giúp bạn biết mình đang ở đâu.*

16.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *bản đồ dày không cải thiện độ chính xác tư thế* — suy từ cấu trúc Jacobian (cửa a) và kiểm được bằng mini project. Nó bị bác bỏ nếu một hệ dày vượt hệ thưa một cách nhất quán trên các chuỗi *có đặc trưng tốt* — tức ngoài ngoại lệ đã nêu. Tôi cho là không, nhưng cần nói rõ một giới hạn của lập luận: nó giả định front-end thưa đã *tốt*. Với một front-end thưa kém, bổ sung thông tin dày đúng là có ích, và lúc đó kết luận đúng là “*hãy sửa front-end*” chứ không phải “*hãy dùng dày*”.

Mệnh đề thứ hai — *ba vùng thất bại của stereo là giới hạn vật lý* — đúng theo định nghĩa cho hai vùng đầu (thông tin không tồn tại). Với vùng thứ ba (phản chiếu) thì *không hoàn toàn*: thông tin

có tồn tại — một hệ có thể nhận ra rằng đây là gương nếu nó suy luận về tính nhất quán của hình học phản xạ qua nhiều góc nhìn. Nên phát biểu chặt hơn: *với hình học hai khung thuần túy thì đó là giới hạn; với suy luận cấp cao hơn thì có thể phát hiện*. Đây là một hướng nghiên cứu chưa được khai thác nhiều.

Mệnh đề thứ ba — đánh giá về 3DGS tại 9/2026 — là một phán đoán về mức trưởng thành, và nó là phần dễ hết hạn nhất của cả cây. Nó bị bác bỏ bởi thời gian: nếu trong hai năm nữa có một hệ 3DGS chạy điều hướng trên SoC nhúng với ESDF trích xuất được tin cậy, đánh giá này sai. Tôi cho rằng khả năng đó không nhỏ, và đó là lý do khối *Cạm bẫy* ở mục 4 yêu cầu rà lại sau sáu tháng.

16.7 Kết nối

Nối lên trên. [A/01-hình-hoc-da-thi](#) cho triangulation và hình học stereo. [A/06-bat-dinh-va-lan-truyen-sai-so](#) cho quan hệ $\sigma_z \approx z^2 \sigma_d / (bf)$, thứ chỉ phối chất lượng của mọi bản đồ dày và giải thích vì sao bản đồ dày ở xa là bản đồ dày sai. [B/08-mo-hinh-phan-du](#) cho phần dư quang trắc và cho lời cảnh báo về prior độ sâu học được, áp dụng nguyên vẹn ở mục 2B.

Nối ngang. [C/13-quan-ly-ban-do](#) là chương mà bài toán kích thước được đặt ra; ở đây nó gay gắt hơn hai tới ba bậc độ lớn, và vấn đề biến dạng khi loop closure ở mục 3B là một bài toán mà chương 13 không gặp. [C/17-hoc-may-trong-slam](#) bàn kỹ hơn về các mô hình sinh ra độ sâu và về việc hiệu chuẩn bất định của chúng.

Nối xuống dưới. [D/20-toi-uu-hieu-nang-va-he-thong](#) là chỗ ngân sách bộ nhớ và tính toán trở thành ràng buộc cứng — và bản đồ dày là thứ tiêu tốn ngân sách ấy nhiều nhất. [D/22-huong-nghien-cuu-con-trong](#) xếp “biểu diễn neural cho localization thực dụng” làm một trong tám khoảng trống, với lý do nằm trọn trong mục 4B.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao bản đồ dày thường không cho tư thế chính xác hơn? Trả lời bằng ngôn ngữ gradient và ma trận thông tin, rồi nêu ngoại lệ.
2. Bộ lập kế hoạch cần biểu diễn nào, và vì sao mesh không phải câu trả lời dù nó mô tả thể giới chính xác hơn?
3. Nêu ba vùng mà stereo thất bại, và nói vùng nào hỏng im lặng — tức nguy hiểm nhất.
4. Vì sao TSDF khó biến dạng sau loop closure còn surfel thì dễ? Nêu hai cách thoả hiệp cho TSDF.
5. Nêu ba lý do một robot sản phẩm có thể vẫn chọn TSDF thay vì 3DGS năm 2026, và nói lý do nào là điểm chặn cứng.
6. Một bản đồ dày trộn giữa phép đo và độ sâu học được. Vì sao đó là nguy hiểm cho ứng dụng an toàn, và cách xử lý đúng là gì?
7. Nếu mục tiêu là định vị chính xác hơn, nêu ba chỗ nên đầu tư thay vì xây bản đồ dày.

Học máy trong SLAM (Learning in SLAM)

PHỤ THUỘC A/06-bat-dinh-va-lan-truyen-sai-so (bất định — chương này lập luận rằng đây là chỗ học máy đóng góp được nhiều nhất và hiện đang đóng góp ít nhất); B/07-dac-trung-va-data-association (front-end học được); B/08-mo-hinh-phan-du (prior học được và ba điều kiện của nó); B/12-relocalization (trường hợp APR — một bài học phương pháp).

TÓM TẮT

Học máy đã thay đổi phần lớn thị giác máy tính, và câu hỏi tự nhiên là nó thay đổi SLAM tới đâu. Chương này lập luận rằng câu trả lời không đồng đều: có những chỗ học máy *đã* thắng rõ ràng và nên dùng ngay, có những chỗ nó chưa thắng và lý do là cấu trúc chứ không phải tạm thời, và có một chỗ nó *ên* đóng góp nhiều mà hiện đang đóng góp ít. Nội dung đi theo ba nhóm ấy. Nhóm một — front-end, prior độ sâu, place recognition — là nơi học máy thay một module có giao diện hẹp trong khi giữ nguyên back-end hình học, và đó là lý do nó thắng. Nhóm hai — SLAM đầu-cuối — là nơi nó chưa thắng, và mục 3 phân tích vì sao lý do là cấu trúc. Nhóm ba là bất định: đây là chỗ mà một mô hình học được *có thể* đóng góp nhiều nhất — học ra hiệp phương sai từ dữ liệu thay vì đoán — và là chỗ hiện có ít công trình nhất. Nếu chỉ nhớ một điều: *front-end học được cộng back-end hình học là hướng đúng*, và lý do không phải là thoả hiệp mà là phân công theo thế mạnh.

KIẾN THỨC CẦN CÓ TRƯỚC

- **front-end học được** — thay thế detector, descriptor hoặc matcher bằng một mạng học từ dữ liệu, giữ nguyên phần ước lượng hình học phía sau.
- **SLAM đầu-cuối (end-to-end)** — học một mạng ánh xạ trực tiếp từ chuỗi ảnh sang quỹ đạo, không có bước hình học tường minh nào.
- **BA khả vi (differentiable bundle adjustment)** — cài đặt bundle adjustment sao cho gradient chảy được qua nó, cho phép huấn luyện các thành phần phía trước bằng lan truyền ngược qua bộ giải.
- **foundation model 3D** — mô hình lớn huấn luyện trên dữ liệu đa dạng, cho ra cấu trúc 3D hoặc tương ứng trực tiếp từ ảnh mà không cần hiệu chuẩn hay tư thế cho trước.
- **hiệu chuẩn bất định (uncertainty calibration)** — tính chất mà một mô hình dự đoán kèm mức tin cậy *đúng* — khi nó nói “90% chắc” thì nó đúng 90% số lần. Phần lớn mạng nơ-ron không có tính chất này mà không được sửa.
- **ngoài phân bố (out-of-distribution)** — dữ liệu khác với dữ liệu huấn luyện. Mọi mô hình học được đều suy giảm ở đây, và vấn đề thật là chúng thường suy giảm *một cách im lặng*.
- **semantic SLAM** — đưa thông tin loại đối tượng vào bài toán ước lượng — landmark không chỉ là một điểm mà là “một góc của cái bàn”.

CÂU HỎI MỞ ĐẦU

- Q1. Học máy thắng rõ ở front-end nhưng chưa thắng ở back-end. Nêu một lý do *cấu trúc* cho sự bất đối xứng đó — không phải “vì chưa ai làm tốt”.
- Q2. DROID-SLAM dùng mạng cho luồng quang và một bộ giải BA ở giữa vòng huấn luyện. Nêu điều mà kiến trúc này giữ lại từ thế giới hình học, và vì sao việc giữ lại đó quan trọng.

- Q3.** Một mô hình độ sâu cho ước lượng ở mọi pixel. Nếu điều nguy hiểm nhất về đầu ra của nó trong ngữ cảnh SLAM, và nói vì sao nó khác với một cảm biến độ sâu kém chính xác.
- Q4.** Nếu bạn được chọn *một* chỗ để áp dụng học máy vào một hệ SLAM sản phẩm đang chạy, chọn chỗ nào và vì sao?
- Q5.** Mô hình của bạn đạt kết quả tốt trên mọi benchmark công khai. Nếu ba câu hỏi phải trả lời trước khi đưa nó vào sản phẩm, và nói câu nào khó nhất.

17.1 Đặt vấn đề

Trong mười năm, học máy đã thay thế các phương pháp thủ công ở gần như mọi bài toán thị giác: phân loại, phát hiện, phân đoạn, sinh ảnh. Câu hỏi tự nhiên: vì sao SLAM vẫn chủ yếu là hình học?

Câu trả lời hời hợt là “vì chưa ai làm tốt”. Câu trả lời thật thú vị hơn, và nó nằm ở *bản chất của bài toán*.

Các bài toán mà học máy thắng có chung một đặc điểm: chúng là bài toán *nhận thức* — ánh xạ từ dữ liệu cảm giác sang một nhãn hoặc một mô tả, nơi quan hệ đúng là phức tạp, khó viết ra, và chỉ học được từ ví dụ. “Con mèo trông thế nào” không có công thức.

SLAM thì khác. Phần lõi của nó — cho các tương ứng, tìm tư thế và cấu trúc — là một bài toán *ước lượng* có một mô hình toán *đúng* và *đã biết*. Phép chiếu phối cảnh không phải thứ cần học từ dữ liệu; nó là hình học, đã đúng từ thế kỷ mười lăm. Một mạng học lại nó từ ví dụ sẽ học được một xấp xỉ tệ hơn, và nó chỉ đúng trong phạm vi dữ liệu huấn luyện — đúng như trường hợp APR ở B/12 mục 3B.

Nhưng SLAM *không chỉ* là bài toán ước lượng. Nó có một phần nhận thức rất lớn, và đó chính là nơi mọi khó khăn thực tế nằm: pixel nào tương ứng pixel nào, ảnh này có phải chỗ tôi đã đi qua không, vật kia có đang chuyển động không, độ sâu ở vùng trơn này khoảng bao nhiêu. Không câu nào trong số đó có công thức.

Từ đó ra một cách phân chia rõ ràng, và nó là luận điểm của chương: *học máy nên làm phần nhận thức, hình học nên làm phần ước lượng*. Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và nó là một lý do cấu trúc chứ không phải một quan sát tạm thời.

CẠM BÃY

Chương này chốt tại tháng 9/2026 và cùng với C/16 là phần hết hạn nhanh nhất của cây. Mọi đánh giá về mức trưởng thành dưới đây là *phán đoán của tôi*, không phải kết quả đo, và nên được rà lại sau sáu tháng.

Tôi cũng **không dẫn con số benchmark** cho bất kỳ so sánh nào trong chương. Lý do đã nêu ở B/07: các con số này phụ thuộc mạnh vào bộ dữ liệu và cách đo, và trong lĩnh vực học máy còn thêm một vấn đề — các baseline hình học thường được cài lại bởi nhóm đề xuất phương pháp học được, và theo *research-rules.md* mục 4 thì đó là một cờ đỏ cần cảnh giác.

Điều tôi đưa ra là phân tích về *cơ chế*: mỗi cách tiếp cận giữ lại gì từ hình học, bỏ đi gì, và điều đó có hệ quả gì. Phân tích ấy bền hơn con số.

TRỰC GIÁC

Hình dung việc đo một căn phòng bằng thước dây, và bạn có một người trợ lý. Có những việc người trợ lý làm tốt hơn bạn: *nhận ra* cái góc tường ở ảnh này chính là cái góc tường ở ảnh kia, dù ánh sáng khác và góc nhìn khác. Đó là việc nhận thức — nó đòi kinh nghiệm, và không có quy tắc nào viết ra được.

Và có những việc bạn *không* nên nhờ trợ lý: cộng các số đo lại. Phép cộng có quy tắc đúng, đã biết, và không có lý do gì để đoán nó từ kinh nghiệm. Một người trợ lý giỏi “ước lượng tổng” vẫn thua một cái máy tính bỏ túi.

Đó là toàn bộ sự phân công trong chương này. Học máy là người trợ lý có kinh nghiệm: giao cho nó việc nhận ra, việc phán đoán, việc điền vào chỗ trống. Hình học là cái máy tính: giao cho nó việc tính.

Và có một chuyện thứ ba, quan trọng hơn cả hai. Người trợ lý tốt không chỉ nói “cái góc này khớp với cái góc kia” — anh ta nói thêm “tôi khá chắc” hoặc “cái này tôi đoán thôi”. Câu thứ hai đáng giá hơn nhiều so với vẻ ngoài, vì nó cho bạn biết nên tin điều gì.

Phần lớn các mô hình học được hiện nay là những người trợ lý *không bao giờ nói câu thứ hai*. Họ trả lời mọi câu hỏi với cùng một giọng tự tin, kể cả những câu họ chưa từng gặp. Và đó là vấn đề lớn nhất khi đưa họ vào một hệ thống phải tự biết mình sai.

17.2 Nơi học máy đã thắng**A. Front-end**

Đây là chỗ rõ ràng nhất, và là câu trả lời cho câu hỏi mở đầu thứ tư.

SuperPoint [49], SuperGlue [50], LightGlue [51], LoFTR [52] — tất cả thay một module với *giao diện hẹp*: vào là hai ảnh, ra là một tập tương ứng. Mọi thứ phía sau không đổi.

Ba lý do vì sao đây là chỗ thắng, và chúng giải thích cả vì sao nó dễ áp dụng.

Một — đây thật sự là bài toán nhận thức. “Hai mảng ảnh này có phải cùng một điểm vật lý không” không có công thức; nó phụ thuộc vào việc bề mặt trông thế nào từ các góc khác nhau dưới các ánh sáng khác nhau, và đó là kiến thức thống kê về thế giới.

Hai — giao diện hẹp và kiểm được. Đầu ra là một tập tương ứng, và mọi tương ứng đều đi qua RANSAC và kiểm định hình học phía sau (B/07 mục 5). Nghĩa là một mô hình sai không phá hệ thống ngay — nó chỉ làm tỉ lệ inlier giảm, và tăng hình học bất được.

Ba — thay thế được từng phần. Đổi detector mà giữ nguyên matcher, hoặc ngược lại. Rủi ro triển khai thấp.

Cái giá đã nêu ở B/07 mục 2B và vẫn đúng: cần GPU, khó dự đoán khi ra ngoài phân bố, và mất tính tất định. Ba điều đó là lý do sản phẩm còn do dự chứ không phải vì chất lượng.

B. Prior độ sâu và place recognition

Hai chỗ thắng nữa với cùng cấu trúc lập luận.

Độ sâu học được (B/08 mục 4C, C/16 mục 2B) giải quyết được vùng không có vân, nơi hình học không có thông tin. Điều kiện dùng an toàn đã nêu: khởi tạo chứ không ràng buộc, và hiệp phương sai phải phản ánh sự thật.

Place recognition học được (B/11 mục 2C) bền hơn nhiều với thay đổi vẻ ngoài — mùa, ngày đêm. Đây rõ ràng là bài toán nhận thức, và các descriptor thủ công không có cách nào bắt được “cùng một chỗ, mùa khác”.

Điểm chung của cả ba chỗ thắng, và đáng nêu thành nguyên tắc: *học máy thắng ở nơi bài toán không có mô hình đúng, và nơi đầu ra được kiểm bởi một tầng hình học phía sau*. Hai điều kiện ấy đi cùng nhau.

17.3 Nơi nó chưa thắng, và vì sao

A. SLAM đầu-cuối

Ý tưởng: học một mạng ánh xạ từ chuỗi ảnh sang quỹ đạo, bỏ hết hình học.

Lập luận vì sao khó, và nó là lý do cấu trúc chứ không phải kỹ thuật.

Một — *nó phải học lại thứ đã biết*. Phép chiếu phối cảnh, hợp thành tư thế, ràng buộc epipolar — tất cả đều là toán đúng. Một mạng học chúng từ dữ liệu sẽ học một xấp xỉ, và xấp xỉ ấy chỉ đúng trong phạm vi dữ liệu huấn luyện. Trường hợp APR ở B/12 mục 3B là minh hoạ rõ nhất: mô hình nội suy giữa các tư thế huấn luyện thay vì học hình học.

Hai — *không có gauge*. SLAM có bảy bậc tự do gauge (A/05); một mạng đầu-cuối phải chọn một quy ước, và nó chọn quy ước của tập huấn luyện. Điều đó nghĩa là nó không tổng quát hoá sang môi trường có thang độ khác.

Ba — *không có hiệp phương sai có nghĩa*. Đầu ra là một tư thế, không phải một phân bố. Với A/06 là một chương của cây, đây là một thiếu sót nghiêm trọng cho ứng dụng.

Bốn — *không mở rộng được về thời gian*. Một mạng không có cơ chế nào tương đương với loop closure: nó không giữ một bản đồ có thể được sửa lại khi có thông tin mới.

B. Kiến trúc lai: DROID-SLAM

DROID-SLAM [92] là ví dụ tốt nhất về cách làm đúng, và nó là câu trả lời cho câu hỏi mở đầu thứ hai.

Kiến trúc: một mạng lập dự đoán luồng quang dày giữa các khung, và một lớp *bundle adjustment* lấy các luồng ấy làm phép đo rồi giải cho tư thế và độ sâu. Lớp BA nằm trong vòng lặp, và nó khả vi nên gradient chảy ngược qua nó để huấn luyện mạng.

Điều kiện trúc này giữ lại từ thế giới hình học — và đây là phần quan trọng: *ràng buộc hình học vẫn được áp đặt cứng*. Mạng không được phép đề xuất một quỹ đạo vi phạm phép chiếu; nó chỉ đề xuất *phép đo*, và hình học quyết định quỹ đạo. Nghĩa là mọi tính chất đúng đắn của BA — tính nhất quán hình học, khả năng dùng hiệp phương sai, khả năng thêm ràng buộc từ cảm biến khác — đều được giữ.

Vì sao việc giữ lại đó quan trọng: nó là thứ cho phép hệ thống *tổng quát hoá* ra ngoài dữ liệu huấn luyện theo hướng hình học. Mạng có thể chưa từng thấy một chuyển động như thế này, nhưng nếu nó cho luồng quang đúng thì BA cho tư thế đúng — vì BA không cần học.

Đây là lập luận chính của chương, phát biểu ở dạng cụ thể: *đặt học máy ở chỗ sinh ra phép đo, đặt hình học ở chỗ biến phép đo thành trạng thái*.

C. Foundation model 3D

DUST3R [93], MAST3R [94], VGGT [95] đi theo một hướng khác và đáng chú ý: cho hai hoặc nhiều ảnh bất kỳ, trực tiếp cho ra cấu trúc 3D và tương ứng, *không cần* hiệu chuẩn hay tư thế cho trước.

Đây là một thay đổi thật về cách nghĩ, vì nó xoá bỏ giả định mà cả A/01 và A/02 dựa vào — rằng ta biết tham số nội. Với ảnh thu thập từ nguồn không rõ, đó là một lợi ích lớn.

Đánh giá của tôi tại 9/2026: rất hứa hẹn cho SfM và cho tái tạo từ ảnh không hiệu chuẩn; *chưa* là lựa chọn cho SLAM thời gian thực trên thiết bị, vì chi phí tính toán và vì thiếu cơ chế cho hoạt động dài hạn (bản đồ, loop closure, quản lý). Cách dùng thực dụng nhất hiện nay có lẽ là làm *bộ khởi tạo* và làm công cụ dựng bản đồ ngoại tuyến, rồi một hệ hình học nhẹ chạy localization trên bản đồ đó. (*Đây là phán đoán của tôi, chưa đối chiếu với một khảo sát có hệ thống.*)

17.4 Nơi nó nên đóng góp mà chưa

Đây là mục mà tôi cho là có giá trị cao nhất của chương, và là câu trả lời cho câu hỏi mở đầu thứ ba.

A. Bất định là chỗ trống

A/06 lập luận rằng hiệp phương sai đáng tin là thứ SLAM thiếu một cách hệ thống, và rằng bốn nguồn của bệnh quá tự tin đều khó chữa bằng phân tích.

Đây chính là loại bài toán mà học máy *nên* giải: ước lượng một phân bố từ dữ liệu, khi ta không viết được mô hình giải tích cho nó. “Khớp điểm này đáng tin tới đâu, cho loại cảnh này, dưới ánh sáng này” không có công thức — nhưng có rất nhiều dữ liệu.

Vậy mà số công trình về bất định học được trong SLAM ít hơn nhiều so với số công trình về front-end học được. (*Đây là ấn tượng của tôi từ việc đọc rải rác, không phải kết quả của một khảo sát có hệ thống — và làm khảo sát ấy là một mục trong 99-chưa-biết.md.*)

Tôi cho rằng có hai lý do. *Một*: khó đo. Để biết một ước lượng bất định có đúng không, cần chân trị và cần thống kê — tức cần NEES (A/06 mục 4A), và đó là một phép đo mà cộng đồng ít dùng. Không có chỉ số dễ đo thì không có tiến bộ dễ công bố. *Hai*: ít hấp dẫn. Một bài báo cải thiện ATE dễ đăng hơn một bài báo cải thiện tính trung thực của hiệp phương sai, dù cái thứ hai có giá trị ứng dụng cao hơn.

B. Vấn đề hiệu chuẩn bất định

Có một khó khăn thật, không chỉ là thiếu quan tâm, và nó đáng nêu rõ.

Mạng nơ-ron nổi tiếng là *không được hiệu chuẩn* về mặt xác suất: khi một mô hình phân loại nói “99% chắc”, nó thường đúng ít hơn 99% số lần. Với SLAM, hệ quả nghiêm trọng hơn so với với phân loại, vì hiệp phương sai không chỉ được *đọc* mà được *dùng làm trọng số* — một hiệp phương sai quá nhỏ kéo cả nghiệm đi (B/08 khỏi *Cạm bẫy*).

Và vấn đề gay gắt nhất: bất định phải đúng ngoài *phân bố huấn luyện*, vì đó chính là lúc ta cần nó nhất. Một mô hình biết mình không chắc khi gặp cảnh lạ là đúng thứ ta muốn — và đó cũng là thứ khó nhất để huấn luyện, vì theo định nghĩa ta không có dữ liệu huấn luyện cho “cảnh lạ”.

Đây là câu trả lời cho câu hỏi mở đầu thứ ba: điều nguy hiểm nhất về đầu ra của một mô hình độ sâu không phải nó sai, mà là nó *sai với vẻ chắc chắn như nhau ở mọi pixel*. Một cảm biến độ sâu kém chính xác vẫn tốt hơn, vì sai số của nó có cấu trúc đã biết — nó tệ ở xa, tệ trên bề mặt bóng, và ta mô hình hoá được điều đó. Mô hình học được không cho ta cấu trúc ấy.

C. Semantic SLAM: một hướng khác

Đưa thông tin loại đối tượng vào bài toán: landmark không chỉ là một điểm mà là “một góc của cái bàn”. CubeSLAM [96] và QuadricSLAM [97] biểu diễn đối tượng bằng các hình đơn giản và đưa chúng vào đồ thị như landmark cấp cao.

Ba lợi ích tiềm năng. *Data association tốt hơn*: hai điểm thuộc hai đối tượng khác loại chắc chắn không phải cùng một điểm — một ràng buộc mạnh và miễn phí. *Xử lý vật động*: biết cái gì là xe thì biết cái gì có thể di chuyển (D/19). *Bản đồ ổn định hơn*: các đối tượng cố định (tường, cột) đáng tin hơn các đối tượng di động (ghế), và bản đồ nên phản ánh điều đó — một hướng tấn công thật vào bài toán lifelong ở C/13 mục 4C.

Đánh giá tại 9/2026: lợi ích thứ ba là lợi ích lớn nhất và ít được khai thác nhất. Phần lớn công trình semantic SLAM tập trung vào việc dựng bản đồ có nhãn, trong khi giá trị thực dụng cao hơn nằm ở việc dùng nhãn để *quyết định tin cái gì*. (Phán đoán của tôi.)

CẦN NHỚ

Học máy làm phần nhận thức, hình học làm phần ước lượng — và ranh giới giữa hai phần là chỗ *phép đo* được sinh ra.

Cụ thể: đặt mô hình học được ở chỗ trả lời các câu không có công thức — pixel nào khớp pixel nào, ảnh này có phải chỗ cũ không, độ sâu ở vùng trơn này khoảng bao nhiêu, điểm này có nằm trên vật động không. Đặt hình học ở chỗ biến các câu trả lời ấy thành trạng thái.

Ba lý do vì sao ranh giới này đúng, không phải một thoả hiệp. *Một*: phần ước lượng có mô hình đúng và đã biết, nên học nó từ dữ liệu chỉ cho một xấp xỉ tệ hơn. *Hai*: giữ hình học nghĩa là giữ được hiệp phương sai, gauge, khả năng thêm cảm biến, và loop closure — bốn thứ mà một mạng đầu-cuối mất hết. *Ba*: đầu ra của mô hình học được đi qua một tầng kiểm định hình học, nên một mô hình sai làm giảm chất lượng chứ không phá hệ thống. Và một điều kiện bắt buộc mà hiện ít được đáp ứng: **mô hình phải nói được nó chắc tới đâu**, và con số ấy phải được hiệu chuẩn trên dữ liệu thật. Không có nó, phép đo do mô hình sinh ra không đưa vào đồ thị một cách hợp lệ được — xem B/08 mục 4C.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
SuperPoint + LightGlue [51]	Front-end học được, giao diện hẹp	Mục 2A — chỗ dễ áp dụng nhất và lợi ích rõ nhất
DROID-SLAM [92]	Luồng học được + lớp BA khả vi trong vòng lặp	Mục 3B — kiến trúc lai làm đúng; đọc để thấy ranh giới đặt ở đâu
DPVO [98]	Phiên bản nhẹ hơn nhiều của cùng ý tưởng	Cho thấy chi phí có thể giảm mạnh mà giữ kiến trúc
MASt3R [94], VGGT [95]	Foundation model 3D không cần hiệu chuẩn	Mục 3C — thay đổi cách nghĩ về SfM; chốt 9/2026
Theseus [99]	Thư viện tối ưu phi tuyến khả vi	Công cụ để xây kiến trúc lai; hạ tầng cho mục 3B
DynaSLAM [100]	Phân đoạn ngữ nghĩa để loại vật động	Ứng dụng thực dụng nhất của semantic; xem D/19

MINI PROJECT SLAM · Thay front-end bằng bản học được, rồi đo cái mà benchmark không đo · 3 ngày

Mục tiêu	đo lợi ích thật của front-end học được trên dữ liệu của bạn, và đo cái giá mà bảng so sánh không nhắc — hành vi ngoài phân bố và tính tất định.
Dữ liệu	một chuỗi chuẩn (EuRoC), cộng <i>ba</i> chuỗi tự thu cổ ý ngoài phân bố: một chuỗi ánh sáng rất kém, một chuỗi trong môi trường khác hẳn dữ liệu huấn luyện phổ biến (nhà kho công nghiệp, ngoài trời ban đêm), và một chuỗi có chuyển động rất nhanh.
Các bước	chạy cùng một hệ SLAM với hai front-end — ORB và SuperPoint+LightGlue — trên cả bốn chuỗi; đo ATE, tỉ lệ tracking, thời gian mỗi khung; rồi chạy <i>mỗi cấu hình hai lần trên cùng dữ liệu</i> và kiểm xem kết quả có bit-exact không.
Xong khi	bạn có một bảng hai front-end nhân bốn chuỗi với bốn cột. Ba điều cần quan sát: (1) trên chuỗi chuẩn, học được thắng hoặc hoà — đúng như mong đợi; (2) trên các chuỗi ngoài phân bố, xem nó thắng ở đâu và <i>thua ở đâu</i> — đây là phần benchmark không cho bạn; (3) kiểm tính tất định — nếu hai lần chạy cho kết quả khác nhau, bạn vừa phát hiện một rào cản triển khai thật mà không bài báo nào nhắc tới (D/20).
Nối với	D/19-ben-vung-trong-the-gioi-thuc cho danh mục điều kiện khó đầy đủ hơn; D/21-tu-prototype-len-mass-production bàn về chi phí cập nhật mô hình và tái kiểm định.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Lý do cấu trúc: phần lõi của SLAM là bài toán *ước lượng* có một mô hình toán *đúng và đã biết* — phép chiếu phối cảnh không phải thứ cần học từ dữ liệu. Một mạng học lại nó sẽ học một xấp xỉ chỉ đúng trong phạm vi dữ liệu huấn luyện, như trường hợp APR ở B/12. Front-end thì ngược lại: “hai mảng ảnh này có phải cùng một điểm không” là bài toán *nhận thức*, không có công thức, phụ thuộc vào kiến thức thống kê về việc bề mặt trông thế nào từ các góc và ánh sáng khác nhau. Học máy thắng ở nơi không có mô hình đúng, và thua ở nơi có.
- A2. Nó giữ lại việc *áp đặt cứng các ràng buộc hình học*: mạng chỉ được đề xuất *phép đo* (luồng quang), còn lớp BA quyết định quỹ đạo. Mạng không được phép đề xuất một quỹ đạo vi phạm phép chiếu. Việc giữ lại đó quan trọng vì nó cho phép hệ tổng quát hoá theo hướng hình học ra ngoài dữ liệu huấn luyện — mạng có thể chưa từng thấy chuyển động này, nhưng nếu nó cho luồng đúng thì BA cho tư thế đúng, vì BA không cần học. Nó cũng giữ được hiệp phương sai, gauge, khả năng thêm cảm biến và loop closure — bốn thứ mà mạng đầu-cuối mất hết.
- A3. Điều nguy hiểm nhất là nó *sai với vẻ chắc chắn như nhau ở mọi pixel* — kể cả ở những pixel mà hình học không nói gì và mô hình chỉ đang nội suy từ tiên nghiệm. Nó khác với một cảm biến độ sâu kém chính xác ở chỗ sai số của cảm biến có *cấu trúc đã biết*: nó tệ ở xa theo z^2 , tệ trên bề mặt bóng, và ta mô hình hoá được điều đó rồi đặt trọng số tương ứng. Mô hình học được không cho ta cấu trúc ấy, nên ta không biết đặt trọng số thế nào — và với ứng dụng an toàn, một bản đồ trộn giữa phép đo và phỏng đoán mà không phân biệt được hai phần là nguy hiểm.
- A4. *Front-end* — cụ thể là matcher. Ba lý do: (1) đây thật sự là bài toán nhận thức nên lợi ích lớn; (2) giao diện hẹp — vào hai ảnh, ra một tập tương ứng — nên thay được mà không đụng gì khác; (3) đầu ra đi qua RANSAC và kiểm định hình học, nên một mô hình kém làm giảm tỉ lệ inlier chứ không phá hệ thống. Rủi ro triển khai thấp nhất trên mỗi đơn vị lợi ích. Điều kiện: phải có ngân sách GPU, và phải chấp nhận mất tính tất định — hai cái giá thuộc về D/20 chứ không phải về chất lượng.
- A5. Ba câu: (1) nó hành xử thế nào ngoài phân bố huấn luyện, và nó có *biết* mình đang ở ngoài không? (2) nó có *tất định* không — cùng đầu vào cho cùng đầu ra bit-exact, điều kiện để tái hiện một lỗi từ hiện trường? (3) *chi phí cập nhật là bao nhiêu* — đổi mô hình đòi tái kiểm định những gì? Câu (1) khó nhất, vì theo định nghĩa ta không có dữ liệu để kiểm “cảnh lạ”, và vì việc huấn luyện một mô hình *biết* mình không chắc khi gặp cảnh lạ là bài toán chưa giải. Điểm số tốt trên benchmark công khai không trả lời câu nào trong ba câu ấy.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đo một căn phòng bằng thước dây và có một người trợ lý giàu kinh nghiệm.

Có việc anh ta làm giỏi hơn bạn: nhận ra cái góc tường trong tấm ảnh này chính là cái góc tường trong tấm ảnh kia, dù chụp lúc khác, sáng khác, đứng ở chỗ khác. Việc đó không có quy tắc — nó là kinh nghiệm.

Và có việc bạn không nên nhờ anh ta: cộng các con số lại. Phép cộng có quy tắc đúng, đã biết từ lâu.

Một người “ước lượng tổng” giỏi đến mấy vẫn thua cái máy tính bỏ túi.

Đó là toàn bộ cách phân công. Giao việc nhận ra cho người có kinh nghiệm; giao việc tính cho cái máy tính. Và ranh giới giữa hai việc nằm đúng ở chỗ con số được sinh ra: người trợ lý đọc ra các con số, cái máy tính xử lý chúng.

Nhưng có một chuyện thứ ba, và nó quan trọng hơn cả hai chuyện trên.

Một người trợ lý tốt không chỉ nói “góc này khớp góc kia”. Anh ta nói thêm: “cái này tôi chắc” hoặc “cái này tôi đoán thôi, ánh sáng tối quá”. Câu thứ hai đáng giá hơn nhiều so với vẻ ngoài, vì nó cho bạn biết nên tin đến đâu — và nên đi kiểm lại chỗ nào.

Phần lớn các mô hình học máy hiện nay là những người trợ lý *không bao giờ nói câu thứ hai*. Họ trả lời mọi câu với cùng một giọng chắc chắn, kể cả những câu họ chưa từng gặp bao giờ. Bạn đưa cho họ một tấm ảnh chụp trong một nhà kho mà họ chưa từng thấy loại nào tương tự, và họ vẫn trả lời rành rọt.

Đó là vấn đề lớn nhất khi đưa họ vào một cái máy phải tự biết mình sai. Và điều trở trêu là: dạy máy *nói không chắc* đúng chỗ lại là thứ mà máy học lẽ ra nên giỏi — ước lượng mức tin cậy từ dữ liệu là đúng loại việc của nó. Chỉ là chưa mấy ai làm.

Chỗ không rút gọn được: chỗ không rút gọn được là việc dạy một mô hình biết mình đang gặp thứ chưa từng thấy. Theo định nghĩa, bạn không có dữ liệu huấn luyện cho “những thứ chưa từng thấy”. Mọi cách tiếp cận hiện có đều là cách vòng quanh vấn đề ấy, và không cách nào thoả đáng.

Một câu: *Giao việc nhận ra cho cái học được, giao việc tính cho cái đã biết — và đòi cái học được phải nói rõ khi nó chỉ đang đoán.*

17.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *học máy nên làm nhận thức, hình học nên làm ước lượng* — bị bác bỏ nếu một hệ đầu-cuối thuần túy vượt một hệ lai trên các chuỗi *ngoài phân bố huấn luyện*, với cùng ngân sách tính toán. Điều kiện “ngoài phân bố” là chỗ phải kiểm nghiệm khác, vì trong phân bố thì một mạng đủ lớn có thể thắng mà không chứng minh gì về khả năng tổng quát hoá. Tôi cho là mệnh đề vững, nhưng ranh giới *đang dịch* — các foundation model ở mục 3C hấp thụ ngày càng nhiều phần hình học vào mô hình, và tôi không chắc phát biểu này còn nguyên vẹn sau vài năm.

Mệnh đề thứ hai — *bất định học được là chỗ trống* — dựa trên ấn tượng đọc rải rác chứ không trên khảo sát, và tôi đã ghi rõ điều đó ở mục 4A. Nó bị bác bỏ đơn giản bằng cách chỉ ra một nhánh công trình đáng kể mà tôi bỏ sót. Đây là mục có rủi ro sai cao nhất trong chương, và việc làm khảo sát cho nó nằm trong 99-chua-biet.md.

Mệnh đề thứ ba — *mạng nơ-ron không được hiệu chuẩn về xác suất* — là một kết quả đã biết rộng rãi trong cộng đồng học máy, nhưng tôi *không* dẫn được một nguồn cụ thể trong ngữ cảnh SLAM, và tôi không biết mức độ nghiêm trọng của nó cho các mô hình độ sâu và luồng quang cụ thể đang dùng. Đây là một khoản nợ kiểm chứng.

Mệnh đề thứ tư — đánh giá về mức trưởng thành của foundation model tại 9/2026 — là phán đoán và sẽ hết hạn. Rà lại sau sáu tháng.

17.6 Kết nối

Nối lên trên. [B/07-dac-trung-va-data-association](#) là chỗ front-end học được được bàn ở mức chi tiết, cùng cái giá của nó. [B/08-mo-hinh-phan-du](#) mục 4C cho ba điều kiện đưa một prior học được vào đồ thị, và chúng áp dụng nguyên vẹn cho mọi nguồn học được. [B/12-relocalization](#) mục 3B cho trường hợp APR, và bài học phương pháp ở đó — so với một đường cơ sở tầm thường — nên áp dụng cho mọi khẳng định trong chương này. [A/06-bat-dinh-va-lan-truyen-sai-so](#) là chương mà mục 4 nối vào: bất định là chỗ trống, và lý do nó là chỗ trống nằm ở chỗ nó khó đo.

Nối ngang. [C/16-dense-mapping-va-bieu-dien](#) chia sẻ đặc điểm hết hạn nhanh, và hai chương nên được rà cùng nhau. [C/18-danh-gia-va-phuong-phap-luan](#) cho công cụ để đánh giá một thành phần học được một cách có kỷ luật — đặc biệt là yêu cầu về baseline và về đánh giá ở đuôi phân bố.

Nối xuống dưới. [D/19-ben-vung-trong-the-gioi-thuc](#) là chỗ semantic segmentation có ứng dụng thực dụng nhất: loại vật động. [D/20-toi-uu-hieu-nang-va-he-thong](#) bàn về chi phí tính toán và về tính tất định — hai cái giá lớn nhất của front-end học được trong sản phẩm. [D/21-tu-prototype-len-mass-production](#) bàn về chi phí cập nhật mô hình, tái kiểm định, và OTA cho mô hình — những thứ không tồn tại với một thuật toán thủ công. [D/22-huong-nghien-cuu-con-trong](#) lấy “front-end học được + back-end hình học với bất định được truyền đúng” làm hướng số năm, và lý do nằm trọn trong mục 4 của chương này.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu lý do *cấu trúc* vì sao học máy thắng ở front-end mà chưa thắng ở back-end.
2. Ba điều kiện chung của các chỗ mà học máy đã thắng là gì, và vì sao chúng đi cùng nhau?
3. DROID-SLAM giữ lại gì từ hình học, và điều đó cho hệ thống khả năng gì mà một mạng đầu-cuối không có?
4. Nêu bốn thứ mà một hệ SLAM đầu-cuối mất so với một hệ hình học.
5. Vì sao bất định là chỗ mà học máy *ên* đóng góp nhiều nhất, và nêu hai lý do vì sao hiện nó đóng góp ít?
6. Vì sao việc huấn luyện một mô hình “biết mình không chắc khi gặp cảnh lạ” là bài toán khó về mặt nguyên tắc?
7. Nêu ba câu hỏi phải trả lời trước khi đưa một mô hình học được vào sản phẩm, và nói vì sao benchmark không trả lời câu nào.

Đánh giá và phương pháp luận (Evaluation and Methodology)

PHỤ THUỘC A/05-observability-gauge-va-suy-bien (số bậc tự do gauge quyết định căn chỉnh mấy bậc — căn sai là che mất lỗi); A/06-bat-dinh-va-lan-truyen-sai-so (NEES và bệnh quá tự tin); B/11-loop-closure-va-nhat-quan-toan-cuc và B/12-relocalization (đánh giá có chi phí lỗi bất đối xứng).

TÓM TẮT

Mọi chương trước nợ chương này một câu trả lời: *làm sao biết một thay đổi thật sự là cải tiến?* Câu hỏi nghe đơn giản và nó bị trả lời sai một cách có hệ thống trong cả lĩnh vực lẫn trong thực hành công nghiệp. Nội dung có bốn phần. Một: các chỉ số quỹ đạo — ATE và RPE — với một chi tiết quyết định mà nhiều người làm sai: *căn chỉnh mấy bậc tự do*, và căn sai thì che mất đúng thứ ta muốn đo. Hai: đo *tính trung thực* chứ không chỉ độ chính xác, tức NEES — một phép đo rẻ mà gần như không ai làm. Ba: đánh giá các thành phần có chi phí lỗi bất đối xứng (loop closure, relocalization), nơi một con số trung bình là vô nghĩa và đường cong precision-recall ở vùng precision cao mới là thứ đáng nhìn. Bốn: kỷ luật phương pháp — ablation, mô phỏng, nhiều seed, và nguyên tắc quan trọng nhất của cả chương: *đánh giá ở đuôi phân bố, không ở trung bình*. Nếu chỉ nhớ một điều: *một hệ tốt hơn về trung bình nhưng tệ hơn ở đuôi là một hệ tệ hơn* — vì sản phẩm hỏng ở đuôi, không hỏng ở trung bình.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ATE** — *absolute trajectory error* — sai số vị trí giữa quỹ đạo ước lượng và chân trị sau khi căn chỉnh hai quỹ đạo. Đo sai số *toàn cục*, nên một loop closure tốt cải thiện nó nhiều.
- **RPE** — *relative pose error* — sai số của phép biến đổi tương đối trên các đoạn có độ dài cố định. Đo *drift cục bộ*, không phụ thuộc vào việc có đóng vòng hay không.
- **căn chỉnh (alignment)** — phép biến đổi áp lên quỹ đạo ước lượng để đưa nó về cùng hệ với chân trị, trước khi tính sai số. Số bậc tự do của phép biến đổi này *phải* bằng số bậc gauge của hệ.
- **ablation study** — thí nghiệm tắt từng thành phần một để đo đóng góp của nó. Chỉ hợp lệ khi mọi thứ khác được giữ nguyên và khi có đủ số lần chạy.
- **đuôi phân bố** — các trường hợp xấu nhất — phân vị 95, 99, hoặc trường hợp hỏng hoàn toàn. Đây là nơi hệ thống sản phẩm thất bại, và là nơi trung bình không nói gì.
- **ý nghĩa thống kê** — mức tin cậy rằng một khác biệt quan sát được không phải do ngẫu nhiên. SLAM có nguồn ngẫu nhiên thật (RANSAC, thứ tự luông), nên một lần chạy không chứng minh gì.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn đánh giá một hệ VIO bằng cách căn chỉnh $Sim(3)$ với chân trị rồi tính ATE — cách làm phổ biến. Vì sao đó là một sai lầm, và nó che mất điều gì cụ thể?
- Q2. Hệ thống mới của bạn có ATE trung bình thấp hơn hệ cũ 15% trên mười chuỗi. Nêu ba lý do vì sao điều đó vẫn chưa chứng minh nó tốt hơn.
- Q3. Đo NEES cần chân trị, nên nó chỉ làm được trong mô phỏng. Vì sao điều đó *không* phải lý do để bỏ qua nó, và nó cho biết gì mà ATE không cho biết?

- Q4.** Bạn so hệ của mình với một baseline mà bạn tự cài đặt, và bạn thắng. Nêu phép kiểm bắt buộc trước khi tin kết quả đó.
- Q5.** Hệ A có ATE trung bình 0,08 m và thất bại hoàn toàn trên 2% số lần chạy. Hệ B có ATE trung bình 0,12 m và chưa bao giờ thất bại. Hệ nào tốt hơn, và câu trả lời phụ thuộc vào gì?

18.1 Đặt vấn đề

Mười bảy chương trước đưa ra hàng chục lựa chọn thiết kế: detector nào, phần dư nào, kiến trúc back-end nào, chính sách bản đồ nào. Mỗi lựa chọn được biện minh bằng lập luận về cơ chế. Không lựa chọn nào được biện minh bằng số liệu — và đó là có chủ ý, vì chương này giải thích vì sao các số liệu phổ biến thường không biện minh được gì.

Bài toán gốc: *cho hai phiên bản của một hệ thống, làm sao biết cái nào tốt hơn?*

Nghe đơn giản. Nó khó vì bốn lý do chồng lên nhau.

Một — “tốt hơn” không phải một chiều. Một hệ có thể chính xác hơn nhưng chậm hơn, hoặc chính xác hơn ở trung bình nhưng hỏng nhiều hơn ở đuôi, hoặc chính xác hơn trong nhà và tệ hơn ngoài trời.

Hai — chân trị khó có. Hệ đo ngoài (motion capture) chỉ có trong phòng lab và chỉ trong một thể tích nhỏ. Ngoài đó, ta không biết sự thật.

Ba — SLAM có tính ngẫu nhiên thật. RANSAC lấy mẫu ngẫu nhiên; thứ tự luồng không tắt định; một khác biệt nhỏ trong thời điểm có thể đổi keyframe nào được chọn. Hai lần chạy cùng một hệ trên cùng dữ liệu cho hai kết quả khác nhau.

Bốn — và đây là lý do nghiêm trọng nhất: điều ta đo thường không phải điều ta quan tâm. Ta đo ATE trung bình trên các chuỗi mà hệ thống chạy được. Cái ta quan tâm là hệ thống có chạy được ở nhà khách hàng suốt tám tiếng hay không — và hai thứ đó tương quan yếu hơn nhiều so với vẻ ngoài.

Trước khi có các chỉ số chuẩn, mỗi bài báo tự định nghĩa cách đo, và so sánh giữa các bài là bất khả. TUM-RGBD [101] chuẩn hoá ATE và RPE; rpg_trajectory_evaluation [39] chuẩn hoá cách căn chỉnh và cách báo cáo drift theo đoạn. Đó là tiền bộ thật.

Nhưng chuẩn hoá chỉ số không giải quyết bốn lý do ở trên. Chương này lập luận rằng phần lớn giá trị của việc đánh giá nằm ở *thiết kế thí nghiệm*, không ở việc chọn chỉ số — và rằng thiết kế thí nghiệm là chỗ được đầu tư ít nhất.

TRỰC GIÁC

Hình dung bạn đánh giá hai tài xế.

Cách đơn giản: cho mỗi người lái một quãng đường và đo thời gian. Tài xế nhanh hơn thắng.

Ba chỗ sai của cách đó, và chúng ánh xạ chính xác sang ba chỗ sai của việc đánh giá SLAM.

Một — bạn đo trên một quãng đường. Tài xế A nhanh trên đường cao tốc, tài xế B nhanh trong phố. Một con số duy nhất giấu mất điều đó, và nếu bạn định thuê người lái trong phố thì bạn vừa chọn sai.

Hai — bạn đo trung bình, không đo tệ nhất. Tài xế A nhanh hơn 10% nhưng thỉnh thoảng đâm xe. Tài xế B chậm hơn nhưng chưa bao giờ. Con số trung bình nói A thắng; bất kỳ ai thuê tài xế đều biết B mới là lựa chọn.

Ba — bạn không hỏi họ có biết mình đang chạy ẩu không. Một người lái nhanh và biết lúc nào nên chậm lại thì an toàn. Một người lái nhanh và luôn nghĩ mình đang an toàn thì không. Hai người này có cùng thời gian và cùng số lần đâm xe cho tới nay — nhưng họ không giống nhau chút nào.

Chuyện thứ ba là chuyện tình tế nhất và là chuyện mà gần như không ai đo. Nó tương ứng với việc đo *tính trung thực của hiệp phương sai*: một hệ biết mình sai bao nhiêu thì dùng được, một hệ luôn nghĩ mình đúng thì không — dù hai hệ có cùng ATE.

Và một chuyện thứ tư, về sự công bằng. Nếu bạn để tài xế A lái chiếc xe quen của anh ta và bắt tài xế B lái một chiếc xe lạ, thì kết quả không nói gì về hai tài xế. Trong đánh giá SLAM, chuyện này xảy ra liên tục dưới dạng: người đề xuất phương pháp mới tự cài đặt baseline.

18.2 Chỉ số quỹ đạo, và cái bẫy căn chỉnh

A. ATE và RPE đo hai thứ khác nhau

ATE căn chỉnh toàn bộ quỹ đạo ước lượng với chân trị rồi tính sai số vị trí ở mỗi thời điểm. Nó đo *sai số toàn cục*, nên nó rất nhạy với việc có đóng vòng hay không: một loop closure tốt cải thiện ATE mạnh.

RPE lấy các đoạn có độ dài cố định — chẳng hạn mọi đoạn 10 m — và đo sai số của phép biến đổi tương đối trên mỗi đoạn. Nó đo *drift cục bộ*, và nó gần như không bị ảnh hưởng bởi loop closure.

Hai chỉ số trả lời hai câu khác nhau, và dùng nhầm là một lỗi thật. Nếu bạn đang cải thiện odometry, RPE là chỉ số đúng — ATE sẽ bị chi phối bởi việc có đóng vòng hay không, và nó che mất đóng góp của bạn. Nếu bạn đang cải thiện loop closure, ATE là chỉ số đúng — RPE gần như không đổi.

Cách báo cáo tốt nhất, theo [39]: *drift theo đoạn*, tức RPE tính trên nhiều độ dài đoạn khác nhau và biểu diễn dưới dạng phần trăm quãng đường. Nó cho một bức tranh đầy đủ hơn một con số, và nó so sánh được giữa các chuỗi có độ dài khác nhau — điều mà ATE tuyệt đối không làm được.

B. Căn chỉnh mấy bậc tự do: cái bẫy chính

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là lỗi mà tôi cho là phổ biến nhất trong đánh giá SLAM.

Quỹ đạo ước lượng sống trong một hệ quy chiếu tùy ý — đó là gauge ở A/05. Trước khi so với chân trị, phải căn chỉnh. Câu hỏi: căn chỉnh *mấy bậc tự do*?

Câu trả lời: *đúng bằng số bậc gauge của hệ*, và A/05 mục 3B cho bảng:

Mono thuần thị giác: $Sim(3)$, bảy bậc — vì tỉ lệ là gauge tự do. Stereo: $SE(3)$, sáu bậc. VIO: *bốn* bậc — ba tịnh tiến và yaw, vì trọng lực ghim roll với pitch.

Căn sai thì sao? Hai hướng sai với hai hậu quả khác nhau.

Căn quá nhiều bậc — ví dụ căn $Sim(3)$ cho một hệ VIO — *che mất lỗi*. Phép căn chỉnh sẽ tự động co giãn quỹ đạo ước lượng cho khớp với chân trị, nên một hệ có sai số tỉ lệ 10% sẽ cho ATE giống hết một hệ có tỉ lệ đúng. Bạn vừa xoá đúng thứ bạn cần đo — và với VIO, tỉ lệ là một trong những thứ quan trọng nhất cần kiểm, vì nó là thứ IMU đóng góp.

Căn quá ít bậc — ví dụ căn bốn bậc cho một hệ mono — cho ATE lớn giả tạo, vì phần gauge chưa được hấp thụ sẽ xuất hiện thành sai số.

Hệ quả thực hành: *luôn ghi rõ căn chỉnh mấy bậc khi báo cáo ATE*, và nghi ngờ mọi bảng so sánh không ghi. Và nếu bạn đang so hai hệ có cấu hình cảm biến khác nhau — mono với VIO chẳng hạn — thì ATE của chúng *không so sánh trực tiếp được*, vì chúng phải căn chỉnh với số bậc khác nhau.

CẠM BẮY

Căn chỉnh $Sim(3)$ cho một hệ có IMU là cách phổ biến nhất để tự lừa mình.

Cơ chế: phép căn chỉnh cơ giãn quỹ đạo ước lượng cho khớp chân trị, nên nó *hấp thụ* sai số tỉ lệ. Một hệ VIO có tỉ lệ sai 10% — một lỗi nghiêm trọng, thường do khởi tạo kém hoặc do kích thích không đủ (B/10) — sẽ cho ATE gần như giống hệt một hệ có tỉ lệ đúng.

Bạn sẽ thấy một con số đẹp và kết luận hệ thống tốt, trong khi thứ mà IMU được thêm vào để cung cấp — đơn vị mét — đang sai.

Ba quy tắc để tránh. *Một*: căn chỉnh đúng số bậc theo cấu hình cảm biến, và **ghi rõ trong mọi báo cáo**. *Hai*: với hệ có tỉ lệ quan sát được, hãy **báo cáo riêng sai số tỉ lệ** như một con số độc lập — nó là thứ đáng theo dõi nhất và nó biến mất khỏi ATE nếu căn sai. *Ba*: khi đọc một bảng so sánh không ghi số bậc căn chỉnh, hãy coi các con số là không diễn giải được, chứ không phải là “chắc họ làm đúng”.

18.3 Đo tính trung thực, không chỉ độ chính xác

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và là mục mà tôi cho là có tỉ lệ giá trị trên mức phổ biến cao nhất trong chương.

A. NEES và điều nó cho biết

Từ A/06 mục 4A: $\varepsilon = \tilde{\mathbf{x}}^\top \Sigma^{-1} \tilde{\mathbf{x}}$, và nếu Σ trung thực thì $\mathbb{E}[\varepsilon]$ bằng số chiều.

ATE trả lời: *bạn sai bao nhiêu*. NEES trả lời: *bạn có thành thật về việc mình sai bao nhiêu không*. Hai câu hỏi khác nhau, và một hệ có thể tốt ở câu một và tệ ở câu hai — thực tế đó là trường hợp thường gặp (A/06 mục 4).

Vì sao câu hai quan trọng với sản phẩm: mọi module phía trên dùng hiệp phương sai để ra quyết định. Bộ lập kế hoạch quyết định có đi qua khe hẹp không; loop closure quyết định có chấp nhận một ứng viên không (B/11 mục 3B); relocalization quyết định có tin kết quả không (B/12 mục 4B). Ba quyết định ấy đều sai nếu hiệp phương sai quá nhỏ.

Cái giá: cần chân trị, nên chỉ làm được trong mô phỏng hoặc trên bộ dữ liệu có hệ đo ngoài.

B. Vì sao “cần chân trị” không phải lý do bỏ qua

Lập luận thường gặp: NEES chỉ đo được trong mô phỏng, mà mô phỏng không giống thực tế, nên nó không có giá trị.

Lập luận ấy sai ở một chỗ quan trọng. NEES trong mô phỏng không đo “hệ thống của tôi tốt tới đâu trong thực tế”. Nó đo một thứ khác và cụ thể hơn: *cài đặt hiệp phương sai của tôi có đúng không, với chính mô hình mà tôi đã giả định*. Đó là một câu hỏi về *tính đúng đắn của cài đặt*, và mô phỏng là chỗ lý tưởng để trả lời nó, vì ở đó mọi giả định đều thoả chính xác.

Nói cách khác: nếu NEES lệch trong mô phỏng — nơi nhiều đúng là Gauss, trọng số đúng là σ bạn đã dùng để sinh dữ liệu — thì bạn có *lỗi cài đặt*, không phải vấn đề mô hình. Nhầm hiệp phương sai biên với điều kiện (A/06 mục 2B), quên nhân Adjoint khi đổi quy ước (A/03), sai một

hệ số ở đâu đó. Đây là những lỗi mà không phép đo nào khác bắt được, và chúng *im lặng* — hệ thống vẫn cho ATE tốt.

Đây là lập luận quan trọng nhất của mục này: *mô phỏng không dùng để chứng minh hệ thống tốt, mà để tách lỗi cài đặt khỏi lỗi mô hình*. Chạy trong mô phỏng trước là cách rẻ nhất để biết phần nào của vấn đề là do bạn viết sai và phần nào là do thế giới khó.

C. Khi không có chân trị

Tại hiện trường, bốn đại lượng thay thế đáng theo dõi, và chúng là nền cho giám sát sức khoẻ ở [D/21](#): số inlier và tỉ lệ inlier; số điều kiện hoặc giá trị riêng nhỏ nhất của Λ ; vết của hiệp phương sai tư thế; và tỉ lệ tracking.

Không đại lượng nào trong bốn cái đó đo sai số. Chúng đo các *điều kiện* mà sai số nhỏ phụ thuộc vào. Quan hệ giữa chúng và sai số thật phải được *hiệu chuẩn một lần* trên dữ liệu có chân trị, rồi dùng ngoài hiện trường. Đó là một quy trình đáng làm và ít ai làm.

I 18.4 Đánh giá thành phần có chi phí lỗi bất đối xứng

Loop closure ([B/11](#)) và relocalization ([B/12](#)) có chung một đặc điểm: chi phí của dương tính giả lớn hơn âm tính giả nhiều bậc. Đánh giá chúng bằng một con số duy nhất là vô nghĩa.

A. Precision-recall, và vùng đáng nhìn

Cách đúng là đường cong precision-recall. Nhưng — và đây là điểm quan trọng — *điểm F1 tối đa là tiêu chí sai*, vì F1 giả định precision và recall có giá trị ngang nhau.

Với chi phí bất đối xứng, vùng đáng nhìn là vùng *precision rất cao*. Câu hỏi đúng là: ở *precision 99,9%*, *recall của tôi là bao nhiêu*? Một hệ có recall 60% ở precision 99,9% hữu ích hơn một hệ có recall 95% ở precision 95%, dù con số thứ hai trông đẹp hơn — vì 5% dương tính giả nghĩa là bản đồ bị phá định kỳ.

Cách báo cáo tốt: chọn một mức precision từ *yêu cầu sản phẩm* — “tôi chấp nhận một loop closure sai trên một nghìn” — rồi báo cáo recall ở mức đó. Con số ấy có nghĩa và so sánh được.

B. Đánh giá trên truy vấn không có câu trả lời

Một thiếu sót có hệ thống của các benchmark, đã nêu ở [B/12](#) khối *Cạm bẫy*: chúng chỉ chứa các truy vấn *có* câu trả lời trong bản đồ.

Trong thực tế, phần lớn truy vấn relocalization xảy ra ở những chỗ mà hệ thống *chưa* lập bản đồ — robot bị đưa sang một phòng mới, hoặc bản đồ đã lỗi thời. Hành vi đúng là trả về “không biết”. Không benchmark nào đo điều đó.

Cách khắc phục cho bộ dữ liệu nội bộ: cố ý đưa vào các truy vấn ngoài bản đồ và đo tỉ lệ hệ thống trả lời sai thay vì nói không biết. Đây là một trong những phép đo có giá trị cao nhất cho sản phẩm và gần như không tồn tại trong tài liệu.

I 18.5 Kỷ luật phương pháp

A. Nhiều lần chạy và ý nghĩa thống kê

Đây là câu trả lời cho câu hỏi mở đầu thứ hai.

SLAM có nguồn ngẫu nhiên thật: RANSAC lấy mẫu ngẫu nhiên, thứ tự luồng không tắt định, và một khác biệt nhỏ về thời điểm đổi keyframe nào được chọn. Độ tán giữa các lần chạy có thể so sánh được với khác biệt giữa hai phương pháp.

Quy tắc tối thiểu: *chạy mỗi cấu hình nhiều lần với các seed khác nhau, và báo cáo cả trung vị lẫn độ tán. Một cải thiện 15% trong khi độ tán giữa các lần chạy là 20% thì chưa chứng minh gì.*

Ba lý do vì sao 15% trên mười chuỗi vẫn chưa đủ: *một*, có thể nằm trong độ tán chạy-tới-chạy; *hai*, có thể do một hoặc hai chuỗi chi phối trong khi các chuỗi khác tệ đi — nên phải nhìn phân bố chứ không nhìn trung bình; *ba*, mười chuỗi có thể không đại diện cho môi trường triển khai.

Một cách rẻ để loại nguồn ngẫu nhiên: làm hệ thống *tắt định* — seed cố định, thứ tự xử lý cố định. Điều này đáng làm vì lý do khác quan trọng hơn: nó là điều kiện để debug một lỗi từ hiện trường ([D/20](#)).

B. Baseline phải tử tế

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và nó là một quy tắc mà `research-rules.md` mục 7 nêu rõ.

Nếu bạn tự cài đặt baseline và bạn thắng, phép kiểm bắt buộc là: *baseline của tôi có đạt được con số mà bài gốc công bố không?* Nếu không, bạn đang so với một phiên bản bị làm yếu, và kết quả không nói gì.

Đây không phải một cảnh báo lý thuyết. Rất nhiều cải tiến biến mất khi baseline được chỉnh tử tế, và lý do có cấu trúc: người đề xuất phương pháp mới dành hàng tháng tinh chỉnh nó và vài ngày cài baseline.

Quy tắc thực hành: dùng mã nguồn gốc của baseline khi có; nếu phải tự cài, hãy tái lập được con số công bố *trước* khi so sánh; và nếu không tái lập được, hãy ghi rõ điều đó thay vì im lặng.

C. Ablation có kỷ luật

Tắt từng thành phần một, giữ nguyên mọi thứ khác, và đo. Nghe hiển nhiên, và nó hay bị làm sai theo ba cách.

Sai một — *tắt nhiều thứ cùng lúc*. Nếu tắt cả loop closure lẫn local BA, bạn không biết cái nào đóng góp gì.

Sai hai — *không chỉnh lại tham số*. Tắt một thành phần có thể làm các tham số tối ưu thay đổi. Nếu bạn tắt M-estimator mà giữ nguyên ngưỡng RANSAC, bạn đang đo một cấu hình chưa được tinh chỉnh, không phải đóng góp của M-estimator.

Sai ba — *không chạy đủ nhiều lần*. Áp dụng đúng như mục 5A.

D. Đánh giá ở đuôi, không ở trung bình

Đây là nguyên tắc quan trọng nhất của chương, và là câu trả lời cho câu hỏi mở đầu thứ năm.

Với hệ A (ATE trung bình 0,08 m, thất bại 2%) so với hệ B (ATE 0,12 m, chưa bao giờ thất bại): câu trả lời phụ thuộc vào *chi phí của một lần thất bại*.

Với một bài báo, thất bại nghĩa là một chuỗi bị loại khỏi bảng, nên hệ A thắng. Với một sản phẩm, thất bại nghĩa là robot dừng giữa nhà khách hàng và ai đó phải tới xử lý — và 2% trên hàng nghìn máy mỗi ngày là một dòng chi phí vận hành thật. Hệ B thắng, và thắng không sát sao.

Đây là chỗ mục tiêu sản phẩm ở **00-map** tạo ra một khác biệt thật về phương pháp: *SOTA thật nằm ở đuôi phân bố, không ở trung bình*. Ba hệ quả cụ thể.

Một — *báo cáo phân vị, không chỉ trung bình*. Phân vị 95 và 99 của ATE, và của thời gian mỗi khung. **B/09** mini project đã nhấn mạnh điều này cho thời gian.

Hai — *đếm và phân loại thất bại*. Số lần mất bám, số lần relocalization sai, số lần bản đồ bị phá. Các con số này thường không xuất hiện trong bài báo và chúng là con số quan trọng nhất cho sản phẩm.

Ba — *thiết kế thí nghiệm nhắm vào đuôi*. Đừng chỉ chạy trên các chuỗi mà hệ thống chạy được. Chạy trên các chuỗi khó, các chuỗi có sự cố, các chuỗi ngoài phân bố. **D/21** lập luận rằng đây là lý do phải có bộ dữ liệu nội bộ phản ánh môi trường khách hàng.

CẦN NHỚ

Bốn câu hỏi mà một đánh giá đầy đủ phải trả lời, và ba câu sau thường bị bỏ.

Một — *chính xác tới đâu?* ATE và RPE, căn chỉnh đúng số bậc. Đây là câu duy nhất mà phần lớn bài báo trả lời.

Hai — *có thành thật không?* NEES trong mô phỏng để kiểm cài đặt hiệp phương sai; các đại lượng thay thế ngoài hiện trường.

Ba — *hỏng thế nào và bao nhiêu lần?* Tỷ lệ thất bại, phân loại nguyên nhân, và hành vi khi hỏng — nó dừng lại hay tiếp tục báo số sai?

Bốn — *ở đuôi thì sao?* Phân vị 95 và 99, không phải trung bình. Với sản phẩm, đây là câu quan trọng nhất.

Và một nguyên tắc bao trùm: **đo trước, cải tiến sau**. Cải tiến trước khi đo là cách nhanh nhất để tối ưu sai thứ — bạn sẽ dành ba tháng làm giảm ATE trung bình 10% trong khi vấn đề thật là 2% số lần chạy bị hỏng vì một nguyên nhân bạn chưa từng đo.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
evo [102]	Công cụ tính ATE, RPE với các chế độ căn chỉnh khác nhau	Dễ dùng; chú ý chọn đúng chế độ căn chỉnh — mặc định không phải lúc nào cũng đúng cho hệ của bạn
rpg_trajectory_evaluation [39]	Căn chỉnh theo cấu hình cảm biến; drift theo đoạn; nhiều lần chạy	Cài đặt tham chiếu của mục 2; đọc tài liệu về lựa chọn căn chỉnh
OpenVINS [42]	Mô phỏng có chân trị; tính NEES trực tiếp	Một trong ít hệ mã nguồn mở đo tính nhất quán chứ không chỉ độ chính xác
TUM-RGBD [101]	Định nghĩa chuẩn của ATE và RPE	Nguồn của mục 2A
OpenLORIS [79]	Bộ dữ liệu nhiều phiên, môi trường thay đổi	Một trong ít bộ dữ liệu cho phép đánh giá hoạt động dài hạn

MINI PROJECT SLAM · Đánh giá lại một kết quả bạn đã tin, theo bốn câu hỏi ở khối Cần nhớ · 3 ngày

Mục tiêu	phát hiện xem một kết luận bạn đã rút ra từ các chương trước có đứng vững dưới một đánh giá có kỷ luật hay không.
Dữ liệu	chọn một mini project bạn đã làm ở các chương trước — so ba phương pháp triangulation, hay so hai front-end, hay so ba back-end — và dùng lại dữ liệu đó, cộng thêm vài chuỗi khó.

Các bước	chạy lại so sánh cũ nhưng lần này: (a) căn chỉnh đúng số bậc theo cấu hình cảm biến và ghi rõ; (b) chạy mỗi cấu hình <i>mười</i> lần với seed khác nhau và báo cáo trung vị cùng khoảng phân vị; (c) báo cáo phân vị 95 và 99 chứ không chỉ trung bình; (d) đếm và phân loại các lần thất bại; (e) nếu có mô phỏng, tính NEES.
Xong khi	bạn trả lời được: kết luận cũ của bạn có còn đúng không, và nếu có thì với mức tin cậy nào? Kết quả có giá trị nhất của bài này là khi câu trả lời là <i>không</i> — bạn vừa tránh được việc xây tiếp trên một kết luận sai. Nếu độ tán giữa mười lần chạy lớn hơn khác biệt giữa hai phương pháp, hãy ghi điều đó vào 99-chua-biet.md như một câu hỏi mở thay vì kết luận bừa.
Nối với	D/21-tu-prototype-len-mass-production biến quy trình này thành regression test tự động chạy mỗi commit; D/22-huong-nghien-cuu-con-trong dựa vào nó để phân biệt một khoảng trống thật với một khoảng trống tưởng tượng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì *Sim(3)* có bảy bậc gồm cả *tỉ lệ*, nên phép căn chỉnh sẽ co giãn quỹ đạo ước lượng cho khớp chân trị và *hấp thụ* sai số tỉ lệ. Một hệ VIO có tỉ lệ sai 10% — lỗi nghiêm trọng, thường do khởi tạo kém hoặc kích thích không đủ — sẽ cho ATE gần giống hệ có tỉ lệ đúng. Bạn vừa xoá đúng thứ cần đo, và đó là thứ mà IMU được thêm vào để cung cấp. Với VIO phải căn *bốn* bậc (ba tịnh tiến + yaw), vì trọng lực ghim roll và pitch. Và nên báo cáo sai số tỉ lệ như một con số riêng.
- A2. (1) Có thể nằm trong *độ tán chạy-tới-chạy* — SLAM có ngẫu nhiên thật từ RANSAC và thứ tự luồng, và độ tán có thể so sánh được với 15%. (2) Có thể do *một hoặc hai chuỗi chi phối* trong khi các chuỗi khác tẻ đi — phải nhìn phân bố chứ không nhìn trung bình. (3) Mười chuỗi có thể *không đại diện* cho môi trường triển khai. Và một lý do thứ tư quan trọng hơn cả ba: 15% ở *trung bình* không nói gì về *đuôi*, và sản phẩm hỏng ở đuôi. Cách kiểm: chạy nhiều seed, báo cáo trung vị và độ tán, báo cáo phân vị 95 và 99, và đếm số lần thất bại.
- A3. Vì NEES trong mô phỏng không đo “hệ tôi tốt tới đâu trong thực tế” — nó đo một thứ khác và cụ thể hơn: *cài đặt hiệp phương sai của tôi có đúng không, với chính mô hình tôi đã giả định*. Mô phỏng là *chỗ lý tưởng* cho câu hỏi ấy vì ở đó mọi giả định thoả chính xác. Nếu NEES lệch trong mô phỏng thì bạn có *lỗi cài đặt* — nhằm hiệp phương sai biên với điều kiện, quên Adjoint khi đổi quy ước, sai một hệ số — và đó là loại lỗi im lặng mà không phép đo nào khác bắt được, vì ATE vẫn tốt. Nguyên tắc: *mô phỏng không dùng để chứng minh hệ tốt, mà để tách lỗi cài đặt khỏi lỗi mô hình*.
- A4. Phép kiểm bắt buộc: *baseline của tôi có đạt con số mà bài gốc công bố không?* Nếu không, bạn đang so với một phiên bản bị làm yếu và kết quả không nói gì. Đây không phải cảnh báo lý thuyết mà có lý do cấu trúc: người đề xuất phương pháp mới dành hàng tháng tinh chỉnh nó và vài ngày cài baseline. Quy tắc: dùng mã nguồn gốc khi có; nếu tự cài thì phải tái lập được con số công bố *trước* khi so sánh; nếu không tái lập được thì ghi rõ điều đó thay vì im lặng.
- A5. Phụ thuộc vào *chi phí của một lần thất bại*. Với một bài báo, thất bại nghĩa là một chuỗi bị loại khỏi bảng, nên A thắng. Với một sản phẩm, thất bại nghĩa là robot dừng giữa nhà khách hàng và ai đó phải tới xử lý — và 2% trên hàng nghìn máy mỗi ngày là một dòng chi phí vận hành thật, chưa kể thiệt hại về uy tín. B thắng, và không sát sao. Với mục tiêu sản phẩm ở *00-map*, nguyên tắc là *SOTA thật nằm ở đuôi phân bố*: báo cáo phân vị thay vì trung bình, đếm và phân loại thất bại, và thiết kế thí nghiệm nhắm vào các chuỗi khó chứ không chỉ các chuỗi chạy được.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn muốn thuê một tài xế và có hai ứng viên. Cách đơn giản nhất: cho mỗi người lái một quãng, đo thời gian, chọn người nhanh hơn.

Cách đó sai ở ba chỗ, và ba chỗ ấy chính là ba lỗi phổ biến nhất khi đánh giá một hệ định vị.

Thứ nhất, bạn đo trên một quãng đường. Người A nhanh trên cao tốc, người B nhanh trong phố. Một con số duy nhất giấu mất điều đó, và nếu bạn cần người lái trong phố thì bạn vừa chọn nhầm.

Thứ hai, bạn đo trung bình chứ không đo lúc tệ nhất. Người A nhanh hơn mười phần trăm nhưng thỉnh thoảng đâm xe. Người B chậm hơn nhưng chưa bao giờ. Con số trung bình bảo chọn A; bất kỳ ai thật sự đi thuê tài xế đều biết phải chọn B.

Thứ ba, và tinh tế nhất: bạn không hỏi họ có *biết* lúc nào mình đang chạy ẩu không. Một người lái nhanh và biết lúc nào nên chậm lại thì an toàn. Một người lái nhanh và lúc nào cũng nghĩ mình đang ổn thì không. Hai người này có cùng thời gian và cùng số lần đâm xe *cho tới nay* — nhưng họ không giống nhau chút nào, và bạn sẽ biết điều đó vào một ngày không đẹp trời.

Cái thứ ba là cái gần như không ai đo. Nó tương ứng với việc hỏi một cái máy: “mày nói mày sai hai centimet — điều đó có thật không?” Rất ít người hỏi câu đó, và câu trả lời thường là không.

Còn một chuyện nữa, về sự công bằng khi so sánh. Nếu bạn để người A lái chiếc xe quen của anh ta và bắt người B lái một chiếc xe lạ, thì kết quả chẳng nói gì về hai người. Chuyện này xảy ra liên tục trong nghiên cứu, dưới dạng: người đề xuất phương pháp mới tự cài lại phương pháp cũ để so sánh — và dành hàng tháng cho cái của mình, vài ngày cho cái của người khác.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *không có sự thật để so* ngoài phòng thí nghiệm. Trong phòng lab có hệ đo ngoài cho biết chân trị; ở nhà khách hàng thì không. Nên mọi thứ ta đo được ngoài hiện trường đều là dấu hiệu gián tiếp, và việc nối các dấu hiệu ấy với sai số thật là một việc phải làm một lần trong lab rồi mang ra dùng — và rất ít nơi làm.

Một câu: *Đo trước rồi hãy cải tiến — và đo ở chỗ tệ nhất, không ở chỗ trung bình.*

18.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *căn chỉnh phải đúng số bậc gauge* — suy từ [A/05](#) (cửa a) và được [\[39\]](#) thiết lập (cửa c). Nó kiểm được trực tiếp: lấy một quỹ đạo, nhân tỉ lệ 1,1, rồi tính ATE với căn $Sim(3)$ và với căn $SE(3)$; nếu con số thứ nhất không đổi thì mệnh đề được xác nhận.

Mệnh đề thứ hai — *NEES trong mô phỏng có giá trị vì nó tách lỗi cài đặt khỏi lỗi mô hình* — là một lập luận về mục đích của phép đo, không phải một sự kiện thực nghiệm. Nó bị bác bỏ nếu trong thực tế, các lỗi cài đặt hiệp phương sai hiểm tới mức phép đo này không đáng công. Tôi cho là ngược lại — tôi ngờ chúng rất phổ biến, dựa trên việc phân biệt biên với điều kiện là một điểm tinh tế — nhưng tôi không có số liệu (*tôi suy ra, chưa đổi chiều nguồn*).

Mệnh đề thứ ba — *SOTA thật nằm ở đuôi* — phụ thuộc vào mục tiêu. Nó đúng cho sản phẩm robot hoạt động không giám sát, là mục tiêu đã nêu ở [00-map](#). Nó *sai* cho một ứng dụng mà thất bại rẻ và độ chính xác trung bình là thứ được trả tiền — quét đo đặc có người vận hành chẳng hạn, nơi một lần hỏng chỉ tốn một lần quét lại. Nếu rõ điều này vì nguyên tắc ở mục 5D không phổ quát.

Mệnh đề thứ tư — *benchmark không đo truy vấn ngoài bản đồ* — là một quan sát về các bộ dữ liệu hiện có, và nó bị bác bỏ nếu một benchmark mới đưa vào phần đó. Nếu điều đó xảy ra thì đó là tin tốt, và tôi mong nó xảy ra.

18.7 Kết nối

Chương này là chương mà mọi chương trước đều nợ, nên nổi lên trên là nổi tới tất cả.

Cụ thể: [A/05-observability-gauge](#)-va-suy-bien cho quy tắc căn chỉnh ở mục 2B — không có chương đó thì quy tắc này trông tùy tiện. [A/06-bat-dinh](#)-va-lan-truyen-sai-so cho NEES và cho lý do vì sao tính trung thực quan trọng. [B/11-loop-closure](#)-va-nhat-quan-toan-cuc và [B/12-relocalization](#) cho hai thành phần có chi phí lỗi bất đối xứng ở mục 4. [C/15-hop-nhat-da-cam-bien](#) khỏi *Cần nhớ* nêu rằng đo một hệ hợp nhất bằng ATE trung bình là bỏ sót phần lớn giá trị của nó — mục 5D ở đây là cách khắc phục.

Nối xuống dưới. [D/21-tu-prototype-len-mass-production](#) là chỗ chương này trở thành quy trình: bộ dữ liệu nội bộ phản ánh môi trường khách hàng, regression test tự động cho mỗi commit, ma trận thử nghiệm, và nguyên tắc không bao giờ tin benchmark công khai là đủ. [D/22-huong-nghien-cuu-con-trong](#) phụ thuộc vào chương này ở một mức sâu hơn: không thể phân biệt một khoảng trống *thật* với một khoảng trống *tưởng tượng* nếu không có kỷ luật đo lường — và ba trong tám hướng ở đó (bất định đáng tin, dự đoán thất bại, xử lý suy biến có nguyên tắc) chỉ có nghĩa khi ta đo được chúng.

Nối sang `research-rules.md`: mục 5B và 5C của chương này là mục 7 của tài liệu đó áp dụng cho SLAM. Nếu bạn định nâng một phần của cây này lên thành một dự án khảo sát hoặc nghiên cứu, chương này là cầu nối.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Căn chỉnh mấy bậc cho mono, stereo, VIO — và nói căn quá nhiều bậc che mất điều gì cụ thể.
2. ATE và RPE đo hai thứ khác nhau. Nếu bạn đang cải thiện odometry thì dùng cái nào, và vì sao cái kia sẽ che mất đóng góp của bạn?
3. NEES cần chân trị nên chỉ đo được trong mô phỏng. Vì sao điều đó không phải lý do bỏ qua, và nó bắt được loại lỗi nào mà ATE không bắt được?
4. Vì sao điểm F1 tối đa là tiêu chí sai cho loop closure, và tiêu chí đúng là gì?
5. Nêu ba cách làm sai một ablation study.
6. Hệ A nhanh và thỉnh thoảng hỏng, hệ B chậm và chưa bao giờ hỏng. Câu trả lời phụ thuộc vào gì, và với mục tiêu của cây này thì chọn cái nào?
7. Nêu bốn câu hỏi mà một đánh giá đầy đủ phải trả lời, và nói vì sao ba câu sau thường bị bỏ.

THỰC TẾ VÀ SẢN XUẤT



TÓM TẮT

Đây là phần ít tài liệu học thuật nhất và tạo ra giá trị thật nhiều nhất. Ba phần trước dựng một hệ thống chạy tốt trên EuRoC; phần này trả lời câu hỏi vì sao cùng hệ thống ấy chết ở nhà khách hàng, và phải làm gì. Bốn chương: thế giới thực và những gì nó ném vào hệ thống (chương 19); ràng buộc tính toán, bộ nhớ và thời gian trên phần cứng thật (chương 20); con đường từ một nguyên mẫu tới hàng nghìn máy xuất xưởng (chương 21); và cuối cùng, tám chỗ mà tôi cho là còn khoảng trống thật nếu mục tiêu là đóng góp mới (chương 22). Nếu chỉ nhớ một điều: *biết mình đang sai quan trọng hơn không bao giờ sai* — một hệ có tỉ lệ lỗi 2% mà phát hiện được 90% số lần lỗi thì dùng được trong sản phẩm; một hệ có tỉ lệ lỗi 1% mà im lặng khi lỗi thì không.

Phần này khác ba phần trước ở một điểm căn bản: *nó không có tài liệu chuẩn*. Không có sách giáo khoa nào về việc đưa SLAM lên dây chuyền, và số bài báo về chủ đề này ít hơn số bài về một biến thể detector đặc trưng vài bậc độ lớn. Lý do có cấu trúc: kiến thức ở đây thuộc về các công ty, nó không tổng quát hoá sạch sẽ thành định lý, và nó khó công bố vì mỗi ràng buộc là riêng của một sản phẩm.

Hệ quả cho cách đọc: nội dung ở đây được viết ở mức *nguyên tắc và danh mục kiểm* chứ không ở mức công thức. Nó cũng là phần mà tôi phải cẩn thận nhất về nguồn — rất nhiều điều “ai cũng biết” trong công nghiệp không có nguồn nào kiểm được, và theo `learning-rules.md` mục 5 thì tôi phải nói rõ chỗ nào là suy luận của tôi. Tôi làm điều đó nhất quán trong cả bốn chương.

Chương 19 và 21 là hai chương xương sống. Chương 19 vì nó liệt kê những gì thế giới thật ném vào hệ thống, và mỗi mục trong danh sách ấy là một chế độ hỏng mà ba phần trước hoặc bỏ qua hoặc chỉ nhắc thoáng. Chương 21 vì nó là chương duy nhất trong cây viết cho mục tiêu mass production, và vì các quyết định nó bàn — dung sai cơ khí, quy trình hiệu chuẩn, giám sát fleet — là những quyết định khó đảo ngược nhất. Chương 20 là chương kỹ thuật thuần túy và có thể đọc lướt nếu bạn chưa chạm vào phần cứng nhúng. Chương 22 khép cây lại bằng một danh sách tám hướng, và nó là chương duy nhất chạm vào địa hạt nghiên cứu — với một cảnh báo rõ ràng rằng nó chưa đạt chuẩn của `research-rules.md` và cần được nâng lên thành một dự án khảo sát riêng trước khi dùng để chọn đề tài thật.

Bảng 18.2. Bản đồ Phần D: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
19	19-ben-vung-trong-thi	Vật động và masking; môi trường nghèo vân; thay đổi ánh sáng; motion blur; kính và gương; vân lặp lại; ngoài trời; suy giảm cảm biến; phát hiện thất bại và suy giảm có kiểm soát	● ● ●
20	20-toi-uu-hieu-nang-	Đa luồng và hàng đợi; xử lý độ trễ và mất khung; SIMD và lượng tử hoá; GPU/NPU; ngân sách bộ nhớ; bộ giải thừa tối ưu; tính tất định; điện năng và nhiệt	● ●
21	21-tu-prototype-len-	Hiệu chuẩn quy mô sản xuất; độ tin cậy và giám sát sức khỏe; kiểm thử và regression; ràng buộc sản phẩm; fleet analytics; OTA; quyền riêng tư	● ● ●
22	22-huong-nghien-cuu-	Tám hướng còn khoảng trống, với đánh giá mức chắc chắn và cách kiểm mỗi hướng có thật là khoảng trống không	● ●

Chương 19 và 21 là xương sống của phần này. Đọc 19 trước ngay cả khi bạn còn đang làm nguyên mẫu — danh mục các chế độ hỏng ở đó nên được dùng làm danh mục kiểm khi thiết kế, không phải khi gỡ lỗi. Chương 20 đọc khi bạn bắt đầu chạm vào phần cứng mục tiêu; trước đó nó là kiến thức không dùng tới. Chương 21 là chương nên đọc *trước khi chốt thiết kế cơ khí*, vì vài quyết định nó bàn — dung sai lắp ráp, chọn global shutter, chỗ đặt cảm biến nhiệt — không sửa được sau khi mở khuôn. Chương 22 đọc cuối, và đọc với thái độ hoài nghi: nó là danh sách các chỗ tôi

cho là còn trống, không phải kết quả của một khảo sát có hệ thống, và chương đó nói rõ điều này ngay từ đầu.

Bền vững trong thế giới thực (Robustness in the Real World)

PHỤ THUỘC A/01-hình-hoc-da-thi (giả thiết cảnh tĩnh — chương này là chỗ trả nợ cho nó); A/02-mô-hình-camera (mô hình quang trắc, rolling shutter); B/07-dac-trung-va-data-association (giới hạn của RANSAC); B/08-mô-hình-phan-du (indirect và direct hồng khác nhau); C/13-quan-ly-ban-do (thế giới thay đổi theo thời gian).

TÓM TẮT

Ba phần trước dựng một hệ thống chạy tốt trên EuRoC. Chương này liệt kê những gì thế giới thật ném vào nó, và mỗi mục trong danh sách là một giả thiết của Phần A bị vi phạm. Nội dung tổ chức theo bốn nhóm chứ không theo danh sách phẳng, vì mỗi nhóm đòi một loại xử lý khác nhau. Nhóm một — *cảnh không tĩnh*: vật động, và một trường hợp đặc biệt tệ là khi vật động chiếm đa số khung hình. Nhóm hai — *thông tin không đủ*: nghèo vân, tối, mờ do chuyển động. Nhóm ba — *thông tin sai lệch*: kính, gương, bề mặt phản chiếu, vân lặp lại; đây là nhóm nguy hiểm nhất vì nó hỏng *im lặng*. Nhóm bốn — *cảm biến suy giảm*: bụi ống kính, lệch tiêu cự, hỏng pixel, và trôi hiệu chuẩn. Mục cuối cùng, và là mục quan trọng nhất, bàn về thứ quyết định một hệ có dùng được trong sản phẩm hay không: *phát hiện thất bại và suy giảm có kiểm soát*. Nếu chỉ nhớ một điều: *biết mình đang sai quan trọng hơn không bao giờ sai* — và đó không phải một câu an ủi mà là một nguyên tắc thiết kế có hệ quả cụ thể ở mọi module.

KIẾN THỨC CẦN CÓ TRƯỚC

- **vật động (dynamic object)** — vật thể di chuyển trong cảnh — người, xe, cửa mở. Nó vi phạm giả thiết cảnh tĩnh mà toàn bộ A/01 dựa vào.
- **masking** — loại bỏ các vùng ảnh khỏi việc xử lý, thường dựa trên phân đoạn ngữ nghĩa (“đây là người”) hoặc trên kiểm tra hình học (“vùng này không nhất quán”).
- **motion blur** — nhòe do vật hoặc camera di chuyển trong thời gian phơi sáng. Nó làm gradient giảm, nên cả detector lẫn direct method đều suy giảm.
- **suy giảm có kiểm soát (graceful degradation)** — hành vi của hệ thống khi điều kiện xấu: thay vì hỏng hoàn toàn, nó chuyển sang chế độ kém chính xác hơn nhưng vẫn hoạt động, và *báo* rằng nó đang ở chế độ đó.
- **phát hiện thất bại (failure detection)** — cơ chế để hệ thống tự nhận ra ước lượng của nó không còn đáng tin. Khác với phục hồi ở chỗ nó chạy *trước* và quyết định có cần phục hồi không.
- **hỏng im lặng (silent failure)** — chế độ hỏng mà hệ thống tiếp tục cho ra kết quả trông hợp lý nhưng sai, không có dấu hiệu nào. Nguy hiểm hơn hỏng ồn ào nhiều bậc.

CÂU HỎI MỞ ĐẦU

- Q1. Một chiếc xe tải đi ngang chiếm 70% khung hình. RANSAC báo 85% inlier. Nêu chính xác cái gì vừa được ước lượng, và vì sao kết quả sai mà vẫn nhất quán.
- Q2. Direct method và indirect method đều suy giảm khi có vật động. Nêu vì sao direct suy giảm *nặng* hơn, và nói điều đó ảnh hưởng tới lựa chọn kiến trúc thế nào.

- Q3.** Trong bốn nhóm vấn đề ở chương này, nhóm nào nguy hiểm nhất và vì sao? Câu trả lời không phải nhóm làm hệ thống hỏng nhiều nhất.
- Q4.** Camera của bạn bị mờ một góc vì bụi bám sau sáu tháng. Nếu ba dấu hiệu đo được, và nói vì sao ATE không phải một trong ba dấu hiệu ấy.
- Q5.** Bạn có ngân sách cho *một* tính năng: cải thiện độ chính xác 20%, hoặc thêm một cơ chế phát hiện thất bại. Chọn cái nào và biện minh bằng chi phí vận hành.

19.1 Đặt vấn đề

A/01 mục 7 liệt kê bốn giả thiết của hình học đa thị và nói rằng ba trong bốn sẽ được trả nợ ở các chương sau. Chương này là chỗ trả nợ, và nó trả nợ cho nhiều hơn thế — gần như mọi giả thiết tiện lợi của ba phần trước đều bị vi phạm ở đây.

Bài toán gốc phát biểu bằng một quan sát thực nghiệm mà ai triển khai đều gặp: *một hệ SLAM chạy tốt trên bộ dữ liệu chuẩn thường thất bại ở môi trường thật, và nó thất bại vì những lý do không có trong bộ dữ liệu chuẩn.*

Lý do cấu trúc, và đáng nói rõ vì nó giải thích cả sự thiếu tài liệu. Bộ dữ liệu chuẩn được thu để *đánh giá thuật toán*, nên chúng cố ý loại bỏ nhiễu loạn: EuRoC quay trong một phòng có motion capture, không có người đi lại; KITTI quay trên đường, nhưng người ta chọn các đoạn mà hệ thống chạy được. Điều đó *đúng* về mặt phương pháp — để so hai thuật toán, cần một môi trường kiểm soát — nhưng nó nghĩa là điểm số trên chúng không dự đoán được hành vi ngoài đời.

Ai gặp bài toán này? Chỉ những người triển khai. Đây là lý do một kỹ sư có sáu tháng kinh nghiệm triển khai thường biết nhiều hơn về chủ đề này so với một người đọc trăm bài báo — và là lý do chương này viết ở mức danh mục kiểm chứ không ở mức công thức.

Cách tổ chức của chương đáng giải thích. Tôi *không* liệt kê phẳng các vấn đề mà chia thành bốn nhóm theo *bản chất của cái sai*, vì mỗi nhóm đòi một loại xử lý khác nhau và nhằm nhóm là nhằm cách chữa:

Nhóm một — cảnh không tĩnh: dữ liệu đúng nhưng mô hình sai (mô hình giả định thế giới đứng yên).

Nhóm hai — thông tin không đủ: mô hình đúng nhưng dữ liệu nghèo. Cách chữa là thêm nguồn thông tin.

Nhóm ba — thông tin sai lệch: dữ liệu *trông* đúng nhưng không phải. Nguy hiểm nhất vì mọi cơ chế phát hiện dựa trên tính nhất quán đều thất bại.

Nhóm bốn — cảm biến suy giảm: mô hình đo không còn mô tả cảm biến. Đây là nhóm phát triển *chậm* theo thời gian, nên nó không xuất hiện trong bất kỳ thí nghiệm ngắn nào.

TRỰC GIÁC

Hình dung bạn đang đi trong một toà nhà và ghi nhớ đường đi bằng cách nhìn các mốc.

Nhóm một: có người đi ngang. Bạn ghi nhớ “có một người mặc áo đỏ ở đây”, rồi lát sau quay lại và người đó đã đi mất. Tệ hơn: nếu bạn đi cùng chiều với người đó một đoạn dài, bạn sẽ nghĩ mình đứng yên trong khi thực ra cả hai đang đi — vì so với người ấy thì bạn không nhúc nhích.

Nhóm hai: đèn tắt, hoặc bạn đi vào một hành lang trắng trơn không có gì để nhớ. Bạn *biết* mình không có thông tin, và bạn đi chậm lại, quờ tay tìm tường. Đây là loại vấn đề dễ chịu nhất, vì bạn biết mình đang gặp nó.

Nhóm ba: có một tấm gương lớn. Bạn nhìn thấy một hành lang tiếp tục ở phía trước và đi thẳng vào đó. Mọi thứ bạn thấy đều hợp lý, nhất quán, rõ ràng — cho tới khi bạn đâm vào gương. Đây là loại tệ nhất, vì *không có gì báo trước*.

Nhóm bốn: mắt bạn mờ dần theo năm tháng. Mỗi ngày nó chỉ tệ hơn hôm qua một chút, nên không ngày nào bạn nhận ra. Sau vài năm bạn nhìn nhầm nhiều thứ, và bạn vẫn tin vào mắt mình như cũ.

Bốn loại, bốn cách xử lý. Loại hai chữa bằng cách thêm nguồn thông tin khác — sờ tường, đếm bước. Loại một và bốn chữa bằng cách nhận ra và điều chỉnh. Loại ba thì *không chữa được bằng cách nhìn kỹ hơn* — cách duy nhất là biết trước rằng gương tồn tại, và nghi ngờ khi thấy một hành lang quá hoàn hảo.

Và có một điều chung cho cả bốn, quan trọng hơn mọi cách chữa: *một người biết mình đang không chắc thì đi chậm lại và sờ tường; một người không biết thì đi bình thường và ngã*.

19.2 Nhóm một: cảnh không tĩnh

A. Vì sao RANSAC không đủ

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất.

RANSAC tìm *tập nhất quán lớn nhất* (B/07 mục 5B). Khi vật động chiếm thiểu số, tập nhất quán lớn nhất là cảnh tĩnh, và RANSAC làm đúng việc của nó.

Khi vật động chiếm *đa số* — một xe tải đi ngang, một đám đông, một bức tường của kho hàng đang được xe nâng dịch chuyển — tập nhất quán lớn nhất là *vật động*. RANSAC trung thành chọn nó làm inlier và cảnh tĩnh làm outlier.

Cái vừa được ước lượng là *tư thế tương đối của camera so với chiếc xe tải*. Kết quả sai so với thế giới, nhưng nó hoàn toàn nhất quán: mọi điểm trên xe tải khớp hoàn hảo với mô hình, phần dư nhỏ, số inlier cao. Không có gì trong đầu ra báo hiệu.

Đây là một ví dụ nữa của nguyên tắc ở A/05: số inlier đo mức nhất quán của dữ liệu với mô hình, không đo mức đúng đắn của mô hình.

B. Ba tầng xử lý

Tầng một — kiểm tra hình học. Với các điểm có lịch sử quan sát đủ dài, kiểm xem chúng có nhất quán với một điểm 3D tĩnh không. Một điểm trên vật động sẽ có phần dư tăng dần khi vật di chuyển. Rẻ, không cần mô hình ngữ nghĩa, và nó bắt được mọi loại vật động — kể cả những loại mà một bộ phân đoạn không biết. Giới hạn: cần thời gian, nên nó không bắt được vật động ngay lập tức.

Tầng hai — masking ngữ nghĩa. Dùng một mạng phân đoạn để loại các vùng thuộc các lớp *có thể di chuyển* — người, xe, động vật. DynaSLAM [100] làm điều này. Nhanh và chủ động, nhưng nó có hai giới hạn đáng nêu: nó chỉ biết các lớp đã huấn luyện, và nó loại cả những vật *đang đứng yên* thuộc lớp đó — một chiếc xe đỗ là một mốc tĩnh rất tốt mà masking ngữ nghĩa sẽ vứt đi.

Cách kết hợp đúng: dùng ngữ nghĩa để *hạ trọng số*, không để loại cứng; rồi để kiểm tra hình học quyết định. Một chiếc xe đỗ sẽ chứng minh mình tĩnh qua vài khung và lấy lại trọng số.

Tầng ba — nhận ra khi vật động chiếm đa số. Đây là tầng hay bị bỏ và là tầng quan trọng nhất. Khi tầng một và hai cùng báo rằng phần lớn khung hình là động, hành vi đúng *không phải* cố ước lượng từ phần còn lại — đó là *từ chối* và dựa vào cảm biến khác (C/15) cho tới khi tình hình qua đi. Đây là nguyên tắc “biết từ chối” của B/10 áp dụng ở một chỗ khác.

C. Direct suy giảm nặng hơn indirect

Đây là câu trả lời cho câu hỏi mở đầu thứ hai.

Với indirect, một vật động làm mất vài chục điểm rời rạc — các điểm trên vật đó trở thành outlier và bị M-estimator hạ trọng số. Hệ thống mất một phần thông tin.

Với direct, một vật động làm hỏng cường độ của *cả một vùng liên thông*. Vì phần dư quang trắc được tính trên mọi pixel có gradient, và vì các pixel trong vùng ấy *tương quan mạnh* với nhau (chúng cùng sai theo cùng một cách), nên đóng góp của vùng ấy vào hàm mục tiêu lớn hơn nhiều so với tỉ lệ diện tích của nó. M-estimator xử lý outlier độc lập tốt hơn nhiều so với outlier tương quan thành khối.

Hệ quả kiến trúc: trong môi trường có nhiều vật động — nhà xưởng có người, đường phố — indirect có lợi thế thật. Đây là một trong những tiêu chí cụ thể để chọn giữa hai họ ở B/08, và nó không xuất hiện trong các so sánh trên bộ dữ liệu tĩnh.

19.3 Nhóm hai: thông tin không đủ

Nhóm này dễ chịu nhất vì hệ thống *biết* nó đang gặp vấn đề — số đặc trưng giảm, phần dư tăng, hiệp phương sai nở ra.

Nghèo nàn: hành lang trắng, tường trơn, sương mù. Cách xử lý theo thứ tự: chuyển sang đặc trưng đường thẳng và mặt phẳng (B/08 mục 4B), vốn tồn tại ngay cả khi không có góc; hoặc chuyển sang direct, vốn khai thác được gradient yếu; hoặc dựa vào cảm biến khác cho tới khi qua.

Ánh sáng thay đổi: đèn bật tắt, đi từ trong nhà ra ngoài trời, auto-exposure phản ứng. Với indirect, descriptor bền một phần. Với direct, đây là vấn đề nghiêm trọng và cách chữa là mô hình quang trắc (A/02 mục 5B) cộng hai tham số affine mỗi khung.

Motion blur: gradient giảm nên cả hai họ đều suy giảm. Cách chữa đúng thuộc về phần cứng — giảm thời gian phơi sáng, tăng gain, dùng cảm biến nhạy hơn — và đó là một quyết định ở D/21. Cách chữa phần mềm là dùng IMU để dự đoán và thu hẹp vùng tìm kiếm (B/07 mục 5C), vốn giúp nhưng không giải quyết được việc thông tin đã mất.

Điểm chung của cả ba, và là lý do nhóm này dễ hơn: *có tín hiệu cảnh báo*. Số đặc trưng, độ tương phản trung bình, và mức gradient đều đo được rõ, và chúng giảm *trước* khi hệ thống hỏng. Một hệ có giám sát các đại lượng này sẽ biết trước vài trăm mili giây — đủ để chuyển chế độ hoặc để báo.

19.4 Nhóm ba: thông tin sai lệch

Đây là câu trả lời cho câu hỏi mở đầu thứ ba: nhóm này nguy hiểm nhất, và không phải vì nó làm hệ thống hỏng nhiều nhất mà vì nó hỏng *im lặng*.

A. Kính, gương, bề mặt phản chiếu

Trong nhóm một và hai, hệ thống hoặc thấy phần dư lớn hoặc thấy ít dữ liệu — có tín hiệu. Ở đây, mọi thứ *trông hoàn hảo*: các đặc trưng trong gương ổn định, khớp tốt qua nhiều khung, triangulate ra các điểm 3D nhất quán. Chúng chỉ ở sai chỗ — ở phía sau mặt gương, tại vị trí ảnh phản chiếu.

Với cửa kính còn tệ hơn: hệ thống thấy cả cảnh phía sau kính (đúng về mặt hình học, vì ánh sáng đi thẳng qua) lẫn ảnh phản chiếu trên mặt kính (sai), trộn vào nhau.

Vì sao khó chữa: mọi cơ chế phát hiện trong cây này dựa trên *tính nhất quán* — phần dư, số inlier, kiểm chéo giữa các nguồn. Ảnh phản chiếu nhất quán với chính nó qua nhiều khung, nên không cơ chế nào bắt được. Đây là giới hạn cấu trúc chứ không phải thiếu sót cài đặt.

Ba hướng giảm thiểu, không cái nào đầy đủ. *Ngữ nghĩa*: một bộ phân đoạn có lớp “kính” và “gương” — hoạt động khi được huấn luyện đúng, và các bề mặt ấy khó phân đoạn chính vì chúng trong suốt. *Nhất quán đa cảm biến*: LiDAR cũng bị lừa bởi gương nhưng theo cách khác, nên bất đồng giữa hai cảm biến là một tín hiệu. *Suy luận hình học cấp cao*: ảnh phản chiếu có cấu trúc hình học đặc biệt — nó là phản xạ của cảnh thật qua một mặt phẳng — và về nguyên tắc một hệ có thể nhận ra điều đó qua nhiều góc nhìn. Tôi không biết một hệ SLAM thực dụng nào làm điều này, và nó là một hướng chưa được khai thác (*tôi suy ra từ lập luận hình học, chưa đổi chiều nguồn*).

B. Vân lặp lại

Đã bàn ở B/07 mục 3B và B/11 mục 5B, và nó thuộc nhóm này vì cùng lý do: một khớp sai giữa hai cánh cửa giống nhau *nhất quán* với chính nó.

Điều đáng nhắc ở đây là một biểu hiện đặc biệt trong sản phẩm: các môi trường lặp lại có hệ thống — kho hàng, bãi đỗ xe, hành lang chung cư — là đúng những môi trường mà robot thương mại hoạt động. Nên đây không phải một trường hợp biên mà là môi trường mặc định của một phần lớn thị trường.

C. Vì sao nhóm này quyết định thiết kế

Ba nhóm kia có thể xử lý bằng cách cải thiện thuật toán. Nhóm này thì không — nó đòi *thay đổi kiến trúc hệ thống*, theo hai hướng.

Một — đa dạng hoá cảm biến. Không phải để chính xác hơn mà để có *bất đồng*. Hai cảm biến bị lừa theo hai cách khác nhau sẽ bất đồng, và bất đồng là tín hiệu duy nhất có sẵn trong nhóm này. Đây là một lập luận cho hợp nhất cảm biến khác với lập luận ở C/15, và có lẽ mạnh hơn.

Hai — giới hạn hậu quả. Nếu không phát hiện được, hãy đảm bảo hậu quả bị chặn: đừng để một loop closure đơn lẻ phá bản đồ (B/11 mục 5), đừng để một relocalization đơn lẻ được tin ngay (B/12 mục 4B), và giữ khả năng quay lại một trạng thái trước đó.

19.5 Nhóm bốn: cảm biến suy giảm

Đây là nhóm mà không thí nghiệm ngắn nào phát hiện được, vì nó phát triển theo tháng và năm. Đây là câu trả lời cho câu hỏi mở đầu thứ tư.

Bụi và vết bẩn trên ống kính. Một vùng ảnh mất tương phản. Ba dấu hiệu đo được: mật độ đặc trưng theo vùng giảm không đều (một góc ảnh sứt trong khi các góc khác bình thường); độ tương phản cục bộ giảm ở cùng vùng đó; và phân bố inlier lệch khỏi vùng ấy. Cả ba đo được mà không cần chân trị, và cả ba nên là một phần của giám sát sức khoẻ ở D/21.

Vì sao ATE *không* phải một dấu hiệu: một vùng ảnh mất đặc trưng không làm hệ thống sai ngay — các đặc trưng còn lại vẫn đủ, và ATE có thể không đổi trong nhiều tháng. Nó chỉ tệ đi khi kết hợp với một điều kiện khó khác, và lúc đó bạn không biết nguyên nhân là bụi. Đây là một ví dụ chung của nguyên tắc: *đo nguyên nhân, không chỉ đo hậu quả*.

Lệch tiêu cự và trôi cơ khí. Ống kính xoay dần do rung; keo lão hoá; khung stereo biến dạng theo nhiệt. Biểu hiện: phần dư có cấu trúc không gian (A/02 mục 4A) hoặc tham số hiệu chuẩn trực tuyến trôi đơn điệu (C/14 mục 4B).

Hỏng pixel và suy giảm cảm biến. Pixel chết tạo ra các đặc trưng giả *đứng yên trong ảnh*. Đây là một chế độ hỏng tinh vi: các đặc trưng ấy ổn định tuyệt đối qua mọi khung, nên chúng trông rất đáng tin — và chúng nói rằng camera không di chuyển. Phát hiện: một đặc trưng có cùng tọa độ pixel qua hàng nghìn khung trong khi hệ thống đang di chuyển là một pixel hỏng, không phải một landmark.

CẦN NHỚ

Do nguyên nhân, không chỉ do hậu quả. Đây là nguyên tắc nổi chương này với D/21.

ATE là hậu quả. Nó tổng hợp mọi nguyên nhân thành một con số, và khi nó xấu đi bạn không biết vì sao. Tệ hơn, nó có thể *không* xấu đi trong nhiều tháng trong khi hệ thống đang suy giảm dần — vì các cơ chế bù trừ vẫn còn dư địa.

Các đại lượng đo nguyên nhân thì rẻ và nhiều thông tin hơn. Danh sách tối thiểu nên giám sát liên tục trong sản phẩm:

Nhóm một: tỉ lệ điểm bị loại vì không nhất quán hình học; diện tích vùng bị mask.

Nhóm hai: số đặc trưng, độ tương phản trung bình, mức gradient trung bình, thời gian phơi sáng.

Nhóm ba: bất đồng giữa các cảm biến; số loop closure bị từ chối ở từng tầng.

Nhóm bốn: mật độ đặc trưng *theo vùng ảnh* (không phải tổng); xu hướng dài hạn của tham số hiệu chuẩn trực tuyến; số đặc trưng có tọa độ pixel không đổi.

Mỗi đại lượng tốn vài byte mỗi khung và có thể tổng hợp thành thống kê hàng ngày. Chúng là thứ biến một báo cáo lỗi “robot bị lạc” thành một chẩn đoán “camera trái bắn từ ba tuần trước”.

19.6 Phát hiện thất bại và suy giảm có kiểm soát

Đây là mục quan trọng nhất của chương, và là câu trả lời cho câu hỏi mở đầu thứ năm.

A. Lập luận chi phí

So sánh hai hệ thống trong một đội hình một nghìn robot.

Hệ A: tỉ lệ lỗi 1% số ca, và khi lỗi nó im lặng — tiếp tục báo tư thế sai. Mười ca mỗi ngày, mỗi ca robot đi sai chỗ, có thể va chạm, và cần người tới xử lý. Chi phí: mười lần can thiệp, cộng rủi ro an toàn, cộng việc khách hàng mất niềm tin.

Hệ B: tỉ lệ lỗi 2%, nhưng nó phát hiện được 90% số ca. Hai mươi ca mỗi ngày, trong đó mười tám ca robot *dừng lại và báo*, và hai ca im lặng. Chi phí: mười tám lần gián đoạn ngắn (robot dừng, tự phục hồi hoặc chờ) cộng hai lần can thiệp.

Hệ B tốt hơn rõ rệt dù tỉ lệ lỗi gấp đôi, vì *một lỗi được phát hiện rẻ hơn một lỗi im lặng nhiều bậc*. Đây là lập luận cho việc chọn phát hiện thất bại thay vì cải thiện độ chính xác khi chỉ có ngân sách cho một cái.

Có một lập luận thứ hai, và nó mạnh hơn về dài hạn: *một lỗi được phát hiện là một lỗi được ghi lại*. Hệ B tạo ra dữ liệu — mười tám bản ghi mỗi ngày về việc nó thất bại ở đâu và trong điều kiện nào. Sau một tháng đó là năm trăm mẫu để phân tích. Hệ A tạo ra các báo cáo mơ hồ từ khách hàng. Đây là vòng lặp cải tiến ở D/21, và nó chỉ khởi động được nếu hệ thống biết mình sai.

B. Bốn tầng phát hiện

Tầng một — các đại lượng tức thời, mỗi khung: số inlier, tỉ lệ tracking, phần dư trung bình. Rẻ nhất, phát hiện sớm nhất, nhiều báo động giả nhất.

Tầng hai — các đại lượng thống kê, trên một cửa sổ: xu hướng của phần dư, phân bố không gian của inlier, số điều kiện của Λ . Bắt được các suy giảm chậm mà tầng một bỏ qua.

Tầng ba — nhất quán chéo giữa các nguồn: bắt đồng giữa thị giác và IMU, giữa thị giác và odometry. Đây là tầng duy nhất có cơ hội bắt được nhóm ba, và nó chỉ tồn tại nếu có nhiều cảm biến.

Tầng bốn — kiểm tính hợp lý vật lý: robot có thể đi nhanh thế này không? Tư thế có nhảy một quãng không thể trong một khung không? Rẻ, và nó bắt được các thất bại thảm họa mà ba tầng kia có thể bỏ qua vì chúng dựa trên các đại lượng nội tại vốn cũng bị ảnh hưởng.

C. Suy giảm có kiểm soát

Phát hiện xong thì làm gì. Nguyên tắc: *chuyển sang chế độ kém chính xác hơn nhưng vẫn hoạt động, và báo rằng mình đang ở chế độ đó.*

Một thang chế độ hợp lý, từ tốt tới xấu: SLAM đầy đủ với bản đồ; SLAM không đóng vòng (khi loop closure không đáng tin); odometry thị giác-quán tính không bản đồ; odometry quán tính-bánh xe (khi thị giác chết); và cuối cùng là dừng và báo.

Hai yêu cầu về kiến trúc, và cả hai phải được thiết kế từ đầu. Một — *chuyển chế độ phải mượt, không có bước nhảy trong đầu ra, vì bộ điều khiển phía trên không chịu được. Hai — chế độ hiện tại phải là một phần của đầu ra, cùng với tư thế và hiệp phương sai. Một module phía trên nhận “tư thế, hiệp phương sai, chế độ” hành xử khác hẳn một module chỉ nhận tư thế — nó biết khi nào nên thận trọng.*

Yêu cầu thứ hai nghe hiển nhiên và nó bị bỏ sót thường xuyên, vì trong nghiên cứu đầu ra của SLAM là một quỹ đạo để so với chân trị, không phải một tín hiệu cho một hệ thống khác dùng.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DynaSLAM [100]	Masking ngữ nghĩa + kiểm tra hình học cho vật động	Mục 2B; chú ý giới hạn của masking cứng
DSO [22]	Mô hình quang trắc + hai tham số affine mỗi khung	Mục 3 — xử lý thay đổi ánh sáng cho direct
ORB-SLAM3 [10]	Atlas cho phục hồi mềm khi mất bám	Mục 6C — một dạng suy giảm có kiểm soát
OpenLORIS [79]	Bộ dữ liệu có người đi lại, ánh sáng đổi, nhiều phiên	Một trong ít bộ dữ liệu chạm vào nội dung chương này
Kimera [63]	Ngữ nghĩa trong đồ thị; hạ trọng số theo lớp đối tượng	Hướng đúng cho mục 2B — hạ trọng số thay vì loại cứng

MINI PROJECT SLAM · Dựng một bộ thử nghiệm nghịch cảnh, rồi đo tỉ lệ phát hiện thất bại · 4 ngày

Mục tiêu	chuyển từ “hệ của tôi chạy tốt” sang “hệ của tôi thất bại trong $X\%$ các tình huống này, và nó <i>biết</i> trong $Y\%$ số lần thất bại”.
Dữ liệu	tự thu, và đây là phần chính của bài: dựng một bộ mười chuỗi nghịch cảnh, mỗi chuỗi nhắm vào một mục trong bốn nhóm — người đi ngang chiếm phần lớn khung hình, hành lang trắng, đèn tắt đột ngột, chuyển động nhanh gây mờ, một tấm gương lớn, một hành lang có các cửa giống nhau, một ống kính bị che một phần bằng băng dính mờ (mô phỏng bụi), và ba chuỗi tổ hợp.
Các bước	chạy hệ thống trên cả mười chuỗi; với mỗi chuỗi ghi lại: có thất bại không (so với chân trị thô hoặc quan sát bằng mắt), và hệ thống có <i>báo</i> rằng nó đang gặp vấn đề không; cài bốn tầng phát hiện ở mục 6B và đo tỉ lệ phát hiện cùng tỉ lệ báo động giả của từng

- tầng; cuối cùng cài một thang chế độ theo mục 6C và đo xem hệ thống có chuyển chế độ đúng lúc không.
- Xong khi** bạn có hai con số cho hệ thống của mình: *tỉ lệ thất bại* trên bộ nghịch cảnh, và *tỉ lệ phát hiện* trong số các lần thất bại. Con số thứ hai là con số mà gần như không ai công bố và nó quan trọng hơn con số thứ nhất. Bạn cũng có một bảng bốn tầng phát hiện với chi phí và hiệu quả của từng tầng — dùng nó để quyết định tầng nào đáng cài trước.
- Nối với** [D/20-toi-uu-hieu-nang-va-he-thong](#) cho chi phí tính toán của các tầng giám sát; [D/21-tu-prototype-len-mass-production](#) biến bộ nghịch cảnh này thành regression test và biến các đại lượng giám sát thành fleet analytics.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1.** Cái vừa được ước lượng là *tư thế tương đối của camera so với chiếc xe tải*, không phải so với thế giới. RANSAC tìm tập nhất quán *lớn nhất*, và khi vật động chiếm đa số thì tập ấy là vật động — nó trung thành chọn xe tải làm inlier và cảnh tĩnh làm outlier. Kết quả sai nhưng *hoàn toàn nhất quán*: mọi điểm trên xe khớp hoàn hảo với mô hình, phần dư nhỏ, số inlier cao, không có gì trong đầu ra báo hiệu. Đây lại là nguyên tắc ở [A/05](#): số inlier đo mức nhất quán của dữ liệu với mô hình, không đo mức đúng đắn của mô hình.
- A2.** Vì một vật động làm hỏng cường độ của *cả một vùng liên thông*, và các pixel trong vùng ấy *tương quan mạnh* — chúng cùng sai theo cùng một cách. M-estimator xử lý outlier độc lập tốt hơn nhiều so với outlier tương quan thành khối, nên đóng góp sai của vùng ấy vào hàm mục tiêu lớn hơn tỉ lệ diện tích của nó. Với indirect, vật động chỉ làm mất vài chục điểm rời rạc và chúng bị hạ trọng số độc lập. Hệ quả kiến trúc: trong môi trường nhiều vật động — nhà xưởng có người, đường phố — indirect có lợi thế thật, và đây là một tiêu chí chọn giữa hai họ ở [B/08](#) không xuất hiện trong các so sánh trên bộ dữ liệu tĩnh.
- A3.** *Nhóm ba — thông tin sai lệch* (kính, gương, vân lặp lại). Không phải vì nó làm hỏng nhiều nhất mà vì nó hỏng *im lặng*: mọi cơ chế phát hiện trong cây này dựa trên tính nhất quán, và ảnh phản chiếu nhất quán với chính nó qua nhiều khung nên không cơ chế nào bắt được. Đây là giới hạn cấu trúc chứ không phải thiếu sót cài đặt. Hệ quả: ba nhóm kia xử lý bằng cách cải thiện thuật toán, nhóm này đòi *thay đổi kiến trúc* — đa dạng hoá cảm biến để có *bất đồng* (tín hiệu duy nhất có sẵn), và giới hạn hậu quả của một lần sai đơn lẻ.
- A4.** Ba dấu hiệu: (1) *mật độ đặc trưng theo vùng ảnh* giảm không đều — một góc sứt trong khi các góc khác bình thường; (2) *độ tương phản cục bộ* giảm ở cùng vùng đó; (3) *phân bố inlier lệch khỏi vùng* ấy. Cả ba đo được không cần chân trị. ATE không phải dấu hiệu vì một vùng mất đặc trưng không làm hệ sai ngay — các đặc trưng còn lại vẫn đủ, và ATE có thể không đổi trong nhiều tháng; nó chỉ tệ đi khi kết hợp với một điều kiện khó khác, và lúc đó bạn không biết nguyên nhân là bại. Nguyên tắc chung: *đo nguyên nhân, không chỉ đo hậu quả*.
- A5.** Chọn *phát hiện thất bại*. Lập luận chi phí trên một nghìn robot: hệ A lỗi 1% và im lặng cho mười ca mỗi ngày, mỗi ca cần người tới xử lý cộng rủi ro an toàn; hệ B lỗi 2% nhưng phát hiện 90% cho mười tám lần gián đoạn ngắn (robot dừng và báo) cộng chỉ hai lần can thiệp. Hệ B tốt hơn rõ rệt dù tỉ lệ lỗi gấp đôi. Có một lập luận thứ hai mạnh hơn về dài hạn: *một lỗi được phát hiện là một lỗi được ghi lại* — hệ B tạo ra năm trăm mẫu mỗi tháng để phân tích, hệ A tạo ra các báo cáo mơ hồ từ khách hàng. Vòng lặp cải tiến ở [D/21](#) chỉ khởi động được nếu hệ thống biết mình sai.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đi trong một toà nhà lạ và nhớ đường bằng cách nhìn các mốc. Có bốn kiểu chuyện có thể xảy ra, và chúng khác nhau về bản chất chứ không chỉ về mức độ.

Kiểu một: có người đi ngang. Bạn ghi nhớ “có anh áo đỏ đứng đây” rồi lát sau quay lại, anh ta đã đi mất. Tệ hơn: nếu bạn đi cùng chiều với anh ta một đoạn dài, bạn sẽ tưởng mình đứng yên — vì so với anh ta thì bạn không nhúc nhích.

Kiểu hai: đèn tắt, hoặc bạn vào một hành lang trắng trơn không có gì để nhớ. Chuyện này khó chịu nhưng dễ chịu nhất trong bốn kiểu, vì *bạn biết* mình đang không có thông tin. Bạn tự khắc đi chậm lại và quờ tay tìm tường.

Kiểu ba: có một tấm gương lớn ở cuối hành lang. Bạn thấy hành lang tiếp tục ở phía trước và đi thẳng vào. Mọi thứ bạn thấy đều hợp lý, rõ ràng, nhất quán — cho tới khi bạn đâm vào gương. Đây là kiểu tệ nhất, vì không có gì báo trước và vì *nhìn kỹ hơn cũng không giúp gì*.

Kiểu bốn: mắt bạn mờ dần theo năm tháng. Mỗi ngày chỉ tệ hơn hôm qua một chút nên không ngày nào bạn nhận ra. Sau vài năm bạn nhìn nhầm nhiều thứ, và bạn vẫn tin vào mắt mình y như cũ.

Bốn kiểu, bốn cách xử lý. Kiểu hai chữa bằng cách dùng giác quan khác — sờ tường, đếm bước. Kiểu một và bốn chữa bằng cách để ý và điều chỉnh. Kiểu ba thì chỉ có một cách: *biết trước rằng gương tồn tại*, và thấy nghi khi một hành lang trông quá hoàn hảo.

Nhưng điều quan trọng nhất thì chung cho cả bốn, và nó không phải một mẹo kỹ thuật. Người biết mình đang không chắc thì đi chậm lại và sờ tường; người không biết thì đi bình thường và ngã. Hai người ấy có thể có cùng thị lực — nhưng một người về được nhà.

Chỗ không rút gọn được: chỗ không rút gọn được là tấm gương. Mọi cách nhận ra sai sót trong chương này đều dựa trên việc “có gì đó không khớp”. Ảnh trong gương khớp hoàn hảo với chính nó, khớp qua nhiều lần nhìn, khớp từ nhiều góc. Không có gì không khớp cả. Cách duy nhất là dùng một giác quan khác — hoặc biết trước.

Một câu: Biết mình đang sai quan trọng hơn không bao giờ sai — vì cái thứ nhất đo được và sửa được, cái thứ hai thì không tồn tại.

19.7 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *phát hiện thất bại có giá trị cao hơn cải thiện độ chính xác* — dựa trên một lập luận chi phí với các con số *tôi tự đặt* để minh họa. Nó bị bác bỏ nếu trong một triển khai thật, chi phí của một lần gián đoạn được phát hiện xấp xỉ chi phí của một lần lỗi im lặng — điều có thể đúng nếu việc robot dừng lại cũng đòi hỏi can thiệp của con người. Với robot có khả năng tự phục hồi thì mệnh đề vững; với robot mà mọi lần dừng đều cần người thì nó yếu đi nhiều. Đây là chỗ tôi phát biểu dựa trên cấu trúc chứ không dựa trên số liệu fleet (*tôi suy ra, chuta đối chiếu nguồn*).

Mệnh đề thứ hai — *nhóm ba nguy hiểm nhất* — là một xếp hạng theo mức khó phát hiện chứ không theo tần suất. Nó bị bác bỏ nếu trong thống kê thật, các thất bại do kính và gương hiểm tới mức không đáng kể so với các thất bại do nghèo vãn. Điều đó *có thể* đúng tùy môi trường — một nhà kho không có gương, một văn phòng hiện đại thì đầy kính. Cách kiểm là thu thống kê nguyên nhân từ fleet, tức đúng việc mà D/21 đề nghị.

Mệnh đề thứ ba — *direct suy giảm nặng hơn indirect với vật động* — suy từ lập luận về tương quan của outlier (cửa a). Nó kiểm được bằng thí nghiệm: chạy cả hai họ trên cùng chuỗi có vật động chiếm các tỉ lệ diện tích khác nhau, và xem đường cong suy giảm. Tôi *chưa* chạy thí nghiệm đó, và nó là một mục trong 99-chua-biet.md.

Mệnh đề thứ tư — *suy luận hình học cấp cao có thể phát hiện gương* — là một phỏng đoán của tôi dựa trên việc ảnh phản chiếu có cấu trúc đặc biệt. Tôi không biết một hệ thực dụng nào làm điều này, và tôi cũng chưa khảo sát xem có ai đã thử và thất bại hay không. Đây là chỗ có rủi ro sai cao nhất trong chương.

19.8 Kết nối

Nói lên trên. [A/01-hình-hoc-da-thi](#) mục 7 nêu bốn giả thiết ngầm; chương này là chỗ trả nợ cho giả thiết cảnh tĩnh. [A/05-observability-gauge-va-suy-bien](#) cho nguyên tắc chung mà mục 2A là một ứng dụng: số inlier không đo mức đúng đắn của mô hình. [A/06-bat-dinh-va-lan-truyen-sai-so](#) cho khái niệm hỏng im lặng, mà nhóm ba là biểu hiện

cực đoan nhất. [B/08-mo-hinh-phan-du](#) là chỗ mục 2C cho thêm một tiêu chí chọn giữa indirect và direct.

Nổi ngang. [C/13-quan-ly-ban-do](#) mục 4C bàn một dạng khác của cùng vấn đề: thế giới thay đổi theo tháng thay vì theo giây. [C/14-hieu-chuan-nhu-mot-nhanh-cua-slam](#) mục 4B cho cơ chế phát hiện trôi hiệu chuẩn, tức nhóm bốn. [C/15-hop-nhat-da-cam-bien](#) cho một lập luận về đa cảm biến; mục 4C ở đây cho một lập luận khác và có lẽ mạnh hơn — đa dạng hoá để có bất đồng. [C/17-hoc-may-trong-slam](#) cho công cụ ngữ nghĩa dùng ở mục 2B.

Nổi xuống dưới. [D/20-toi-uu-hieu-nang-va-he-thong](#) cho ngân sách tính toán của các tầng giám sát ở mục 6B — chúng không miễn phí. [D/21-tu-prototype-len-mass-production](#) là chỗ chương này trở thành quy trình: bộ nghịch cảnh thành regression test, các đại lượng giám sát thành fleet analytics, và tỉ lệ phát hiện thất bại thành một chỉ tiêu nghiệm thu. [D/22-huong-nghien-cuu-con-trong](#) lấy *dự đoán thất bại* — dự đoán trước khi mất bám thay vì phục hồi sau — làm hướng số bốn, và mục 6 của chương này là lập luận vì sao nó đáng giá.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao RANSAC chọn vật động khi nó chiếm đa số, và cái gì thực sự được ước lượng lúc đó?
2. Nêu ba tầng xử lý vật động, và nói vì sao masking ngữ nghĩa nên hạ trọng số thay vì loại cứng.
3. Vì sao direct suy giảm nặng hơn indirect khi có vật động? Trả lời bằng ngôn ngữ tương quan của outlier.
4. Nhóm nào trong bốn nhóm nguy hiểm nhất và vì sao? Nêu hai hướng thay đổi kiến trúc mà nó đòi hỏi.
5. Nêu ba dấu hiệu của bụi ống kính, và nói vì sao ATE không phải một trong ba.
6. Nêu bốn tầng phát hiện thất bại, và nói tầng nào là tầng duy nhất có cơ hội bắt được nhóm ba.
7. Trình bày lập luận chi phí cho việc ưu tiên phát hiện thất bại hơn độ chính xác, và nêu điều kiện mà lập luận đó yếu đi.

Tối ưu hiệu năng và hệ thống (Performance and Systems Engineering)

PHỤ THUỘC A/04-uoc-luong-map-va-bundle-adjustment (cấu trúc thừa — nguồn của phần lớn chi phí và của phần lớn cơ hội tối ưu); B/09-paradigm-uoc-luong-back-end (lựa chọn kiến trúc quyết định hồ sơ chi phí); C/13-quan-ly-ban-do mục 4A (kiến trúc nhiều luồng — chương này bàn chi tiết kỹ thuật của nó).

TÓM TẮT

Chương này về khoảng cách giữa một thuật toán đúng và một hệ thống chạy được trên phần cứng thật, trong thời gian thật, với ngân sách điện thật. Đó là khoảng cách mà bài báo không mô tả và nó chiếm phần lớn thời gian kỹ thuật của một dự án. Nội dung có bốn phần. Một: thời gian — và luận điểm trung tâm là *phân vị chứ không phải trung bình*, cùng cách xử lý độ trễ và mất khung mà không làm hỏng tính đúng đắn. Hai: kiến trúc đa luồng, nơi ba nhịp độ của C/13 gặp thực tế của khoá, hàng đợi và cạnh tranh dữ liệu — và nơi *tính tất định* nổi lên như một yêu cầu không thể thương lượng. Ba: ngân sách tính toán và bộ nhớ — SIMD, lượng tử hoá, bộ giải thừa, và các quyết định về nơi đặt mỗi phần của pipeline. Bốn: một vòng lặp vật lý mà gần như mọi tài liệu bỏ qua — điện năng, nhiệt, và việc nhiệt làm trôi hiệu chuẩn. Nếu chỉ nhớ một điều: *một hệ nhanh trung bình nhưng thỉnh thoảng chậm là một hệ không dùng được trong vòng điều khiển* — và mọi quyết định trong chương này xoay quanh việc làm cho trường hợp xấu nhất có giới hạn.

KIẾN THỨC CẦN CÓ TRƯỚC

- **phân vị (percentile)** — giá trị mà một tỉ lệ nhất định các mẫu nằm dưới nó. Phân vị 99 của thời gian xử lý là ngưỡng mà 99% số khung xử lý xong dưới đó.
- **jitter** — độ biến thiên của thời gian xử lý giữa các khung. Một hệ có jitter lớn khó dùng trong vòng điều khiển ngay cả khi trung bình thấp.
- **tính tất định (determinism)** — cùng đầu vào cho cùng đầu ra, chính xác tới từng bit, qua nhiều lần chạy. Điều kiện để tái hiện một lỗi từ hiện trường.
- **SIMD** — *single instruction multiple data* — chỉ thị xử lý nhiều phần tử cùng lúc. NEON trên ARM, AVX trên x86.
- **allocation-free hot loop** — vòng lặp chính không cấp phát bộ nhớ động. Cấp phát có thời gian không xác định và có thể gây phân mảnh, nên nó là nguồn jitter.
- **thermal throttling** — việc giảm tần số xử lý khi nhiệt độ vượt ngưỡng. Trên thiết bị nhúng không có quạt, đây là ràng buộc thật chứ không phải trường hợp biên.
- **ISP** — *image signal processor* — khối phần cứng xử lý ảnh thô từ cảm biến. Các thao tác của nó (auto-exposure, gamma) ảnh hưởng tới SLAM và thường không điều khiển được đầy đủ.

CÂU HỎI MỞ ĐẦU

- Q1. Hệ của bạn xử lý trung bình 18 ms mỗi khung, dư dả cho 30 Hz. Nó vẫn không dùng được trong vòng điều khiển. Nêu đại lượng bạn chưa đo, và giải thích vì sao nó quyết định.
- Q2. Bạn tối ưu và giảm thời gian trung bình từ 18 ms xuống 12 ms. Nêu một trường hợp mà thay đổi đó làm hệ thống *tệ đi*.

- Q3.** Đồng nghiệp đề nghị dùng số thực độ chính xác đơn thay vì kép để tăng tốc gấp đôi. Nên điều kiện để làm được điều đó một cách an toàn, và chỉ ra chương nào trong cây cung cấp điều kiện ấy.
- Q4.** Một lỗi xảy ra ở hiện trường và bạn có log đầy đủ. Bạn chạy lại trên máy của mình và không tái hiện được. Nên nguyên nhân có khả năng nhất, và nói nó lẽ ra phải được ngăn từ lúc nào.
- Q5.** Thiết bị chạy tốt trong mười phút rồi bắt đầu kém chính xác dần, và nó hồi phục sau khi nghỉ. Nên hai cơ chế hoàn toàn khác nhau có thể gây ra điều đó.

20.1 Đặt vấn đề

Ba phần trước nói về thuật toán đúng. Chương này về khoảng cách giữa một thuật toán đúng và một hệ thống chạy được, và khoảng cách ấy lớn hơn nhiều so với vẻ ngoài.

Bài toán gốc: thuật toán chạy trên laptop với bộ dữ liệu đọc từ đĩa không phải thứ chạy trên robot. Ba khác biệt.

Một — thời gian thật. Camera gửi khung ở 30 Hz bất kể bạn đã xử lý xong hay chưa. Không có nút tạm dừng.

Hai — tài nguyên hữu hạn. Một SoC nhúng có vài watt, vài gigabyte RAM, và không có quạt.

Ba — và đây là khác biệt ít được nói nhất: hệ thống là một phần của một hệ thống lớn hơn. Bộ điều khiển cần tư thế đúng lúc; nó không chờ được. Một tư thế chính xác đến sau khi cần thì vô dụng.

Ai gặp bài toán này? Ai đưa SLAM lên thiết bị. Và đây là phần mà kiến thức thuộc về kỹ thuật phần mềm hệ thống chứ không thuộc về thị giác máy tính — một lý do nữa vì sao nó thiếu trong tài liệu SLAM.

Có một cách nhìn xuyên suốt chương, và nó gọn: *mọi thứ trong chương này là về việc làm cho trường hợp xấu nhất có giới hạn.* Không phải làm cho trung bình nhanh hơn — trung bình gần như luôn đủ nhanh. Cái gây ra thất bại là một khung mất một giây, một lần cấp phát gây phân mảnh, một lần throttling làm mọi thứ chậm gấp đôi.

TRỰC GIÁC

Hình dung bạn đi xe đạp và cần nhìn đường.

Nếu bạn nhắm mắt trung bình 0,1 giây mỗi lần chớp mắt, không sao — bạn vẫn đạp được cả ngày.

Nếu thỉnh thoảng bạn nhắm mắt *hai giây*, bạn sẽ đâm vào cột điện. Và điều quan trọng: việc trung bình chỉ có 0,1 giây *không cứu bạn* ở lần hai giây ấy. Các lần nhanh không bù được cho lần chậm.

Đó là toàn bộ lý do phân vị quan trọng hơn trung bình. Với một hệ chỉ cần cho ra kết quả cuối cùng — một phép quét để in 3D chẳng hạn — trung bình là đại lượng đúng, vì tổng thời gian là thứ ta quan tâm. Với một hệ nằm trong vòng điều khiển, mỗi chu kỳ là một hạn chót riêng, và trễ một lần là hỏng một lần.

Có một chuyện thứ hai, cũng về sự bất đối xứng. Nếu bạn phải chọn giữa “nhìn kỹ và biết chính xác mình đang ở đâu, nhưng chậm” và “nhìn lướt và biết đại khái, nhưng kịp”, thì với xe đạp, lựa chọn thứ hai đúng. Một câu trả lời gần đúng đúng lúc hữu ích hơn một câu trả lời chính xác đến muộn. Đây là nguyên tắc chi phối cách thiết kế kiến trúc ở mục 3.

Và chuyện thứ ba, ít ai nghĩ tới. Đạp xe lâu thì bạn nóng người và mệt, và khi mệt thì bạn nhìn kém đi. Cái máy cũng vậy: chạy lâu thì nóng, nóng thì tự giảm tốc, và — điều bất ngờ

— nóng còn làm *ống kính giãn ra*, tức làm sai chính phép đo. Một vòng lặp vật lý mà không tài liệu phần mềm nào nhắc tới.

20.2 Thời gian: phân vị, không phải trung bình

A. Vì sao trung bình không đủ

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất: đại lượng chưa đo là *phân vị 99 của thời gian xử lý*.

Lập luận. Một hệ trong vòng điều khiển có một hạn chót *mỗi chu kỳ*. Nếu chu kỳ là 33 ms và một khung mất 200 ms, thì sáu chu kỳ điều khiển không có dữ liệu mới. Bộ điều khiển phải ngoại suy, và với robot đang di chuyển, sáu chu kỳ ngoại suy có thể là mười centimet sai lệch.

Và điều quan trọng: *các khung nhanh không bù được*. Trung bình là một đại lượng cộng gộp, phù hợp khi ta quan tâm tổng; ở đây mỗi chu kỳ là một sự kiện độc lập với một hạn chót riêng.

Ba nguồn của các khung chậm, theo thứ tự phổ biến trong SLAM.

Một — *keyframe*. Khi một khung được chọn làm keyframe, local mapping chạy, và chi phí nhảy vọt. Nếu nó chạy trên cùng luồng với tracking thì mỗi keyframe là một lần trễ.

Hai — *loop closure*. Hiếm hơn nhưng đắt hơn nhiều: pose graph cộng cập nhật bản đồ có thể mất hàng giây.

Ba — *cấp phát bộ nhớ và thu gom*. Một lần cấp phát lớn có thể mất mili giây và có thời gian không xác định.

B. Vì sao tối ưu trung bình có thể làm tệ đi

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một cái bẫy thật.

Ví dụ cụ thể: bạn thêm một bộ nhớ đệm để tránh tính lại một thứ. Trung bình giảm vì hầu hết khung trúng đệm. Nhưng khi trượt đệm, chi phí cao hơn trước — vì giờ phải tính lại *và* cập nhật đệm. Phân vị 99 tăng lên.

Ví dụ thứ hai, phổ biến hơn: bạn tăng số đặc trưng để chính xác hơn. Trung bình tăng một chút. Nhưng trong các khung khó — nhiều đặc trưng hơn nghĩa là nhiều ứng viên hơn nghĩa là RANSAC lặp lâu hơn — phân vị 99 tăng nhiều.

Quy tắc thực hành: *đo phân vị 99 trước và sau mọi thay đổi hiệu năng*, và coi một thay đổi làm tăng phân vị 99 là một hồi quy kể cả khi trung bình giảm.

C. Xử lý mất khung và độ trễ

Nguyên tắc quan trọng nhất: *không bao giờ được chặn*. Nếu tracking chưa xử lý xong khung này và khung tiếp theo tới, hành vi đúng là *bỏ khung* chứ không phải xếp hàng.

Vì sao xếp hàng sai: hàng đợi làm độ trễ tăng dần và không bao giờ phục hồi. Sau một khung chậm, mọi khung sau đều trễ một chút, và độ trễ tích lũy. Với một vòng điều khiển, một tư thế đúng nhưng cũ 300 ms tệ hơn một tư thế kém chính xác hơn nhưng mới.

Ba chi tiết cài đặt đáng nhớ. *Một*: hàng đợi giữa các luồng nên có kích thước *một*, và ghi đè thay vì xếp hàng. *Hai*: mỗi khung phải mang *mốc thời gian thu*, không phải thời gian xử lý — mọi nhân tử trong đồ thị dùng mốc thu. *Ba*: đầu ra phải kèm *tuổi* của nó, để module phía trên biết mà chiết khấu; đây là phần mở rộng của nguyên tắc ở D/19 mục 6C.

20.3 Kiến trúc đa luồng và tính tất định

A. Ba luồng, ba nhịp độ

C/13 mục 4A nêu ba nhịp độ: tracking ở tần số khung, local mapping ở tần số keyframe, loop closing hiếm và lâu. Chương này bàn chi tiết kỹ thuật.

Nguyên tắc phân chia: *luồng có hạn chót cứng phải được cô lập*. Tracking có hạn chót; hai luồng kia không. Nên tracking không bao giờ được chờ hai luồng kia, kể cả khi nó cần dữ liệu mà chúng đang sửa.

Ba mẫu thiết kế giải quyết điều đó.

Bản sao đọc. Tracking làm việc trên một bản sao của bản đồ cục bộ, được cập nhật ở các điểm an toàn. Tồn bộ nhớ, nhưng tracking không bao giờ bị chặn.

Hoán đổi con trỏ. Local mapping xây dựng phiên bản mới của một cấu trúc rồi hoán đổi con trỏ trong một thao tác nguyên tử. Tracking thấy hoặc phiên bản cũ hoặc mới, không bao giờ thấy trạng thái nửa vời.

Hàng đợi một chiều. Tracking gửi keyframe cho local mapping qua một hàng đợi và *không chờ kết quả*. Kết quả quay lại qua một trong hai cơ chế trên.

B. Tính tất định là một yêu cầu, không phải xa xỉ

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và là luận điểm mà tôi cho là quan trọng nhất của chương.

Kịch bản: một robot ở hiện trường gặp lỗi. Bạn có log đầy đủ — mọi ảnh, mọi phép đo IMU, mọi mốc thời gian. Bạn chạy lại và *không tái hiện được*.

Nguyên nhân có khả năng nhất: *hệ thống không tất định*. Thứ tự luồng khác nhau làm keyframe khác được chọn; RANSAC với seed khác cho inlier khác; một khung bị bỏ ở lần chạy này mà không bị bỏ ở lần kia.

Hậu quả: bạn không debug được. Bạn có thể đoán, sửa, và không bao giờ biết mình đã sửa đúng chưa. Với một lỗi xảy ra một lần trong nghìn giờ, đây là sự khác biệt giữa sửa được và không.

Ba việc phải làm, và từ đầu dự án chứ không phải khi gặp lỗi.

Một — seed cố định và có thể tái lập. Mọi nguồn ngẫu nhiên (RANSAC, khởi tạo) phải lấy seed từ một nguồn ghi vào log.

Hai — chế độ chạy lại tất định. Một chế độ trong đó dữ liệu được đưa vào theo mốc thời gian ghi trong log, không theo thời gian thực, và các luồng chạy theo một thứ tự cố định. Chế độ này chạy chậm hơn thời gian thực và điều đó chấp nhận được — mục đích của nó là chẩn đoán.

Ba — ghi lại mọi quyết định không tất định. Khung nào bị bỏ, keyframe nào được chọn, cổng nào đóng mở. Nếu không tái lập được hoàn toàn, ít nhất có thể tái lập các quyết định.

Cái giá của tính tất định là thật: nó hạn chế một số tối ưu (xử lý song song không thứ tự), và nó có thể chậm hơn vài phần trăm. Đó là một cái giá đáng trả, và nó là một trong những lý do chính đáng nhất để chấp nhận chạy chậm hơn một chút — cùng lập luận với việc chọn ORB thay vì một front-end học được ở **B/07** mục 2B.

20.4 Ngân sách tính toán và bộ nhớ

A. Biết chi phí nằm ở đâu

Trước khi tối ưu, đo. Hồ sơ chi phí điển hình của một hệ SLAM thị giác, theo thứ tự thường gặp: trích và mô tả đặc trưng thường là mục lớn nhất; khớp và RANSAC; bundle adjustment cục bộ; và các phần còn lại.

Tôi *không* đưa ra tỉ lệ phần trăm cụ thể, vì chúng phụ thuộc hoàn toàn vào độ phân giải, số đặc trưng, kích thước cửa sổ và phần cứng. Điều đáng nói là một nguyên tắc: *đo trước khi tối ưu*, và đo trên *phần cứng mục tiêu* — hồ sơ chi phí trên laptop x86 và trên SoC ARM khác nhau đáng kể, vì tỉ lệ giữa tốc độ CPU và băng thông bộ nhớ khác nhau.

B. Các kỹ thuật, theo thứ tự tỉ lệ lợi ích trên công sức

Giảm công việc trước khi tăng tốc công việc. Cách rẻ nhất để làm một phép tính nhanh hơn là không làm nó. Ví dụ cụ thể trong SLAM: thu hẹp vùng tìm kiếm bằng tiên nghiệm chuyển động (B/07 mục 5C) giảm chi phí khớp hàng chục lần; chọn keyframe tiết kiệm (C/13 mục 2) giảm chi phí BA theo lũy thừa ba.

SIMD cho các vòng lặp nóng. Tính descriptor, tính khoảng cách Hamming, warp ảnh — các thao tác này song song hoá hoàn hảo ở mức dữ liệu. Lợi ích lớn và công sức vừa phải.

Vòng lặp nóng không cấp phát. Cấp phát trước mọi bộ đệm và tái sử dụng. Lợi ích chính không phải tốc độ trung bình mà là *giảm jitter* — tức đúng thứ mục 2 nói là quan trọng.

Bộ giải thừa tối ưu. Khai thác cấu trúc khối cụ thể của bài toán thay vì dùng bộ giải thừa tổng quát (A/04 mục 4). Công sức lớn, lợi ích lớn.

Độ chính xác đơn. Đây là câu trả lời cho câu hỏi mở đầu thứ ba. Nó nhanh gấp đôi và tốn nửa bộ nhớ, nhưng nó nguy hiểm với bài toán có số điều kiện lớn — và BA có số điều kiện lớn (B/09 mục 5A). Điều kiện để làm an toàn: *dùng dạng căn bậc hai*, vốn có số điều kiện là căn bậc hai của dạng hệ chuẩn [37]. B/09 mục 5A là chương cung cấp điều kiện ấy, và đây là một ví dụ đẹp về việc một lựa chọn toán học mở ra một tối ưu hệ thống.

C. Đặt cái gì ở đâu

Trên SoC hiện đại có nhiều khối tính toán, và việc phân bổ là một quyết định thiết kế.

CPU cho logic điều khiển và bộ giải thừa — các phép toán có nhiều nhánh và truy cập bộ nhớ không đều, nơi GPU không giúp. GPU cho các thao tác dày và đều — trích đặc trưng, mạng nơ-ron, stereo dày. NPU cho mạng, khi có. ISP cho tiền xử lý — nhưng với một cảnh báo quan trọng.

Cảnh báo về ISP: nó thường thực hiện các thao tác mà SLAM không muốn — auto-exposure thay đổi giữa các khung (vấn đề cho direct, A/02 mục 5B), khử nhiễu làm mất chi tiết nhỏ (vấn đề cho detector), và gamma phi tuyến (vấn đề cho mô hình quang trắc). Và các thao tác ấy thường không tắt được hoặc không điều khiển được đầy đủ.

Quy tắc: *lấy ảnh càng gần raw càng tốt*, và nếu phải dùng ISP thì *ghi lại tham số của nó cho mỗi khung* — thời gian phơi sáng và gain phải là một phần của dữ liệu, vì A/02 cần chúng. Đây là một yêu cầu với nhà cung cấp module camera, và nó nên được nêu *trước khi chọn module*.

I 20.5 Điện năng, nhiệt, và một vòng lặp bị bỏ quên

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và là mục mà tôi cho là đóng góp riêng của chương.

Hai cơ chế hoàn toàn khác nhau giải thích hiện tượng “chạy tốt mười phút rồi kém dần, hồi phục sau khi nghỉ”.

Cơ chế một — thermal throttling. SoC nóng lên, tần số giảm, thời gian xử lý tăng. Hệ quả theo mục 2: nhiều khung bị bỏ hơn, cửa sổ BA phải thu nhỏ, độ trễ tăng. Đây là cơ chế mà mọi kỹ sư nhúng đều biết.

Cơ chế hai — trôi hiệu chuẩn do nhiệt. Ống kính và khung giãn nở, tiêu cự đổi vài phần vạn, và với stereo thì đường cơ sở cùng góc giữa hai camera đổi. Theo A/02 mục 4C, sai số hiệu chuẩn chảy vào tư thế và cấu trúc như một sai lệch có hệ thống.

Hai cơ chế cho cùng triệu chứng và cần hai cách chữa hoàn toàn khác nhau. Cơ chế một chữa bằng quản lý nhiệt và giảm tải; cơ chế hai chữa bằng hiệu chuẩn trực tuyến (C/14 mục 3) hoặc bằng mô hình bù theo nhiệt độ.

Phép thử phân biệt, và nó rẻ: *ghi nhiệt độ vỏ máy cùng với các đại lượng giám sát.* Nếu thời gian xử lý tăng theo nhiệt thì là cơ chế một; nếu *phần dư có cấu trúc* hoặc *tham số hiệu chuẩn trực tuyến* tương quan với nhiệt thì là cơ chế hai. Hai dấu hiệu ấy độc lập, nên phép thử phân biệt được rõ ràng — và cả hai có thể xảy ra cùng lúc.

Điều đáng nói: vòng lặp “điện năng → nhiệt → trôi ống kính → trôi hiệu chuẩn → sai số thị giác” gần như không xuất hiện trong tài liệu SLAM, vì nó nằm ở giao của ba lĩnh vực — nhiệt học, quang cơ, và thị giác — và không ai sở hữu nó. Tôi nêu nó ở đây vì tôi cho rằng nó là một trong những nguyên nhân hay bị chần đoán nhầm nhất trong triển khai thật (*đây là suy luận của tôi từ A/02 mục 4C và C/14 mục 4B; tôi không có số liệu đo và không dẫn được nguồn nào đã đo vòng lặp này một cách hệ thống*).

Hệ quả thiết kế, và nó thuộc về D/21: *đặt một cảm biến nhiệt gần module camera và ghi giá trị của nó cùng mọi dữ liệu.* Chi phí vài xu; giá trị chẩn đoán rất cao; và không thêm được sau khi đã mở khuôn.

CẦN NHỚ

Bốn quy tắc của chương này, xếp theo mức quan trọng.

Một — đo phân vị, không đo trung bình. Một hệ nhanh trung bình nhưng thỉnh thoảng chậm là hệ không dùng được trong vòng điều khiển, vì các lần nhanh không bù được cho lần chậm. Coi một thay đổi làm tăng phân vị 99 là hồi quy kể cả khi trung bình giảm.

Hai — không bao giờ chặn; bỏ khung thay vì xếp hàng. Hàng đợi làm độ trễ tăng dần và không phục hồi. Một tư thế kém chính xác đúng lúc hữu ích hơn một tư thế chính xác đến muộn.

Ba — tắt định từ đầu dự án. Không có nó thì không tái hiện được lỗi hiện trường, và với một lỗi xảy ra một lần trong nghìn giờ, đó là khác biệt giữa sửa được và không. Cái giá vài phần trăm tốc độ là đáng trả.

Bốn — ghi nhiệt độ. Vài xu cho một cảm biến, và nó là thứ phân biệt được hai cơ chế suy giảm hoàn toàn khác nhau mà nếu không có thì bạn sẽ chẩn đoán nhầm.

Và một quy tắc bao trùm: **mọi thứ trong chương này là về giới hạn trường hợp xấu nhất**, không về cải thiện trung bình. Trung bình gần như luôn đủ nhanh.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Basalt [23]	Dạng căn bậc hai; chất lượng kỹ thuật phần mềm rất cao	Mục 4B — ví dụ về việc lựa chọn toán học mở ra tối ưu hệ thống

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [10]	Ba luồng với hàng đợi và khoá	Mục 3A; đọc mã để thấy các mẫu thiết kế thật, gồm cả chỗ chúng phức tạp hơn mô tả trong bài báo
VINS-Mono [13]	Tách luồng estimator và luồng loop closure	Kiến trúc gọn, dễ đọc để học nguyên tắc cô lập luồng có hạn chót
Ceres [35]	Bộ giải thưa với nhiều lựa chọn ordering và linear solver	Mục 4B; thử các lựa chọn trên bài toán của bạn thay vì dùng mặc định
OpenCV	Nhiều hàm đã có phiên bản SIMD	Mục 4B; kiểm xem build của bạn có bật đúng cờ kiến trúc không — một lỗi phổ biến

MINI PROJECT SLAM · Đo hồ sơ thời gian trên phần cứng mục tiêu, rồi tấn công phân vị 99 · 4 ngày

Mục tiêu	chuyển từ “hệ của tôi chạy 30 Hz” sang “phân vị 99 của tôi là X ms, và nguồn của nó là Y ”.
Dữ liệu	một chuỗi dài (ít nhất mười phút) chạy trên <i>phần cứng mục tiêu</i> , không phải laptop. Nếu chưa có phần cứng mục tiêu, dùng thứ gần nhất và ghi rõ rằng kết quả sẽ khác.
Các bước	đo thời gian của từng giai đoạn pipeline cho <i>mỗi khung</i> và lưu lại toàn bộ; vẽ histogram và phân vị 50, 95, 99, 99,9; xác định các khung ở đuôi và <i>tìm ra chúng có gì chung</i> — có phải keyframe không, có loop closure không, có phải lần cấp phát lớn không; ghi nhiệt độ trong suốt và vẽ thời gian xử lý theo nhiệt độ; rồi chọn <i>một</i> nguồn ở đuôi và sửa nó, đo lại.
Xong khi	bạn có một biểu đồ phân vị trước và sau, và một lời giải thích cho <i>từng</i> đỉnh ở đuôi. Điều quan trọng nhất: bạn phải trả lời được câu “khung chậm nhất của tôi chậm vì cái gì” cho ít nhất ba nguồn khác nhau. Nếu đường cong thời gian theo nhiệt độ có xu hướng tăng, bạn vừa đo được cơ chế một ở mục 5 trên chính thiết bị của mình.
Nối với	D/21-tu-prototype-len-mass-production biến các đại lượng này thành giám sát fleet; D/19-ben-vung-trong-the-gioi-thuc mục 6 dùng chúng làm đầu vào cho phát hiện thất bại.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Đại lượng chưa đo là *phân vị 99 của thời gian xử lý*. Một hệ trong vòng điều khiển có hạn chót *mỗi chu kỳ*, và các khung nhanh không bù được cho khung chậm — trung bình là đại lượng cộng gộp, phù hợp khi ta quan tâm tổng, nhưng ở đây mỗi chu kỳ là một sự kiện độc lập với hạn chót riêng. Nếu một khung mất 200 ms thì sáu chu kỳ điều khiển không có dữ liệu mới, và với robot đang di chuyển đó có thể là mười centimet sai lệch. Ba nguồn thường gặp của khung chậm: keyframe kích hoạt local mapping, loop closure, và cấp phát bộ nhớ.
- A2. Ví dụ: bạn thêm một bộ nhớ đệm. Trung bình giảm vì hầu hết khung trúng đệm, nhưng khi *trượt* đệm thì chi phí cao hơn trước — phải tính lại *và* cấp nhật đệm — nên phân vị 99 tăng. Ví dụ thứ hai phổ biến hơn: tăng số đặc trưng để chính xác hơn làm trung bình tăng một chút, nhưng trong các khung khó thì nhiều ứng viên hơn nghĩa là RANSAC lặp lâu hơn, và phân vị 99 tăng nhiều. Quy tắc: đo phân vị 99 trước và sau mọi thay đổi hiệu năng, và coi việc nó tăng là một hồi quy kể cả khi trung bình giảm.
- A3. Điều kiện: *dùng dạng căn bậc hai* thay vì hệ chuẩn. Vì $\text{cond}(\Lambda) = \text{cond}(R)^2$, việc dựng Λ tương minh phá huỷ một nửa số chữ số có nghĩa — với bài toán BA có số điều kiện 10^8 , độ chính xác đơn (khoảng bảy chữ số) không đủ. Làm việc với R giữ lại số điều kiện ở mức căn bậc hai, và [37] cho thấy điều đó cho phép dùng độ chính xác đơn trong nhiều trường hợp. Chương cung cấp điều kiện là [B/09](#) mục 5A — một ví dụ đẹp về việc một lựa chọn toán học mở ra một tối ưu hệ thống.

- A4. Nguyên nhân có khả năng nhất: *hệ thống không tắt định* — thứ tự luồng khác làm keyframe khác được chọn, RANSAC với seed khác cho inlier khác, một khung bị bỏ ở lần này mà không bị ở lần kia. Hậu quả: bạn không debug được; bạn đoán, sửa, và không bao giờ biết đã sửa đúng chưa. Lẽ ra phải ngăn từ đầu dự án, bằng ba việc: seed cố định ghi vào log; một chế độ chạy lại tắt định trong đó dữ liệu vào theo mốc thời gian ghi trong log và luồng chạy theo thứ tự cố định; và ghi lại mọi quyết định không tắt định (khung nào bị bỏ, keyframe nào được chọn). Cái giá vài phần trăm tốc độ là đáng trả.
- A5. Hai cơ chế. (1) *Thermal throttling*: SoC nóng, tần số giảm, thời gian xử lý tăng, nhiều khung bị bỏ hơn. (2) *Trôi hiệu chuẩn do nhiệt*: ống kính và khung giãn nở, tiêu cự đổi, và với stereo thì đường cơ sở cùng góc giữa hai camera đổi — sai số hiệu chuẩn chảy vào tư thế như một sai lệch có hệ thống (A/02 mục 4C). Hai cơ chế cho cùng triệu chứng và cần hai cách chữa ngược nhau: quản lý nhiệt so với hiệu chuẩn trực tuyến. Phép thử phân biệt rõ: ghi nhiệt độ vỏ máy; nếu *thời gian xử lý* tăng theo nhiệt thì là (1), nếu *phần dư có cấu trúc* hoặc tham số hiệu chuẩn trực tuyến tương quan với nhiệt thì là (2). Cả hai có thể xảy ra cùng lúc.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ về việc đạp xe và cần nhìn đường.

Nếu trung bình bạn chớp mắt một phần mười giây, không sao cả. Nhưng nếu thỉnh thoảng bạn nhắm mắt hai giây, bạn sẽ đâm vào cột điện — và việc trung bình chỉ có một phần mười giây *không cứu bạn* ở lần hai giây ấy.

Đó là chuyện quan trọng nhất của chương. Khi một cái máy phải trả lời liên tục cho một thứ khác đang chuyển động, thì thứ đáng lo không phải là nó trả lời nhanh bao nhiêu *trung bình*, mà là lần chậm nhất của nó chậm bao lâu. Các lần nhanh không bù được cho lần chậm.

Chuyện thứ hai cũng về sự lựa chọn giữa nhanh và đúng. Nếu phải chọn giữa “nhìn thật kỹ rồi mới trả lời” và “nhìn lướt nhưng trả lời kịp”, thì với người đang đạp xe, lựa chọn thứ hai đúng. Một câu trả lời gần đúng đúng lúc hữu ích hơn một câu trả lời chính xác đến muộn. Và có một hệ quả cụ thể: khi máy xử lý không kịp, đừng để nó xếp hàng — hãy để nó *bỏ qua* và nhìn tấm ảnh mới nhất. Xếp hàng thì nó sẽ luôn trả lời về chuyện đã xảy ra vài giây trước.

Chuyện thứ ba ít ai nghĩ tới nhưng lại là chuyện tốn nhiều thời gian nhất. Giả sử máy hỏng ở nhà khách hàng và bạn có đầy đủ ghi chép về mọi thứ nó thấy. Bạn chạy lại y hệt trên máy của mình — và nó chạy bình thường. Chuyện này xảy ra khi máy *không chạy giống nhau qua hai lần*, vì có chỗ nào đó dựa vào may rủi hoặc vào thứ tự các việc chạy song song. Lúc đó bạn không thể sửa, vì bạn không tái hiện được lỗi. Và cách duy nhất tránh là làm cho nó chạy giống hệt nhau *ngay từ đầu* — thêm vào sau thì gần như không làm được.

Và chuyện cuối, thuộc về vật lý chứ không phải phần mềm. Chạy lâu thì máy nóng. Nóng thì nó tự chạy chậm lại — điều ai cũng biết. Nhưng nóng còn làm *ống kính giãn ra một chút*, tức làm sai chính phép đo — điều gần như không ai nhắc. Hai chuyện này gây ra cùng một triệu chứng: máy chạy tốt mười phút rồi kém dần, rồi hồi phục sau khi nghỉ. Chúng cần hai cách chữa hoàn toàn khác nhau, và cách phân biệt chỉ tốn một cái cảm biến nhiệt giá vài xu.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *phải quyết định các chuyện này từ rất sớm*. Tính tắt định không thêm vào được sau; cảm biến nhiệt không gắn thêm được sau khi đã mở khuôn nhựa. Đó là những quyết định phải đưa ra lúc chưa ai thấy chúng cần thiết, và đó là lý do chúng hay bị bỏ qua.

Một câu: *Đừng hỏi trung bình bao nhanh — hỏi lần chậm nhất chậm bao lâu.*

20.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *phân vị quan trọng hơn trung bình* — đúng cho hệ nằm trong vòng điều khiển và *sai* cho hệ xử lý ngoại tuyến, nơi tổng thời gian là thứ đáng quan tâm. Nó cũng yếu đi nếu module phía trên có khả năng ngoại suy tốt và chịu được độ trễ — một bộ điều khiển có mô hình

động học tốt có thể chịu được vài chu kỳ thiếu dữ liệu. Nên phát biểu chặt hơn: *phân vị quan trọng theo tỉ lệ với mức mà module phía trên không chịu được độ trễ*.

Mệnh đề thứ hai — *tính tất định là yêu cầu bắt buộc* — bị bác bỏ nếu trong thực tế, phần lớn lỗi hiện trường tái hiện được mà không cần nó, hoặc nếu các kỹ thuật ghi log đủ chi tiết thay thế được. Tôi cho là không, nhưng mức độ phụ thuộc vào tần suất các lỗi hiếm — và với một hệ ổn định thì có thể chúng đủ hiếm để không đáng đầu tư. Đây là một đánh đổi phụ thuộc dự án.

Mệnh đề thứ ba — *vòng lặp nhiệt gây trôi hiệu chuẩn* — là suy luận của tôi từ hai chương khác, và tôi đã ghi rõ rằng *không có số liệu đo và không dẫn được nguồn nào đã đo nó một cách hệ thống*. Đây là mệnh đề có rủi ro sai cao nhất trong chương. Nó dễ kiểm: ghi nhiệt độ và tham số hiệu chuẩn trực tuyến trên một thiết bị thật trong vài giờ, và xem chúng có tương quan không. Nếu không, mệnh đề sai và tôi nên rút nó. Phép đo này nằm trong 99-chua-biet.md như một mục ưu tiên cao, vì nó rẻ và vì kết quả đổi cách chẩn đoán.

20.7 Kết nối

Nối lên trên. [A/04-uoc-luong-map-va-bundle-adjustment](#) là nguồn của cả chi phí lẫn cơ hội tối ưu lớn nhất — cấu trúc thừa. [B/09-paradigm-uoc-luong-back-end](#) mục 5A cung cấp điều kiện cho việc dùng độ chính xác đơn, và nói chung là chỗ lựa chọn kiến trúc quyết định hồ sơ chi phí. [C/13-quan-ly-ban-do](#) mục 4A nêu kiến trúc ba luồng mà chương này bàn chi tiết kỹ thuật.

Nối ngang. [D/19-ben-vung-trong-the-gioi-thuc](#) mục 6 đề nghị bốn tầng giám sát; chương này nhắc rằng chúng không miễn phí và phải nằm trong ngân sách. Ngược lại, mục 5 ở đây cho [D/19](#) một chế độ hồng thuộc nhóm bốn mà chương đó không nêu: trôi do nhiệt theo chu kỳ hoạt động chứ không theo tuổi thiết bị.

Nối xuống dưới. [D/21-tu-prototype-len-mass-production](#) là chỗ các quyết định ở đây trở thành yêu cầu phân cứng: chọn SoC theo ngân sách nhiệt chứ không chỉ theo hiệu năng đỉnh, yêu cầu truy cập ảnh gần raw từ nhà cung cấp module, và đặt cảm biến nhiệt gần camera. Cả ba phải quyết *trước khi chốt thiết kế*, và đó là lý do chương 21 khuyến nghị đọc chương này sớm hơn thứ tự tự nhiên của nó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao trung bình không đủ cho một hệ trong vòng điều khiển, và nêu ba nguồn thường gặp của các khung ở đuôi.
2. Nêu một thay đổi làm giảm thời gian trung bình mà tăng phân vị 99, và giải thích cơ chế.
3. Vì sao bỏ khung đúng hơn xếp hàng? Nêu ba chi tiết cài đặt đi kèm nguyên tắc đó.
4. Nêu ba việc phải làm để có tính tất định, và nói vì sao chúng phải làm từ đầu dự án.
5. Điều kiện để dùng độ chính xác đơn an toàn là gì, và chương nào cung cấp điều kiện ấy?
6. Nêu cảnh báo về ISP và hai yêu cầu nên đặt ra với nhà cung cấp module camera.
7. Hai cơ chế gây “chạy tốt rồi kém dần rồi hồi phục” là gì, và phép thử nào phân biệt chúng?

Từ prototype lên mass production (From Prototype to Mass Production)

PHỤ THUỘC C/14-hieu-chuan-nhu-mot-nhanh-cua-slam (hiệu chuẩn — ở đây nó trở thành một quy trình có chỉ tiêu thời gian và chất lượng); C/18-danh-gia-va-phuong-phap-luan (đánh giá — ở đây nó trở thành regression test và chỉ tiêu nghiệm thu); D/19-ben-vung-trong-the-gioi-thuc (chế độ hồng — ở đây thành fleet analytics); D/20-toi-uu-hieu-nang-va-he-thong (ràng buộc phần cứng — nên đọc trước chương này).

TÓM TẮT

Đây là chương duy nhất trong cây viết cho mục tiêu sản xuất hàng loạt, và nó là chương ít tài liệu nhất. Nội dung tổ chức theo bốn nhóm, tương ứng bốn nhóm quyết định mà một dự án phải đưa ra. *A — hiệu chuẩn ở quy mô sản xuất*: đồ gá tự động, chu trình dưới một phút, dung sai cơ khí, trôi theo thời gian, và một quyết định tốn nhiều tiền là hiệu chuẩn từng máy hay dùng tham số chung cho cả lô. *B — độ tin cậy*: định nghĩa chế độ hồng, giám sát sức khỏe tại hiện trường, watchdog, logging, và fleet analytics — vòng lặp biến hàng nghìn máy thành một công cụ nghiên cứu. *C — kiểm thử*: bộ dữ liệu nội bộ, regression tự động, hardware-in-the-loop, ma trận thử nghiệm, và nguyên tắc không bao giờ tin benchmark công khai là đủ. *D — ràng buộc sản phẩm*: chọn cấu hình cảm biến, chi phí BOM, nhiệt, điện, OTA, quyền riêng tư. Điều quan trọng nhất về chương này là *thứ tự*: phần lớn các quyết định ở nhóm A và D phải đưa ra *trước khi chốt thiết kế cơ khí*, khi chưa ai thấy chúng cần thiết — và không sửa được sau khi mở khuôn.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đồ gá hiệu chuẩn (calibration fixture)** — thiết bị cơ khí giữ sản phẩm và target ở các tư thế đã biết, để quy trình hiệu chuẩn chạy tự động và lặp lại được trên dây chuyền.
- **golden sample** — một hoặc vài đơn vị sản phẩm được đo rất kỹ và giữ làm chuẩn tham chiếu, dùng để phát hiện trôi của chính quy trình sản xuất và quy trình đo.
- **SPC — statistical process control** — theo dõi thống kê của một tham số qua các lô sản xuất, để phát hiện khi quy trình bắt đầu trôi *trước khi* sản phẩm bị lỗi.
- **phân tích độ nhạy** — đo xem sai lệch của một tham số đầu vào (dung sai cơ khí, sai số hiệu chuẩn) gây ra sai lệch bao nhiêu ở đầu ra, để biết dung sai nào cần siết và dung sai nào nới được.
- **fleet analytics** — thu thập và phân tích dữ liệu vận hành từ toàn bộ đội thiết bị đang hoạt động, để tìm ra các chế độ hồng mà phòng thí nghiệm không tái hiện được.
- **HIL — hardware-in-the-loop** — chạy phần mềm trên phần cứng thật với dữ liệu được phát lại, để kiểm thử mà không cần môi trường thật.
- **MTBF — mean time between failures** — thời gian trung bình giữa hai lần hỏng. Với SLAM, cần định nghĩa rõ “hỏng” nghĩa là gì trước khi con số này có nghĩa.

CÂU HỎI MỞ ĐẦU

- Q1.** Bạn có một nguyên mẫu chạy tốt và cần sản xuất một nghìn máy. Nếu quyết định quan trọng nhất phải đưa ra, và nói vì sao nó phải đưa ra *trước* khi chốt thiết kế cơ khí.

- Q2.** Hiệu chuẩn từng máy mất ba phút và tốn tiền; dùng tham số chung cho cả lô thì miễn phí. Nếu đại lượng quyết định lựa chọn ấy, và cách đo nó.
- Q3.** Hệ thống của bạn chạy hoàn hảo trên EuRoC và trên bộ dữ liệu nội bộ. Nếu ba lý do vì sao nó vẫn có thể thất bại ở nhà khách hàng đầu tiên.
- Q4.** Một nghìn máy đang chạy và bạn nhận được một trăm báo cáo lỗi mỗi tháng, phần lớn mơ hồ. Nếu thứ bạn cần thiết kế *từ trước* để biến các báo cáo ấy thành thông tin sửa được.
- Q5.** Bạn phải chọn cấu hình cảm biến cho một robot dịch vụ trong nhà. Trình bày cách biến câu hỏi này thành một bài toán có tiêu chí, và nêu kết luận cùng điều kiện của nó.

21.1 Đặt vấn đề

Ba phần trước và hai chương trước dựng một hệ thống chạy được. Chương này về việc biến nó thành một nghìn hệ thống chạy được ở một nghìn nơi khác nhau, không có bạn ở đó.

Bài toán gốc: khoảng cách giữa *một máy chạy được* và *mọi máy chạy được* lớn hơn nhiều so với trực giác, và nó lớn theo những hướng mà kỹ thuật thuật toán không chạm tới.

Ba nguồn của khoảng cách ấy.

Một — biến thiên giữa các đơn vị. Nguyên mẫu của bạn có một camera cụ thể lắp ở một vị trí cụ thể. Một nghìn máy có một nghìn camera hơi khác nhau, lắp ở các vị trí hơi khác nhau trong dung sai cơ khí. Tham số hiệu chuẩn của máy này không dùng cho máy kia — hoặc có dùng được, và câu hỏi “có hay không” là một quyết định tốn nhiều tiền (mục 2C).

Hai — biến thiên giữa các môi trường. Bạn thử trong văn phòng của mình. Khách hàng dùng nó trong một nhà kho có sàn bóng, một căn hộ có nhiều gương, một nhà xưởng có bụi. Mỗi môi trường kích hoạt một mục khác trong danh sách ở [D/19](#).

Ba — biến thiên theo thời gian. Nguyên mẫu chạy vài giờ. Sản phẩm chạy vài năm, trong đó ống kính bám bụi, keo lão hoá, môi trường thay đổi, và phần mềm được cập nhật.

Ai gặp bài toán này? Chỉ những người thật sự sản xuất, và đó là lý do tài liệu về nó gần như không tồn tại trong học thuật. Kiến thức thuộc về các công ty; nó không tổng quát hoá sạch sẽ thành định lý; và mỗi ràng buộc là riêng của một sản phẩm.

CẠM BẮY

Toàn bộ chương này là nguyên tắc và danh mục kiểm, không phải số liệu.

Tôi *không* có dữ liệu fleet thật, và mọi con số xuất hiện trong chương — “dưới sáu mươi giây”, “một nghìn máy”, các tỉ lệ phần trăm — là **ví dụ minh họa tự đặt** để lập luận có chỗ bám, không phải kết quả đo. Theo `learning-rules.md` mục 5, tôi phải nói rõ điều này một lần ở đầu chương thay vì rải cảnh báo khắp nơi.

Điều tôi đưa ra là *cấu trúc của các quyết định*: quyết định nào phải đưa ra trước quyết định nào, đại lượng nào quyết định mỗi lựa chọn, và cách đo đại lượng ấy. Cấu trúc ấy tôi tin là đúng và nó suy ra được từ các chương trước. Các con số thì phải do chính dự án của bạn đo.

Và một cảnh báo về cách đọc: chương này liệt kê rất nhiều việc phải làm. Không dự án nào làm hết ngay từ đầu. Cách dùng đúng là coi nó như một *danh mục để quyết định bỏ qua có ý thức* — biết rằng bạn đang bỏ qua điều gì và vì sao, thay vì phát hiện nó vào lúc đã quá muộn để sửa.

TRỰC GIÁC

Hình dung sự khác nhau giữa nấu một bữa ăn ngon và mở một nhà hàng.

Nấu một bữa ngon: bạn nêm, bạn chỉnh, bạn dùng đúng nguyên liệu mình thích. Kết quả phụ thuộc vào tay nghề của bạn ở thời điểm đó.

Mở nhà hàng thì khác về bản chất. Món ăn phải giống nhau ở mọi bàn, mọi ngày, do nhiều đầu bếp khác nhau nấu, với nguyên liệu mua từ nhiều nguồn khác nhau. Bạn không thể nêm từng đĩa.

Từ đó ra ba việc mà người nấu ăn ở nhà không bao giờ nghĩ tới, và chúng ánh xạ chính xác sang ba nhóm trong chương này.

Một — công thức phải đo được, không phải “nêm vừa ăn”. Bao nhiêu gram, bao nhiêu phút. Vì người nấu không phải bạn, và “vừa ăn” của họ khác của bạn. Đây là hiệu chuẩn và quy trình.

Hai — phải biết món nào bị hỏng mà không cần nêm. Nhiệt độ bếp, màu sắc, thời gian — các dấu hiệu gián tiếp thay cho việc nêm. Đây là giám sát sức khỏe.

Ba — phải có cách biết khách hàng nghĩ gì. Không phải mỗi lời phàn nàn lẻ tẻ, mà là: món nào bị trả lại nhiều nhất, vào giờ nào, ở bàn nào. Đây là fleet analytics, và nó là thứ biến một trăm lời phàn nàn mơ hồ thành một chẩn đoán.

Và có một việc thứ tư, quan trọng hơn cả ba và dễ bỏ quên nhất: *bạn phải quyết định kích thước bếp trước khi xây nhà hàng.* Sau khi đã xây, muốn thêm một cái lò nữa thì phải đập tường. Trong sản phẩm, những quyết định kiểu “đập tường” là dung sai cơ khí, chỗ đặt cảm biến, và loại camera — và chúng phải quyết vào lúc chưa ai thấy chúng quan trọng.

21.2 A — Hiệu chuẩn ở quy mô sản xuất

A. Phân tích độ nhạy phải chạy trước

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là quyết định quan trọng nhất trong chương.

Câu hỏi: *dung sai cơ khí bao nhiêu là chấp nhận được?* Cụ thể: hai camera stereo lệch góc bao nhiêu độ thì vẫn đạt yêu cầu độ chính xác? Module camera xoay bao nhiêu so với thân máy?

Câu hỏi ấy *phải* được trả lời trước khi chốt thiết kế cơ khí, vì siết dung sai sau khi mở khuôn là bất khả về mặt kinh tế.

Quy trình trả lời, và nó là một quy trình chạy được chứ không phải một lời khuyên:

Bước một — xác định ngân sách sai số đầu ra. Từ yêu cầu sản phẩm: robot phải định vị chính xác bao nhiêu để làm được việc của nó.

Bước hai — lập danh sách các tham số đầu vào và độ tán dự kiến của chúng. Với mỗi tham số, một khoảng: tiêu cự lệch bao nhiêu giữa các đơn vị, góc lắp lệch bao nhiêu, đường cơ sở lệch bao nhiêu.

Bước ba — chạy Monte-Carlo. Nhiễu loạn từng tham số theo độ tán của nó trong mô phỏng, đo sai số đầu ra. Đây là C/18 mục 3B áp dụng cho một mục đích khác: mô phỏng để tách các nguồn.

Bước bốn — đọc kết quả theo hai chiều. Tham số nào mà sai lệch nhỏ gây sai số lớn thì *siết dung sai* hoặc *hiệu chuẩn từng máy*. Tham số nào mà sai lệch lớn cũng không sao thì *nới dung sai* để tiết kiệm chi phí cơ khí.

Chiều thứ hai là chiều hay bị bỏ quên và nó tiết kiệm tiền thật: phần lớn dự án siết mọi dung sai vì không biết cái nào quan trọng, và điều đó đắt một cách không cần thiết.

B. Quy trình hiệu chuẩn trên dây chuyền

Bốn ràng buộc mà quy trình phòng lab không có.

Thời gian. Mỗi giây trên dây chuyền là tiền. Một quy trình dùng Kalibr với người thao tác cầm bảng đi lại mất vài phút và không lặp lại được — không dùng được.

Tính lặp lại. Kết quả không được phụ thuộc vào người thao tác. Điều này đòi hỏi một *đồ gá*: một cơ cấu giữ sản phẩm và di chuyển target (hoặc ngược lại) qua một chuỗi tư thế cố định.

Chuỗi tư thế phải thoả điều kiện kích thích. Đây là chỗ [A/05](#) Bảng kích thích và [C/14](#) trả công trực tiếp. Chuỗi tư thế của đồ gá phải được *thiết kế* để mỗi tham số cần ước lượng đều được kích thích: đa dạng độ sâu cho tiêu cự, target ở bốn góc ảnh cho điểm chính và méo, xoay quanh nhiều trục nếu có hiệu chuẩn camera-IMU.

Điều đáng nhấn mạnh: chuỗi ấy nên được *kiểm bằng phân tích observability* trước khi chế tạo đồ gá — dựng bài toán MAP với chuỗi tư thế dự kiến và xem hiệp phương sai của các tham số. Nếu một tham số có hiệp phương sai lớn thì chuỗi chưa đủ, và phát hiện điều đó trên máy tính rẻ hơn nhiều so với phát hiện sau khi đã chế tạo đồ gá.

Tiêu chí đạt/không đạt tự động. Không có kỹ sư nhìn từng kết quả. Cần một bộ tiêu chí chạy tự động, và theo [A/02](#) mục 4 thì RMS *không* phải tiêu chí đúng. Bốn tiêu chí đúng: độ phủ của target trên khung hình (kiểm được *trước* khi tối ưu — rẻ nhất, nên chạy đầu tiên); hiệp phương sai của từng tham số; hoa văn phần dư; và so với phân bố của lô (mục 2D).

C. Hiệu chuẩn từng máy hay dùng chung

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một quyết định có ảnh hưởng lớn tới chi phí.

Đại lượng quyết định là một *tỉ số*:

$$\frac{\text{độ tán của tham số trong lô}}{\text{mức tham số đó được phép sai theo ngân sách}}.$$

Từ số đo bằng cách hiệu chuẩn một mẫu vài chục máy và tính độ lệch chuẩn. Mẫu số đến từ phân tích độ nhảy ở mục 2A.

Nếu tỉ số nhỏ hơn một, tham số chung dùng được và bạn tiết kiệm được thời gian dây chuyền. Nếu lớn hơn một, phải hiệu chuẩn từng máy — hoặc siết dung sai cơ khí, vốn có thể rẻ hơn tùy trường hợp.

Và quyết định không nhất thiết đồng nhất cho mọi tham số. Thường gặp: *tiêu cự và méo* khá đồng đều trong một lô module cùng nguồn nên dùng chung được; *ngoại* — vị trí tương đối giữa các camera — phụ thuộc vào lắp ráp nên độ tán lớn hơn nhiều và thường phải đo từng máy. Một quy trình hỗn hợp — dùng chung nội, đo riêng ngoại — thường là điểm tối ưu, và nó nhanh hơn nhiều so với đo tất cả.

D. Theo dõi qua thời gian: golden sample và SPC

Hai cơ chế cần thiết và hay bị bỏ.

Golden sample. Giữ vài đơn vị đã được đo rất kỹ bằng một phương pháp độc lập và chính xác hơn. Chạy chúng qua dây chuyền định kỳ. Nếu kết quả trôi, thì *quy trình đo* đang trôi — không phải sản phẩm. Không có cơ chế này thì bạn không phân biệt được “lô này kém” với “máy đo bị lệch”.

SPC trên tham số hiệu chuẩn. Vẽ phân bố của từng tham số qua các lô. Một dịch chuyển của trung bình hoặc một giãn nở của độ tán là dấu hiệu quy trình sản xuất đang trôi — *trước khi* sản phẩm bị lỗi. Đây là công cụ chuẩn của sản xuất công nghiệp, và nó áp dụng trực tiếp cho tham số hiệu chuẩn mà ít ai làm.

Trôi sau khi xuất xưởng là chuyện của [C/14](#) mục 4B và [D/20](#) mục 5: nhiệt độ, rung, va đập, lão hoá keo. Hai cơ chế xử lý: hiệu chuẩn trực tuyến có cổng observability, và phát hiện để yêu cầu hiệu chuẩn lại.

21.3 B – Độ tin cậy và fleet analytics

A. Định nghĩa “hỏng” trước khi đo MTBF

MTBF là một con số vô nghĩa cho tới khi “hỏng” được định nghĩa. Với SLAM, cần một thang chứ không phải một ngưỡng, và nó nên khớp với thang chế độ suy giảm ở [D/19](#) mục 6C.

Một thang hợp lý, từ nhẹ tới nặng: suy giảm có phát hiện (hệ chuyển chế độ và báo — không tính là hỏng); mất bám tạm thời có tự phục hồi; mất bám cần khởi động lại phần mềm; sai vị trí im lặng (ngghiêm trọng nhất, vì hậu quả không giới hạn); và bản đồ bị phá (mất dữ liệu).

Mỗi mức có một tỉ lệ chấp nhận khác nhau, và *sai vị trí im lặng* nên có tỉ lệ chấp nhận thấp hơn nhiều bậc so với các mức khác — đúng lập luận ở [D/19](#) mục 6A.

B. Giám sát sức khoẻ tại hiện trường

Các đại lượng đã được liệt kê rải rác: [C/18](#) mục 3C, [D/19](#) khối *Cần nhớ*, [D/20](#) mục 2. Ở đây là ba nguyên tắc để biến chúng thành một hệ thống.

Một — đo nguyên nhân, không chỉ hậu quả (D/19). Mật độ đặc trưng theo vùng ảnh nói nhiều hơn ATE.

Hai — hiệu chuẩn quan hệ giữa đại lượng thay thế và sai số thật, một lần, trong lab. Các đại lượng giám sát không đo sai số; chúng đo các điều kiện mà sai số nhỏ phụ thuộc vào. Quan hệ giữa chúng phải được thiết lập trên dữ liệu có chân trị rồi mang ra dùng. Bước này bị bỏ qua gần như mọi nơi, và không có nó thì các ngưỡng cảnh báo là số đoán.

Ba — ngân sách phải nằm trong ngân sách. Theo [D/20](#), mọi thứ chạy mỗi khung đều tốn. Phần lớn đại lượng giám sát rẻ (vài chục con số mỗi khung, tổng hợp thành thống kê mỗi giây), nhưng vài cái không — tính phổ giá trị riêng của Λ chẳng hạn. Chạy cái đắt theo chu kỳ thưa hơn.

C. Logging: đủ để tái hiện, không nhiều hơn

Đây là một phần của câu trả lời cho câu hỏi mở đầu thứ tư.

Yêu cầu mâu thuẫn: log phải đủ để tái hiện một lỗi, nhưng không được tốn băng thông, bộ nhớ, hay quyền riêng tư.

Cách giải quyết là *ba tầng*.

Tầng một — luôn bật, rất rẻ. Các đại lượng giám sát tổng hợp theo giây, kèm nhiệt độ và trạng thái chế độ. Vài trăm byte mỗi giây. Đây là thứ cho phép fleet analytics.

Tầng hai — vòng đệm quay vòng, giữ vài chục giây gần nhất. Đủ chi tiết để tái hiện: mốc thời gian, quyết định keyframe, kết quả các cổng. Chỉ được *ghi ra* khi có sự kiện đáng chú ý — một lần mất bám, một cảnh báo. Đây là cơ chế quan trọng nhất, vì nó cho bạn dữ liệu về *chính khoảnh khắc* trước khi hỏng.

Tầng ba — ghi ảnh, chỉ khi được phép. Đắt và nhạy cảm về quyền riêng tư. Chỉ bật theo yêu cầu và có sự đồng ý.

Và theo [D/20](#) mục 3B: log tầng hai chỉ có giá trị nếu hệ thống *tất định*. Nếu không, bạn có dữ liệu mà không tái hiện được.

D. Fleet analytics: vòng lặp quan trọng nhất

Đây là phần còn lại của câu trả lời cho câu hỏi mở đầu thứ tư, và tôi cho là đóng góp lớn nhất của chương.

Một nghìn máy chạy tám tiếng mỗi ngày là tám nghìn giờ dữ liệu mỗi ngày — nhiều hơn bất kỳ bộ dữ liệu nghiên cứu nào nhiều bậc. Nếu bạn thu được các đại lượng tổng hợp từ chúng, bạn có một công cụ mà không phòng thí nghiệm nào có.

Vòng lặp, bốn bước.

Một — thu thập các đại lượng tầng một từ toàn đội, cộng các bản ghi tầng hai khi có sự kiện.

Hai — phân cụm các sự kiện theo dấu hiệu, không theo mô tả của khách hàng. Các lần mất bắm có cùng hồ sơ đại lượng giám sát — cùng kiểu sụt đặc trưng, cùng kiểu tăng phần dư — nhiều khả năng có cùng nguyên nhân, dù khách hàng mô tả chúng bằng những từ khác nhau.

Ba — xếp hạng các cụm theo tần suất nhân mức nghiêm trọng. Đây là danh sách việc cần làm, và nó khác rất nhiều so với danh sách mà kỹ sư tự nghĩ ra.

Bốn — tái hiện cụm lớn nhất trong lab bằng dữ liệu tầng hai, sửa, và đo lại trên fleet ở phiên bản sau.

Điều quan trọng nhất về vòng lặp này: nó biến “một trăm báo cáo mơ hồ” thành “ba nguyên nhân xếp theo mức quan trọng”. Và nó chỉ khởi động được nếu ba thứ đã được thiết kế *từ trước*: các đại lượng giám sát, cơ chế log ba tầng, và tính tất định. Không thứ nào thêm vào được sau một cách rẻ.

Đây cũng là chỗ [D/19](#) mục 6A trả công: một hệ biết mình sai tạo ra dữ liệu; một hệ im lặng thì không. Toàn bộ giá trị của fleet analytics phụ thuộc vào việc hệ thống phát hiện được thất bại của chính nó.

I 21.4 C — Kiểm thử

A. Bộ dữ liệu nội bộ

Đây là câu trả lời cho câu hỏi mở đầu thứ ba.

Ba lý do một hệ chạy hoàn hảo trên benchmark vẫn thất bại ở nhà khách hàng đầu tiên.

Một — benchmark không chứa các chế độ hỏng ở [D/19](#). Chúng được thu trong điều kiện kiểm soát, không có gương, ít người đi lại, không có bụi ống kính.

Hai — benchmark quá ngắn. Không bộ dữ liệu chuẩn nào dài tám tiếng, nên các vấn đề ở [C/13](#) mục 4 — kích thước bản đồ, lão hoá — không xuất hiện.

Ba — benchmark đo sai thứ. Chúng đo ATE trung bình trên các chuỗi hệ thống chạy được; sản phẩm cần tỉ lệ thất bại và hành vi ở đuôi ([C/18](#) mục 5D).

Kết luận, và nó là một nguyên tắc cứng: *không bao giờ tin benchmark công khai là đủ.* Cần một bộ dữ liệu nội bộ phản ánh môi trường khách hàng, và nó nên gồm ba phần: các chuỗi *đại diện* (môi trường điển hình, dài), các chuỗi *ngịch cảnh* (bộ ở [D/19](#) mini project), và các chuỗi *từ hiện trường* (dữ liệu tầng hai từ các lần hỏng thật — đây là phần có giá trị nhất và nó chỉ có được nhờ vòng lặp ở mục 3D).

B. Regression test tự động

Mỗi commit chạy toàn bộ bộ dữ liệu nội bộ và so với đường cơ sở. Ba chi tiết quyết định nó có dùng được hay không.

Một — so với phân vị, không chỉ trung bình (C/18, D/20). Một thay đổi làm giảm ATE trung bình mà tăng tỉ lệ thất bại là một hồi quy.

Hai — chạy nhiều seed nếu hệ không tắt định; nếu tắt định thì một lần là đủ, và đó là một lý do nữa để đầu tư vào tính tắt định.

Ba — nhiều chỉ tiêu, không một. ATE, tỉ lệ thất bại, tỉ lệ phát hiện thất bại, phân vị thời gian, bộ nhớ. Một bảng chứ không một con số.

C. HIL và ma trận thử nghiệm

HIL: chạy phần mềm trên phần cứng thật với dữ liệu phát lại. Nó bắt được các vấn đề mà chạy trên máy tính bỏ sót: thermal throttling (D/20 mục 5), giới hạn bộ nhớ, hành vi của ISP. Điều kiện để có giá trị: phát lại phải đúng thời gian thực, vì phần lớn vấn đề ở đây là vấn đề thời gian.

Ma trận thử nghiệm: liệt kê các chiều biến thiên — ánh sáng, loại sàn, tốc độ, tải, mức độ động của môi trường — và thử các tổ hợp. Không thể thử hết, nên chọn theo hai tiêu chí: các tổ hợp có khả năng xảy ra ở khách hàng, và các tổ hợp mà lý thuyết nói là khó (từ A/05 và D/19).

Điều đáng nhớ: ma trận nên được xây từ hai nguồn — kinh nghiệm về khách hàng, và lý thuyết về chỗ hệ thống suy biến. Chỉ dùng nguồn thứ nhất thì bỏ sót các trường hợp chưa gặp; chỉ dùng nguồn thứ hai thì thử nhiều thứ không bao giờ xảy ra.

I 21.5 D — Ràng buộc sản phẩm

A. Chọn cấu hình cảm biến

Đây là câu trả lời cho câu hỏi mở đầu thứ năm.

Cách biến nó thành bài toán có tiêu chí, ba bước, dùng lại các công cụ đã có.

Bước một — liệt kê gauge cần phá. Từ A/05: mono thuần thị giác có bảy bậc không quan sát được. Ứng dụng cần bao nhiêu trong số đó? Robot dịch vụ cần tỉ lệ metric (để đo khoảng cách thật) và cần hướng ổn định; nó không cần vị trí tuyệt đối toàn cục.

Bước hai — với mỗi ứng viên cảm biến, xem nó phá gauge nào và đòi điều kiện gì (C/15 Bảng cảm biến).

Bước ba — kiểm tính độc lập của chế độ hỏng (D/19). Hai cảm biến cùng chết trong bóng tối không cho khả năng phục hồi.

Áp dụng cho robot dịch vụ trong nhà, kết luận của tôi: stereo + IMU + odometry bánh xe, với global shutter.

Lập luận: stereo phá gauge tỉ lệ ngay từ khung đầu, làm khởi tạo trở nên tầm thường và xoá cả một lớp chế độ hỏng (B/10 mục 3A); IMU ghim roll với pitch, cho chuyển động tần số cao, và quan trọng nhất là nó sống khi thị giác chết; odometry bánh xe cho tỉ lệ độc lập và ràng buộc nonholonomic miễn phí, và nó sống khi cả thị giác lẫn IMU đều kém (C/15 mục 4A). Ba cảm biến có ba chế độ hỏng gần như độc lập: thị giác chết trong tối, IMU trôi khi đứng yên lâu, odometry nói dối khi trượt — và chúng kiểm chéo lẫn nhau.

Global shutter: theo A/02 mục 5A, nó xoá bỏ một lớp mô hình, một lớp tham số phải hiệu chuẩn, và một lớp chế độ hỏng chỉ xuất hiện khi rung. Chênh lệch giá vài độ la mỗi máy đổi lấy điều đó là một trong số ít quyết định trong cây mà phần cứng thắng phần mềm rõ ràng.

Điều kiện của kết luận này: nó dành cho robot mặt đất, trong nhà, hoạt động không giám sát, với ngân sách cho phép ba cảm biến. Nó không áp dụng cho drone (không có bánh xe), cho thiết bị cầm tay (không có odometry), hay cho sản phẩm cực rẻ (ngân sách chỉ đủ một camera). (Đây là

suy luận của tôi từ các chương trước, không phải kết quả của một khảo sát thị trường hay một thử nghiệm so sánh.)

B. OTA, phiên bản, và quyền riêng tư

Ba ràng buộc còn lại, mỗi cái một đoạn.

Cập nhật OTA cho phần mềm, bản đồ, và mô hình. Ba yêu cầu: định dạng bản đồ phải có phiên bản (C/13 mục 5A) — thêm sau rất đau; phải có cơ chế quay lui khi bản cập nhật gây hồi quy; và với mô hình học được, mỗi lần đổi mô hình đòi *tái kiểm định toàn bộ* (C/17), nên chi phí cập nhật cao hơn nhiều so với đổi một ngưỡng.

Quyền riêng tư. Theo C/13 mục 5B, bản đồ gồm điểm và descriptor là dữ liệu riêng tư — tái tạo được ảnh từ nó [80]. Ba hệ quả: tải bản đồ lên đám mây là truyền dữ liệu cá nhân; lưu trên thiết bị thì nó là mục tiêu; và log tầng ba (ảnh) phải có sự đồng ý rõ ràng. Điều tối thiểu: coi bản đồ là dữ liệu cá nhân trong mọi quyết định thiết kế, và nói rõ điều đó với khách hàng.

Nhiệt và điện. Theo D/20 mục 5: chọn SoC theo ngân sách nhiệt liên tục chứ không theo hiệu năng đỉnh, và *đặt một cảm biến nhiệt gần module camera*. Chi phí vài xu, giá trị chẩn đoán cao, và không gắn thêm được sau khi mở khuôn.

CẦN NHỚ

Thứ tự quan trọng hơn nội dung. Chương này liệt kê nhiều việc; điều quyết định là việc nào phải làm *trước*.

Bốn quyết định **không sửa được sau khi chốt cơ khí**, xếp theo mức tổn kém nếu sai:

Một — dung sai cơ khí, quyết bởi phân tích độ nhạy ở mục 2A. Sai thì hoặc phải mở khuôn lại, hoặc phải hiệu chuẩn từng máy suốt đời sản phẩm.

Hai — loại cảm biến và loại shutter. Global shutter và cấu hình cảm biến quyết định cả một lớp chế độ hồng có tồn tại hay không.

Ba — các cảm biến phụ trợ rẻ tiền: cảm biến nhiệt gần camera, encoder bánh xe. Vài xu mỗi cái, và chúng là thứ phân biệt được các nguyên nhân suy giảm.

Bốn — truy cập ảnh gần raw và điều khiển ISP, vốn là một yêu cầu với nhà cung cấp module phải nêu *trước khi chọn module*.

Và ba quyết định **phần mềm** nên đưa ra từ đầu vì thêm sau rất đắt: *tính tất định* (D/20 mục 3B); *định dạng bản đồ có phiên bản*; và *cơ chế giám sát cùng log ba tầng*, vì không có chúng thì vòng lặp fleet analytics ở mục 3D không khởi động được.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Kalibr [21]	Hiệu chuẩn đa cảm biến; quy trình thu dữ liệu	Điểm khởi đầu cho mục 2B — nhưng quy trình dây chuyền phải tự thiết kế, vì Kalibr không nhắm tới dây chuyền
maplab [73]	Nhiều phiên, quản lý bản đồ dài hạn	Hiếm framework nghĩ về hoạt động nhiều phiên; liên quan mục 5B
OpenLORIS [79]	Bộ dữ liệu nhiều phiên trong môi trường thật có thay đổi	Gần nhất với tinh thần mục 4A trong các bộ dữ liệu công khai
evo [102], rpg_trajectory_evaluation [39]	Công cụ đánh giá quỹ đạo	Nền cho regression test ở mục 4B; nhớ chọn đúng chế độ căn chỉnh

MINI PROJECT SLAM · Chạy phân tích độ nhạy cho sản phẩm của bạn, trước khi chốt cơ khí · 4 ngày

Mục tiêu	biến câu hỏi “dung sai bao nhiêu là đủ” từ một cuộc tranh luận thành một bảng số — và làm điều đó vào lúc còn sửa được.
Dữ liệu	không cần phần cứng: toàn bộ bài này chạy trong mô phỏng, và đó chính là điểm mạnh của nó.
Các bước	viết ra ngân sách sai số đầu ra từ yêu cầu sản phẩm; lập danh sách mười tham số đầu vào với độ tán dự kiến (nếu chưa biết, dùng dung sai mà nhà cung cấp cơ khí đề xuất làm điểm khởi đầu); chạy Monte-Carlo nhiều lần từng tham số riêng lẻ rồi nhiều lần tất cả cùng lúc; xếp hạng theo độ nhạy; với ba tham số nhạy nhất, tính tỉ số ở mục 2C để quyết định siết dung sai, hiệu chuẩn từng máy, hay cả hai; cuối cùng thiết kế chuỗi tư thế cho đồ gá và kiểm nó bằng phân tích observability theo mục 2B trước khi vẽ cơ khí.
Xong khi	bạn có một bảng mười tham số với ba cột: độ nhạy, độ tán dự kiến, và quyết định (nói / siết / hiệu chuẩn từng máy). Và bạn có một chuỗi tư thế đồ gá đã được kiểm rằng nó kích thích đủ mọi tham số cần ước lượng. Hai sản phẩm ấy là hai thứ đắt nhất để làm lại nếu bỏ qua, và chúng tốn bốn ngày trên máy tính.
Nối với	C/14-hieu-chuan-nhu-mot-nhanh-cua-slam cho phần lý thuyết của việc kiểm chuỗi tư thế; C/18-danh-gia-va-phuong-phap-luan cho phương pháp Monte-Carlo có kỷ luật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Quyết định quan trọng nhất là *dung sai cơ khí*, và nó được trả lời bằng phân tích độ nhạy: nhiều loạn từng tham số trong mô phỏng theo độ tán dự kiến, đo sai số đầu ra, rồi siết dung sai cho các tham số nhạy và nói cho các tham số không nhạy. Nó phải đưa ra trước khi chốt cơ khí vì siết dung sai sau khi mở khuôn là bất khả về mặt kinh tế — lựa chọn còn lại là hiệu chuẩn từng máy suốt đời sản phẩm, vốn đắt hơn nhiều. Chiều *nói* cũng quan trọng và hay bị bỏ: phần lớn dự án siết mọi dung sai vì không biết cái nào quan trọng, và điều đó đắt một cách không cần thiết.
- A2. Đại lượng quyết định là tỉ số (*độ tán của tham số trong lô*) / (*mức tham số được phép sai theo ngân sách*). Từ số đo bằng cách hiệu chuẩn một mẫu vài chục máy và tính độ lệch chuẩn; mẫu số đến từ phân tích độ nhạy. Tỉ số nhỏ hơn một thì dùng chung được. Quyết định không cần đồng nhất cho mọi tham số: thường thì tiêu cự và méo khá đồng đều trong một lô module cùng nguồn nên dùng chung được, còn *ngoại* phụ thuộc lắp ráp nên độ tán lớn hơn và phải đo từng máy. Quy trình hỗn hợp — dùng chung nội, đo riêng ngoại — thường là điểm tối ưu và nhanh hơn nhiều so với đo tất cả.
- A3. (1) Benchmark *không chứa các chế độ hồng ở D/19* — không có gương, ít người đi lại, không có bụi ống kính, vì chúng được thu trong điều kiện kiểm soát. (2) Benchmark *quá ngắn* — không bộ nào dài tám tiếng, nên các vấn đề về kích thước bản đồ và lão hoá ở [C/13](#) không xuất hiện. (3) Benchmark *đo sai thứ* — chúng đo ATE trung bình trên các chuỗi hệ thống chạy được, trong khi sản phẩm cần tỉ lệ thất bại và hành vi ở đuôi. Kết luận: không bao giờ tin benchmark công khai là đủ; cần bộ dữ liệu nội bộ gồm chuỗi đại diện, chuỗi nghịch cảnh, và — có giá trị nhất — chuỗi từ hiện trường.
- A4. Ba thứ phải thiết kế từ trước: (1) *các đại lượng giám sát* đo nguyên nhân chứ không chỉ hậu quả; (2) *cơ chế log ba tầng* — tầng một luôn bật và rẻ, tầng hai là vòng đệm quay vòng chỉ ghi ra khi có sự kiện (đây là tầng quan trọng nhất vì nó cho dữ liệu về chính khoảnh khắc trước khi hồng), tầng ba là ảnh và chỉ khi được phép; (3) *tính tất định*, vì không có nó thì log tầng hai vô dụng. Có ba thứ đó thì vòng lặp fleet analytics chạy được: phân cụm sự kiện *theo dấu hiệu* chứ không theo mô tả của khách hàng, xếp hạng cụm theo tần suất nhân mức nghiêm trọng, rồi tái hiện cụm lớn nhất trong lab. Nó biến một trăm báo cáo mơ hồ thành ba nguyên nhân xếp hạng.
- A5. Ba bước: liệt kê gauge cần phá ([A/05](#)); với mỗi ứng viên xem nó phá gauge nào và đòi điều kiện gì ([C/15](#)); kiểm tính *độc lập của chế độ hồng ở D/19*. Với robot dịch vụ trong nhà, kết luận của tôi là *stereo + IMU + odometry bánh xe, với global shutter*: stereo phá gauge tỉ lệ ngay từ khung đầu và xóa cả một lớp chế độ hồng khởi tạo; IMU ghim roll-pitch và sống khi thị giác chết; odometry cho tỉ lệ độc lập, ràng buộc nonholonomic miễn phí, và sống khi cả hai kia kém. Ba chế độ hồng gần như độc lập nên chúng kiểm chéo lẫn nhau. Điều kiện: kết luận này dành cho robot mặt

đất trong nhà, hoạt động không giám sát, có ngân sách cho ba cảm biến — nó không áp dụng cho drone, thiết bị cầm tay, hay sản phẩm cực rẻ.

NÓI LẠI THẬT ĐƠN GIẢN

Nấu một bữa ăn ngon và mở một nhà hàng là hai việc khác nhau về bản chất, không chỉ về quy mô. Nấu ở nhà: bạn nêm, bạn chỉnh, bạn dùng nguyên liệu mình thích. Kết quả phụ thuộc vào tay nghề của bạn hôm đó.

Mở nhà hàng: món phải giống nhau ở mọi bàn, mọi ngày, do nhiều người nấu, với nguyên liệu từ nhiều nguồn. Bạn không thể nêm từng đĩa.

Từ đó ra bốn việc mà người nấu ăn ở nhà không bao giờ nghĩ tới.

Công thức phải đo được, không phải “nêm vừa ăn” — vì người nấu không phải bạn và “vừa ăn” của họ khác của bạn.

Phải biết món nào hồng mà không cần nêm — nhìn màu, đo nhiệt độ bếp, canh thời gian. Các dấu hiệu gián tiếp thay cho việc nêm.

Phải biết khách nghĩ gì một cách có hệ thống — không phải từng lời phàn nàn lẻ tẻ mà là: món nào bị trả lại nhiều nhất, vào giờ nào, ở bàn nào. Một trăm lời phàn nàn mơ hồ gộp lại có thể chỉ ra đúng một cái bếp bị lệch nhiệt.

Và việc thứ tư, quan trọng nhất và dễ quên nhất: *phải quyết kích thước bếp trước khi xây nhà hàng*. Sau khi xây xong, muốn thêm một cái lò thì phải đập tường.

Trong một sản phẩm có camera, những quyết định kiểu “đập tường” là: hai camera cách nhau bao xa và được phép lệch bao nhiêu độ; dùng loại cảm biến ảnh nào; có gắn một cái đo nhiệt độ bên cạnh ống kính hay không. Ba thứ ấy phải quyết vào lúc chưa ai thấy chúng quan trọng — và đó chính là lý do chúng hay bị bỏ qua.

Cách duy nhất biết trước là *thử trên máy tính*: giả sử camera lệch nửa độ thì kết quả sai bao nhiêu? Nếu sai nhiều thì phải làm cơ khí chính xác hơn, hoặc phải đo riêng từng máy. Nếu sai ít thì có thể làm lỏng tay và tiết kiệm. Bốn ngày ngồi máy tính đổi lấy việc không phải mở lại khuôn nhựa — đó là phép tính dễ nhất trong cả chương.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *phải quyết định khi chưa có dữ liệu*. Bạn cần biết độ tán của tham số trong lô để quyết dung sai, nhưng bạn chỉ đo được độ tán sau khi đã sản xuất một lô — tức sau khi đã chốt dung sai. Cách duy nhất phá vòng là làm một lô mẫu nhỏ trước, và đó là một khoản chi phải thuyết phục được người trả tiền vào lúc chưa ai thấy nó cần.

Một câu: *Khoảng cách từ một máy chạy được tới nghìn máy chạy được nằm ở những quyết định phải đưa ra trước khi ai thấy chúng quan trọng.*

21.6 Điều gì chứng minh cách hiểu này sai

Toàn bộ chương này là *cấu trúc của các quyết định* chứ không phải các sự kiện đo được, nên nó bị bác bỏ theo một cách khác với các chương trước: không phải bằng một phép đo mà bằng một kinh nghiệm triển khai mâu thuẫn.

Mệnh đề chính — *phân tích độ nhạy phải chạy trước khi chốt cơ khí* — bị bác bỏ nếu trong thực tế, việc hiệu chuẩn từng máy rẻ tới mức không đáng siết dung sai cho bất kỳ tham số nào. Điều đó *có thể* đúng với sản phẩm sản lượng thấp giá cao, nơi ba phút dây chuyền không đáng kể. Nên phát biểu chặt hơn: mệnh đề đúng khi thời gian dây chuyền là ràng buộc, và mất hiệu lực khi không.

Mệnh đề thứ hai — *stereo + IMU + odometry là điểm ngọt cho robot mặt đất* — là suy luận của tôi từ ba chương trước, *không phải kết quả của một khảo sát hay một thử nghiệm so sánh*. Nó bị bác bỏ nếu một sản phẩm thành công trên thị trường dùng cấu hình rẻ hơn — một camera mono cộng IMU chẳng hạn — với tỉ lệ hỏng chấp nhận được. Tôi ngờ rằng điều này *có* xảy ra ở phân khúc giá rẻ, và nếu vậy thì kết luận đúng là “điểm ngọt phụ thuộc vào tỉ lệ hỏng mà thị trường chấp nhận”, không phải “ba cảm biến là câu trả lời”. Đây là mệnh đề tôi ít chắc nhất trong chương.

Mệnh đề thứ ba — *fleet analytics là vòng lặp quan trọng nhất* — bị bác bỏ nếu các chế độ hồng thật đủ đơn giản để tái hiện trong lab mà không cần dữ liệu hiện trường. Với một sản phẩm hoạt động trong môi trường kiểm soát — một nhà máy có điều kiện ổn định — điều đó có thể đúng. Với sản phẩm tiêu dùng thì tôi cho là không.

Và một cảnh báo bao trùm: chương này được viết *không có kinh nghiệm vận hành fleet thật*. Nó suy ra từ nguyên lý và từ các chương trước. Ai đã vận hành một đội thiết bị thật sẽ biết những thứ không có ở đây, và nếu bạn trở thành người đó thì hãy sửa chương này bằng kinh nghiệm của mình.

21.7 Kết nối

Chương này là chỗ hội tụ của gần như mọi chương trong cây, và cách đọc đúng là đọc nó như một bản đồ ngược.

Nối lên trên theo từng mục. Mục 2 (hiệu chuẩn) lấy từ [A/02](#) (mô hình và chẩn đoán), [A/05](#) (điều kiện kích thích), [A/06](#) (hiệp phương sai tham số), và [C/14](#) (khung hợp nhất) — và biến chúng thành một quy trình có chỉ tiêu. Mục 3 (độ tin cậy) lấy từ [D/19](#) (chế độ hồng và phát hiện) và [D/20](#) (logging và tính tất định). Mục 4 (kiểm thử) lấy từ [C/18](#) (phương pháp đánh giá) và [D/19](#) (bộ nghịch cảnh). Mục 5 (ràng buộc sản phẩm) lấy từ [A/05](#) và [C/15](#) (chọn cảm biến theo gauge), [A/02](#) (global shutter), [C/13](#) (quyền riêng tư và định dạng bản đồ), [D/20](#) (nhiệt).

Nối xuống dưới: [D/22-huong-nghien-cuu-con-trong](#) là chương cuối, và ba trong tám hướng ở đó — hiệu chuẩn có ý thức observability, dự đoán thất bại, hệ tự chữa — đều mọc ra từ các vấn đề nêu trong chương này. Đó là một ví dụ của điều mà `research-rules.md` mục 7 nói: câu hỏi nghiên cứu tốt mọc ra từ một vấn đề thật, không từ cảm hứng ngẫu nhiên.

Nối ngang: `../marker/` là cây bàn về một giải pháp thực dụng cho bài toán định vị tuyệt đối trong môi trường có thể sửa đổi, và nó có một phần riêng về ngân sách sai số và nghiệm thu — rất gần tinh thần chương này, ở một quy mô nhỏ hơn và cụ thể hơn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu bốn quyết định không sửa được sau khi chốt cơ khí, và nói mỗi cái được quyết bởi phân tích nào.
2. Trình bày quy trình phân tích độ nhảy bốn bước, và nói vì sao chiều “nói dung sai” cũng quan trọng.
3. Nêu tỉ số quyết định việc hiệu chuẩn từng máy hay dùng chung, và nói vì sao quyết định có thể khác nhau giữa tham số nội và tham số ngoại.
4. Vì sao chuỗi tư thế của đồ gá nên được kiểm bằng phân tích observability trước khi chế tạo?
5. Nêu ba tầng logging và nói tầng nào quan trọng nhất cho việc chẩn đoán, cùng điều kiện để nó có giá trị.
6. Mô tả vòng lặp fleet analytics bốn bước, và nói ba thứ phải thiết kế từ trước để nó khởi động được.
7. Trình bày ba bước biến việc chọn cảm biến thành một bài toán có tiêu chí, và nêu kết luận cùng *điều kiện* của nó.

Tám hướng còn khoảng trống (Eight Open Directions)

PHỤ THUỘC Toàn bộ cây. Mỗi hướng dưới đây mọc ra từ một vấn đề cụ thể đã nêu ở một chương trước, và phần “từ đâu ra” của mỗi mục trở về chương đó.

TÓM TẮT

Chương này là chương duy nhất trong cây chạm vào địa hạt nghiên cứu, và nó mở đầu bằng một cảnh báo về chính nó: *nó chưa đạt chuẩn của research-rules.md*. Nó không dựa trên một khảo sát có hệ thống — không có nhật ký tìm kiếm, không có ma trận tổng hợp, không đạt chỉ tiêu số công trình cho bất kỳ hướng nào. Nó là danh sách các chỗ mà *tôi cho là còn trống*, dựa trên việc đọc rải rác và trên lập luận cấu trúc từ hai mươi mốt chương trước. Vì vậy mỗi hướng được trình bày theo một khuôn cố định gồm bốn phần: *từ đâu ra* (vấn đề cụ thể ở chương nào), *vì sao tôi cho là còn trống*, *mức chắc chắn của tôi*, và — phần quan trọng nhất — *phép kiểm* để xác minh nó có thật là khoảng trống hay chỉ là chỗ tôi chưa đọc tới. Tám hướng, xếp theo mức tôi tin là chúng còn trống và đáng làm. Nếu chỉ nhớ một điều: *trước khi chọn bất kỳ hướng nào, hãy chạy phép kiểm của nó* — vì cách nhanh nhất để lãng phí một năm là phát minh lại thứ người ta đã giải quyết ba năm trước.

CẢM BẤY

Cảnh báo về chính chương này, và nó là phần quan trọng nhất cần đọc.

Theo *research-rules.md*, một khảo sát đạt chuẩn cần: nhật ký tìm kiếm tái lập được; sàng ít nhất 25–40 công trình cho mỗi chủ đề; ít nhất ba nhóm nghiên cứu độc lập; ma trận tổng hợp viết theo cột; và mọi khẳng định lõi tựa vào một nguồn mức A hoặc hai nguồn mức B độc lập.

Chương này không đạt bất kỳ điều kiện nào trong số đó. Nó được viết từ lập luận cấu trúc và từ việc đọc rải rác, không từ một khảo sát. Điều đó nghĩa là:

Rủi ro chính: một “khoảng trống” ở đây có thể đã được lấp bởi các công trình tôi chưa đọc. Đây là rủi ro có thật và không nhỏ, đặc biệt với các hướng liên quan tới học máy, nơi tài liệu tăng rất nhanh.

Cách dùng đúng: coi mỗi mục dưới đây là một **giả thuyết cần kiểm**, không phải một kết luận. Mỗi mục kèm một phép kiểm cụ thể, và phép kiểm ấy nên được chạy *trước* khi đầu tư thời gian vào hướng đó.

Việc cần làm nếu dùng chương này để chọn đề tài thật: nâng hướng đã chọn lên thành một dự án survey/ đúng chuẩn — bước 0 tới bước 4 của *research-rules.md* — rồi mới quyết. Chương này chỉ là bước chọn *ứng viên*, không phải bước kết luận.

22.1 Vì sao một danh sách như thế này lại có giá trị

Nếu chương này không đạt chuẩn khảo sát, vì sao viết nó?

Vì nó làm một việc mà một khảo sát không làm: nó *nối các vấn đề đã gặp trong cây thành các câu hỏi*. Hai mươi mốt chương trước liên tục chạm vào các chỗ mà tôi phải viết “chưa có lời giải tốt” —

bài toán lifelong ở [C/13](#), bệnh quá tự tin ở [A/06](#), perceptual aliasing ở [B/11](#). Những chỗ ấy rải rác và dễ quên; gom chúng lại là một việc có giá trị riêng.

Theo `research-rules.md` mục 7, một câu hỏi nghiên cứu tốt mọc ra từ một vấn đề thật chứ không từ cảm hứng ngẫu nhiên. Tám mục dưới đây đều mọc ra từ một vấn đề đã gặp, và phần “từ đâu ra” của mỗi mục là bằng chứng cho điều đó.

Tiêu chí xếp thứ tự: *mức tôi tin là nó còn trống*, nhân với *mức tôi tin là nó đáng giải*. Hai yếu tố ấy độc lập — một bài toán có thể còn trống vì nó không đáng giải.

22.2 Hướng 1 – Hiệu chuẩn trực tuyến có ý thức observability

Từ đâu ra: [C/14](#) mục 3. Hiệu chuẩn trực tuyến cần một cổng quyết định khi nào cập nhật tham số nào, và các cơ chế hiện có còn thô sơ: hoặc cập nhật mù quáng, hoặc dùng các quy tắc thủ công dựa trên một đại lượng chuyển động duy nhất.

Bài toán cụ thể: cho một hệ đang chạy, quyết định *theo thời gian thực* tham số nào đang quan sát được đủ để cập nhật. Cần một chỉ số rõ, một ngưỡng có cơ sở, và một cơ chế chuyển đổi mượt giữa cập nhật và đóng băng.

Vì sao tôi cho là còn trống: các công trình tôi biết [\[81\]](#), [\[82\]](#) thiết lập ý tưởng nhưng dừng ở mức phân tích phổ, vốn đắt cho thời gian thực. Phần lớn hệ mã nguồn mở hoặc không có cổng hoặc dùng ngưỡng thủ công.

Mức chắc chắn: trung bình. Ý tưởng đã có; điều tôi cho là còn trống là một cơ chế *thực dụng và được kiểm chứng*.

Phép kiểm: tìm các công trình sau 2020 với từ khoá “observability-aware calibration”, “degeneracy-aware state estimation”, “online extrinsic gating”. Kiểm xem có công trình nào (a) chạy thời gian thực, (b) có ngưỡng suy ra từ nguyên lý chứ không chỉnh tay, và (c) được đánh giá bằng *tính nhất quán* chứ chỉ bằng độ chính xác. Nếu cả ba đã có, hướng này đã được lấp.

22.3 Hướng 2 – Bất định đáng tin

Từ đâu ra: [A/06](#) mục 4 và [C/17](#) mục 4. SLAM quá tự tin một cách hệ thống, và bốn nguồn của nó đều khó chữa bằng phân tích.

Bài toán cụ thể: một hiệp phương sai mà module phía trên *dùng được* — tức có ANEES gần số chiều trên dữ liệu thật, không chỉ trong mô phỏng, và trong các điều kiện khó chứ không chỉ điều kiện tốt.

Vì sao tôi cho là còn trống: hai lý do cấu trúc đã nêu ở [C/17](#) mục 4A. Nó khó đo (cần chân trị và thống kê), nên không có chỉ số dễ công bố; và một bài cải thiện ATE dễ đăng hơn một bài cải thiện tính trung thực.

Mức chắc chắn: **cao** — đây là hướng tôi tin nhất trong tám hướng. Lý do: tôi có thể chỉ ra bốn cơ chế cụ thể gây quá tự tin ([A/06](#) mục 4B) và không cơ chế nào có lời giải sạch.

Phép kiểm: khảo sát xem có bao nhiêu công trình SLAM báo cáo NEES hoặc ANEES bên cạnh ATE. Nếu tỉ lệ rất thấp thì hướng còn trống được xác nhận theo một chiều — ít người đo, nên ít người cải thiện. Bổ sung: tìm các công trình về “uncertainty calibration” trong học máy và kiểm xem chúng đã được áp dụng cho SLAM chưa.

Ghi chú: hướng này có một ưu điểm phương pháp đáng kể — nó có một chỉ số rõ ràng (ANEES) và một tiêu chí thành công rõ ràng (gần số chiều). Nhiều hướng khác trong danh sách này thiếu điều đó.

22.4 Hướng 3 – Bản đồ sống lâu trong thế giới thay đổi

Từ đâu ra: C/13 mục 4C và B/12 mục 4C. Gần như mọi hệ SLAM giả định thế giới tĩnh ở thang thời gian dài, và đó là giả định sai nhất trong triển khai thực tế.

Bài toán cụ thể: phân biệt “bản đồ sai” với “tôi ở sai chỗ” — hai kết luận cho cùng triệu chứng và đòi hai hành động ngược nhau. Và cái vòng luẩn quẩn: để biết bản đồ sai, phải chắc mình ở đúng chỗ; để chắc mình ở đúng chỗ, phải tin bản đồ.

Vì sao tôi cho là còn trống: các cách tiếp cận hiện có đều là cách phá vòng một cách thô — quên theo thời gian, đếm phiếu qua nhiều lần đi qua, lưu nhiều phiên bản — và không cách nào giải quyết trường hợp thay đổi cấu trúc lớn.

Mức chắc chắn: cao về việc nó là vấn đề đau nhất trong triển khai; trung bình về việc nó còn trống, vì tôi biết có một nhánh công trình về lifelong SLAM mà tôi chưa khảo sát.

Phép kiểm: đây là hướng có phép kiểm rõ ràng nhất vì nó có một chỉ tiêu tự nhiên. Tìm xem có hệ nào duy trì được tỉ lệ *relocalization ổn định qua nhiều tháng* trong một môi trường thay đổi thật, được kiểm định độc lập. Bộ dữ liệu để kiểm: OpenLORIS [79] và các bộ nhiều phiên tương tự. Nếu không hệ nào làm được, hướng còn trống.

Ghi chú: hướng này cần dữ liệu dài hạn, vốn khó thu — và đó vừa là rào cản vừa là lý do nó còn trống. Mini project ở C/13 là một cách bắt đầu thu.

22.5 Hướng 4 – Dự đoán thất bại thay vì phục hồi sau

Từ đâu ra: D/19 mục 6. Mọi cơ chế hiện có là *phản ứng*: phát hiện sau khi đã mất bám, rồi phục hồi.

Bài toán cụ thể: dự đoán trước vài giây rằng hệ thống sắp mất bám, để module phía trên có thể hành động — robot đi chậm lại, quay về vùng đã biết, hoặc chuyển chế độ trước khi hỏng.

Vì sao tôi cho là còn trống: các đại lượng cần thiết đã có sẵn (D/19 khối *Cần nhớ*) và chúng thay đổi trước khi hỏng — số đặc trưng giảm, số điều kiện tăng. Việc nối chúng thành một bộ dự đoán là một bài toán chuỗi thời gian thông thường. Tôi không biết công trình nào làm điều này một cách hệ thống trong SLAM.

Mức chắc chắn: trung bình. Ý tưởng đủ hiển nhiên khiến tôi ngờ rằng ai đó đã làm, và việc tôi không biết có thể chỉ phản ánh việc tôi chưa tìm.

Phép kiểm: tìm với từ khoá “failure prediction SLAM”, “predictive tracking failure”, “proactive relocalization”. Nếu có công trình, kiểm xem chúng dự đoán trước bao lâu và với tỉ lệ báo động giả nào — vì một bộ dự đoán báo trước 0,2 giây thì vô dụng cho việc ra quyết định.

Ghi chú: hướng này có lợi thế là *dữ liệu dễ có* — mọi lần mất bám trong bất kỳ chuỗi nào đều là một mẫu, và bộ nghịch cảnh ở D/19 mini project sinh ra chúng có chủ đích.

22.6 Hướng 5 – Bất định được truyền đúng qua ranh giới học/hình học

Từ đâu ra: C/17 mục 4B và B/08 mục 4C. Kiến trúc lai là hướng đúng, nhưng phần lớn công trình nổi hai khối một cách lỏng lẻo: mô hình học được cho một phép đo, và hiệp phương sai của phép đo ấy được đặt bằng một hằng số chính tay.

Bài toán cụ thể: một mô hình học được sinh ra phép đo kèm hiệp phương sai đã hiệu chuẩn, và hiệp phương sai ấy đúng cả khi dữ liệu nằm ngoài phân bố huấn luyện.

Vì sao tôi cho là còn trống: về cuối là phần khó và là phần tôi cho là chưa giải. Một mô hình biết mình không chắc khi gặp cảnh lạ đòi hỏi phát hiện ngoài phân bố, và đó là bài toán mở trong chính học máy.

Mức chắc chắn: trung bình-cao. Cao về việc bài toán chưa giải; trung bình về việc nó là một khoảng trống riêng của SLAM — có thể nó chỉ là bài toán chung của học máy được nhìn qua lăng kính SLAM.

Phép kiểm: hai bước. Một, tìm trong tài liệu học máy các công trình về hiệu chuẩn bất định và phát hiện ngoài phân bố sau 2023 — nếu có tiến bộ lớn thì hướng này trở thành việc áp dụng chứ không phải nghiên cứu. Hai, kiểm xem có công trình SLAM nào dùng hiệp phương sai học được trong đồ thị nhân tử và đo ANEES của kết quả.

Ghi chú: hướng này gắn chặt với hướng 2, và chúng có thể là một — hoặc nên được làm cùng nhau.

22.7 Hướng 6 — Xử lý suy biến có nguyên tắc

Từ đâu ra: A/05 mục 2C và A/01 mục 6. Việc phát hiện và xử lý chuyển động suy biến hiện chủ yếu là heuristic: các ngưỡng thủ công cho parallax, cho tỉ số điểm số mô hình, cho số điều kiện.

Bài toán cụ thể: một khung thống nhất để (a) phát hiện hướng suy biến theo thời gian thực với chi phí chấp nhận được, (b) phân biệt ba loại suy biến ở A/05 mục 2C, và (c) quyết định hành động — đóng băng biến nào, dùng mô hình nào, hay từ chối.

Vì sao tôi cho là còn trống: mỗi hệ thống tự xử lý theo cách riêng với các ngưỡng riêng, và không có một khung chung. Đặc biệt phần (b) — phân biệt ba loại — tôi không biết cài đặt nào làm một cách tường minh.

Mức chắc chắn: trung bình. Có nhiều công trình về observability phân tích; điều tôi cho là còn trống là một khung chạy được và thống nhất.

Phép kiểm: kiểm xem các thư viện lớn (GTSAM, Ceres, g2o) có cung cấp một cơ chế báo hướng suy biến không, và các hệ SLAM lớn có dùng nó không. Nếu mỗi hệ vẫn tự cài ngưỡng riêng thì khoảng trống được xác nhận. Bổ sung: tìm “degeneracy detection” trong tài liệu LiDAR SLAM, nơi vấn đề này (hành lang dài) gay gắt hơn và có thể đã được giải tốt hơn.

22.8 Hướng 7 — Hệ tự nhận biết lệch và tự sửa

Từ đâu ra: C/14 mục 4B và D/21 mục 2D. Một thiết bị ở hiện trường phải tự biết hiệu chuẩn của nó đã trôi và tự xử lý, vì không ai mang target tới nhà khách hàng.

Bài toán cụ thể: một vòng khép kín gồm phát hiện trôi (không có target, không có chân trị), phân biệt nguyên nhân (nhiệt so với va đập so với lão hoá), tự sửa khi có thể, và báo khi không.

Vì sao tôi cho là còn trống: các mảnh đã có — hiệu chuẩn trực tuyến, phân tích phần dư — nhưng tôi không biết một hệ nào ghép chúng thành một vòng khép kín có đánh giá.

Mức chắc chắn: trung bình-thấp. Đây có thể là một vấn đề kỹ thuật đã được các công ty giải quyết mà không công bố, chứ không phải một khoảng trống nghiên cứu.

Phép kiểm: khác với các hướng khác, phép kiểm ở đây không phải tìm trong tài liệu học thuật mà là tìm trong tài liệu kỹ thuật của sản phẩm — các thiết bị AR và robot thương mại có công bố cách xử lý trôi hiệu chuẩn không. Nếu có thì đây là vấn đề đã giải nhưng không công bố, và giá trị nghiên cứu thấp hơn giá trị kỹ thuật.

22.9 Hướng 8 — Biểu diễn neural cho localization thực dụng

Từ đâu ra: C/16 mục 4B. 3DGS cho hình ảnh đẹp nhưng ba điểm chặn cho robot: không trả lời được truy vấn occupancy, tổn bộ nhớ và cần GPU, và chế độ hồng chưa được hiểu.

Bài toán cụ thể: một biểu diễn neural mà từ đó trích xuất được ESDF tin cậy, chạy trong ngân sách của một SoC nhúng, và có chế độ hồng dự đoán được.

Vì sao tôi cho là còn trống: phần lớn công trình tối ưu cho chất lượng render, không cho truy vấn hình học. Ba điểm chặn ở C/16 mục 4B đều còn.

Mức chắc chắn: **thấp** — đây là hướng tôi ít chắc nhất, vì lĩnh vực chuyển động rất nhanh và đánh giá của tôi chốt tại 9/2026. Rất có thể một hoặc cả ba điểm chặn đã được giải trong thời gian tôi viết chương này.

Phép kiểm: rà lại tài liệu 3DGS-SLAM sau mỗi sáu tháng, và kiểm cụ thể ba điểm: có công trình nào trích xuất occupancy hoặc ESDF từ Gaussian với đánh giá định lượng không; có công trình nào chạy trên phần cứng nhúng không; có công trình nào phân tích chế độ hồng không.

CẦN NHỚ

Cách dùng danh sách này, theo thứ tự.

Một — chạy phép kiểm trước. Mỗi mục có một phép kiểm cụ thể. Chạy nó mất vài ngày và nó có thể tiết kiệm một năm. Cách nhanh nhất để lãng phí thời gian là phát minh lại thứ đã được giải quyết.

Hai — nếu phép kiểm xác nhận khoảng trống, nâng lên thành khảo sát đúng chuẩn. research-rules.md bước 0 tới bước 4: dựng khung câu hỏi, săn nguồn theo giao thức lần cầu tuyệt, sàng theo mức tin A/B/C, đọc ba lượt, lập ma trận tổng hợp. Chỉ sau bước đó mới đủ cơ sở để chọn.

Ba — ưu tiên các hướng có chỉ số rõ ràng. Hướng 2 (ANEES gần số chiều) và hướng 3 (tỉ lệ relocalization ổn định qua tháng) có tiêu chí thành công đo được. Các hướng thiếu điều đó khó chứng minh tiến bộ, và một bài toán không đo được là một bài toán không giải được.

Bốn — ưu tiên các hướng mà bạn có dữ liệu. Hướng 3 và 4 cần dữ liệu dài hạn và dữ liệu thất bại; nếu bạn đang vận hành thiết bị thật thì bạn có lợi thế mà phòng thí nghiệm không có — và đó là lợi thế đáng khai thác hơn là cố cạnh tranh ở chỗ mọi người đều mạnh.

Và một gợi ý cuối, thuộc về E/24: một bài toán ứng dụng cụ thể mà bạn đang làm thường là chỗ tốt hơn để bắt đầu so với một hướng trong danh sách này — vì nó có ràng buộc thật, có dữ liệu thật, và có tiêu chí thành công thật.

22.10 Điều gì chứng minh cách hiểu này sai

Chương này khác các chương khác: mệnh đề của nó không phải một khẳng định kỹ thuật mà là *tám phán đoán về trạng thái của lĩnh vực*, và mỗi phán đoán đã kèm mức chắc chắn cùng phép kiểm riêng ở mục tương ứng.

Mệnh đề bao trùm — *tám hướng này còn trống* — bị bác bỏ từng phần bởi chính các phép kiểm ấy. Tôi kỳ vọng ít nhất hai trong tám sẽ bị bác bỏ khi kiểm nghiệm túc, và hai ứng viên có khả năng nhất là hướng 4 (dự đoán thất bại — ý tưởng đủ hiển nhiên) và hướng 8 (3DGS — lĩnh vực chuyển động quá nhanh).

Có một mệnh đề thứ hai, ẩn hơn và đáng nêu: *rằng một danh sách như thế này có giá trị*. Nó bị bác bỏ nếu việc chọn đề tài nghiên cứu từ một danh sách viết sẵn cho kết quả tệ hơn việc chọn từ một vấn đề bạn đang thật sự gặp. Tôi *ngghiêng về* khả năng đó — và tôi đã nói điều đó ở cuối khối *Cần nhớ*. Danh sách này hữu ích nhất không phải như một thực đơn để chọn mà như một *bản đồ để nhận ra* rằng vấn đề bạn đang gặp trùng với một chỗ mà lĩnh vực cũng chưa giải.

Và một cảnh báo cuối, nhắc lại từ đầu chương vì nó quan trọng: chương này được viết *không có khảo sát*. Nếu bạn dùng nó để chọn đề tài mà không chạy phép kiểm, bạn đang dựa vào phán đoán của một người đọc rải rác — và theo chính `research-rules.md` mục 8, đó là đúng thứ không nên tin.

22.11 Kết nối

Chương này nói ngược về gần như toàn bộ cây, và bảng dưới đây là bản đồ ấy.

Hướng 1 về [C/14](#) và [A/05](#). Hướng 2 về [A/06](#) và [C/18](#). Hướng 3 về [C/13](#) và [B/12](#). Hướng 4 về [D/19](#). Hướng 5 về [C/17](#) và [B/08](#). Hướng 6 về [A/05](#) và [A/01](#). Hướng 7 về [C/14](#) và [D/21](#). Hướng 8 về [C/16](#).

Nối xuống dưới: [E/24-lo-trình](#) giai đoạn năm là chỗ chương này được dùng, và nó nêu rõ quy trình — chọn một hướng, đọc 20–30 công trình gần nhất *của riêng hướng đó*, tái hiện hai tới ba baseline, rồi mới đề xuất. Thứ tự ấy quan trọng: tái hiện baseline trước khi đề xuất là cách duy nhất biết mình đang đứng ở đâu.

Nối sang `research-rules.md`: chương này là bước *trước* bước 0 của tài liệu đó. Nó chọn ứng viên; `research-rules.md` bắt đầu từ việc dựng khung câu hỏi cho ứng viên đã chọn. Nhầm hai bước — coi chương này là kết quả của một khảo sát — là lỗi phương pháp nghiêm trọng nhất có thể mắc với nó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chương này không đạt chuẩn `research-rules.md`, và điều đó đổi cách dùng nó thế nào?
2. Với mỗi hướng, phần nào của mục là quan trọng nhất và vì sao?
3. Hướng nào tôi chắc chắn nhất và hướng nào ít chắc nhất? Nêu lý do của cả hai.
4. Vì sao hướng 2 và hướng 3 được đề nghị ưu tiên, và tiêu chí chung của chúng là gì?
5. Hướng 7 có phép kiểm khác các hướng khác. Khác thế nào, và điều đó nói gì về bản chất của hướng ấy?
6. Nêu bốn bước dùng danh sách này, và nói bước nào tiết kiệm nhiều thời gian nhất.
7. Vì sao một vấn đề bạn đang thật sự gặp thường là chỗ tốt hơn để bắt đầu so với một hướng trong danh sách?

FRAMEWORK VÀ LỘ TRÌNH

TÓM TẮT

Hai chương khép cây. Chương 23 là bản đồ công cụ — ai đã cài sẵn cái gì, và quan trọng hơn, *đọc mã nguồn nào để học được gì*; nó được xếp cuối vì nó hết hạn nhanh nhất và vì nó chỉ có nghĩa khi bạn đã biết mình đang tìm gì. Chương 24 là lộ trình năm giai đoạn: thứ tự học, với lý do cho từng thứ tự, và một lập luận rằng giai đoạn bốn — tự dựng dữ liệu và đo thất bại — là giai đoạn bị bỏ qua nhiều nhất và có giá trị cao nhất. Nếu chỉ nhớ một điều: *đọc mã nguồn của một hệ trường thành dạy nhiều hơn đọc bài báo của nó*, vì phần lớn kiến thức ở Phần C và Phần D không tồn tại trong bài báo.

Phần này ngắn và có vai trò khác hẳn bốn phần trước. Bốn phần trước xây dựng hiểu biết; phần này là điều hướng — nó nói bạn nên đi đâu tiếp và theo thứ tự nào.

Một lưu ý về tính hết hạn. Chương 23 chốt tại tháng 9/2026 và nó là phần hết hạn nhanh nhất của cả cây cùng với C/16 và C/17. Các dự án mã nguồn mở đổi, các thư viện đổi API, và các hệ thống mới xuất hiện. Điều *không* đổi nhanh là *nguyên tắc chọn* — đọc cái gì để học cái gì — nên chương 23 được viết xoay quanh nguyên tắc ấy chứ không xoay quanh danh sách.

Chương 24 thì ngược lại: nó gần như không hết hạn, vì nó nói về thứ tự học, và thứ tự ấy suy ra từ cấu trúc phụ thuộc của chính lĩnh vực. Lie group phải đến trước bundle adjustment vì BA cần Jacobian trên đa tạp; observability phải đến sau BA vì nó là câu hỏi về ma trận của BA. Những quan hệ ấy không đổi.

Bảng 22.1. Bản đồ Phần E: hai chương khép cây.

Ch.	Thư mục	Nội dung	Mức
23	23-framework-va-cor	Nguyên tắc chọn mã nguồn để đọc; sáu hệ phải đọc và học được gì từ mỗi hệ; thư viện nền; bộ dữ liệu và công cụ đánh giá; chốt 9/2026	•
24	24-lo-trình	Lộ trình năm giai đoạn với lý do cho từng thứ tự; lộ trình rút gọn cho người có ít thời gian; và lập luận vì sao giai đoạn bốn quan trọng nhất	••

Chương 23 là tham khảo — đọc lướt một lần để biết có gì, rồi quay lại khi cần chọn công cụ hoặc khi muốn đọc mã nguồn một chủ đề cụ thể. Nó chốt tại 9/2026 và nên rà lại mỗi sáu tháng. Chương 24 thì nên đọc *sớm*, thậm chí trước khi đọc Phần A — vì nó nói thứ tự nên đi, và biết thứ tự trước khi bắt đầu tiết kiệm rất nhiều thời gian. Nếu bạn đang đọc cây này lần đầu và chưa xem chương 24, hãy xem nó ngay bây giờ rồi quay lại.

Framework và công cụ (Frameworks and Tools)

PHỤ THUỘC Toàn bộ cây — chương này chỉ có nghĩa khi bạn đã biết mình đang tìm gì. Đọc nó trước khi đọc Phần A là lãng phí.

TÓM TẮT

Chương này là bản đồ công cụ, và nó được viết xoay quanh một câu hỏi duy nhất: *đọc mã nguồn nào để học được gì?* Cách tổ chức ấy là có chủ ý — một danh sách dự án hết hạn trong hai năm, nhưng nguyên tắc “muốn hiểu marginalization thì đọc VINS-Mono” thì bền hơn nhiều. Nội dung gồm: nguyên tắc chọn mã nguồn để đọc, và vì sao đọc mã dạy nhiều hơn đọc bài báo với phần lớn nội dung Phần C và Phần D; sáu hệ thống đáng đọc trọn vẹn, mỗi hệ kèm chỉ dẫn *đọc phần nào trước*; các thư viện nền và tiêu chí chọn giữa chúng; và bộ dữ liệu cùng công cụ đánh giá. Nếu chỉ nhớ một điều: *phần lớn kiến thức ở Phần C và Phần D không tồn tại trong bài báo* — kiến trúc ba luồng, chính sách quản lý bản đồ, các cổng và ngưỡng — và cách duy nhất học chúng là đọc mã của một hệ đã trưởng thành.

CẠM BÃY

Chương này chót tại tháng 9/2026 và là phần hết hạn nhanh nhất của cây cùng với C/16 và C/17. Dự án đổi, API đổi, hệ mới xuất hiện. Rà lại mỗi sáu tháng. Tôi cũng **không xếp hạng chất lượng** các dự án. Lý do: “tốt hơn” phụ thuộc vào việc bạn cần gì, và một bảng xếp hạng sẽ sai ngay khi ai đó có ràng buộc khác tôi. Thay vào đó, mỗi mục trả lời câu “*đọc nó để học được gì*” — một câu hỏi có câu trả lời ổn định hơn nhiều.

23.1 Vì sao đọc mã nguồn

Có một sự bất đối xứng đáng nói giữa hai nửa của cây này.

Với Phần A và phần lớn Phần B — hình học, MAP, bundle adjustment, các paradigm back-end — *bài báo là nguồn tốt*. Nội dung là toán, nó viết ra được, và một bài báo tốt trình bày nó rõ hơn mã nguồn.

Với Phần C và Phần D thì ngược lại, và lý do có cấu trúc. Kiến trúc ba luồng ở C/13 mục 4A, chính sách chọn và loại keyframe, các cổng ở B/10 và B/12, cách hoà nhập loop closure với tracking — những thứ ấy *không viết được thành định lý*, chúng không phải đóng góp mà một bài báo khoe, và chúng chiếm phần lớn mã nguồn. Một bài báo mô tả hệ thống như thể nó chạy tuần tự; mã nguồn thật đầy hàng đợi, khoá, và cờ trạng thái.

Hệ quả thực hành: *với Phần C và Phần D, đọc mã là cách học chính, không phải cách bổ sung*. Và đọc mã của một hệ *đã trưởng thành* — đã chạy trên nhiều bộ dữ liệu, đã được nhiều người dùng, đã sửa nhiều lỗi — dạy nhiều hơn đọc mã của một hệ mới hơn nhưng chưa va chạm.

Cách đọc mã có hiệu quả, ba nguyên tắc.

Một — đọc theo câu hỏi, không đọc tuần tự. Mở mã với một câu hỏi cụ thể: “nó quyết định tạo keyframe thế nào?” Tìm chỗ ấy, đọc kỹ, rồi đóng lại. Đọc từ đầu tới cuối một hệ vài chục nghìn dòng là cách chắc chắn bỏ cuộc.

Hai — tìm các ngưỡng và hỏi chúng từ đâu ra. Mỗi hằng số trong mã là một quyết định. Phần lớn được chỉnh bằng thực nghiệm, và biết *cái nào* có cơ sở lý thuyết và cái nào không là kiến thức có giá trị.

Ba — so mã với bài báo của chính nó. Chỗ chúng khác nhau là chỗ thú vị nhất: đó là chỗ lý thuyết gặp thực tế, và thường là chỗ có một bài học không được viết ra.

23.2 Sáu hệ thống đáng đọc

Bảng 23.1. Sáu hệ thống và cái đáng học từ mỗi hệ. Cột cuối là chỉ dẫn đọc — phần nào trước.

Hệ thống	Đọc để học	Đọc phần nào trước
ORB-SLAM3	Kiến trúc SLAM đầy đủ nhất: ba luồng, covisibility, Atlas, quản lý bản đồ	Bắt đầu ở vòng lặp tracking; rồi chính sách tạo và loại keyframe (C/13); rồi loop closing và cách nó hoà nhập với tracking
VINS-Fusion	Cửa sổ trượt và marginalization, ngắn và dễ đọc nhất trong sáu	Bắt đầu ở chỗ dựng bài toán tối ưu; rồi marginalization (A/04 mục 5); rồi khởi tạo VI
OpenVINS	Bộ lọc chuẩn mực; hiệu chuẩn trực tuyến; <i>có mô phỏng có chân trị</i>	Chỗ định nghĩa vector trạng thái; rồi cờ FEJ và chỗ Jacobian được tính; rồi mô-đun mô phỏng — đây là chỗ tính NEES được (C/18)
GTSAM	Đồ thị nhân tử đúng cách; iSAM2; cách viết một loại nhân tử mới	Ví dụ về custom factor; rồi cấu trúc Bayes tree; rồi Marginals (A/06 mục 2B)
Ceres	Manifold, auto-diff, bộ giải thừa với Schur	Tài liệu về Manifold (A/03 mục 6); rồi các lựa chọn linear solver và ordering (A/04 mục 4D)
Kalibr	Toàn bộ tư duy hiệu chuẩn = batch MAP	<i>Tài liệu quy trình thu dữ liệu trước mã</i> — mỗi yêu cầu trong đó là một hàng của Bảng kích thích ở A/05

Một gợi ý về thứ tự nếu bạn đọc cả sáu: *VINS-Fusion* trước vì nó ngắn nhất và cấu trúc rõ; *OpenVINS* tiếp theo để thấy thế giới bộ lọc và để có mô phỏng; *ORB-SLAM3* thứ ba vì nó đầy đủ nhất và cũng phức tạp nhất — đọc nó trước khi có nền là nản. *Kalibr* đọc song song khi học [C/14](#). *GTSAM* và *Ceres* đọc như tài liệu tham khảo khi cần, không đọc tuần tự.

23.3 Các hệ thống khác, theo chủ đề

Xếp theo *chủ đề bạn muốn học* chứ không theo mức độ nổi tiếng.

Direct method và mô hình quang trắc: [DSO](#) là cài đặt tham chiếu; [DM-VIO](#) thêm quán tính và dynamic marginalization. Đọc [DSO](#) để hiểu [B/08](#) mục 2B và [A/02](#) mục 5B.

Kỹ thuật số học và chất lượng phần mềm: **Basalt**. Dạng căn bậc hai (B/09 mục 5A), spline $SE(3)$ cho hiệu chuẩn, và chất lượng kỹ thuật rất cao — đáng đọc như một ví dụ về cách viết phần mềm SLAM tử tế.

Semi-direct và tốc độ: **SVO-Pro**. Đọc để thấy mục 2C của B/08 — phân công theo thể mạnh.

SfM và localization quy mô lớn: **COLMAP** cho SfM chuẩn công nghiệp; **HLoc** cho pipeline localization phân cấp (B/12 mục 2).

Kiến trúc lai học được: **DROID-SLAM** và **DPVO**. Đọc để thấy ranh giới ở C/17 mục 3B được đặt ở đâu trong mã.

Quản lý bản đồ nhiều phiên: **maplab**. Hiếm framework coi C/13 là chủ đề chính chứ không phải phụ.

Ngữ nghĩa trong đồ thị: **Kimera**. Ví dụ về việc thông tin ngữ nghĩa cũng chỉ là một loại nhân tử.

Biểu diễn neural: **SplaTAM**, **MonoGS**. Chốt 9/2026; xem cảnh báo ở C/16 mục 4.

Fiducial: **TagSLAM**. Liên quan ../marker/.

23.4 Thư viện nền

Lie group: **Sophus** là lựa chọn phổ biến nhất trong hệ sinh thái C++ SLAM; **manif** có API hiện đại hơn và Jacobian tường minh cho mọi phép toán, và nó đi kèm [25] vốn là tài liệu tham chiếu tốt nhất về quy ước. Tiêu chí chọn: nếu bạn đang học và cần hiểu quy ước rõ ràng, dùng **manif**; nếu bạn đang tích hợp vào hệ sinh thái có sẵn, dùng cái mà hệ đó dùng.

Tối ưu: **Ceres** cho auto-diff và sự trưởng thành; **GTSAM** cho đồ thị nhân tử và smoothing gia tăng; **g2o** cho mã ngắn dễ đọc (tốt để học, không nhất thiết để dùng); **SymForce** cho việc sinh mã Jacobian tối ưu; **Theseus** cho tối ưu khả vi (C/17 mục 3B).

Tiêu chí chọn giữa Ceres và GTSAM, vốn là lựa chọn hay gặp nhất: nếu bài toán của bạn là một bài toán tối ưu tĩnh — hiệu chuẩn, BA ngoại tuyến — thì Ceres đơn giản hơn. Nếu nó là một bài toán *gia tăng* với các biến được thêm liên tục — SLAM thời gian thực — thì GTSAM với iSAM2 có lợi thế cấu trúc.

Thị giác: **OpenCV** cho gần như mọi thứ ở B/07 và A/02. Một cảnh báo thực dụng: kiểm xem build của bạn có bật đúng cờ kiến trúc (SIMD) không — đây là một lỗi hiệu năng phổ biến và dễ sửa (D/20 mục 4B).

23.5 Bộ dữ liệu và công cụ đánh giá

Công cụ: **evo** dễ dùng; **rpg_trajectory_evaluation** [39] chặt chẽ hơn về căn chỉnh. Với cả hai, chọn đúng chế độ căn chỉnh theo C/18 mục 2B — mặc định không phải lúc nào cũng đúng cho cấu hình của bạn, và đây là lỗi phổ biến nhất trong đánh giá.

Bộ dữ liệu, xếp theo cái chúng dùng để đo:

Đo độ chính xác trong điều kiện tốt: EuRoC [103] (drone, trong nhà, có motion capture), TUM-VI [104] (cầm tay, có hiệu chuẩn quang trắc), TUM-RGBD [101] (RGB-D, định nghĩa gốc của ATE/RPE), KITTI [105] (xe, ngoài trời).

Đo hoạt động dài hạn và thay đổi môi trường: OpenLORIS [79] — một trong rất ít bộ chạm vào C/13 mục 4C.

Đo localization trong điều kiện thay đổi: Aachen Day-Night [106].

CẦN NHỚ

Không bộ dữ liệu công khai nào đủ, và đây là nhắc lại [D/21](#) mục 4A vì nó là điều quan trọng nhất của mục này.

Ba lý do: chúng không chứa các chế độ hỏng ở [D/19](#); chúng quá ngắn cho các vấn đề ở [C/13](#); và chúng đo ATE trung bình trên các chuỗi hệ thống chạy được, trong khi sản phẩm cần tỉ lệ thất bại và hành vi ở đuôi.

Dùng chúng cho đúng việc: *so sánh thuật toán trong điều kiện kiểm soát, và phát hiện hồi quy*. Không dùng chúng để kết luận hệ thống sẵn sàng cho khách hàng.

Việc cần làm song song: xây bộ dữ liệu nội bộ theo ba phần ở [D/21](#) mục 4A — chuỗi đại diện, chuỗi nghịch cảnh, và chuỗi từ hiện trường. Phần thứ ba có giá trị nhất và chỉ có được nhờ vòng lặp fleet analytics.

23.6 Kết nối

Chương này là tham khảo cho toàn cây, và cách dùng đúng là tra ngược: khi đọc một chương và muốn xem nó được cài đặt thế nào, quay lại [Bảng 23.1](#) và mục 3.

Nối xuống dưới: [E/24-lo-trình](#) giai đoạn hai là chỗ chương này được dùng nhiều nhất — đọc ba hệ thống theo thứ tự, và với mỗi hệ vẽ lại đồ thị nhân tử cùng danh sách biến nào bị fix. Đó là bài tập biến việc đọc mã từ tiêu thụ thụ động thành một việc có sản phẩm kiểm được.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao với Phần C và Phần D thì đọc mã là cách học chính chứ không phải bổ sung?
2. Nêu ba nguyên tắc đọc mã nguồn có hiệu quả, và nói nguyên tắc nào tiết kiệm nhiều thời gian nhất.
3. Với mỗi hệ trong sáu hệ ở [Bảng 23.1](#), nói nó dạy được gì mà năm hệ kia không dạy.
4. Tiêu chí chọn giữa Ceres và GTSAM là gì?
5. Vì sao không bộ dữ liệu công khai nào đủ, và chúng nên được dùng cho việc gì?
6. Khi dùng `evo` hoặc `rpg_trajectory_evaluation`, lỗi phổ biến nhất là gì?

Lộ trình (Roadmap)

PHỤ THUỘC Không có — chương này nên đọc trước Phần A, rồi quay lại sau mỗi giai đoạn. Nó là bản đồ chứ không phải nội dung.

TÓM TẮT

Chương này nói thứ tự nên đi, và quan trọng hơn, *lý do* cho từng thứ tự. Năm giai đoạn: nền tảng không rút ngắn được; đọc hệ thống thật; hợp nhất với hiệu chuẩn; đối mặt thực tế; và cuối cùng là nghiên cứu. Điểm khác biệt của chương so với một danh sách đọc thông thường nằm ở hai chỗ. Thứ nhất, mỗi thứ tự đều có lý do suy ra từ *cấu trúc phụ thuộc* của lĩnh vực — Lie group phải trước bundle adjustment vì BA cần Jacobian trên đa tạp; observability phải sau BA vì nó là câu hỏi về ma trận của BA. Những quan hệ ấy không đổi theo thời gian, nên chương này gần như không hết hạn. Thứ hai, và là luận điểm riêng của chương: *giai đoạn bốn — tự dựng dữ liệu và đo thất bại — bị bỏ qua nhiều nhất và có giá trị cao nhất*. Phần lớn người học nhảy từ giai đoạn ba sang giai đoạn năm, và hậu quả là họ cải tiến những thứ không phải vấn đề. Nếu chỉ nhớ một điều: *cải tiến trước khi đo là cách nhanh nhất để tối ưu sai thứ*.

24.1 Vì sao thứ tự quan trọng

Một lộ trình có thể chỉ là một danh sách sắp xếp tùy tiện. Lộ trình này thì không, và lý do đáng nói vì nó cũng là lý do chương này không hết hạn.

Lĩnh vực này có một *cấu trúc phụ thuộc* thật: một số thứ không hiểu được nếu chưa hiểu thứ khác, và quan hệ ấy là quan hệ logic chứ không phải quy ước sự phạm.

Ba ví dụ cụ thể từ chính cây này. *Lie group phải đến trước bundle adjustment*, vì BA cần Jacobian của phần dư theo tư thế, và tư thế sống trên đa tạp — không có A/03 thì công thức ở A/04 là hộp đen. *Observability phải đến sau BA*, vì toàn bộ A/05 là một câu hỏi về ma trận thông tin mà A/04 dựng lên. *Hiệu chuẩn phải đến sau cả hai*, vì C/14 chỉ có nghĩa khi đã có khung MAP để thả biến vào và đã có lý thuyết kích thích để biết khi nào thả được.

Những quan hệ ấy không phụ thuộc vào công nghệ hiện tại. Chúng vẫn đúng nếu mọi framework trong E/23 biến mất.

Có một nguyên tắc thứ hai chi phối lộ trình, ít hiển nhiên hơn: *học một thứ trước khi cần nó thì quên; học nó đúng lúc cần thì nhớ*. Đây là lý do lộ trình xen kẽ lý thuyết với thực hành thay vì học hết lý thuyết rồi mới làm — và là lý do mỗi chương trong cây có một mini project.

24.2 Giai đoạn 1 — Nền tảng, không rút ngắn được

Nội dung: A/01 (hình học đa thị) → A/02 (mô hình camera) → A/03 (Lie group) → A/04 (MAP và BA) → A/05 (observability) → A/06 (bất định).

Vì sao thứ tự này: đã giải thích ở mục 1 cho ba quan hệ chính. Thêm hai quan hệ: chương 1 trước chương 2 vì chương 2 lắp vào chỗ chương 1 bỏ trống; chương 6 cuối cùng vì nó cần cả Λ từ chương 4 lẫn điều kiện khả nghịch từ chương 5.

Vì sao không rút ngắn được: đây là phần duy nhất trong cây mà tôi khẳng định như vậy, nên nó đáng biện minh. Mọi thứ ở bốn phần sau đều là các cách khác nhau để dùng sáu chương này. Một người bỏ qua A/03 sẽ đọc mọi Jacobian như bùa chú; một người bỏ qua A/05 sẽ chẩn đoán sai mọi lần hệ thống hỏng. Không có đường tắt, và cố đi tắt sẽ tốn nhiều thời gian hơn.

Cách đo tiến độ, và đây là điểm quan trọng: *tự dẫn lại được trên giấy.* Cụ thể: dẫn lại ràng buộc epipolar từ tính đồng phẳng; dẫn lại $\partial(R\mathbf{p})/\partial\phi$; dẫn lại phần bù Schur; dẫn lại bảy bậc tự do của SfM mono. Nếu chưa dẫn lại được thì chưa xong, bất kể đã đọc bao nhiêu lần.

Mini project nên làm: chương 1 (pipeline hai khung và ba cấu hình suy biến), chương 3 (kiểm mọi Jacobian bằng sai phân số — bài này tạo ra bộ unit test dùng suốt về sau), và chương 5 (phổ giá trị riêng trên bốn quỹ đạo).

Thời gian: đây là phần dài nhất và nên là phần dài nhất.

24.3 Giai đoạn 2 – Đọc hệ thống thật

Nội dung: đọc B/07 tới B/12 song song với việc đọc mã nguồn ba hệ thống, theo thứ tự ở E/23 mục 2: VINS-Fusion, rồi OpenVINS, rồi ORB-SLAM3.

Vì sao thứ tự hệ thống này: VINS-Fusion ngắn nhất và cấu trúc rõ nhất, nên nó là chỗ học cách đọc mã SLAM; OpenVINS cho thể giới bộ lọc và — quan trọng hơn — cho một bộ mô phỏng có chân trị, thứ cần cho C/18; ORB-SLAM3 đầy đủ nhất và phức tạp nhất, nên đọc nó cuối.

Bài tập bắt buộc cho mỗi hệ, và đây là phần biến việc đọc mã từ tiêu thụ thụ động thành một việc có sản phẩm: *vẽ lại đồ thị nhân tử và liệt kê chính xác biến nào được fix.* Với mỗi hệ, trả lời: những biến nào trong vector trạng thái? Những loại nhân tử nào? Cái gì bị fix và vì sao? Nó xử lý gauge thế nào?

Bài tập ấy có giá trị vượt xa việc hiểu ba hệ thống: nó buộc bạn đặt mỗi hệ vào Bảng ở 00-map, và sau ba lần bạn sẽ thấy chúng là ba ô của cùng một bảng.

Cách đo tiến độ: bạn giải thích được vì sao mỗi hệ đưa ra các lựa chọn kiến trúc của nó, và bạn chỉ ra được ít nhất một chỗ mà mã khác với bài báo.

24.4 Giai đoạn 3 – Hợp nhất với hiệu chuẩn

Nội dung: C/14 cùng mini project của nó, cộng C/13 và C/15.

Vì sao giai đoạn này đứng riêng: vì nó là chỗ luận đề trung tâm của cây được chứng minh bằng tay. Mini project ở C/14 — viết một bộ giải rồi chạy bốn bài toán khác nhau chỉ bằng cách đổi danh sách biến — là bài tập làm sáng tỏ luận đề rõ nhất trong cả cây.

Quy trình đề nghị: chạy Kalibr trên dữ liệu của mình để có cảm giác; rồi tự viết một bài toán hiệu chuẩn bằng Ceres hoặc GTSAM; rồi *thêm dần biến vào* cho tới khi nó trở thành VIO. Mỗi lần thêm một biến, hỏi: biến này cần chuyển động gì để quan sát được? Đó là Bảng kích thích ở A/05 được kiểm bằng tay.

Cách đo tiến độ: bạn có một bộ mã chạy được cả bốn cấu hình, và bạn giải thích được phổ giá trị riêng của từng cấu hình.

Vì sao [C/13](#) thuộc giai đoạn này: vì quản lý bản đồ và hiệu chuẩn chia nhau một câu hỏi — cái gì được coi là cố định và cái gì được phép đổi. Và vì chương 13 là chương bị coi nhẹ nhất, nên đặt nó cạnh một chương mà người học đang hào hứng là cách để nó không bị bỏ.

24.5 Giai đoạn 4 – Đối mặt thực tế

Nội dung: [C/18](#) (phương pháp đánh giá) và [D/19](#) (chế độ hỏng), rồi *tự dựng dữ liệu trong môi trường mục tiêu*, đo thất bại, phân loại nguyên nhân.

Đây là giai đoạn mà tôi cho là quan trọng nhất và bị bỏ qua nhiều nhất, nên nó đáng một lập luận đầy đủ.

A. Vì sao nó bị bỏ qua

Ba lý do, và cả ba đều dễ hiểu.

Một — nó không có cảm giác tiến bộ. Đọc một bài báo mới cho cảm giác học được điều gì đó; thu ba giờ dữ liệu trong hành lang văn phòng thì không.

Hai — nó không có sản phẩm khoe được. Không có bảng số nào để đưa vào báo cáo, không có thuật toán mới nào.

Ba — nó đòi hỏi thừa nhận rằng mình chưa biết vấn đề là gì. Sau ba giai đoạn học, cảm giác tự nhiên là bắt tay vào cải tiến. Việc dừng lại để đo là một sự thừa nhận rằng ta chưa biết cần cải tiến cái gì.

B. Vì sao nó quan trọng nhất

Lập luận gọn: *cải tiến trước khi đo là cách nhanh nhất để tối ưu sai thứ.*

Cụ thể hơn. Sau ba giai đoạn, bạn có một danh sách các thứ có thể cải tiến — front-end tốt hơn, back-end tốt hơn, bản đồ tốt hơn. Danh sách ấy đến từ việc đọc, nên nó phản ánh *những gì cộng đồng đang quan tâm*, không phản ánh *vấn đề của bạn*.

Trong thực tế, khi đo, người ta thường phát hiện rằng vấn đề thật là một thứ không có trong danh sách: một chỗ trong môi trường có gương; ánh sáng thay đổi vào một giờ nhất định; bánh xe trượt trên một loại sàn cụ thể; hiệu chuẩn trôi sau hai giờ chạy. Không vấn đề nào trong số đó được giải bằng một detector tốt hơn.

Theo [C/18](#) mục 5D: *SOTA thật nằm ở đuôi phân bố*. Và đuôi phân bố của *bạn* chỉ đo được trên dữ liệu của bạn.

C. Làm gì cụ thể

Bước một — *dựng bộ dữ liệu*, theo ba phần ở [D/21](#) mục 4A: chuỗi đại diện (dài, môi trường điển hình), chuỗi nghịch cảnh (theo mini project [D/19](#)), và — nếu có thiết bị chạy thật — chuỗi từ hiện trường.

Bước hai — *đo theo bốn câu hỏi ở C/18 khối Cần nhớ*: chính xác tới đâu, có thành thật không, hỏng thế nào và bao nhiêu lần, ở đuôi thì sao.

Bước ba — *phân loại nguyên nhân*, không phân loại triệu chứng. Mười lần mất bám có thể có ba nguyên nhân, và việc gộp chúng theo triệu chứng che mất điều đó. Dùng các đại lượng ở [D/19](#) khối *Cần nhớ* để phân cụm.

Bước bốn — *chỉ bây giờ mới bắt đầu cải tiến*, và bắt đầu từ cụm lớn nhất.

Cách đo tiến độ: bạn có một bảng xếp hạng nguyên nhân thất bại trong môi trường của bạn, với tần suất. Nếu bảng ấy trùng với danh sách bạn đoán trước khi đo thì hoặc bạn rất giỏi hoặc bạn chưa đo đủ khó.

24.6 Giai đoạn 5 — Nghiên cứu

Nội dung: [D/22](#) để chọn ứng viên, rồi `research-rules.md` để làm đúng cách.

Quy trình, và thứ tự quan trọng.

Một — chọn một hướng, từ [D/22](#) hoặc — tốt hơn — từ bảng xếp hạng nguyên nhân ở giai đoạn 4. Hướng đến từ vấn đề thật của bạn có ba lợi thế: bạn có dữ liệu, bạn có tiêu chí thành công, và bạn biết nó đáng giải.

Hai — chạy phép kiểm của hướng đó ([D/22](#) khỏi *Cần nhớ*). Vài ngày, và nó có thể tiết kiệm một năm.

Ba — nếu phép kiểm xác nhận, nâng lên thành khảo sát đúng chuẩn: `research-rules.md` bước 0 tới bước 4. Đọc 20–30 công trình gần nhất của riêng hướng đó, có nhật ký tìm kiếm, có ma trận tổng hợp.

Bốn — tái hiện hai tới ba baseline trước khi đề xuất bất cứ gì. Theo [C/18](#) mục 5B: nếu baseline của bạn không đạt con số công bố thì mọi so sánh về sau vô nghĩa. Và bước này thường dạy nhiều hơn cả bước năm.

Năm — chỉ bây giờ mới đề xuất.

Cách đo tiến độ: bạn có một `00-question.md` theo `research-rules.md` mục 7 — một câu hỏi đủ cụ thể để có đáp án đúng/sai, một giả thuyết kèm mức tin bằng số, và một tiêu chí falsify viết *trước* khi chạy.

CẦN NHỚ

Lộ trình rút gọn, nếu bạn chỉ có thời gian cho phần xương sống.

Bảy chương phải đọc kỹ và tự dẫn lại được: [A/03](#) (Lie group), [A/04](#) (MAP và BA), [A/05](#) (observability), [B/07](#) (front-end), [B/09](#) (paradigm back-end), [C/13](#) (quản lý bản đồ), [C/14](#) (hiệu chuẩn).

Ba mini project có tỉ lệ giá trị trên thời gian cao nhất: kiểm Jacobian bằng sai phân số ([A/03](#) — tạo ra bộ test dùng suốt); phổ giá trị riêng trên bốn quỹ đạo ([A/05](#) — làm observability thành thứ nhìn thấy được); và đi từ hiệu chuẩn tới SLAM bằng cách thả biến ([C/14](#) — chứng minh luận đề trung tâm).

Một hệ thống đọc trọn vẹn: VINS-Fusion, vì nó ngắn nhất mà vẫn đủ các thành phần.

Một việc không được bỏ: giai đoạn 4. Kể cả khi rút gọn mọi thứ khác, hãy tự thu dữ liệu trong môi trường của mình và đo thất bại. Đây là việc phân biệt một người hiểu lĩnh vực với một người hiểu *bài toán của mình* — và chỉ có cái thứ hai mới tạo ra thứ dùng được.

24.7 Một gợi ý về chỗ bắt đầu

Một nhận xét cuối, và nó là một nhận xét về phương pháp chứ không về nội dung.

Nếu bạn đang có một bài toán ứng dụng cụ thể — ví dụ docking bằng marker với yêu cầu độ chính xác milimet, chủ đề của `../marker/` — thì bài toán ấy thường là chỗ tốt hơn để bắt đầu so với việc đi tuần tự qua cây này.

Lý do: một bài toán ứng dụng hẹp chứa *bản thu nhỏ* của gần như toàn bộ lộ trình. Nó có hiệu chuẩn (C/14); nó có observability và lưỡng nghĩa (A/05); nó có ngân sách sai số (A/06); nó có yêu cầu sản xuất (D/21); và nó có một tiêu chí thành công rõ ràng mà không benchmark nào cho bạn.

Quan trọng hơn cả: nó cho bạn *lý do để cần* mỗi chương. Học A/05 vì tò mò thì khó nhớ; học nó vì tư thế của bạn đang lưỡng nghĩa và bạn cần biết vì sao thì nhớ rất lâu. Đây là nguyên tắc “học đúng lúc cần” ở mục 1, áp dụng ở quy mô lớn nhất.

Cách kết hợp đề nghị: dùng bài toán ứng dụng làm *trực*, và dùng cây này làm *tài liệu tra* — mỗi lần gặp một chỗ không hiểu, tìm chương tương ứng và đọc nó trọn vẹn. Sau vài tháng bạn sẽ đã đọc phần lớn cây, và mỗi chương đều gắn với một vấn đề thật bạn từng gặp.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba quan hệ phụ thuộc trong giai đoạn 1 và lý do logic của mỗi quan hệ.
2. Cách đo tiến độ của giai đoạn 1 là gì, và vì sao “đã đọc xong” không phải một cách đo?
3. Bài tập bắt buộc khi đọc mỗi hệ thống ở giai đoạn 2 là gì, và nó cho bạn thấy điều gì sau ba lần?
4. Nêu ba lý do vì sao giai đoạn 4 bị bỏ qua, và lập luận vì sao nó quan trọng nhất.
5. Vì sao một hướng nghiên cứu đến từ bảng xếp hạng nguyên nhân của bạn tốt hơn một hướng chọn từ danh sách?
6. Trong quy trình năm bước của giai đoạn 5, bước nào thường dạy nhiều nhất và vì sao?
7. Vì sao một bài toán ứng dụng cụ thể thường là chỗ tốt hơn để bắt đầu so với đi tuần tự qua cây?

Sổ những điều chưa biết

TÓM TẮT

Đây là danh sách công việc học tập của cây, mô phỏng quyển “*Notebook of things I don’t know about*” mà Feynman mở khi làm nghiên cứu sinh ở Princeton. Mỗi mục là một **câu hỏi cụ thể** mà tôi biết là mình chưa trả lời được, chứ không phải một chủ đề chung chung. Nếu chỉ nhớ một điều: sổ này chỉ có giá trị khi được *thu ngắn dần* — mục nào giải quyết xong thì chuyển thành một đoạn trong bài chính và ghi ngày.

Bản đầy đủ và được cập nhật thường xuyên nằm ở tệp 99-chua-biet.md tại gốc cây; bản dưới đây là ảnh chụp tại ngày lập, giữ trong PDF để người đọc bản in cũng thấy cây này còn nợ những gì. Khi hai bản lệch nhau thì bản .md là bản đúng.

Điều đáng nói ngay: mục quan trọng nhất không phải một câu hỏi tri thức mà là một khoản **nợ kiểm chứng**, và nó đứng trên mọi câu hỏi về nội dung.

Nợ kiểm chứng

Toàn bộ refs.bib được soạn từ trí nhớ về công trình gốc và *chưa* được đối chiếu với trang nhà xuất bản hay arXiv. Theo learning-rules.md mục 5, như vậy là chưa qua cửa kiểm chứng nào, và một nguồn bịa nằm im trong cây nhiều năm rồi lộ ra đúng lúc cần dựa vào nó là kịch bản tệ nhất.

Ưu tiên đối chiếu theo mức cây dựa vào chúng: hartley2004, triggs2000ba và barfoot2017 là nền của cả Phần A; civera2008inversedepth chống đỡ lập luận nghịch đảo độ sâu dùng lại ở ba chương; sattler2019understanding và pittaluga2019revealing là hai khẳng định mạnh về mặt kết luận nên phải kiểm kỹ nhất. Mỗi entry sau khi đối chiếu cần được dán lại mức tin A/B/C *sau khi* đã đọc được ít nhất abstract gốc; mức tin hiện tại là phán đoán chưa đối chiếu.

Hai khoản nợ khác cùng loại. *Chưa có bài nền tảng nào được đọc tới lượt ba* theo phương pháp Keshav — ít nhất triggs2000ba và một trong hai sách nền phải qua lượt ba trước khi coi Phần A là hoàn chỉnh. Và *chưa có thư mục code/* nào: mọi công thức được đánh dấu “cửa (b)” trong header các chương đều là tính tay, chưa có script chạy được; ưu tiên là Jacobian ở A/03, phần bù Schur ở A/04, và quan hệ σ_z ở A/06.

Câu hỏi về nguyên lý và toán học

Công thức Rodrigues có kỳ dị 0/0 khi góc tiến về không, và mọi thư viện chuyển sang khai triển Taylor dưới một ngưỡng — nhưng ngưỡng ấy đặt ở đâu thì đúng, và sai số số học ở lân cận ngưỡng lớn cỡ nào? Mini project ở A/03 đo được con số này. Liên quan: nhiễu loạn trái và phải cho cùng cực tiểu với bài toán trơn và bước nhỏ, nhưng với LM damping mạnh trên bài toán không lồi, hai quy ước có thể đi hai đường và rơi vào hai cực tiểu địa phương khác nhau; tôi chưa kiểm và chưa tìm được nguồn khẳng định. Còn một khoản nợ suy dẫn: A/03 khẳng định mọi tham số hoá ba số của $SO(3)$ đều có kỳ dị, đối chiếu được với hai nguồn nhưng tôi *không tự chứng minh được* phần topology.

Hai câu hỏi về tính tách được. Lệch điểm chính Δc_x tương đương xấp xỉ một yaw $\Delta c_x/f$ theo A/02; nhưng tôi ngờ chúng tách được *một phần* khi chuyển động đủ đa dạng, vì lệch điểm chính gắn với vị trí trong ảnh còn yaw thì không — làm rõ điều này cần một phân tích observability cụ thể. Và ngưỡng parallax ở B/10 được dẫn từ hình học tam giác nhỏ, bỏ qua ảnh hưởng của số lượng quan sát và bỏ qua phân bố lệch ở A/06; một công thức đúng hơn nên tính từ ma trận thông tin đầy đủ.

Câu hỏi về observability và bất định

Câu hỏi cụ thể nhất và dễ kiểm nhất: thành phần của $\mathbf{p}_{\text{cam}}^{\text{imu}}$ song song với trục quay có quan sát được *yếu* không? C/14 nêu rằng ngoài số hạng $\omega \times \mathbf{p}$ còn có $\dot{\omega} \times \mathbf{p}$, và số hạng sau đưa vào một cơ chế quan sát khác nếu tốc độ góc *thay đổi*. Phát biểu chặt hơn phải là: với ω hằng quanh một trục thì không quan sát được; với ω biến thiên thì có thể quan sát được yếu. Mini project ở C/14 đo được điều này chỉ bằng cách thêm một quỹ đạo.

Câu hỏi lớn hơn và quan trọng hơn: SLAM hiện đại *quá tự tin* tới mức nào? Bệnh được thiết lập cho EKF-SLAM; với các hệ tối ưu hoá hiện đại tôi *suy ra* từ bốn cơ chế ở A/06 rằng nó vẫn còn, nhưng không có số liệu về mức độ. Một khảo sát ANEES trên nhiều hệ và nhiều bộ dữ liệu là thí nghiệm đáng làm mà tôi không biết ai đã làm — và nó cũng chính là phép kiểm cho hướng số hai ở D/22. Hai câu hỏi định lượng cùng nhóm: bao nhiêu phần trăm cạnh sai thì GNC sụp (B/11 đo được), và marginalization trong cửa sổ trượt tối ưu hoá gây bất nhất tới mức nào so với trong bộ lọc.

Câu hỏi về phương pháp và triển khai

Mệnh đề rủi ro cao nhất trong cả cây nằm ở D/20: vòng lặp nhiệt $\rightarrow$ trôi ống kính $\rightarrow$ trôi hiệu chuẩn. Tôi suy ra nó từ A/02 và C/14, *không có số liệu đo và không dẫn được nguồn nào đã đo nó một cách hệ thống*. Phép kiểm rẻ và nên làm sớm: ghi nhiệt độ vỏ máy cùng tham số hiệu chuẩn trực tuyến trên một thiết bị thật trong vài giờ, xem chúng có tương quan không. Nếu không, phải rút mệnh đề.

Ba mệnh đề khác cùng loại, đều dễ kiểm và đều chưa kiểm. *Direct suy giảm nặng hơn indirect với vật động* (D/19) — chạy cả hai họ trên cùng chuỗi với vật động chiếm các tỉ lệ diện tích khác nhau. *Số inlier không tương quan với sai số tư thế* (A/01, B/10) — đo tương quan trên một tập đủ lớn gồm cả cấu hình suy biến, so với parallax trung vị. *Dạng căn bậc hai đáng dùng* (B/09) — lợi ích có thể không đo được khi số điều kiện nhỏ, và ngưỡng chưa biết. Cuối cùng, một con số mà tôi không biết ai đã công bố cho môi trường thật: *tốc độ lão hoá của bản đồ*, tức tỉ lệ landmark từ phiên đầu còn sống sau nhiều phiên (C/13 đo được).

Câu hỏi về học máy

Mục có rủi ro sai cao nhất trong C/17: khẳng định rằng *bất định học được là chỗ trống*. Nó dựa trên ấn tượng từ việc đọc rải rác, không phải một khảo sát — và tôi đã ghi rõ điều đó trong chương. Làm khảo sát cho nó là việc cần làm *trước* khi dùng D/22 hướng hai và hướng năm để chọn đề tài. Hai câu hỏi phụ: mức độ nghiêm trọng của việc mạng nơ-ron không được hiệu chuẩn xác suất, trong ngữ cảnh SLAM cụ thể (tôi không dẫn được nguồn cho điều này); và kết luận về APR ở B/12 có còn đúng với các kiến trúc sau 2019 không.

Câu hỏi nổi sang nhánh khác

Cây `../IMU_Fusion/` có một dự án thực nghiệm thật ở `../imu_initialization/` trên một chuỗi EuRoC. Những con số đó nói gì về [B/10](#), và có chỗ nào hai cây phát biểu khác nhau không? Có dữ liệu thật trong tay mà chưa đối chiếu với bài tổng quan là một dạng tự lừa mình rẻ tiền và dễ tránh — nên đây là việc nên làm sớm.

Cây `../marker/` làm ngân sách sai số pixel sang tư thế ở mức milimet. Các quan hệ ở [A/06](#) có nhất quán với ngân sách ở đó không? Nếu hai cây cho hai con số khác nhau cho cùng một đại lượng thì ít nhất một cây sai. Và một việc chưa làm: [B/11](#) nói công nghiệp giải bài toán môi trường lặp lại bằng cách dán marker, còn `../marker/` cho cái giá của giải pháp ấy; ghép hai phía thành một phân tích đánh đổi đầy đủ.

Khoản nợ ở mức cây

Ba điều. Thứ nhất, [D/22](#) chưa đạt chuẩn `research-rules.md` và nói rõ điều đó; trước khi dùng nó để chọn đề tài thật, hướng đã chọn phải được nâng lên thành một dự án survey/ đúng chuẩn.

Thứ hai, cả cây *chưa có con số thực nghiệm nào do tôi đo*. Điều này là có chủ ý và được ghi ở [00-map](#), nhưng nó nghĩa là mọi khẳng định định lượng đều thuộc một trong ba loại: dẫn nguồn, tính tay từ ví dụ tự đặt, hoặc đánh dấu “*tôi suy ra, chưa đối chiếu nguồn*”. Danh sách các mệnh đề loại thứ ba nằm rải trong mục “Điều gì chứng minh cách hiểu này sai” của từng chương; gom chúng thành một danh sách duy nhất là việc nên làm.

Thứ ba, phần hết hạn nhanh gồm [C/16](#) mục 4, toàn bộ [C/17](#) và toàn bộ [E/23](#). Cả ba chốt tại tháng 9/2026. **Rà lại tháng 3/2027.**

- [1] H. C. Longuet-Higgins, "A computer algorithm for reconstructing a scene from two projections," in *Nature* 293, mức tin A; công trình khai sinh essential matrix, 1981.
- [2] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004, mức tin A; sách chuẩn cho toàn Phần A.
- [3] Y. Ma, S. Soatto, J. Košecák, et al., *An Invitation to 3-D Vision: From Images to Geometric Models*. Springer, 2004, mức tin A.
- [4] D. Nistér, "An Efficient Solution to the Five-Point Relative Pose Problem," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 26, no. 6, pp. 756–770, 2004, mức tin A.
- [5] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A Versatile and Accurate Monocular SLAM System," *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [6] R. I. Hartley and P. Sturm, "Triangulation," *Computer Vision and Image Understanding*, vol. 68, no. 2, pp. 146–157, 1997, mức tin A; nguồn của phân biệt DLT / midpoint / optimal.
- [7] J. Civera, A. J. Davison, and J. M. M. Montiel, "Inverse Depth Parametrization for Monocular SLAM," *IEEE Transactions on Robotics*, vol. 24, no. 5, pp. 932–945, 2008, mức tin A; nguồn của lập luận vì sao inverse depth tuyến tính hoá tốt hơn.
- [8] L. Kneip, D. Scaramuzza, and R. Siegwart, "A Novel Parametrization of the Perspective-Three-Point Problem for a Direct Computation of Absolute Camera Position and Orientation," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011, mức tin A.
- [9] V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate $O(n)$ Solution to the PnP Problem," *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [10] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM," *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [11] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [12] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2019.
- [13] T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [14] Z. Zhang, "A Flexible New Technique for Camera Calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; nền của mọi công cụ hiệu chuẩn từ bảng phẳng.
- [15] J. Kannala and S. S. Brandt, "A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 8, pp. 1335–1340, 2006, mức tin A; mô hình KB/equidistant.

- [16] C. Mei and P. Rives, "Single View Point Omnidirectional Camera Calibration from Planar Grids," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; mô hình unified camera, 2007.
- [17] V. Usenko, N. Demmel, and D. Cremers, "The Double Sphere Camera Model," in *International Conference on 3D Vision (3DV)*, mức tin B, 2018.
- [18] P. Furgale, T. D. Barfoot, and G. Sibley, "Continuous-Time Batch Estimation Using Temporal Basis Functions," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2012.
- [19] D. Schubert, N. Demmel, V. Usenko, et al., "Direct Sparse Odometry with Rolling Shutter," in *European Conference on Computer Vision (ECCV)*, mức tin B, 2018.
- [20] J. Engel, V. Usenko, and D. Cremers, "A Photometrically Calibrated Benchmark for Monocular Visual Odometry," in *arXiv:1607.02555*, mức tin B; nguồn của quy trình hiệu chuẩn quang trắc (vignette, CRF, exposure), 2016.
- [21] P. Furgale, J. Rehder, and R. Siegwart, "Unified Temporal and Spatial Calibration for Multi-Sensor Systems," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [22] J. Engel, V. Koltun, and D. Cremers, "Direct Sparse Odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [23] V. Usenko, N. Demmel, D. Schubert, et al., "Visual-Inertial Mapping with Non-Linear Factor Recovery," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [24] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017, mức tin A; nguồn chuẩn cho ước lượng trên đa tạp.
- [25] J. Solà, J. Deray, and D. Atchuthan, "A micro Lie theory for state estimation in robotics," *arXiv:1812.01537*, 2018, mức tin B; tiền ấn nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [26] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, mức tin A; nguồn của loop closure trên Sim(3), 2010.
- [27] F. Dellaert and M. Kaess, "Factor Graphs for Robot Perception," *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.
- [28] B. Triggs, P. F. McLauchlan, R. I. Hartley, et al., "Bundle Adjustment — A Modern Synthesis," in *Vision Algorithms: Theory and Practice*, mức tin A; tài liệu tham chiếu chuẩn cho BA, gauge và độ thừa, Springer, 2000, pp. 298–372.
- [29] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005, mức tin A.
- [30] M. J. Black and A. Rangarajan, "On the Unification of Line Processes, Outlier Rejection, and Robust Statistics with Applications in Early Vision," *International Journal of Computer Vision*, vol. 19, no. 1, pp. 57–91, 1996, mức tin A; nguồn của đối ngẫu Black-Rangarajan.
- [31] M. Kaess, H. Johannsson, R. Roberts, et al., "iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012, mức tin A.
- [32] G. P. Huang, A. I. Mourikis, and S. I. Roumeliotis, "Analysis and Improvement of the Consistency of Extended Kalman Filter Based SLAM," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; nguồn của First-Estimates Jacobian, 2008.
- [33] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.

- [34] H. Yang, P. Antonante, V. Tzoumas, et al., “Graduated Non-Convexity for Robust Spatial Perception: From Non-Minimal Solvers to Global Outlier Rejection,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1127–1134, 2020, mức tin A.
- [35] S. Agarwal, K. Mierle, and The Ceres Solver Team, *Ceres Solver*, phần mềm mã nguồn mở, mức tin B; tài liệu phần mềm, không phải công trình phản biện, 2022.
- [36] R. Kümmerle, G. Grisetti, H. Strasdat, et al., “g2o: A General Framework for Graph Optimization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2011.
- [37] N. Demmel, C. Sommer, D. Cremers, et al., “Square Root Bundle Adjustment for Large-Scale Reconstruction,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [38] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. Wiley, 2001, mức tin A; nguồn chuẩn cho NEES và tính nhất quán.
- [39] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [40] Z. Zhang, G. Gallego, and D. Scaramuzza, “On the Comparison of Gauge Freedom Handling in Optimization-Based Visual-Inertial State Estimation,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 2710–2717, 2018, mức tin B; nguồn của so sánh ba cách xử lý gauge.
- [41] A. Barrau and S. Bonnabel, “The Invariant Extended Kalman Filter as a Stable Observer,” *IEEE Transactions on Automatic Control*, vol. 62, no. 4, pp. 1797–1812, 2017, mức tin A.
- [42] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [43] J. A. Hesch, D. G. Kottas, S. L. Bowman, et al., “Consistency Analysis and Improvement of Vision-Aided Inertial Navigation,” *IEEE Transactions on Robotics*, vol. 30, no. 1, pp. 158–176, 2014, mức tin A.
- [44] M. Kaess and F. Dellaert, “Covariance Recovery from a Square Root Information Matrix for Data Association,” in *Robotics and Autonomous Systems*, mức tin B, 2009.
- [45] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, mức tin A.
- [46] J. Shi and C. Tomasi, “Good Features to Track,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 1994.
- [47] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, mức tin A, 2006.
- [48] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2011.
- [49] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [50] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [51] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.

- [52] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [53] J.-Y. Bouguet, “Pyramidal Implementation of the Affine Lucas Kanade Feature Tracker: Description of the Algorithm,” Intel Corporation, tech. rep., 2001, mức tin B; báo cáo kỹ thuật, là tài liệu tham chiếu de-facto cho KLT thấp.
- [54] S. Baker and I. Matthews, “Lucas-Kanade 20 Years On: A Unifying Framework,” *International Journal of Computer Vision*, vol. 56, no. 3, pp. 221–255, 2004, mức tin A; nguồn của inverse compositional.
- [55] O. Chum and J. Matas, “Matching with PROSAC — Progressive Sample Consensus,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2005.
- [56] D. Barath, J. Noskova, M. Ivashechkin, et al., “MAGSAC++, a Fast, Reliable and Accurate Robust Estimator,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2020.
- [57] C. Forster, Z. Zhang, M. Gassner, et al., “SVO: Semidirect Visual Odometry for Monocular and Multicamera Systems,” *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 249–265, 2017, mức tin A.
- [58] G. Klein and D. Murray, “Parallel Tracking and Mapping for Small AR Workspaces,” in *IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A; công trình tách tracking khỏi mapping thành hai luồng, 2007.
- [59] J. Engel, T. Schöps, and D. Cremers, “LSD-SLAM: Large-Scale Direct Monocular SLAM,” in *European Conference on Computer Vision (ECCV)*, mức tin A, 2014.
- [60] A. I. Mourikis and S. I. Roumeliotis, “A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; nguồn của cách khử landmark bằng chiếu lên null space, 2007.
- [61] L. Yang, B. Kang, Z. Huang, et al., “Depth Anything V2,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2024.
- [62] L. Piccinelli, Y.-H. Yang, C. Sakaridis, et al., “UniDepth: Universal Monocular Metric Depth Estimation,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [63] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [64] D. M. Rosen, L. Carlone, A. S. Bandeira, et al., “SE-Sync: A Certifiably Correct Algorithm for Synchronization over the Special Euclidean Group,” *The International Journal of Robotics Research*, vol. 38, no. 2-3, pp. 95–125, 2019, mức tin A.
- [65] R. Hartley, J. Trumpf, Y. Dai, et al., “Rotation Averaging,” *International Journal of Computer Vision*, vol. 103, no. 3, pp. 267–305, 2013, mức tin A.
- [66] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012, mức tin A.
- [67] D. Nistér and H. Stewénus, “Scalable Recognition with a Vocabulary Tree,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2006.
- [68] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 6, pp. 1437–1451, 2018, mức tin A.

- [69] M. J. Milford and G. F. Wyeth, "SeqSLAM: Visual Route-Based Navigation for Sunny Summer Days and Stormy Winter Nights," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2012.
- [70] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013, mức tin A; thuật toán RRR.
- [71] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2012.
- [72] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2013.
- [73] T. Schneider, M. Dymczyk, M. Fehr, et al., "maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [74] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2022, mức tin B.
- [75] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2023.
- [76] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2015.
- [77] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-Based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của khẳng định APR không tốt hơn image retrieval, 2019.
- [78] M. Dymczyk, S. Lynen, T. Cieslewski, et al., "The Gist of Maps — Summarizing Experience for Lifelong Localization," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; nguồn của map summarization, 2015.
- [79] X. Shi, D. Li, P. Zhao, et al., "Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [80] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., "Revealing Scenes by Inverting Structure from Motion Reconstructions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [81] J. Maye, P. Furgale, and R. Siegwart, "Self-Supervised Calibration for Robotic Systems," in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [82] T. Schneider, M. Li, C. Cadena, et al., "Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation," in *IEEE Sensors Journal*, mức tin B, 2019.
- [83] H. Hirschmüller, "Stereo Processing by Semiglobal Matching and Mutual Information," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, mức tin A.
- [84] T. Whelan, R. F. Salas-Moreno, B. Glocker, et al., "ElasticFusion: Real-Time Dense SLAM and Light Source Estimation," *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1697–1716, 2016, mức tin A.

- [85] E. Sucar, S. Liu, J. Ortiz, et al., “iMAP: Implicit Mapping and Positioning in Real-Time,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2021.
- [86] Z. Zhu, S. Peng, V. Larsson, et al., “NICE-SLAM: Neural Implicit Scalable Encoding for SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2022.
- [87] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023, mức tin A.
- [88] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track and Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [89] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [90] A. Hornung, K. M. Wurm, M. Bennewitz, et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013, mức tin A.
- [91] R. A. Newcombe, S. Izadi, O. Hilliges, et al., “KinectFusion: Real-Time Dense Surface Mapping and Tracking,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A, 2011.
- [92] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin A, 2021.
- [93] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [94] V. Leroy, Y. Cabon, and J. Revaud, “Grounding Image Matching in 3D with MAST3R,” in *European Conference on Computer Vision (ECCV)*, mức tin B, 2024.
- [95] J. Wang, M. Chen, N. Karaev, et al., “VGGT: Visual Geometry Grounded Transformer,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin C; rất mới, chưa có tái lập độc lập rộng rãi, 2025.
- [96] S. Yang and S. Scherer, “CubeSLAM: Monocular 3-D Object SLAM,” in *IEEE Transactions on Robotics*, mức tin B, 2019.
- [97] L. Nicholson, M. Milford, and N. Sünderhauf, “QuadricSLAM: Dual Quadrics from Object Detections as Landmarks in Object-Oriented SLAM,” in *IEEE Robotics and Automation Letters*, mức tin B, 2019.
- [98] Z. Teed, L. Lipson, and J. Deng, “Deep Patch Visual Odometry,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2023.
- [99] L. Pineda, T. Fan, M. Monge, et al., “Theseus: A Library for Differentiable Nonlinear Optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2022.
- [100] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [101] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [102] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phân biện, 2017.

- [103] M. Burri, J. Nikolic, P. Gohl, et al., "The EuRoC Micro Aerial Vehicle Datasets," *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016, mức tin A.
- [104] D. Schubert, T. Goll, N. Demmel, et al., "The TUM VI Benchmark for Evaluating Visual-Inertial Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2018.
- [105] A. Geiger, P. Lenz, and R. Urtasun, "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2012.
- [106] T. Sattler, W. Maddern, C. Toft, et al., "Benchmarking 6DOF Outdoor Visual Localization in Changing Conditions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; bộ Aachen Day-Night, 2018.